

Официальное учебное пособие для самоподготовки

Учебный курс

Microsoft®

Microsoft®

SQL Server 2008

Реализация
и обслуживание

Экзамен **70-432**

Майк Хотек

MCTS

РУССКАЯ РЕДАКЦИЯ

Microsoft®

Посвящается Дженилин.
Она мой компас в шторм и маяк, указующий путь домой.

MCTS
Self-Paced Training Kit

Exam **70-432**

Microsoft®
SQL Server® 2008
Implementation
and Maintenance

Mike Hoteck

Официальное пособие для самоподготовки

Учебный курс *Microsoft®*

Microsoft® SQL Server® 2008

Реализация и обслуживание

Майк Хотек

экзамен **70-432**

MCTS

Москва 2011

 РУССКАЯ РЕДАКЦИЯ

УДК 004.738.5
ББК 32.973.202
X85

Хотек Майк

X85 Microsoft SQL Server 2008. Реализация и обслуживание. Учебный курс Microsoft / Пер. с англ. — М. : Издательство «Русская редакция», 2011. — 576 стр. : ил.

ISBN 978-5-7502-0402-1

Данная книга — подробное руководство по установке и управлению SQL Server в корпоративной среде. В ней описывается, как выбрать подходящую редакцию и как устанавливать и конфигурировать экземпляры SQL Server; приводятся сведения о конфигурировании, управлении, резервном копировании и восстановлении баз данных; содержится детальная информация о создании секционированных таблиц, индексов и индексированных представлений, а также о конфигурировании служб и компонентов. Кроме того, здесь рассказывается о том, как обеспечить безопасность SQL Server и как проектировать и развертывать решения высокой доступности, в том числе, управлять заданиями, предупреждениями и операторами Агента SQL Server, а также устранять неполадки служб, параллелизма и заданий.

Этот учебный курс предназначен администраторам баз данных и всем, кто хочет получить исчерпывающие знания в области развертывания и обслуживания Microsoft SQL Server 2008. Помимо теоретического материала курс содержит практикумы, упражнения и контрольные вопросы для самопроверки. Он поможет вам самостоятельно подготовиться к экзамену по программе сертификации MCTS (Microsoft Certified Technology Specialist) № 70-432: Microsoft SQL Server 2008 Implementation and Maintenance.

Книга состоит из 17 глав, содержит множество иллюстраций и примеров из практики. Дополнительные материалы к книге могут находиться на ресурсах, указанных в тексте, и на сайте www.microsoft.com

УДК 004.738.5
ББК 32.973.202

© 2009-2012, Translation Russian Edition Publishers.

Authorized Russian translation of the English edition of MCTS Self-Paced Training Kit (Exam 70-432): Microsoft® SQL Server® 2008 - Implementation and Maintenance, ISBN 9780735626058 © Mike Hotek.

This translation is published and sold by permission of O'Reilly Media, Inc., which owns or controls all rights to publish and sell the same.

© 2009-2012, перевод ООО «Издательство «Русская редакция».

Авторизованный перевод с английского на русский язык произведения MCTS Self-Paced Training Kit (Exam 70-432): Microsoft® SQL Server® 2008 - Implementation and Maintenance, ISBN 9780735626058 © Mike Hotek.

Этот перевод оригинального издания публикуется и продается с разрешения O'Reilly Media, Inc., которая владеет или распоряжается всеми правами на его публикацию и продажу.

© 2009-2012, оформление и подготовка к изданию, ООО «Издательство «Русская редакция».

Microsoft, а также товарные знаки, перечисленные в списке, расположенном по адресу:

<http://www.microsoft.com/about/legal/en/us/IntellectualProperty/Trademarks/EN-US.aspx> являются товарными знаками или охраняемыми товарными знаками корпорации Microsoft в США и/или других странах. Все другие товарные знаки являются собственностью соответствующих фирм.

Все названия компаний, организаций и продуктов, а также имена лиц, используемые в примерах, вымышлены и не имеют никакого отношения к реальным компаниям, организациям, продуктам и лицам.

Содержание

Благодарности	XVIII
Об этой книге	XIX
Глава 1 Установка и настройка SQL Server 2008.....	1
Занятие 1. Определение аппаратных и программных требований.....	3
Минимальные аппаратные требования.....	3
Поддерживаемые операционные системы	4
Программные требования	4
Практикум. Проверка выполнения минимальных требований.....	5
Резюме занятия.....	6
Закрепление материала.....	6
Занятие 2. Выбор выпуска SQL Server	7
Службы SQL Server.....	7
Выпуски SQL Server	11
Резюме занятия.....	14
Закрепление материала.....	14
Занятие 3. Установка и конфигурирование экземпляров SQL Server	15
Учетные записи служб	15
Параметры сортировки	16
Режимы проверки подлинности	16
Экземпляры SQL Server	16
Диспетчер конфигурации SQL Server	17
Практикум. Установка и конфигурирование экземпляра SQL Server	19
Резюме занятия.....	24
Закрепление материала.....	25
Занятие 4. Конфигурирование компонента Database Mail	25
Компонент Database Mail.....	26
Практикум. Конфигурирование Database Mail.....	27
Резюме занятия.....	28
Закрепление материала.....	29
Закрепление материала главы.....	29
Резюме главы	29
Основные термины.....	30
Упражнения	30
Рекомендуемые упражнения	31
Установка SQL Server	31

Управление службами SQL Server	31
Пробный экзамен.....	31
Глава 2 Настройка и обслуживание базы данных.....	32
Занятие 1. Настройка файлов и файловых групп.....	33
Файлы и файловые группы	34
Журналы транзакций	37
Данные FILESTREAM.....	38
База данных tempdb	38
Практикум. Создание баз данных	39
Резюме занятия.....	39
Закрепление материала.....	40
Занятие 2. Настройка параметров БД.....	40
Параметры настройки БД	40
Схемы сортировки.....	46
Практикум. Изменение модели восстановления БД.....	46
Резюме занятия.....	47
Закрепление материала.....	47
Занятие 3. Обеспечение целостности БД	47
Проверка целостности БД	48
Практикум. Проверка целостности БД.....	49
Резюме занятия.....	50
Закрепление материала.....	50
Закрепление материала главы.....	50
Резюме главы	50
Основные термины	51
Упражнения	51
Рекомендуемые упражнения	53
Настройка баз данных	53
Пробный экзамен.....	53
Глава 3 Таблицы.....	54
Занятие 1. Создание таблиц	55
Схемы	56
Типы данных.....	56
Практикум. Создание таблиц	68
Резюме занятия.....	69
Закрепление материала.....	69
Занятие 2. Реализация ограничений	69
Первичные ключи	70
Внешние ключи	70
Ограничения уникальности	70
Ограничения по умолчанию	71
Проверочные ограничения	71
Практикум. Создание ограничений.....	72
Резюме занятия.....	73

Закрепление материала.....	73
Закрепление материала главы.....	73
Резюме главы.....	73
Основные термины.....	74
Упражнения.....	74
Рекомендуемые упражнения.....	75
Создание таблиц.....	75
Создание ограничений.....	75
Пробный экзамен.....	75
Глава 4 Проектирование индексов SQL Server	76
Занятие 1. Архитектура индексов	77
Структура индексов.....	78
Сбалансированные деревья.....	78
Уровни индекса.....	80
Резюме занятия.....	82
Закрепление материала.....	82
Занятие 2. Проектирование индексов.....	82
Кластеризованные индексы.....	83
Некластеризованные индексы.....	85
Параметры индексов.....	87
XML-индексы.....	89
Пространственные индексы.....	89
Практикум. Создание индексов.....	92
Резюме занятия.....	92
Закрепление материала.....	93
Занятие 3. Поддержка индексов	93
Управление и обслуживание индексов.....	94
Отключение индекса.....	95
Практикум. Поддержка индексов.....	96
Резюме занятия.....	96
Закрепление материала.....	97
Закрепление материала главы.....	97
Резюме главы.....	97
Основные термины.....	97
Упражнения.....	98
Рекомендуемые упражнения.....	99
Создание индексов.....	99
Пробный экзамен.....	100
Глава 5 Полнотекстовое индексирование	101
Занятие 1. Создание и заполнение полнотекстовых индексов.....	102
Полнотекстовые каталоги.....	103
Полнотекстовые индексы.....	104
Отслеживание изменений.....	105
Язык, средства разбиения по словам и парадигматические модули.....	105

Практикум. Создание полнотекстовых индексов.....	107
Резюме занятия.....	108
Закрепление материала.....	108
Занятие 2. Запросы полнотекстовых данных.....	109
Запросы с использованием FREETEXT.....	109
Запросы с использованием CONTAINS.....	111
Практикум. Выполнение запросов с использованием полнотекстовых индексов	113
Резюме занятия.....	114
Закрепление материала.....	115
Занятие 3. Управление полнотекстовыми индексами	115
Тезаурусы.....	116
Списки стоп-слов	116
Заполнение полнотекстовых индексов	117
Практикум. Управление полнотекстовыми индексами	118
Резюме занятия.....	120
Закрепление материала.....	120
Закрепление материала главы.....	120
Резюме главы.....	120
Основные термины	121
Упражнения	121
Рекомендуемые упражнения	122
Создание полнотекстового индекса.....	122
Запрос с использованием полнотекстового индекса.....	122
Создание файла тезауруса	122
Создание списка стоп-слов.....	122
Пробный экзамен.....	122
Глава 6 Распределение и секционирование данных	123
Занятие 1. Создание функции секционирования	124
Функции секционирования	124
Практикум. Создание функции секционирования	127
Резюме занятия.....	129
Закрепление материала.....	129
Занятие 2. Создание схемы секционирования.....	129
Схемы секционирования.....	130
Практикум. Создание схемы секционирования.....	131
Резюме занятия.....	132
Занятие 3. Создание секционированных таблиц и индексов	133
Создание секционированной таблицы	133
Создание секционированного индекса.....	134
Практикум. Секционирование таблицы	135
Резюме занятия.....	136
Закрепление материала.....	136
Занятие 4. Управление секциями.....	136
Операции SPLIT и MERGE.....	137

Изменение схемы секционирования	137
Выравнивание индексов	137
Операция SWITCH	138
Практикум. «Скользящее окно»	141
Резюме занятия	142
Закрепление материала	142
Закрепление материала главы	143
Резюме главы	143
Основные термины	143
Упражнения	144
Рекомендуемые упражнения	146
Секционирование	146
Пробный экзамен	147
Глава 7 Импорт и экспорт данных	148
Занятие 1. Импорт и экспорт данных	149
Утилита BCP	150
Команда BULK INSERT	152
Мастер импорта и экспорта SQL Server	152
Практикум. Экспорт данных	153
Резюме занятия	157
Закрепление материала	157
Закрепление материала главы	157
Резюме главы	158
Основные термины	158
Упражнения	158
Рекомендуемые упражнения	160
Пробный экзамен	160
Глава 8 Проектирование управления на основе политик	161
Занятие 1. Разработка политик	162
Аспекты	162
Условия	163
Цели политик	164
Политики	164
Категории политик	164
Соответствие политикам	165
Импорт и экспорт политик	167
Практикум. Определение политик и выполнение проверок на соответствие	168
Резюме занятия	175
Закрепление материала	175
Закрепление материала главы	175
Резюме главы	175
Основные термины	176
Упражнения	176
Рекомендуемые упражнения	178

Реализация управления, основанного на политиках	178
Пробный экзамен.....	178
Глава 9 Резервное копирование и восстановление базы данных	179
Занятие 1. Резервное копирование баз данных	180
Типы резервных копий	181
Частичные резервные копии	187
Поврежденные страницы	188
Планы обслуживания	188
Сертификаты и главные ключи	189
Проверка резервных копий	190
Практикум. Создание резервных копий баз данных	191
Резюме занятия.....	192
Закрепление материала.....	193
Занятие 2. Восстановление баз данных	193
Внутреннее устройство журнала транзакций	193
Восстановление баз данных	196
Практикум. Восстановление базы данных.....	201
Резюме занятия.....	203
Закрепление материала.....	204
Занятие 3. Моментальный снимок базы данных	204
Создание моментального снимка базы данных.....	205
Возвращение базы данных к прежнему состоянию при помощи моментального снимка	206
Практикум. Создание моментального снимка базы данных	208
Резюме занятия.....	208
Закрепление материала.....	208
Закрепление материала главы.....	209
Резюме главы	209
Основные термины	209
Упражнения	210
Рекомендуемые упражнения	212
Создание резервных копий базы данных	212
Восстановление базы данных	212
Пробный экзамен.....	212
Глава 10 Автоматизация SQL Server	213
Занятие 1. Создание заданий	213
Шаги задания	214
Расписания заданий.....	215
Журнал заданий	215
Операторы	215
Практикум. Создание заданий и операторов	216
Резюме занятия.....	220
Закрепление материала.....	220

Занятие 2. Создание предупреждений.....	220
Предупреждения агента SQL Server	220
Практикум. Создание предупреждений.....	221
Резюме занятия.....	223
Закрепление материала.....	223
Закрепление материала главы.....	224
Резюме главы.....	224
Основные термины.....	224
Упражнения.....	224
Рекомендуемые упражнения	227
Создание заданий.....	227
Создание предупреждений	227
Пробный экзамен.....	227
Глава 11 Проектирование безопасности SQL Server	228
Занятие 1. Конечные точки TCP	229
Типы конечных точек и полезные данные	229
Доступ к конечным точкам.....	229
Конечные точки TCP	230
Аргументы конечных точек TCP.....	230
Общие аргументы для конечных точек DATABASE_MIRRORING и SERVICE_BROKER.....	231
Аргументы для конечных точек DATABASE_MIRRORING.....	232
Аргументы для конечных точек SERVICE_BROKER.....	232
Практикум. Сбор данных об имеющихся конечных точках	234
Закрепление материала.....	234
Занятие 2. Настройка контактной зоны SQL Server	235
Настройка контактной зоны	235
Практикум. Настройка контактной зоны.....	237
Резюме занятия.....	238
Закрепление материала.....	238
Занятие 3. Создание участников.....	238
Имена входа	239
Предопределенные роли сервера.....	240
Пользователи базы данных.....	241
Пользователи без имени входа.....	242
Предопределенные роли базы данных.....	242
Роли пользователей базы данных	243
Практикум. Создание имен входа и пользователей базы данных	244
Резюме занятия.....	245
Закрепление материала.....	245
Занятие 4. Управление разрешениями.....	246
Защищаемые объекты	247
Разрешения	247
Безопасность метаданных	249
Цепочки владения.....	249

Олицетворение.....	250
Главные ключи.....	251
Сертификаты.....	252
Подписи.....	252
Практикум. Управление разрешениями.....	253
Резюме занятия.....	258
Закрепление материала.....	259
Занятие 5. Аудит экземпляров SQL Server.....	259
Триггеры DDL.....	259
Спецификации аудита.....	261
Аудит C2.....	262
Практикум. Создание спецификации аудита базы данных.....	263
Резюме занятия.....	265
Закрепление материала.....	265
Занятие 6. Шифрование данных.....	265
Шифрование данных.....	266
Алгоритмы хеширования.....	267
Симметричные ключи.....	268
Сертификаты и асимметричные ключи.....	268
Прозрачное шифрование данных.....	268
Управление ключами шифрования.....	269
Практикум. Шифрование данных.....	270
Резюме занятия.....	273
Закрепление материала.....	274
Закрепление материала главы.....	274
Резюме главы.....	275
Основные термины.....	275
Упражнения.....	276
Рекомендуемые упражнения.....	277
Управление именами входа и ролями сервера.....	277
Управление пользователями и ролями базы данных.....	277
Управление разрешениями экземпляра SQL Server.....	277
Управление разрешениями базы данных.....	278
Управление разрешениями схемы и разрешениями объектов.....	278
Аудит экземпляров SQL Server.....	278
Управление прозрачным шифрованием данных.....	278
Пробный экзамен.....	278
Глава 12 Мониторинг Microsoft SQL Server.....	279
Занятие 1. Использование Системного монитора.....	280
Общие сведения о Системном мониторе.....	281
Запись журналов счетчиков.....	282
Счетчики производительности.....	284
Практикум. Создание журнала счетчиков.....	285
Резюме занятия.....	287
Закрепление материала.....	287

Занятие 2. Использование SQL Server Profiler.....	287
Определение трассировки.....	288
Определение событий для трассировки	291
Выбор столбцов данных.....	293
Использование фильтров.....	294
Управление трассировками.....	295
Связь между производительностью и мониторингом данных.....	296
Практикум. Создание трассировки	297
Резюме занятия.....	302
Закрепление материала.....	302
Занятие 3. Диагностика сбоев баз данных.....	303
Журналы SQL Server.....	303
Проблемы с нехваткой пространства в базе данных	305
Практикум. Создание предупреждений о неполадках.....	307
Резюме занятия.....	309
Закрепление материала.....	310
Занятие 4. Диагностирование сбоев служб.....	310
Обнаружение сбоев запуска служб.....	310
Практикум. Устранение неполадок запуска служб	318
Резюме занятия.....	319
Закрепление материала.....	319
Занятие 5. Диагностирование неполадок оборудования	320
Дисководы	320
Память и процессоры.....	321
Резюме занятия.....	322
Закрепление материала.....	322
Занятие 6. Устранение неполадок, связанных с кратковременной и обычной блокировкой.....	323
Блокировки	323
Уровни изоляции транзакции	324
Взаимные блокировки.....	326
Практикум. Устранение взаимной блокировки.....	327
Резюме занятия.....	330
Закрепление материала.....	330
Закрепление материала главы.....	330
Резюме главы.....	330
Основные термины.....	331
Упражнения.....	331
Рекомендуемые упражнения	333
Настройка баз данных	333
Пробный экзамен.....	334
Глава 13 Оптимизация производительности.....	335
Занятие 1. Применение помощника по настройке ядра СУБД.....	336
Помощник по настройке ядра СУБД.....	337
Практикум. Анализ рабочей нагрузки запроса.....	341

Резюме занятия.....	342
Закрепление материала.....	342
Занятие 2. Использование Регулятора ресурсов	343
Регулятор ресурсов	343
Практикум. Настройка Регулятора ресурсов	345
Резюме занятия.....	352
Закрепление материала.....	352
Занятие 3. Динамические административные представления и функции	352
Категории динамических административных представлений.....	353
Практикум. Оценка нехватки индексов.....	358
Резюме занятия.....	359
Закрепление материала.....	360
Занятие 4. Использование хранилища данных о производительности	360
Хранилище данных о производительности	360
Практикум. Настройка хранилища данных о производительности	362
Резюме занятия.....	365
Закрепление материала.....	365
Закрепление материала главы.....	366
Резюме главы	366
Основные термины	366
Упражнения	367
Рекомендуемые упражнения	369
Сбор информации средствами хранилища данных о производительности для оптимизации производительности	369
Сбор информации средствами Помощника по настройке ядра СУБД для оптимизации производительности	370
Сбор информации средствами динамических административных представлений для оптимизации производительности	370
Пробный экзамен.....	370
Глава 14 Отказоустойчивые кластеры	371
Занятие 1. Проектирование кластеров Windows.....	373
Компоненты кластеров Windows	373
Типы кластеров.....	374
Конфигурация безопасности	376
Дисковая конфигурация.....	376
Конфигурация сети	377
Ресурсы кластера.....	378
Группы кластера	379
Практикум. Создание кластера Windows.....	380
Резюме.....	390
Закрепление материала.....	391
Занятие 2. Проектирование отказоустойчивых кластеризованных экземпляров SQL Server 2008.....	391
Терминология.....	392
Компоненты отказоустойчивого кластеризованного экземпляра.....	393

Проверка работоспособности	395
Переход на резервный ресурс кластера при сбое	395
Практикум. Установка отказоустойчивого кластеризованного экземпляра SQL Server	397
Резюме	405
Закрепление материала	405
Закрепление материала главы	406
Резюме главы	406
Основные термины	407
Упражнения	407
Рекомендуемые упражнения	410
Кластеры Windows	410
Отказоустойчивые кластеры SQL Server	410
Пробный экзамен	411
Глава 15 Зеркальное отображение базы данных	412
Занятие 1. Обзор зеркального отображения базы данных	413
Роли зеркального отображения базы данных	413
Роль основной базы данных	413
Роль зеркальной базы данных	414
Следящий сервер	414
Конечные точки зеркального отображения базы данных	415
Режимы работы	415
Кеширование	419
Прозрачное перенаправление клиента	419
Организация потоков при зеркальном отображении базы данных	420
Моментальные снимки баз данных	420
Практикум. Создание конечных точек для зеркального отображения базы данных	421
Резюме занятия	423
Закрепление материала	423
Занятие 2. Инициализация зеркального отображения базы данных	424
Модели восстановления	425
Резервное копирование и восстановление	426
Копирование системных объектов	427
Практикум. Конфигурирование зеркального отображения базы данных	427
Резюме занятия	429
Закрепление материала	430
Занятие 3. Проектирование перехода на резервный ресурс и возврата на основной ресурс	430
Проектирование перехода на резервный ресурс в сеансе зеркального отображения	431
Проектирование возврата на основной ресурс в сеансе зеркального отображения	431
Практикум. Переход на резервный ресурс в сеансе зеркального отображения базы данных	434
Резюме	435
Закрепление материала	435
Закрепление материала главы	435

Резюме главы	435
Основные термины	436
Упражнения	436
Рекомендуемые упражнения	439
Настройка зеркального отображения базы данных	439
Создание моментального снимка зеркальной базы данных	440
Пробный экзамен	440
Глава 16 Доставка журналов	441
Занятие 1. Знакомство с доставкой журналов	442
Ситуации, в которых применяется доставка журналов	442
Компоненты механизма доставки журналов	443
Типы доставки журналов	445
Резюме занятия	446
Закрепление материала	446
Занятие 2. Запуск доставки журналов	446
Порядок запуска доставки журналов	447
Практикум. Настройка доставки журналов	452
Резюме занятия	456
Закрепление материала	457
Занятие 3. Разработка стратегии перехода на другой ресурс и возврата на исходный ресурс	457
Переход на другой ресурс	459
Возврат на исходный ресурс	460
Практикум. Переход на другой ресурс при использовании доставки журналов	462
Резюме занятия	462
Закрепление материала	463
Закрепление материала главы	463
Резюме главы	463
Основные термины	463
Упражнения	464
Рекомендуемые упражнения	468
Запуск доставки журналов	468
Переход на другой ресурс и возврат на исходный ресурс при использовании доставки журналов	469
Пробный экзамен	469
Глава 17 Репликация	470
Занятие 1. Общие сведения о репликации	470
Компоненты репликации	471
Роли репликации	472
Топология репликации	473
Агенты репликации	474
Методы репликации	475
Конфликты данных	477
Практикум. Настройка публикации	480

Резюме занятия.....	481
Закрепление материала.....	481
Занятие 2. Репликация транзакций.....	482
Отслеживание изменений.....	482
Архитектуры репликации транзакций.....	486
Мониторинг.....	488
Проверка синхронизации.....	488
Практикум. Реализация репликации транзакций.....	489
Резюме занятия.....	492
Закрепление материала.....	493
Занятие 3. Репликация слиянием.....	493
Отслеживание изменений.....	494
Проверка синхронизации.....	496
Практикум. Реализация репликации слиянием.....	497
Резюме занятия.....	498
Закрепление материала.....	499
Закрепление материала главы.....	500
Резюме главы.....	500
Основные термины.....	500
Упражнения.....	501
Рекомендуемые упражнения.....	505
Репликация транзакций.....	505
Репликация слиянием.....	506
Пробный экзамен.....	507
Ответы.....	508
Системные требования.....	552
Об авторе.....	553

Благодарности

Огромное спасибо вам, мои верные читатели, более десятка лет читающие мои книги; трудно поверить, но это моя седьмая книга по SQL Server, и она была бы невозможна без вас. Хочу поблагодарить редакторскую команду в Microsoft Press: Денизу Банкайтис (Denise Bankaitis) и Лору Сакерман (Laura Sackerman). Особую благодарность выражаю Кену Джонсу (Ken Jones), который участвовал в выпуске пяти моих книг и доказал, что является одним из бриллиантов в короне издательства Microsoft Press. Спасибо Розан Уолен (Rozanne Whalen), техническому редактору трех моих книг. Я не знаю, как она это делает, но именно магическое умение работать со словом Сюзан Маккланг (Susan McClung) превратило мою писанину в замечательную книгу, которую вы держите в руках. Целостность содержания этой книги — результат многодневных и тяжких усилий Розанн, Сюзан и всей остальной редакторской команды. Спасибо!

Об этой книге

Этот учебный курс предназначен для администраторов баз данных ответственных за развертывание, управление и устранение неполадок экземпляров Microsoft SQL Server 2008, а также ИТ-специалистов, планирующих сдать сертификационный экзамен 70-432 по программе Майкрософт MCTS. Предполагается, что у читателей этого курса есть базовые знания Microsoft Windows и SQL Server 2008 и опыт работы с SQL Server или другой базой данных.

Освоив материал этого учебного курса, вы научитесь:

- устанавливать и конфигурировать SQL Server 2008;
- создавать и реализовывать объекты баз данных;
- реализовывать высокую доступность и отказоустойчивость;
- обеспечивать безопасность экземпляров СУБД, баз данных и объектов баз данных;
- выполнять мониторинг и устранение неполадок экземпляров SQL Server.

О прилагаемом компакт-диске

В состав учебного курса входит компакт-диск со следующим содержимым:

■ Пробный экзамен

Проверить и закрепить полученные знания и навыки развертывания и поддержки SQL Server 2008 помогут электронные тесты, составленные из вопросов раздела «Закрепление материала» каждой главы этой книги; читатели смогут самостоятельно выбирать вопросы. Также можно пройти пробный сертификационный экзамен 70-432. Для этого на диске содержится более 200 вопросов реального экзамена — их достаточно, чтобы освоить процедуру опроса и оценить степень своей готовности к экзамену.

■ Примеры кода

Для большинства практических занятий в этой книге имеются файлы примеров кода и другие дополнительные материалы. Вы можете самостоятельно вводить код из книги или открывать готовые файлы с кодом в окне запросов.

■ Электронная версия книги

На прилагаемом диске также находится электронная версия этой книги (на английском языке), которую можно носить с собой вместо печатной копии. Электронная книга записана в формате PDF, для ее просмотра необходима программа Adobe Acrobat или Adobe Reader. В процессе работы над книгой вы можете копировать примеры кода из PDF-файлов.

■ Главы из других книг

На прилагаемом диске вы также найдете главы из других книг о Windows Server 2008, вышедших в издательстве Microsoft Press. Главы представлены в формате PDF, и для их просмотра необходима программа Adobe Acrobat или Adobe Reader.

Пробная версия ПО

180-дневную пробную версию SQL Server 2008, которую можно использовать при работе с этой книгой вместо полной версии этой СУБД, загрузите со страницы, расположенной по адресу <http://www.microsoft.com/sqlserver/2008/ru/ru/default.aspx>.

Установка заданий пробного экзамена

Чтобы установить задания пробного экзамена с прилагаемого диска на жесткий диск своего компьютера, выполните следующее:

1. Вставьте прилагаемый диск в дисковод компьютера и примите лицензионное соглашение — откроется меню компакт-диска.

ПРИМЕЧАНИЕ Если меню не открывается

Если меню не открывается и лицензионное соглашение не выводится, возможно, на вашем компьютере отключена функция AutoRun. О других способах установки см. в файле Readme.txt на прилагаемом диске.

2. Щелкните **Practice Tests** и следуйте инструкциям программы установки.

О пробном экзамене

Чтобы запустить пробный экзамен, выполните следующее:

1. Выберите **Пуск/Все программы/Microsoft Press Training Kit Exam Prep** (Start/All Programs/Microsoft Press Training Kit Exam Prep) — откроется окно со списком пробных экзаменов из учебных курсов Майкрософт, установленных на компьютере.
2. Дважды щелкните занятие, материал которого необходимо закрепить (lesson review), либо пробный экзамен (practice test).

Закрепление материала

При выборе закрепления материала открывается окно **Custom Mode**, которое позволяет настроить процедуру тестирования. Щелкнув **ОК**, можно принять параметры по умолчанию либо настроить число вопросов, режим опроса, выбрать темы, а также включить опрос «на время». При повторном закреплении материала можно ответить на все вопросы либо только на те, на которые вы не смогли ответить правильно в прошлый раз.

После щелчка **ОК** начнется опрос.

1. Во время теста для навигации по вопросам используются кнопки **Next**, **Previous** и **Go To**.
2. Ответив на вопрос, вы можете проверить правильность своего ответа и получить правильный ответ с пояснениями, щелкнув кнопку **Explanation**.

3. Для просмотра результатов теста ответьте на все вопросы и щелкните **Score Test**. Вы увидите процент правильных ответов по каждой из тем экзамена, а также сводные итоги. Далее можно распечатать итоги теста, проанализировать свои ответы либо повторить тест.

Пробный экзамен

Пробный экзамен поддерживает режим сертификационного экзамена (Certification Mode), режим обучения (Study Mode) и пользовательский режим (Custom Mode):

- Certification Mode — воспроизводит реальный сертификационный экзамен. В этом режиме необходимо ответить на предлагаемые вопросы за ограниченное время, при этом невозможно ни приостановить, ни перезапустить таймер.
- Study Mode — тест без ограничения по времени с возможностью просмотра правильных ответов с пояснениями для каждого вопроса.
- Custom Mode — допускает произвольную настройку параметров тестирования.

Пользовательский интерфейс в разных режимах практически идентичен, различаются лишь доступные параметры (см. выше).

С пояснениями правильных ответов для каждого вопроса отображается список ссылок (References) на соответствующие разделы учебного курса и источники дополнительной информации. Щелчок **Test Results** выводит итоги теста, на вкладке **Learning Plan** отображается список материалов, которые необходимо изучить для освоения каждой темы.

Удаление ПО пробного экзамена

Чтобы удалить ПО пробного экзамена, воспользуйтесь апплетом **Установка и удаление программ** (Add Or Remove Programs) (в Windows XP или Windows Server 2003) или **Программы и компоненты** (Programs And Features) (в Windows Vista или Windows Server 2008) панели управления Windows.

Программа сертификации специалистов Майкрософт

Программа сертификации специалистов Майкрософт (Microsoft Certified Professional, MCP) — отличная возможность подтвердить ваше знание современных технологий и программных продуктов этой фирмы. Майкрософт разработала современные методы тестирования; экзамены и программы сертификации подтвердят вашу квалификацию разработчика или специалиста по реализации решений на основе технологий и программных продуктов Майкрософт. Сертифицированные Майкрософт профессионалы квалифицируются как эксперты и высоко ценятся на рынке труда. Сертифицированным специалистам и организациям доступен ряд преимуществ (подробнее см. по ссылке, приведенной ниже).

ПРИМЕЧАНИЕ Сведения о сертификации специалистов Майкрософт

Полные сведения о сертификатах Майкрософт см. по адресу <http://www.microsoft.com/learning/mcp/default.aspx>.

Техническая поддержка

Мы постарались сделать все от нас зависящее, чтобы и учебный курс, и прилагаемый к нему компакт-диск не содержали ошибок. Если все же у вас возникнут вопросы или вы захотите поделиться своими предложениями и комментариями, обращайтесь в издательство Microsoft Press по одному из указанных ниже адресов.

Электронная почта: tkinput@Microsoft.com

Почтовый адрес:

Microsoft Press

Attn: *MCITP Self-Paced Training Kit (Exam 70-432): Microsoft SQL Server 2008 Implementation and Maintenance* Editor

One Microsoft Way

Redmond, WA 98052-6399

Дополнительные сведения о данном курсе и прилагаемом компакт-диске (включая ответы на типичные вопросы об установке и использовании) см. на веб-узле технической поддержки издательства Microsoft Press по адресу <http://www.microsoft.com/learning/support/books>. Самостоятельно найти ответы на свои вопросы можно в базе знаний Microsoft Press Knowledge Base на сайте <http://support.microsoft.com/search>. Консультацию по вопросам, связанным с поддержкой программных продуктов Майкрософт, можно получить по адресу <http://support.microsoft.com>.

Поддержка пробных версий ПО

180-дневная пробная версия не является полным коммерческим продуктом и предлагается исключительно для обучения и ознакомления. Поддержка оценочной версии не предоставляется.

Подробнее о данной книге и прилагаемом компакт-диске (в том числе ответы на часто задаваемые вопросы) — на веб-узле технической поддержки Microsoft Press Web (<http://www.microsoft.com/learning/support/books/>). Подробнее о получении полных версий программных продуктов Майкрософт вы можете узнать, позвонив в службу Microsoft Sales по телефону (800) 426-9400 или по адресу <http://www.microsoft.com>.

Установка и настройка SQL Server 2008

Занятие 1. Определение аппаратных и программных требований	3
Занятие 2. Выбор выпуска SQL Server	7
Занятие 3. Установка и конфигурирование экземпляров SQL Server	15
Занятие 4. Конфигурирование компонента Database Mail	25

Эта глава рассказывает, как установить Microsoft SQL Server. Вы узнаете о возможностях разных выпусков SQL Server, а также о требованиях к оборудованию для установки SQL Server. Также вы научитесь настраивать службы и компоненты SQL Server. Кроме того, вы узнаете, как конфигурировать почту базы данных, которая будет использоваться для уведомления о выполнении различных задач SQL Server.

Темы экзамена

- Установка SQL Server 2008 и относящихся к нему служб.
- Настройка экземпляра SQL Server.
- Настройка служб SQL Server.
- Настройка дополнительных компонентов SQL Server.
- Настройка почты БД.

Прежде всего

Для изучения материалов этой главы необходимо иметь:

- компьютер, соответствующий или превышающий минимальные требования к ПО и оборудованию, определенные в занятии 1;
- носители с установочным выпуском SQL Server 2008.

Практический совет

Майк Хотек

SQL Server 2008 не просто база данных, а полноценная платформа баз данных, состоящая из многих служб и сотен функций. Очень часто администраторы просто «прощелкивают» мастер установки SQL Server, после чего начинают загружать данные. Но до установки надо решить, как планируется использовать SQL Server, а также какие аппаратные ресурсы ему понадобятся.

Недавно мне пришлось работать с компанией, где только что установили серверы с SQL Server и требовалось изменить конфигурацию в соответствии с текущими потребностями. К сожалению, никто предварительно не разобрался как положено с новым приложением, которое планировалось установить. SQL Server был установлен, группа администраторов баз данных развернула структуру баз данных (БД) и приступила к загрузке данных. Внезапно процедура загрузки прервалась, и СУБД перестала реагировать на какие бы то ни было действия. Оказалось, что свободного пространства на дисках было явно мало и в процессе загрузки оно быстро исчерпалось. После выделения дополнительного дискового пространства и повторного запуска загрузки, горе-администраторы столкнулись с другой проблемой, в результате которой SQL Server снова стал недоступным. Дополнительное дисковое пространство для базы данных выделили, но о БД Tempdb забыли и снова дискового пространства не хватило. После многих попыток наконец удалось загрузить данные, но тут оказалось, что проект предусматривает репликацию, наличие компонента Service Broker и CLR.

После установки поддержки репликации и конфигурирования Service Broker и CLR, систему передали в промышленную эксплуатацию с 16-дневной опозданием относительно графика. На следующий же день пользователи поголовно стали жаловаться на слишком большое время реакции приложения. Администраторы БД запланировали одновременную работу не более 20 пользователей приложения, тогда как к новому приложению пытались «достучаться» более 2000 человек. Машины с одним процессором и 2 Гб оперативной памяти было явно недостаточно для обработки 2000 одновременных подключений пользователей, пытающихся обратиться к данным, общий объем которых превышал 400 Гб.

После перевода приложения в автономный режим, приобретения нового оборудования и повторного развертывания системы новое приложение вернули в онлайн-режим, с 43-дневной задержкой после намеченной даты. Большинство пользователей к тому времени предпочло системы конкурентов. Миллионы долларов на рекламу были потрачены впустую из-за плохого планирования на стадиях развертывания и установки.

Занятие 1. Определение аппаратных и программных требований

SQL Server 2008 должен устанавливаться на компьютер, удовлетворяющий минимальным требованиям к ПО и оборудованию. Здесь рассказывается, каковы минимальные аппаратные требования для тех или иных выпусков операционных систем, а также, какое дополнительное ПО необходимо для нормальной работы SQL Server 2008.

ВНИМАНИЕ! Минимальные аппаратные требования

На этом занятии речь идет о минимальных требованиях для установки SQL Server. Промышленным системам обычно требуется значительно больше, чтобы удовлетворить требования по производительности и емкости. Для определения требований к оперативной памяти, дисковому пространству и процессору, обусловленных конкретными приложениями, потребуются знания последующих глав данного курса.

Изучив материал этого занятия, вы сможете:

- ✓ проверить выполнение минимальных требований к оборудованию;
- ✓ проверить наличие необходимых ресурсов операционной системы;
- ✓ проверить наличие дополнительного программного обеспечения.

Продолжительность занятия — 20 минут.

Минимальные аппаратные требования

Требования для SQL Server 2005 зависели от выпуска и разрядности приложения. В SQL Server 2008 минимальные аппаратные требования существенно унифицированы (табл. 1-1).

Табл. 1-1. Требования к оборудованию

Оборудование	32-разрядный выпуск SQL Server	64-разрядный выпуск SQL Server
Процессор	Pentium III или выше	Itanium, Opteron, Athlon или Xeon/Pentium с поддержкой EM64T
Тактовая частота процессора	1,0 ГГц или выше	1,6 ГГц или выше
Память	512 Мб	512 Мб

Объем дискового пространства, занимаемого экземпляром SQL Server зависит от состава установленных служб и утилит. Подробнее об определении объема требуемого дискового пространства см. статью «Требования к оборудованию и программному обеспечению для установки SQL Server 2008» (Hardware and Software Requirements for Installing SQL Server 2008) на веб-странице <http://technet.microsoft.com/ru-ru/library/ms143506.aspx> (<http://technet.microsoft.com/en-us/library/ms143506.aspx>).

ВНИМАНИЕ! Дополнительные аппаратные компоненты

В онлайн-документации по SQL Server сказано, что для установки необходима мышь, привод CD- или DVD-дисков и монитор с разрешением не менее 1024x768. Однако SQL Server можно установить на компьютер, не имеющий ни одного из перечисленных устройств, что вполне обычно, если речь идет о серверной среде. Привод CD- или DVD-дисков требуется, только если планируется установка SQL Server с диска. Монитор необходим, только если планируется использовать инструментальные средства с графическим интерфейсом.

Поддерживаемые операционные системы

SQL Server 2008 может устанавливаться на 32- и 64-разрядных выпусках Microsoft Windows. 64-разрядный выпуск SQL Server может устанавливаться только на выпуске Windows с такой же разрядностью, тогда как 32-разрядный выпуск SQL Server можно установить как на 32-разрядную ОС Windows, так и на 64-разрядную ОС Windows с включенным режимом WOW (Windows on Windows).

Любой из выпусков SQL Server можно установить на:

- Windows Server 2008 Standard или выше;
- Windows Server 2003 Standard SP2 или выше.

На следующие операционные системы можно установить выпуски Developer и Express, а также оценочный выпуск (Evaluation) сервера SQL Server:

- Windows XP Professional SP2 или выше;
- Windows Vista Home Basic или выше.

SQL Server Express также можно установить на:

- Windows XP Home Edition SP2 или выше;
- Windows XP Home Reduced Media Edition;
- Windows XP Tablet Edition SP2 или выше;
- Windows XP Media Center 2002 SP2 или выше;
- Windows XP Professional Reduced Media Edition;
- Windows XP Professional Embedded Edition Feature Pack 2007 SP2;
- Windows XP Professional Embedded Edition for Point of Service SP2;
- Windows Server 2003 Small Business Server Standard Edition R2 или выше.

ПРИМЕЧАНИЕ На экзамене

SQL Server 2008 нельзя установить на ядро сервера (Server Core) Windows Server 2008. Ядро сервера Windows Server 2008 не поддерживается, потому что на нем нельзя установить .NET Framework. Возможности .NET Framework нужны SQL Server 2008 для поддержки типов данных *FILESTREAM*, *SPATIAL* и *DATE*, а также для некоторых других функций.

Программные требования

Для SQL Server 2008 требуется .NET Framework 3.5. Программа установки разворачивает нужные версии .NET Framework, но дополнительно для установки SQL Server нужен компонент Windows Installer 4.5.

ВНИМАНИЕ! .NET Framework

В составе .NET Framework 2.0 поставляется Windows Installer 3.1. Этого достаточно для выполнения минимальных требований, но для сокращения времени установки рекомендуется установить .NET Framework через Windows Installer 3.5.

Программа установки SQL Server также требует следующие компоненты:

- Microsoft Data Access Components (MDAC) 2.8 SP1 или выше;
- поддержку сетей Shared Memory, Named Pipes или TCP/IP;
- Internet Explorer 6 SP1 или выше.

Контрольные вопросы

1. На каком выпуске Windows Server 2008 нельзя установить SQL Server 2008?
2. На какие операционные системы можно установить любой из выпусков SQL Server?

Ответы на контрольные вопросы

1. SQL Server 2008 нельзя установить на ядро сервера (Server Core) Windows Server 2008.
2. На Windows Server 2003 Standard SP2 и выше или на Windows Server 2008 Standard RC0 и выше.

Практикум. Проверка выполнения минимальных требований

Вы проверите, удовлетворяет ли ваша машина минимальным требованиям к оборудованию, ОС и ПО для установки SQL Server.

Задание 1. Проверка выполнения требований к ОС и оборудованию

Вы проверите, удовлетворяет ли ваша машина минимальным требованиям к оборудованию и ОС для установки SQL Server 2008.

1. Щелкните **Пуск** (Start). Щелкните правой кнопкой **Мой компьютер** (My Computer) и выберите **Свойства** (Properties).
2. На вкладке **Общие** (General) в разделе **Система** (System) проверьте, удовлетворяет ли ваша операционная система минимальным требованиям.
3. На вкладке **Общие** в разделе **Компьютер** (Computer) проверьте, удовлетворяет ли ваш компьютер минимальным требованиям к оборудованию.

Задание 2. Проверка выполнения требований к ПО

Вы проверите, удовлетворяет ли ваша машина минимальным требованиям к программному обеспечению.

1. Щелкните **Пуск** (Start) и выберите **Панель управления** (Control Panel).
2. Дважды щелкните **Установка и удаление программ** (Add/Remove Programs).

3. Проверьте наличие минимально необходимых версий Windows Internet Explorer и .NET Framework в следующей последовательности:
 - Щелкните **Пуск** и выберите **Запуск программы (Run)**.
 - Введите в текстовом поле *regedit*.
 - В открывшемся редакторе реестра перейдите к разделу *HKEY_LOCAL_MACHINE\Software\Microsoft\DataAccess*.
 - Проверьте версию MDAC в параметре FullInstallVer.

Резюме занятия

- SQL Server 2008 может устанавливаться как на 32-, так и на 64-разрядные выпуски операционных систем.
- Все выпуски SQL Server 2008 можно устанавливать на Windows Server 2003 Standard SP2 и выше или на Windows Server 2008 Standard и выше.
- SQL Server 2008 нельзя установить на ядро сервера (Server Core) Windows Server 2008.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Вы развертываете новый сервер в компании Wide World Importers, на который планируется установить экземпляр SQL Server 2008, обслуживающий новое приложение. Требуется установить выпуск SQL Server 2008 Enterprise. Какую ОС можно установить на сервер? (Выберите все подходящие варианты.)
 - A. Windows 2000 Server Enterprise SP4 или выше.
 - B. Windows Server 2003 Enterprise.
 - C. Windows Server 2003 Enterprise SP2.
 - D. Windows Server 2008 Enterprise.
2. Вы развертываете SQL Server 2008 Express для поддержки нового веб-приложения, которое позволит клиентам делать заказы непосредственно в компании Coho Vineyards. Какая ОС должна быть на сервере, где планируется установить SQL Server?
 - A. Windows XP Home Edition SP2.
 - B. Ядро сервера Windows Server 2008.
 - C. Windows Server 2003 Enterprise SP2.
 - D. Windows XP Tablet Edition SP2.

Занятие 2. Выбор выпуска SQL Server

Существует несколько выпусков SQL Server 2008, область применения которых разнится от поддержки небольших мобильных встроенных приложений до обслуживания петабайт данных или данных, с которыми одновременно работают миллионы пользователей. Здесь рассказывается о службах SQL Server 2008 и различиях между выпусками SQL Server.

Изучив материал этого занятия, вы сможете:

- ✓ объяснить различия между выпусками Enterprise, Workgroup, Standard и Express сервера SQL Server 2008;
- ✓ объяснить роль каждой службы SQL Server 2008.

Продолжительность занятия — 20 минут.

Службы SQL Server

SQL Server 2008 не просто база данных, способная хранить данные. В SQL Server 2008 есть несколько служб, позволяющих построить практически любое необходимое на предприятии приложение.

Основной компонент SQL Server, Database Engine, включает службы, служащие для хранения, управления, резервного копирования и восстановления данных. Ядро СУБД также обеспечивает расширенные функции безопасности, а также службы обеспечения максимальной доступности. Инфраструктуру данных можно расширить возможностями обработки неструктурированного текста и синхронизации множественных копий базы данных.

Компонент Service Broker

Service Broker был впервые представлен в SQL Server 2005, где служил для организации системы очередей, интегрированной с платформой данных SQL Server. Работа Service Broker базируется на нестандартных сообщениях и их обработке и может использоваться для организации асинхронной обработки данных. Service Broker не только хорошая система обмена сообщениями — с его помощью можно организовать бизнес-процессы и обработку данных в огромном числе различных платформ, при этом пользователям не потребуется ждать завершения процесса, и вообще пользователь может полностью остаться за скобками этого процесса.

Службы Integration Services

Службы Integration Services (SSIS) предоставляют все возможности масштаба предприятия, которые обычно есть в ETL-приложениях (Extract, Transform, and Load), а также позволяют организациям создавать приложения, способные управлять базами данных и системными ресурсами, реагировать на системные события и события баз данных и даже взаимодействовать с пользователями.

В SSIS есть задачи, позволяющие получать или загружать файлы на FTP-сайты, управлять файлами в каталогах, импортировать файлы в базы данных или экспортировать данные в файлы. Службы SSIS могут также выполнять приложения, взаимодействовать с веб-службами, отправлять и получать сообще-

ния из MSMQ-очередей и реагировать на WMI-события (Windows Management Instrumentation). Контейнеры позволяют службам SSIS выполнять целые задачи и потоки операций (workflows) в рамках цикла с разнообразными входными данными — от простого счетчика до файлов в каталоге или результатов запроса. Есть также специализированные задачи, позволяющие копировать объекты SQL Server с одного места среды в другое, а также управлять резервными копиями баз данных, переиндексацией и другими операциями обслуживания. Если в SSIS нет нужной вам задачи в готовом виде, вы можете создать собственные процессы, используя Visual Studio Tools for Applications или даже спроектировать собственные пользовательские задачи, которые можно зарегистрировать и использовать в SSIS.

Ограничения очередностью (precedence constraints) позволяют создавать самые сложные потоки операций, где путь обработки данных может меняться в зависимости от результата деятельности компонента — успех, неудача или простое завершение работы. В дополнение к статической маршрутизации, основанной на состоянии завершения, можно объединять выражения, меняя путь потока операций в зависимости от условий. Обработчики событий позволяют выполнять целые потоки операций в ответ на события, происходящие на уровне пакета или задачи. Например, автоматически выполнять поток операций, перемещающий файл в каталог, если тот не удастся обработать, регистрирующий ошибку в журнале и отправляющий сообщение электронной почты администратору.

Конфигурации пакетов (package configuration) позволяют разработчикам предоставить доступ ко внутренним свойствам пакета, чтобы их можно было менять в зависимости от среды, в которой выполняется пакет. Предоставление доступа к свойствам в конфигурации позволяет администраторам просто переконфигурировать пакет, например изменить имя сервера базы данных или каталогов, не редактируя сам пакет.

Помимо потоков операций службы SSIS поставляются с развитыми компонентами перемещения и модификации данных. Хотя в большинстве случаев будет достаточно простой потоковой задачи, перемещающей данные с одного места в другое, вы можете предусмотреть выполнение массы операций к данным по мере их перемещения — убрать недействительные данные, выполнить вычисления и преобразовать данных из одного типа в другой. Потоки входящих данных можно направить по многим адресатам в зависимости от определенных условий. Задача потока данных позволяет выполнять запросы данных в источниках — для проверки корректности входящих данных или включения дополнительной информации перед передачей данных в пункт назначения. Для расширения возможностей стандартных подстановочных знаков и обеспечения гибкости сверки и группировки можно использовать нечеткие запросы и группировки. Несколько входящих потоков данных можно объединять для передачи в один конечный пункт. Возможен и противоположный сценарий: отправка одного потока данных во многие конечные пункты. В рамках потоковой задачи SSIS можно также менять сопоставления символов, применять операторы *PIVOT* и *UNPIVOT* к наборам данных, вычислять агрегаты, упорядочивать данные, выполнять выборки данных и интеллектуальный анализ текста (text mining). Если в SSIS нет адаптера для обработки формата вашего источника данных или получателя данных или преобразования с нужной логикой, вы всегда можете воспользоваться компонентом

сценариев, позволяющим задействовать все возможности Visual Studio Tools for Applications для обработки своих данных.

Службы Reporting Services

Всем организациям независимо от их размера нужен доступ к обширным объемам данных, причем храниться данные должны так, чтобы обеспечивалась их стандартизация и непротиворечивость. Конечно, неплохо было бы, чтобы каждый знал, как создать запрос источника данных, возвращающий нужные данные, или чтобы всегда под рукой были разработчики, умеющие создать пользовательский интерфейс, предоставляющий все необходимые данные, но большинство организаций не может себе это позволить. Поэтому необходимы инструментальные средства, позволяющие создавать стандартизированные отчеты, которые должны быть доступны любому сотруднику организации, а также надо предоставить пользователям возможность самостоятельно создавать нестандартные отчеты.

Службы SSRS (SQL Server Reporting Services) удовлетворяют потребность в доставке данных, предоставляя гибкую платформу для разработки отчетов, а также распространения данных в рамках организации. ИТ-отдел может быстро создавать сложные отчеты и развертывать их в одном или нескольких порталах, доступом к которым можно управлять, используя гибкие правила безопасности. ИТ-отдел может также проектировать и публиковать модели отчетов, позволяющие пользователям создавать собственные отчеты, не вникая в сложности исходной базы данных. Созданные сотрудниками ИТ-отдела или пользователями отчеты могут развертываться в централизованном портале отчетов, что позволит сотрудникам организации получать информацию, которая им нужна для выполнения их служебных обязанностей.

Пользователи могут пользоваться отчетами, которые генерируются на лету или создаются на основе данных, хранящихся в обновляемом по графику кеше. Пользователи могут подписываться на отчеты — в этом случае службы SSRS создают график генерации отчета и отправки его пользователям по заданному ими каналу и в нужном пользователю формате. Например, менеджер по продажам может подписаться на отчет о ежедневных продажах, который генерируется в полночь, по завершении всех сбытовых операций, поставляется в виде PDF-документа и отправляется в почтовый ящик менеджера, чтобы тот мог просмотреть его утром.

В состав SSRS входят два основных компонента: сервер отчетов и конструктор отчетов.

Сервер отчетов хранит и обеспечивает безопасность всех отчетов. При запросе отчета сервер отчетов «обязан» подключиться к нужным источникам данных, собрать данные и подготовить отчет в нужном формате. Подготовка отчетов выполняется по требованию пользователя или в рамках запланированного задания, позволяющего создавать отчеты во время минимальной загрузки сервера.

Чтобы сервер отчетов мог предоставить пользователям какие-либо отчеты, их нужно создать. Для создания и отладки отчетов служит конструктор отчетов. В SSRS есть также компоненты, позволяющие пользователям создавать как простые табличные или матричные, так и более сложные отчеты со многими под-

чиненными или вложенными отчетами, диаграммами, связанными отчетами и ссылками на внешние ресурсы. В самих отчетах можно использовать вычисления и функции, объединять таблицы и даже изменять результаты отчета в зависимости от того, какой пользователь заказал его. Конструктор отчетов также служит для создания моделей отчетов, являющихся мощным уровнем обработки семантики, маскирующим от пользователей сложности источника данных и предоставляющим им возможность сосредоточиться на создании отчетов.

Службы Analysis Services

Взрывообразный рост объема данных в организации требует развертывать инструментальные средства, которые позволили бы пользователям принимать бизнес-решения практически в масштабе реального времени. Пользователи больше не могут ждать, пока в ИТ-отделе спроектируют отчеты, отвечающие на сотни вопросов, ответы на которые нужны пользователям немедленно. Вместе с тем у ИТ-отдела нет возможности создавать сотни отчетов, необходимых сотрудникам компании для нормальной работы.

Службы SSAS (SQL Server Analysis Services) призваны заполнить пропасть между потребностями в данных рядовых пользователей и возможностями ИТ-отдела по обеспечению этих данных. В SSAS два компонента — OLAP (Online Analytical Processing) и компонент интеллектуального анализа данных (Data Mining).

Компонент OLAP позволяет развертывать, запрашивать и управлять кубами, спроектированными в среде Business Intelligence Development Studio (BIDS). Вы можете добавить множественные измерения и иерархии в рамках одного измерения, а также выбирать разные варианты настройки, например выбрать, какие атрибуты должны отображаться и как сортировать элементы. Можно конструировать меры — от простых аддитивных элементов до сложных пользовательских схем агрегирования. Можно добавлять ключевые индикаторы производительности (Key Performance Indicators, KPI), которые предоставляют наглядные очереди, информирующие пользователей о состоянии бизнес-объектов. Кубы могут содержать перспективы, определенные как подмножество данных в рамках одного куба и упрощающие его просмотр. Встроенный уровень метаданных позволяет определять преобразования языков на любом уровне куба, предоставляя пользователям возможность просматривать данные на своем родном языке.

Модуль интеллектуального анализа (Data Mining) расширяет возможности бизнес-анализа, позволяя пользователям находить шаблоны (pattern) и создавать прогнозы. Используя поставляемые в составе SQL Server алгоритмы интеллектуального анализа, можно исследовать тенденции изменения данных во времени, определять, какие факторы влияют на решение о покупке или даже использовать обнаруженные шаблоны поведения клиентов при покупке для совершенствования методик продаж.

ПРИМЕЧАНИЕ Подробнее о службах SQL Server

Подробнее о каждом виде функциональности платформы для работы с данными в SQL Server 2008 см. книгу «Microsoft SQL Server 2008 Step by Step» (Microsoft Press, 2008), где каждой функции SQL Server 2008 посвящена отдельная обзорная глава.

Выпуски SQL Server

- **Enterprise** предназначен для крупных предприятий и тем, кому нужна вся мощь платформы SQL Server 2008.
- **Standard** ориентирован на малые и средние предприятия, которым не нужны все возможности, имеющиеся в выпуске Enterprise.
- **Workgroup** подходит для мелких внутриведомственных проектов, где нужен ограниченный набор функциональных возможностей SQL Server 2008.
- **Express** свободно распространяемый выпуск SQL Server, обслуживающий внедренные приложения, а также удовлетворяющий базовые потребности в хранении данных в серверных приложениях, например в веб-приложениях с небольшим числом пользователей.
- **Compact** представляет собой внедренную базу данных.
- **Developer** адресован разработчиками и предназначен для создания приложений для SQL Server. В SQL Server 2008 Developer имеются все те же функции и возможности, что и в SQL Server 2008 Enterprise, за исключением того, что этот выпуск нельзя использовать в промышленной среде.
- **Evaluation** позволяет оценить возможности SQL Server 2008. В этом оценочном выпуске SQL Server 2008 есть все те же функции и возможности, что и в SQL Server 2008 Enterprise, за исключением того, что этот выпуск нельзя использовать дольше 180 дней.

ПРИМЕЧАНИЕ Изучаемые выпуски SQL Server

В этой книге мы расскажем только о выпусках, которые можно развертывать в промышленной среде: Express, Workgroup, Standard и Enterprise.

Основные различия между выпусками SQL Server заключаются в поддерживаемом оборудовании и наборе функций. Приведенная ниже таблица содержит обзорную информацию о различных выпусках SQL Server.

Табл. 1-2. Поддерживаемое оборудование

Оборудование	Standard	Workgroup	Express	Compact
Число процессоров	4	4	1	1
Размер базы данных	Неограниченный	Неограниченный	4 Гб	4 Гб
Размер оперативной памяти	Неограниченный	Неограниченный	1 Гб	1 Гб

Табл. 1-3. Компоненты СУБД

Компонент	Standard	Workgroup	Express	Compact
SQL Server Management Studio	Да	Да	Доступен отдельно для загрузки с веб-сайта	Нет
Полнотекстовый поиск	Да	Да	Только при наличии дополнительных служб	Нет
Секционирование	Нет	Нет	Нет	Нет

Табл. 1-3. (окончание)

Компонент	Standard	Workgroup	Express	Compact
Параллельные операции	Нет	Нет	Нет	Нет
Множественные экземпляры	Нет	Нет	Нет	Нет
Моментальные снимки баз данных	Нет	Нет	Нет	Нет
Масштабируемые общие базы данных	Нет	Нет	Нет	Нет
Индексированные представления	Нет	Нет	Нет	Нет
Сжатие журналов	Нет	Нет	Нет	Нет
Кластеризация	2 узла	Нет	Нет	Нет
Зеркальное отображение баз данных	Один поток	Нет	Нет	Нет
Оперативные операции	Нет	Нет	Нет	Нет
Регулятор ресурсов	Нет	Нет	Нет	Нет
Сжатие резервных копий	Нет	Нет	Нет	Нет
Горячее добавление модулей памяти и процессора	Нет	Нет	Нет	Нет
Шифрование данных	Ограничено	Ограничено	Ограничено	Только на основе пароля
Отслеживание изменений в данных	Нет	Нет	Нет	Нет
Сжатие данных	Нет	Нет	Нет	Нет
Управление на основе политик	Да	Нет	Нет	Нет
Сбор данных производительности	Да	Нет	Нет	Нет
CLR	Да			
XML	Встроенная поддержка	Встроенная поддержка	Встроенная поддержка	Хранение в виде текста
Пространственные данные	Да			
Сохраненные процедуры, триггеры и представления	Да	Да	Да	Нет
Репликация слияния	Да	Да	Только в режиме подписчика	Только в режиме подписчика
Репликация транзакций	Да	Только в режиме подписчика	Только в режиме подписчика	Нет

Табл. 1-4. Поддержка SSIS

Функциональность	Standard	Workgroup	Express	Compact
Мастера импорта и экспорта	Да	Да	–	–
Конструктор пакетов	Да	Да	–	–
Интеллектуальная обработка данных (Data Mining)	Нет	Нет	–	–
Нечеткая группировка и поиск	Нет	Нет	–	–
Извлечение/поиск терминов	Нет	Нет	–	–
OLAP-обработка	Нет	Нет	–	–

Табл. 1-5. Поддержка SSRS

Функциональность	Standard	Workgroup	Express	Compact
Интеграция с Microsoft Office	Да	Да	Только при наличии дополнительных служб	N/A
Построитель отчетов	Да	Да	Только при наличии дополнительных служб	N/A
Масштабное развертывание отчетов	Нет	Нет	Нет	N/A
Управление подпиской на основе данных	Нет	Нет	Нет	N/A

Табл. 1-6. Поддержка SSAS — OLAP

Функциональность	Standard	Workgroup	Express	Compact
Связанные меры/измерения	Нет	Нет	–	–
Перспективы	Нет	Нет	–	–
Секционированные кубы	Нет	Нет	–	–

Табл. 1-7. Поддержка SSAS — Интеллектуальная обработка данных

Функциональность	Standard	Workgroup	Express	Compact
Временные ряды	Нет	Нет	–	–
Параллельная обработка и прогнозирование	Нет	Нет	–	–
Дополнительные возможности интеллектуального анализа данных	Нет	Нет	–	–

ПРИМЕЧАНИЕ На экзамене

Вы должны знать основное предназначение каждого выпуска SQL Server, в том числе набор функциональности и различия в требованиях к памяти и тактовой частоте процессора.

Контрольные вопросы

1. Какие выпуски содержат полный набор функциональности SQL Server? Какие из этих выпусков нельзя использовать в промышленной среде?
2. Какие выпуски SQL Server предназначены для хранения данных во внедренных приложениях и ограничены в плане функциональности и оборудования, на котором они могут работать?

Ответы на контрольные вопросы

1. Выпуски Enterprise, Developer и Evaluation содержат полный набор функциональности SQL Server 2008. Выпуски Developer и Evaluation не лицензируются для использования в промышленной среде.
2. Выпуски Express и Compact разработаны для использования в качестве баз данных во внедренных приложениях и могут работать в системе только с одним процессором, объемом оперативной памяти не более 1 Гб, а размер базы данных не может превышать 4 Гб.

Резюме занятия

- Несколько выпусков SQL Server 2008 доступны для использования в промышленной среде — Enterprise, Standard, Workgroup, Express и Compact.
- Помимо базовых технологий баз данных выпуск SQL Server 2008 Enterprise содержит компонент Service Broker, поддерживающий асинхронную обработку.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Компания Margie's Travel открывает новый отдел, предоставляющий клиентам возможность заказывать туры в онлайн. Руководство ожидает быстрого роста трафика, когда в каждый момент времени сотни пользователей станут просматривать предложения и заказывать туры. Руководство также хотело бы синхронизировать множественные копии базы данных о заказах туров, чтобы обеспечить поддержку как онлайн-овых, так и личных продаж в офисах продаж. Какой выпуск SQL Server 2008 позволит удовлетворить эти требования? (Выберите все подходящие варианты.)
 - A. Express.
 - B. Standard.
 - C. Enterprise.
 - D. Compact.
2. Руководство Margie's Travel решило сэкономить деньги и приобрести выпуск SQL Server 2008 Standard для нового отдела. После успешного запуска менеджеры стали испытывать трудности с бизнес-операциями, поэтому пришлось развернуть дополнительные аналитические средства. Планируется

установить новый сервер с SQL Server. Какой выпуск SQL Server нужно установить на новом сервере, чтобы предоставить менеджерам доступ к аналитическим средствам?

- A. SQL Server 2008 Standard.
- B. SQL Server 2008 Express with Advanced Services.
- C. SQL Server 2008 Workgroup.
- D. SQL Server 2008 Enterprise.

Занятие 3. Установка и конфигурирование экземпляров SQL Server

На этом занятии вы узнаете, как создавать и конфигурировать учетные записи служб, которые будут использоваться для работы выбранных для установки служб SQL Server. Здесь также рассказывается о режиме проверки подлинности и параметрах сортировки, определяемых во время установки экземпляра. В конце занятия речь пойдет о конфигурировании и управлении службами SQL Server после установки.

Изучив материал этого занятия, вы сможете:

- ✓ создавать учетные записи служб;
- ✓ устанавливать SQL Server 2008;
- ✓ разбираться в параметрах сортировки;
- ✓ разбираться в режимах проверки подлинности;
- ✓ устанавливать образцы баз данных;
- ✓ конфигурировать экземпляр SQL Server.

Продолжительность занятия — 40 минут.

Учетные записи служб

Все ключевые компоненты SQL Server реализованы как службы. Чтобы должным образом конфигурировать компоненты, до установки нужно создать несколько учетных записей служб. Необходимы выделенные учетные записи служб для следующих компонентов:

- ядро базы данных;
- агент SQL Server.

Учетные записи служб SQL Server служат не только для предоставления приложениям доступа к данным и планированию заданий, но и для определения границ безопасности. Ядру SQL Server требуется доступ ко многим ресурсам на компьютере, таким как память, процессоры, дисковое пространство и сети. Однако служба SQL Server работает в рамках инфраструктуры безопасности Windows. SQL Server получает доступ только к тем ресурсам Windows, к которым такой доступ предоставлен разрешениями учетной записи службы.

ПРИМЕЧАНИЕ Безопасность SQL Server

Подробнее о безопасности SQL Server см. главу 11.

ПРИМЕЧАНИЕ Версия операционной системы

Для выполнения всех упражнений в этой книге я использовал Windows XP Professional. Вам придется сделать поправку на версию Windows, которую будете использовать. Кроме того, если ваш компьютер является членом домена, учетные записи служб должны быть учетными записями домена, а не локальными.

Параметры сортировки

Параметры сортировки (collation sequence) определяют порядок обработки сервером SQL Server символьных данных в процессе их хранения, поиска, сортировки и сравнения. Механизмы определения параметров сортировки в SQL Server 2008 позволяют обеспечить поддержку любого существующего в настоящее время в мире языка.

Параметры сортировки могут определяться на уровне экземпляра, базы данных, таблицы или даже столбца. Единственный обязательный порядок сортировки определен на уровне экземпляра и по умолчанию применяется на всех остальных уровнях, если явно не задан другой порядок.

Параметры сортировки определяют поддерживаемый набор символов, в том числе чувствительность к регистру, диакритическим знакам и выбору таблицы каны (в японском языке). Например, выбрав сортировку SQL_Latin1_General_CP1_CI_AI, вы получаете поддержку западноевропейского набора символов, в котором не учитывается регистр и диакритические знаки. То есть в операциях сортировки и сравнения символы е, Е, è, é, ê и ë не различаются, а французская сортировка с учетом регистра (CS) диакритических знаков (AS) заставит считать эти символы разными.

ПРИМЕЧАНИЕ Параметры сортировки

Подробнее о параметрах сортировки см. главу 2.

Режимы проверки подлинности

Один из определяемых при установке параметров конфигурации экземпляра — режим проверки подлинности, который SQL Server использует для управления порядком входа в систему. Есть два режима проверки подлинности:

- Windows (интегрированная защита) [Windows Only (integrated security)];
- Windows и SQL Server (смешанный режим) [Windows and SQL Server (mixed mode)].

При выборе первого варианта для входа в систему экземпляра SQL Server могут использоваться только учетные записи Windows. Во втором варианте для этого годятся учетные записи как Windows, так и SQL Server.

ПРИМЕЧАНИЕ Вход в систему

Подробнее о входе в систему см. главу 11.

Экземпляры SQL Server

Экземпляр SQL Server фактически является контейнером всех операций, выполняемых в рамках SQL Server. Каждый экземпляр содержит собственный

набор баз данных, учетные данные безопасности, параметры конфигурации, служб Windows и других объектов SQL Server.

SQL Server 2008 позволяет установить на одной машине до 50 экземпляров SQL Server. Можно установить один экземпляр как экземпляр по умолчанию, а остальные 49 — как дополнительные именованные экземпляры, или 50 именованных экземпляров без экземпляра по умолчанию.

При подключении к экземпляру SQL Server по умолчанию используется имя машины, где он установлен. При подключении к именованному экземпляру нужно указывать им имя машины, и имя экземпляра в такой последовательности: `<имя_машины>\<имя_экземпляра>`.

Есть несколько причин установки более одного экземпляра SQL Server на одной машине, вот основные из них:

- нужны экземпляры для тестирования или разработки;
- нужна поддержка экземпляров с разными пакетами исправлений;
- в организации существуют различные группы администраторов, которым разрешен доступ только к определенному подмножеству баз данных организации;
- нужны экземпляры SQL Server с различными конфигурациями.

ПРИМЕЧАНИЕ Множественные экземпляры SQL Server

Установку множественных экземпляров на одной машине поддерживает только выпуск SQL Server 2008 Enterprise.

ПРИМЕЧАНИЕ На экзамене

Вы должны знать, как параметры сортировки определяют порядок обработки символьных данных сервером SQL Server.

Диспетчер конфигурации SQL Server

Диспетчер конфигурации SQL Server (SQL Server Configuration Manager) (рис. 1-1) служит для управления службами и протоколами SQL Server. Основные задачи, выполняемые с помощью диспетчера конфигурации:

- запуск, остановка, приостановка и перезапуск службы;
- смена учетных записей служб и паролей учетных записей;
- управление режимом запуска службы;
- определение параметров запуска службы.

По завершении начальной установки и конфигурирования служб SQL Server Диспетчер конфигурации SQL Server в основном используется для периодической смены пароля учетной записи службы. При изменении пароля учетной записи службы больше не нужно перезапускать экземпляр SQL Server, чтобы изменения вступили в силу.

ВНИМАНИЕ! Консоль «Службы» в Windows

Службы SQL Server также видны в Windows-консоли «Службы» (Services). Консоль позволяет изменять учетные записи и пароли служб, но этого никогда не нужно делать. Для этих целей следует использовать только Диспетчер конфигурации SQL Server, так как только он «умеет» пересоздавать главный ключ службы, который критически важен для нормальной работы служб SQL Server.

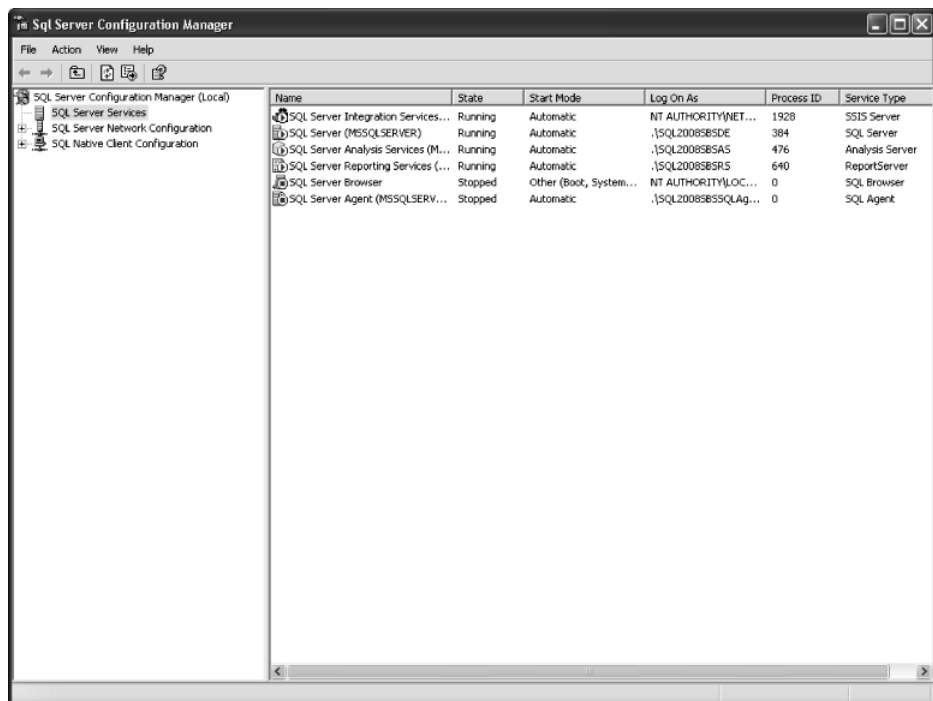


Рис. 1-1. Список служб в Диспетчере конфигурации SQL Server

Вы вправе запускать, останавливать, приостанавливать и перезапускать службы SQL Server, однако SQL Server обладает обширными возможностями управления, которые гарантируют, что вам вряд ли когда-либо придется останавливать или перезапускать службу SQL Server.

Диспетчер конфигурации SQL Server также позволяет конфигурировать протоколы связи, используемые для подключения клиентов. Помимо конфигурирования специфических параметров протоколов, можно задавать, требуется ли шифрование подключений или должен ли экземпляр реагировать на запрос перечисления (рис. 1-2).

ПРИМЕЧАНИЕ Консоль «Службы» в Windows

Приложения могут направлять в сеть специальную команду, которая называется *запрос перечисления*, для определения местонахождения серверов, где работает SQL Server. Возможность перечисления серверов с SQL Server полезна в среде разработки и тестирования, где экземпляры могут появляться, исчезать и восстанавливаться относительно часто, но в промышленной среде перечисление нежелательно. Отключая реакцию на запрос перечисления путем присвоения параметру «Скрыть экземпляр» (Hide Instance) значение «Да» (Yes), вы не позволяете злоумышленникам использовать методы перечисления для определения местонахождения серверов с SQL Server.

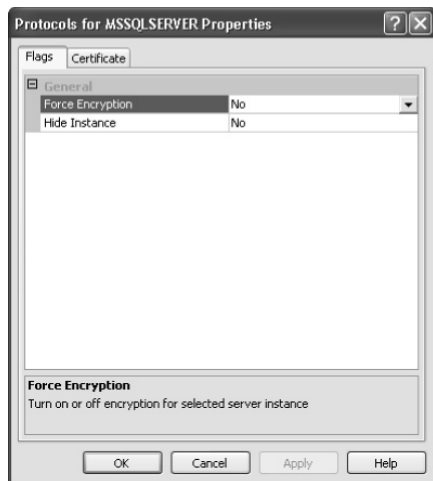


Рис. 1-2. Окно свойств протокола

Контрольные вопросы

1. Какой выпуск SQL Server поддерживает установку на одной машине более одного экземпляра SQL Server?
2. Каковы поддерживаемые SQL Server режимы проверки подлинности?

Ответы на контрольные вопросы

1. Устанавливать множественные экземпляры SQL Server на одной машине позволяет только выпуск SQL Server Enterprise.
2. SQL Server позволяет выбрать один из двух режимов проверки подлинности: интегрированная защита (только Windows) и смешанный (Windows и SQL Server).

Практикум. Установка и конфигурирование экземпляра SQL Server

Вы создадите учетные записи служб, установите экземпляр SQL Server по умолчанию и установите тестовую базу данных AdventureWorks, которая будет использоваться в этой книге.

Задание 1. Создание учетных записей служб

Вы создадите учетную запись службы, которая будет использоваться для работы службы SQL Server.

1. Щелкните **Пуск** (Start), а затем щелкните правой кнопкой **Мой компьютер** (My computer) и выберите **Управление** (Manage).
2. Последовательно разверните узлы **Локальные пользователи и группы** (Local Users And Groups) и **Пользователи** (Users).
3. Щелкните правой кнопкой в правой панели окна и выберите **Новый пользователь** (New User).

4. В поле **Пользователь** (User Name) укажите значение **SQL2008TK432DE**, задайте надежный пароль, сбросьте флажок **Требовать смены пароля при следующем входе в систему** (User Must Change Password At Next Logon) и установите флажок **Срок действия пароля не ограничен** (Password Never Expires).
5. Повторите пп. 3 и 4, чтобы создать еще одну такую же учетную запись по имени **SQL2008TK432SQLAgent**.

Задание 2. Установка экземпляра SQL Server

Вы установите экземпляр SQL Server.

1. Запустите программу установки SQL Server.
2. Если вы еще не установили ПО, необходимое для работы SQL Server 2008, программа установки установит необходимые компоненты.
3. После установки необходимого ПО откроется основное окно установки SQL Server (рис. 1-3).

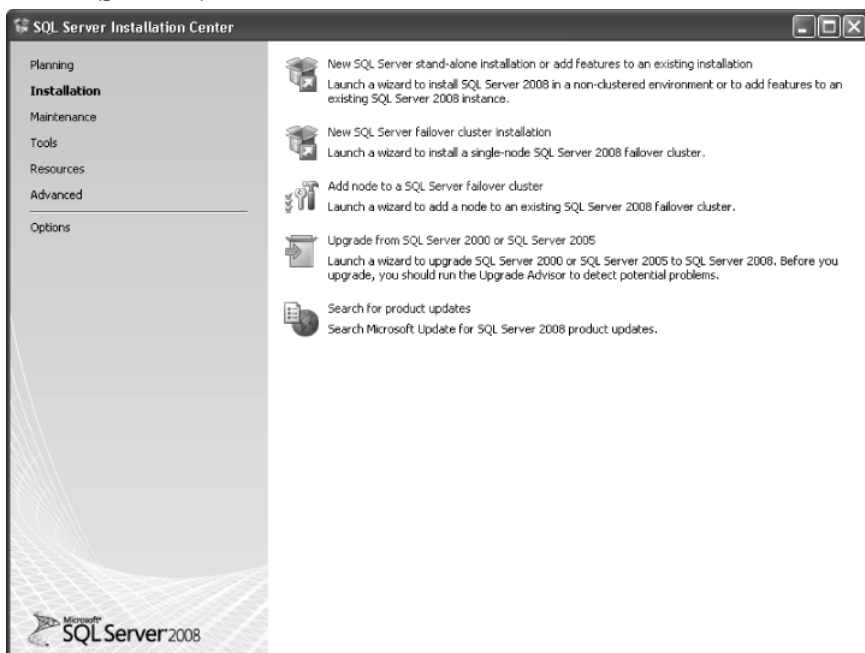


Рис. 1-3. Основное окно установки SQL Server

4. Щелкните **Новая установка изолированного SQL Server** (New SQL Server stand-alone Installation), чтобы запустить установку SQL Server.
5. Программа установки выполнит проверку конфигурации системы. В случае успешной проверки откроется окно, показанное на рис. 1-4.

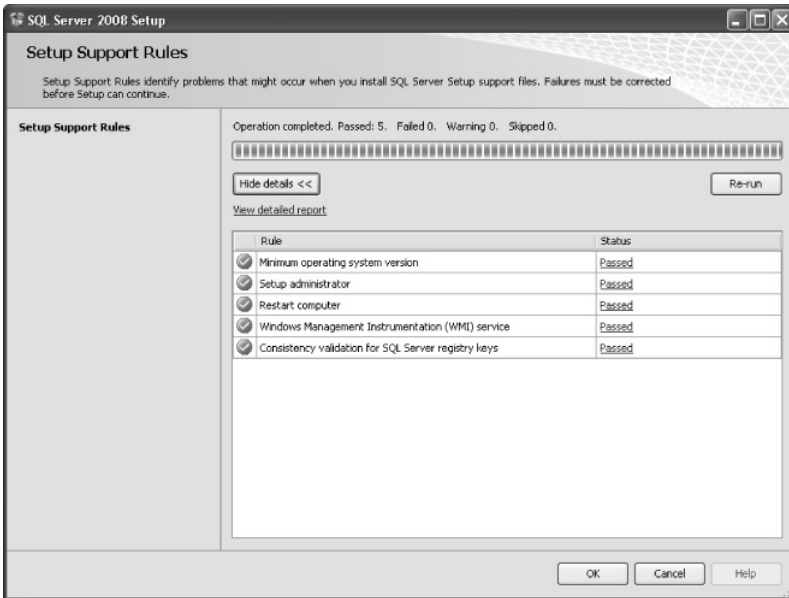


Рис. 1-4. Окно успешного завершения проверки конфигурации системы

6. Щелкните **ОК**. Выберите нужный выпуск SQL Server и щелкните **Далее** (Next).
7. Щелкните **Далее**. Выберите необходимые компоненты (рис. 1-5) и щелкните **Далее**.

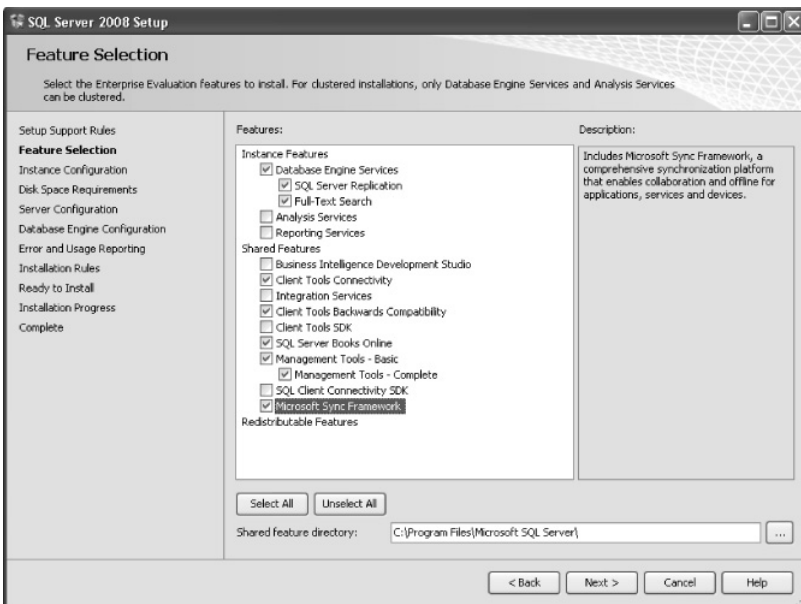


Рис. 1-5. Окно выбора компонентов

8. Проверьте требования к дисковому пространству и щелкните **Далее**.

9. На странице **Настройка экземпляра** (Instance Configuration) выберите **Экземпляр по умолчанию** (Default Instance) и щелкните **Далее**.
10. Введите имена учетных записей служб, созданных в задании 1. После этого страница должна выглядеть, как на рис. 1-6.

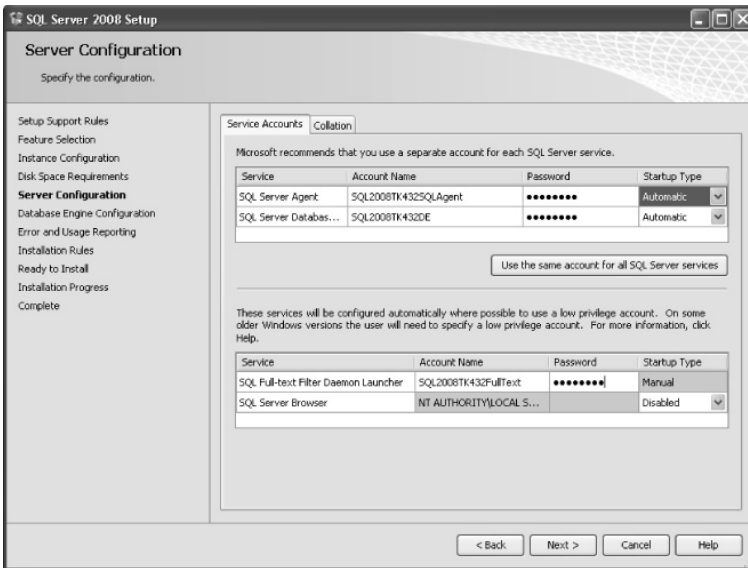


Рис. 1-6. Определение учетных записей служб

11. Перейдите на вкладку **Параметры сортировки** (Collation), чтобы проверить порядок сортировки компонента Database Engine. Выполните необходимую корректировку в соответствии с поддерживаемым языком. Щелкните **Далее**.
12. На странице **Настройка компонента Database Engine** (Database Engine Configuration) выберите **Смешанный режим проверки подлинности SQL Server и Windows** [Mixed Mode (SQL Server Authentication And Windows Authentication)] и задайте пароль. Щелкните **Добавить текущего пользователя** (Add Current User) и добавьте учетную запись Windows, которая будет администратором SQL Server и под которой будет выполняться установка. Щелкните **Добавить** (Add) и добавьте другие учетные записи Windows, которые должны быть администраторами SQL Server.
13. Перейдите на вкладку **Каталоги данных** (Data Directories) и проверьте параметры настройки.

ПРИМЕЧАНИЕ Базы данных и каталоги

Подробнее о создании баз данных, файлов баз данных и каталогах см. главу 2.

14. Перейдите на вкладку **FILESTREAM** и последовательно установите флажки **Разрешить FILESTREAM при доступе через Transact-SQL** (Enable FILESTREAM for Transact-SQL Access) и **Разрешить FILESTREAM при потоковом доступе файлового ввода-вывода** (Enable FILESTREAM For File I/O Streaming Access). В поле общего ресурса Windows оставьте заданное по умолчанию значение MSSQLSERVER. Щелкните **Далее**.

- Щелкните **Пуск** (Start), а затем последовательно выберите **Все программы** (All Programs), **Microsoft SQL Server 2008** и **Среда SQL Server Management Studio** (SQL Server Management Studio).
- Если вы еще не указали, задайте имя машины, на которой вы ранее установили свой экземпляр SQL Server. Щелкните **Соединить** (Connect). Ваш экран должен выглядеть, как показано на рис. 1-7.
- Щелкните **Создать запрос** (New Query). Введите следующий код и щелкните **Выполнить** (Execute):

```
EXEC sp_configure 'filesystem_access_level', 2;  
GO  
RECONFIGURE  
GO  
RESTORE DATABASE AdventureWorks FROM DISK='C:\Program Files\Microsoft SQL  
Server\100\Tools\Samples\AdventureWorks2008.bak' WITH RECOVERY;  
GO  
RESTORE DATABASE AdventureWorksDW FROM DISK='C:\Program Files\Microsoft  
SQL Server\100\Tools\Samples\AdventureWorksDW2008.bak' WITH RECOVERY;  
GO
```

- Когда вы развернете узел **Базы данных** (Database), ваш экран должен выглядеть, как показано на рис. 1-8.

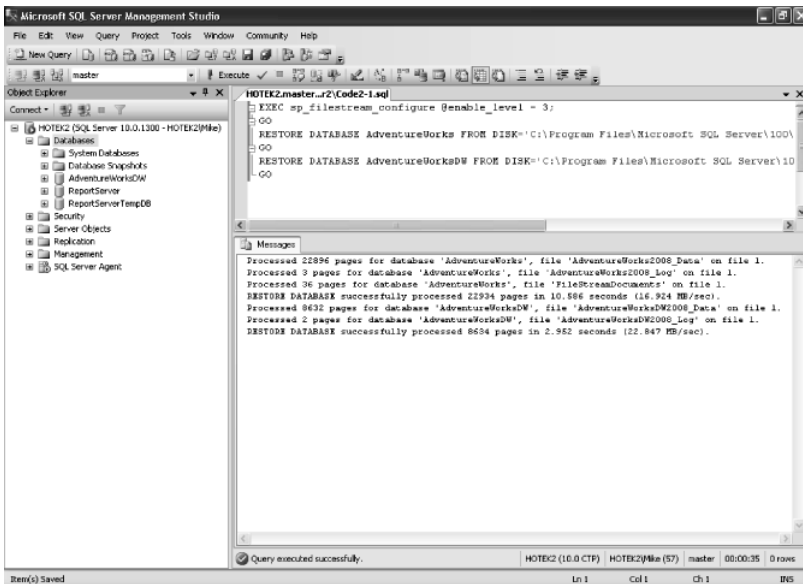


Рис. 1-8. Установка базы данных AdventureWorks

Резюме занятия

- Ядро SQL Server и Агент SQL Server работают как службы, и для них нужно создать учетные записи служб с соответствующими разрешениями Windows, предоставляющими доступ к нужным ресурсам Windows.

- SQL Server позволяет выбрать один из двух режимов проверки подлинности: интегрированная защита (только Windows) и смешанный (Windows и SQL Server).
- Параметры сортировки определяют, как SQL Server хранит и управляет символьными данными.
- На одной машине можно установить до 50 экземпляров SQL Server; однако множественные экземпляры поддерживает только выпуск Enterprise Edition.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. В компании Wide World Importers планируют использовать новый тип данных *FILESTREAM* для хранения отсканированных изображений путевых листов. Какую команду нужно выполнить в экземпляре SQL Server, прежде чем в нем можно будет хранить данные типа *FILESTREAM*?
A. *ALTER DATABASE*
B. *DBCC*
C. *sp_configure*
D. *sp_filestream_configure*
2. В компании Contoso реализовали новую политику, которая требует каждые 30 дней менять пароли всех учетных записей служб. Какой инструмент администраторы баз данных Contoso должны использовать для изменения пароля учетной записи службы SQL Server, чтобы удовлетворить требованиям новой политики?
A. Windows-консоль «Службы» (Services).
B. Среду SQL Server Management Studio.
C. Диспетчер конфигурации SQL Server (SQL Server Configuration Manager).
D. Диспетчер настройки контактной зоны SQL Server (SQL Server Surface Area Configuration Manager).

Занятие 4. Конфигурирование компонента Database Mail

Компонент Database Mail обеспечивает поддержку уведомлений в экземплярах SQL Server. На этом занятии рассказывается об особенностях компонента Database Mail и о том, как его конфигурировать в экземпляре SQL Server 2008.

Изучив материал этого занятия, вы сможете:

- ✓ конфигурировать компонент Database Mail;
- ✓ отправлять сообщения средствами Database Mail.

Продолжительность занятия — 30 минут.

Компонент Database Mail

Он позволяет SQL Server отправлять исходящие почтовые сообщения. Хотя сообщения могут содержать результаты запросов, Database Mail прежде всего используется для отправки предупреждений администраторам об изменении поведения сервера или объектов.

Компонент Database Mail впервые появился в SQL Server 2005 вместо SQL Mail. Причины замены были очень просты:

- избавиться от зависимости от MAPI (Microsoft Mail Application Programming Interface);
- упростить конфигурирование и управление;
- предоставить быстрый и надежный механизм отправки почтовых сообщений.

Database Mail использует службу ретрансляции SMTP (Simple Mail Transfer Protocol), имеющуюся на всех Windows-машинах и служащую для пересылки почтовых сообщений. При инициировании отправки почты сообщение вместе со всеми его свойствами вносится в таблицу в базе данных *Msdb*. Периодически выполняется фоновая задача, управляемая Агентом SQL Server. При выполнении процесса отправки почты все еще не отправленные сообщения из очереди отправки собираются и пересылаются с использованием соответствующего почтового профиля.

Почтовые профили составляют основу Database Mail. Профиль может содержать несколько почтовых учетных записей, чтобы обеспечить переход на случай недоступности почтового сервера. Почтовые учетные записи содержат все свойства, относящиеся к определенной почтовой учетной записи, такие как адреса электронной почты, адрес для ответа, имя почтового сервера, номер порта и учетные данные для проверки подлинности.

Доступ к почтовому профилю можно защитить, ограничив возможности пользователей отправлять почту, используя данный профиль. Профиль может создаваться открытый или закрытый. Открытый профиль доступен всем пользователям, способным отправлять почту, а закрытый — только тем пользователям, которым доступ к почтовому профилю предоставлен явно.

Почтовый профиль также можно назначить профилем по умолчанию. SQL Server будет использовать его для отправки сообщений, когда почтовый профиль не указан.

Контрольный вопрос

- Какие существуют два типа почтовых профилей?

Ответ на контрольный вопрос

- Почтовые профили делятся на открытые и закрытые.

Практикум. Конфигурирование Database Mail

Вы конфигурируете Database Mail и отправите тестовое почтовое сообщение.

1. Откройте Среду SQL Server Management Studio, подключитесь к своему экземпляру SQL Server и щелкните **Создать запрос (New Query)**. Выполните следующий код, чтобы активизировать Database Mail:

```
EXEC sp_configure 'Database Mail XPs',1  
GO  
RECONFIGURE WITH OVERRIDE  
GO
```

2. В панели **Обозреватель объектов (Object Explorer)** откройте узел **Управление**, щелкните правой кнопкой **Компонент Database Mail (Database Mail)** и выберите команду **Настроить компонент Database Mail (Configure Database Mail)**.
3. На первой странице мастера щелкните **Далее (Next)**.
4. Выберите **Установить компонент Database Mail (Set Up Database Mail By Performing The Following Tasks)** и щелкните **Далее**.
5. Задайте имя профиля и щелкните **Добавить (Add)**, чтобы определить параметры учетной записи электронной почты.
6. Заполните поля **Имя учетной записи (Account Name)**, **Адрес электронной почты (E-mail Address)**, **Отображаемое имя (Display Name)**, **Адрес электронной почты для ответа (Reply E-mail)** и **Имя сервера (Server Name)** (рис. 1-9).

Рис. 1-9. Настройка компонента Database Mail

ПРИМЕЧАНИЕ Настройка Database Mail

Я воспользовался своей интернетовской учетной записью электронной почты и преднамеренно оставил пустыми поля «Имя сервера», «Имя пользователя» и «Пароль» на рис. 1-9. Если вы используете внутренний почтовый сервер, придется указать имя сервера.

7. Выберите нужный режим проверки подлинности SMTP и, в случае выбора варианта **Обычная проверка подлинности** (Basic authentication), укажите имя пользователя и пароль.
8. Щелкните **ОК**, а затем — **Далее**.
9. Отметьте вновь созданный профиль флажком в столбце **Открытый** (Public) и выберите вариант **Да** (Yes) в столбце **Профиль по умолчанию** (Default Profile). Щелкните **Далее**.
10. Просмотрите параметры настройки на странице **Установка системных параметров** (Configure System Parameters). Щелкните **Далее**.
11. Щелкните **ОК**, а затем — **Далее**. Щелкните **Готово** (Finish).
12. Страница завершения работы мастера должна информировать, что все четыре этапа создания профиля завершены успешно. Щелкните **Заккрыть** (Close).
13. Щелкните правой кнопкой **Компонент Database Mail** (Database Mail) и выберите **Отправить тестовое сообщение** (Send Test E-mail).
14. Выберите только что созданный профиль Database Mail, укажите адрес электронной почты в поле **Кому** (To) и щелкните **Отправить тестовое сообщение** (Send Test E-Mail) (рис. 1-10).

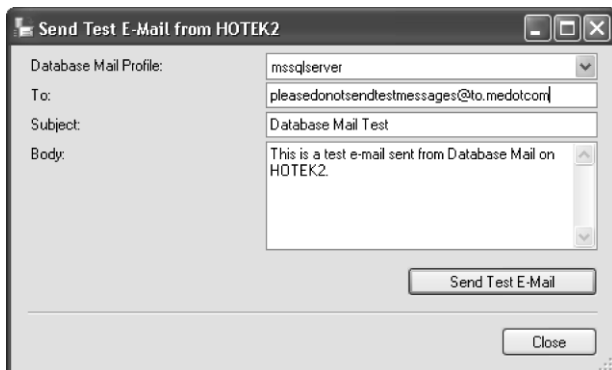


Рис. 1-10. Отправка тестового почтового сообщения

15. Откройте свой почтовый клиент и убедитесь в получении тестового сообщения.

ПРИМЕЧАНИЕ Отправка почты

Database Mail использует службы Агента SQL Server для отправки сообщений в фоновом режиме. Если Агент SQL Server остановлен, сообщения накапливаются в очереди в базе данных *MsdB*.

Резюме занятия

- Компонент Database Mail служит для отправки почтовых сообщений экземпляром SQL Server.
- Почтовые сообщения не отправляются, если не работает Агент SQL Server.
- Почтовый профиль может содержать одну или более почтовых учетных записей.

- Почтовые профили делятся на открытые и закрытые.
- Если при отправке сообщения не указать профиль, будет использован профиль по умолчанию.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. В рамках реализации новой системы заказа туров через Интернет в компании Margie's Travel планируется предоставить клиентам возможность получать уведомления об успешном заказе тура. Какие технологии или функции могут задействовать разработчики Margie's Travel для реализации системы уведомлений? (Выберите все подходящие варианты.)
 - A. Службы Notification Services.
 - B. Компонент Database Mail.
 - C. Библиотеки кода Microsoft Visual Studio.NET.
 - D. Монитор активности (Activity Monitor).
2. Разработчики компании Margie's Travel решили использовать Database Mail для отправки сообщений клиентам. Нужно ограничить возможность отправлять почтовые сообщения через данный профиль, но при этом не нужно, чтобы одобренный пользователь определял почтовый профиль при отправке уведомлений. Какие параметры настройки надо сконфигурировать, чтобы удовлетворить этим требованиям? (Выберите все подходящие варианты.)
 - A. Сконфигурировать почтовый профиль как открытый.
 - B. Сконфигурировать почтовый профиль как закрытый.
 - C. Сконфигурировать почтовый профиль как открытый и предоставить доступ к нему одобренным пользователям.
 - D. Сконфигурировать почтовый профиль как профиль по умолчанию.

Закрепление материала главы

Для закрепления знаний и навыков, полученных при изучении этой главы, выполните следующее:

- изучите резюме главы;
- повторите основные термины, использованные в этой главе;
- выполните задания по темам этой главы;
- сделайте предложенные упражнения;
- пройдите пробный экзамен.

Резюме главы

- Существует несколько выпусков SQL Server 2008: Enterprise, Standard, Workgroup, Express и Compact.
- SQL Server работает как Windows-служба, поэтому в процессе установки для него нужно задать учетную запись службы.

- SQL Server позволяет выбрать один из двух режимов проверки подлинности: интегрированная защита (только Windows) и смешанный (Windows и SQL Server).
- Диспетчер конфигурации SQL Server служит для управления всеми службами SQL Server.
- В SQL Server можно активизировать и сконфигурировать компонент Database Mail, который позволяет пользователям и приложениям отправлять почтовые сообщения.

Основные термины

Понятны ли вам значения приведенных здесь основных терминов? Чтобы проверить свои ответы, загляните в глоссарий на прилагаемом компакт-диске.

- Параметры сортировки ~ collation sequence.
- Компонент Database Mail ~ Database Mail.
- Интеллектуальный анализ данных ~ data mining.
- Почтовый профиль ~ mail profile.

Упражнения

Сейчас вы примените на практике знания и навыки, полученные при изучении этой главы. Ответы на вопросы см. в разделе «Ответы» в конце книги.

Упражнение 1. Проектирование инфраструктуры SQL Server

Компания Wide World Importers собирается внедрять новый набор приложений для управления несколькими направлениями деятельности. Нужно обеспечить возможность хранить большой объем данных в корпоративном вычислительном центре и доступ к ним из любой точки земного шара.

Нескольким менеджерам требуется доступ к оперативным отчетам о текущей нагрузке своих сотрудников, а также о новых и незавершенных заказах клиентов. Еще им необходим доступ к большим объемам архивных данных для определения тенденций, оптимизации штата и уровня складских запасов.

В отделе продаж работает большое количество менеджеров по продажам, которые постоянно звонят клиентам, и им нужен доступ к данным по клиентам, которых обслуживает тот или иной менеджер, а также к информации о потенциальных клиентах. Доступ к информации менеджерам по продажам нужен даже в отсутствие подключения к Интернету или корпоративной сети. Периодически менеджеры по продажам подключаются к корпоративной сети и синхронизируют свои данные с корпоративной базой данных.

Средствами Microsoft Visual Studio .NET был разработан ряд приложений для работы в среде Microsoft Windows, а весь доступ к данным осуществляется через хранимые процедуры. Набор приложений одинаков как у пользователей, подключенных напрямую к корпоративному серверу баз данных, так и у торговых представителей, подключающихся к своим локальным серверам баз данных.

Ответьте на следующие вопросы:

1. Какой выпуск SQL Server 2008 надо установить на ноутбуках менеджеров по продажам, чтобы минимизировать затраты на приобретение лицензий?

2. Какой выпуск SQL Server 2008 надо установить в корпоративном вычислительном центре?
3. Какие службы SQL Server надо установить, чтобы удовлетворить требования менеджеров?
4. Какую версию Windows нужно установить на корпоративном сервере базы данных?

Рекомендуемые упражнения

Чтобы подготовиться к экзамену по темам этой главы, выполните следующие задания.

Установка SQL Server

- **Задание 1. Установка экземпляров SQL Server.** Установите еще два экземпляра компонента Database Engine сервера SQL Server 2008.

Управление службами SQL Server

- **Задание 1. Управление экземпляром SQL Server.** Измените учетную запись службы одного из установленных экземпляров.

Пробный экзамен

На прилагаемом компакт-диске содержится пробный экзамен, обеспечивающий широкие возможности самоподготовки. Например, можно проверить, как усвоены темы этой главы либо все темы экзамена 70-432. Можно симитировать настоящий экзамен или выбрать режим обучения, когда после каждого вопроса приводятся правильные ответы с пояснениями.

ПРИМЕЧАНИЕ Пробный экзамен

Подробнее о пробном экзамене см. «Введение».

Настройка и обслуживание базы данных

Занятие 1. Настройка файлов и файловых групп	33
Занятие 2. Настройка параметров БД	40
Занятие 3. Обеспечение целостности БД	47

От правильности выбора конфигурации базы данных зависит ее производительность, масштабируемость и обслуживание. В этой главе рассказывается, как проектировать структуры хранилищ баз данных на основе файлов и файловых групп. Вы научитесь настраивать параметры базы данных и модели восстановления, а также проверять и управлять целостностью базы данных.

Темы экзамена

- Создание резервной копии базы данных.
- Настройка и управление базами данных.
- Обеспечение целостности базы данных.
- Управление сортировкой.

Прежде всего

Для изучения материалов этой главы необходимо иметь:

- установленный экземпляр Microsoft SQL Server 2008;
- подключенную базу данных AdventureWorks.

Практический совет

Майк Хотек

На протяжении моей карьеры, связанной с SQL Server, я работал с миллионами баз данных для тысяч заказчиков. За это время я выработал много основанных на практике правил, которые получили неоднозначную оценку со стороны «пуристов». Все мои рекомендации и подходы к разработке архитектуры и управлению серверами SQL Server выработаны с учетом реальности, но при этом основаны на отличном знании SQL Serv-

ег, оборудования, сетевых технологий и многих других элементов, плохо вписывающихся в теорию совершенного мира.

Разработка дисковых структур для баз данных — одна из областей, где я отхожу от теоретических процессов и расчетов, которые представлены в литературе. Существуют целые руководящие документы и даже разделы в тренингах, посвященные расчету скорости обмена данными с диском и соотношению скоростей при произвольных и последовательных операциях записи, но дело в том, что перед созданием системы у меня никогда не было для этого ни времени, ни возможности.

Конечно, замечательно, что есть формулы вычисления скорости обмена данными для конкретной конфигурации дисков. Более того, статистическими методами можно дополнительно уточнить расчеты, выполненные на основе произвольных/последовательных операций ввода-вывода в системе. Однако бесполезно тратить время на это, пока не известны объемы чтения и записи базы данных, размещаемой на дисковой подсистеме. Кроме того, очень трудно проектировать дисковое хранилище базы данных в соответствии со всеми теориями, пока не куплена новая система хранения данных, предназначенная для конкретного приложения.

Главное в достижении оптимальной производительности — разнесение журналов транзакций и файлов данных так, чтобы изолировать дисковый ввод-вывод. Журналы транзакций — ключ к достижению высокой производительности записи, поскольку максимальная скорость транзакций определяется возможностями записи в файл журнала транзакций. Разобравшись с журналом транзакций, следует добавить достаточное количество файлов и файловых групп, чтобы обеспечить приемлемую пропускную способность диска для операций чтения и записи. Однако наиболее важный элемент производительности — это разработка эффективного кода приложения, который работает только с минимально необходимым объемом данных для выполнения той или иной бизнес-задачи.

Занятие 1. Настройка файлов и файловых групп

Информация базы данных хранится на диске в виде одного или нескольких файлов данных. Перед записью в файл(ы) данных, каждая транзакция записывается в файл журнала транзакций. На этом занятии рассказывается, как управлять журналом транзакций, проектировать файлы данных базы данных и объединять их в группы, чтобы из физического хранилища сформировать базу данных. Кроме того вы узнаете, как настраивать базу данных *tempdb* для получения оптимальной производительности.

Изучив материал этого занятия, вы сможете:

- ✓ создавать файловые группы;
- ✓ добавлять файлы в группы;
- ✓ работать с данными *FILESTREAM*;
- ✓ настраивать журнал транзакций.

Продолжительность занятия — 20 минут.

Файлы и файловые группы

Доступ к данным в памяти выполняется исключительно быстро, однако после выключения компьютера вся информация теряется. Чтобы защитить данные, их следует сохранить на диске. Информация баз данных хранится в одном или нескольких файлах.

В SQL Server используются два типа файлов — файлы данных и файлы журнала транзакций. Файлы данных служат для долговременного хранения всей информации в базе данных, а файлы журнала транзакций — для транзакций, выполненных в базе данных.

Вместо отображения хранилища объектов напрямую в файл данных, SQL Server для большей гибкости предоставляет уровень абстракции, называемый *файловой группой*. Файловая группа — это логическая структура, определенная в базе данных и служащая для сопоставления базы данных и ее объектов на файлы данных на диске. Файловые группы могут содержать несколько файлов данных.

Все содержащие данные объекты, такие как таблицы, индексы и индексируемые представления, имеют предложение `ON`, с помощью которого при создании объекта можно задавать файловую группу, где SQL Server будет их хранить. При записи информации в объекты файловые группы используются сервером SQL Server для определения, в каком файле хранить данные.

При добавлении файла в базу данных указывается его начальный размер. Кроме этого, можно определить максимальный размер файла и его автоматическое увеличение в случае заполнения данными. В последнем случае можно задать, как увеличивать файл — на заданную величину (в процентах) относительно его текущего размера или на фиксированную величину.

Если файловая группа состоит из нескольких файлов, невозможно узнать, в котором из них хранится та или иная строка данных. При записи в файлы SQL Server применяет алгоритм пропорционального заполнения. Алгоритм пропорционального заполнения обеспечивает одновременное достижение заданного максимального размера всеми файлами в группе. Например, если есть два файла данных размером соответственно 10 Гб и 1 Гб, то SQL Server будет писать на каждые 10 строк в файл размером 10 Гб одну строку в файл размером 1 Гб.

Алгоритм пропорционального заполнения позволяет выполнять операцию изменения размера на уровне файловой группы, иначе говоря, все файлы в группе увеличиваются одновременно.

Расширения файлов

В SQL Server используются три расширения файлов: `mdf`, `ndf` и `ldf`. К сожалению, многие придают им слишком большое значение, хотя те и не несут особого смысла. Так же, как документы Microsoft Office Word имеют расширение `doc` или `docx`, а файлы Microsoft Office Excel — расширение `xls` или `xlsx`, это всего лишь соглашение о наименованиях. Можно запросто создать документ Word с расширением `.bob`, или вообще без расширения, который все равно будет документом Word.

Как правило, в базе данных сначала создается файл данных с расширением `mdf`, он обычно связывается с первичной файловой группой и счита-

ется основным файлом данных, содержащим все необходимые базе данных системные объекты. Расширение `ndf` используется для всех остальных файлов данных базы данных независимо от файловой группы, с которой он связан. Расширение `ldf` служит для хранения файлов транзакций.

Расширения файлов SQL Server есть не что иное, как соглашения о наименованиях. SQL Server не заботится о расширениях файлов и даже об их наличии. Если вам очень надо, можете использовать расширение `ldf` для основного файла данных, а `mdf` — для файла журнала транзакций. Хотя такой подход к расширениям файлов и не повлияет на SQL Server, однако он может ввести в заблуждение других администраторов баз данных организации. Чтобы этого не происходило, рекомендуем использовать принятые в SQL Server соглашения о наименованиях для расширений `mdf`, `ndf` и `ldf`, но не забывать, что это всего лишь соглашение, которое не оказывает никакого влияния на сам SQL Server.

Все операции с данными в SQL Server выполняются в буферах, расположенных в оперативной памяти. При добавлении в базу данных новой информации, она сначала помещается в буфер памяти, после чего регистрируется в журнале транзакций и, наконец, фоновым процессом, выполняющим *запись контрольных точек* (check pointing), сохраняется в файле данных. При изменении или удалении существующей строки, SQL Server сначала считывает данные с диска, если их еще нет в памяти, а затем вносит изменения. Точно так же при чтении информации, которая пока не была загружена в буфер памяти, SQL Server должен сначала прочитать ее с диска из файлов данных.

Если позаботиться, чтобы на компьютере, содержащем базу данных, было достаточно оперативной памяти для размещения всей информации БД, то для обеспечения высокой производительности SQL Server может загружать все данные с диска в буфера памяти при загрузке. Однако, как правило, размеры баз данных много больше оперативной памяти любого компьютера, поэтому SQL Server получает данные с диска только по мере необходимости. Если в памяти недостаточно места для новых данных, для их размещения SQL Server очищает пулы буферов, которые дольше всего не использовались.

Поскольку доступ к диску значительно медленнее доступа к оперативной памяти, структура файлов данных базы данных оказывает большое влияние на производительность.

Первый уровень этой структуры — дисковая подсистема. Чем больше дисков в томе, тем выше пропускная способность операций чтения и записи. Однако скорость дискового ввода-вывода ограничена пропускной способностью RAID-контроллера, адаптера главной шины (HBA) и шины диска. Поэтому добавлением дисков невозможно устранить бутылочное горлышко, которым является дисковая система ввода-вывода. Хотя написана масса объемных технических документов о произвольной и последовательной записи, скорости передачи, скорости ротации, вычислении общей скорости чтения и записи диска и т. п., процесс проектирования дисковой подсистемы сводится к добавлению достаточного количества дисков, выбору подходящих контроллеров, нужного размера дисковых кешей, чтобы обеспечить требуемую для базы данных производительность операций чтения-записи.

Если бы дело было только в количестве дисков, дисковый ввод-вывод гораздо реже был бы узким местом в системах. Но есть второй уровень структуры файлов данных — определение необходимого количества файлов данных и их расположение.

Для каждого файла базы данных SQL Server создает отдельный поток. По мере увеличения количества файлов в базе данных, SQL Server создает больше потоков для чтения и записи данных. Однако нельзя просто создать базу данных с тысячами файлов, чтобы увеличить количество потоков. Дело в том что каждый поток потребляет память, отнимая ее у кешируемых данных. Даже если бы удалось писать одновременно во все потоки, это заблокировало бы физические диски, хранящие файлы данных. Более того, управление тысячами файлов данных — трудоемкая задача, а если к тому же большую часть этих файлов открывать одновременно, то это создаст нагрузку, способную заблокировать поток информации в базе данных.

Противоречивые требования и тот простой факт, что в реальном мире мало у кого из администраторов БД есть время для выполнения сложных расчетов скорости передачи байт или даже для проектирования дискового уровня на основе точных знаний требуемой пропускной способности данных, приводят к необходимости проектировать уровень данных итеративно.

Проектирование уровня данных базы данных начинается с ее создания. Вновь созданная база данных должна содержать три файла и две файловые группы. Необходим файл с расширением *mdf* в первичной файловой группе (с именем PRIMARY), файл с расширением *ndf* в пользовательской файловой группе (с произвольным названием) и журнал транзакций с расширением *ldf*.

ПРИМЕЧАНИЕ Расширения файлов

Как было сказано во вставке выше в этой главе, расширения файлов не больше, чем просто соглашение о наименованиях. Они не предоставляют никаких особых возможностей.

Файловые группы, кроме того, что являются логическим определением одного или нескольких файлов, представляющих границы хранения объектов, имеют свойство DEFAULT (группа по умолчанию). Свойство DEFAULT определяет файловую группу, где SQL Server размещает объекты, для которых при создании не было задано предложение ON.

При создании базы данных, ее первичная файловая группа помечается как файловая группа по умолчанию. После создания базы данных рекомендуется группой по умолчанию назначить другую файловую группу. Смена файловой группы по умолчанию гарантирует, что ни один создаваемый пользовательский объект не будет случайно помещен в первичную файловую группу, где останутся только системные объекты базы данных. Следующая команда меняет файловую группу по умолчанию:

```
ALTER DATABASE <имя_базы_данных> MODIFY FILEGROUP <имя_файловой_группы> DEFAULT
```

Основная причина отказа от размещения пользовательских объектов в первичной файловой группе — обеспечение максимально возможной изоляции операций ввода-вывода. Данные в системных объектах изменяются не так часто, как в пользовательских. Уменьшая количество операций записи в первичный файл данных, вы снижаете вероятность повреждений из-за аппаратных сбоев.

Кроме того, поскольку состояние первичной файловой группы определяет состояние базы данных, то при уменьшении изменений в первичной файловой группе, увеличивается доступность базы данных.

Сразу после создания базы данных нужно по необходимости создать дополнительные файловые группы, чтобы разнести хранение объектов внутри базы данных. Кроме этого следует создать дополнительные файлы в файловых группах для увеличения возможностей дискового ввода-вывода объектов этих групп и, тем самым, устранения узкого места, которым является дисковая подсистема.

Журналы транзакций

Подтверждая фиксацию транзакции, SQL Server должен гарантировать, что все изменения записаны в долговременное хранилище. Хотя операции записи и выполняются через буферы памяти, сохранность данных обеспечивается тем, что до завершения фиксации все изменения записываются в журнал транзакций. Кроме того, запись в журнал транзакций должна осуществляться напрямую на диск.

Поскольку все изменения в базе данных записываются непосредственно на диск, для получения максимальной производительности записи транзакций важно выбрать правильную архитектуру дискового хранилища журнала транзакций.

SQL Server всегда выполняет запись в журнал транзакций последовательно, и никогда не считывает его за исключением времени восстановления. Поскольку SQL Server выполняет произвольное чтение и запись в файлы данных базы данных, вынос журналов транзакций в отдельную группу дисков избавляет головки дисков от хаотического перемещения и обеспечивают их предпочтительное линейное движение.

ПРИМЕЧАНИЕ На экзамене

Максимальное число транзакций, выполняемых в единицу времени, для любой базы данных определяется тем, сколько информации в секунду SQL Server может записать в журнал транзакций.

Тесты производительности

Отчеты с тестами производительности — лучший источник информации для оптимизации производительности при проектировании дискового хранилища. Пресса и многие организации уделяют особое внимание различным тестам производительности. Однако тщательные исследования показали, что сам SQL Server не оказывает настолько большого влияния на общие показатели, как это принято считать. Отвечающий за обработку транзакций модуль в SQL Server настолько эффективен, что очень слабо влияет на производительность обработки транзакций, а истинным ключом к увеличению скорости транзакций является дисковое хранилище. При одинаковой конфигурации дисков накопители со скоростью вращения шпинделя 7200 оборотов в минуту обеспечивают примерно вдвое меньшую скорость транзакций SQL Server, чем со скоростью вращения 15000 оборотов в минуту. Выделение 100 дисков для журнала транзакций обыч-

но удваивает скорость выполнения транзакций по сравнению с конфигурацией из 50 дисков. Кроме того один из трюков, применяемых при измерении производительности, заключается в таком разбиении диска, чтобы SQL Server записывал все данные во внешнюю половину дисковой пластины. Согласно законам физики линейная скорость возрастает по мере приближения читающей/записывающей головки к краю круглого диска, тем самым за единицу времени под головкой диска проходит больший сегмент дисковой пластины.

Данные FILESTREAM

Объем данных в организациях резко вырос, причем основной рост пришелся на неструктурированную информацию. Для решения проблемы хранения и управления большими объемами неструктурированных баз данных, а также объединения их со структурированной информацией в базах данных в SQL Server представлен новый тип данных — *FILESTREAM*.

FILESTREAM позволяет ассоциировать файлы с базой данных. Файлы хранятся в папках под управлением операционной системы, но связаны непосредственно с базой данных, где могут участвовать в резервном копировании, восстановлении, объединяться со структурированными данными, а также проходить полнотекстовую индексацию.

Подробнее о *FILESTREAM* рассказывается в главах 3 и 5, но сейчас вам достаточно знать, что для хранения данных *FILESTREAM* в базе данных, необходимо указать, где они будут храниться. Расположение данных *FILESTREAM* в базе данных определяется выбором для их хранения одной из файловых групп при помощи свойства *CONTAINSFILESTREAM*. Свойство *FILENAME* файловой группы *FILESTREAM* задает путь к папке. Начальный участок пути к папке должен существовать за исключением последней папки — она создается автоматически. После создания папки *FILESTREAM*, в ней создается файл *filestream.hdr*, являющийся системным файлом и используемый для управления файлами, которые в дальнейшем будут записываться в эту папку.

База данных tempdb

Так как в данной версии база данных *tempdb* стала использоваться гораздо интенсивнее, чем в предыдущих, особое внимание следует уделить проектированию ее хранилища.

Теперь кроме хранения временных объектов SQL Server использует ее для размещения рабочих таблиц, используемых в операциях группировок и сортировок, для поддержки в них курсоров, а также для хранилища версий с поддержкой уровня изоляции моментального снимка, и для большого количества табличных переменных. Кроме этого можно настроить построение индекса так, чтобы задействовать пространство *tempdb*.

Из-за высокой вероятности большой активности операций записи следует перенести *tempdb* на группу дисков, размещенных отдельно от файлов самой базы данных и любых резервных копий. Иногда для ускорения дисковых операций ввода-вывода имеет смысл добавить в *tempdb* дополнительные файлы.

ПРИМЕЧАНИЕ Количество файлов в tempdb

Общепринятая практика для *tempdb* — создание по одному файлу на процессор. Причем под процессором подразумевается то, что сам SQL Server считает процессором, а не физический процессор, у которого может быть несколько ядер и который поддерживает технологию многопоточности (hyperthreading).

Контрольные вопросы

1. Какие типы файлов создаются для баз данных, и какие расширения принято для них использовать?
2. Для чего служит журнал транзакций?

Ответы на контрольные вопросы

1. Для баз данных создаются файлы данных и журнала транзакций. Файлы данных обычно имеют расширение .mdf или .ndf, а файлы транзакций — .ldf.
2. В журнал транзакций записываются все изменения в базе данных для сохранения всех транзакций на диске.

Практикум. Создание баз данных

Вы создадите базы данных с несколькими файлами и с поддержкой данных *FILESTREAM*.

1. Выполните следующий код, чтобы создать базу данных:

```
CREATE DATABASE TK432 ON PRIMARY
( NAME = N'TK432_Data', FILENAME = N'c:\test\TK432.mdf' ,
  SIZE = 8MB , MAXSIZE = UNLIMITED, FILEGROWTH = 16MB ),
FILEGROUP FG1
( NAME = N'TK432_Data2', FILENAME = N'c:\test\TK432.ndf' ,
  SIZE = 8MB , MAXSIZE = UNLIMITED, FILEGROWTH = 16MB ),
FILEGROUP Documents CONTAINS FILESTREAM DEFAULT
( NAME = N'Documents', FILENAME = N'c:\test\TK432Documents' )
LOG ON
( NAME = N'TK432_Log', FILENAME = N'c:\test\TK432.ldf' ,
  SIZE = 8MB , MAXSIZE = 2048GB , FILEGROWTH = 16MB )
GO
```

2. Смените файловую группу по умолчанию следующей командой:

```
ALTER DATABASE TK432
MODIFY FILEGROUP FG1
DEFAULT
GO
```

Резюме занятия

- В качестве физического хранилища базы данных можно использовать один или несколько файлов данных и файлов журнала транзакций.

- В рамках базы данных файлы данных объединяются в файловые группы.
- Файловые группы являются логическими контейнерами хранения объектов базы данных.
- Файлы могут храниться при помощи новой функциональности — *FILE-STREAM*.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Имеется эталонная БД *OrderHistory*, данные в которой не должны изменяться. Как с наименьшими усилиями добиться, чтобы пользователи могли только читать информацию из БД.
 - A. Добавить всех пользователей БД в роль *db_datareader*.
 - B. Создать представления для всех таблиц и предоставить пользователям БД разрешение SELECT только на эти представления.
 - C. Установить у БД атрибут READ_ONLY.
 - D. Всем пользователям БД предоставить разрешение SELECT и отменить разрешения INSERT, UPDATE и DELETE.

Занятие 2. Настройка параметров БД

Информация БД хранится на диске в одном или нескольких файлах данных. Перед записью в файл(ы) данных каждая транзакция фиксируется в файле журнала транзакций. Здесь рассказывается, как проектировать файлы БД, объединять их в файловые группы, чтобы связать физическое хранилище с базой данных, и управлять журналом транзакций.

Изучив материал этого занятия, вы сможете:

- ✓ определять модель восстановления БД;
- ✓ настраивать параметры БД;
- ✓ управлять схемами сортировки;
- ✓ контролировать и поддерживать целостность БД.

Продолжительность занятия — 20 минут.

Параметры настройки БД

Существует много параметров настройки БД, определяющих ее поведение. Эти параметры подразделяются на несколько категорий, в том числе:

- восстановление;
- автоматические операции;
- отслеживание изменений;
- доступ;
- параметризация.

Параметры восстановления

Параметры восстановления определяют поведение журнала транзакций и способ обработки поврежденных страниц.

Модели восстановления

У каждой БД SQL Server есть свойство, называемое *моделью восстановления* (recovery model). Она определяет типы резервных копий, которые можно создавать в этой БД. В SQL Server возможны следующие модели восстановления:

- полная;
- с неполным протоколированием;
- простая.

Модель полного восстановления

Если в базе данных выбрана модель полного восстановления, все изменения, выполненные операторами языков DML (Data Manipulation Language) и DDL (Data Definition Language), записываются в журнал транзакций. Регистрация всех изменений в журнале транзакций позволяет восстановить БД на любой требуемый момент времени, поэтому при восстановлении после сбоя можно свести потерю данных к минимуму или вообще избежать ее. Изменения хранятся в журнале транзакций неограниченное время и удаляются только при создании его резервной копии.

ПРАКТИЧЕСКИЙ СОВЕТ Модели восстановления

Во всех рабочих базах данных, поддерживающих транзакции, должна быть выбрана модель полного восстановления — это обеспечит максимальные возможности по восстановлению.

Модель восстановления с неполным протоколированием

Иногда приходится обрабатывать большие объемы данных, однако дополнительные операции по их записи в журнал транзакций могут сильно снижать производительность. Модель восстановления с неполным протоколированием позволяет выполнять такие задачи с минимальным журналированием. В этом случае SQL Server вместо записи изменений каждой строки регистрирует только экстенды, за счет чего сокращаются дополнительная нагрузка и повышается производительность. В модели с неполным протоколированием минимальное журналирование используется при следующих операциях :

- BCP
- BULK INSERT
- SELECT . . . INTO
- CREATE INDEX
- ALTER INDEX . . . REBUILD

В этой модели в журнал транзакций записываются не все изменения, поэтому в ней невозможно восстановить БД на момент выполнения транзакций с минимальным журналированием.

Простая модель восстановления

Третья модель восстановления — простая. В базе данных, где используется эта модель все операции записываются точно так же, как в модели полного восстановления, но в момент записи контрольной точки в базу данных, зафиксированная часть журнала транзакций отбрасывается. В простой модели восстановления нельзя восстановить базу данных на любой момент времени, так как нет возможности создать резервную копию журнала транзакций БД.

Поскольку модель восстановления является свойством базы данных, задать ее можно командой *ALTER DATABASE*:

```
ALTER DATABASE <имя_базы_данных>  
SET RECOVERY {FULL | BULK_LOGGED | SIMPLE}
```

Типы резервных копий, возможные в каждой модели восстановления, перечислены в табл. 2-1.

Табл. 2-1. Типы резервных копий, поддерживаемые разными моделями восстановления

Модель восстановления	Тип резервной копии		
	Полная	Разностная	Журнал транзакций
	Полная	Да	Да
	С неполным протоколированием	Да	Да, без транзакций с минимальным журналированием
Простая	Да	Да	Нет

ПРИМЕЧАНИЕ На экзамене

Следует знать, какие типы резервных копий доступны в каждой модели восстановления.

Поврежденные страницы

Во время операций записи возможно повреждение страниц данных в случае отключения питания или при сбоях дисковой подсистемы. Если операцию записи не удастся корректно завершить, в базе данных появляется неполная страница, которую невозможно прочитать. Поскольку повреждается страница, расположенная на диске, узнать об этом можно, только когда SQL Server попытается прочитать эту страницу с диска.

В конфигурации по умолчанию SQL Server не выполняет проверку целостности страниц, из-за чего база данных может перейти в автономный режим работы, «наткнувшись» на поврежденную страницу. Чтобы обнаруживать и регистрировать в журнале поврежденные страницы, нужно присвоить параметру *PAGE_VERIFY* значение *CHECKSUM*. Тогда при записи страницы на диск будет рассчитываться и сохраняться в заголовке ее контрольная сумма. При чтении страницы с диска будет вычисляться ее контрольная сумма и сравниваться с суммой из заголовка. В случае расхождения вызывающему приложению возвращается ошибка 824, кроме того она регистрируется в журнале ошибок SQL Server и журнале событий «Приложение» (Application). Также идентификатор (ID) поврежденной страницы записывается в таблицу *suspect_pages* базы данных *msdb*.

В SQL Server 2005 единственным способом исправить поврежденную страницу было выполнение операции восстановления страницы, о чем говорится в главе 9. В дополнение к этому в SQL Server 2008 можно заменить страницу ее копией с зеркала, если база данных участвует в сеансе зеркального отображения. Если монитор зеркального отображения БД автоматически исправляет поврежденную страницу, запись об этом добавляется в журнал и доступна для просмотра в представлении `sys.dm_db_mirroring_auto_page_repair`.

Параметры с приставкой AUTO

Есть пять параметров базы данных, управляющих автоматическим выполнением определенных действий:

- `AUTO_CLOSE`
- `AUTO_SHRINK`
- `AUTO_CREATE_STATISTICS`
- `AUTO_UPDATE_STATISTICS`
- `AUTO_UPDATE_STATISTICS_ASYNC`

Каждой БД на сервере требуются различные ресурсы, наиболее важный из них — набор буферов памяти. Каждая открытая база данных занимает несколько байт в памяти, а результаты запросов БД поступают в кеш данных и запросов. Если включен параметр `AUTO_CLOSE`, при закрытии БД SQL Server освобождает все занимаемые ею ресурсы. При создании нового соединения с базой данных SQL Server запускает ее и выделяет для нее ресурсы.

Параметр `AUTO_CLOSE` по умолчанию отключен. Его стоит включать только в случае нехватки памяти на сервере. Кроме того, не рекомендуется включать этот параметр, если к БД часто обращаются, так как это вызовет сильное снижение производительности из-за невозможности надлежащим образом использовать кеш данных и запросов.

Можно настроить автоматическое увеличение файлов данных при возникновении потребности в дополнительном пространстве. Хотя большинство действий по увеличению размера оказывают долговременное влияние на базу данных, некоторым из них достаточно временного увеличения пространства. При включенном параметре `AUTO_SHRINK` SQL Server периодически проверяет использование места в файлах данных и журнала транзакций и, обнаружив файлы данных, объем которых задействован менее чем на 25%, автоматически уменьшает их, освобождая дисковое пространство.

Увеличение размеров файлов БД — очень дорогая операция, впрочем как и их сжатие. Когда размер файлов БД увеличивается в результате обычных операций, велика вероятность, что после сжатия, последующие действия снова увеличат их размер. Единственными операциями, временно увеличивающими размер БД, являются административные задачи по созданию или обновлению индексов, архивации данных или загрузке информации. Из-за ресурсоемкости операции увеличения файлов рекомендуется оставлять параметр `AUTO_SHRINK` выключенным, а при крайней необходимости сжимать файлы вручную.

Статистика позволяет оптимизатору запросов (Query Optimizer) строить более эффективные планы запросов. Если включен параметр `AUTO_CREATE_STATISTICS`, SQL Server автоматически собирает статистику, отсутствующую

в фазе оптимизации обработки запросов. Хотя вычисление статистики подразумевает некоторую дополнительную нагрузку, получаемый в результате выигрыш в производительности запросов оправдывает затраты на автоматический сбор статистики.

В статистике собираются данные об относительном распределении значений в одном или нескольких столбцах таблицы. Как правило, после выхода БД «на режим» статистическое распределение обычных операций практически не меняется. Однако массовые изменения данных или кардинальная перестройка бизнес-процессов могут резко внести значительные отклонения в распределение данных. Если в связи с этим не обновить статистику, оптимизатор может выбрать неэффективный план запросов.

В базах данных имеется два параметра, управляющих автоматическим обновлением устаревшей статистики SQL Server. Параметр `AUTO_UPDATE_STATISTICS` обеспечивает обновление статистики во время оптимизации запроса. Если этот параметр включен, то параметр `AUTO_UPDATE_STATISTICS_ASYNC` определяет, будет ли обновление статистики выполняться в процессе оптимизации запроса или же она будет выполняться асинхронно.

Отслеживание изменений

Одна из сложных задач в многопользовательских системах — защита изменений одного пользователя от случайной перезаписи изменениями другого. Для предотвращения перезаписи изменений разных пользователей друг другом в приложениях обычно предусматривают механизмы определения, не менялась ли строка таблицы между чтением данных и записью их обратно в базу данных. В таких отслеживающих механизмах обычно используются столбцы с данными типа *datetime* или *timestamp* или даже целые системы отслеживания версий.

Параметр `CHANGE_TRACKING` представляет функциональность, появившуюся в SQL Server 2008 — отслеживание изменений, представляющее собой облегченный механизм, который каждой строке таблицы, где включено отслеживание изменений, соотносит версию. При каждом изменении строки номер версии увеличивается на единицу. Вместо построения системы, предотвращающей перезапись изменений от разных пользователей, достаточно сделать так, чтобы приложения сравнивали версию строки, таким образом определяя, были ли изменения между ее чтением и записью.

После включения отслеживания изменений в базе данных, можно выбрать таблицы, для которых будет собираться информация для отслеживания изменений. Со временем информация для отслеживания изменений накапливается в базе данных, поэтому есть возможность определить при помощи параметра `CHANGE_RETENTION`, как долго такая информация должна храниться, а параметром `AUTO_CLEANUP` указать, должна ли она автоматически вычищаться.

Параметры управления доступом

Состояние БД можно явно переключить в `ONLINE`, `OFFLINE` или `EMERGENCY`. В базе данных в состоянии `ONLINE` разрешены все операции, БД в состоянии `OFFLINE` недоступна, а база данных в состоянии `EMERGENCY` доступна только членам роли `db_owner`, а единственная доступная команда — `SELECT`.

Контролировать возможность изменения информации в оперативной базе данных можно, переведя ее в режим `READ_ONLY` или `READ_WRITE`. В базе данных в режиме `READ_ONLY` запрещена запись. Кроме того при переключении БД в этот режим SQL Server удаляет все ее файлы журнала транзакций. При изменении режима с `READ_ONLY` на `READ_WRITE` SQL Server воссоздает файл журнала транзакций.

Доступ пользователей к базе данных управляется параметрами `SINGLE_USER`, `RESTRICTED_USER` и `MULTI_USER`. В базе данных в режиме `SINGLE_USER` только одному пользователю разрешен доступ к базе данных. В режиме `RESTRICTED_USER` доступ разрешен только членам ролей `db_owner`, `dbcreator` и `sysadmin`.

Если в момент переключения БД в режим `SINGLE_USER` с ней работает несколько пользователей или если при переключении в режим `RESTRICTED_USER` есть пользователи, не входящие в разрешенные роли, команда `ALTER DATABASE` блокируется, пока все неразрешенные пользователи не отключатся. Можно не ждать, пока пользователи завершат свои операции и отключатся от БД, и разорвать подключения принудительно, задав параметр `ROLLBACK`. Значение `IMMEDIATE` параметра `ROLLBACK` принудительно откатывает все открытые транзакции и закрывает соединения со всеми пользователями, не входящими в разрешенные роли. Кроме этого, можно позволить пользователям завершить транзакции и отключиться от БД, задав параметр `ROLLBACK AFTER <количество_секунд>`; в этом случае сервер ожидает указанное количество секунд, и только затем откатывает транзакции и отключает пользователей.

Обычный рабочий режим большинства баз данных — `ONLINE`, `READ_WRITE` и `MULTI_USER`.

Параметризация

Одна из самых обсуждаемых тем в разработке приложений — вопрос о том, стоит ли использовать параметризацию в запросах к БД. В параметризованных запросах значения передаются как переменные. У каждой из точек зрения есть свои приверженцы. К сожалению, разработка приложений значительно облегчается использованием параметризованных запросов БД.

SQL Server кеширует планы всех выполненных запросов. В кеше запросов находится каждый выполненный запрос с момента запуска сервера, если только нет давления на кеш запросов, что вынуждает удалять из него планы. Приступая к обработке запроса, SQL Server сначала его анализирует и компилирует. Затем используя алгоритм сравнения строк, сервер ищет запрос в кеше запросов, и, обнаружив его, загружает уже готовый план и выполняет запрос.

Вероятность нахождения параметризованного запроса гораздо выше, так как его строка не меняется даже при изменении его параметров. Поэтому в параметризованных запросах могут гораздо чаще использоваться хранящиеся в кеше планы запросов, что избавляет от необходимости тратить время на построение плана запроса.

Поскольку не во всех приложениях используются параметризованные запросы базы данных, можно заставить SQL Server параметризовать каждый запрос к определенной базе данных, задав параметру `PARAMETRIZATION` значение `FORCED`.

По умолчанию в базе данных принудительная параметризация отключена. Польза от повторного использования планов запросов есть только тогда, когда они описывают наиболее эффективный способ обработки данных. В таблицах с неравномерным распределением данных для разных значений могут создаваться совершенно разные планы. Кроме того, эффект от параметризации заметен в приложениях, только когда длительность большинства запросов БД исключительно мала.

Итак, принудительная параметризация может обеспечить увеличение производительности, если большинство запросов БД выполняются быстро и создаваемые планы запросов не меняются с изменением передаваемых параметров.

Схемы сортировки

SQL Server способен хранить символьную информацию на всех возможных письменных языках. Однако в разных языках разные правила сортировки и сравнения данных. Выбор сортировки (Collation Sequences) в SQL Server позволяет определить правила сравнения, сортировки, учета регистра или диакритических знаков.

Во время установки SQL Server указывается схема сортировки по умолчанию для всех баз данных, таблиц и столбцов. На каждом уровне можно переопределить используемую по умолчанию схему сортировки. На уровне БД схему сортировки можно переопределить, указав предложение COLLATE в команде *CREATE DATABASE* или *ALTER DATABASE*.

Контрольные вопросы

1. Как одновременно разрешить доступ к базе данных только для членов роли db_owner и закрыть все активные транзакции и подключения?
2. Какие резервные копии могут создаваться в каждой из моделей восстановления?

Ответы на контрольные вопросы

1. Следует выполнить следующую команду: *ALTER DATABASE <имя_базы_данных> SET RESTRICTED_USER WITH ROLLBACK IMMEDIATE*.
2. В простой модели восстановления можно создать полную и разностную резервные копии, а также копии файла/файловой группы. В модели восстановления с неполным протоколированием возможны все типы резервных копий, однако при этом база данных не может быть восстановлена на момент выполнения транзакций с минимальным журналированием. В полной модели восстановления доступны все типы резервных копий.

Практикум. Изменение модели восстановления БД

Вы настроите полную модель восстановления в базе данных *AdventureWorks*, чтобы гарантировать, что в случае сбоя ее можно было восстановить в состоянии на любой момент времени.

1. Выполните следующий код:

```
ALTER DATABASE AdventureWorks  
SET RECOVERY FULL  
GO
```
2. Чтобы просмотреть модель восстановления и убедиться, что установлена полная модель, щелкните правой кнопкой базу данных *AdventureWorks*, выберите **Свойства** (Properties), а затем — вкладку **Параметры** (Options).

Резюме занятия

- Для БД можно выбрать одну из следующих моделей восстановления: полную, с неполным протоколированием или простую.
- Создавать резервную копию журнала транзакций можно в базах данных с полной моделью восстановления и с моделью с неполным протоколированием.
- Параметр AUTO_SHRINK сжимает базу данных, если в ее файлах свободно более чем 25% дискового пространства.
- Поврежденные страницы можно отслеживать и регистрировать в журнале, включив параметр PAGE_VERIFY CHECKSUM.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Вы администратор баз данных в компании Blue Yonder Airlines и отвечаете в основном за БД *Reservation*, расположенную на SQL Server 2008. Клиенты резервируют авиабилеты через веб-узел компании, а также нескольких партнеров. Раз в час поступает несколько файлов от партнеров, которые загружаются в базу данных *Reservation* утилитой массового копирования BCP. Требуется обеспечить возможность восстановления БД на любой момент времени, гарантировав при этом максимальную производительность программ импорта данных. Как настроить базу данных, чтобы удовлетворить этим бизнес-требованиям?
 - A. Включить параметр AUTO_SHRINK.
 - B. Включить параметр PARAMETERIZATION FORCED базы данных.
 - C. Настроить в базе данных модель восстановления с неполным протоколированием.
 - D. Настроить в базе данных полную модель восстановления.

Занятие 3. Обеспечение целостности БД

В идеальном мире все, что сохраняется на дисковое хранилище, всегда правильно записывается, считывается и никогда не возникает никаких проблем. К сожалению базы данных SQL Server существуют в несовершенном мире, где что-нибудь может пойти не так, как надо. Хотя это и случается крайне редко,

но данные в базе данных могут повреждаться, если при записи страницы произойдет сбой дисковой подсистемы. Страницы данных имеют размер 8 Кб, но при записи SQL Server делит страницу на 16 блоков по 512 байт. Если произошел сбой дисковой подсистемы, когда SQL Server начал запись блоков страницы, то часть страницы, которая к этому времени была успешно записана и вызывает в дальнейшем ошибку, называется *разорванной страницей* (torn page). На этом занятии рассказывается, как выявлять и исправлять повреждения в базе данных.

Изучив материал этого занятия, вы сможете:

- ✓ проверять целостность БД;
- ✓ использовать утилиту DMV для диагностики повреждений.

Продолжительность занятия — 20 минут.

Проверка целостности БД

Как говорилось на занятии 2, у БД есть параметр проверки страниц, называемый `PAGE_VERIFY` и принимающий значение `TORN_PAGE_DETECTION` или `CHECKSUM`. Оставленный для совместимости с предыдущими версиями параметр `PAGE_VERIFY TORN_PAGE_DETECTION` использовать не рекомендуется. При включенном параметре `PAGE_VERIFY CHECKSUM` перед записью страницы SQL Server вычисляет ее контрольную сумму. При каждом чтении страницы контрольная сумма пересчитывается и сверяется с сохраненной контрольной суммой. Несовпадение сумм свидетельствует о повреждении страницы.

При обнаружении поврежденной страницы возникает ошибка, а команда, обращавшаяся к поврежденной странице, прерывается и в таблицу `suspect_pages` БД `msdb` заносится запись об этом.

ПРАКТИЧЕСКИЙ СОВЕТ Проверка страниц

Во всех рабочих базах данных должен быть включен параметр `PAGE_VERIFY CHECKSUM`.

Хотя проверка позволяет обнаруживать поврежденные страницы, она выполняется только при их чтении с диска. Обычно данные считываются с диска только при обращении к ним пользователей или приложений. Но лучше предотвратить получение пользователями сообщений об ошибках, заранее выявляя повреждения и исправляя их с использованием резервных копий.

Команда `DBCC CHECKDB` заставляет SQL Server прочитать с диска все страницы и проверить их целостность. Общий синтаксис этой команды таков:

```
DBCC CHECKDB ([ '<имя_базы_данных>' | database_id | 0
    [ , NOINDEX | { REPAIR_ALLOW_DATA_LOSS | REPAIR_FAST
    | REPAIR_REBUILD } ] ])
    [ WITH { [ ALL_ERRORMSG ] [ , [ NO_INFOMSGS ] ] [ , [ TABLOCK ] ]
    [ , [ ESTIMATEONLY ] ] [ , [ PHYSICAL_ONLY ] ] | [ , [ DATA_PURITY ] ] } ]
```

Выполняя команду *DBCC CHECKDB*, SQL Server делает следующее:

- проверяет выделение страниц в базе данных;
- проверяет целостность структуры всех таблиц и индексируемых представлений;
- пересчитывает контрольные суммы всех страниц данных и индексов и сравнивает их с хранящимися контрольными суммами;
- проверяет содержимое всех индексируемых представлений;
- проверяет каталог базы данных;
- в базе данных проверяет данные компонента Service Broker.

В процессе этих проверок *DBCC CHECKDB* исполняет следующие команды:

- *DBCC CHECKALLOC*, чтобы проверить выделение страниц в базе данных;
- *DBCC CHECKCATALOG*, чтобы проверить каталог БД;
- для каждой таблицы и представления в базе данных вызывается команда *DBCC CHECKTABLE*, чтобы проверить целостность их структуры.

При этом сообщается обо всех обнаруженных ошибках, чтобы можно было устранить их. Если ошибка целостности найдена в индексе, его необходимо удалить и создать заново. Для исправления ошибки целостности в таблице следует восстановить ее из самой свежей резервной копии.

ПРИМЕЧАНИЕ Зеркальное отображение базы данных

Если база данных участвует в зеркальном отображении, SQL Server сначала пытается получить копию страницы с зеркального отображения. Если нужная страница есть в зеркальном отображении и ее содержание корректно, она автоматически заменяется. Никакого дополнительного вмешательства не требуется. Замену поврежденной страницы с зеркального отображения SQL Server регистрирует в представлении *sys.dm_db_mirroring_auto_page_repair*.

Контрольные вопросы

1. Какой параметр должен быть включен во всех рабочих базах данных?
2. Какие проверки выполняет команда *DBCC CHECKDB*?

Ответы на контрольные вопросы

1. Во всех рабочих базах данных следует включить параметр *PAGE_VERIFY CHECKSUM*.
2. Команда *DBCC CHECKDB* проверяет логическую и физическую целостность всех таблиц, индексов и индексируемых представлений базы данных, а также выделение страниц, данные компонента Service Broker, каталог БД и содержимое всех индексируемых представлений.

Практикум. Проверка целостности БД

Вы проверите целостность базы данных *AdventureWorks*.

1. Выполните следующий код:


```
DBCC CHECKDB ('AdventureWorks') WITH NO_INFOMSGS, ALL_ERRORMSG  
GO
```

2. Ознакомьтесь с результатом.

Резюме занятия

- В рабочих базах данных должен быть включен параметр *PAGE_VERIFY CHECKSUM*, чтобы выявлять ошибки целостности структуры.
- Информация об обнаружении поврежденной страницы помещается в таблицу *suspect_pages* БД *msdb*. Если база данных участвует в сеансе зеркального отображения, SQL Server автоматически получает копию страницы с зеркального сервера, замещает ею страницу на основном сервере и регистрирует событие в представлении *sys.dm_db_mirroring_auto_page_repair*.
- Команда *DBCC CHECKDB* служит для проверки логической и физической целостности БД.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Какие команды выполняются в процессе работы команды *DBCC CHECKDB*? (Выберите все подходящие ответы).
 - A. *DBCC CHECKTABLE*
 - B. *DBCC CHECKIDENT*
 - C. *DBCC CHECKCATALOG*
 - D. *DBCC FREEPROCCACHE*

Закрепление материала главы

Для закрепления знаний и навыков, полученных при изучении этой главы, выполните следующее:

- изучите резюме главы;
- повторите основные термины, использованные в этой главе;
- выполните задания по темам этой главы;
- сделайте предложенные упражнения;
- пройдите пробный экзамен.

Резюме главы

- В базе данных может быть настроена одна из следующих моделей восстановления: полная, простая или с неполным протоколированием.
- Модель восстановления определяет типы резервных копий, которые могут быть созданы, а также ограничения на допустимые параметры восстановления.

- В базе данных можно настроить схему сортировки, отличающуюся от схемы сортировки, определенной на сервере.

Основные термины

Понятны ли вам значения приведенных здесь основных терминов? Чтобы проверить свои ответы, загляните в глоссарий на прилагаемом компакт-диске.

- Поврежденная страница ~ corrupt page.
- Файловая группа ~ filegroup.
- Модель восстановления ~ recovery model.

Упражнения

Сейчас вы примените на практике знания и навыки, полученные при изучении этой главы. Ответы на вопросы см. в разделе «Ответы» в конце книги.

Упражнение 1. Настройка базы данных компании Coho Vineyard

Исходные данные

Основные сведения о компании

Компания Coho Vineyard была основана в 1947 г. как частный семейный винный завод. Вина, производимые Coho Vineyard, удостоивались многих наград и пользовались популярностью, что позволило компании значительно расширить производство. Через несколько лет в рамках стратегии по расширению компании были приобретены несколько существующих винных заводов. Сейчас компания владеет 16 винными заводами — 9 в штатах Вашингтон, Орегон и Калифорния, а остальные 7 в штатах Висконсин и Мичиган. В компании работает 532 человека, из которых 162 — в центральном офисе, где размещаются критически важные для бизнеса серверы. В компании трудится 122 мобильных торговых агента, которым требуется доступ к актуальной информации об имеющихся товарах.

Планируемые изменения

До сих пор у каждого из 16 винных заводов Coho Vineyard был собственный веб-узел, размещенный на площадях завода. Руководство требует объединить сайты всех заводов, так пользователи смогли покупать продукцию всех 16 предприятий в одном интернет-магазине. Вся информация веб-узла должна храниться в базе данных центрального офиса.

После объединения всей информации в центральном офисе, для доставки информации торговым агентам, а также для получения от них заказов, будет использоваться репликация слиянием. Чтобы до завершения проекта по объединению удовлетворить требованиям торговых агентов, информация о наличии товаров на каждом из заводов отправляется в центральный офис ежедневно в конце дня. Репликация слиянием настроена так, чтобы торговые агенты получали локальные копии данных по клиентам, складским запасам и заказам.

Существующая среда

Базы данных

Сейчас каждый винный завод поддерживает собственную базу данных, где хранит всю бизнес-информацию. В конце месяца вся информация переносится в центральный офис и загружается в базы данных, перечисленные в табл. 2-2.

Табл. 2-2. Базы данных Coho Vineyard

База данных	Размер
<i>Customer</i> (клиенты)	180 Мб
<i>Accounting</i> (бухгалтерия)	500 Мб
<i>HR</i> (персонал)	100 Мб
<i>Inventory</i> (склад)	250 Мб
<i>Promotions</i> (реклама)	80 Мб

По завершении объединения баз данных хранилищем информации нового интернет-магазина служит новая база данных *Order*. Кроме того в своей ежедневной работе сотрудники будут обращаться к данным БД *Order* с применением нового разработанного в компании собственными силами веб-приложения.

База данных *HR* содержит конфиденциальную информацию и защищена TDE (Transparent Data Encryption). Кроме этого, данные в таблице *Salary* шифруются с использованием сертификата.

Серверы баз данных

Один сервер под управлением Windows Server 2003 Enterprise в центральном офисе по имени DB1 хранит все БД. На нем установлен SQL Server 2008 Enterprise.

Бизнес-требования

От вас требуется разработать решение для архивирования баз данных *Customer* и *Order*. Стратегия архивирования должна обеспечивать хранение информации о клиентах как минимум на протяжении 6 лет.

В процессе подготовки БД *Order* для архивирования вы создали секционированную таблицу *Order.Sales*, состоящую из двух секций: в первой хранится информация о продажах за текущий, а во второй — за предыдущий месяц. Заказы, размещенные еще раньше, должны перемещаться в другую секционированную таблицу — *Order.Archive*, после чего в секции 1 содержатся все архивные данные, а секция 2 остается пустой.

Новый процесс должен ежедневно, не позднее 4 часов утра загружать информацию о складских запасах со всех 16 заводов.

Четыре крупных клиента осуществляют заказы, используя XML-схему Coho Vineyards для EDI-транзакций (Electronic Data Interchange). EDI-файлы поступают в 5 часов вечера и должны обрабатываться и загружаться в БД *Customer*, *Accounting* и *Inventory*, каждая из которых содержит таблицы с данными, относящимися к размещаемым заказам. В настоящее время утилита импорта

EDI-файлов представляет собой однопоточное C++-приложение, которому требуется от 3—6 часов на обработку всех файлов. Чтобы выполнить соглашение с клиентами об условиях обслуживания (Service Level Agreement, SLA), необходимо заканчивать обработку EDI-файлов до 17:30. К моменту завершения проекта по объединению EDI-утилита должна загружать информацию в новую базу данных *Order*.

Требуется обеспечить создание резервных копий всех баз данных всех филиалов. В самом плохом случае считается приемлемым потеря данных максимум за последние 5 минут. В случае аварии БД *Customer*, *Inventory*, *Promotions* и *Order* могут быть недоступны максимум 20 минут. В случае аварийного сбоя информация БД *Customer* и *Order* старше полугода может быть недоступна до 12 часов.

Ответьте на следующие вопросы:

1. Как настроить базу данных так, чтобы добиться максимальной производительности?
2. Как надо настроить базу данных, чтобы удовлетворить требованиям по восстановлению?

Рекомендуемые упражнения

Чтобы подготовиться к экзамену по темам этой главы, выполните следующие задания.

Настройка баз данных

Выполните все задания.

- **Задание 1.** Создайте базу данных, способную хранить данные FILESTREAM.
- **Задание 2.** Измените модель восстановления и изучите ее влияние на параметры резервного копирования и восстановления.
- **Задание 3.** Переведите базу данных в состояние READ_ONLY и посмотрите, как это сказалось на файле журнала транзакций.
- **Задание 4.** Создайте несколько соединений с базой данных, установите доступ к ней RESTRICTED_USER и задайте параметр ROLLBACK IMMEDIATE. Изучите результат.

Пробный экзамен

На прилагаемом компакт-диске содержится пробный экзамен, обеспечивающий широкие возможности самоподготовки. Например, можно проверить, как усвоены темы этой главы либо все темы экзамена 70-432. Можно симитировать настоящий экзамен или выбрать режим обучения, когда после каждого вопроса приводятся правильные ответы с пояснениями.

ПРИМЕЧАНИЕ Пробный экзамен

Подробнее о пробном экзамене см. «Введение».

ГЛАВА 3

Таблицы

Занятие 1. Создание таблиц 55

Занятие 2. Реализация ограничений 69

Таблицы — основа баз данных, определяющая структуру хранящейся в ней информации. База данных без таблиц практически бесполезна для бизнес-приложений. В этой главе вы узнаете, как создавать эффективные таблицы, позволяющие добиться высокой производительности в различных условиях, одновременно обеспечивая реализацию бизнес-правил.

Темы экзамена

- Сжатие данных.

Прежде всего

Для изучения материалов этой главы необходимо иметь:

- установленный экземпляр Microsoft SQL Server 2008;
- подключенную базу данных AdventureWorks.

Практический совет

Майк Хотек

Я познакомился с SQL Server почти 20 лет назад. Конечно, тогда это был Sybase SQL Server, который лег в основу первой версии Microsoft SQL Server. С того времени я мне пришлось работать с миллионами баз данных тысяч компаний. Кроме того, я обучил работе с SQL Server свыше 50 тыс. человек. Все это время меня поражало, насколько люди усложняли все при проектировании или обучении проектированию баз данных.

Интересно, что независимо от того, что вы разрабатываете — реляционную базу данных или хранилище данных, оказывается, что суть проектирования баз данных можно выразить одной фразой: «Кладите вещи на свое место». Именно так — сотни тысяч страниц публикаций по разработке реляционных баз данных и хранилищ данных на самом деле сводятся ровно к этому утверждению и навыкам логического мышления, доступным любому человеку, перед которым поставлена задача по проектированию базы данных.

Если руководствоваться этим простым принципом при разработке всех своих баз данных, можно разрабатывать базы данных в очень сжатые сроки и, кроме того, эти базы данных будут наиболее полно соответствовать потребностям вашей компании. Разработчики баз данных не создают всю базу данных сразу, а по одной таблице, основываясь на своем понимании, какие данные нужно хранить.

Базовые правила работы с таблицами весьма просты: столбцы задают, какие наборы данных будут храниться в таблице, а каждая строка соответствует одному уникальному набору информации. Определяемые вами столбцы представляют различные фрагменты информации, с которой потребуются работать пользователям вашей базы данных, например населенный пункт, наименование товара, имя, фамилия или цена.

Допустим, вы проектируете базу данных для хранения информации о заказах, размещаемых покупателями товаров. Тогда потребуется определить три основные таблицы: для данных о покупателях, заказах и товарах. Когда покупатель размещает заказ, необходимо знать, кто это. Поэтому таблица с данными о покупателях должна содержать один или несколько столбцов для имени и фамилии, в зависимости от того, собираетесь ли хранить имя и фамилию по отдельности, и нужно ли хранить обращение, такое как «г-н» или «г-жа».

Если покупатели заказывают товары в вашей компании, скорее всего нужно будет доставлять покупателям их заказы, поэтому потребуется один или несколько столбцов для хранения адреса. Вполне допустимо поместить один или несколько столбцов для хранения адреса в таблицу данных о покупателях. Такая структура подойдет, только если вы собираетесь хранить только по одному адресу каждого покупателя. Если необходимо более гибкое обслуживание покупателей и требуется хранить несколько адресов, придется создать еще одну таблицу для хранения адресов и связать ее с таблицей покупателей. Таблица адресов покупателей нужна, так как в таблицу можно добавить сколько угодно строк, тогда как число столбцов ограничено.

Продолжив этот процесс, вы вскоре определите десятки или даже сотни таблиц для хранения данных, требуемых вашему бизнес-приложению. Понятно, что создавая таким образом свою базу данных, вы не забивали себе голову такими сложностями, как первая, вторая и третья нормальные схемы «звезда» и «снежинка». Вы создавали базу данных, просто «положив вещи на свои места». После определения структуры базы данных остается определить, данные каких типов должны храниться в столбцах и являются ли те или иные столбцы обязательными для заполнения, и вы получите базу данных, готовую к использованию приложениями.

Занятие 1. Создание таблиц

Таблицы — минимальная конструктивная единица приложений, определяющая структуру хранящихся данных. При проектировании таблиц стоит задача создать таблицы, где будут храниться данные, необходимые бизнес-приложениям,

и, в то же время, свести к минимуму объем используемого таблицами дискового пространства и памяти. На этом занятии вы узнаете, что нужно принимать во внимание при определении таблиц, чтобы корректно обрабатывать бизнес-данные и чтобы эти данные потребляли минимальное количество ресурсов.

Изучив материал этого занятия, вы сможете:

- ✓ создавать схемы;
- ✓ выбирать подходящие типы данных;
- ✓ использовать свойства столбцов для реализации бизнес-правил;
- ✓ реализовать вычисления в таблицах;
- ✓ определять параметры хранения данных, сокращая пространство, занимаемое строкой или страницей.

Продолжительность занятия — 40 минут.

Схемы

Помимо обеспечения безопасности (подробнее см. главу 11) *схема* (schema) является удобным механизмом группирования объектов в базе данных. Схема также является контейнером, владеющим всеми объектами в базе данных.

С точки зрения управления дисковым пространством, памятью и транзакциями вы управляете каждой базой данных, созданной в экземпляре, по отдельности. Если ваше приложение в настоящее время обращается к нескольким базам данных или если вы разрабатываете приложение, использующее несколько баз данных, которые не требуется хранить в разных экземплярах для увеличения объема данных, который способна хранить система, следует объединить объекты в одной базе данных и с помощью схем разбить объекты на группы.

Простейший синтаксис создания схемы:

```
CREATE SCHEMA <имя схемы> AUTHORIZATION <имя владельца>
```

ПРИМЕЧАНИЕ Создание схем

Инструкция *CREATE SCHEMA* позволяет одной инструкцией создать схему вместе с таблицами и представлениями и задать разрешения. Но программирование для SQL Server — это не упражнение по написанию максимально запутанного кода и не борьба за решение задачи минимальным числом инструкций. Ваш код придется поддерживать другим людям, и лучше добавить лишние операции, если это сделает сценарий удобнее в сопровождении. Поэтому не рекомендуется создавать таблицы и представления и назначать разрешения в инструкции *CREATE SCHEMA*. Каждая инструкция *CREATE SCHEMA* должна выполняться в отдельном пакете.

Типы данных

Хотя типы данных обычно не рассматривают как ограничения, применение их в столбцах определяет самые существенные ограничения, задаваемые в таблице. Выбирая тип данных, вы ограничиваете диапазон возможных значений столбца,

а также задаете максимальное количество места, занимаемого столбцом внутри строки.

Также, решения о выборе типов данных оказывают огромное влияние на производительность базы данных. Типы данных надо выбирать так, чтобы они обеспечивали хранение нужных бизнес-данных, но при этом данные не занимали ни байта лишнего на диске. Стремление сэкономить всего лишь байт может показаться странным, но в базах данных с миллионами и миллиардами строк выделение даже одного лишнего байта в строке приведет к значительным затратам дискового пространства. Еще важнее то, что при выделении лишних байт впустую расходуется самый ценный ресурс — память сервера, поскольку все данные, с которыми работают приложения, проходят через память.

Числовые типы данных

В SQL Server 2008 имеется 9 числовых типов данных, предназначенных для хранения целых чисел, десятичных дробей и денежных величин. В табл. 3-1 перечислены доступные числовые типы данных, диапазоны их значений и занимаемое ими место.

Табл. 3-1. Числовые типы данных

Тип данных	Диапазон значений	Занимаемое пространство
<i>TINYINT</i>	от 0 до 255	1 байт
<i>SMALLINT</i>	от -32 768 до 32 767	2 байта
<i>INT</i>	от -2 ³¹ до 2 ³¹ -1	4 байта
<i>BIGINT</i>	от -2 ⁶³ до 2 ⁶³ -1	8 байт
<i>DECIMAL(P,S)</i> и <i>NUMERIC(P,S)</i>	от -10 ³⁸ +1 до 10 ³⁸ -1	От 5 до 17 байт
<i>SMALLMONEY</i>	от -214 748,3648 до 214 748,3647	4 байта
<i>MONEY</i>	от -922 337 203 685 477,5808 до 922 337 203 685 477,5807	8 байт
<i>REAL</i>	от -3,4E+38 до -1,18E-38, 0 и от 1,18E-38 до 3,4E+3	4 байта
<i>FLOAT(N)</i>	от -1,79E+308 до -2,23E-308, 0 и от 2,23E-308 до 1,79E+308	4 байта или 8 байт

ПРИМЕЧАНИЕ Типы данных NUMERIC и DECIMAL

Типы данных *NUMERIC* и *DECIMAL* совершенно одинаковы. Они сосуществуют в SQL Server для обеспечения обратной совместимости.

Типы *MONEY* и *SMALLMONEY* предназначены, прежде всего для хранения денежных величин и позволяют хранить не более четырех десятичных знаков.

У *FLOAT* есть дополнительный параметр — количество цифр после запятой, называемое мантиссой. Если задана мантисса от 1 до 24, тип *FLOAT* занимает 4 байта, а при мантиссе от 25 до 53 тип *FLOAT* занимает 8 байт.

ПРИМЕЧАНИЕ Точность числовых типов

Типы *FLOAT* и *REAL* относятся к приближительным числовым типам или числовым типам с плавающей запятой. Значения, хранящиеся в столбцах типа *FLOAT* или *REAL*, зависят от используемой процессорной архитектуры. При переносе базы данных с сервера с процессорами Intel на сервер с процессорами AMD значения этих столбцов могут измениться. Если вы используете *FLOAT* или *REAL* из-за поддерживаемых ими диапазонов значений, вы обязаны учитывать погрешность, накапливающуюся при расчетах.

Десятичные типы данных

У десятичных типов данных два параметра — точность и масштаб. Точность (precision) задает общее количество цифр до и после запятой, а масштаб (scale) задает максимальное количество цифр после запятой. Например, в столбце с типом *DECIMAL*(8,3) SQL Server сможет хранить значения из восьми знаков с тремя знаками после запятой, т.е. от -99999,999 до 99999,999.

Как показано в табл. 3-2, пространство, занимаемое десятичным типом данных, зависит от заданной для него точности.

Табл. 3-2. Пространство, занимаемое типами *DECIMAL* и *NUMERIC*

Точность	Занимаемое пространство
от 1 до 9	5 байт
от 10 до 19	9 байт
от 20 до 28	13 байт
от 29 до 38	17 байт

Символьные типы данных

В SQL Server 2008 есть 4 типа для хранения символьных данных. Выбор символьного типа данных определяется длиной значений — постоянная или переменная, и тем, планируется ли хранить данные в кодировке Unicode. В табл. 3-3 показано, сколько пространства занимают символьные типы данных.

Табл. 3-3. Символьные типы данных

Тип данных	Занимаемое пространство
<i>CHAR</i> (<i>n</i>)	Не поддерживает Unicode, <i>n</i> байт (пространство выделяется на все <i>n</i> символов, по одному байту на каждый), максимальный размер — 8000 байт
<i>VARCHAR</i> (<i>n</i>)	Не поддерживает Unicode, по одному байту на каждый символ значения столбца, максимальный размер — 8000 байт
<i>NCHAR</i> (<i>n</i>)	Кодировка Unicode, 2 × <i>n</i> байт (пространство выделяется на все <i>n</i> символов, по два байта на каждый), максимальный размер — 8000 байт
<i>NVARCHAR</i> (<i>n</i>)	Кодировка Unicode, по два байта на каждый символ значения столбца, максимальный размер — 8000 байт

Вместо количества символов можно указать ключевое слово *MAX*, например: *VARCHAR(MAX)*. Типы данных *VARCHAR(MAX)* и *NVARCHAR(MAX)* позволяют хранить до 2 гигабайт данных.

Типы данных для даты и времени

Одно из важнейших недавних усовершенствований SQL Server — расширение системы типов данных, благодаря которому стало возможно хранить дату и время по отдельности (табл. 3-4).

Табл. 3-4. Типы данных для хранения дат и времени

Тип	Диапазон значений	Точность	Занимаемое пространство
<i>SMALLDATETIME</i>	от 01.01.1990 до 06.06.2079	1 минута	4 байта
<i>DATETIME</i>	от 01.01.1753 до 31.12.9999	0,00333 секунды	8 байт
<i>DATETIME2</i>	от 01.01.0001 до 31.12.9999	100 наносекунд	От 6 до 8 байт
<i>DATETIMEOFFSET</i>	от 01.01.0001 до 31.12.9999	100 наносекунд	От 8 до 10 байт
<i>DATE</i>	от 01.01.0001 до 31.12.9999	1 день	3 байта
<i>TIME</i>	от 00:00:00.0000000 до 23:59:59.9999999	100 наносекунд	От 3 до 5 байт

Типы *SMALLDATETIME* и *DATETIME* существовали в предыдущих версиях SQL Server; в них дата и время хранятся вместе, в одном значении. Диапазон значений типа *DATETIME* был слишком узким для исторических приложений, поэтому в SQL Server 2008 ввели *DATETIME2*, обеспечивающий лучшую точность, чем *SMALLDATETIME* или *DATETIME*, и поддерживающий более широкий диапазон значений.

Тип *DATETIMEOFFSET* позволяет хранить часовые пояса в приложениях, работающих с местными временами и датами.

И, наконец, самые востребованные новые типы данных — *DATE* и *TIME*. Теперь можно хранить значения, содержащие только дату или только время, благодаря чему исчезли многие проблемы синтаксического анализа и сравнения значений, с которыми сталкивались разработчики в предыдущих версиях SQL Server.

Двоичные данные

Для хранения двоичных данных предназначены три типа данных (табл. 3-5).

Табл. 3-5. Двоичные типы данных

Тип данных	Диапазон значений	Занимаемое пространство
<i>BIT</i>	NULL, 0 или 1	1 бит
<i>BINARY</i>	Двоичные данные фиксированной длины	До 8000 байт
<i>VARBINARY</i>	Двоичные данные переменной длины	До 8000 байт

Как и в случае символьных типов данных переменной длины, при определении данных типа *VARBINARY* можно указывать ключевое слово *MAX*. Это позволяет хранить до 2 Гб данных и, в то же время, использовать все программные функции работы с двоичными данными.

Тип данных XML

Этот тип позволяет хранить и обрабатывать XML-документы (Extensible Markup Language), используя встроенные средства SQL Server. При хранении XML-документов их максимальный размер ограничен 2 Гб, а количество уровней вложенности документа не может превышать 128. XML-документы можно хранить в символьных столбцах, однако для типа данных *XML* реализованы встроенные механизмы, «понимающие» структуру XML-данных и значение XML-тэгов документа.

Поскольку тип *XML* «понимает» XML-структуры, при необходимости можно реализовать проверку XML-столбцов, разрешив хранить только документы, соответствующие одной или нескольким XML-схемам.

Для хранения XML-схем в SQL Server служит структура, называемая *коллекцией схем* (schema collection). Она содержит одну или несколько XML-схем. Если коллекцию схем применить к XML-столбцу, то в нем можно будет хранить только документы, которые прошли проверку на соответствие XML-схемам из связанной со столбцом коллекции.

Приведенная ниже команда создает коллекцию XML-схем:

```
CREATE XML SCHEMA COLLECTION ProductAttributes AS
'<xsd:schema xmlns:schema="PowerTools"
xmlns:xsd=http://www.w3.org/2001/XMLSchema
xmlns:sqltypes=
http://schemas.microsoft.com/sqlserver/2004/sqltypes
targetNamespace="PowerTools" elementFormDefault="qualified">
<xsd:import namespace=
«http://schemas.microsoft.com/sqlserver/2004/sqltypes»
schemaLocation=
«http://schemas.microsoft.com/sqlserver/2004/
sqltypes/sqltypes.xsd» />
<xsd:element name="dbo.PowerTools">
<xsd:complexType>
<xsd:sequence>
<xsd:element name="Category">
<xsd:simpleType>
<xsd:restriction base="sqltypes:varchar"
sqltypes:localeId="1033"
sqltypes:sqlCompareOptions=
«IgnoreCase IgnoreKanaType
IgnoreWidth» sqltypes:sqlSortId="52">
<xsd:maxLength value="30" />
</xsd:restriction>
</xsd:simpleType>
```

```
</xsd:element>
<xsd:element name="Amperage">
  <xsd:simpleType>
    <xsd:restriction base="sqltypes:decimal">
      <xsd:totalDigits value="3" />
      <xsd:fractionDigits value="1" />
    </xsd:restriction>
  </xsd:simpleType>
</xsd:element>
<xsd:element name="Voltage">
  <xsd:simpleType>
    <xsd:restriction base="sqltypes:char">
      sqltypes:localeId="1033"
      sqltypes:sqlCompareOptions="IgnoreCase
      IgnoreKanaType IgnoreWidth"
      sqltypes:sqlSortId="52">
      <xsd:maxLength value="7" />
    </xsd:restriction>
  </xsd:simpleType>
</xsd:element>
</xsd:sequence>
</xsd:complexType>
</xsd:element>
</xsd:schema>
```

Пространственные типы данных

SQL Server 2008 поддерживает два типа для хранения пространственных данных — *GEOMETRY* и *GEOGRAPHY*. Оба реализованы с использованием функций CLR (Common Language Runtime), ставших доступными в SQL Server 2005. Геометрический тип данных основан на евклидовой геометрии и служит для хранения точек, прямых, кривых и многоугольников. Географический тип данных основан на эллиптических координатах и служит для хранения таких данных как широта и долгота.

Столбцы с пространственными данными создают, задавая в таблицах тип *GEOMETRY* или *GEOGRAPHY*. Чтобы сохранить значение в пространственном столбце, необходимо создать экземпляр значения с помощью одной из нескольких пространственных функций, специфичных для типа данных, хранящихся в столбце. Столбец типа *GEOMETRY* может содержать один из 7 геометрических объектов (табл. 3-6), причем координаты в определении объекта отделяются друг от друга пробелами.

Табл. 3-6. Экземпляры геометрических типов данных

Экземпляр	Описание
<i>Point</i>	Имеет координаты <i>x</i> и <i>y</i> , также может содержать значения уровня (elevation) и меры (measure)
<i>LineString</i>	Набор точек, определяющих начало, конец и все изгибы ломаной линии. Каждая из точек может содержать значения уровня и меры

Табл. 3-6. (окончание)

Экземпляр	Описание
<i>Polygon</i>	Поверхность, определяемая последовательностью точек, задающих внешнюю границу, также может содержать несколько внутренних колец. Содержит как минимум три различные точки
<i>GeometryCollection</i>	Содержит один или несколько экземпляров других геометрических фигур, таких как <i>Point</i> и <i>LineString</i>
<i>MultiPolygon</i>	Содержит координаты нескольких экземпляров <i>Polygon</i>
<i>MultiLineString</i>	Содержит координаты нескольких экземпляров <i>LineString</i>
<i>MultiPoint</i>	Содержит координаты нескольких экземпляров <i>Point</i>

Экземпляры типов с приставкой *Multi* позволяют хранить сразу несколько фигур в одном экземпляре. *GeometryCollection* позволяет объединить несколько фигур в одном экземпляре, получив составную фигуру. Когда создается экземпляр объекта, сохраняемый в столбце типа *GEOMETRY* или *GEOGRAPHY*, данные и определения экземпляра записываются в столбец. Поскольку тип объекта и значения координат неотделимы, можно хранить в одном столбце объекты различных типов.

Географические данные хранятся в виде точек широты и долготы. На географические данные накладывается единственное ограничение: они и операции их сравнения не могут быть выходить за пределы полушария.

Тип данных **HIERARCHYID**

Тип данных *HIERARCHYID* служит для систематизации иерархических данных, таких как структуры организаций, спецификации изделий и карты технологических процессов. Тип *HIERARCHYID* хранит положение в древовидной иерархии. Он позволяет быстро находить узлы иерархии или перемещать данные между ними.

Свойства столбцов

У столбцов может быть семь свойств: возможность наличия значений *NULL* (nullability), *COLLATE*, *IDENTITY*, *ROWGUIDCOL*, *FILESTREAM*, *NOT FOR REPLICATION* и *SPARSE*.

Возможность наличия значения **NULL**

Чтобы указать, допускает ли столбец значения *NULL*, в его свойствах задают *NULL* или *NOT NULL*. В любой выполняемой команде всегда следует явно задавать необходимые параметры, особенно при создании объектов. Если не указать свойство nullability, SQL Server при создании таблицы примет значение параметра по умолчанию, что иногда приводит к неожиданным результатам. Кроме того, не гарантируется, что значение по умолчанию будет одинаковым во всех базах данных, поскольку можно его изменить, задав свойство *ANSI_NULL_DEFAULT*.

Свойство **COLLATE**

Параметры сортировки (collation sequences) определяют порядок обработки символов различных языков. При установке экземпляра SQL Server указывают

параметры сортировки по умолчанию. Можно задать свойство *COLLATE* базы данных, переопределив параметры сортировки экземпляра, которые SQL Server использует как параметры сортировки по умолчанию для объектов базы данных. Аналогично тому, как переопределяют параметры сортировки по умолчанию на уровне базы данных, можно переопределять параметры сортировки для всей таблицы или для отдельных столбцов.

Указывая свойство *COLLATE* для символьных столбцов, можно менять поведение столбцов в соответствии с потребностями того или иного языка.

Свойство *IDENTITY*

Идентификаторы (*identities*) служат для автоматического задания значений столбцов при вставке данных. Столбец со свойством *IDENTITY* обновлять нельзя. Поскольку для столбцов идентификаторов требуется указывать начальное значение и шаг приращения, используемые в дальнейшем при добавлении каждой строки, эти столбцы должны быть числовых типов (кроме *float* и *real*). В таблице может быть только один столбец идентификаторов.

Часто столбцы идентификаторов являются уникальными, но это не обязательно. Чтобы сделать столбец идентификаторов уникальным, необходимо применить к нему ограничение (подробнее об ограничениях см. занятие 2).

Хотя SQL Server автоматически вычисляет следующее значение последовательности, можно явно задавать значения в столбце идентификаторов, выполнив команду *SET IDENTITY_INSERT <имя_таблицы> ON*. Кроме того, можно задать следующее автоматически генерируемое значение, изменив начальное значение командой *DBCC CHECKIDENT*.

Свойство *ROWGUIDCOL*

Это свойство используется в основном при репликации слиянием для определения столбца, уникально идентифицирующего строки в базах данных. Свойство *ROWGUIDCOL* служит для обеспечения того, что в таблице будет не более одного столбца этого типа и этот столбец имеет тип *UNIQUEIDENTIFIER*.

Свойство *FILESTREAM*

Базы данных предназначены для хранения хорошо структурированных данных, состоящих из отдельных элементов. Поскольку организации используют все более разнообразные типы данных, возникает необходимость консолидировать данные всевозможных форматов в единой архитектуре хранения. SQL Server позволяет хранить все имеющиеся у организации данные, большинство из которых существует в виде документов, электронных таблиц и других типов файлов.

До появления SQL Server 2008 приходилось извлекать содержимое файла и сохранять его в столбцах типов *VARBINARY(MAX)*, *VARCHAR(MAX)* или *NVARCHAR(MAX)*. Однако таким образом можно было сохранить всего лишь не более 2 Гб данных (максимальный размер типов данных, предназначенных для хранения больших объемов информации). Чтобы обойти это ограничение, многие организации хранили в SQL Server имена файлов, а сами файлы хранили в операционной системе. Основным недостатком хранения файлов вне базы данных являлось то, что запросто можно было перенести, удалить или переименовать файл, не внося соответствующие изменения в базу данных.

В SQL Server 2008 введено новое свойство столбца — *FILESTREAM*. Оно позволяет сочетать лучшие черты обоих подходов. Крупные двоичные объекты (Binary large objects, BLOBs), хранящиеся в столбцах *FILESTREAM*, находятся под контролем SQL Server, обеспечивающего их корректность; однако их данные содержатся в файлах операционной системы. Благодаря хранению информации в файловой системе, а не в базе данных, размер BLOB-данных больше не ограничен 2 Гб. Кроме того, при резервном копировании базы данных все файлы копируются одновременно с базой данных, следовательно, обеспечивается синхронизация состояния каждого файла с состоянием базы данных.

Свойство *FILESTREAM* применяется к столбцам типа *VARBINARY(MAX)*. Столбец, хранящийся в таблице, содержит 16-байтный идентификатор файла. Доступом к файлам, хранящимся в операционной системе, управляет SQL Server.

ПРИМЕЧАНИЕ На экзамене

Файловые группы (*FILEGROUP*), предназначенные для хранения файловых потоков (*FILESTREAM*), недоступны в моментальных снимках баз данных (Database Snapshot) и отмечаются в них как автономные. Кроме того в базах данных с файловыми группами *FILESTREAM* невозможно реализовать зеркальное отображение баз данных (Database Mirroring).

Свойство NOT FOR REPLICATION

Это свойство присваивается столбцам, определенным со свойством *IDENTITY*. При определении столбца идентификаторов указывают *начальное значение* (seed) и *шаг приращения* (increment), прибавляемый при генерации следующего значения. При явной вставке значения в столбец идентификаторов SQL Server автоматически задает столбцу новое очередное значение. Если таблица участвует в репликации, нет необходимости в повторной генерации нового очередного значения столбца идентификаторов при каждой синхронизации данных. При указании параметра *NOT FOR REPLICATION* SQL Server не генерирует очередное значение для столбца идентификаторов, когда механизм репликации применяет изменения.

Свойство SPARSE

Этот параметр является нововведением SQL Server 2008 и позволяет объявлять столбцы как разреженные, чтобы экономить пространство для хранения столбцов, содержащих много значений *NULL*. Чтобы столбец можно было пометить как *SPARSE*, он должен поддерживать значения *NULL*. При хранении *NULL* в столбцах со свойством *SPARSE* дисковое пространство вообще не расходуется. Однако для значений, отличных от *NULL*, расходуется на 4 байта больше, чем при обычном хранении соответствующего типа данных. Свойство *SPARSE* не следует указывать, если доля строк, содержащих *NULL*, не достаточна для обеспечения экономии пространства с учетом того, что значения, отличные от *NULL*, занимают больше места.

Свойство *SPARSE* неприменимо:

- к столбцам со свойствами *ROWGUIDCOL* или *IDENTITY*;
- к столбцам типов *TEXT*, *NTEXT*, *IMAGE*, *TIMESTAMP*, *GEOMETRY*, *GEOGRAPHY* или типов, определяемых пользователем;

- к столбцам типа *VARBINARY(MAX)* со свойством *FILESTREAM*;
- к вычисляемым столбцам или столбцам, для которых задано правило или значение по умолчанию;
- столбцам, входящим в кластеризованный индекс или первичный ключ;
- к столбцам, указываемым в инструкции *ALTER TABLE*.

Ограничение на размер строки

Если максимальный размер строки таблицы превышает 4009 байт, нельзя ни выполнить инструкцию *ALTER*, устанавливающую свойство *SPARSE*, ни добавить столбец со свойством *SPARSE*. При выполнении *ALTER* каждая строка формируется заново, причем ее вторая копия записывается на ту же самую страницу данных. Поскольку две копии строки размером более 4009 байт занимают более 8018 байт, инструкция *ALTER* потерпит неудачу.

Есть только три способа обойти это ограничение:

- уменьшить количество данных в строке, добившись, чтобы ее размер не превышал 4009 байт;
- создать новую таблицу, скопировать в нее все данные из старой, удалить старую таблицу, а затем переименовать новую таблицу в старую;
- экспортировать данные, усесть (*truncate*) существующую таблицу, внести изменения, затем импортировать данные обратно в таблицу.

Вычисляемые столбцы

Вычисляемые столбцы — это столбцы таблицы, которые не заполняются данными, а вычисляются с использованием значений других столбцов строки. Например, если в таблице хранятся промежуточные итоговые суммы и суммы оплаты за погрузку, можно создать вычисляемый столбец с общей суммой, которая автоматически будет меняться при изменении промежуточной суммы или суммы оплаты за погрузку.

Для вычисляемого столбца в базе хранится только его определение. При указании вычисляемого столбца в DML-инструкциях (*data manipulation language*) его значения вычисляются во время выполнения. Если вы не хотите, чтобы во время выполнения возникали издержки, связанные с такими вычислениями, задайте свойство *PERSISTED*. В вычисляемом столбце с таким свойством SQL Server хранит результат вычисления в строке и обновляет его при каждом изменении данных, от которых оно зависит.

Сжатие строк и страниц

В SQL Server 2008 появилась поддержка сжатия строк и страниц таблиц, не содержащих столбцы со свойством *SPARSE*, а также индексов и индексированных представлений.

Сжатие на уровне строк позволяет сжимать отдельные строки, чтобы размещать больше строк на странице, что, в свою очередь, сокращает пространство, необходимое для хранения таблицы, поскольку на диске размещается меньше страниц. Так как распаковать данные можно в любой момент, а операция рас-

паковки всегда должна быть успешной, сжатие не позволяет хранить более 8060 байт в одной строке.

Сжатие страниц — это просто сокращение пространства, требуемого для хранения. Когда SQL Server выполняет сжатие страниц кучи (таблицы без кластеризованного индекса), сжимаются только страницы, которые уже есть в таблице. SQL Server сжимает новые данные, добавляемые в кучу, только при выполнении инструкций *BULK INSERT* или *INSERT INTO...WITH (TABLOCK)*. Страницы, добавляемые в таблицу с применением *BCP* или инструкцией *INSERT* без указания на блокировку таблицы, не сжимаются. Чтобы сжимались добавляемые несжатые страницы, необходимо выполнить инструкцию *ALTER TABLE...REBUILD*, указав вариант сжатия *PAGE*.

Параметры сжатия таблицы не распространяются ни на какие некластеризованные индексы или индексируемые представления этой таблицы. Необходимо по отдельности задавать эти параметры для каждого сжимаемого некластеризованного индекса или индексируемого представления. Если таблица секционирована (подробнее см. главу 6), можно применять сжатие на уровне секций.

Данные столбцов типов *VARCHAR(MAX)*, *NVARCHAR(MAX)* и *VARBINARY(MAX)* хранятся в специализированных структурах вне строк. Кроме того, в столбцах *VARBINARY(MAX)* со свойством *FILESTREAM* документ хранится в каталоге, внешнем по отношению к базе данных. Нельзя сжать данные, хранящиеся вне строк.

Создание таблиц

Вот часть синтаксиса создания таблиц:

```
CREATE TABLE
    [ имя_базы_данных . [ имя_схемы ] . | имя_схемы . ]
    имя_таблицы
    ( { <определение_столбца >
      | <определение_вычисляемого_столбца>
      | <определение_набора_столбцов> }
    [ <ограничение_таблицы> ] [ ,...n ] )
    [ ON { имя_схемы_секционирования
      ( имя_столбца_секционирования )
      | файловая группа | «default» } ]
    [ { TEXTIMAGE_ON { файловая группа | «default» } } ]
    [ FILESTREAM_ON { имя_схемы_секционирования
      | файловая группа | «default» } ]
    [ WITH ( <параметр_таблицы> [ ,...n ] ) ] [ ; ]

<определение_столбца> ::=
    имя_столбца <тип_данных>
    [ FILESTREAM ]
    [ COLLATE имя_параметров_сортировки ]
    [ NULL | NOT NULL ]
    [ [ CONSTRAINT имя_ограничения ]
      DEFAULT константное_выражение ]
    [ [ IDENTITY [ ( начальное_значение , шаг_приращения ) ]
```

```

[ NOT FOR REPLICATION ] ]
[ ROWGUIDCOL ] [ <ограничение_столбца> [ ...n ] ] [ SPARSE ]
<тип_данных> ::=
[ имя_схемы_типа . ] имя_типа
[ ( точность [ , масштаб ] | max |
  [ { CONTENT | DOCUMENT } ] коллекция_XML_схем ) ]
<определение_вычисляемого_столбца> ::=
имя_столбца AS выражение_вычисляемого_столбца
[ PERSISTED [ NOT NULL ] ]
<определение_набора_столбцов> ::=
имя_набора_столбцов XML COLUMN_SET FOR ALL_SPARSE_COLUMNS
<параметр_таблицы> ::=
{ DATA_COMPRESSION = { NONE | ROW | PAGE }
[ ON PARTITIONS ( { <выражение_номера_секции>
  | <диапазон> } [ , ...n ] ) ] }

```

Стандартная таблица SQL Server 2008 может содержать 1024 столбца. Однако с помощью новых возможностей — определения набора столбцов и поддержки разреженных столбцов — можно создать таблицу с 30 000 столбцов. Таблицы, в которых благодаря использованию определений наборов столбцов содержится более 1024 столбцов, называют *широкими таблицами*. Но объем данных, хранящихся в любой строке, все равно не может превышать 8019 байт, при этом не учитываются столбцы *VARCHAR(MAX)*, *NVARCHAR(MAX)* или *VARBINARY(MAX)*.

Помимо таблиц, постоянно хранящихся в базе данных, имеется 4 временных табличных структуры.

Временные таблицы хранятся в базе данных *tempdb* и бывают локальными или глобальными. Локальные временные таблицы обозначаются символом # в начале имени. Они видимы только в рамках соединения, в котором созданы. У глобальной временной таблицы имя начинается с символов ##, и она видима для всех соединений экземпляра. И глобальные, и локальные временные таблицы автоматически удаляются при разрыве соединения, в котором они были созданы.

Табличные переменные создают, чтобы передавать наборы данных внутри объектов, таких как хранимые процедуры и функции. Табличные переменные можно заполнять инструкциями *INSERT*, *UPDATE*, *DELETE* и *MERGE* и даже указывать в предложении *JOIN*, так же как таблицы. Табличная переменная — структура, хранящаяся в памяти, видимая только в том соединении, в котором она объявлена, и освобождаемая по завершении выполнения кода, в котором объявлена.

Табличный тип данных — нововведение SQL Server 2008, позволяющее создавать параметры хранимых процедур и функций для передачи наборов данных между объектами.

Контрольные вопросы

1. Как надо проектировать базы данных?
2. Какие три новых возможности конфигурирования столбцов, строк и страниц таблиц появились в SQL Server 2008?

Ответы на контрольные вопросы

1. Главный принцип проектирования баз данных — «класть вещи на свое место». Если требуется хранить много строк информации, ссылающихся на одну сущность, заведите для этих строк отдельную таблицу. С другой стороны, каждая таблица определяет некий важный для системы объект, данные экземпляров которого требуется хранить, а столбцы таблицы определяют, какие именно данные вы будете хранить.
2. Можно объявить столбцы как *SPARSE*, чтобы оптимизировать хранение значений *NULL*. К столбцу *VARBINARY(MAX)* можно применить свойство *FILESTREAM* — это позволит хранить в каталоге операционной системы, документы размером более 2 Гб. Можно сжимать строки, чтобы на каждой странице умещалось больше строк, и сжимать страницы, чтобы уменьшить пространство, необходимое для хранения таблиц, индексов или индексированных представлений.

Практикум. Создание таблиц

Вы создадите схему для хранения набора таблиц, а затем создадите таблицы, указав ограничения и параметры хранения строк и страниц.

1. Выполните следующий код, чтобы создать схему *test* в базе данных *AdventureWorks*:

```
USE AdventureWorks
GO

CREATE SCHEMA test AUTHORIZATION dbo
GO
```

2. Выполните следующий код, чтобы создать таблицу со столбцами, имеющими свойства *IDENTITY* и *SPARSE*:

```
CREATE TABLE test.Customer
(CustomerID INT IDENTITY(1,1),
LastName VARCHAR(50) NOT NULL,
FirstName VARCHAR(50) NOT NULL,
CreditLine MONEY SPARSE NULL,
CreationDate DATE NOT NULL)
GO
```

3. Выполните следующий код, чтобы создать таблицу с вычисляемым столбцом и сжатием строк:

```
CREATE TABLE test.OrderHeader
(OrderID INT IDENTITY(1,1),
CustomerID INT NOT NULL,
```

```
OrderDate DATE NOT NULL,  
OrderTime TIME NOT NULL,  
SubTotal MONEY NOT NULL,  
ShippingAmt MONEY NOT NULL,  
OrderTotal AS (SubTotal + ShippingAmt))  
WITH (DATA_COMPRESSION = ROW)  
GO
```

Резюме занятия

- Схемы позволяют группировать взаимосвязанные объекты, а также служат для защиты объектов.
- Самое важное решение, принимаемое при проектировании таблицы, — определение типов данных столбцов.
- С помощью определений наборов столбцов и объявления разреженных столбцов можно создавать таблицы, содержащие до 30 000 столбцов.
- Таблицы, индексы и индексированные представления можно сжимать, используя сжатие на уровне строк или страниц; однако сжатие не совместимо с разреженными столбцами.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите соответствующий материал. Ответы для самопроверки — в разделе «Ответы» в конце главы.

1. Какие варианты не совместимы со сжатием строк или страниц? (Выберите два варианта, каждый из которых является верным ответом).
 - A. Столбец типа *VARCHAR(MAX)*.
 - B. Разреженный столбец.
 - C. Таблица с набором столбцов.
 - D. Столбец *VARBINARY(MAX)* со свойством *FILESTREAM*.

Занятие 2. Реализация ограничений

Ограничения используют для реализации бизнес-правил или обеспечения целостности данных. В этом занятии вы ознакомитесь с ограничениями и с тем, как реализовывать ограничения различных типов в базе данных.

Изучив материал этого занятия, вы сможете:

- ✓ создавать первичный ключ;
- ✓ создавать внешний ключ;
- ✓ создавать ограничения уникальности;
- ✓ реализовывать ограничения по умолчанию;
- ✓ применять проверочные ограничения.

Продолжительность занятия — 40 минут.

Первичные ключи

В таблице можно определить не более одного первичного ключа. *Первичный ключ* (primary key) задает столбец (или столбцы), уникально идентифицирующий строки таблицы. Вы должны определить все столбцы, входящие в первичный ключ, как *NOT NULL*.

При создании первичного ключа указывают, каким он будет — кластеризованным или некластеризованным. При выборе кластеризованного первичного ключа (это вариант по умолчанию) SQL Server хранит таблицу отсортированной в соответствии со значениями первичного ключа.

ПРИМЕЧАНИЕ На экзамене

По умолчанию создается кластеризованный первичный ключ. Если создать кластеризованный первичный ключ в сжатой таблице, сжатие будет применено при перестроении таблицы.

Внешние ключи

Внешние ключи (foreign keys) используют для обеспечения ссылочной целостности таблиц базы данных. Создание внешних ключей позволяет гарантировать, что связанные таблицы не будут содержать некорректные строки, не имеющие пары. Внешний ключ создает отношение «родитель-потомок» между двумя таблицами, обеспечивая, чтобы в дочернюю таблицу нельзя было записать значение, которого нет в родительской таблице. Так, например, нет смысла хранить данные о заказе несуществующего клиента.

Чтобы можно было создать внешний ключ, у родительской таблицы должен быть первичный ключ, через который будет устанавливаться связь с дочерней таблицей. Кроме того, типы данных одного или нескольких столбцов родительской и дочерней таблиц, входящих во внешний ключ, должны быть совместимыми. Если первичный ключ состоит из нескольких столбцов, для определения внешнего ключа необходимо, чтобы в дочерней таблице существовали те же столбцы, что входят в первичный ключ родителя.

ВНИМАНИЕ! Каскадные внешние ключи

Одним из параметров внешнего ключа является *CASCADE*. Внешний ключ можно настроить так, чтобы изменения родительской таблицы «по каскаду» автоматически распространялись на дочернюю таблицу. Например, при удалении клиента SQL Server удалит и все его заказы. Каскадные операции — на редкость неудачное решение. Очень часто бывает, что все таблицы базы данных связаны внешними ключами. Если выполнить инструкцию *DELETE* без предложения *WHERE* по отношению к «не той» таблице, SQL Server очень быстро удалит все строки всех таблиц базы данных. А если создать внешний ключ без параметра *CASCADE*, то при попытке удалить родительскую строку, на которую ссылаются дочерние, вы получите ошибку.

Ограничения уникальности

Ограничения уникальности позволяют определить столбец или столбцы, значения которых должны быть уникальными в пределах таблицы. Дублирование

значений не допускается. Например, можно потребовать, чтобы в базе данных не было клиентов с одинаковыми именами. Ограничения уникальности похожи на первичные ключи, но во входящих в них столбцах допускаются значения *NULL*.

ПРИМЕЧАНИЕ На экзамене

В общем случае разные значения *NULL* не равны друг другу и их нельзя сравнивать, но в ограничениях уникальности *NULL* рассматриваются так же, как любые другие значения. Если ограничение уникальности определено для одного столбца, то значение *NULL* в этом столбце может быть не более, чем у одной строки. Если ограничение уникальности определено для нескольких столбцов, вы можете хранить в этих столбцах значения *NULL*, пока не появятся две строки с одинаковыми комбинациями *NULL* и значений, отличных от *NULL*.

Ограничения по умолчанию

Ограничения по умолчанию позволяют определить значения, которые будут записываться в столбец, если значение столбца не задано приложением. Ограничения по умолчанию применяются только к новым строкам, добавляемым инструкциями *INSERT*, *BCP* или *BULK INSERT*. Ограничения по умолчанию можно определить как для столбцов *NULL*, так и для *NOT NULL*. Если для столбца определено ограничение по умолчанию, а приложение передает значение *NULL* для этого столбца, то SQL Server запишет в столбец *NULL* вместо значения по умолчанию. SQL Server записывает в столбец значение по умолчанию, только если приложение выполняет инструкцию *INSERT*, в которой не указан этот столбец.

Проверочные ограничения

Проверочные ограничения задают диапазон значений столбца. Их можно создавать на уровне столбцов — тогда в них нельзя ссылаться на другие столбцы таблицы. Проверочные ограничения можно создавать на уровне таблиц — тогда в них можно ссылаться на столбцы только данной таблицы, но не других.

В результате вычисления выражения, задаваемого в проверочном ограничении, должно возвращаться значение *true* или *false*. Любые другие значения не допускаются. Данные, успешно проходящие проверку, вносятся в столбец или таблицу, а не прошедшие — отвергаются, и в приложение возвращается ошибка.

В проверочных ограничениях могут выполняться простые операции сравнения, такие как *>*, *<*, *>=*, *<=*, *<>* и *=*. Можно создавать более сложные проверочные ограничения, используя операции *AND*, *OR* или *NOT*. Кроме того, в проверочных ограничениях можно использовать шаблоны *%* и *_*, и выполнять операции проверки соответствия шаблонам. Например, такое ограничение проверяет выражение на соответствие формату номеров социального обеспечения США (три цифры, тире, две цифры, тире, четыре цифры):

```
CHECK (Column1 LIKE '[0-9][0-9][0-9]-[0-9][0-9]-[0-9][0-9][0-9][0-9]')
```

Контрольные вопросы

1. Чем отличаются первичный ключ и ограничение уникальности?
2. Какие ограничения должны быть у родительской таблицы при создании внешнего ключа?

Ответы на контрольные вопросы

1. В первичном ключе не допускаются значения *NULL*.
2. У родительской таблицы должен быть первичный ключ, используемый при определении связи между родительской и дочерней таблицами. Кроме того, если первичный ключ родителя содержит несколько столбцов, то столбцы соответствующих типов должны существовать в дочерней таблице, для которой создается внешний ключ.

Практикум. Создание ограничений

Вы добавите ограничения в таблицы, созданные на занятии 1.

1. Выполните следующий код, чтобы создать первичные ключи в таблицах *Customer* и *OrderHeader*:

```
ALTER TABLE test.Customer
    ADD CONSTRAINT pk_customer PRIMARY KEY CLUSTERED (CustomerID)
GO
ALTER TABLE test.OrderHeader
    ADD CONSTRAINT pk_orderheader PRIMARY KEY CLUSTERED (OrderID)
GO
```

2. Выполните следующий код, чтобы создать внешний ключ, связывающий таблицы *Customer* и *OrderHeader*:

```
ALTER TABLE test.OrderHeader
    ADD CONSTRAINT fk_orderheadertocustomer
        FOREIGN KEY (CustomerID)
        REFERENCES test.Customer (CustomerID)
GO
```

3. Выполните следующий код, чтобы задать значения по умолчанию в столбцах *CreationDate* и *OrderDate*:

```
ALTER TABLE test.Customer
    ADD CONSTRAINT df_creationdate DEFAULT (GETDATE())
        FOR CreationDate
GO
ALTER TABLE test.OrderHeader
    ADD CONSTRAINT df_orderdate DEFAULT (GETDATE())
        FOR OrderDate
GO
```

4. Выполните следующий код, чтобы добавить проверочное ограничение для столбца *SubTotal*:

```
ALTER TABLE test.OrderHeader  
  ADD CONSTRAINT ck_subtotal CHECK (SubTotal > 0)  
GO
```

Резюме занятия

- Первичный ключ — это столбец (или столбцы), уникально идентифицирующий каждую строку таблицы.
- Внешние ключи служат для обеспечения ссылочной целостности при связывании таблиц.
- Ограничения по умолчанию задают значения, записываемые в столбцы, если значение столбца не задается приложением.
- Проверочные ограничения определяют, какие значения допустимы в столбцах.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Столбцы с какими свойствами не могут быть разреженными? (Выберите два варианта, каждый из которых является верным ответом).

A. *FILESTREAM*.

B. *NULL*.

C. *NOT FOR REPLICATION*.

D. *COLLATE*.

Закрепление материала главы

Для закрепления знаний и навыков, полученных при изучении этой главы, выполните следующее:

- изучите резюме главы;
- повторите основные термины, использованные в этой главе;
- выполните задания по темам этой главы;
- сделайте предложенные упражнения;
- пройдите пробный экзамен.

Резюме главы

- Таблицы служат основой любой базы данных, а выбор типа данных в столбцах — самое важное решение, определяющее производительность базы данных.
- Если имеется много строк, в которых определенный столбец содержит значение *NULL*, можно оптимизировать хранение данных, объявив этот столбец как *SPARSE*.
- Сжатие строк и страниц позволяет экономить пространство для хранения и повысить производительность обработки данных.

- Для уникальной идентификации каждой строки таблиц следует создавать первичные ключи.

Основные термины

Понятны ли вам значения приведенных здесь основных терминов? Чтобы проверить свои ответы, загляните в глоссарий на прилагаемом компакт-диске.

- Проверочное ограничение ~ check constraint.
- Ограничение по умолчанию ~ default constraint.
- Свойство *FILESTREAM* ~ *FILESTREAM*.
- Внешний ключ ~ foreign key.
- Столбец идентификаторов ~ identity column.
- Первичный ключ ~ primary key.
- Схема ~ schema.
- Коллекция схем ~ schema collection.
- Разреженный столбец ~ sparse column.
- Ограничение уникальности ~ unique constraint.

Упражнения

Сейчас вы примените на практике знания и навыки, полученные при изучении этой главы. Ответы на вопросы см. в разделе «Ответы» в конце книги.

Упражнение 1. Управление данными

Компания Wide World Importers собирается внедрять новый набор приложений для управления несколькими направлениями деятельности. Нужно обеспечить возможность хранить большой объем данных в корпоративном вычислительном центре и доступ к ним из любой точки земного шара.

Нескольким менеджерам требуется доступ к оперативным отчетам о текущей нагрузке своих сотрудников, а также о новых и незавершенных заказах клиентов. Еще им необходим доступ к большим объемам архивных данных для определения тенденций, оптимизации штата и уровня складских запасов.

Менеджеры хотят разместить руководства по пользованию продуктами на веб-сайте компании, а не поставлять их в составе самих продуктов. Пользователям нужно предоставить возможность просматривать руководства по названию продуктов и искать руководства по определенному тексту в них. Менеджеры по продажам также требуют усовершенствовать веб-сайт компании: описания продуктов должны быть доступны на нескольких языках с возможностью поиска.

В отделе продаж работает большое количество менеджеров по продажам, которые постоянно звонят клиентам, и им нужен доступ к данным по клиентам, которых обслуживает тот или иной менеджер, а также к информации о потенциальных клиентах. Доступ к информации менеджерам по продажам нужен даже в отсутствие подключения к Интернету или корпоративной сети. Периодически менеджеры по продажам подключаются к корпоративной сети и синхронизируют свои данные с корпоративной базой данных.

Средствами Microsoft Visual Studio .NET был разработан ряд приложений для работы в среде Microsoft Windows, а весь доступ к данным осуществляется через хранимые процедуры. Набор приложений одинаков как у пользователей, подключенных напрямую к корпоративному серверу баз данных, так и у менеджеров по продажам, подключающихся к своим локальным серверам баз данных.

Ответьте на следующие вопросы:

1. Как спроектировать таблицы для хранения руководств по продуктам?
2. Как спроектировать таблицы для хранения описаний продуктов на разных языках?
3. Как спроектировать таблицы так, чтобы можно было назначать клиентам менеджеров по продажам, исключая возможность назначить клиенту несуществующего менеджера по продажам?

Рекомендуемые упражнения

Чтобы подготовиться к экзамену по темам этой главы, выполните следующие задания.

Создание таблиц

- **Задание 1.** Вставьте строку в таблицу Customer и посмотрите значение столбца CustomerID. Измените начальное значение или шаг приращения столбца идентификаторов или и то, и другое. Вставьте еще несколько строк в таблицу Customer и посмотрите значения CustomerID. Соответствуют ли они вашим ожиданиям?

Создание ограничений

- **Задание 1.** Попробуйте вставить строку в таблицу данных о клиентах (Customer), не указав фамилию. Соответствует ли результат вашим ожиданиям?
- **Задание 2.** Попытайтесь добавить данные о заказе с неверным CustomerID. Что произойдет?
- **Задание 3.** Что будет записано в столбцы CreationDate или OrderDate при добавлении данных о клиенте или заказе?
- **Задание 4.** Попытайтесь добавить данные о заказе с отрицательным промежуточным итогом. Что произойдет?

Пробный экзамен

На прилагаемом компакт-диске содержится пробный экзамен, обеспечивающий широкие возможности самоподготовки. Например, можно проверить, как усвоены темы этой главы либо все темы экзамена 70-432. Можно симитировать настоящий экзамен или выбрать режим обучения, когда после каждого вопроса приводятся правильные ответы с пояснениями.

ПРИМЕЧАНИЕ Пробный экзамен

Подробнее о пробном экзамене см. «Введение».

Проектирование индексов SQL Server

Занятие 1. Архитектура индексов	77
Занятие 2. Проектирование индексов	82
Занятие 3. Поддержка индексов	93

В этой главе рассказывается о внутренней архитектуре индекса, а также о том, как создавать кластеризованные, некластеризованные, пространственные и XML-индексы. Вы узнаете, как управлять индексами, чтобы обеспечить максимальную производительность.

Темы экзамена

- Поддержка индексов.

Прежде всего

Для изучения материалов этой главы необходимо иметь:

- установленный экземпляр Microsoft SQL Server 2008;
- подключенную базу данных AdventureWorks.

Практический совет

Майк Хотек

У одного из моих клиентов была среда со среднего размера хранилищем данных, которое использовалась для поддержки принятия решения в области ценообразования и выпуска продуктов. Раз в месяц в хранилище поступали данные из нескольких систем-источников. Последние данные объединялись с уже имеющимися на промежуточном сервере. После импорта данных выполнялись несколько процессов для вычисления агрегатов, производных таблиц, преобразования данных в таблицы фактических данных и измерений, а также для денормализации данных, которые предполагалось использовать для активного обслуживания запросов.

Основная проблема заключалась в производительности и доступности данных. Для исправления ситуации был предпринят многомесячный проект по замене всех серверов, сетевого оборудования и сетей хранения (SAN), на который потратили более миллиона долларов.

После обновления оборудования время обработки сократилось до 12–16 дней. Некоторые процессы занимали 12–18 часов. Один из консультантов проанализировал базы данных и сказал, что многие индексы сильно фрагментированы. В течение следующих 2 недель консультанты выполнили дефрагментацию индексов, а также добавили несколько дополнительных индексов. В процессе работы они уверяли клиента, что благодаря этому производительность должна улучшиться. В компании радовались тому, что «наконец им оказывают настоящую помощь» и что «корень проблем — этот чертов SQL Server».

Однако уже следующая ежемесячная обработка продемонстрировала очень небольшое повышение производительности.

Дальнейший анализ показал, что индексы снова сильно фрагментированы. Консультант был не в состоянии понять, что вычислительные процедуры обрабатывали практически все данные каждой таблицы данных. Можно было потратить сколько угодно усилий на дефрагментацию индексов, но при существующих процедурах обработки SQL Server не мог воспользоваться преимуществами всех индексов, скорее наоборот — вычисление индексов только добавляло ненужной работы.

Дальнейший анализ вскрыл главную проблему. Типы данных, используемые в объединениях, не совпадали, предложения *GROUP BY* были включены во многие запросы, которые при этом не содержали агрегатов, временные таблицы заполнялись миллионами строк данных, которые потом нигде не использовались, создавались временные таблицы с десятками или сотнями миллионов строк лишь затем, чтобы создать другие временные таблицы, а те порождали третьи временные таблицы, инструкции *INSERT...SELECT* содержали предложения *ORDER BY*, объединения выполнялись на вычисляемых значениях, схема таблиц была неэффективна, — и этот список можно было продолжать бесконечно.

Мораль истории проста: индексы повышают производительность операций поиска данных, но сами по себе они не в состоянии побороть такие недостатки, как неэффективный код или неудачный проект таблицы.

Занятие 1. Архитектура индексов

Конструкция индексов такова, что позволяет найти нужную информацию в большом объеме данных, выполнив лишь немного операций чтения. На этом занятии рассказывается о внутренней структуре индекса, а также как SQL Server создает и работает с индексами. Эта информация позволит вам более грамотно принимать решения о числе, типе и определении создаваемых индексов.

Изучив материал этого занятия, вы сможете:

- ✓ рассказать, как устроено и управляется сбалансированное дерево;
- ✓ рассказать, почему для индексов в SQL Server используется сбалансированное дерево.

Продолжительность занятия — 20 минут.

Структура индексов

Для получения данных из таблицы серверу SQL Server не обязательно нужны индексы. Чтобы найти нужные данные, таблицу можно просто просканировать, однако в этом случае время поиска нужных данных будет прямо пропорционально объему данных в таблице. Пользователям нужно хранить в таблицах постоянно растущие объемы и иметь разумное время поиска нужных данных независимо от их объема, поэтому без индексов никак не обойтись, если нужно удовлетворить потребности бизнес-приложений, от которых зависит функционирование предприятия.

Индексы не представляют собой ничего нового — мы используем их каждый день. Один из видов индекса — предметный указатель, имеющийся во многих книгах. Если нужно узнать, где в книге речь идет о каком-то понятии, найти нужную информацию можно двумя способами. Можно открыть книгу на первой странице и последовательно листать ее, отыскивая нужную тему. Другой способ — открыть предметный указатель, найти номер страницы, посвященной нужной теме, и открыть ее. Оба способа дадут нужный результат, но предметный указатель позволяет найти нужную информацию, просмотрев минимальное число страниц.

Индекс полезен, только если он позволяет быстро найти данные независимо от их объема. Посмотрите на любой предметный указатель: он организован в привычном для человека алфавитном порядке, позволяя моментально отбросить массу ненужных страниц и сразу найти нужную тему или термин. В SQL Server индексы организованы похожим образом.

Сбалансированные деревья

Структура, используемая SQL Server для построения и поддержания индексов называется *сбалансированным деревом* (B-tree). Пример сбалансированного дерева показан на рис. 4-1.

Сбалансированное дерево состоит из корневого узла, который содержит единственную страницу данных, одной или нескольких страниц промежуточного уровня и одной или нескольких страниц конечного уровня. Основной принцип сбалансированного дерева содержится в первом слове его названия — «сбалансированное». Сбалансированное дерево всегда симметрично, с одинаковым числом страниц справа и слева на каждом уровне.

Страницы конечного уровня содержат элементы, отсортированные в указанном создателем порядке. Данные на конечном уровне содержат все комбинации значений индексируемых столбцов, а число индексных строк на странице определяется местом, необходимым для хранения столбцов, определенных в индексе.

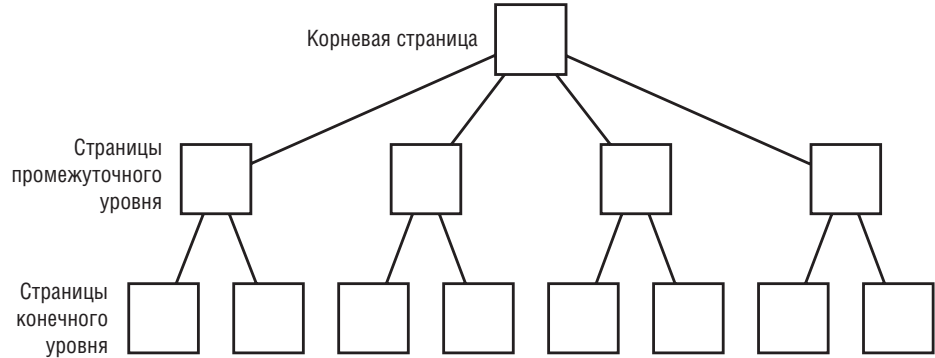


Рис. 4-1. Структура сбалансированного дерева

ПРИМЕЧАНИЕ Хранение элементов индекса

Страницы в SQL Server могут хранить до 8060 байт данных. Поэтому индекс, созданный для столбца с данными типа *INT*, может хранить 2015 значений на странице индекса, а индекс для столбца с данными типа *datetime2* — примерно половину этой цифры, а точнее 1007 значений на страницу.

Корневой и промежуточные уровни индекса создаются из первых элементов каждой страницы нижележащего уровня, а также указателей на страницу, где хранится это значение данных (рис. 4-2).

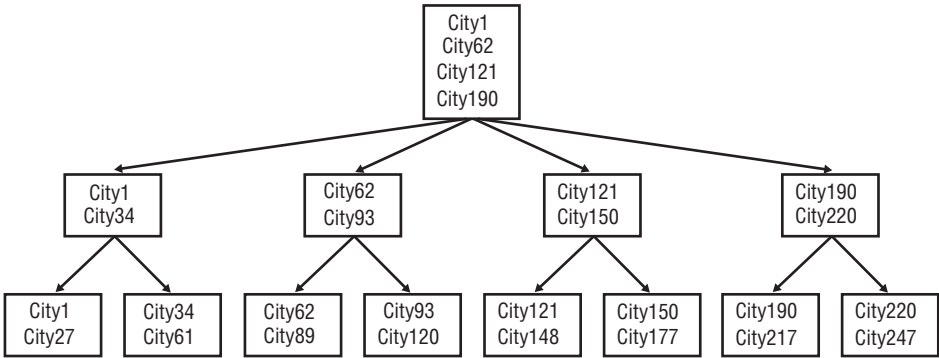


Рис. 4-2. Построение промежуточных и корневого уровней

Запрос просматривает корневую страницу, пока не найдет страницу, которая содержит искомое значение. Далее он использует указатель страницы для перехода на следующий уровень и снова просматривает строки в новой странице, пока не найдет страницу с нужными данными. Этот процесс повторяется на последующих уровнях, пока не будет достигнут конечный уровень.

Например, если мы ищем в сбалансированном дереве рис. 4-2 значение City132, процесс запроса начинается на корневом уровне и просматриваются строки. Так как City132 находится между City121 и City190, SQL Server определяет, что City132 скорее всего находится на странице, начинающейся с City121. Далее SQL Server переходит на страницу промежуточного уровня, начинающуюся с City121. После просмотра страницы SQL Server снова решает,

что City132 находится между City121 и City150, поэтому переходит на страницу конечного уровня, начинающуюся с City121, и пытается найти City132 на ней. Это страница конечного уровня и искать больше нигде. Если бы City132 не было в таблице, SQL Server не нашел бы запись для City132. Прочитав запись City148, SQL Server «понимает», что ниже в таблице нет значения City132, и возвращает запрос без результатов. Обратите внимание, что в показанной здесь структуре серверу SQL Server надо прочитать максимум 3 страницы, чтобы определить местонахождение любого города в базе данных.

Вот что означает сбалансированное дерево. Выполняя поиск нужных данных, SQL Server проходит одно и то же число уровней в индексе и такое же число страниц индекса.

Уровни индекса

Число уровней в индексе и число страниц на каждом уровне индекса определяется просто. Размер страницы данных в SQL Server — 8192 байт, где может храниться до 8060 байт пользовательских данных. Зная число байт, необходимых для хранения ключа индекса, что определяется типом данных, легко простым делением вычислить число строк, уместящихся на странице.

Следующий пример иллюстрирует не только построение индекса, но и вычисление размера индекса. Сейчас мы увидим насколько ценными оказываются индексы, когда надо найти данные в очень больших таблицах, а также узнаем, почему время на поиск нужных данных несильно изменяется, даже если размер базы данных увеличивается очень значительно. Ясно, что время поиска нужных данных также зависит от грамотности написания запросов.

Если создавать индекс столбца данных типа *INT*, каждая строка в таблице занимать в индексе 4 байт.

Если таблица содержит только 1200 строк данных, требуется 4800 байт свободного пространства. Поскольку все записи помещаются на одной странице данных, индекс будет состоять из одной страницы, которая одновременно будет страницей корневого и конечного уровня. В действительности в таблице может располагаться даже 2015 строк, а для индекса все равно будет достаточно одной страницы.

Однако при добавлении 2016-ой строки записи больше не помещаются на одной странице, поэтому в индекс добавляется две дополнительные страницы — этот процесс называется *разбиение страниц*. Существующая корневая страница перемещается вниз иерархии и становится страницей конечного уровня. Половину данных страницы индекса SQL Server перемещает в одну из только что созданных страниц. Другая новая страница размещается вверху иерархии структуры индекса и становится новой корневой страницей. Завершающая операция — копирование первых записей каждой из страниц конечного уровня в только что созданную корневую страницу. У нас теперь одна корневая страница и две страницы конечного уровня. Этот индекс не нуждается в создании промежуточного уровня, потому что в корневой странице достаточно места для размещения всех значений из первых строк страниц конечного уровня. Теперь для нахождения любой строки в таблице надо просмотреть ровно две страницы в индексе.

ПРИМЕЧАНИЕ Разбиение страниц

Следует помнить, что строки на странице индекса упорядочены, поэтому SQL Server всегда вносит новые записи в «правильное» место при разбиении страниц. При этом строки могут перемещаться между страницами, а разбиение страниц может происходить на любом уровне структуры хранения.

Можно добавлять строки в таблицу, не меняя число уровней в индексе, пока число строк не достигнет 4 060 225. В этом случае в индексе 2015 страниц конечного уровня с 2015 записями в каждой. Корневая страница содержит 2015 записей — по одной первой строке каждой из страниц конечного уровня. Таким образом, чтобы найти любую строку среди этих 4 060 255 строк таблицы, SQL Server требуется прочитать ровно две страницы. В момент добавления 4 060 226-ой строки данных в таблицу нужно добавить еще одну страницу индекса на конечном уровне, но корневая страница не может держать 2016 записей, так как тогда она выйдет за пределы разрешенных 8060 байт. Поэтому SQL Server выполняет разбиение страниц. Страница корневого уровня становится страницей промежуточного уровня, где создается еще одна страница. Бывшая корневая страница подвергается разбиению, причем половина ее записей переходит к только что созданной второй странице промежуточного уровня, а первые записи обеих страниц промежуточного уровня записываются во вновь созданную корневую страницу.

В следующий раз SQL Server потребуется создать промежуточный уровень, когда потребуется добавить 8 181 353 376-ую строку данных в таблицу — 2015 строк корневой страницы, которым соответствует 2015 страниц промежуточного уровня, в каждой из которых в свою очередь содержится 2015 записей, соответствующих 2015 страницам на конечном уровне, и, собственно, еще одна строка данных, которая не помещается.

Как видите, такая структура позволяет SQL Server очень быстро находить строки в исключительно больших таблицах. В данном случае, чтобы найти строку в таблице, содержащей чуть более 4 миллионов строк, SQL Server нужно прочитать лишь две страницы данных, а прочитывать три страницы потребуется, только если размер таблицы перевалит за 8 миллиардов строк.

ПРИМЕЧАНИЕ На экзамене

В разреженных столбцах для обеспечения максимальной компактности и производительности следует создавать фильтруемые индексы.

Контрольные вопросы

1. Какая структура SQL Server используется для создания индексов?
2. Каковы три типа страниц индекса?

Ответы на контрольные вопросы

1. Для хранения индексов в SQL Server используется сбалансированное дерево.
2. Индекс может содержать страницы корневого, промежуточного и конечного уровня. В индексе всегда одна корневая страница, находящаяся вверху иерархии индекса. У индекса может (но не обязательно) быть один или более промежуточных уровней. Страницы конечного уровня находятся в самом низу иерархии индекса.

Резюме занятия

- SQL Server создает индекс, используя структуру, которая называется *сбалансированное дерево*.
- У каждого индекса единственная страница корневого уровня и, если все записи не помещаются на этой странице, в индексе могут создаваться страницы конечного и промежуточного уровней.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. В компании Fabrikam информация о продуктах хранится в такой таблице:

```
CREATE TABLE Products.Product
(ProductID          INT          IDENTITY(1,1),
 ProductName        VARCHAR(30)  NOT NULL,
 SKU                CHAR(8)      NOT NULL,
 Cost               MONEY        NOT NULL,
 ListPrice          MONEY        NOT NULL,
 ShortDescription   VARCHAR(200) NOT NULL,
 LongDescription    VARCHAR(MAX) NULL,
 CONSTRAINT pk_product PRIMARY KEY CLUSTERED (ProductID))
```

Таблица запрашивается по столбцу ProductID, ProductName или SKU. Приложение отображает ProductName, SKU, ListPrice, и ShortDescription. Помимо этого возвращается ProductID, чтобы можно было выполнять любые последующие операции. Недавно в таблицу внесли информацию о нескольких тысячах новых продуктов, и производительность сервера упала. Какой индекс нужно реализовать, чтобы обеспечить максимальную производительность обработки запросов?

- A. CREATE NONCLUSTERED INDEX idx_product ON Products.Product (ProductID, ProductName, SKU)
- B. CREATE NONCLUSTERED INDEX idx_product ON Products.Product (ProductName)
- C. CREATE NONCLUSTERED INDEX idx_product ON Products.Product (ProductName) INCLUDE (SKU, ListPrice, ShortDescription, ProductID)
- D. CREATE NONCLUSTERED INDEX idx_product ON Products.Product (ProductName, SKU, ProductID, ListPrice, ShortDescription)

Занятие 2. Проектирование индексов

Индексы позволяют максимально эффективно находить нужную информацию в огромных базах данных. Здесь вы узнаете, как создавать кластеризованные и некластеризованные индексы, а также какой тип индекса использовать в той или иной ситуации. Вы научитесь создавать фильтруемые индексы и индексы с включенными столбцами для расширения возможностей по обработке запро-

сов с использованием индексов. Наконец, вы узнаете, как создавать пространственные и XML-индексы, которые позволяют быстро находить информацию в XML-документах и приложениях, в которых используются пространственные данные.

Изучив материал этого занятия, вы сможете:

- ✓ создавать кластеризованные индексы;
- ✓ создавать некластеризованные индексы;
- ✓ рассказать об указателях перенаправления;
- ✓ создавать фильтруемые индексы;
- ✓ определять включенные столбцы в некластеризованном индексе;
- ✓ создавать индексы XML;
- ✓ создавать пространственные индексы.

Продолжительность занятия — 30 минут.

Кластеризованные индексы

Можно создавать индексы на основе одно или нескольких столбцов таблицы — такой индекс называется индексным ключом и у него есть ряд ограничений:

- индекс может охватывать не более 16 столбцов;
- максимальный размер индексного ключа — 900 байт.

Столбец (или столбцы) кластеризованного индекса называется ключом кластеризации (clustering key). Кластеризованный индекс оказывает особое влияние на SQL Server, так как заставляет его упорядочивать данные в таблице согласно ключу кластеризации. Поскольку таблица может упорядочиваться только одним способом, в ней можно задать лишь один кластеризованный индекс.

Кластеризованные индексы задают порядок сортировки данных в таблице. Однако кластеризованные индексы не обеспечивают порядок физической сортировки. Кластеризованный индекс не приводит к физическому упорядочению данных на диске, потому что это привело бы к большому числу операций дискового ввода-вывода при разбиении страниц. Он лишь гарантирует, что индексированная цепочка страниц упорядочена логически, что позволяет SQL Server при поиске данных переходить прямо по цепочке страниц. В процессе движения сервера SQL Server по индексированной цепочке страниц строки данных считываются в порядке ключа кластеризации.

Поскольку конечный уровень кластеризованного индекса представляет собой строку данных в таблице, пройдя кластеризованный индекс вниз до конечного уровня, SQL Server получает нужные данные. Никаких дополнительных операций чтения не нужно для поиска нужных данных.

В общем случае, у каждой таблицы должен быть кластеризованный индекс. Одна из основных целей создания кластеризованного индекса — устранение указателей перенаправления.

Указатели перенаправления

Таблица без кластеризованного индекса называется кучей. В ней цепочки страниц не хранятся в упорядоченном виде. SQL Server выделяет страницы и сохраняет данные по мере их записи в таблицу. На основе хранящихся данных строятся некластеризованные индексы, а на конечном уровне индексов содержится указатель на местоположение строки в страницах таблицы данных.

Если SQL Server переместит строку по причине последующих изменений, таких как разбиение страницы или невозможность уместить строку на странице данных, SQL Server не обновит некластеризованный индекс, задав в нем новое местоположение строки, а создаст на странице данных указатель перенаправления, указывающий на новое местоположение строки.

Когда в таблице немного указателей перенаправления, это не создает особых проблем, но большое их количество может вызвать значительное снижение производительности. Если бы указателей перенаправления не было, после просмотра некластеризованного индекса SQL Server пришлось бы выполнить лишь одну дополнительную операцию, чтобы получить данные из строки. Однако если указатели перенаправления есть, SQL Server приходится выполнять дополнительную операцию получения данных строки, на которую указывает указатель, после чего возвращаться, чтобы продолжить чтение цепочки страницы. В самых трудных случаях SQL Server приходится выполнять в 10-15 раз больше операций чтения, чем число строк, возвращенных в результате выполнения запроса.

Общий синтаксис создания реляционного индекса таков:

```
CREATE [ UNIQUE ] [ CLUSTERED | NONCLUSTERED ] INDEX имя_индекса
ON <объект> ( column [ ASC | DESC ] [ ,...n ] )
[ INCLUDE ( имя_столба [ ,...n ] ) ]
[ WHERE <предикат_фильтра> ]
[ WITH ( <параметры_реляционного_индекса> [ ,...n ] ) ]
[ ON { имя_схемы_секции ( имя_столба ) | имя_файловой_группы | default } ]
[ FILESTREAM_ON { имя_файловой_группы_filestream | имя_схемы_секции |
«NULL» } ][ ; ]
```

ПРИМЕЧАНИЕ Данные FILESTREAM

Предложение *FILESTREAM_ON* используется при создании кластеризованных индексов в таблице, содержащей данные *FILESTREAM*. Если в предложении *FILESTREAM_ON* задать файловую группу, отличную от той, где хранятся данные, в течение создания кластеризованного индекса все данные *FILESTREAM* будут перемещены в новую файловую группу.

Вы наверняка помните сценарии создания таблиц из главы 3, которые содержали ключевое слово «кластеризованный» в определении первичного ключа. Хотя первичный ключ и является ограничением, на физическом уровне SQL Server реализует первичный ключ как индекс. Поскольку по умолчанию первичный ключ создается кластеризованным, SQL Server создает для такого пер-

вичного ключа кластеризованный индекс. Аналогично уникальное ограничение на физическом уровне реализовано как уникальный индекс. Поскольку первичный ключ также по умолчанию уникален (если только не создан как некластеризованный), SQL Server на физическом уровне реализует каждый первичный ключ как уникальный, кластеризованный индекс.

Как мы уже обсуждали в главе 3, предложение *ON* определяет файловую группу, в которой создан индекс. Однако поскольку конечный уровень кластеризованного индекса является строкой данных таблицы, таблица и кластеризованный индекс всегда хранятся в одной файловой группе.

ПРИМЕЧАНИЕ Схемы секций

Подробнее о схемах секций см. главу 6.

Некластеризованные индексы

Другой тип реляционных индексов — некластеризованный индекс. Они не налагают никаких ограничений на упорядочение записей в таблице, поэтому в одной таблице можно создать много некластеризованных индексов. У таких индексов те же ограничения, что у кластеризованных: максимальный размер индексного ключа — 900 байт и максимум 16 столбцов. Кроме того, в таблице может быть не больше 1000 некластеризованных индексов.

Конечный уровень некластеризованного индекса содержит указатель на нужные данные. Если в таблице есть кластеризованный индекс, конечный уровень некластеризованного индекса указывает на ключ кластеризации. Если же кластеризованного индекса нет, страницы конечного уровня указывают на строки данных в таблице. В любом случае, при просмотре некластеризованного индекса до конечного уровня чтобы найти данные в строке таблицы, SQL Server приходится выполнять одну дополнительную операцию.

Поддержка индексов

На первый взгляд может показаться, что достаточно создать в таблице десятки или сотни индексов, чтобы «обслужить» любой возможный запрос. Индекс представляет собой структуру сбалансированного дерева, которая состоит из всех записей таблицы, соответствующих ключу индекса. Записи в индексе хранятся на страницах индексов согласно заданному для индекса порядку сортировки.

При добавлении в таблицу новой строки операция не завершается, пока SQL Server не добавит нужное значение этой новой строки в правильное место индекса. При каждой записи в таблицу SQL Server должен внести записи во все затронутые индексы.

Если в странице индекса конечного уровня нет места для нового значения, SQL Server должен выполнить разбиение страниц и записать половину строк из полной страницы во вновь созданную страницу. Если это также приводит к переполнению страницы промежуточного уровня, разбиение страницы выполняется и на этом уровне. Если новая строка вызывает переполнение корневой страницы, последняя разбивается, при этом создается новый промежуточный уровень и новая корневая страница.

Индексы позволяют повысить производительность запросов, но каждый создаваемый индекс снижает производительность всех операций *INSERT*,

UPDATE, *DELETE*, *BULK*, *INSERT* и *BCP*. Поэтому надо аккуратно рассчитывать число индексов, чтобы добиться оптимального времени выполнения операций. Есть эмпирическое правило: не следует без серьезных причин создавать более 5 индексов в таблице, предназначенной для OLTP-операций. У таблиц, предназначенных для операций чтения или обслуживания запросов в хранилищах данных запросов обычно много индексов, потому что запись в хранилище данных выполняется настроенными администратором пакетными заданиями в часы невысокой нагрузки.

Индекс для покрытия запросов

При создании индекса в него включаются все значения ключей индекса. В результате индекс представляет собой мини-таблицу, содержащую все значения, соответствующие только столбцам в ключах индекса. Поэтому возможно, что запрос можно выполнить, используя только данные индекса. Индекс, конструкция которого позволяет SQL Server выполнять запросы, используя только информацию индекса, называют *индексом для покрытия запросов* (covering index).

Создав такой индекс для данных, к которым часто обращаются, можно сократить время выполнения запросов за счет исключения дополнительных операций чтений основной таблицы. В принципе, можно организовать параллельное выполнение запросов на основе только данных индекса и операций внесения изменений в основную таблицу при условии, что те не влияют на индекс.

SQL Server также может использовать для выполнения запроса несколько индексов. Если у двух индексов по крайней мере один общий столбец, SQL Server может при выполнении запроса применить к этим индексам операцию соединения.

Включенные столбцы

Индексы — замечательный инструмент для быстрого выполнения запросов. Но еще лучше в этом смысле индексы для покрытия запросов. Однако мы ограничены 16-ю столбцами и 900 байтами для ключа индекса. Эти ограничения фактически исключают столбцы с объемными типами данных, которые было бы удобно разместить в индексе покрытия запросов, чтобы при их выполнении не нужно было извлекать данные из основной таблицы.

Индексы могут создаваться средствами дополнительного предложения *INCLUDE*. Включенные столбцы становятся частью индекса только на конечном уровне. Значения включенных столбцов не размещаются на корневом или промежуточном уровнях индекса и не подпадают под ограничение в 900 байт. Таким образом, используя предложение *INCLUDE*, можно создавать индексы для покрытия запросов, состоящие из более 16 столбцов и 900 байт.

Статистика распределения

Компонент, отвечающий за определение, нужно ли вообще использовать индекс при выполнении того или иного запроса, называют *оптимизатором запросов* (query optimizer). Он принимает решение, основываясь на статистике распределения индекса.

При создании индекса SQL Server генерирует структуру, называемую гистограммой — она хранит информацию об относительном распределении значений данных в столбце. Степень, до какой значения в столбце позволяют находить

маленькие наборы данных, называется избирательностью индекса. По мере роста числа уникальных значений в столбце растет избирательность индекса. При выполнении запросов оптимизатор запросов выбирает самые избирательные индексы, потому что высокоизбирательный индекс позволяет обработчику запросов сразу отсечь значительную часть таблицы и при выполнении запроса работать с меньшими объемами данных. Индексы с низкой избирательностью и низкой долей уникальных значений оптимизатором запроса игнорируются, но создают дополнительную нагрузку на операции записи.

Фильтруемые индексы

Ключ индекса может содержать перекося в значениях данных, в которых большая доля таблицы содержит дублирующиеся значения, находящиеся в пределах узкого диапазона значений. После выполнения запроса, получающего данные из очень избирательной части таблицы, очень возможно, что последующие запросы, выполняемые по отношению к части с низкой избирательностью, будут использовать тот же индекс, но это явно неуместно.

Для обработки случаев с существенным перекосом в данных в SQL Server 2008 существуют фильтруемые индексы. *Фильтруемый индекс* (filtered index) — это просто индекс с предложением *WHERE*. В индекс добавляются только ключи, удовлетворяющие предложению *WHERE*, что позволяет создавать индексы, ориентированные на высокоизбирательные части таблицы, позволяя SQL Server использовать другие методы в случае менее избирательного диапазона.

У фильтруемых индексов есть ряд ограничений:

- они должны представлять собой некластеризованные индексы;
- их нельзя создавать в вычисляемых столбцах;
- столбцы не могут подвергаться явному или неявному преобразованию типов данных.

Параметры индексов

В процессе создания индекса можно задать несколько параметров. Наиболее важный из них — *FILLFACTOR*. Когда страница индексов полна и SQL Server пытается внести новую запись в страницу, должно произойти разбиение страницы. Результат разбиения страницы — две наполовину заполненные страницы индексов. Если разбиения страниц индекса происходят часто, может образоваться большое количество страниц индексов, содержащих частичный набор данных. Аналогично файлам на диске, индексы фрагментируются из-за разбиения страниц. Для поиска информации с использованием сильно фрагментированных индексов требуется больше операций чтения.

Для управления разбиением страниц служит *коэффициент заполнения* (fill factor) индекса, *FILLFACTOR*, определяющий долю свободного пространства, которая должна оставаться на уровне страницы конечного уровня в процессе создания или перестроения индекса. Оставляя побольше свободного пространства на уровне листа, можно обеспечить разбиение страниц при небольшом количестве строк на странице конечного уровня, что позволит замедлить скорость фрагментации индекса.

FILLFACTOR применяется только к странице индекса конечного уровня. Страницы промежуточного уровня (если он есть) и корневая страница запол-

няются почти «до упора». На таких страницах SQL Server резервирует не более одной строки. Однако если планируется создать большое число страниц конечного уровня, что, в свою очередь, вызовет много разбиений страниц промежуточного и, возможно, корневого уровня, можете задействовать параметр *PAD_INDEX*, который распространяет действие *FILLFACTOR* на страницы промежуточного и корневого уровня.

В процессе создания индекса считываются все значения данных индексного ключа. После этого SQL Server создает ряд внутренних рабочих таблиц для сортировки значений перед построением структуры сбалансированного дерева. По умолчанию эти таблицы создаются в той же базе данных, что и индекс. Если вы не хотите занимать пространство базы данных, где находится индекс, можете задать параметр *SORT_IN_TEMPDB*, который вынудит сервер создать эти рабочие таблицы в базе данных tempdb.

Как кластеризованные, так и некластеризованные индексы могут определяться как уникальные. В таких индексах нельзя дублировать записи. Если попытаться вставить дублирующее значение, сервер вернет ошибку и отменит транзакцию. По умолчанию откатываются многостроковые транзакции даже если дублируется хотя бы одна строка. Чтобы разрешить вставку любых строк многостроковой транзакции, отклоняя только те, что приводят к созданию дубликатов, можно определить параметр *IGNORE_DUP_KEY*. В этом случае при попытке вставки дублирующих строк сервер выдает предупреждение и отбрасывает такие строки.

Создание оперативных индексов

При построении индекса считываются и используются для построения все значения в ключе индекса. Чтение всех значений и построение индекса не происходит моментально, и за это время данные в ключе могут измениться. SQL Server контролирует изменения данных в таблице, гарантируя согласованность данных в процессе построения индекса в соответствии с заданными параметрами. Индексы делятся на *оперативные* (online) и *автономные* (offline). При создании индекса с использованием команды *WITH ONLINE = OFF* сервер SQL Server блокирует всю таблицу, запрещая любые изменения до завершения процесса создания индекса. Если же задействовать команду *WITH ONLINE = ON*, SQL Server разрешит вносить изменения в таблицу в процессе создания индекса, храня информацию о версии в базе данных tempdb.

Управляют созданием индекса с использованием одной из указанных выше команд. Значение по умолчанию — *WITH ONLINE = OFF*. При создании кластеризованного индекса в автономном режиме таблица блокируется и запрещены операции *SELECT* или изменения данных. При создании некластеризованного индекса в автономном режиме применяется общая блокировка таблицы, позволяющая только операцию *SELECT*, но не изменение данных.

В процессе создания оперативного индекса в основной таблице данных или представлении разрешены операции запроса и изменения данных. При создании индекса в оперативном режиме функциональность поддержки версий строк в SQL Server, 2005 гарантирует отсутствие конфликтов с другими операциями, выполняемыми в таблице. Оперативный режим создания индексов доступен только в выпуске SQL Server 2008 Enterprise Edition.

ПРИМЕЧАНИЕ На экзамене

Такие операции, как создание или перестроение оперативного индекса или оперативное восстановление доступны только в выпуске SQL Server 2008 Enterprise Edition.

XML-индексы

Тип данных XML позволяет хранить в столбце до 2 гигабайт данных. Хотя структура XML-данных позволяет выполнять запросы этих данных, при поиске информации в XML-документе SQL Server должен просмотреть его структуру. Чтобы повысить производительность запросов XML-данных, можно создать особый тип индекса, который называется XML-индексом.

Есть два типа XML-индексов: первичный и вторичный.

Первичный XML-индекс (primary XML index) строится на основе всех узлов в XML-столбце. Первичный XML-индекс привязывается к таблице ссылкой на соответствующую строку в кластеризованном индексе. Это означает, что на момент создания первичного XML-индекса должен существовать кластеризованный индекс.

После создания первичного XML-индекса, можно создавать дополнительные вторичные индексы. Вторичные индексы могут создаваться на основе параметров *PATH*, *VALUE* или *PROPERTY*. Необходимость первичного XML-индекса обусловлена тем, что вторичные XML-индексы создаются на основе данных, содержащихся в первичном XML-индексе.

Вторичные XML-индексы, созданные в режиме *FOR PATH* строятся на основе значений *PATH* и *NODE* первичного XML-индекса. XML-индекс *PATH* используется для оптимизации запросов поиска пути в XML-документе. Индексы, созданные в режиме *FOR VALUE*, строятся на основе параметров *PATH* и *VALUE* первичного XML-индекса и служат для поиска значений в XML-документах. Индексы, созданные в режиме *FOR PROPERTY*, строятся на основе первичного ключа и параметров *PATH* и *NODE*. XML-индексы на основе *PROPERTY* служат для эффективного поиска данных в XML-столбце наряду с дополнительными столбцами таблицы.

Универсальный синтаксис создания XML-индекса таков:

```
CREATE [ PRIMARY ] XML INDEX имя_индекса
ON <объект> ( имя_XML_столбца )
[ USING XML INDEX имя_XML_индекса
  [ FOR { VALUE | PATH | PROPERTY } ] ]
[ WITH ( <параметры_XML_индекса> [ ,...n ] ) ][ ; ]
```

Пространственные индексы

Пространственные индексы создаются на основе столбцов пространственных данных, относящихся к геометрическим или географическим типам.

```
CREATE SPATIAL INDEX имя_индекса
ON <объект> ( имя_столбца_пространственных_данных )
[ USING <тесселяция_геометрической_решетки> ]
WITH ( <ограничивающий_прямоугольник>
```



```

        [ [,] <параметры_тесселяции> [ ,...n ] ]
        [ [,] <параметры_пространственного_индекса> [ ,...n ] ] )
    | [ USING <geography_grid_tessellation> ]
        [ WITH ( [ <тесселяция_геометрической_решетки> [ ,...n ] ]
        [ [,] <параметры_пространственного_индекса>
        [ ,...n ] ] ) ) ]
    } [ ON { filegroup_name | «default» } ];

```

Пространственные данные определяются с использованием двумерной системы координат. Индексы строятся на основе сбалансированных деревьев, которые представляют собой одномерную структуру. Поэтому чтобы проиндексировать пространственные данные, SQL Server должен преобразовать двумерное пространство пространственных данных в линейную цепочку. Процесс такого преобразования называется *тесселяцией* (tessellation).

Индексируя географический тип данных, SQL Server сопоставляет эллипсоидальные данные двумерному, неевклидовому пространству. Поверхность эллипсоида делится на полушария, после чего каждое полушарие проецируется на четырехугольную пирамиду. Далее пирамиды проецируются на двумерную плоскость. Затем плоскости, представляющие верхнее и нижнее полушария, сшиваются по краю. Заключительный процесс индексации географических данных — выполнение тесселяции.

Перед тесселяцией SQL Server создает однородную четырехуровневую иерархическую декомпозицию представляемого пространства. Уровень 1 — верхний уровень иерархии. Уровень 2 содержит декомпозицию каждой ячейки сетки Уровня 1 в сетку равной размерности. Уровень 3 содержит декомпозицию каждой ячейки Уровня 2 в сетку равной размерности. Аналогично уровень 4 содержит декомпозицию каждой ячейки Уровня 3 в сетку равного измерения (рис. 4-3).

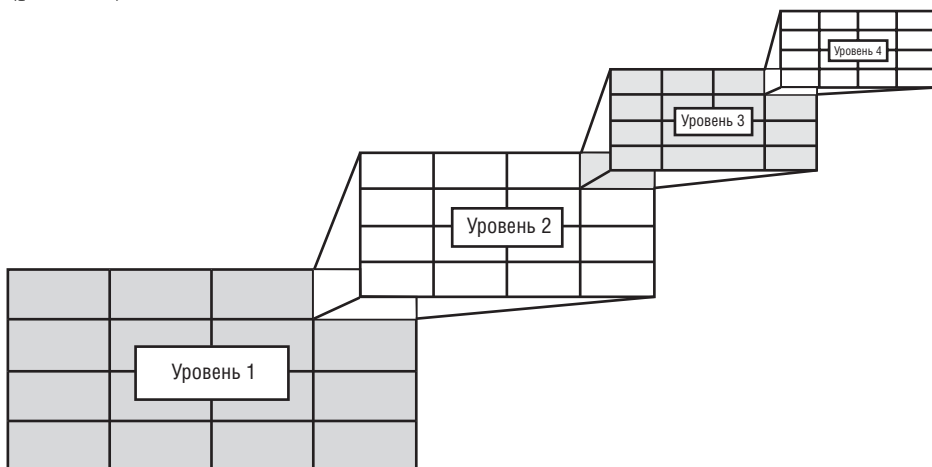


Рис. 4-3. Четырехуровневая однородная сеточная иерархия

Ячейки в сеточной иерархии нумеруются в линейном порядке с использованием варианта заполнения пространства кривой Гильберта.

Тесселяция

После создания четырехуровневой сеточной иерархии последовательно считываются все строки пространственных данных в сетке. Начиная с уровня 1, процесс тесселяции строит график пространственного объекта в наборе ячеек сетки, которых касается этот объект. Набор этих ячеек после этого записывается в индекс.

Очень маленькие объекты касаются лишь небольшого числа ячеек в сеточной иерархии, тогда как крупные объекты касаются очень большого числа ячеек. Чтобы ограничить размер индекса без снижения его точности, при тесселяции применяется ряд правил определения конечного результата, записываемого в индекс:

- **Правило покрытия** Если объект полностью покрывает ячейку, говорят, что эта ячейка накрыта объектом. Накрытая ячейка считается и не подвергается тесселяции. Это правило применяется на всех уровнях сеточной иерархии.
- **Правило ячеек на объект** Это правило вводит ограничение ячеек на объект (*CELLS_PER_OBJECT*), определяющее максимальное число ячеек, которые можно посчитать для каждого объекта (за исключением уровня 1). На более низких уровнях правило числа ячеек на объект определяет объем данных, которые можно записать об объекте.
- **Правило самой глубокой ячейки** Создает наилучшее приближение для объекта, записывая только самые нижние ячейки, созданные при тесселяции объекта. Родительские ячейки не включаются в счетчик ячеек на объект и не записываются в индекс.

Если ячейка не попала в тесселяцию, никакой информации о ней и для всех последующих уровней сеточной иерархии не попадает в индекс. Если требование правила ячеек на объект не превышено и объект полностью не покрывает ячейку, она попадает в тесселяцию. Когда ячейка попадает в тесселяцию, часть объекта, содержавшегося в ячейке, строится в сетке на следующем уровне иерархии, где к ячейкам этой сетки применяются правила тесселяции.

Процесс тесселяции выполняется по отношению к каждой ячейке в сетке, а затем на каждом последующем уровне сеточной иерархии. По достижении предела ячеек на объект или Уровня 4 иерархии тесселяция завершается. Ячейки, попавшие в тесселяцию на самом низком уровне, определяют ключ строки в пространственном индексе.

Ограничивающий прямоугольник

При индексации геометрических данных надо определить еще один дополнительный параметр пространственного индекса — *ограничивающий прямоугольник* (bounding box). Сбалансированное дерево — конечная линейная структура, у которой есть четко определенное начало и конец. Поскольку геометрическая плоскость бесконечна, невозможно определить сбалансированное дерево на всем двумерном пространстве. Параметр *BOUNDING_BOX* позволяет задать максимальную и минимальную точки по координатным осям X и Y, которые используются при построении иерархии сеток и тесселяции строк геометрических данных.

Все объекты или части объекта, выходящие за пределы ограничивающего прямоугольника, не учитываются при расчете индекса. Выбирать пределы огра-

ничающего прямоугольника следует так, чтобы в него попадало большинство объектов, которые надо проиндексировать в таблице.

ПРИМЕЧАНИЕ Пространственные индексы

Подробнее о пространственных индексах и процессе тесселяции см. статью на странице <http://technet.microsoft.com/ru-ru/library/bb964712.aspx>.

Контрольные вопросы

1. Каково различие между кластеризованным и некластеризованным индексами?
2. Как параметр *FILLFACTOR* влияет на процесс построения индекса?

Ответы на контрольные вопросы

1. Кластеризованный индекс заставляет SQL Server упорядочивать данные в таблице согласно ключу кластеризации. Некластеризованный индекс такого требования не налагает.
2. Параметр *FILLFACTOR* резервирует свободное пространство на страницах промежуточного и конечного уровней.

Практикум. Создание индексов

Вы создадите индексы в базе данных AdventureWorks.

1. Выполните следующий код, чтобы создать некластеризованный индекс в таблице Person.Address:

```
CREATE NONCLUSTERED INDEX idx_city ON Person.Address(City) INCLUDE (AddressLine1)
```

2. Выполните следующий код, чтобы создать фильтруемый индекс в таблице Person.Address:

```
CREATE NONCLUSTERED INDEX idx_city2 ON Person.Address(City)
INCLUDE (AddressLine1, AddressLine2)
WHERE AddressLine2 IS NOT NULL
```

3. Выполните следующий код, чтобы создать пространственный индекс в таблице Person.Address:

```
CREATE SPATIAL INDEX sidx_spatiallocation
ON Person.Address(Spatiallocation)
USING GEOGRAPHY_GRID
WITH (GRIDS = (MEDIUM, LOW, MEDIUM, HIGH ),
CELLS_PER_OBJECT = 64);
```

Резюме занятия

- Кластеризованные индексы определяют порядок сортировки страниц данных в таблице.
- В таблице можно создать не более 1000 некластеризованных индексов.

- Некластеризованные индексы могут содержать данные столбцов на конечном уровне индекса, чтобы обеспечить выполнение большего числа запросов.
- К некластеризованному индексу можно применить предложение *WHERE*, ограничивающее набор данных, на основании которых строится индекс.
- Существуют три типа XML-индексов; *PATH*, *VALUE* и *PROPERTY*.
- Пространственные индексы определяются для геометрических или географических типов данных.
- При индексировании геометрических данных параметр *BOUNDING_BOX* служит для ограничения размеров двумерной плоскости.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Вы администратор базы данных в компании, торгующей в розницу заготовками и комплектам инструментов для мастеров, делающих эксклюзивные ручки для письма. Вы разрабатываете базу данных, где будет храниться информация о характеристиках предлагаемых товаров. У каждого товара свой набор характеристик, который может отличаться у разных товаров. Вы планируете стратегию индексации базы данных. Самый популярный запрос выглядит так:

```
SELECT a.ProductName, b.ProductType, b.WoodSpecies, b.Color
FROM Products a INNER JOIN ProductAttributes b ON a.ProductID = b.ProductID
WHERE b.Color = «X»
```

Не у всех товаров есть атрибут цвета Color. Какая стратегия индексации будет максимально эффективной?

- A. Некластеризованный индекс по атрибуту Color.
- B. Некластеризованный индекс по атрибуту Color, содержащий столбцы ProductType и WoodSpecies.
- C. Фильтруемый, некластеризованный индекс по атрибуту Color.
- D. Фильтруемый, некластеризованный индекс по атрибуту Color, содержащий столбцы ProductType и WoodSpecies.

Занятие 3. Поддержка индексов

Со временем изменения в данных приведут к фрагментации индексов. Чтобы обеспечить максимальную эффективность операций запроса данных, нужно добиваться минимизации фрагментации. На этом занятии вы узнаете, как управлять уровнем фрагментации, а также как избавиться от фрагментации индекса.

Изучив материал этого занятия, вы сможете:

- ✓ перестраивать индексы для устранения фрагментации;
- ✓ отключать индекс.

Продолжительность занятия — 30 минут.

Управление и обслуживание индексов

Данные индекса хранятся в упорядоченном виде, поэтому через какое-то время, значения могут перемещаться в пределах индекса из-за разбиений страницы или изменения значений. Чтобы контролировать фрагментацию индекса со временем, нужно периодически предпринимать профилактические меры.

Фрагментация индексов

Файлы операционной системы обычно со временем фрагментируются из-за многократных операций записи. Индексы тоже могут становиться фрагментированными, но фрагментация индексов отличается от фрагментации файлов.

При создании индекса все значения ключа индекса записываются в упорядоченном виде на страницах индекса. При удалении строки из таблицы SQL Server должен удалить соответствующую запись в индексе, что создает «дырку» на странице индекса. SQL Server не возвращает освобожденное пространство из-за слишком высокой стоимости операций обнаружения и повторного использования «дырки» в индексе. Если значение в базовой таблице изменяется, SQL Server перемещает запись с указателем в другое место, что оставляет еще одну «дырку». При переполнении страниц индексов и потребности разбиения страниц снова происходит фрагментация индекса. Со временем индексы таблицы, в которой происходит много изменений данных, становятся фрагментированными.

Для управления степенью фрагментации индекса обычно используют параметр, который называется *коэффициентом заполнения* (fill factor). Для устранения фрагментации можно также задействовать инструкцию *ALTER INDEX*.

Параметр FillFactor

Этот параметр индекса определяет долю свободного пространства, которое резервируется на каждой странице конечного уровня при создании или перестроении индекса. Зарезервированное пространство позволяет в дальнейшем размещать дополнительные значения, снижая таким образом число разбиений страниц. Коэффициент заполнения измеряется в целых процентах, например значение 75 означает, что каждая создаваемая страница конечного уровня должна содержать 25% свободного пространства для размещения будущих значений.

Дефрагментация индексов

Поскольку SQL Server не возвращает пространство в систему, надо периодически освобождать пустое пространство в индексе, чтобы сохранить тот выигрыш в производительности, из-за которого индекс изначально создавался. Для дефрагментации индексов используют инструкцию *ALTER INDEX*.

```
ALTER INDEX { имя_индекса | ALL }  
ON <объект>  
{ REBUILD  
  [ [ WITH ( <параметры_перестроения_индекса> [ ,...n ] ) ]  
    | [ PARTITION = partition_number  
      [ WITH ( <параметры_перестроения_отдельной_секции_индекса>  
        [ ,...n ] ) ] ] ]  
| DISABLE | REORGANIZE  
  [ PARTITION = номер_секции ]  
  [ WITH ( LOB_COMPACTION = { ON | OFF } ) ]  
| SET ( <параметры_определения_индекса> [ ,...n ] ) } [ ; ]
```

При дефрагментации индексов можно выбрать параметры *REBUILD* или *REORGANIZE*.

Первый параметр перестраивает все уровни индекса и заполняет страницы в соответствии с параметром *FILLFACTOR*. При перестроении кластеризованного индекса перестраивается только он, однако если задать параметр *ALL*, будет перестроен как кластеризованный индекс, так и все некластеризованные индексы таблицы. Перестроение индекса обновляет всю структуру сбалансированного дерева, поэтому, если не задан параметр *ONLINE*, таблица блокируется и до завершения перестроения никакие изменения данных невозможны.

Параметр *REORGANIZE* устраняет фрагментацию только на конечном уровне. Страницы промежуточного уровня и корневая страница не дефрагментируются. Операция *REORGANIZE* всегда выполняется в оперативном режиме, поэтому не вызывает долгосрочной блокировки таблицы.

Отключение индекса

Индекс можно отключить инструкцией *ALTER INDEX*:

```
ALTER INDEX { имя_индекса | ALL }  
ON <объект>  
DISABLE [ ; ]
```

Определение отключенного индекса остается в системном каталоге, но индекс не используется. SQL Server не обновляет индекс при изменении данных в таблице и не использует при выполнении запросов. При отключении кластеризованного индекса становится недоступной вся таблица.

Чтобы включить индекс, его нужно перестроить для восстановления и заполнения структуры сбалансированного дерева. Это можно сделать следующей командой:

```
ALTER INDEX { имя_индекса | ALL }  
ON <объект>  
REBUILD [ ; ]
```

Контрольные вопросы

1. Каково различие между параметрами *REBUILD* и *REORGANIZE* команды *ALTER INDEX*?
2. Что происходит при отключении индекса?

Ответы на контрольные вопросы

1. Параметр *REBUILD* инициирует дефрагментацию всех уровней индекса, а *REORGANIZE* — только конечного уровня.
2. Отключенный индекс не используется оптимизатором, а при изменении данных в таблице индекс не обновляется.

Практикум. Поддержка индексов

Вы выполните дефрагментацию, отключите и вновь включите индексы.

1. Перестройте все индексы в таблице `Person.Address` следующим запросом:

```
ALTER INDEX ALL  
ON Person.Address  
REBUILD
```
2. Перестройте конечный уровень индекса таблицы `Person.Address` запросом:

```
ALTER INDEX IX_Person_LastName_FirstName_MiddleName  
ON Person.Person  
REORGANIZE
```
3. Отключите кластеризованный индекс в таблице `Person.Address`:

```
ALTER INDEX PK_Address_AddressID  
ON Person.Address  
DISABLE
```
4. Убедитесь, что таблица `Person.Address` недоступна, выполнив запрос:

```
SELECT * FROM Person.Address
```
5. Обратно включите кластеризованный индекс в таблице `Person.Address`:

```
ALTER INDEX PK_Address_AddressID  
ON Person.Address  
REBUILD
```
6. Убедитесь в доступности таблицы `Person.Address`, выполнив запрос:

```
SELECT * FROM Person.Address
```

Резюме занятия

- Выполнять дефрагментацию индексов можно в одном из двух режимов в зависимости от выбранного параметра — *REBUILD* или *REORGANIZE*.
- Параметр *REBUILD* обеспечивает перестроение всех уровней индекса. Если не задан параметр *ONLINE*, параметр *REBUILD* вызывает общую блокировку таблицы и она становится недоступной для каких-либо изменений данных.
- Параметр *REORGANIZE* обеспечивает перестроение только конечного уровня индекса и не вызывает блокировку таблицы.
- Можно отключить индекс, тогда его перестанет использовать оптимизатор, и индекс не будет обновляться при изменении данных в таблице. При отключении кластеризованного индекса становится недоступной вся таблица.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Вы отвечаете за разработку процесса ежемесячной загрузки приблизительно 150 Гбайт данных в хранилище данных предприятия. В каждой таблице хранилища данных есть не менее 8 индексов, служащих для повышения производительности операций анализа данных. Вы хотите, чтобы данные максимально быстро загружались непосредственно в таблицы. Какая операция обеспечит максимальную производительность при минимальных усилиях со стороны администратора?
 - A. Использование команды *BULK INSERT*.
 - B. Удаление и перестроение индексов.
 - C. Отключение и включение индексов.
 - D. Использование служб Integration Services для импорта данных.

Закрепление материала главы

Для закрепления знаний и навыков, полученных при изучении этой главы, выполните следующее:

- изучите резюме главы;
- повторите основные термины, использованные в этой главе;
- выполните задания по темам этой главы;
- сделайте предложенные упражнения;
- пройдите пробный экзамен.

Резюме главы

- В таблице можно создать не более одного кластеризованного индекса, который к тому же задает порядок сортировки данных в таблице.
- В таблице можно создавать до 1000 некластеризованных индексов. Некластеризованные индексы могут быть фильтруемыми и содержать дополнительные столбцы на конечном уровне индекса.
- Пространственный индекс создается путем проецирования пространственных объектов на четырехуровневую однородную сетку и применения правил тесселяции.
- Выполнять дефрагментацию индексов можно в одном из двух режимов в зависимости от выбранного параметра — *REBUILD* или *REORGANIZE*.

Основные термины

Понятны ли вам значения приведенных здесь основных терминов? Чтобы проверить свои ответы, загляните в глоссарий на прилагаемом компакт-диске.

- Сбалансированное дерево ~ balanced tree (B-tree).

- Кластеризованный индекс ~ clustered index.
- Индекс для покрытия запросов ~ covering index.
- Некластеризованный индекс ~ nonclustered index.
- Тесселяция ~ tessellation.

Упражнения

Сейчас вы примените на практике знания и навыки, полученные при изучении этой главы. Ответы на вопросы см. в разделе «Ответы» в конце книги.

Упражнение 1. Управление данными

Компания Wide World Importers собирается внедрять новый набор приложений для управления несколькими направлениями деятельности. Нужно обеспечить возможность хранить большой объем данных в корпоративном вычислительном центре и доступ к ним из любой точки земного шара.

Нескольким менеджерам требуется доступ к оперативным отчетам о текущей нагрузке своих сотрудников, а также о новых и незавершенных заказах клиентов. Еще им необходим доступ к большим объемам архивных данных для определения тенденций, оптимизации штата и уровня складских запасов.

Менеджеры хотят разместить руководства по пользованию продуктами на веб-сайте компании, а не поставлять их в составе самих продуктов. Пользователям нужно предоставить возможность просматривать руководства по названию продуктов и искать руководства по определенному тексту в них. Менеджеры по продажам также требуют усовершенствовать веб-сайт компании: описания продуктов должны быть доступны на нескольких языках с возможностью поиска.

В отделе продаж работает большое количество менеджеров по продажам, которые постоянно звонят клиентам, и им нужен доступ к данным по клиентам, которых обслуживает тот или иной менеджер, а также к информации о потенциальных клиентах. Доступ к информации менеджерам по продажам нужен даже в отсутствие подключения к Интернету или корпоративной сети. Периодически менеджеры по продажам подключаются к корпоративной сети и синхронизируют свои данные с корпоративной базой данных.

Некоторые из основных таблиц в пределах базы данных перечислены в Таблице 4-1.

Табл. 4-1. База данных компании Wide World Importers

Таблица	Описание
Customer	Содержит имя клиента, а также информацию о кредитной линии, номер счета, имя и пароль входа на веб-сайт
CustomerAddress	Содержит один или несколько адресов каждого клиента. Адрес может содержать до трех строк помимо названия города, штата и страны, почтового кода и широты и долготы. Обязательно наличие по крайней мере одной строки адреса, а также названия города и страны

Табл. 4-1. (окончание)

Таблица	Описание
CustomerContact	Содержит одну или более строк на каждого клиента, содержащих такую контактную информацию, как телефон, сотовый телефон, адрес электронной почты и номер факса
SalesPerson	Содержит список сотрудников отдела продаж с указанием территории, за которую каждый из них отвечает, процента комиссии и квоты продаж
Product	Содержит идентификатор продукта, артикул, запас продукта на складе, минимальный объем складского хранения, цену и нормативную цену
CustomerOrder	Содержит информацию о заказах клиентов, в том числе имя сотрудника отдела продаж, дату заказа, флажок, указывающий, отправлен ли заказ, и общую сумму заказа
CustomerOrder-Detail	Содержит строку для каждого заказа
CustomerSales-Person	Связывает клиентов и сотрудников отдела продаж

Средствами Microsoft Visual Studio .NET был разработан ряд приложений для работы в среде Microsoft Windows, а весь доступ к данным осуществляется через хранимые процедуры. Набор приложений одинаков как у пользователей, подключенных напрямую к корпоративному серверу баз данных, так и у менеджеров по продажам, подключающихся к своим локальным серверам баз данных.

Вот перечень самых популярных запросов:

- поиск клиентов по имени, городу или сотруднику отдела продаж;
- поиск клиентов, заказавших определенный продукт;
- поиск всех еще не отправленных заказов;
- поиск адреса доставки клиенту;
- поиск всех продуктов, заказанных в данном месяце;
- поиск всех клиентов, находящихся на данном расстоянии от текущего местоположения сотрудника отдела продаж.

Что нужно сделать, чтобы обеспечить высокую производительность запросов данных в БД?

Рекомендуемые упражнения

Чтобы подготовиться к экзамену по темам этой главы, выполните следующие задания.

Создание индексов

- **Задание 1.** В своей базе данных создайте первичные ключи во всех таблицах, в которых их еще нет.
- **Задание 2.** Найдите все таблицы в своей базе данных, у которых нет кластеризованного индекса, и создайте его или измените кластеризуемый первичный ключ.

- **Задание 3.** Создайте индексы для покрытия часто выполняемых запросов, потребности которых индексы сейчас удовлетворяют не полностью.
- **Задание 4.** Преобразуйте индекс, который используется для обслуживания запросов, обращающихся только к подмножеству строк, в фильтрованный индекс, чтобы обеспечить повышение производительности.

Пробный экзамен

На прилагаемом компакт-диске содержится пробный экзамен, обеспечивающий широкие возможности самоподготовки. Например, можно проверить, как усвоены темы этой главы либо все темы экзамена 70-432. Можно симитировать настоящий экзамен или выбрать режим обучения, когда после каждого вопроса приводятся правильные ответы с пояснениями.

ПРИМЕЧАНИЕ Пробный экзамен

Подробнее о пробном экзамене см. «Введение».

Полнотекстовое индексирование

Занятие 1. Создание и заполнение полнотекстовых индексов	102
Занятие 2. Запросы полнотекстовых данных	109
Занятие 3. Управление полнотекстовыми индексами	115

В этой главе рассказывается, как создавать и настраивать полнотекстовые индексы, что позволит приложениям эффективно работать с неструктурированными данными, хранящимися в столбцах *FILESTREAM*, *XML* или больших символьных полях.

Темы экзамена

- Установка SQL Server 2008 и связанных с ним служб.
- Настройка дополнительных компонентов SQL Server.
- Настройка полнотекстового индексирования.
- Обслуживание индексов.

Прежде всего

Для изучения материалов этой главы необходимо иметь:

- подключенную базу данных AdventureWorks.

Практический совет

Майк Хотек

Одна из компаний, с которой я работал, продавала большое количество сложной продукции. Каждый продукт поставлялся с одним или несколькими руководствами, которые могли состоять из тысяч страниц и нескольких томов. В целях сокращения затрат на бумагу и принтерный тонер в 2002 году компания отказалась от поставки документов на бумаге.

Руководства разрабатывались средствами Microsoft Office Word, а затем преобразовывались в формат PDF и размещались на веб-сайте компании, откуда их могли получить конечные пользователи. Пользователей ранее вполне устраивали указатели терминов, имеющиеся в конце бумажных руководств, но теперь они ожидают возможности выполнять контекстный поиск по всем руководствам, размещенным на веб-сайте компании.

Для решения этой проблемы компания обратилась в консалтинговую фирму, чтобы та создала поисковые индексы руководств. После двух месяцев разработки прототипа программы индексирования стоимостью свыше 40 тыс. долларов и содержавшего индексы 15 руководств, была определена общая стоимости проекта по индексированию всех руководств — 8 000 000 долларов, причем на поддержку программы и индексирование новых руководств и изменений в существующих пришлось бы еще тратить по миллиону долларов ежегодно. Мы узнали об этом проекте на общекорпоративном совещании.

Без особого шума мы создали простую базу данных, использующую преимущества файловых потоков *FILESTREAM* в Microsoft SQL Server 2008. Используя API-функции Win32, мы за выходные загрузили свыше 20 тыс. документов Word в столбец *VARBINARY*, предназначенный для *FILESTREAM*. Затем мы создали и заполнили полнотекстовый индекс по всем документам. На загрузку документов и создание полнотекстовых индексов ушло около 32 часов. Кроме этого, один из наших ASP-разработчиков набросал прототип страницы, позволявшей выполнять поиск из браузера.

В понедельник утром мы назначили встречу с командой, работающей над проектом по индексированию документов, и представили им наше решение. Начальство было поражено не столько тем, что за выходные мы умудрились проиндексировать все руководства, сколько качеством нашего поиска — наше решение оказалось значительно лучше «консалтингового», было практически готово к применению, при любых изменениях обновление выполнялось автоматически. После встречи мы за пару часов развернули функцию поиска руководств на веб-сайте компании, сэкономив тем самым более 8 миллионов долларов, не говоря уже о миллионе долларов ежегодно, запрашиваемом консалтинговой фирмой.

Занятие 1. Создание и заполнение полнотекстовых индексов

SQL Server 2008 позволяет создавать индексы, с помощью которых можно быстро выполнять запросы в больших объемах неструктурированных данных. На этом занятии рассказывается, как создавать полнотекстовые каталоги и индексы.

Изучив материал этого занятия, вы сможете:

- ✓ создавать полнотекстовые каталоги;
- ✓ создавать полнотекстовые индексы;
- ✓ настраивать режимы заполнения полнотекстовых индексов;
- ✓ управлять заполнением полнотекстовых индексов.

Продолжительность занятия — 20 минут.

Полнотекстовые каталоги

Первый шаг в построении полнотекстового индекса — создание структуры хранилища. В отличие от реляционных индексов полнотекстовые индексы обладают уникальной внутренней структурой, которая поддерживается в хранилище с особым форматом, называемом *полнотекстовым каталогом* (full text catalog). Каждый полнотекстовый каталог содержит один или несколько *полнотекстовых индексов*.

Общий синтаксис создания полнотекстового каталога следующий:

```
CREATE FULLTEXT CATALOG имя_каталога  
[ON FILEGROUP файловая_группа ]  
[IN PATH 'корневая_папка']  
[WITH <параметр_каталога>]  
[AS DEFAULT]  
[AUTHORIZATION имя_владельца ]
```

<параметр_каталога>:=

```
ACCENT_SENSITIVITY = {ON|OFF}
```

ПРИМЕЧАНИЕ На экзамене

До SQL Server 2008 связывать полнотекстовый каталог с файловой группой надо было только в целях резервного копирования, а все содержимое самого каталога хранилось в структуре каталогов операционной системы. В SQL Server 2008 больше нет внешней файловой структуры, а содержимое полнотекстового каталога теперь хранится в базе данных.

Предложение *FILEGROUP* определяет файловую группу, где будут храниться все полнотекстовые индексы этого каталога. Предложение *IN PATH* больше не поддерживается, так как полнотекстовые индексы теперь хранятся в базе данных.

Параметр *ACCENT_SENSITIVITY* определяют, должен ли обработчик полнотекстового поиска учитывать диакритические знаки при построении полнотекстового индекса и при поиске. При изменении параметра *ACCENT_SENSITIVITY* необходимо перестраивать все полнотекстовые индексы каталога.

Предложение *AS DEFAULT* работает точно так же, как и параметр *DEFAULT* файловой группы. Если при создании полнотекстового индекса не указать имя каталога, то SQL Server создаст его в каталоге, отмеченном как полнотекстовый каталог по умолчанию.

Параметр *AUTHORIZATION* задает владельца полнотекстового каталога. Указанный владелец должен обладать на нем разрешением *TAKE OWNERSHIP*.

ПРИМЕЧАНИЕ Размещение файловой группы

Несмотря на возможность размещения полнотекстового каталога в файловой группе, содержащей также и реляционные данные, рекомендуется создавать отдельную файловую группу для полнотекстовых индексов, чтобы разделить операции ввода-вывода полнотекстового каталога и реляционных данных.

Полнотекстовые индексы

Создав полнотекстовый каталог, можно переходить к созданию полнотекстовых индексов, служащих для поиска неструктурированной информации.

Полнотекстовые индексы можно создавать на основе столбцов следующих типов: *CHAR/VARCHAR*, *XML* и *VARBINARY*. В случае столбца *CHAR/VARCHAR*, обработчик полнотекстового поиска может непосредственно анализировать данные и строить подходящий индекс. При построении полнотекстовых индексов по столбцам *XML* загружается специальный обработчик, «умеющий» анализировать *XML*-документы и индексировалось их содержимое, а не *XML*-теги.

Наиболее популярные столбцы типа *VARBINARY(MAX)* хранят документы, используя появившиеся в *SQL Server 2008* возможности *FILESTREAM*. Хотя обработчик полнотекстового поиска способен индексировать прямо при создании и редактировании документов, устраняя тем самым дорогостоящий процесс преобразования, ему требуются специальные сборки, предназначенные для различных типов хранимых документов. При обработке столбцов *VARBINARY(MAX)* требуется указывать столбец, описывающий тип документа, чтобы обработчик полнотекстового поиска мог загрузить подходящую сборку. *SQL Server 2008* поставляется с 50-ю фильтрами, позволяющими анализировать документы различных типов, в том числе документы *HTML*, *Word*, *Microsoft Office PowerPoint* и *Microsoft Office Excel*.

Обработчик полнотекстового поиска при построении индексов полагается на вспомогательные службы, такие как средства разбиения по словам или парадигматические модули, которые различны для разных языков. Первым делом при построении эффективных индексов неструктурированных данных надо создать список встречающихся в них слов. *Средства разбиения по словам* (*word breakers*) — это сборки, определяющие разрывы между словами для создания списка индексируемых слов. Глаголы могут иметь различные формы, такие как прошедшее, настоящее или будущее время, поэтому парадигматические модули объединяют глаголы так, чтобы при обработке запросов обнаруживалась информация даже в глаголах разных временных форм. Для каждого языка имеются свои средства разбиения по словам и парадигматические модули, так как однокоренные глаголы и даже разделители слов различны в разных языках.

Из получившегося списка слов отфильтровываются широко распространенные слова, или *стоп-слова*. Стоп-слова задаются, чтобы индекс не засорялся большим объемом обычно не используемых в поиске слов. Например, артикли *the*, *a* и *an* в английском и *le* и *la* — во французском считаются стоп-словами.

Можно создавать полнотекстовый индекс на основе нескольких столбцов, но только один на таблицу или индексированное представление. Общий синтаксис создания полнотекстового индекса таков:

```
CREATE FULLTEXT INDEX ON имя_таблицы
[ ( { имя_столбца
    [ TYPE COLUMN имя_столбца_типа ]
    [ LANGUAGE термин_языка ]
} [ ,...n ]
) ]
KEY INDEX имя_индекса
```

```

[ ON <параметры_файловой_группы_каталога> ]
[ WITH [ ( ) <параметры> [ ,...n ] [ ) ] ]
[;]

< параметры_файловой_группы_каталога >::=
{имя_полнотекстового_каталога
 | (имя_полнотекстового_каталога, FILEGROUP имя_файловой_группы )
 | ( FILEGROUP имя_файловой_группы, имя_полнотекстового_каталога)
 | ( FILEGROUP имя_файловой_группы)}

<параметры with>::=
{CHANGE_TRACKING [ = ] { MANUAL | AUTO | OFF [, NO POPULATION ] }
 | STOPLIST [ = ] { OFF | SYSTEM | имя_списка_стоп_слов }}

```

Параметр *TYPE COLUMN* задает колонку, содержащую тип фильтра, который должен применяться обработчиком полнотекстового поиска при анализе столбца *VARBINARY(MAX)*. Параметр *LANGUAGE* служит для определения языка индексируемых данных. Параметр *KEY INDEX* — это единственный столбец таблицы или индексируемого представления, который однозначно определяет запись.

Отслеживание изменений

Параметр полнотекстового индекса *CHANGE_TRACKING* определяет, как SQL Server обращается с индексом при изменении исходных данных.

Если задать значение *MANUAL* или *AUTO*, SQL Server будет поддерживать список изменений индексируемых данных. Если выбрать значение *MANUAL*, придется вручную периодически обновлять полнотекстовый индекс. Если задано значение *AUTO*, SQL Server автоматически будет обновлять полнотекстовый индекс при изменении данных. В отличие от реляционного индекса наложение полнотекстового индекса не происходит немедленно, поскольку прежде чем добавить изменения в существующий индекс, данные нужно передать в обработчик индексов, который затем вызывает средства разбиения по словам, парадигматические модули, языковые фильтры и списки стоп-слов.

Если параметру *CHANGE_TRACKING* задано значение *OFF*, SQL Server не поддерживает списка изменений исходных данных. Поэтому когда потребуется обновить индекс, чтобы он соответствовал текущим данным в индексируемой колонке, придется полностью перезаполнить индекс. Полнотекстовый индекс можно создать изначально незаполненным, если отключить *CHANGE_TRACKING* и указать параметр *NO POPULATION*.

Язык, средства разбиения по словам и парадигматические модули

Спецификация языка — основной элемент построения эффективного полнотекстового индекса. Для всех данных можно использовать одно средство разбиения по словам, однако можно получить непредсказуемые результаты, если информация представлена на нескольких языках. Например, слова в английском языке разделяются пробелами, тогда как в языках вроде немецкого или французского слова могут сливаться. Если средство разбиения по словам рас-

познает только пробелы между словами, полнотекстовый индекс будет пригодным только для поиска информации на английском языке.

Спецификация языка служит для управления тем, какие средства разбиения по словам и парадигматические модули будут загружаться обработчиком полнотекстового поиска. Выбранное средство разбиения по словам и парадигматический модуль задаются для всего полнотекстового индекса и не могут динамически меняться в зависимости от типа столбца, как это можно сделать для столбца *VARBINARY*. Однако нет необходимости разбивать информацию колонок по языкам. Слова могут различаться, но все множество языков можно разбить на несколько языковых семейств, в каждом из которых использовать средство разбиения по словам, способное работать со словами узкого семейства языков.

Допустим, что хранимые данные содержат тексты на западноевропейских языках, таких как английский, немецкий, французский и испанский. Для индексирования столбца можно использовать средство разбиения одного языка, которое корректно выделит слова для индекса. Если информация хранится на нескольких языках, следует выбрать средство для самого сложного языка. Например, средство разбиения по словам на немецком языке сумеет корректно обработать английский, испанский и французский языки, в то время как английское средство разбиения по словам столкнется с проблемами в некоторых языковых конструкциях немецкого языка.

Если языки сильно различаются, например арабский, китайский, английский и исландский, придется разнести языки по разным столбцам. В противном случае не получится правильно разбить текст по словам и построить полнотекстовый индекс с ожидаемым поведением.

В набор поставки SQL Server 2008 входит 50 средств разбиения по словам и парадигматических модулей для разных языков. Кроме того, можно зарегистрироваться в службе поддержки и использовать в SQL Server средства разбиения по словам и парадигматические модули сторонних разработчиков. В частности, средства разбиения по словам для турецкого, датского и польского языков созданы сторонними разработчиками и поставляются в составе SQL Server 2008.

Средства разбиения по словам находят и отмечают границы слов в тексте. Затем полнотекстовый индекс объединяет все лексемы и строит для поиска статистику распределения. Кроме того, средства разбиения по словам выявляют взаимное расположение слов в наборе данных и встраивают эту информацию в полнотекстовый индекс. Возможность поиска с учетом расположения слов является уникальной особенностью полнотекстовых индексов, что позволяет использовать сложные критерии поиска, учитывающие расположение слов по отношению друг к другу.

SQL Server применяет парадигматические модули, чтобы предоставить возможность полнотекстового поиска запрашиваемого слова с учетом всех словоформ, например в английском при поиске слова *drive* учитываются такие формы, как *drove*, *driving* и *driven*. Морфологический анализ специфичен для каждого языка. Хотя и допускается использование немецкого средства разбиения по словам для разметки английского языка, но немецкий парадигматический модуль непригоден для анализа англоязычного текста.

Контрольные вопросы

1. Что необходимо создать до создания полнотекстовых индексов?
2. Что необходимо определить при создании полнотекстового индекса по документам, хранимым в *FILESTREAM*?

Ответы на контрольные вопросы

1. Полнотекстовые индексы содержатся в полнотекстовых каталогах. Поэтому перед созданием полнотекстового индекса необходимо создать полнотекстовый каталог.
2. SQL Server хранит документы в столбце *VARBINARY(MAX)* с параметром *FILESTREAM*. Чтобы создать полнотекстовый индекс, следует задать столбец, где указан тип документов, хранимых в столбце *VARBINARY (MAX)*. Это необходимо для загрузки подходящего фильтра, используемого средством разбиения по словам.

Практикум. Создание полнотекстовых индексов

Вы создадите полнотекстовый каталог и полнотекстовый индекс.

Задание 1. Создание полнотекстового каталога

Вы создадите полнотекстовый каталог.

1. Выполните следующий код, чтобы в базу данных *AdventureWorks* добавить файловую группу и файл, которые будут использоваться при полнотекстовом индексировании:

```
ALTER DATABASE AdventureWorks
    ADD FILEGROUP AWFullTextFG
GO

ALTER DATABASE AdventureWorks
    ADD FILE (NAME = N'S AdventureWorksFT', FILENAME =
        N'C:\Program Files\Microsoft SQL Server\
        MSSQL10.MSSQLSERVER\MSSQL\DATA\AdventureWorks FT.ndf')
    TO FILEGROUP AWFullTextFG
GO
```

2. Чтобы создать полнотекстовый каталог, в *AdventureWorks* выполните следующий код:

```
USE AdventureWorks
GO
CREATE FULLTEXT CATALOG ProductsFTC
    ON FILEGROUP AWFullTextFG
GO
```

Задание 2. Создание полнотекстового индекса

Вы создадите полнотекстовый индекс.

1. Выполните следующий код, чтобы создать полнотекстовый индекс для описания продукции:

```
CREATE FULLTEXT INDEX ON Production.ProductDescription(Description)
    KEY INDEX PK_ProductDescription_ProductDescriptionID
    ON ProductsFTC
    WITH CHANGE_TRACKING = AUTO
GO
```

2. В базе данных *AdventureWorks* откройте **Хранилище (Storage)** и узел **Полнотекстовые каталоги (Full Text Catalogs)**, щелкните правой кнопкой каталог *ProductFTC* и выберите **Свойства (Properties)**.
3. Выберите страницу **Таблицы или представления (Tables/Views)**, чтобы изучить только что созданный полнотекстовый индекс.
4. Ознакомьтесь с полнотекстовыми индексами в каталоге *AW2008FullTextCatalog*, поставляемом вместе с базой данных *AdventureWorks*.

Резюме занятия

- Перед созданием полнотекстового индекса необходимо создать полнотекстовый каталог, сопоставленный файловой группе, для хранения одного или нескольких полнотекстовых индексов.
- Полнотекстовые индексы можно создавать на основе содержания столбцов *CHAR/VARCHAR, XML* и *VARBINARY*.
- При создании полнотекстового индекса на основе столбца *VARBINARY (MAX)* необходимо указать столбец для параметра *COLUMN TYPE*, чтобы обработчик полнотекстового поиска мог загрузить правильный фильтр для выполнения анализа.
- Параметр *LANGUAGE* определяет средства разбиения по словам и парадигматические модули, которые SQL Server должен загрузить для разметки текста и создания списка словоформ для индекса.
- В отличие от средств разбиения по словам, которые с приемлемым результатом могут использоваться на разных близких по грамматике языках, парадигматические модули пригодны только для конкретных языков.
- Параметр *CHANGE_TRACKING* определяет, будет ли SQL Server отслеживать изменения в исходных столбцах, а также будут ли эти изменения автоматически учитываться в индексе.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Вы администратор баз данных своей компании. От вас требуется предоставить группе поддержки продаж возможность выполнять поиск с нечеткими критериями по описаниям продукции. Как удовлетворить требованиям пользователей с минимальными усилиями? (Выберите два варианта, которые в совокупности являются правильным ответом.)
 - A. Создайте полнотекстовый каталог, указав файловую группу для резервного копирования и корневой путь, где в файловой системе будет храниться содержимое каталога.

- В. Создайте полнотекстовый каталог и укажите файловую группу для хранения содержимого каталога.
 - С. В таблице с описанием продукции создайте полнотекстовый индекс для столбца с описанием и задайте параметр *NO POPULATION*.
 - Д. В таблице с описанием продукции создайте полнотекстовый индекс для столбца с описанием и задайте параметр *CHANGE_TRACKING AUTO*.
2. Вам надо настроить полнотекстовые индексы так, чтобы изменения учитывались в индексе максимально быстро, при минимуме усилий со стороны администратора. Какую команду следует выполнить?
- A. `ALTER FULLTEXT INDEX ON <имя_таблицы> START FULL POPULATION`
 - B. `ALTER FULLTEXT INDEX ON <имя_таблицы> START INCREMENTAL POPULATION`
 - C. `ALTER FULLTEXT INDEX ON <имя_таблицы> SET CHANGE_TRACKING AUTO`
 - D. `ALTER FULLTEXT INDEX ON <имя_таблицы> START UPDATE POPULATION`

Занятие 2. Запросы полнотекстовых данных

SQL Server предоставляет две команды для выполнения запросов полнотекстовых данных: *CONTAINS* и *FREETEXT*. Есть еще две дополнительные команды, которые возвращают результирующий набор с дополнительными столбцами данных: *CONTAINSTABLE* и *FREETEXTTABLE*.

Основное различие между этими четырьмя командами в том, что *CONTAINS* и *FREETEXT* возвращают значения True/False, применяемые для ограничения результирующего набора, а *CONTAINSTABLE* и *FREETEXTTABLE* возвращают результирующий набор результатов, который может применяться для расширения функциональности запросов.

При включении в запрос полнотекстовых предикатов, SQL Server передает полнотекстовую часть условия обработчику полнотекстового поиска для выполнения разбиения по словам и разметки аргумента поиска. Статистика распределения, основанная на размеченном условии поиска, возвращается оптимизатору, который соединяет полнотекстовую часть запроса с реляционной и строит план запроса.

Изучив материал этого занятия, вы сможете:

- ✓ выполнять запросы неструктурированной информации при помощи полнотекстовых индексов.

Продолжительность занятия — 20 минут.

Запросы с использованием FREETEXT

Запросы с *FREETEXT* — самая простая форма полнотекстового поиска. Общий синтаксис таков:

```
FREETEXT ( { имя_столбца | (список_столбца) | * }  
          , 'произвольная_строка' [ , LANGUAGE термин_языка ] )
```

Вот пример запроса с *FREETEXT*:

```
SELECT ProductDescriptionID, Description  
FROM Production.ProductDescription  
WHERE FREETEXT(Description,N'bike')  
GO
```

Параметр *LANGUAGE* позволяет выбрать средство разбиения по словам и парадигматический модуль для анализа введенного аргумента поиска. К примеру, хотя при создании полнотекстового индекса мог использоваться немецкий язык, можно указать английский в качестве языкового параметра для обработки запроса английским модулем разбиения по словам.

ПРИМЕЧАНИЕ Языковые параметры

В запросе можно выбрать язык отличный от использовавшегося при создании индекса, но точность полнотекстового поиска при этом вряд ли автоматически улучшится. Для каждого языка существует свой парадигматический модуль, поэтому если индекс построен с применением языка отличного от языка, указанного в запросе, такой запрос не позволит найти какие-либо производные формы слов.

ПРИМЕЧАНИЕ На экзамене

Все полнотекстовые условия поиска являются строками в Юникоде. Если передать в запрос строку не в Юникоде, то он все равно будет работать, хотя и не столь эффективно, так как оптимизатор не сможет выполнить пробную проверку параметра для оценки статистики распределения в полнотекстовом индексе. Для достижения максимальной производительности старайтесь всегда все условия полнотекстового поиска вводить в Юникоде.

FREETEXTTABLE возвращает результирующий набор с дополнительной информацией, которая ранжирует результаты в соответствии с тем, насколько точно они совпадают с условием поиска. Общий синтаксис *FREETEXTTABLE* таков:

```
FREETEXTTABLE (таблица, { имя_столбца | (список_столбцов) | * }  
              , 'произвольная_строка'  
              [ , LANGUAGE термин_языка]  
              [ , первые_n_по_ранжиру ] )
```

Тот же самый запрос с командой *FREETEXTTABLE*:

```
SELECT a.ProductDescriptionID, a.Description, b.*  
FROM Production.ProductDescription a  
     INNER JOIN FREETEXTTABLE(Production.ProductDescription,  
                             Description,N'bike') b ON a.ProductDescriptionID = b.[Key]  
ORDER BY b.[Rank]  
GO
```

Запросы с использованием CONTAINS

В запросах, требующих большей гибкости, следует использовать предикат *CONTAINS*, позволяющий:

- искать формы слов;
- искать с учетом взаимного расположения слов;
- определять относительный вес терминов.

Общий синтаксис *CONTAINS* таков:

CONTAINS

```
( { имя_столбца | (список_столбцов) | * }
, '< условие_поиска >'
[ , LANGUAGE термин_языка ] )
```

< условие_поиска_contains > ::=

```
{ < простое_условие > | < префиксное_условие > | < собирательное_условие >
| < условие_взаимного_расположения > | < взвешенное_условие > }
| { ( < условие_поиска_contains > )
[ { < AND > | < AND NOT > | < OR > } ]
< условие_поиска_contains > [ ...n ] }
```

< простое_условие > ::=

```
«слово» | «фраза»
```

< префиксное_условие > ::=

```
{ «слово*» | «фраза*» }
```

< собирательное_условие > ::=

```
FORMSOF ( { INFLECTIONAL | THESAURUS } , < простое_условие > [ ,...n ] )
```

< условие_взаимного_расположения > ::=

```
{ < простое_условие > | < префиксное_условие > }
{ { NEAR | ~ }
{ < простое_условие > | < префиксное_условие > } } [ ...n ]
```

< взвешенное_условие > ::=

```
ISABOUT ( { { < простое_условие > | < префиксное_условие > | <
собирательное_условие >
| < условие_взаимного_расположения > }
[ WEIGHT (весовое_значение ) ] } [ ,...n ] )
```

Термины поиска могут использоваться как для точного сравнения, так и для префиксного. Следующий запрос возвращает все товары при условии точного совпадения со словом *bike*. Хотя этот запрос выглядит почти так же, как и с *REETEXT*, но возвращает на 2 строки меньше из-за требования точного совпадения:

```
SELECT ProductDescriptionID, Description
FROM Production.ProductDescription
WHERE CONTAINS(Description,N'bike')
GO
```

Если требуется выполнить поиск слов с простыми знаками подстановки в условии, можно выполнить такой запрос:

```
SELECT ProductDescriptionID, Description
FROM Production.ProductDescription
WHERE CONTAINS(Description,N'»bike*»')
GO
```

Если сравнить результат этого запроса с тем, что вернул *FREETEXT*-запрос, окажется, что они возвращают одинаковый набор записей. При помощи *CONTAINS* можно явно указать, что требуется выполнять нечеткий поиск с префиксами слов, а *FREETEXT* такой поиск выполняет по умолчанию.

Если надо найти формы слов, можно воспользоваться параметрами *FORMSOF*, *INFLECTIONAL* и *THESAURUS*. *INFLECTIONAL* заставляет обработчик полнотекстового поиска учитывать основы слов. Например, при поиске слова *driven*, также будут найдены *drive*, *driving*, *drove* и т. д. *THESAURUS* предоставляет синонимы для условия поиска. Ниже приведены примеры поиска слов и их форм:

```
SELECT ProductDescriptionID, Description
FROM Production.ProductDescription
WHERE CONTAINS(Description,N' FORMSOF (INFLECTIONAL,ride) ')
GO
```

```
SELECT ProductDescriptionID, Description
FROM Production.ProductDescription
WHERE CONTAINS(Description,N' FORMSOF (THESAURUS,metal) ')
GO
```

ПРИМЕЧАНИЕ Файлы тезаурусов

Для каждого поддерживаемого языка существует файл тезауруса. Все тезаурусы представляют собой XML-файлы и хранятся в каталоге *FTDATA*, расположенном в папке, где установлен SQL Server. Файлы тезаурусов не заполнены, а потому, перед выполнением поиска синонимов следует заполнить файлы тезаурусов. Подробнее о файлах тезаурусов см. занятие 3.

Поскольку полнотекстовые индексы строятся на основе неструктурированных данных, то кроме индексов они хранят информацию о взаимном расположении слов. Для поиска с учетом взаимного расположения слов служит ключевое слово *NEAR*. Поиск с учетом взаимного расположения и взвешенного расположения можно выполнять, используя *CONTAINS*, однако поиск такого типа обычно выполняется при помощи *CONTAINSTABLE* с использованием вычисляемого значения *RANK*.

Следующий запрос возвращает все строки, где слово *bike* находится недалеко от слова *performance*. На значение ранга влияет расстояние между этими двумя словами:

```
SELECT a.ProductDescriptionID, a.Description, b.*
FROM Production.ProductDescription a INNER JOIN
    CONTAINSTABLE(Production.ProductDescription, Description,
        N'bike NEAR performance') b ON a.ProductDescriptionID = b.[Key]
ORDER BY b.[Rank]
GO
```

Следующий запрос возвращает первые 10 строк в соответствии со средневзвешенными индексами слов performance, comfortable, smooth, safe и competition:

```
SELECT a.ProductDescriptionID, a.Description, b.*
FROM Production.ProductDescription a INNER JOIN
    CONTAINSTABLE(Production.ProductDescription, Description,
        N'ISABOUT (performance WEIGHT (.8), comfortable WEIGHT (.6),
        smooth WEIGHT (.2) , safe WEIGHT (.5), competition WEIGHT (.5))', 10)
    b ON a.ProductDescriptionID = b.[Key]
ORDER BY b.[Rank] DESC
GO
```

Контрольные вопросы

1. Какой предикат выполняет нечеткий поиск по умолчанию?
2. Какой предикат используется для поиска с учетом взаимного расположения слов и синонимов?

Ответы на контрольные вопросы

1. Предикаты *FREETEXT* и *FREETEXTTABLE* по умолчанию служат для поиска по шаблону.
2. *CONTAINS* и *CONTAINSTABLE* применяются для поиска с учетом взаимного расположения слов, синонимов и словоформ.

Практикум. Выполнение запросов с использованием полнотекстовых индексов

Вы выполните несколько запросов и сравните результаты выполнения команд *CONTAINS*, *CONTAINSTABLE*, *FREETEXT* и *FREETEXTTABLE*.

1. Выполните следующий запрос и ознакомьтесь с содержимым столбца Description:

```
SELECT Description
FROM Production.ProductDescription
GO
```

2. Выполните следующий запрос и ознакомьтесь с полученными строками:

```
SELECT ProductDescriptionID, Description
FROM Production.ProductDescription
WHERE FREETEXT(Description, N'bike')
GO
```

3. Выполните следующий запрос и ознакомьтесь с полученными строками:

```
SELECT a.ProductDescriptionID, a.Description, b.*
FROM Production.ProductDescription a
    INNER JOIN FREETEXTTABLE(Production.ProductDescription,
        Description, N'bike') b ON a.ProductDescriptionID = b.[Key]
ORDER BY b.[Rank]
GO
```


4. Выполните следующий запрос и ознакомьтесь с полученными строками:

```
SELECT ProductDescriptionID, Description
FROM Production.ProductDescription
WHERE CONTAINS(Description,N'bike')
GO
```

5. Выполните следующий запрос и ознакомьтесь с полученными строками:

```
SELECT ProductDescriptionID, Description
FROM Production.ProductDescription
WHERE CONTAINS(Description,N'»bike*»')
GO
```

6. Выполните следующий запрос и ознакомьтесь с полученными строками:

```
SELECT ProductDescriptionID, Description
FROM Production.ProductDescription
WHERE CONTAINS(Description,N' FORMSOF (INFLECTIONAL,ride) ')
GO
```

7. Выполните следующий запрос и обратите внимание, что не возвращено ни одной строки, так как файл тезауруса еще не заполнен:

```
SELECT ProductDescriptionID, Description
FROM Production.ProductDescription
WHERE CONTAINS(Description,N' FORMSOF (THESAURUS,metal) ')
GO
```

8. Выполните следующий запрос и ознакомьтесь с полученными строками:

```
SELECT a.ProductDescriptionID, a.Description, b.*
FROM Production.ProductDescription a INNER JOIN
    CONTAINSTABLE(Production.ProductDescription, Description,
        N'bike NEAR performance') b ON a.ProductDescriptionID = b.[Key]
ORDER BY b.[Rank]
GO
```

9. Выполните следующий запрос и ознакомьтесь с полученными строками:

```
SELECT a.ProductDescriptionID, a.Description, b.*
FROM Production.ProductDescription a INNER JOIN
    CONTAINSTABLE(Production.ProductDescription, Description,
        N'ISABOUT (performance WEIGHT (.8), comfortable WEIGHT (.6),
        smooth WEIGHT (.2) , safe WEIGHT (.5), competition WEIGHT (.5))', 10)
    b ON a.ProductDescriptionID = b.[Key]
ORDER BY b.[Rank] DESC
GO
```

Резюме занятия

- Предикаты *FREETEXT* и *CONTAINS* возвращают значения True или False, которые можно затем использовать в запросах, таких как предложение *EXISTS*, для сужения результирующего набора.

- Предикаты *FREETEXTTABLE* и *CONTAINSTABLE* возвращают результирующий набор, включая столбец ранга, который сообщает, насколько точно строка соответствует условиям поиска.
- *FREETEXT* и *FREETEXTTABLE* по умолчанию выполняют поиск по шаблону.
- *CONTAINS* и *CONTAINSTABLE* могут выполнять поиск по шаблону с учетом взаимного расположения слов, словоформ и синонимов.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Требуется найти два слова с учетом их взаимного расположения в строке. Какие полнотекстовые предикаты следует использовать для выполнения такого поиска? (Выберите два варианта, которые в совокупности образуют правильный ответ.)
 - A. *CONTAINS*
 - B. *FREETEXT*
 - C. *CONTAINSTABLE*
 - D. *FREETEXTTABLE*
2. Требуется найти два слова с учетом их взаимного расположения на основе взвешенных значений аргументов поиска. Какими параметрами предиката *CONTAINSTABLE* следует воспользоваться?
 - A. *FORMSOF* с ключевым словом *THESAURUS*.
 - B. *FORMSOF* с ключевым словом *INFLECTIONAL*.
 - C. *ISABOUT*.
 - D. *ISABOUT* с ключевым словом *WEIGHT*.

Занятие 3. Управление полнотекстовыми индексами

Простое создание и автоматическое заполнение полнотекстовых индексов само по себе очень полезно, но эту возможность можно расширить, создав файлы тезаурусов и списков стоп-слов для фильтрации индексов. На этом занятии рассказывается, как создавать и использовать файлы тезаурусов и как строить списки стоп-слов и перестраивать полнотекстовые индексы.

Изучив материал этого занятия, вы сможете:

- ✓ поддерживать файлы тезаурусов;
- ✓ создавать списки стоп-слов и управлять ими;
- ✓ управлять заполнением полнотекстовых индексов.

Продолжительность занятия — 15 минут.

Тезаурусы

Файл *тезауруса* используется в полнотекстовых запросах, возвращающих строки, где содержится аргумент поиска либо с его синонимы. Тезаурус — это XML-файл, ориентированный на конкретный язык и расположенный в каталоге *FTDATA*. Заданный для того или иного языка тезаурус SQL Server автоматически использует при выполнении запросов *FREETEXT* и *FREETEXTTABLE*. В запросах *CONTAINS* и *CONTAINSTABLE* тезаурус используется, только если задан параметр *FORMSOF THESAURUS*.

Тезаурус может содержать наборы расширения или замены. Набор замены определяет термин или термины, которые заменяются в аргументе поиска до его передачи средству разбиения по словам. Набор расширения определяет термины, применяемые для расширения аргумента поиска. При использовании списка расширения SQL Server дополнительно возвращает строки, содержащие любой из терминов списка.

Общая структура файла тезауруса такова:

```
<XML ID=»Microsoft Search Thesaurus»>
<!-- Commented out
<thesaurus xmlns=»x-schema:tsSchema.xml»>
<diacritics_sensitive>0</diacritics_sensitive>
  <expansion>
    <sub>Internet Explorer</sub>
    <sub>IE</sub>
    <sub>IE5</sub>
  </expansion>
  <replacement>
    <pat>NT5</pat>
    <pat>W2K</pat>
    <sub>Windows 2000</sub>
  </replacement>
  <expansion>
    <sub>run</sub>
    <sub>jog</sub>
  </expansion>
</thesaurus>
-->
</XML>
```

Диакритический параметр определяет, чувствителен ли тезаурус к диакритическим знакам.

Списки стоп-слов

Списки стоп-слов (stop lists), называемые в предыдущей версии SQL Server файлами неучитываемых слов (noise word files), применяются для исключения слов, нежелательных в полнотекстовом индексе. Часто встречающиеся слова исключаются из индекса, чтобы правильно сформированный запрос возвращал правильные и уместные результаты. Вполне разумно искать слово «Microsoft» в Интернете, но совершенно глупо искать его в документации к продуктам

Microsoft — он вернет все документы в системе или, в правильно разработанной системе, такой запрос будет сочтен неверным.

Самые распространенные в каждом языке стоп-слова, например как артикли *the*, *a* и *an* в английском, уже учтены в обработчике полнотекстовых запросов. В дополнение к общепринятым словам, администратор может добавить слова, характерные для организации и с высокой вероятностью встречающиеся в индексируемых данных. Когда стоп-слово встречается как аргумент поиска или в индексируемых данных, средство разбиения по словам помечает термин как несущественный и удаляет его. Если все аргументы, передаваемые в полнотекстовом предикате, являются стоп-словами, запрос не возвращает никаких результатов, даже не обращаясь при этом к данным.

ПРИМЕЧАНИЕ На экзамене

В то время как одни функции SQL Server работают одинаково от версии к версии, другие улучшаются или изменяются. Можно ожидать, что на экзамене встретятся вопросы для проверки знаний об изменившихся функциях в новой версии. В SQL Server 2005 и более ранних версиях, настраиваемые файлы неучитываемых слов находились в каталоге *FTDATA*. В SQL Server 2008 нужно настраивать списки стоп-слов, которые хранятся в базе данных. Весьма вероятно, что если встретится вопрос о настройке стоп-слов, то среди ответов будут описывающие настройку как SQL Server 2005, так и SQL Server 2008. Понятно, что все ответы, описывающие поведение SQL Server 2005 будут сочтены неверными.

Заполнение полнотекстовых индексов

Полнотекстовые индексы заполняются вручную — по запросу или по расписанию — или автоматически при изменении исходных данных. Заполнение индексов можно останавливать, приостанавливать или возобновлять — это позволяет управлять загрузкой ресурсов при выполнении больших объемов изменений в полнотекстовом индексе.

Есть несколько параметров заполнения полнотекстовых индексов:

- **FULL** — заново обрабатываются все строки исходных данных и полностью перестраивается полнотекстовый индекс;
- **INCREMENTAL** — обрабатываются только строки, изменившиеся с момента последнего заполнения. Чтобы эта функция была доступна, в таблице должен быть столбец временной метки;
- **UPDATE** — обрабатываются любые изменения, произошедшие после последнего обновления индекса. Чтобы эта функция была доступна, необходимо для индекса включить параметр *CHANGE_TRACKING* и присвоить ему значение *MANUAL*.

Запуск заполнения полнотекстового индекса выполняется командой **ALTER FULLTEXT INDEX**.

Контрольные вопросы

1. Какие файлы обеспечивают возможность поиска с использованием синонимов?
2. Что следует настроить, чтобы исключить слова из индекса и аргументов поиска?

Ответы на контрольные вопросы

1. Поддержку синонимов в аргументах поиска обеспечивает файл тезауруса.
2. Список стоп-слов содержит слова, исключаемые как из полнотекстового индекса, так и из аргументов поиска.

Практикум. Управление полнотекстовыми индексами

Вы заполните файл тезауруса и сравните результаты поиска. Также вы создадите список стоп-слов, чтобы исключать часто встречающиеся слова из аргументов поиска и полнотекстового индекса.

Задание 1. Заполнение тезауруса

Вы заполните файл тезауруса.

1. Выполните следующий запрос и убедитесь, что он не возвращает ни одной строки:

```
SELECT ProductDescriptionID, Description
FROM Production.ProductDescription
WHERE CONTAINS(Description,N' FORMSOF (THESAURUS,metal) ')
GO
```

2. Откройте файл TSenu.xml (английский США), расположенный в папке *Program Files\Microsoft SQL Server\MSSQL10.MSSQLSERVER\MSSQL\FTData*.
3. Замените его содержимое на:

```
<XML ID="Microsoft Search Thesaurus">
  <thesaurus xmlns="x-schema:tsSchema.xml">
    <diacritics_sensitive>0</diacritics_sensitive>
    <expansion>
      <sub>metal</sub>
      <sub>steel</sub>
      <sub>aluminum</sub>
      <sub>alloy</sub>
    </expansion>
  </thesaurus>
</XML>
```

4. Перезагрузите файл тезауруса следующей командой (1033 указывает на английский язык США):

```
USE AdventureWorks
GO
```

```
EXEC sys.sp_fulltext_load_thesaurus_file 1033;  
GO
```

5. Выполните следующий запрос и убедитесь, что теперь он возвращает 33 строки данных и сравните этот результат с тем, что вы ожидали на основе тезауруса:

```
SELECT ProductDescriptionID, Description  
FROM Production.ProductDescription  
WHERE CONTAINS(Description,N' FORMSOF (THESAURUS,metal) ')  
GO
```

Задание 2. Создание списка стоп-слов

Вы создадите список стоп-слов и сравните полученные результаты.

ПРИМЕЧАНИЕ Разделители команд

Точка с запятой является разделителем команд Transact-SQL. В большинстве случаев нет необходимости явно указывать разделитель команд. Однако некоторые команды, такие как *CREATE FULLTEXT STOPLIST* требуют явного указания разделителя — только в этом случае команда будет выполнена успешно.

1. В базе данных *AdventureWorks* выполните следующий запрос и ознакомьтесь с полученными 16 строками:

```
SELECT ProductDescriptionID, Description  
FROM Production.ProductDescription  
WHERE CONTAINS(Description,N'»bike*»')  
GO
```

2. Создайте список стоп-слов командой:

```
CREATE FULLTEXT STOPLIST ProductStopList;  
GO
```

3. Добавьте слово *bike* в список стоп-слов командой:

```
ALTER FULLTEXT STOPLIST ProductStopList ADD 'bike' LANGUAGE 1033;  
GO
```

4. Свяжите список стоп слов с полнотекстовым индексом таблицы *ProductDescription*:

```
ALTER FULLTEXT INDEX ON Production.ProductDescription  
SET STOPLIST ProductStopList  
GO
```

5. В базе данных *AdventureWorks* выполните следующий запрос и ознакомьтесь с результатами:

```
SELECT ProductDescriptionID, Description  
FROM Production.ProductDescription  
WHERE CONTAINS(Description,N'»bike*»')  
GO
```

Резюме занятия

- Файлами тезаурусов управляют путем редактирования файлов для каждого конкретного языка. Файлы хранятся в каталоге *FTDATA* экземпляра сервера.
- Список стоп-слов, которые должны исключаться из аргументов поиска и полнотекстового индекса, строится при помощи команд *CREATE FULLTEXT STOPLIST* и *ALTER FULLTEXT STOPLIST*.
- Готовый список стоп-слов можно связать с полнотекстовым индексом командой *ALTER FULLTEXT INDEX*.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Имеется список слов, которые должны исключаться из аргументов поиска. Что нужно выполнить на SQL Server 2008, чтобы решить задачу минимальными усилиями?
 - A. Создать список стоп-слов и связать его с полнотекстовым индексом.
 - B. Создать файл со списком неучитываемых слов и связать его с полнотекстовым индексом.
 - C. Заполнить файл тезауруса и связать его с полнотекстовым индексом.
 - D. Разбирать поступающий запрос, удаляя из его аргументов поиска часто встречающиеся слова.

Закрепление материала главы

Для закрепления знаний и навыков, полученных при изучении этой главы, выполните следующее:

- изучите резюме главы;
- повторите основные термины, использованные в этой главе;
- выполните задания по темам этой главы;
- сделайте предложенные упражнения;
- пройдите пробный экзамен.

Резюме главы

- Полнотекстовые индексы могут быть созданы на основе столбцов *CHAR/VARCHAR*, *XML* и *VARBINARY*.
- Для полнотекстового индекса по столбцу *VARBINARY* необходимо указать фильтр, который будет применяться средством разбиения по словам при интерпретации содержимого документа.
- Файлы тезаурусов позволяют создать список синонимов или список подстановки слов для терминов поиска.
- Списки стоп-слов служат для исключения определенных слов из аргументов поиска и полнотекстового индекса.

Основные термины

Понятны ли вам значения приведенных здесь основных терминов? Чтобы проверить свои ответы, загляните в глоссарий на прилагаемом компакт-диске.

- Полнотекстовый каталог ~ full text catalog.
- Полнотекстовый фильтр ~ full text filter.
- Полнотекстовый индекс ~ full text index.
- Парадигматический модуль ~ stemmer.
- Список стоп-слов ~ stop list.
- Файл тезауруса ~ thesaurus file.
- Средство разбиения по словам ~ word breaker.

Упражнения

Сейчас вы примените на практике знания и навыки, полученные при изучении этой главы. Ответы на вопросы см. в разделе «Ответы» в конце книги.

Упражнение 1. Установка и настройка экземпляра SQL Server 2008

Компания Wide World Importers собирается внедрять новый набор приложений для управления несколькими направлениями деятельности. Нужно обеспечить возможность хранить большой объем данных в корпоративном вычислительном центре и доступ к ним из любой точки земного шара.

Нескольким менеджерам требуется доступ к оперативным отчетам о текущей нагрузке своих сотрудников, а также о новых и незавершенных заказах клиентов. Еще им необходим доступ к большим объемам архивных данных для определения тенденций, оптимизации штата и уровня складских запасов.

Менеджеры хотят разместить руководства по использованию продуктами на веб-сайте компании, а не поставлять их в составе самих продуктов. Пользователям нужно предоставить возможность просматривать руководства по названию продуктов и искать руководства по определенному тексту в них. Менеджеры по продажам также требуют усовершенствовать веб-сайт компании: описания продуктов должны быть доступны на нескольких языках с возможностью поиска.

В отделе продаж работает большое количество менеджеров по продажам, которые постоянно звонят клиентам, и им нужен доступ к данным по клиентам, которых обслуживает тот или иной менеджер, а также к информации о потенциальных клиентах. Доступ к информации менеджерам по продажам нужен даже в отсутствие подключения к Интернету или корпоративной сети. Периодически менеджеры по продажам подключаются к корпоративной сети и синхронизируют свои данные с корпоративной базой данных.

Средствами Microsoft Visual Studio .NET был разработан ряд приложений для работы в среде Microsoft Windows, а весь доступ к данным осуществляется через хранимые процедуры. Набор приложений одинаков как у пользователей, подключенных напрямую к корпоративному серверу баз данных, так и у торговых представителей, подключающихся к своим локальным серверам баз данных.

1. Какие возможности SQL Server 2008 следует задействовать для хранения руководств по использованию продуктов?

2. Что следует настроить, чтобы пользователи могли выполнять поиск по руководствам?
3. Какие объекты нужно настроить, чтобы обеспечить максимальное качество результатов поиска?

Рекомендуемые упражнения

Чтобы подготовиться к экзамену по темам этой главы, выполните следующие задания.

Создание полнотекстового индекса

- Создайте полнотекстовый индекс на основе большого объема тестовых данных.

Запрос с использованием полнотекстового индекса

- Выполните различные запросы данных, на основе которых создан полнотекстовый индекс, используя *CONTAINS*, *CONTAINSTABLE*, *FREETEXT* и *FREETEXTTABLE*, и сравните результаты с тем, что вы ожидали получить.

Создание файла тезауруса

- Заполните файл тезауруса, содержащий слова-заменители или синонимы. Выполните дополнительные запросы, чтобы изучить эффект от изменений.

Создание списка стоп-слов

- Создайте список стоп-слов, чтобы исключить из поиска часто встречающиеся слова и проверьте эффект от него при попытке поиска исключенных слов.

Пробный экзамен

На прилагаемом компакт-диске содержится пробный экзамен, обеспечивающий широкие возможности самоподготовки. Например, можно проверить, как усвоены темы этой главы либо все темы экзамена 70-432. Можно симитировать настоящий экзамен или выбрать режим обучения, когда после каждого вопроса приводятся правильные ответы с пояснениями.

ПРИМЕЧАНИЕ Пробный экзамен

Подробнее о пробном экзамене см. «Введение».

Распределение и секционирование данных

Занятие 1. Создание функции секционирования	124
Занятие 2. Создание схемы секционирования	129
Занятие 3. Создание секционированных таблиц и индексов	133
Занятие 4. Управление секциями	136

Секционирование (partitioning) таблиц появилось в Microsoft SQL Server 2005 как механизм, позволяющий распределять крупные таблицы между несколькими структурами хранения данных. Раньше каждый объект должен был храниться только в одной файловой группе, которая могла содержать несколько файлов. Однако размещение данных в файловой группе все равно определялось самим SQL Server.

Секционирование таблиц позволят создавать таблицы, индексы и индексированные представления, хранящиеся в нескольких файловых группах, причем администратор баз данных (DBA) может указывать, в какой файловой группе будет храниться та или иная часть объекта.

Секционированную таблицу, индекс или индексированное представление создают в несколько этапов:

1. Создают функцию секционирования.
2. Создают схему секционирования, которой соответствует функция секционирования.
3. Создают таблицу, индекс или индексированное представление, используя эту схему секционирования.

Темы экзамена

- Управление секционированием данных.

Прежде всего

Для изучения материалов этой главы необходимо иметь:

- экземпляр SQL Server 2008 выпуска Enterprise, Developer или Evaluation.

Практический совет*Майк Хотек*

У одной из обслуживаемых мной компаний возникли серьезные проблемы, связанные с конфликтами доступа к данным на рабочих серверах. Из-за этих конфликтов клиенты компании иногда даже не могли войти в систему, чтобы выполнить платеж, просмотреть остатки на счетах или выполнить другие операции. Удалось выяснить, что конфликты возникают из-за операций архивирования, выполняемых в соответствии с новыми политиками сохранения данных, создание которых обусловили новые требования нового закона — закона Сарбейнса-Оксли (SOX). Несмотря на все усилия администраторам не удалось полностью исключить отрицательное влияние процесса архивирования на работу клиентов. Ежедневные операции архивирования выполнялись 3–4 часа, а еженедельное архивирование данных аудита — целых 22 часа.

Чтобы решить проблему конфликтов, мы секционировали все таблицы, данные которых архивировались в соответствии с новыми политиками. Затем мы реализовали новый процесс, в котором для перемещения сегментов данных из OLTP-таблиц в набор промежуточных таблиц использовались операции для работы с секционированными данными — *SPLIT*, *MERGE* и *SWITCH*. В результате нам удалось не только сократить время архивирования с часов до менее чем 5 секунд, но и полностью избавиться от конфликтов при обращениях к таблицам.

Занятие 1. Создание функции секционирования

Функция секционирования задает набор граничных точек, которые делят данные на секции. Здесь рассказывается, как выполнить первый этап секционирования объекта — создание функции секционирования.

Изучив материал этого занятия, вы сможете:

- ✓ создавать функции секционирования.

Продолжительность занятия — 15 минут.

Функции секционирования

Функция секционирования определяет граничные точки, используемые для разбиения данных на секции в соответствии со схемой секционирования. На рис. 6-1 приведен пример простейшей секционированной таблицы.

Вот пример функции секционирования:

```
CREATE PARTITION FUNCTION  
mypartfunction (int)  
AS RANGE LEFT  
FOR VALUES (10, 20, 30, 40, 50, 60)
```

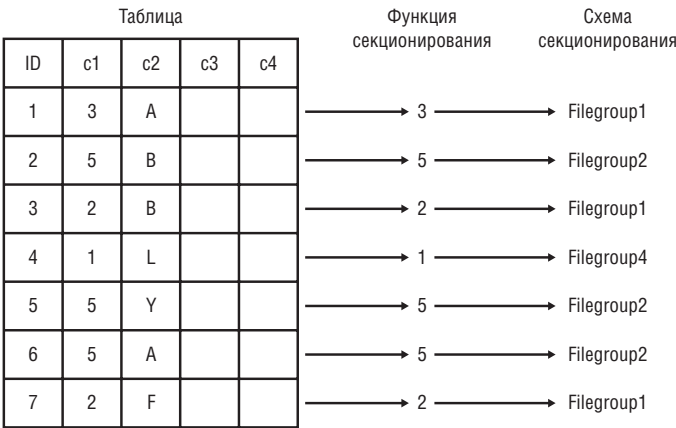


Рис. 6-1. Секционированная таблица

При определении любой функции секционирования задают ее имя и тип данных. Тип данных определяет, в каких пределах лежат граничные точки, используемые при секционировании, и должен определять такой же или менее широкий диапазон данных, что и у типа данных столбца таблицы, индекса или индексированного представления, к которому применяется функция секционирования.

Типом данных функции секционирования может быть любой собственный тип данных SQL Server, за исключением *text*, *ntext*, *image*, *varbinary(max)*, *timestamp*, *xml* и *varchar(max)*. Кроме того, нельзя указывать пользовательские типы данных Transact-SQL и типы данных CLR (Common Language Runtime). Столбцы неточных типов, например вычисляемые или *real*, должны быть материализованными (*persisted*). Столбцы, используемые при секционировании, должны быть детерминированными (*deterministic*).

Предложение *AS* позволяет определить создаваемую функцию секционирования как *RANGE LEFT* или *RANGE RIGHT*. Параметры *LEFT* и *RIGHT* задают, какая секция содержит граничную точку.

Предложение *FOR VALUES* служит для указания граничных точек функции секционирования. Если функция секционирования определена как *RANGE LEFT*, то граничная точка принадлежит секции, находящейся слева от нее, а если функция секционирования определена как *RANGE RIGHT* — секции, находящейся справа.

Функция секционирования всегда сопоставляет весь диапазон данных, без пропусков. Нельзя указывать одинаковые граничные точки. Таким образом, гарантируется, что любому значению столбца всегда соответствует единственная секция. Значения *null* всегда хранятся в секции, крайней слева, если только вы явно не укажете граничную точку *null* и предложение *RANGE RIGHT*, — тогда они будут храниться в самой правой секции.

Поскольку функция секционирования всегда сопоставляется всему диапазону значений, количество создаваемых секций на единицу превышает количество граничных точек. В табл. 6-1 показано, какие диапазоны значений соответствуют такой функции секционирования:

```
CREATE PARTITION FUNCTION
mypartition (int)
AS RANGE LEFT
FOR VALUES (10, 20, 30, 40, 50, 60)
```

Табл. 6-1. Диапазоны значений функции секционирования *RANGE LEFT*

Номер секции	Минимальное значение	Максимальное значение
1	$-\infty$	10
2	11	20
3	21	30
4	31	40
5	41	50
6	51	60
7	61	$+\infty$

ПРИМЕЧАНИЕ Повторное использование кода

В определениях функций секционирования отсутствуют предложения, задающие объект, столбец или хранилище. Таким образом функция секционирования представляет собой отдельный объект, который можно применять к различным таблицам, индексам или индексированным представлениям по своему усмотрению.

В табл. 6-2 показано, как изменятся разделы, если определить функцию секционирования как *RANGE RIGHT*:

```
CREATE PARTITION FUNCTION
mypartition (int)
AS RANGE RIGHT
FOR VALUES (10, 20, 30, 40, 50, 60)
```

Табл. 6-2. Диапазоны значений функции секционирования *RANGE RIGHT*

Номер секции	Минимальное значение	Максимальное значение
1	$-\infty$	9
2	10	19
3	20	29
4	30	39
5	40	49
6	50	59
7	60	$+\infty$

У объекта может быть не более 1000 секций, поэтому возможно определить не более 999 граничных точек.

ПРИМЕЧАНИЕ На экзамене

Можно секционировать существующие объекты, уже хранящие данные. Чтобы секционировать существующую таблицу, удалите кластеризованный индекс и пересоздайте его, используя схему секционирования. Чтобы секционировать существующий индекс или индексированное представление, удалите индекс и пересоздайте его, используя схему секционирования. При секционировании существующих объектов, уже содержащих данные, необходимо быть крайне осторожным, так как при секционировании выполняется значительное количество операций дискового ввода/вывода.

Контрольные вопросы

1. Какие типы данных нельзя использовать в функциях секционирования?
2. Какое максимальное количество секций допускается для таблиц?
3. Какое максимальное количество граничных точек можно указывать в функциях секционирования?

Ответы на контрольные вопросы

1. Нельзя использовать типы данных *text*, *ntext*, *image*, *xml*, *varbinary(max)*, *varchar(max)*, а также любые типы данных CLR.
2. Максимальное количество секций таблицы — 1000.
3. Максимальное количество граничных точек функции секционирования — 999.

Практикум. Создание функции секционирования

Вы создадите тестовую базу данных, а затем сформируете в ней функцию секционирования.

1. В Проводнике Windows (Windows Explorer) создайте каталог *C:\Test* (если он еще не существует).
2. Откройте новое окно запроса в Среде SQL Server Management Studio (SQL Server Management Studio).
3. Выполните следующие инструкции создания тестовой базы данных:

```
-- Создаем базу данных с несколькими файловыми группами.  
USE master  
GO  
CREATE DATABASE partitiontest  
ON PRIMARY  
    (NAME = primary_data, FILENAME = 'c:\test\db.mdf',  
     SIZE = 2MB),  
FILEGROUP FG1  
    (NAME = FG1_data, FILENAME = 'c:\test\FG1.ndf',  
     SIZE = 2MB),  
FILEGROUP FG2
```

```
(NAME = FG2_data, FILENAME = 'c:\test\FG2.ndf',
    SIZE = 2MB),
FILEGROUP FG3
    (NAME = FG3_data, FILENAME = 'c:\test\FG3.ndf',
        SIZE = 2MB),
FILEGROUP FG4
    (NAME = FG4_data, FILENAME = 'c:\test\FG4.ndf',
        SIZE = 2MB),
FILEGROUP FG5
    (NAME = FG5_data, FILENAME = 'c:\test\FG5.ndf',
        SIZE = 2MB),
FILEGROUP FG6
    (NAME = FG6_data, FILENAME = 'c:\test\FG6.ndf',
        SIZE = 2MB),
FILEGROUP FG7
    (NAME = FG7_data, FILENAME = 'c:\test\FG7.ndf',
        SIZE = 2MB),
FILEGROUP FG8
    (NAME = FG8_data, FILENAME = 'c:\test\FG8.ndf',
        SIZE = 2MB),
FILEGROUP FG9
    (NAME = FG9_data, FILENAME = 'c:\test\FG9.ndf',
        SIZE = 2MB),
FILEGROUP FG10
    (NAME = FG10_data, FILENAME = 'c:\test\FG10.ndf',
        SIZE = 2MB),
FILEGROUP FG11
    (NAME = FG11_data, FILENAME = 'c:\test\FG11.ndf',
        SIZE = 2MB),
FILEGROUP FG12
    (NAME = FG12_data, FILENAME = 'c:\test\FG12.ndf',
        SIZE = 2MB),
FILEGROUP FG13
    (NAME = FG13_data, FILENAME = 'c:\test\FG13.ndf',
        SIZE = 2MB)
LOG ON
    (NAME = db_log, FILENAME = 'c:\test\log.ndf',
        SIZE = 2MB, FILEGROWTH = 10% );
GO
USE partitiontest
GO
```

2. Создайте функцию секционирования с граничными точками, делящими год на месяцы:

```
-- Создаем функцию секционирования с граничными точками,
-- делящими год на месяцы
CREATE PARTITION FUNCTION partfunc (datetime) AS
RANGE RIGHT FOR VALUES ('1/1/2005', '2/1/2005', '3/1/2005',
```

```
'4/1/2005', '5/1/2005', '6/1/2005', '7/1/2005', '8/1/2005',  
'9/1/2005', '10/1/2005', '11/1/2005', '12/1/2005')  
GO
```

3. Выполните следующую команду, чтобы посмотреть результаты шага 4:

```
SELECT * FROM sys.partition_range_values;
```

Резюме занятия

- Функция секционирования определяет граничные точки для набора секций.
- Можно создать функцию секционирования *RANGE LEFT* или *RANGE RIGHT*.
- Можно указывать любые типы данных, кроме *text*, *ntext*, *image*, *varbinary(max)*, *varchar(max)*, *XML* и типов данных CLR.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Компания Contoso использует транзакционную систему, работающую с очень большими объемами данных. Памяти сервера баз данных не хватает для хранения активного набора данных, поэтому очень большое количество операций чтения и записи выполняется с непосредственным доступом к дискам. Добавление нескольких индексов не помогло добиться приемлемой производительности. К сожалению, администраторы не могут придумать, какие еще индексы добавить. Бюджет компании не позволяет приобрести дополнительную память или серверы или увеличить мощность серверов. Однако недавно в компании развернули сеть хранения данных (storage area network, SAN). Какая технология поможет компании Contoso увеличить производительность?
 - A. Доставка журналов.
 - B. Репликация.
 - C. Секционирование.
 - D. Зеркальное отображение баз данных.

Занятие 2. Создание схемы секционирования

Схема секционирования определяет структуры хранения и набор файловых групп, используемые в связке с той или иной функцией секционирования. Здесь рассказывается, как создавать схемы секционирования, сопоставляя секции файловым группам.

Изучив материал этого занятия, вы сможете:

- ✓ создавать схемы секционирования.

Продолжительность занятия — 10 минут.

Схемы секционирования

Схемы секционирования позволяют определить альтернативный порядок хранения объекта и служат для его распределения среди одной или нескольких файловых групп. Синтаксис создания схем секционирования таков:

```
CREATE PARTITION SCHEME имя_схемы_секционирования  
AS PARTITION имя_функции_секционирования  
[ ALL ] TO ( { имя_файловой_группы | [ PRIMARY ] } [ ,...n ] )
```

Вот три примера схем секционирования:

```
CREATE PARTITION SCHEME mypartscheme  
AS PARTITION mypartfunction T0 (Filegroup1, Filegroup2,  
Filegroup3, Filegroup4, Filegroup5, Filegroup6, Filegroup7)
```

```
CREATE PARTITION SCHEME mypartscheme  
AS PARTITION mypartfunction T0 (Filegroup1, Filegroup1,  
Filegroup2, Filegroup2, Filegroup3)
```

```
CREATE PARTITION SCHEME mypartscheme  
AS PARTITION mypartfunction ALL T0 (Filegroup1)
```

У каждой схемы секционирования должно быть имя, удовлетворяющее требованиям к идентификаторам. Предложение *AS PARTITION* задает имя функции секционирования, соответствующей схеме секционирования. В предложении *TO* указывают список файловых групп, включаемых в схему секционирования.

ВНИМАНИЕ! Файловые группы

Все файловые группы, указываемые в инструкции *CREATE PARTITION SCHEME*, должны уже существовать в базе данных.

Необходимо определять схему секционирования так, чтобы она содержала файловую группу для каждой секции, которая создается в соответствии с соответствующей схеме функцией секционирования. Как говорилось ранее, в SQL Server 2008 можно указать ключевое слово *ALL*, позволяющее создать все секции, заданные в функции секционирования, в одной файловой группе. Если ключевое слово *ALL* не указано, схема секционирования должна содержать как минимум одну файловую группу для каждой секции, определенной функцией секционирования. Например, если в функции секционирования определено шесть граничных точек (семь секций), то в схеме секционирования, которой она сопоставляется, необходимо определить как минимум семь файловых групп. Если количество файловых групп, указанных в схеме секционирования, превышает количество секций, то лишние файловые группы не будут использоваться для хранения данных, если только схему секционирования не изменить командой *ALTER PARTITION SCHEME*.

ПРИМЕЧАНИЕ На экзамене

Если при создании схемы секционирования указано ключевое слово *ALL*, можно задать только одну файловую группу.

В табл. 6-3 показано, какие файловые группы сопоставляются функции секционирования и схеме секционирования, определенным в следующем коде:

```
CREATE PARTITION FUNCTION
mypartfunction (int)
AS RANGE LEFT
FOR VALUES (10,20,30,40,50,60);
GO
CREATE PARTITION SCHEME mypartscheme
AS PARTITION mypartfunction
TO (Filegroup1, Filegroup2, Filegroup2, Filegroup4,
Filegroup5, Filegroup6, Filegroup7);
GO
```

Табл. 6-3. Схема секционирования, соответствующая функции секционирования

Файловая группа	Номер секции	Минимальное значение	Максимальное значение
Filegroup1	1	$-\infty$	10
Filegroup2	2	11	20
Filegroup2	3	21	30
Filegroup4	4	31	40
Filegroup5	5	41	50
Filegroup6	6	51	60
Filegroup7	7	61	$+\infty$

Контрольные вопросы

- 1. Сколько файловых групп можно задать, если при определении схемы секционирования используется ключевое слово *ALL*?
- 2. Можно ли создать новую файловую группу при создании схемы секционирования?

Ответы на контрольные вопросы

- 1. При использовании ключевого слова *ALL* можно указать только одну файловую группу.
- 2. Нет. Все файловые группы, указываемые в *CREATE PARTITION SCHEME*, должны уже существовать в базе данных.

Практикум. Создание схемы секционирования

Вы создадите схему секционирования, сопоставленную определенной в предыдущем упражнении функции секционирования.

- 1. Откройте в Среде SQL Server Management Studio (SQL Server Management Studio) новое окно запроса и измените контекст на базу данных *Partition-test*.
- 2. Выполните следующие инструкции, чтобы создать схему секционирования, соответствующую функции секционирования:

```
CREATE PARTITION SCHEME partscheme AS
```

```
PARTITION partfunc TO
([FG1], [FG2], [FG3], [FG4], [FG5], [FG6], [FG7],
 [FG8], [FG9], [FG10], [FG11], [FG12], [FG13])
GO
-- Отображаем схемы секционирования
SELECT * FROM sys.partition_schemes;
```

Резюме занятия

- Схема секционирования — это определение схемы хранения данных, задающее набор файловых групп.
- Если вы указали ключевое слово *ALL*, схема секционирования может содержать только одну файловую группу.
- Если ключевое слово *ALL* не указано, нужно предусмотреть как минимум одну файловую группу для каждой секции, определенной функцией секционирования.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Руководство компании Margie's Travel требует хранить в онлайн-системе обработки транзакций данные о заказах, сделанных не более 30 дней назад. Таблица данных о заказах содержит столбец *OrderDate*, в котором хранится дата размещения заказа. Как администратор Margie's Travel должен организовать перенос данных о заказах, сделанных более 30 дней назад, из соответствующей таблицы так, чтобы это оказало минимальное влияние на пользовательские транзакции? (Выберите два варианта, каждый из которых является верным ответом).
 - A. Организовать перенос секции данных, содержащие информацию о заказах более чем 30-дневной давности, операцией *SWITCH*.
 - B. Создать хранимую процедуру, удаляющую данные о заказах, сделанных более 30 дней назад.
 - C. Секционировать таблицу с данными о заказах по столбцу *OrderDate*, используя функцию секционирования, определенную для типа данных *datetime*.
 - D. Создать задание, удаляющее данные о заказах, сделанных более 30 дней назад.

Занятие 3. Создание секционированных таблиц и индексов

Завершающий этап создания секционированной таблицы или индекса — создание таблицы или индекса, разбитого на секции в соответствии со схемой секционирования и значениями столбца секционирования. Здесь рассказывается о том, как создавать секционированные таблицы и индексы.

Изучив материал этого занятия, вы сможете:

- ✓ создавать секционированные таблицы;
- ✓ создавать секционированные индексы.

Продолжительность занятия — 10 минут.

Создание секционированной таблицы

Создание секционированной таблицы, индекса или индексируемого представления очень похоже на создание обычной, несекционированной таблицы, индекса или индексируемого представления. При создании каждого объекта можно указать предложение *ON*, в котором определяется, где SQL Server должен хранить объект. Обычно предложение *ON* опускают, и тогда SQL Server создает объект в файловой группе по умолчанию. Поскольку само определение того, как хранится объект, содержится в схеме секционирования, секционирование таблицы, индекса или индексируемого представления — очень простая операция.

Вот пример создания секционированной таблицы:

```
CREATE TABLE Employee (EmployeeID int NOT NULL,  
                        FirstName varchar(50) NOT NULL,  
                        LastName varchar(50) NOT NULL)  
ON mypartscheme(EmployeeID);  
GO
```

Здесь ключевую роль играет предложение *ON*. Вместо файловой группы, в которой создается таблица, в нем указана схема секционирования. Вы уже определили схему секционирования, сопоставленную функции секционирования. При создании таблицы указывается ключ секционирования — столбец, к которому будет применяться функция секционирования. В предыдущем примере мы создали таблицу *Employee* и секционировали ее по столбцу *EmployeeID*, основываясь на определении функции секционирования, сопоставленной схеме секционирования, в соответствии с которой будет храниться таблица. В табл. 6-4 показано, как секционируются данные таблицы *Employee*, определенной в следующем коде:

```
CREATE PARTITION FUNCTION  
mypartfunction (int)  
AS RANGE LEFT  
FOR VALUES (10,20,30,40,50,60);  
GO
```

```
CREATE PARTITION SCHEME mypartscheme
AS PARTITION mypartfunction
TO (Filegroup1, Filegroup2, Filegroup3, Filegroup4,
Filegroup5, Filegroup6, Filegroup7);
GO
CREATE TABLE Employee (EmployeeID      int          NOT NULL,
                        FirstName         varchar(50) NOT NULL,
                        LastName          varchar(50) NOT NULL)
ON mypartscheme(EmployeeID);
GO
```

Табл. 6-4. Схема секционирования, сопоставленная функции секционирования

Файловая группа	Номер секции	Минимальное значение	Максимальное значение
Filegroup1	1	−∞	10
Filegroup2	2	11	20
Filegroup3	3	21	30
Filegroup4	4	31	40
Filegroup5	5	41	50
Filegroup6	6	51	60
Filegroup7	7	61	+∞

Тип данных, длина и точность ключа секционирования должны соответствовать функции секционирования. Если ключ секционирования — вычисляемый столбец, то он должен иметь свойство *PERSISTED*.

ПРИМЕЧАНИЕ Частичное резервное копирование и восстановление

Секционирование предоставляет интересные возможности управления таблицами и индексами. По определению функции секционирования и схемы секционирования можно определить, какой набор записей хранится в той или иной файловой группе. Используя эту информацию, можно выполнять резервное копирование и восстановление фрагментов таблицы, не влияя на другие части таблицы.

Создание секционированного индекса

Как и при создании секционированной таблицы, для секционирования индекса указывают схему секционирования в предложении *ON*, например:

```
CREATE NONCLUSTERED INDEX idx_employeefirstname
ON dbo.Employee(FirstName) ON mypartscheme(EmployeeID);
GO
```

Ключом секционирования для индекса не обязательно должен быть столбец, по которому определен индекс. Как рассказывалось в главе 4, при определении индексов можно указывать необязательное предложение *INCLUDE*. Когда вы создаете индекс секционированной таблицы, SQL Server автоматически включает ключ секционирования в определение каждого индекса, что позволяет секционировать индекс так же, как таблицу.

Контрольные вопросы

1. Какое свойство должен иметь вычисляемый столбец, чтобы он мог быть ключом секционирования?
2. Какое предложение инструкции *CREATE TABLE* или *CREATE INDEX* используется для секционирования объекта?

Ответы на контрольные вопросы

1. Вычисляемый столбец должен иметь свойство *PERSISTED*.
2. Для указания того, как будет храниться таблица или индекс (в файловой группе или по схеме секционирования), служит предложение *ON*.

Практикум. Секционирование таблицы

Вы создадите секционированную таблицу, воспользовавшись функцией секционирования и схемой секционирования, созданными в предыдущих упражнениях.

1. Откройте в Среде SQL Server Management Studio (SQL Server Management Studio) новое окно запроса и измените контекст на базу данных *Partitiontest*.
2. Создайте таблицу для хранения данных о заказах, использующую схему секционирования:

```
CREATE TABLE dbo.orders (  
    OrderID      int          identity(1,1),  
    OrderDate    datetime     NOT NULL,  
    OrderAmount  money        NOT NULL  
    CONSTRAINT pk_orders  
        PRIMARY KEY CLUSTERED (OrderDate, OrderID))  
ON partscheme(OrderDate)  
GO
```

3. Занесите в таблицу данные о заказах, выполнив следующий код:

```
SET NOCOUNT ON  
DECLARE @month int,  
        @day int  
  
SET @month = 1  
SET @day = 1  
  
WHILE @month <= 12  
BEGIN  
    WHILE @day <= 28  
    BEGIN  
        INSERT dbo.orders (OrderDate, OrderAmount)  
        SELECT cast(@month as varchar(2)) + '/'  
            + cast(@day as varchar(2))  
            + '/2005', @day * 20  
    END  
END
```

```
SET @day = @day + 1
END

SET @day = 1
SET @month = @month + 1
END
GO
```

4. Посмотрите распределение данных, выполнив следующую инструкцию:

```
SELECT * FROM sys.partitions
WHERE object_id = OBJECT_ID('dbo.orders')
```

Резюме занятия

- Для указания структуры хранения, файловой группы или схемы секционирования таблицы или индекса служит предложение *ON*.
- Тип данных, длина и точность ключа секционирования должны соответствовать функции секционирования.
- Вычисляемый столбец, используемый в качестве ключа секционирования, должен быть материализованным (*PERSISTED*).

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. У компании Wide World Importers имеется очень большое и активно используемое хранилище данных, которое должно быть доступно 24 часа в сутки, 7 дней в неделю. Для поддержки бизнес-операций необходимо, чтобы группа администраторов еженедельно загружала новые наборы данных, причем добавление большого количества данных не должно мешать работе пользователей. Какими средствами следует воспользоваться, чтобы обеспечить еженедельную загрузку данных и при этом создать минимум проблем пользователям?
 - A. Репликация транзакций.
 - B. Применение операции *SWITCH* к секционированным данным.
 - C. Зеркальное отображение баз данных.
 - D. Моментальные снимки баз данных.

Занятие 4. Управление секциями

После секционирования таблицы или индекса SQL Server автоматически хранит данные в соответствии с требованиями функции и схемы секционирования. Со временем требования к секционированию данных могут измениться. Здесь рассказывается, как вносить изменения в определения функций секционирования и схем секционирования и управлять секциями объектов баз данных.

Изучив материал этого занятия, вы сможете:

- ✓ добавлять и удалять граничные точки функций секционирования;
- ✓ добавлять файловые группы в схемы секционирования;
- ✓ помечать файловые группы как группы, в которых будет храниться следующая созданная секция;
- ✓ переносить секции из одной таблицы в другую.

Продолжительность занятия — 20 минут.

Операции **SPLIT** и **MERGE**

Когда данные постоянно меняются, структура секций редко остается статичной. Для управления изменениями граничных точек служат две операции — *SPLIT* и *MERGE*.

Операция *SPLIT* добавляет граничную точку в функцию секционирования. *MERGE* удаляет граничную точку из функции секционирования. Синтаксис этих операций таков:

```
ALTER PARTITION FUNCTION имя_функции_секционирования()  
{SPLIT RANGE ( значение_граничной_точки )  
 | MERGE RANGE (значение_граничной_точки ) } [ ; ]
```

При использовании *SPLIT* и *MERGE* необходимо быть крайне осторожным, ведь вы добавляете или удаляете из функции секционирования целую секцию. Эти операции не модифицируют данные таблицы, но изменяют их распределение по секциям. Поскольку секция может храниться только в одной файловой группе, выполнение *SPLIT* или *MERGE* может приводить к большому количеству операций дискового ввода-вывода, так как SQL Server будет считывать записи с диска и переносить их в другое место.

Изменение схемы секционирования

Можно добавлять файловые группы в существующую схему секционирования, чтобы выделять дополнительное пространство для хранения данных секционированной таблицы. Вот синтаксис инструкции, которая это делает:

```
ALTER PARTITION SCHEME имя_схемы_секционирования  
NEXT USED [ имя_файловой_группы ] [ ; ]
```

Предложение *NEXT USED* служит для выполнения двух задач.

1. Добавляет файловую группу в схему секционирования, если указанная в нем файловая группа еще не входит в схему секционирования.
2. Задает свойство *NEXT USED* файловой группы.

После выполнения операции *SPLIT* помеченная флагом *NEXT USED* файловая группа будет содержать следующую секцию.

Выравнивание индексов

Таблицу и связанные с ней индексы можно секционировать по-разному. Единственное исключение — секционирование таблицы и ее кластеризованного

индекса должно быть одинаковым, потому что SQL Server не может хранить таблицу и ее кластеризованный индекс в разных структурах.

Если таблица и ее индексы секционированы с применением одной функции секционирования, говорят, что они *выровнены* (aligned). Если таблица и все ее индексы используют одну функцию секционирования и одну схему секционирования, их структура хранения также называется выровненной (рис. 6-2).

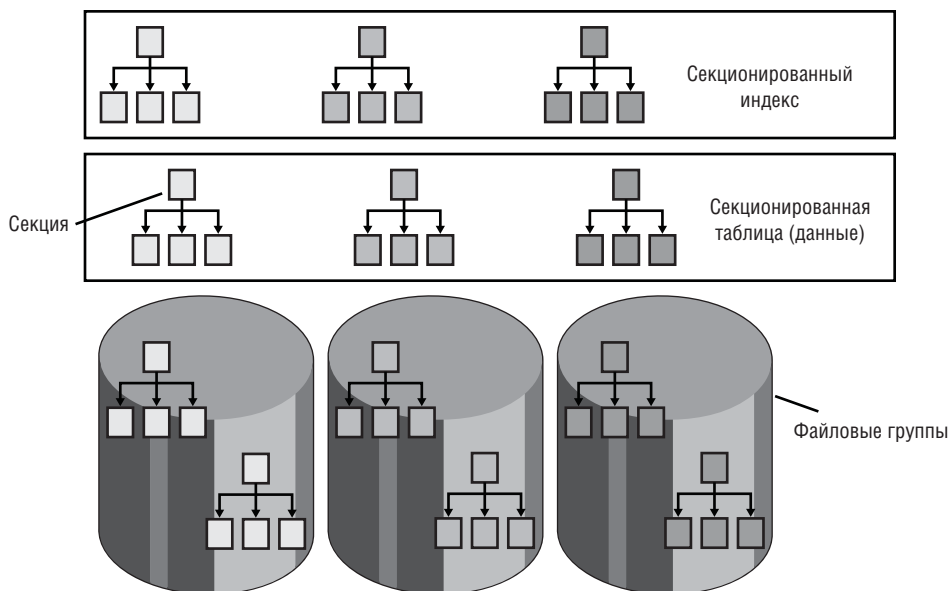


Рис. 6-2. Выровненная структура хранения

Когда структура хранения выровнена, записи таблицы и соответствующие им значения индексов хранятся в одних файловых группах. В результате при резервном копировании или восстановлении одной секции данные и соответствующие им индексные значения будут храниться вместе, в одной копии.

Операция SWITCH

Скорее всего, на данный момент секционирование вызывает у вас много вопросов. Понятно, что цель секционирования — распределить таблицу и связанные с ней индексы среди нескольких структур хранения, а предназначение каждой операции — управление несколькими структурами хранения. Однако секционирование предоставляет расширенные возможности управления данными, выходящие далеко за рамки простого хранения части таблицы в файловой группе. Чтобы понять, что происходит при секционировании, необходимо вернуться на шаг назад и вспомнить основы хранения данных в SQL Server.

SQL Server хранит данные в страницах в виде двунаправленного списка. Поиск данных и обращение к ним SQL Server выполняет в следующей последовательности:

1. Разрешает имя таблицы в идентификатор объекта.
2. Находит запись для данного идентификатора объекта в *sysindexes* чтобы получить указатель на первую страницу объекта.

- 3. Считывает первую страницу объекта.
- 4. Ориентируясь по значениям записей Next Page (следующая страница) и Previous Page (предыдущая страница) каждой страницы данных, проходит цепочку страниц и находит требуемые данные.

У первой страницы объекта нет предыдущей страницы; поэтому ее запись «предыдущая страница» содержит значение 0:0. У последней страницы объекта нет следующей страницы, поэтому запись «следующая страница» содержит 0:0. Когда обнаруживается запись «следующая страница» со значением 0:0, SQL Server прекращает чтение данных.

Что произойдет с цепочкой страниц при секционировании? При секционировании таблицы производится физическая сортировка данных, их разбиение на секции и сохранение в файловых группах. С точки зрения цепочек страниц, SQL ищет первую страницу объекта в секции 1, проходит цепочку страниц, достигает последней страницы секции 1, которая указывает на первую страницу секции 2, продолжает этот цикл до конца таблицы образом. Физическое упорядочение данных открывает весьма интересные возможности.



Рис. 6-3. Двухнаправленный список

Если изменить значение указателя на следующую страницу в последней странице секции 1 на 0:0, SQL Server прекратит считывание данных, достигнув этой

точки, и оставшиеся данные как бы «исчезнут» из таблицы. Не будет происходить никаких блокировок или взаимоблокировок, поскольку при обновлении указателя на страницу выполняется простая операция, затрагивающая только метаданные. Общая идея такой операции над метаданными показана на рис. 6-3.

Было бы здорово иметь возможность удалить часть таблицы таким простым способом. Однако SQL Server не позволяет «взять и выкинуть» данные. В таких случаях применяется операция *SWITCH*. Основное предназначение *SWITCH* — позволить осуществлять обмен секциями между таблицами, обеспечивая высокий уровень масштабирования, без каких-либо блокировок или взаимоблокировок.

SWITCH предъявляет несколько требований, призванных обеспечить максимально корректное масштабирование. Вот самые важные из них:

- данные и индексы исходной и целевой таблиц должны быть выровнены;
- исходная и целевая таблицы должны иметь одинаковую структуру;
- нельзя переносить данные из одной файловой группы в другую;
- нельзя менять местами две секции, содержащие данные;
- целевая секция должна быть пустой;
- исходная и целевая таблицы не должны участвовать в репликации;
- исходная и целевая таблицы не могут иметь полнотекстовые индексы или содержать данные типа *FILESTREAM*.

Выполнив эти требования, можно добиться такого эффекта, как на рис. 6-4.

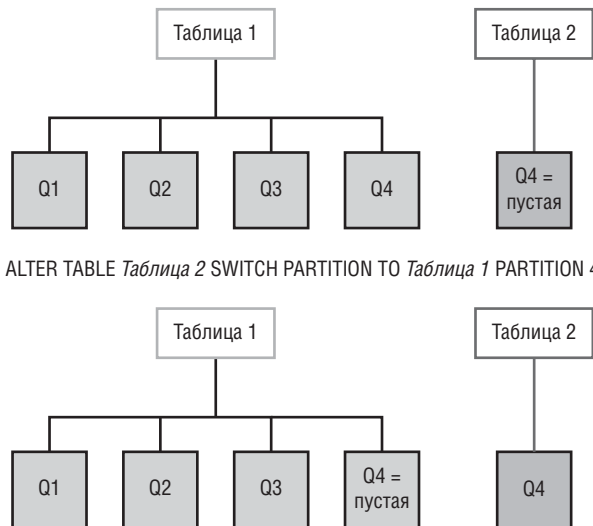


Рис. 6-4. Переключение секций

ПРИМЕЧАНИЕ На экзамене

На экзамене 70-432 необходимо помнить, что нельзя переносить данные из одной файловой группы в другую или менять местами две секции с данными.

Контрольные вопросы

- 1. Какие операции служат для добавления или удаления граничных точек функции секционирования?
- 2. Какая операция служит для перемещения секций между таблицами?

Ответы на контрольные вопросы

- 1. Операция *SPLIT* служит для добавления граничной точки, а операция *MERGE* — для удаления.
- 2. Операция *SWITCH* предназначена для перемещения секций между таблицами.

Практикум. «Скользящее окно»

С помощью *SPLIT*, *MERGE* и *SWITCH* вы удалите данные из таблицы, с которой работают пользователи, и перенесете их в архив, не повлияв на производительность запросов к ней. На практикуме к занятию 3 вы занесли в таблицу с данными о заказах информацию о заказах за 12 месяцев. Теперь вы создадите секцию для данных за январь 2006 с помощью операции *SPLIT*. Затем с помощью операции *SWITCH* вы удалите секцию за январь 2005, перенеся ее в архив. Наконец, вы удалите граничную точку для января 2005, выполнив операцию *MERGE*.

Данные о заказах должны иметь следующий вид, показанный в табл. 6-5.

Табл. 6-5. Распределение информации в таблице данных о заказах

Файловая группа	Минимальная дата	Максимальная дата
FG1	$-\infty$	31.12.2004
FG2	01.01.2005	31.01.2005
FG3	01.02.2005	28.02.2005
FG4	01.03.2005	31.03.2005
FG5	01.04.2005	30.04.2005
FG6	01.05.2005	31.05.2005
FG7	01.06.2005	30.06.2005
FG8	01.07.2005	31.07.2005
FG9	01.08.2005	31.08.2005
FG10	01.09.2005	30.09.2005
FG11	01.10.2005	31.10.2005
FG12	01.11.2005	30.11.2005
FG13	01.12.2005	$+\infty$

- 1. Измените схему секционирования, установив флаг *NEXT USED* для *FG1*:

```
ALTER PARTITION SCHEME partscheme
NEXT USED [FG1];
GO
```

2. Добавьте граничную точку для января 2006:

```
ALTER PARTITION FUNCTION partfunc()  
SPLIT RANGE ('1/1/2006');  
GO
```

3. Создайте архивную таблицу для информации о заказах за январь 2005:

```
CREATE TABLE dbo.ordersarchive (  
    OrderID      int          NOT NULL,  
    OrderDate    datetime    NOT NULL  
        CONSTRAINT ck_orderdate CHECK (OrderDate<'2/1/2005'),  
    OrderAmount  money NOT NULL  
        CONSTRAINT pk_ordersarchive  
        PRIMARY KEY CLUSTERED (OrderDate,OrderID)  
)  
ON FG2  
GO
```

4. С помощью инструкции *SWITCH* отсоедините секцию за январь 2005 от таблицы *orders* и присоедините ее к таблице *ordersarchive*:

```
ALTER TABLE dbo.orders  
SWITCH PARTITION 2 TO dbo.ordersarchive  
GO
```

5. Удалите граничную точку для января 2005:

```
ALTER PARTITION FUNCTION partfunc()  
MERGE RANGE ('1/1/2005');  
GO
```

6. Убедитесь, что таблицы *orders* и *ordersarchive* содержат правильные данные о заказах.

Резюме занятия

- Операция *SPLIT* служит для добавления граничной точки в функцию секционирования.
- Операция *MERGE* служит для удаления граничной точки из функции секционирования.
- Операция *SWITCH* служит для перемещения секций между таблицами.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. В компании Contoso Limited используется система ввода информации о заказах, хранящая очень большие объемы данных. Руководство решило, что данные о заказах должны храниться в производственной системе не более полугода, после чего их нужно переносить в архив. При архивировании данные удаляются из таблицы и загружаются в хранилище данных. Такая

выгрузка данных будет осуществляться раз в месяц. Какая технология лучше всего подходит для архивирования данных системы ввода информации о заказах?

- A. Зеркальное отображение баз данных.
- B. Репликация транзакций.
- C. Моментальные снимки баз данных.
- D. Секционирование.

Закрепление материала главы

Для закрепления знаний и навыков, полученных при изучении этой главы, выполните следующее:

- изучите резюме главы;
- повторите основные термины, использованные в этой главе;
- выполните задания по темам этой главы;
- сделайте предложенные упражнения;
- пройдите пробный экзамен.

Резюме главы

- Секционирование позволяет распределить таблицу или индекс среди нескольких файловых групп.
- Таблицы и индексы секционируются горизонтально, т. е. разбиение выполняется по строкам, а критерием служит определенный для таблицы или индекса столбец секционирования).
- Для создания секционированной таблицы или индекса необходимо выполнить следующие операции:
 - создать функцию секционирования;
 - создать схему секционирования, соответствующую функции секционирования;
 - создать таблицу или индекс по схеме секционирования.
- Функция *\$PARTITION* позволяет при выполнении запросов отбирать только те записи, которые находятся в заданной секции.
- Операция *SPLIT* служит для добавления граничной точки и, следовательно, добавления секции.
- Операция *MERGE* служит для удаления граничной точки и, следовательно, удаления секции.
- Операция *SWITCH* служит для перемещения секций данных между таблицами.

Основные термины

Понятны ли вам значения приведенных здесь основных терминов? Чтобы проверить свои ответы, загляните в глоссарий на прилагаемом компакт-диске.

- Index alignment ~ выравнивание индекса (с таблицей)

- Функция секционирования ~ partition function.
- Ключ секционирования ~ partitioning key.
- Схема секционирования ~ partition scheme.

Упражнения

Сейчас вы примените на практике знания и навыки, полученные при изучении этой главы. Ответы на вопросы см. в разделе «Ответы» в конце книги.

Упражнение 1. Проектирование инфраструктуры SQL Server

Исходные данные

Основные сведения о компании

Компания Coho Vineyard была основана в 1947 г. как частный семейный винный завод. Вина, производимые Coho Vineyard, удостоивались многих наград и пользовались популярностью, что позволило компании значительно расширить производство. Сейчас компания владеет 12 винными заводами, расположенными в верхней части Среднего Запада США. В компании работает 400 человек, из которых 74 — в штаб-квартире, где размещаются критические важные для бизнеса серверы.

Планируемые изменения

До настоящего времени у каждого из 12 заводов Coho Vineyard был собственный локальный веб-сайт. Компания Coho Vineyard планирует объединить веб-ресурсы всех заводов, чтобы клиенты могли купить продукцию любого из 12 заводов в едином онлайн-магазине. Все данные этого веб-сайта будут храниться в базе данных в штаб-квартире.

Существующая инфраструктура данных

Базы данных

Сейчас каждый винный завод поддерживает собственную базу данных, где хранит всю бизнес-информацию. В конце месяца вся информация переносится в центральный офис и загружается в базы данных, перечисленные в табл. 6-6.

Табл. 6-6. Базы данных компании Coho Vineyard

База данных	Размер
Customer (данные о клиентах)	180 Мб
Accounting (данные бухучета)	500 Мб
HR (данные о сотрудниках)	100 Мб
Inventory (данные товарного учета)	250 Мб
Promotions (данные о рекламных мероприятиях)	80 Мб

После реализации проекта по консолидации баз данных добавится база данных *Order* (данные о заказах), обслуживающая новый Интернет-магазин. Со-

трудники также будут периодически обращаться к базе данных *Order* при выполнении своих повседневных обязанностей, используя новое веб-приложение, выполняемое в штаб-квартире.

Серверы баз данных

Все базы данных штаб-квартиры размещаются на единственном сервере DB1, на котором установлены операционная система Windows Server 2003 Enterprise Edition и SQL Server 2008 Enterprise Edition.

Бизнес-требования

Производительность

Требуется спроектировать решение по архивированию информации из баз данных *Customer* и *Order*. Стратегия архивирования должна позволять сохранять данные о клиентах (из базы данных *Customer*) за последние 6 лет.

Чтобы подготовить базу данных *Order* к архивированию, создана секционированная таблица *Order.Sales*, состоящая из двух секций. Секция 1 содержит данные о продажах за текущий месяц, а секция 2 — данные о продажах за предыдущий месяц. Данные о заказах, размещенных ранее предыдущего месяца, предполагается переносить в еще одну секционированную таблицу — *Order.Archive*. Секция 1 таблицы *Order.Archive* будет содержать все архивные данные, а секция 2 останется пустой (рис. 6-5).

Order.Sales

Секция 1	Секция 2
Данные за текущий месяц	Данные за предыдущий месяц

Order.Archive

Секция 1	Секция 2
Данные двухмесячной давности	(Пустая)

Рис. 6-5. Секционированные таблицы с данными о заказах

Для перемещения данных между этими секциями используются операции *SPLIT*, *MERGE* и *SWITCH*.

Ответьте на следующие вопросы:

1. Какой из перечисленных способов позволит выполнить требования к архивированию информации из базы данных *Customer* с минимальными затратами на администрирование?
 - A. Импортировать все данные из базы данных *Customer* в электронные таблицы Microsoft Office Excel. Хранить такие таблицы на диске в течение 6 лет.
 - B. Ежемесячно выполнять резервное копирование базы данных *Customer* на ленту. Хранить ленты с копиями в безопасном месте в течение 6 лет.
 - C. Создать новую базу данных *ArchiveData* и каждый месяц реплицировать данные из базы *Customer* в новую базу данных. Хранить все данные в базе *ArchiveData* как минимум 6 лет.
 - D. Не копировать данные из базы *Customer* в другие места. Просто хранить все данные в базе данных *Customer* в течение как минимум 6 лет.
2. Как перенести данные из Секции 2 таблицы Order.Sales в таблицу Order.Archive?
 - A. Перенести данные в Секцию 1 таблицы Order.Archive операцией *SPLIT*.
 - B. Перенести данные в Секцию 2 таблицы Order.Archive операцией *SPLIT*.
 - C. Перенести данные в Секцию 1 таблицы Order.Archive операцией *SWITCH*.
 - D. Перенести данные в Секцию 2 таблицы Order.Archive операцией *SWITCH*.
3. Какие две секции должны находиться в одной файловой группе?
 - A. Секции 1 и 2 таблицы Order.Sales.
 - B. Секции 1 и 2 таблицы Order.Archive.
 - C. Секция 2 таблицы Order.Sales и Секция 2 таблицы Order.Archive.
 - D. Секция 1 таблицы Order.Sales и Секция 1 таблицы Order.Archive.

Рекомендуемые упражнения

Чтобы подготовиться к экзамену по темам этой главы, выполните следующие задания.

Секционирование

Выполните следующие задания, чтобы закрепить навыки секционирования таблиц и архивирования и загрузки данных с помощью операций *SPLIT*, *MERGE* и *SWITCH*.

- **Задание 1.** Создайте секционированную таблицу и поэкспериментируйте с добавлением файловых групп, разбиением и объединением секций.
- **Задание 2.** Возьмите за основу таблицу из задания 1 и создайте архивную таблицу, в которую операция *SWITCH* перенесет удаляемые данные.
- **Задание 3.** Перенесите данные из исходной таблицы в архивную таблицу, созданную в задании 2, выполнив операцию *SWITCH*.

Пробный экзамен

На прилагаемом компакт-диске содержится пробный экзамен, обеспечивающий широкие возможности самоподготовки. Например, можно проверить, как усвоены темы этой главы либо все темы экзамена 70-432. Можно симитировать настоящий экзамен или выбрать режим обучения, когда после каждого вопроса приводятся правильные ответы с пояснениями.

ПРИМЕЧАНИЕ Пробный экзамен

Подробнее о пробном экзамене см. «Введение».

Импорт и экспорт данных

Занятие 1. Импорт и экспорт данных

149

Большинство приложений позволяют пользователям работать лишь с небольшими объемами данных. Однако бывают ситуации, когда необходимо экспортировать или импортировать значительный объем данных. При этом использование команды *INSERT* для такого импорта далеко не самый эффективный способ. Аналогично при экспорте нецелесообразно передавать данные приложению в виде результирующего набора данных для дальнейшей их записи, например, в файл. В этой главе рассказывается о возможностях SQL Server по эффективному импорту-экспорту больших объемов данных.

Темы экзамена

- Импорт и экспорт данных.

Прежде всего

Для изучения материалов этой главы необходимо иметь:

- установленный экземпляр Microsoft SQL Server 2008;
- подключенную базу данных AdventureWorks.

Практический совет

Майк Хотек

Пару лет назад я работал с заказчиком, одно из подразделений организации которого занималось обработкой заказов от партнерских фирм. Раз в день партнеры загружали файлы со своими заказами на наш FTP-сервер, которые далее проходили разборку, загружались в базу данных, после чего заказы направлялись на подбор, упаковку, доставку и оформление счетов. К сожалению, процесс обработки заказов каждого партнера занимал от двух до семи часов, в то время как соглашения с заказчиками требовали, чтобы обработка файлов от каждого партнера укладывалась в один час. Длина очереди на обработку составляла в среднем 30 и более файлов. Кроме того, только 5% партнеров была разрешена групповая загрузка данных, так как система не справлялась с дополнительной нагрузкой.

Система импорта заказов состояла из десятка приложений, написанных на C++, более 30 папок, распределенных по трем серверам, и небольшого объема кода базы данных, куда импортировались заказы. В процессе работы вся эта система, написанная около 10 лет назад, расходовала до 90% времени на перенос обрабатываемых фалов между каталогами, основная цель которого состояла в изоляции файлов на время обработки. При дальнейшем изучении системы был найден код управления несколькими приложениями, одновременно получающими доступ к одному и тому же файлу, что собственно и позволяло системе функционировать и справляться с задачей.

Мы обнаружили, что после того, как файлы наконец попадали в точку, где проходили обработку, они считывались и обрабатывались приложением строка за строкой. При обработке каждой строки выполнялось порядка 100 запросов для проверки кодов товара, его наличия, цен и других бизнес-правил.

Мы переписали всю систему так, чтобы максимально задействовать возможности SQL Server по массовому импорту данных. На первом этапе мы переписали весь код, написанный на C++, заменили всю структуру папок на один каталог и написали две хранимых процедуры — одну для обработки файлов утилитой BCP и вторую для обработки данных после импорта. В дальнейшем мы заменили процедуру с утилитой BCP на SSIS-пакет, позволяющий обрабатывать файлы параллельно и более гибко работать с файлами разных форматов.

По окончании первого этапа работы процедура импорта превосходила по скорости возможности партнеров по загрузке данных. В течение одной минуты после поступления файла данные импортировались в базу SQL Server, а заказы были обработаны и стояли в очереди на упаковку на складе. По окончании второго этапа работ система загрузки заказов обслуживала оставшихся 95% партнеров, причем время обработки файла с заказами составляло не более 15 секунд даже для самых больших файлов. Прямым результатом внедрения этой системы стало не только сохранение, но и расширение клиентской базы, что привело к росту оборотов на 400% в течение последующего полугодия.

Занятие 1. Импорт и экспорт данных

Основные задачи импорта-экспорта данных решаются с помощью команды *BULK INSERT* и утилиты командной строки BCP (Bulk Copy Program). На этом занятии вы научитесь использовать утилиту BCP как для импорта, так и экспорта данных. Кроме того вы узнаете как импортировать данные с помощью команды *BULK INSERT*, а также мастеров импорта и экспорта среды SQL Server Management Studio (SSMS).

Изучив материал этого занятия, вы сможете:

- ✓ экспортировать данные в файл с помощью утилиты BCP;
- ✓ импортировать данные из файла с помощью утилиты BCP;
- ✓ импортировать данные из файла с помощью команды *BULK INSERT*;
- ✓ импортировать и экспортировать данные с помощью мастеров.

Продолжительность занятия — 20 минут.

Утилита BCP

BCP.exe наверное самое старое средство в составе SQL Server. Эта утилита использовалась для импорта-экспорта данных еще до того, как Microsoft приобрела у Sybase первую версию SQL Server. Несмотря на все усовершенствования SQL Server за последние 20 лет синтаксис, назначение и производительность утилиты BCP не поменялись. Короче говоря, это наиболее эффективное средство импорта форматированных данных из файлов в базы SQL Server и экспорта таблиц из баз в файлы.

Утилита BCP — производительное и простое решение импорта-экспорта данных. BCP также служит для работы с форматированными данными. Если в процессе импорта-экспорта необходимо преобразование или обработка ошибок, лучше воспользоваться службами интеграции SQL Server Integration Services (SSIS).

При экспорте данных учетная запись, в контексте которой выполняется задача, должна иметь как минимум разрешение *SELECT* на таблицу или представление. При импорте данных необходимы разрешения *SELECT*, *INSERT* и *ALTER TABLE*.

Синтаксис утилиты BCP таков:

```
bcp {[[database_name.][owner].]{table_name | view_name} | «query»}  
{in | out | queryout | format} data_file  
[-mmax_errors] [-fformat_file] [-x] [-eerr_file]  
[-Ffirst_row] [-Llast_row] [-bbatch_size]  
[-n] [-c] [-w] [-N] [-V (60 | 65 | 70 | 80)] [-6]  
[-q] [-C { ACP | OEM | RAW | code_page } ] [-tfield_term]  
[-rrow_term] [-iinput_file] [-ooutput_file] [-apacket_size]  
[-Sserver_name[instance_name]] [-Ulogin_id] [-Ppassword]  
[-T] [-v] [-R] [-k] [-E] [-h»hint [,...n]»]
```

У BCP много параметров, но наиболее часто используется только небольшая их часть.

ВНИМАНИЕ! Чувствительность к регистру

Все параметры командной строки утилиты BCP чувствительны к регистру. Например: параметр *-e* используется для указания файла ошибок, в то время как *-E* указывает утилите BCP сохранять идентификаторы при импорте.

Вот три примера базовых команд утилиты BCP:

```
bcp AdventureWorks.HumanResources.Department out c:\test\department.txt -n  
-SHOTEK -T
```

```
bcpl AdventureWorks.HumanResources.Department in c:\test\department.txt -c  
-SHOTEK -U<имя_пользователя> -P<пароль>
```

```
bcpl "SELECT Name, GroupName FROM HumanResources.Department" queryout c:\test\  
department.txt -n -SHOTEK -T
```

ПРИМЕЧАНИЕ Источники экспорта данных

При обсуждении импорта и экспорта данных в этой главе для простоты предполагается, что речь идет о таблице. Можно выполнять экспорт из таблиц и представлений. Также можно импортировать данные в представления при условии их соответствия требованиям для использования команды *INSERT*.

Первый аргумент определяет таблицу или запрос, обрабатываемые утилитой BCP. Третий аргумент определяет файл с исходными данными или результатом работы BCP.

Второй аргумент BCP — выражение *in*, *out* или *queryout*. При выборе значения *in* BCP осуществляет импорт данных из указанного файла в заданную таблицу. Если второму аргументу присвоено значение *out*, BCP экспортирует содержимое таблицы в указанный файл. При необходимости экспортировать подмножество таблицы или результат запроса, можно заменить имя таблицы запросом, заключенным в двойные кавычки, и использовать параметр *queryout*. Как видно из названия, параметр *queryout* позволяет только экспортировать данные.

ПРИМЕЧАНИЕ На экзамене

На экзамене проверяются знания вариантов импорта и экспорта, наиболее подходящих в той или иной ситуации.

Остальные аргументы BCP можно указывать в произвольном порядке, при этом большинство администраторов баз данных придерживаются соглашения о порядке указания аргументов в той последовательности, в какой они перечислены в основном синтаксисе команды.

Аргументы *-n* и *-c* являются взаимно исключающими. Первый указывает на файл с данными в собственном формате SQL Server, а второй — на файл с данными в символьном формате. При экспорте данных, которые планируется импортировать в другой экземпляр SQL Server, следует использовать аргумент *-n*, так как при этом данные сохраняются во внутреннем формате SQL Server для хранения страниц данных, что обеспечит лучшую производительность. При необходимости переноса файла между разными платформами баз данных или в иную не-SQL систему, следует использовать аргумент *-c*, что позволит преобразовать данные из собственного формата хранения SQL Server в стандартный символьный формат. Выбор аргумента при импорте данных из файла определяется форматом полученного файла, поскольку при выполнении операции импорта BCP не может осуществлять преобразование данных из одного формата в другой.

При выполнении команд Transact-SQL (T-SQL) нет необходимости указывать название экземпляра или базы данных, поскольку эти параметры указаны в свойствах подключения к базе данных. Утилита BCP является приложением, поэтому не обладает информацией о контексте экземпляра и базе данных. Поэтому в ней нужно указывать информацию для подключения. Аргумент *-S*

определяет имя экземпляра, к которому необходимо подключиться. Аутентификация при подключении может осуществляться с применением учетной записи как самого SQL Server, так и Microsoft Windows. Аргумент *-T* определяет доверенное соединение, при этом BCP использует аутентификационную информацию учетной записи Windows, в контексте которой выполняется BCP. Также можно задействовать учетную запись SQL Server, используя аргументы *-U* и *-P*. Аргумент *-U* задает имя пользователя, а *-P* — пароль.

ПРИМЕЧАНИЕ Использование проверки ограничений и триггеров

При импорте данных в таблицу с использованием BCP, триггеры и проверка ограничений по умолчанию отключаются. При необходимости проверки ограничений и триггеров при импорте следует использовать аргумент *-h*. Если при импорте не отключить триггеры и проверку ограничений, то не потребуются разрешения *ALTER TABLE*.

Команда BULK INSERT

Одним из недостатков утилиты BCP является то, что это приложение командной строки. У команды *BULK INSERT* много общих с утилитой BCP возможностей и особенностей использования за исключением следующих двух:

- *BULK INSERT* не поддерживает экспорт данных;
- *BULK INSERT* является командой T-SQL и не требует указания имени экземпляра или идентификационной информации учетной записи.

Общий синтаксис команды *BULK INSERT* таков:

BULK INSERT

```
[ database_name . [ schema_name ] . | schema_name . ] [ table_name | view_name ]
    FROM 'data_file'
    [ WITH
        ( [ [ , ] BATCHSIZE = batch_size ] [ [ , ] CHECK_CONSTRAINTS ]
        [ [ , ] CODEPAGE = { 'ACP' | 'OEM' | 'RAW' | 'code_page' } ]
        [ [ , ] DATAFILETYPE = { 'char' | 'native' | 'widechar' | 'widenative' } ]
        [ [ , ] FIELDTERMINATOR = 'field_terminator' ] [ [ , ] FIRSTROW = first_row ]
        [ [ , ] FIRE_TRIGGERS ] [ [ , ] FORMATFILE = 'format_file_path' ]
        [ [ , ] KEEPIDENTITY ] [ [ , ] KEEPNULLS ]
        [ [ , ] KILOBYTES_PER_BATCH = kilobytes_per_batch ] [ [ , ] LASTROW = last_row ]
        [ [ , ] MAXERRORS = max_errors ] [ [ , ] ORDER ( { column [ ASC | DESC ] }
        [ ,...n ] ) ]
        [ [ , ] ROWS_PER_BATCH = rows_per_batch ] [ [ , ] ROWTERMINATOR = 'row_terminator' ]
        [ [ , ] TABLOCK ] [ [ , ] ERRORFILE = 'file_name' ] ) ]
```

Мастер импорта и экспорта SQL Server

Утилита BCP и команда *BULK INSERT* позволяют просто и легко импортировать и экспортировать данные с использованием файлов. Если надо импортировать и экспортировать данные напрямую между источником и получателем, при этом осуществлять преобразования и обработку ошибок в данных, можно задействовать возможности SSIS по построению пакетов.

Мастер импорта и экспорта SQL Server предоставляет администратору подмножество возможностей SSIS в среде SSMS для перемещения данных между источником и получателем. Доступ к мастеру можно получить в контекстном меню базы данных в Обозревателе объектов (Object Explorer), выбрав в панели **Задачи** (Tasks) **Импорт данных** (Import Data) или **Экспорт данных** (Export Data).

Хотя BCP и *BULK INSERT* работают с файлами, мастер импорта и экспорта поддерживает любые источники данных, распознаваемые SSIS, такие как файлы Microsoft Office Excel, Microsoft Office Access или XML. Кроме того, мастер импорта и экспорта поддерживает любые источники и получатели данных, для которых имеется провайдер OLE DB (Object Linking and Embedding Database). Для работы BCP и *BULK INSERT* требуется, чтобы экземпляр SQL Server был либо источником, либо получателем данных, в то время как мастер импорта и экспорта не требует, чтобы экземпляр SQL Server был источником или получателем. Кроме того, мастер импорта и экспорта может перемещать данные из нескольких таблиц или файлов в одной операции, а BCP и *BULK INSERT* могут работать только с одной таблицей, представлением или запросом.

ПРИМЕЧАНИЕ Компонент Integration Services

Полное описание возможностей SSIS-пакетов выходит за рамки данной книги. Подробнее о возможностях SSIS см. книгу Майка Хотека (Mike Hotek) «Microsoft SQL Server 2008 Step by Step» (Microsoft Press, 2008).

Контрольные вопросы

1. Какие форматы данных поддерживает BCP и какие существуют аргументы командной строки для каждого формата?
2. Какой параметр необходимо указать для экспорта данных с использованием запроса?
3. На какой функциональности SQL Server базируется мастер импорта и экспорта?
4. Какие виды источников и получателей может использовать мастер импорта и экспорта?

Ответы на контрольные вопросы

1. Утилита BCP может работать с данными либо в собственном, либо в символьном форматах. Аргумент *-c* указывает на символьные данные, аргумент *-n* указывает на данные в собственном формате.
2. Для экспорта результата запроса используется параметр *queryout*.
3. Мастер импорта и экспорта базируется на функциональности SSIS.
4. Можно указать любой источник и получатель, для которого есть провайдер OLE DB.

Практикум. Экспорт данных

Вы выполните экспорт содержимого таблицы *HumanResources.Department* в файлы собственного и символьного форматов. Также вы задействуете мастер импорта и экспорта для экспорта всего содержимого базы данных *Adventure-Works*.

Задание 1. Экспорт данных с помощью ВСП

Вы выполните экспорт содержимого таблицы HumanResources.Department.

1. Откройте окно командной строки и выполните команду:

```
bcp AdventureWorks.HumanResources.Department out c:\test\department.txt -c  
-S<имя_экземпляра> -T
```

2. Откройте созданный файл в Блокноте и просмотрите результаты.

3. Выполните следующую команду в командной строке:

```
bcp AdventureWorks.HumanResources.Department out c:\test\department.bcp -n  
-S<имя_экземпляра> -T
```

4. Откройте созданный файл в Блокноте и просмотрите результаты.

Задание 2. Экспорт таблиц

Вы выполните экспорт данных из базы AdventureWorks в новую базу данных по имени AdventureWorksTest.

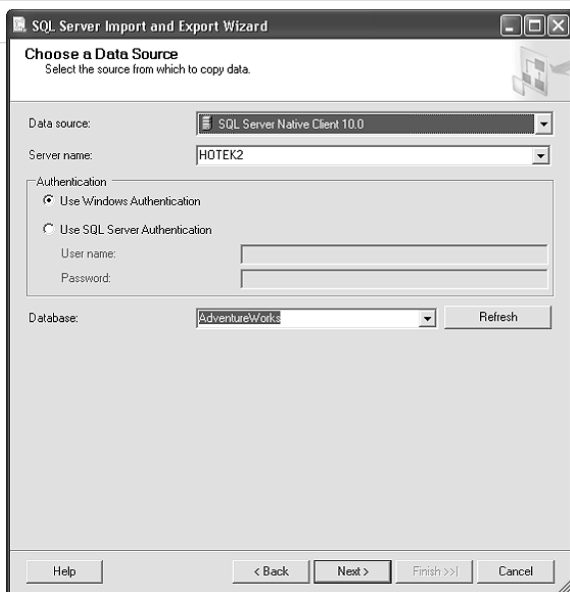
1. В среде SSMS выполните следующую команду в окне запроса:

```
CREATE DATABASE AdventureWorksTest
```

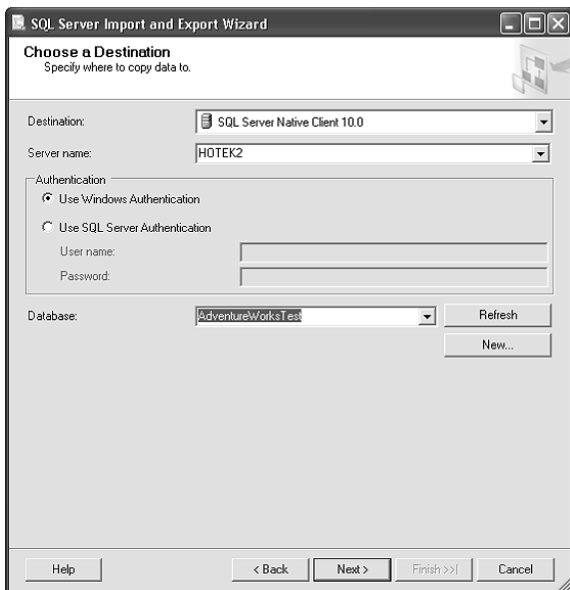
2. В окне **Обозреватель объектов** (Object Explorer) щелкните правой кнопкой базу данных AdventureWorks, выберите **Задачи** (Tasks), а затем — **Экспорт данных** (Export Data).

3. На странице с приветствием Мастера импорта и экспорта SQL Server (SQL Server Import And Export Wizard) щелкните **Далее** (Next) .

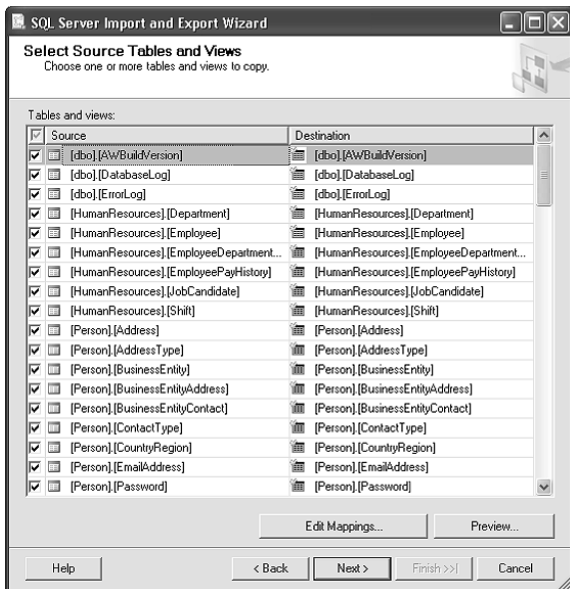
4. Выберите базу данных AdventureWorks в качестве источника и щелкните **Далее** (Next).



5. Выберите базу данных AdventureWorksTest в качестве получателя и щелкните **Далее** (Next).

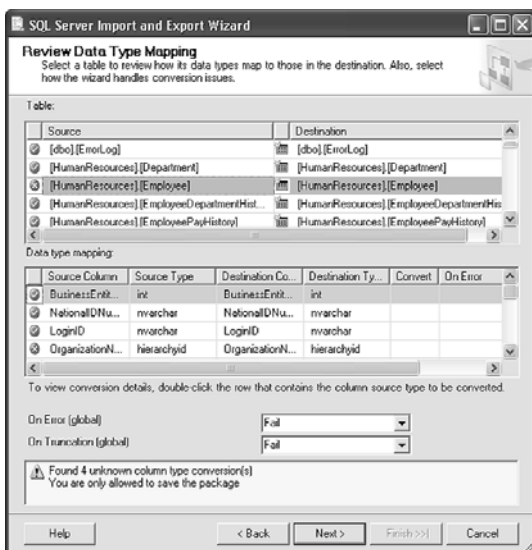
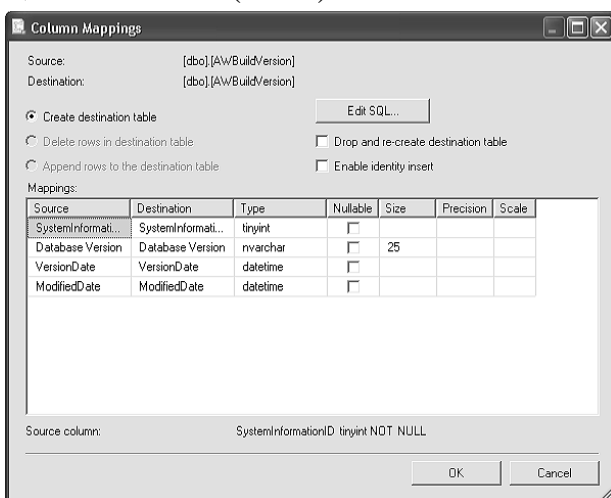


6. Выберите **Скопировать данные из одной или нескольких таблиц или представлений** (Copy Data From One Or More Tables Or Views) и щелкните **Далее** (Next).
7. Выберите все таблицы базы данных AdventureWorks.



8. В списке источников выберите базу данных AWBuildVersion и щелкните **Изменить сопоставления** (Edit Mappings).
9. Просмотрите параметры, доступные при переносе данных от источника к получателю. Щелкните **Отмена** (Cancel), а затем **Далее** (Next).

10. При переносе данных с помощью SSIS SQL Server преобразует их в типы .NET, поскольку SSIS основан на C# .NET, где нет эквивалентов для типов данных *hierarchy*, *geography* и *geometry*. Поэтом мастер импорта и экспорта не может применяться для обработки данных этих типов. При наличии данных одного или нескольких указанных типов вы получите сообщение об ошибке. Щелкните **Назад** (Back).
11. Снимите флажки рядом с таблицами **[HumanResources].[Employee]**, **[Person].[Address]**, **[Production].[Document]** и **[Production].[ProductDocument]** и щелкните **Далее** (Next).
12. Удостоверьтесь, что выбран вариант **Запустить немедленно** (Run Immediately) и щелкните **Далее** (Next).
13. Проконтролируйте выполняющиеся операции. По окончании выполнения щелкните **Готово** (Finish).



Резюме занятия

- BCP — простой инструмент командной строки, позволяющий импортировать и экспортировать данные.
- Утилита BCP не предназначена для преобразования данных и обработки ошибок.
- Кроме экспорта содержимого таблицы или представления с помощью утилиты BCP можно экспортировать результат запроса, задействовав аргумент *queryout*.
- **BULK INSERT** — команда T-SQL, позволяющая выполнять только импорт данных.
- Основанный на SSIS Мастер импорта и экспорта позволяет перемещать данные напрямую между источником и получателем без использования промежуточных файлов.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Вам необходимо импортировать данные в таблицу Orders. В таблице определены триггеры и проверки ограничений для обеспечения целостности. Вы планируете использовать утилиту BCP, задав аргумент *-h CHECK_CONSTRAINTS, FIRE_TRIGGERS*. Какие из перечисленных разрешений должны быть заданы?
 - A. *SELECT* в таблице Orders.
 - B. *ALTER TABLE* в таблице Orders.
 - C. *INSERT* в таблице Orders.
 - D. Членство в роли bulkadmin.
2. Вы переносите с Oracle на SQL Server базу данных Order, содержащую несколько сотен таблиц. В таблице CustomerAddress есть XML-столбец AddressBook. Как наиболее эффективно и с наименьшими затратами перенести данные в новую базу на SQL Server?
 - A. Перенести базу данных Order с Oracle на SQL Server, используя репликацию.
 - B. Экспортировать базу данных с помощью утилит Oracle и импортировать ее в SQL Server с помощью BCP.
 - C. Перенести базу данных Order с помощью мастера импорта и экспорта.
 - D. Перенести данные из Oracle в SQL с помощью функции *OPENROWSET*.

Закрепление материала главы

Для закрепления знаний и навыков, полученных при изучении этой главы, выполните следующее:

- изучите резюме главы;

- повторите основные термины, использованные в этой главе;
- выполните задания по темам этой главы;
- сделайте предложенные упражнения;
- пройдите пробный экзамен.

Резюме главы

- BCP является программой, позволяющей импортировать данные из файла в таблицу, а также экспортировать данные из таблицы в файл.
- *BULK INSERT* — команда T-SQL, позволяющая импортировать данные из файла в таблицу.
- Мастер импорта и экспорта основан на возможностях SSIS для переноса данных между источником и получателем.

Основные термины

Понятны ли вам значения приведенных здесь основных терминов? Чтобы проверить свои ответы, загляните в глоссарий на прилагаемом компакт-диске.

- Утилита BCP ~ Bulk Copy Program (BCP).
- *BULK INSERT*.

Упражнения

Сейчас вы примените на практике знания и навыки, полученные при изучении этой главы. Ответы на вопросы см. в разделе «Ответы» в конце книги.

Упражнение 1. Разработка стратегии импорта для компании Coho Vineyard

Исходные данные

Информация о компании

Компания Coho Vineyard основана в 1947 году как семейная винодельня. За последние десятилетия вина Coho Vineyards неоднократно отмечались премиями на выставках, благодаря чему предприятие существенно выросло. В последнее время компания приобрела несколько винодельческих заводов. Сегодня компания состоит из 16 заводов: 9 находятся в Вашингтоне, Орегоне и Калифорнии, и 7 заводов расположены в Висконсине и Мичигане. На заводах работает 532 человека, 162 из которых — в штаб-квартире, где расположены критически важные для работы предприятия серверы. В компании также работают 122 коммивояжера, путешествующих по всему миру, которым необходим доступ к информации о текущем состоянии складов.

Запланированные изменения

До настоящего времени у каждого из 16 заводов Coho Vineyard был собственный локальный веб-сайт. Компания Coho Vineyard планирует объединить веб-ресурсы всех заводов, чтобы клиенты могли купить продукцию любого из 16 заводов в едином онлайн-магазине. Все данные этого веб-сайта будут храниться в базе данных в штаб-квартире.

Данные будут консолидированы в базе штаб-квартиры, репликация слиянием будет использоваться для доставки данных коммивояжерам, а также для ввода ими заказов. Пока проект консолидации данных не завершен, для обслуживания коммивояжеров в конце рабочего дня данные о запасах с каждого завода пересылаются в штаб-квартиру.

Существующая система работы с данными

Базы данных

Сейчас каждый завод поддерживает собственную БД для хранения своей коммерческой информации. В конце каждого месяца эта информация переносится в штаб-квартиру и загружается в базы данных, указанные в табл.7-1.

Табл. 7-1. Базы данных Coho Vineyard

База данных	Объем
<i>Customer</i> (данные о клиентах)	180 MB
<i>Accounting</i> (бухгалтерские данные)	500 MB
<i>HR</i> (данные о кадрах)	100 MB
<i>Inventory</i> (складские запасы)	250 MB
<i>Promotions</i> (сведения о поощрениях)	80 MB

В результате выполнения работ по консолидации баз данных будет создана новая база данных *Order*, которая будет работать хранилищем данных нового онлайн-магазина. В процессе своей ежедневной работы сотрудники будут периодически обращаться к базе данных *Order*, используя новое самостоятельно разработанное веб-приложение.

База данных *HR* содержит конфиденциальную информацию и защищена с использованием TDE (Transparent Data Encryption). Кроме того, данные в таблице *Salary* (данные о зарплатах) зашифрованы с использованием сертификатов.

Серверы баз данных

Все базы данных в центральном офисе хранятся в SQL Server 2008 Enterprise на одном сервере DB1 под управлением Windows Server 2003 Enterprise.

Бизнес-требования

Вам необходимо разработать решение архивирования баз данных *Customer* и *Order*. Стратегия архивирования должна обеспечивать хранение данные базы *Customer* в течение 6 лет.

Для подготовки базы данных *Order* к архивированию, вы создали секционированную таблицу (partitioned table) *Order.Sales*, состоящую из двух секций: Секция 1 содержит информацию о продажах в текущем месяце, а Секция 2 — о продажах в предыдущем месяце. Заказы, полученные в позапрошлом месяце нужно перенести в другую секционированную таблицу *Order.Archive*, где Секция 1 содержит все архивные данные, а Секция 2 пуста.

Процесс нужно спроектировать так, чтобы данные о складских запасах с каждого из 16 заводов ежедневно загружались в 4 часа утра.

Четыре крупных клиента направляют заказы, используя разработанную компанией Coho Vineyards XML-схему для обмена электронными транзакциями (Electronic Data Interchange, EDI). Файлы в EDI-формате поступают в 5 часов вечера, их нужно проанализировать и загрузить в базы данных *Customer*, *Accounting* и *Inventory*, каждая из которых содержит таблицы, относящиеся в содержимому заказа. Программа импорта EDI-заказов представляет собой разработанное на C++ однопоточное приложение, которому на обработку файлов требуется от 3 до 6 часов. В соответствии с соглашением о качестве обслуживания клиентов (SLA) необходимо, чтобы процесс обработки EDI-заказов заканчивался к 5:30 утра. По окончании проекта консолидации EDI-программа должна загружать все данные в новую базу *Order*.

Должно выполняться резервное копирование всех баз данных на всех заводах. В самом крайнем случае допускается потеря максимум 5 минут данных. При возникновении сбоев базы данных *Customer*, *Account*, *Inventory*, *Promotions* и *Order* могут быть недоступны не более 20 минут. Данные старше 6 месяцев в базах *Customer* и *Order* могут быть недоступны при сбоях до 12 часов.

Ответьте на следующие вопросы.

1. Какой метод лучше всего использовать для переноса данных с каждого завода в центральную базу данных?
2. Какой метод обеспечит наиболее гибкий способ обработки всех EDI-данных?

Рекомендуемые упражнения

Чтобы подготовиться к экзамену по темам этой главы, выполните следующие задания:

- **Задание 1.** Используйте утилиту BCP в символьном режиме для экспорта данных из таблицы в файл.
- **Задание 2.** Воспользуйтесь командой *BULK INSERT* для импорта содержимого файла, созданного в задании 1, в таблицу.
- **Задание 3.** Изучите SSIS. Платформа SSIS позволяет выполнять любые операции импорта-экспорта и обработки данных, которые могут понадобиться в вашей среде.

Пробный экзамен

На прилагаемом компакт-диске содержится пробный экзамен, обеспечивающий широкие возможности самоподготовки. Например, можно проверить, как усвоены темы этой главы либо все темы экзамена 70-432. Можно симитировать настоящий экзамен или выбрать режим обучения, когда после каждого вопроса приводятся правильные ответы с пояснениями.

ПРИМЕЧАНИЕ Пробный экзамен

Подробнее о пробном экзамене см. «Введение».

Проектирование управления на основе политик

Занятие 1. Разработка политик

162

До выхода Microsoft SQL Server 2008 управлять конфигурацией окружения приходилось при помощи массы документов, сценариев и ручных операций. Параметры конфигурации, соглашения о наименовании и разрешенный набор функций регламентировались в одном или нескольких документах. Для обеспечения выбранных стандартов приходилось подключаться к каждому экземпляру и выполнять сценарии, которые следовало поддерживать и обновлять с выходом каждой новой версии и пакета обновления. В этой главе вы познакомитесь с новой инфраструктурой управления на основе политик, позволяющей проверять и обеспечивать соответствие политикам всей инфраструктуры SQL Server.

Темы экзамена

- Внедрение инфраструктуры DMF.
- Настройка контактной зоны.

Прежде всего

Для изучения материалов этой главы необходимо иметь:

- установленный экземпляр Microsoft SQL Server 2008;
- подключенную базу данных AdventureWorks.

Практический совет

Майкл Хотек

Управление одним сервером SQL Server или их небольшой группой по одному в каждый момент времени всегда представлялось довольно простым делом. Однако когда требуется единообразно управлять целой средой серверов SQL Server или большой группой экземпляров, приходится либо писать много пользовательского кода, либо покупать дополнительные продукты.

У одного из клиентов, с которым я работал, была среда из более чем 5000 экземпляров SQL Server. До выхода SQL Server 2008 требовалось два администратора баз данных для управления кодом объемом более 50 000 строк, который проверял экземпляры на соответствие корпоративным политикам. У них уходило больше 70 часов в неделю на поддержание этого кода и проверку систем.

После установки SQL Server 2008 они начали переводить свой код в политики. По их оценкам после завершения перехода осталось менее 1000 строк пользовательской логики. Функции централизованного управления для проверки и обеспечения политик во всей среде позволили ежегодно экономить более 3000 часов администраторского труда по управлению и сопровождению.

Занятие 1. Разработка политик

В SQL Server 2008 появилась новая функциональность, называемая *управление на основе политик* (Policy Based Management), известная также как инфраструктура DMF (Declarative Management Framework), предназначенная для стандартизации экземпляров SQL Server. Управление на основе политик можно просто использовать для предупреждения администратора о несоответствии объектов принятым стандартам, но при наличии определенных типов политик DMF позволяет обеспечивать это соответствие, предотвращая изменения, нарушающие политики.

Управление на основе политик вводит следующие новые объекты, используемые при разработке и проверке на соответствие стандартам:

- *аспекты*;
- *условия*;
- *политики*;
- *цели политик*;
- *категории политик*.

Изучив материал этого занятия, вы сможете:

- ✓ создавать условия;
- ✓ определять политики;
- ✓ задавать цели проверок политик;
- ✓ настраивать категории политик;
- ✓ выполнять проверку на соответствие политикам;
- ✓ импортировать и экспортировать политики.

Продолжительность занятия — 30 минут.

Аспекты

Аспекты (facets) — ключевые объекты, на основе которых строятся стандарты. Аспекты определяют подлежащие контролю типы объектов или параметры,

такие как база данных, контактная зона или учетная запись. В комплект поставки SQL Server входит 74 аспекта, выполненных в виде сборок .NET, каждая с уникальным набором параметров.

Все объекты управления на основе политик находятся в базе данных *msdb*. Получить список имеющихся аспектов можно, выполнив запрос таблицы *dbo.syspolicy_management_facets*. К сожалению, если вы не желаете писать код для взаимодействия с управляющими объектами сервера (Server Management Objects, SMO), то единственный способ получить список свойств аспекта — по одному открывать каждый аспект в среде SQL Server Management Studio (SSMS) и просматривать список свойств.

Условия

При определении предложения *WHERE* в команде языка обработки данных DML, задаются условия для DML-инструкции, которые определяют набор строк, удовлетворяющих определенным критериям включения. В рамках инфраструктуры управления на основе политик *условия* аналогичны предложению *WHERE* и определяют критерии для проверки.

Условия для контроля или применения, которые должна накладывать политика, определяются путем определения критериев для свойств аспекта. Точно так же как и предложение *WHERE*, условие может быть определено для одного или нескольких свойств аспекта, а одно свойство аспекта может контролироваться несколькими критериями. Допустимые операторы сравнения зависят от типа данных конкретного свойства. Например, свойство типа *string* может проверяться операторами *=*, *<>*, *LIKE*, *NOT LIKE*, *IN* или *NOT IN*, в то время как для типа *boolean* возможны только *=* и *<>*.

Если для условия аспекта, которое требуется контролировать, нет подходящего отдельного свойства, можно воспользоваться расширенным редактором для определения сложных условий, сравнивающих множество свойств и содержащих функции. Например, можно проверять, что у всех таблиц есть первичные ключи, а таблицы с единственным индексом — кластеризованы. К сожалению, если определить условия в расширенном редакторе, политика, содержащая это условие, должна выполняться вручную и не может быть помещена в расписание.

Условия проверяются в одно действие. Условия не могут получать список объектов, перечислять их итеративно по одному и затем выполнять последующие проверки. Чтобы работать в инфраструктуре управления на основе политик, условия должны возвращать только *True* или *False*. Поэтому при построении сложных условий в расширенном редакторе невозможно вернуть список объектов, не удовлетворяющих критериям. Необходимо формулировать условие так, чтобы в случае неудовлетворения критериям хотя бы одним объектом, возвращалось значение *False*.

Хотя в одном условии можно проверять несколько свойств аспекта, одно условие нельзя определить для нескольких аспектов. Например, в одном условии можно проверить все 10 свойств аспекта настройки контактной зоны, но для проверки одного свойства настройки контактной зоны службы Analysis Services придется определить второе условие.

Цели политик

Условия лежат в основе политик. Однако не всегда требуется контролировать политики всех имеющихся объектов, таких как все базы данных в экземпляре или все индексы в каждой базе данных. Условия могут применяться также для указания множества объектов, на котором должны проверяться условия, они называются целями для политик.

Назначать цели для политики можно на уровне серверов, например экземпляры SQL Server 2005 или SQL Server 2008. Политике можно также назначать цели на уровне баз данных, к примеру все пользовательские базы данных или все системные базы данных.

Политики

Политики создаются только для одного условия и либо обеспечивают, либо контролируют соответствие требованиям. Можно выбрать один из следующих режимов выполнения:

- **По запросу (On demand)** Выполнение политики инициируется пользователем.
- **При изменении: запретить (On change, prevent)** Для предотвращения нарушения политики используются триггеры DDL.
- **При изменении: только внесение в журнал (On change, log only)** Используется уведомление о событии для определения политики при возникновении соответствующего изменения.
- **По расписанию (On schedule)** Создается задание агента SQL Server для контроля политики по определенному расписанию.

Если политика содержит условие, определенное в расширенном редакторе, то единственный возможный режим выполнения — «По запросу».

Для режимов выполнения «При изменении: запретить» и «При изменении: только внесение в журнал» политика должна быть предназначена для экземпляров SQL Server 2005 или выше. В режиме «При изменении: только внесение в журнал» используется инфраструктура уведомления о событии, доступная только в SQL Server 2005 или выше. Для предотвращения не соответствующих политике изменений режим «При изменении: запретить» требует триггеры DDL, которые поддерживаются только в SQL Server 2005 или выше. Кроме того, политике можно задать режим «При изменении: запретить», только если триггер DDL может предотвратить изменение. Например, можно предотвратить создание объекта, не удовлетворяющего соглашению об именовании, но невозможно обеспечить соблюдение политики, обязывающей все базы данных иметь модель полного восстановления, так как команда *ALTER DATABASE* выполняется вне контекста транзакции.

Категории политик

Категории политик (policy categories) служат для объединения одной или нескольких политик в единый блок. Если не указано иного, все политики принадлежат категории *DEFAULT*. Для проверок соответствия политикам или их обеспечения, необходимо создать подписку к одной или нескольким политикам.

Подписка возможна на двух уровнях — экземпляра сервера или базы данных. Члены роли `sysadmin` могут подписать экземпляр сервера к категории политик. После этого владельцы баз данных экземпляра могут подписывать свои базы данных к этой категории политик.

В каждой категории политик есть свойство *Mandate*, применяемое к базам данных. Когда для категории политик устанавливается свойство *Mandate* и системный администратор подписывает к ней экземпляр сервера, все базы данных, входящие в набор целей, подчиняются политикам этой категории. Подписка политики к категории политик, у которой задано свойство *Mandate*, не может переопределяться владельцем базы данных.

Соответствие политикам

Поскольку невозможно обеспечить соответствие всем политикам, то те политики, которые невозможно применять на регулярной основе, приходится применять вручную. Заданные в экземпляре политики можно просмотреть, щелкнув правой кнопкой имя экземпляра в обозревателе объектов, а затем выбрав **Политики** (Policies) и **Просмотр** (View). Проверить установленные на экземпляре политики можно, щелкнув правой кнопкой имя экземпляра в обозревателе объектов и выбрав **Политики** (Policies), а затем — **Выполнить** (Evaluate).

Проверить все политики экземпляра можно, щелкнув правой кнопкой узел **Политики** (Policies), а затем выбрав **Выполнить** (Evaluate), как показано на рис. 8-1.

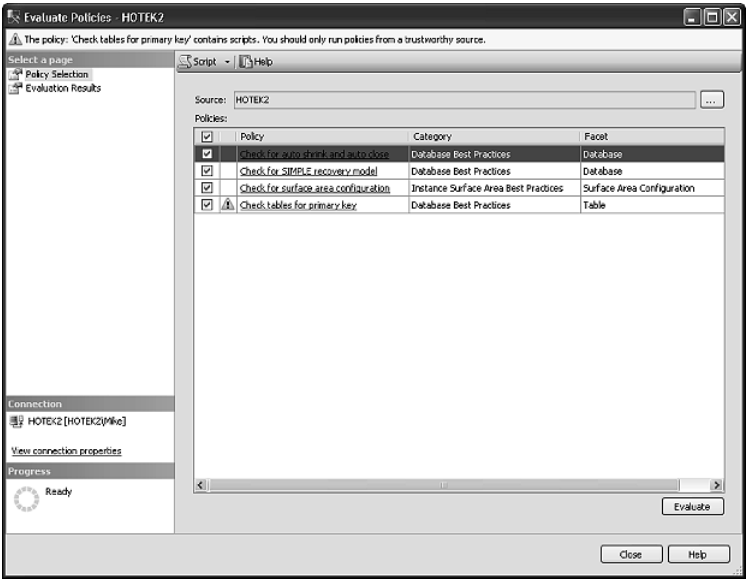


Рис. 8-1. Выполнение политик

Щелкнув **Выполнить** (Evaluate), вы можете выполнить политики и просмотреть результаты, как показано на рис. 8-2.

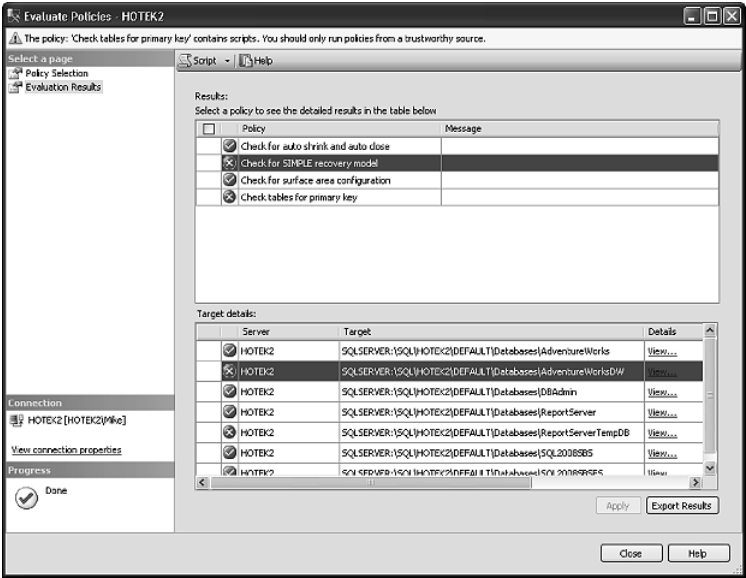


Рис. 8-2. Результаты проверки политик

ПРИМЕЧАНИЕ На экзамене

Разработка условий, используемых в качестве целей политик, критически важна для определения корректных политик. Политика не проходит проверку, если объект не удовлетворяет критериям или если отсутствуют соответствующие свойства. Например, попытка проверить, что на экземпляре SQL Server 2008 отключен помощник Web Assistant, завершится с ошибкой, поскольку такая функция отсутствует.

Серверы централизованного управления

Управление на основе политик ограничено серверами SQL Server 2008 и может быть очень утомительно, если требуется выполнить следующие операции:

- скопировать политики на все экземпляры;
- по отдельности создать подписки для всех экземпляров в среде;
- по отдельности проверить все экземпляры на соответствие правилам.

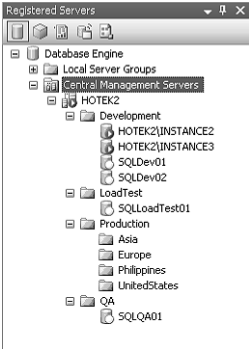


Рис. 8-3. Центральный сервер управления

Серверы централизованного управления настраиваются в панели «Зарегистрированные серверы» (Registered Servers) среды SSMS. В узле Central Management Servers управления можно создать несколько уровней папок и зарегистрировать экземпляры в подходящих папках. После того, как структура серверов централизованного управления настроена в среде SSMS, можно выполнять политики на любом экземпляре, папке или всех экземплярах центрального сервера управления. На рис. 8-3 показан пример узла Central Management Servers.

Импорт и экспорт политик

Политики и условия можно как экспортировать в файлы, так и импортировать из них. В поставку SQL Server входит 53 политики, которые находятся в папке Microsoft SQL Server\100\Tools\Policies. Из них 50 политик для ядра СУБД, 2 политики для служб Reporting Services и 1 политика для служб Analysis Services. На веб-сайте CodePlex (<http://www.codeplex.com>) можно загрузить и импортировать дополнительные политики.

Импортируют в панели **Зарегистрированные серверы** (Registered Servers) или обозревателя объектов. Чтобы импортировать политику средствами Обозревателя объектов, щелкните правой кнопкой подузлы **Политики** (Политики) узла **Управление политиками** (Policy Management) и выберите **Импортировать политику** (Import Policy). Чтобы импортировать политику средствами панели **Зарегистрированные серверы** щелкните правой кнопкой **Центральный сервер управления** (Central Management Server), папку или экземпляр и выберите выбрать **Импортировать политику** (Import Policies). При импорте политик на центральном сервере управления они импортируются для всех экземпляров ниже узла Central Management Servers, но не на нем самом. Точно также, щелчок правой кнопки папки импортирует политики во все экземпляры иерархии этой папки. Чтобы импортировать политики на центральный сервер, надо подключиться к его экземпляру из обозревателя объектов и выполнить импорт из узла **Политики** (Policies).

Контрольные вопросы

1. Какие пять объектов применяются в управлении на основе политик?
2. Какие существуют режимы выполнения политик?
3. У каких объектов есть свойства, позволяющие сделать контроль всех баз данных экземпляра обязательным?
4. Сколько аспектов может проверяться в одном условии?
5. Сколько условий может проверяться в одной политике?

Ответы на контрольные вопросы

1. В управлении на основе политик применяются следующие объекты: аспекты, условия, политики, цели политик и категории политик.
2. Есть следующие режимы выполнения политик: «По запросу», «По расписанию», «При изменении: только внесение в журнал» и «При изменении: запретить».
3. Категории политик позволяют сделать обязательной проверку всех баз данных экземпляра.

4. Условие может определяться только на одном аспекте.
5. Политика может контролировать только одно условие.

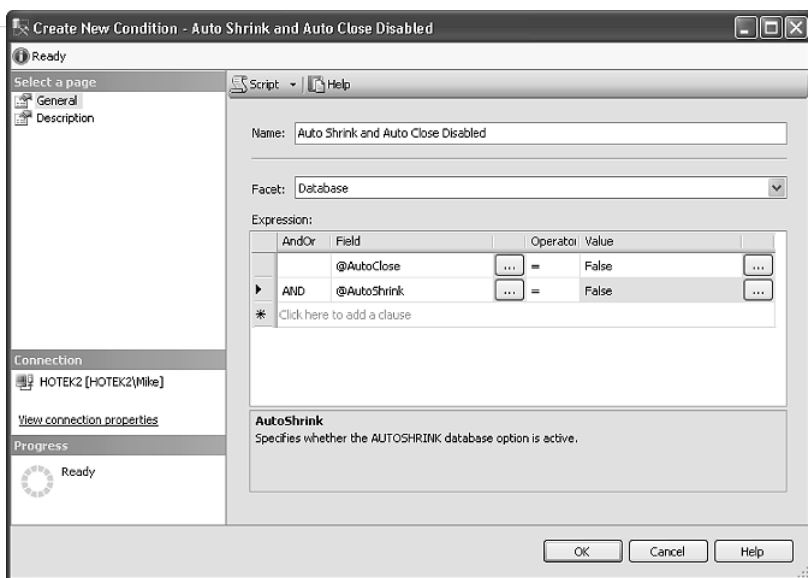
Практикум. Определение политик и выполнение проверок на соответствие

Вы определите несколько политик в своей среде и выполните их проверку.

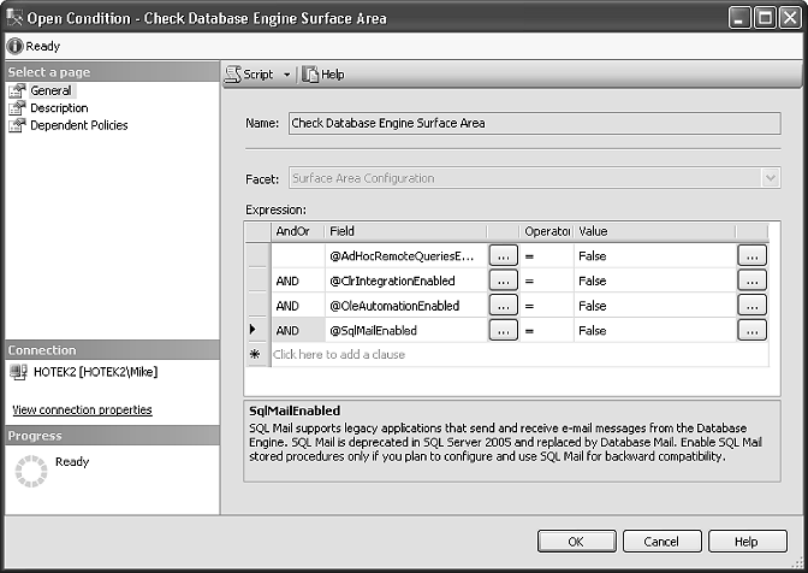
Задание 1. Создание условия

Вы создадите условие, чтобы:

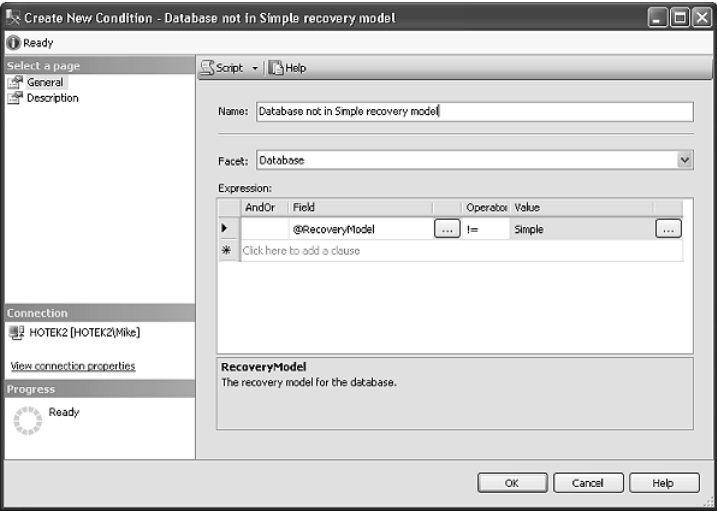
- проверять, что в базе данных не установлены свойства *auto shrink* или *auto close*;
 - проверять, что отключены поддержка CLR, OLE-автоматизация, нерегламентированные удаленные запросы (Ad Hoc Remote Queries) и служба SQL Mail;
 - проверять, что в базе данных не выбрана простая модель восстановления;
 - проверять, что у всех таблиц есть первичный ключ.
1. В Обозревателе объектов в узле **Управление** (Management) раскройте узел **Управление политиками** (Policy Management).
 2. Щелкните правой кнопкой узел **Условия** (Conditions) и выберите **Создать условие** (New Condition).
 3. Задайте условие, как показано на рисунке и щелкните **ОК**.



4. Снова щелкните правой кнопкой узел **Условия**, выберите **Создать условие** и настройте условие, как показано на рисунке. Щелкните **ОК**.



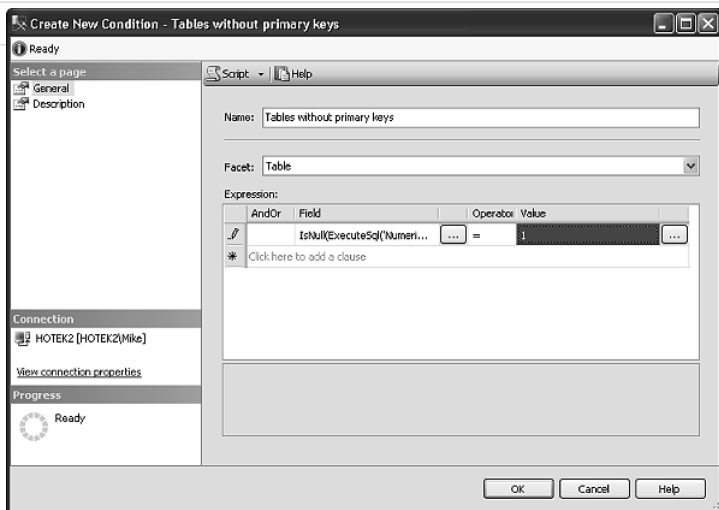
5. Опять щелкните правой кнопкой узел **Условия**, выберите **Создать условие** и настройте условие, как показано на следующем рисунке. Щелкните **ОК**.



6. Щелкните правой кнопкой узел **Условия**, выберите **Создать условие**. Выберите аспект **Таблица** (Table), щелкните кнопку с многоточием рядом со столбцом **Поле** (Field). Откроется диалоговое окно **Расширенное редактирование** (Advanced Edit). Введите следующий код в поле **Значение ячейки** (Cell Value):

```
IsNull(ExecuteSql('Numeric', 'SELECT 1 FROM sys.tables a INNER JOIN sys.  
indexes b  
ON a.object_id = b.object_id WHERE b.is_primary_key = 1  
AND a.name = @@ObjectName AND a.schema_id = SCHEMA_ID(@@SchemaName)', 0))
```

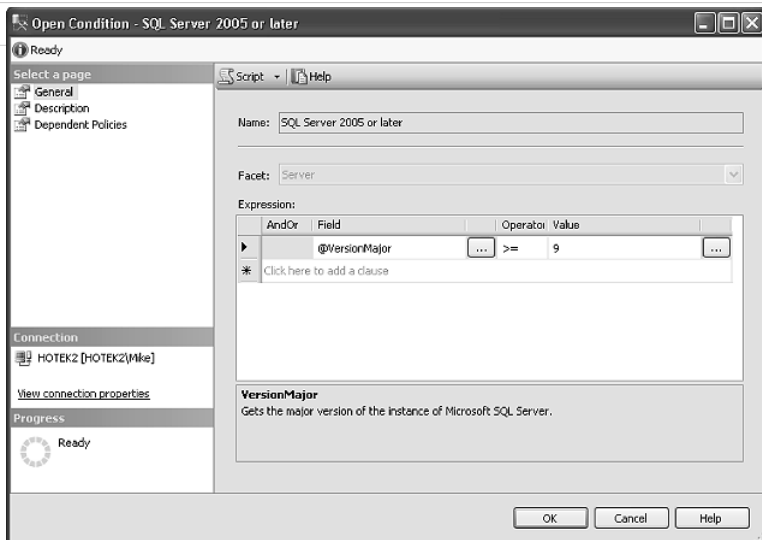

7. Настройте **Имя** (Name), **Оператор** (Operator) и **Значение** (Value), как показано на рисунке и щелкните **ОК**.



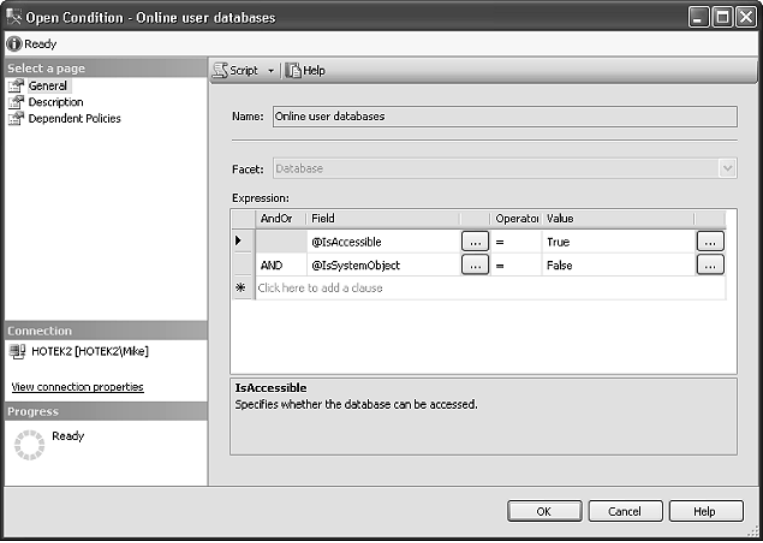
Задание 2. Создание условия для набора целей

Вы создадите условие выбора всех экземпляров SQL Server 2005, а также более поздних версий, и условие для отбора всех пользовательских баз данных, находящихся в оперативном режиме.

1. Щелкните правой кнопкой узел **Условия** и выберите **Создать условие**. Задайте условие, как показано на следующем рисунке и щелкните **ОК**.



2. Щелкните правой кнопкой узел **Условия** и выберите **Создать условие**. Задайте условие, как показано на следующем рисунке и щелкните **ОК**.

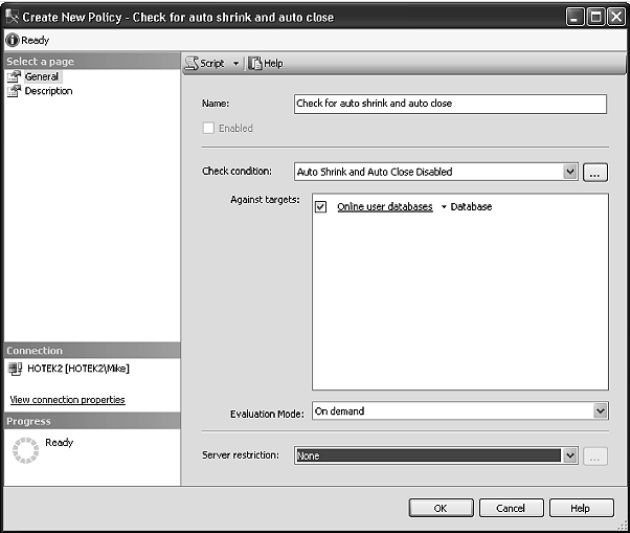


Задание 3. Создание политики

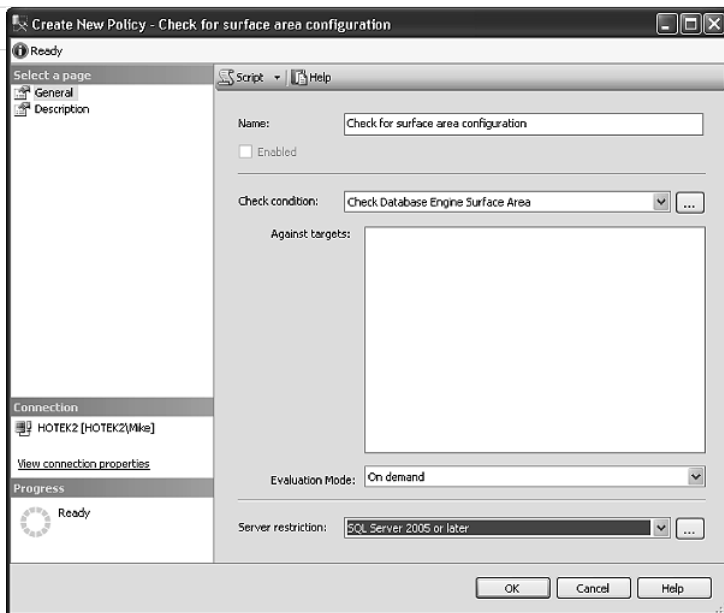
Вы создадите политику, использующую только что созданные условия, чтобы проверять, что:

- в базе данных не установлены свойства *auto shrink* или *auto close*;
- отключены поддержка CLR, OLE-автоматизация, нерегламентированные удаленные запросы (Ad Hoc Remote Queries) и служба SQL Mail;
- в базе данных не выбрана простая модель восстановления;
- у всех таблиц есть первичный ключ.

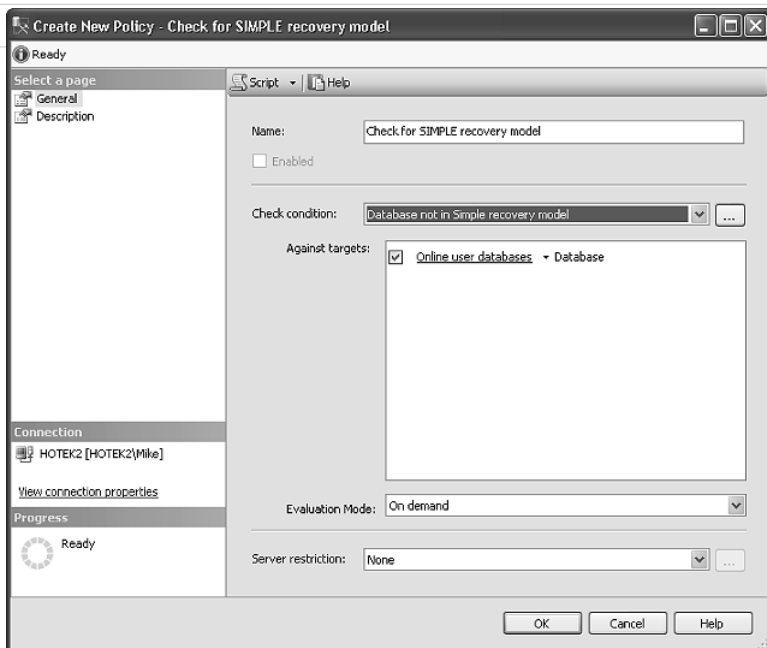
1. Щелкните правой кнопкой узел **Политики** (Policies), выберите **Создать политику** (New Policy) и настройте политику, как показано ниже, затем щелкните **ОК**.



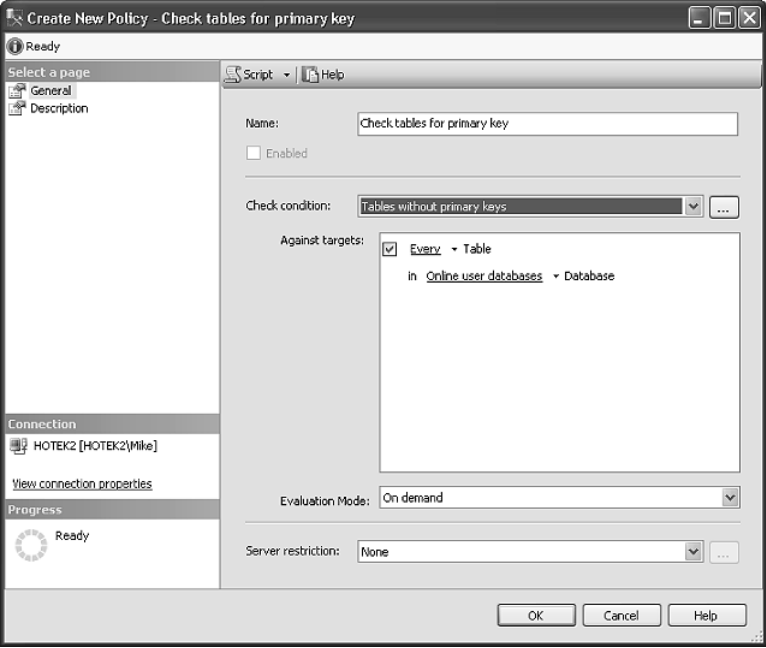
2. Щелкните правой кнопкой узел **Политики**, выберите **Создать политику** и настройте политику, как показано ниже, затем щелкните **ОК**.



3. Щелкните правой кнопкой узел **Политики**, выберите **Создать политику** и настройте политику, как показано ниже, затем щелкните **ОК**.



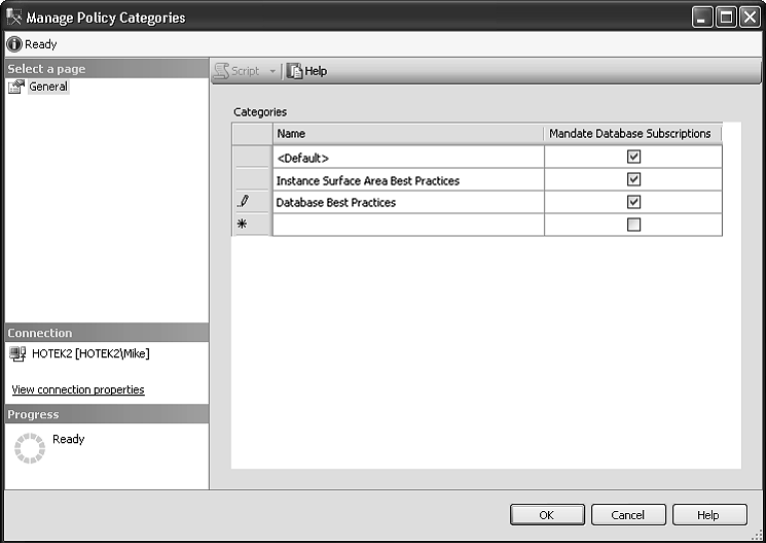
4. Щелкните правой кнопкой узел **Политики**, выберите **Создать политику** и настройте политику, как показано ниже, затем щелкните **ОК**.



Задание 4. Создание категории политик

Вы создадите две категории для созданных политик.

- 1. Щелкните правой кнопкой **Управление политиками** (Policy Management), выберите **Управление категориями** (Manage Categories) и создайте категорию, как показано на рисунке. Щелкните **ОК**.



- 2. В дереве консоли среды SSMS раскройте папку **Политики** (Policies). Щелкните правой кнопкой **Check For Auto Shrink And Auto Close Policy** (по-

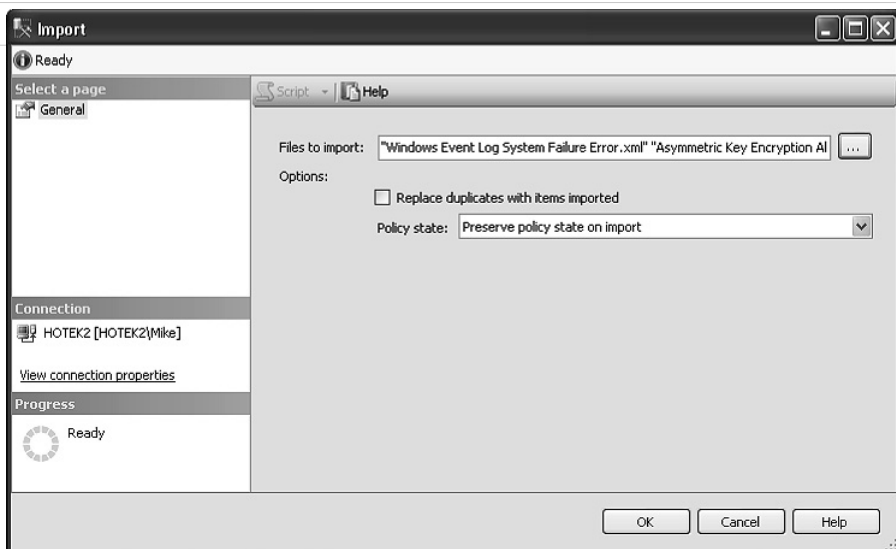
литика для проверки на автоматическое сжатие и автоматическое закрытие), выберите **Properties** (Свойства), щелкните вкладку **Описание** (Description) и измените категорию на **Рекомендации для баз данных** (Database Best Practices). Щелкните **ОК**.

- Щелкните правой кнопкой **Check For Simple Recovery Model Policy** (политика проверки на простую модель восстановления), выберите **Свойства** (Properties), затем вкладку **Описание** (Description) и измените категорию на **Рекомендации для баз данных** (Database Best Practices), затем щелкните **ОК**.
- Щелкните правой кнопкой **Check For Surface Area Configuration Policy** (политика проверки настройки контактной зоны), выберите **Свойства** (Properties), затем вкладку **Описание** (Description) и измените категорию на **Рекомендации контактной зоны экземпляра** (Instance Surface Area Best Practices), затем щелкните **ОК**.
- Щелкните правой кнопкой **Check For Primary Key Policy** (политика проверки первичных ключей), выберите **Свойства** (Properties), затем вкладку **Описание** (Description) и измените категорию на **Рекомендации для баз данных** (Database Best Practices), затем щелкните **ОК**.

Задание 5. Импорт политик

Вы выполните импорт политик, поставляемых в составе SQL Server.

- В узле **Управление политиками** (Policy Management) щелкните правой кнопкой **Политики** (Policies) и выберите **Импортировать политику** (Import Policy).
- Щелкните кнопку с многоточием, рядом с полем **Файлы, подлежащие импорту** (Files To Import), перейдите к папке Microsoft SQL Server\100\Tools\Policies\DatabaseEngine\1033, выберите в ней все файлы, как показано на рисунке, и щелкните **Открыть** (Open).



3. Установите флажок **Заменять повторы импортируемыми элементами** (Replace Duplicates With Items Imported), выберите **Сохранить состояние политик для импорта** (Preserve Policy State On Import) и щелкните **ОК**.
4. Внимательно изучите политики и условия, созданные при импорте.

Резюме занятия

- Политики можно создавать для обеспечения выполнения определенных условий на серверах SQL Server любых версий.
- Каждая политика обеспечивает выполнение только одного условия, а каждое условие основывается на единственном аспекте.
- Категории политик позволяют объединять политики для проверки на соответствие.
- У категорий политик можно задавать свойство *Mandate*, чтобы сделать их контроль обязательным на всех базах данных экземпляра.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Вы создали несколько политик, которые должны применяться ко всем базам данных экземпляра. Как с минимальными усилиями обеспечить, чтобы владельцы баз данных не смогли обойти проверку этих политик?
 - A. Создать условие, проверяющее все базы данных.
 - B. Добавить политику в пользовательскую категорию политик и задать в ней свойство *Mandate*.
 - C. Добавить политики в категорию политик по умолчанию.
 - D. Вручную контролировать политики на экземпляре.

Закрепление материала главы

Для закрепления знаний и навыков, полученных при изучении этой главы, выполните следующее:

- изучите резюме главы;
- повторите основные термины, использованные в этой главе;
- выполните задания по темам этой главы;
- сделайте предложенные упражнения;
- пройдите пробный экзамен.

Резюме главы

- Аспекты — это .NET-сборки, определяющие набор свойств для объектов, на основе которых создаются условия.
- Условие может определяться только для одного аспекта, а политика может контролироваться только на одном экземпляре.

- Политики могут контролироваться вручную или автоматически. Автоматический контроль политик может осуществляться по расписанию или при помощи инфраструктуры уведомления о событиях.
- Владелец базы данных может подписать ее на одну или несколько политик. Однако политики, входящие в категорию политик с заданным свойством *Mandate*, обязательны для выполнения во всех базах данных.

Основные термины

Понятны ли вам значения приведенных здесь основных терминов? Чтобы проверить свои ответы, загляните в глоссарий на прилагаемом компакт-диске.

- Условие ~ condition.
- Аспект ~ facet.
- Категория политик ~ policy category.
- Цель политики ~ policy target.

Упражнения

Сейчас вы примените на практике знания и навыки, полученные при изучении этой главы. Ответы на вопросы см. в разделе «Ответы» в конце книги.

Упражнение 1. Разработка стратегии управления для компании Coho Vineyard

Исходные данные

Основные сведения о компании

Компания Coho Vineyard была основана в 1947 г. как частный семейный винный завод. Вина, производимые Coho Vineyard, удостоивались многих наград и пользовались популярностью, что позволило компании значительно расширить производство. Через несколько лет в рамках стратегии по расширению компании были приобретены несколько существующих винных заводов. Сейчас компания владеет 16 винными заводами — 9 в штатах Вашингтон, Орегон и Калифорния, а остальные 7 в штатах Висконсин и Мичиган. В компании работает 532 человека, из которых 162 — в центральном офисе, где размещаются критически важные для бизнеса серверы. В компании трудится 122 мобильных торговых агента, которым требуется доступ к актуальной информации об имеющихся товарах.

Планируемые изменения

До настоящего времени у каждого из 16 заводов Coho Vineyard был собственный локальный веб-сайт. Компания Coho Vineyard планирует объединить веб-ресурсы всех заводов, чтобы клиенты могли покупать продукцию любого из 16 заводов в едином онлайн-магазине. Все данные этого веб-сайта будут храниться в базе данных в штаб-квартире.

Данные будут консолидированы в базе штаб-квартиры, репликация слиянием будет использоваться для доставки данных торговым агентам, а также для ввода ими заказов. Пока проект консолидации данных не завершен, для обслу-

живания торговых агентов в конце рабочего дня данные о запасах с каждого завода пересылаются в штаб-квартиру.

Руководство требует, чтобы нельзя было выполнять хранимые процедуры на C#.NET или использовать команды *OPENROWSET* и *OPENDATASOURCE*.

Существующая инфраструктура данных

Базы данных

Сейчас каждый винный завод поддерживает собственную базу данных, где хранит всю бизнес-информацию. В конце месяца вся информация переносится в центральный офис и загружается в базы данных, перечисленные в табл. 8-1.

Табл. 8-1. Базы данных Coho Vineyard

База данных	Объем
<i>Customer</i> (данные о клиентах)	180 MB
<i>Accounting</i> (бухгалтерские данные)	500 MB
<i>HR</i> (данные о кадрах)	100 MB
<i>Inventory</i> (складские запасы)	250 MB
<i>Promotions</i> (сведения о поощрениях)	80 MB

В результате выполнения работ по консолидации баз данных будет создана новая база данных *Order*, которая будет работать хранилищем данных нового онлайн-магазина. В процессе своей ежедневной работы сотрудники будут периодически обращаться к базе данных *Order*, используя новое самостоятельно разработанное веб-приложение.

База данных *HR* содержит конфиденциальную информацию и защищена с использованием TDE (Transparent Data Encryption). Кроме того, данные в таблице *Salary* (данные о зарплатах) зашифрованы с использованием сертификатов.

Серверы баз данных

Все базы данных в центральном офисе хранятся в SQL Server 2008 Enterprise на одном сервере DB1 под управлением Windows Server 2003 Enterprise.

Бизнес-требования

Вам необходимо разработать решение архивирования баз данных *Customer* и *Order*. Стратегия архивирования должна обеспечивать хранение данные базы *Customer* в течение 6 лет.

В процессе подготовки БД *Order* для архивирования вы создали секционированную таблицу *Order.Sales*, состоящую из двух секций: в первой хранится информация о продажах за текущий, а во второй — за предыдущий месяц. Заказы, размещенные еще раньше, должны перемещаться в другую секционированную таблицу — *Order.Archive*, после чего в секции 1 содержатся все архивные данные, а секция 2 остается пустой.

Новый процесс должен ежедневно, не позднее 4 часов утра загружать информацию о складских запасах со всех 16 заводов.

Четыре крупных клиента осуществляют заказы, используя XML-схему Coho Vineyards для EDI-транзакций (Electronic Data Interchange). EDI-файлы поступа-

ют в 5 часов вечера и должны обрабатываться и загружаться в БД *Custo-mer*, *Accounting* и *Inventory*, каждая из которых содержит таблицы с данными, относящимися к размещаемым заказам. В настоящее время утилита импорта EDI-файлов представляет собой однопоточное приложение на C++, которому требуется от 3 до 6 часов на обработку всех файлов. Чтобы выполнить соглашение с клиентами об условиях обслуживания (Service Level Agreement, SLA), необходимо заканчивать обработку EDI-файлов до 17:30. К моменту завершения проекта по объединению EDI-утилиты должна загружать информацию в новую базу данных *Order*.

Требуется обеспечить создание резервных копий всех баз данных всех филиалов. Все производственные базы данных должны работать в режиме полной модели восстановления. В самом плохом случае считается приемлемым потеря данных за последние 5 минут. В случае аварии БД *Customer*, *Inventory*, *Promotions* и *Order* могут быть недоступны максимум 20 минут. В случае аварийного сбоя информация БД *Customer* и *Order* старше полугода может быть недоступна максимум 12 часов.

Ответьте на следующий вопрос.

1. Какие политики нужно определить для проверки и обеспечения выполнения бизнес-требований компании Coho Vineyard?

Рекомендуемые упражнения

Чтобы подготовиться к экзамену по темам этой главы, выполните следующие задания.

Реализация управления, основанного на политиках

- **Задание 1.** Создайте политику для контроля настройки контактной зоны всех экземпляров SQL Server.
- **Задание 2.** Создайте политику для проверки времени последнего успешного создания резервной копии.
- **Задание 3.** Создайте политику для контроля состава ролей sysadmin и db_owner.
- **Задание 4.** Создайте политику, чтобы гарантировать, что в базах данных не заданы свойства *auto shrink* или *auto close*.
- **Задание 5.** Из политик, входящих в состав поставки SQL Server 2008, выберите подходящие для вашей среды и реализуйте их контроль.

Пробный экзамен

На прилагаемом компакт-диске содержится пробный экзамен, обеспечивающий широкие возможности самоподготовки. Например, можно проверить, как усвоены темы этой главы либо все темы экзамена 70-432. Можно симитировать настоящий экзамен или выбрать режим обучения, когда после каждого вопроса приводятся правильные ответы с пояснениями.

ПРИМЕЧАНИЕ Пробный экзамен

Подробнее о пробном экзамене см. «Введение».

Резервное копирование и восстановление базы данных

Занятие 1. Резервное копирование баз данных	180
Занятие 2. Восстановление баз данных	193
Занятие 3. Моментальный снимок базы данных	204

Обеспечение безопасности — не единственная первоочередная задача администратора баз данных — он также должен обеспечивать возможность восстановления базы данных в случае ее повреждения. Тысячи возможностей Microsoft SQL Server 2008, позволяющие использовать высокопроизводительные и масштабируемые приложения, становятся недоступными без должной защиты данных. В этой главе вы узнаете о возможностях модуля резервного копирования и восстановления данных, а также процедурах восстановления при различных аварийных ситуациях.

Темы экзамена

- Резервное копирование среды SQL Server.
- Резервное копирование баз данных.
- Восстановление баз данных.
- Управление моментальными снимками баз данных.
- Сопровождение базы данных посредством плана обслуживания.

Прежде всего

Для изучения материалов этой главы необходимо иметь:

- установленный экземпляр Microsoft SQL Server 2008;
- подключенную базу данных AdventureWorks.

Практический совет

Майк Хотек

Несколько лет назад меня пригласили в одну компанию помочь им в восстановлении после стихийного бедствия, обусловившего полный переезд

в другое помещение. К тому времени новый офис и вычислительный центр были уже восстановлены, а моей задачей было восстановление работы серверов баз данных и синхронизация со всеми восстановленными базами данных.

Я получил множество документов и подробных описаний процедур резервного копирования для баз данных и резервных серверов, где избыточные копии данных были доступны в онлайн на случай сбоя основного сервера. Все, казалось, было в порядке, поэтому я сел в самолет и уже через несколько часов был в новом вычислительном центре.

И только по прибытии туда, начав собирать все факты вместе, я узнал плохие новости — документация была абсолютно бесполезна. В компании имелись резервные серверы, куда производилась доставка журналов, но они размещались в том же вычислительном центре, где и основные серверы. Размещение резервных серверов в одном вычислительном центре не рекомендуется, к тому же проблема усугублялась тем, что старый вычислительный центр был на глубине 17-футов под водой. Хуже всего то, что хотя для всего и были созданы резервные копии на лентах, но они хранились в шкафу... все в том же плавательном бассейне, в который превратился вычислительный центр после затопления.

У нас была только одна пригодная для использования резервная копия — комплект лент приблизительно двухмесячной давности. В то время один из новых администраторов баз данных догадался заняться хранением лент с резервными копиями вне офиса, но проект был закрыт из-за проблем с финансированием. К счастью, этот новый администратор был загружен работой и забыл, что, на первом этапе проекта по хранению данных вне офиса, сотрудник, работающий по вызову администратора, должен был забирать домой ленту с архивом предыдущего дня и что он сам отнес домой комплект лент два месяца назад.

При управлении базами данных рекомендуется иметь стратегию резервного копирования, резервные серверы и несколько копий баз данных. Но к этому следует добавить еще немного здравого смысла. Если целью резервного копирования является защита от стихийных бедствий, то очень нежелательно хранить ленты с копиями близко от основных компьютеров, которые могут пострадать от стихийного бедствия.

Занятие 1. Резервное копирование баз данных

Резервное копирование баз данных — основа основ любого плана восстановления. Стратегии резервного копирования разрабатываются в порядке, обратном представлению материала данной главы. Начинать следует с требований и процедур восстановления, а затем определить какие типы резервных копий лучше всего соответствуют вашим потребностям. Если не разрабатывать стратегию резервного копирования, ориентированную на восстановление, то вряд ли удастся удовлетворить требованиям восстановления после катастроф. Однако очень сложно обучать восстановлению данных людей, ничего не знающих о резервном копировании. Поэтому прежде чем на этом занятии рассказывать

о восстановлении баз данных, мы поговорим о видах резервных копий и о том, как их создавать в процессе сопровождения баз данных.

Изучив материал этого занятия, вы сможете:

- ✓ создавать полные и разностные резервные копии, а также резервные копии журналов транзакций;
- ✓ разрабатывать планы обслуживания.

Продолжительность занятия — 20 минут.

Безопасность резервного копирования

Резервное копирование всегда выполняется в контексте безопасности учетной записи службы SQL Server. Хотя можно и напрямую предоставить учетной записи службы SQL Server права доступа на чтение и запись в папку резервного копирования, вместо этого следует предоставлять доступ Windows-группе `SQLServerMSSQLUser$<имя_компьютера>$<имя_экземпляра>`, в которую входит учетная запись службы SQL Server.

Члены серверной роли *sysadmin* могут создавать резервные копии любых баз данных экземпляра сервера, а члены роли базы данных *db_owner* могут создавать резервные копии только своей базы данных. Также можно включать пользователей в фиксированную роль базы данных *db_backupoperator*, чтобы разрешить им создавать резервные копии базы данных, запретив при этом любой другой доступ к этой базе.

Типы резервных копий

В SQL Server 2008 можно создавать резервные копии четыре типов:

- *полная копия* (full backup);
- *разностная копия* (differential backup);
- *копия журнала транзакций* (transaction log backup);
- *копия файловой группы* (filegroup backup).

Полные резервные копии

Полная резервная копия включает все содержащиеся информацию таблицы базы данных. Страницы, не содержащие данные, в резервную копию не попадают. Поэтому резервная копия не может быть больше, а зачастую она меньше базы данных, для которой создана. Полная резервная копия является основой для восстановления базы данных и должна использоваться до применения разностной резервной копии или копии журнала транзакций.

Резервное копирование базы данных выполняется чаще, чем восстановление, поэтому модуль резервного копирования оптимизирован для создания резервной копии. При запуске резервного копирования модуль резервного копирования «захватывает» страницы с информацией из файлов с максимально возможной скоростью, не обращая внимания на их порядок. Поэтому при записи страниц на устройство резервного копирования может использоваться несколько

ко потоков. Единственным ограничителем скорости создания резервной копии является производительность устройства, куда записывается резервная копия.

Резервное копирование может выполняться одновременно с другими операциями в базе данных. Во время резервного копирования в самой базе данных могут происходить изменения, поэтому SQL Server должен «уметь» принимать изменения и при этом гарантировать целостность базы данных. Для обеспечения доступности базы данных и целостности ее резервной копии, SQL Server выполняет следующие действия в процессе создания резервной копии:

1. Запирает базу данных, блокируя все транзакции.
2. Ставит отметку в журнал транзакций.
3. Освобождает базу данных.
4. Из файлов данных получает все страницы и записывает их в устройство резервного копирования.
5. Запирает базу данных, блокируя все транзакции.
6. Ставит отметку в журнал транзакций.
7. Освобождает базу данных.
8. Извлекает часть журнала между отметками и добавляет ее к резервной копии.

Во время создания полной копии запрещены только следующие действия:

- добавление или удаление файлов базы данных;
- сжатие базы данных.

Общий синтаксис создания резервной копии базы данных таков:

```
BACKUP DATABASE { имя_базы_данных | @переменная_с_именем_базы_данных }
    TO <устройство_резервного_копирования> [ ,...n ]
    [ <предложение MIRROR TO> ] [ next-mirror-to ]
    [ WITH { DIFFERENTIAL | <общие_параметры_WITH> [ ,...n ] } ]
```

```
<устройство_резервного_копирования> ::= { { имя_логического_устройства
| @переменная_с_именем_логического_устройства }
| { DISK | TAPE } =
    { 'имя_физического_устройства' | @переменная_с_именем_физического_
устройства } }
```

```
<предложение MIRROR TO> ::= MIRROR TO <устройство_резервного_копирования>
[ ,...n ]
```

```
<общие_параметры_WITH> [ ,...n ] ::=
```

```
--Набор параметров резервного копирования
```

```
    COPY_ONLY | { COMPRESSION | NO_COMPRESSION }
    | DESCRIPTION = { 'описание' | @переменная_с_описанием }
    | NAME = { имя_резервного_набора_данных | @переменная_с_именем_резервного_
набора_данных }
    | PASSWORD = { пароль | @переменная_с_паролем }
    | { EXPIREDATE = { 'дата' | @переменная_с_датой }
    | RETAINDAYS = { дни | @переменная_с_днями } }
```

```
--Набор параметров носителя
```

```
{ NOINIT | INIT } | { NOSKIP | SKIP } | { NOFORMAT | FORMAT }  
| MEDIADESCRIPTION = { 'текст_описания_носителя' | @переменная_с_описанием_  
носителя }  
| MEDIANAME = { имя_носителя | @переменная_с_именем_носителя }  
| MEDIAPASSWORD = { пароль_носителя | @переменная_с_паролем_носителя }  
| BLOCKSIZE = { размер_блока | @переменная_с_размером_блока }  
  
--Параметры управления ошибками  
{ NO_CHECKSUM | CHECKSUM }  
| { STOP_ON_ERROR | CONTINUE_AFTER_ERROR }
```

Обязательны только два параметра — имя базы данных и устройства резервного копирования. При назначении дискового устройства резервного копирования могут указываться каталог и имя файла. Если каталог не задан, SQL Server, выполняя резервное копирование на диск, записывает файл в каталог, настроенный по умолчанию для данного экземпляра сервера. Хотя большинство резервных копий размещается в одном файле или одном ленточном устройстве, можно задать до 64 устройств резервного копирования. При выборе нескольких устройств резервного копирования SQL Server создает чередующуюся резервную копию, распределенную между всеми заданными устройствами.

ПРИМЕЧАНИЕ Восстановление из чередующейся резервной копии

Когда SQL Server распределяет *чередующуюся* (striped) резервную копию по нескольким устройствам, то все эти копии необходимы для успешного восстановления. В чередующемся наборе SQL Server не обеспечивает ни избыточности, ни устойчивости к сбоям. Чередующийся набор служит исключительно для увеличения производительности.

Вот пример создания чередующейся резервной копии:

```
BACKUP DATABASE AdventureWorks  
TO DISK = 'AdventureWorks_1.bak', DISK = ' AdventureWorks_2.bak'  
GO
```

Одна из максим аварийного восстановления гласит, что не бывает лишних резервных копий. Предложение *MIRROR TO* предоставляет встроенные возможности создания до четырех экземпляров резервной копии за одну операцию. При наличии предложения *MIRROR TO* SQL Server получает каждую страницу из базы данных только раз, а затем записывает ее копию сразу во все зеркальные резервные копии. В процессе восстановления можно использовать любой зеркальный экземпляр. К зеркальным резервным копиям предъявляется немного требований:

- типы носителя всех устройств резервного копирования должны быть одинаковы;
- все зеркала должны иметь одинаковое количество устройств резервного копирования;
- в команде создания резервной копии должен быть указать *WITH FORMAT*.

Если резервная копия создается на ленте, то и зеркало должно размещаться на ленте. При создании резервной копии на диске, зеркало тоже должно быть на диске. Нельзя создавать резервную копию на ленте, а зеркало на диске и

наоборот. Кроме того, на зеркале должно быть столько же устройств, как и в резервной копии, например, если резервная копия размещается на 64 дисковых устройствах, то и у зеркала должно быть 64 дисковых устройства.

Ограничителем производительности резервного копирования является скорость устройств резервного копирования. Сжав резервную копию, можно записать необходимую информацию, уменьшив при этом объем данных, передаваемых на устройство резервного копирования. Сжатие резервной копии подразумевает дополнительные затраты на обработку. А резервное копирование без сжатия позволяет обойтись минимумом вычислительных ресурсов, тогда как при сжатии потребность в них может быть весьма заметна. Коэффициент сжатия при резервном копировании обычно лежит в диапазоне от 4:1 до 10:1.

ПРАКТИЧЕСКИЙ СОВЕТ Сокращение времени резервного копирования

Дополнительные затраты на сжатие *всегда* стоят того. Время, сэкономленное при сжатии резервной копии, значительно превышает дополнительную нагрузку, связанную с операцией сжатия. К счастью, в SQL Server предусмотрен параметр, называемый сжатием резервной копии по умолчанию, задав который, можно обеспечить выполнение сжатия всегда, независимо от того, задано оно явно или нет. К сожалению, сжатие доступно только в SQL Server 2008 Enterprise.

Каждое устройство резервного копирования может содержать несколько резервных копий. Параметрами *INIT/NOINIT* команды *BACKUP* определяется, будет ли существующий файл резервной копии замещаться или дополняться. Если при создании резервной копии задан параметр *NOINIT*, а файл уже существует, SQL Server добавит новую копию в конец этого файла. При наличии параметра *INIT* существующий файл будет перезаписан содержимым новой резервной копии.

ПРАКТИЧЕСКИЙ СОВЕТ Предотвращение проблем при резервном копировании

Чтобы избежать путаницы, рекомендуется использовать схему уникального именования, в которой дата и время включаются в имя файла, благодаря чему время создания резервной копии можно определить по имени ее файла. Поскольку целью создания резервных копий является снижение риска потери данных, не рекомендуется помещать несколько резервных копий в один файл.

При заданном параметре *CHECKSUM* перед сохранением страницы в резервной копии SQL Server проверяет ее контрольную сумму, если та существует. Кроме этого вычисляется контрольная сумма самой резервной копии, которая может использоваться для проверки на отсутствие повреждений резервной копии. Параметр *STOP_ON_ERROR* определяет поведение по умолчанию при обнаружении ошибок в процессе создания резервной копии. При обнаружении расхождения контрольной суммы страницы создание резервной копии прерывается с ошибкой. Чтобы при обнаружении ошибок создание резервной копии продолжалось и в нее попало как можно больше страниц, следует указать параметр *CONTINUE_PAST_ERROR*.

ПРИМЕЧАНИЕ Выявление поврежденных страниц

Для как можно более раннего обнаружения поврежденных страниц рекомендуется указывать параметр *CHECKSUM*. При восстановлении базы данных из резервной копии любые сюрпризы очень неприятны.

Резервные копии журнала транзакций

Каждому изменению в базе данных соответствует своя запись в журнале транзакций. Каждой строке назначен уникальный номер, называемый *порядковым номером транзакции в журнале* (*Log Sequence Number, LSN*). LSN — это целое число, начинающееся с 1 при создании базы данных и увеличивающееся до бесконечности. В одной базе данных LSN никогда не используется повторно. Фактически LSN является порядковым номером каждого изменения в базе данных.

Содержимое журнала транзакций состоит из двух частей — активной и неактивной. Неактивная часть журнала транзакций содержит уже зафиксированные в базе данных изменения, а активная — еще не зафиксированные. В процессе создания резервной копии журнала транзакций, SQL Server начинает с наименьших номеров LSN журнала транзакций и последовательно помещает в резервную копию записи журнала транзакций. Достигнув LSN-номер первой незафиксированной записи (т. е. старейшей открытой транзакции), SQL Server завершает создание резервной копии журнала транзакций. После этого та часть журнала, которая попала в резервную копию, удаляется, а освободившееся место используется повторно.

Следуя порядковыми номерами, можно восстанавливать резервные копии журнала транзакций одну за другой, воссоздавая базу данных на любой момент времени, просто следуя цепочке транзакций, упорядоченной по номерам LSN.

Поскольку резервные копии журнала транзакций должны восстанавливаться одна за другой, на них накладывается ограничение — в них не должна прерываться последовательность номеров LSN. Любая операция, создающая разрыв в последовательности LSN, препятствует восстановлению любой последующей резервной копии журнала транзакций. При наличии разрыва в LSN до создания резервных копий журнала транзакций необходимо создать полную резервную копию.

Резервная копия журнала транзакций работает в точности, как добавочная архивация в Microsoft Windows. Резервная копия журнала транзакций собирает все транзакции журнала, зафиксированные после создания его предыдущей копии. Однако, поскольку резервные копии журнала транзакций содержат только транзакции, выполненные в базе данных, то необходимо иметь начальную точку для *цепочки резервных копий журнала транзакций* (*transaction log chain*).

Перед созданием резервной копии журнала транзакций необходимо создать полную резервную копию. Если цепочка копий журнала транзакций не прерывалась, то базу данных можно восстановить на любой момент времени, начиная с первой резервной копии. Можно дополнительно создавать полные резервные копии, чтобы иметь более поздние начальные точки для восстановления. Независимо от количества созданных полных резервных копий, можно начать восстановление с любой из них и восстановить все копии журнала транзакций с этой точки, но при условии, что цепочка LSN не прерывалась.

Общий синтаксис создания резервной копии журнала транзакций:

```
BACKUP LOG { имя_базы_данных | @переменная_с_именем_базы_данных }  
TO <устройство_резервного_копирования> [ ,...n ]  
[ <предложение MIRROR TO> ] [ next-mirror-to ]  
[ WITH { <общие_свойства_WITH> | <особые_параметры_журнала> } [ ,...n ] ][:]
```


Разностные резервные копии

Разностная резервная копия содержит экстенды, изменившиеся с момента создания предыдущей полной резервной копии. Основное назначение разностной резервной копии — сокращение числа резервных копий журнала транзакций, необходимых для восстановления. Разностная резервная копия должна применяться к полной резервной копии и не создается, пока та не создана.

SQL Server отслеживает все экстенды, измененные с момента создания полной резервной копии, используя особую страницу в заголовке базы данных, называемой картой разностных изменений (Differential Change Map, DCM). При создании полной резервной копии содержимое карты DCM очищается. При изменении экстенда в базе данных, SQL Server присваивает значение 1 биту, соответствующему этому экстенду. При создании разностной резервной копии, SQL Server считывает содержимое карты DCM, чтобы определить экстенды, изменившиеся с момента создания последней полной резервной копии.

Разностная резервная копия совсем не то же самое, что добавочная резервная копия. Так, резервная копия журнала транзакций является добавочной резервной копией, поскольку она включает все изменения, произошедшие после создания предыдущей резервной копии журнала транзакций. Разностная резервная копия содержит все страницы, измененные после создания последней полной резервной копии. Например, если в полночь создается полная резервная копия, а разностная резервная копия создается каждые четыре часа, то копии, созданные как в 4:00, так и в 8:00 будут содержать все изменения, произошедшие в базе данных после полуночи.

Параметр **COPY_ONLY**

Один из параметров, который может задаваться для резервной копии любого типа, — *COPY_ONLY*. Создание каждой резервной копии оказывает влияние на начальную точку восстановления и на то, какие резервные копии могут использоваться. Разностные резервные копии содержат все экстенды, измененные после создания последней полной резервной копии, поэтому создание полной резервной копии меняет начальную точку, относительно которой строятся разностные резервные копии. При создании резервной копии журнала транзакций все попавшие в нее транзакции удаляются из журнала транзакций.

Иногда бывает необходимо создать резервную копию, чтобы затем получить из нее базу данных для среды разработки или тестирования. При этом требуется получить самые последние данные, не изменив резервный набор данных в рабочей среде. Параметр *COPY_ONLY* позволяет создать резервную копию, которую можно использовать для создания среды разработки или тестирования, не оказывая влияния на состояние рабочей базы данных и набор резервных данных. Создание полной резервной копии с параметром *COPY_ONLY* не сбрасывает страницу карты DCM, а значит, не влияет на разностные резервные копии. Создание резервной копии журнала транзакций с параметром *COPY_ONLY* не удаляет транзакции из журнала транзакций.

Резервные копии файловых групп

Полная резервная копия содержит все используемые страницы базы данных, но в больших базах данных ее создание может потребовать значительного места и времени. Если требуется уменьшить область применимости резервной копии, можно воспользоваться резервным копированием файлов или файловых групп. Как понятно из названия, резервные копии файлов или файловых групп позволяют создать резервную копию определенной части базы данных.

ВНИМАНИЕ! Создание резервной копии отдельных файлов

Несмотря на возможность создания резервных копий отдельных файлов, рекомендуется делать резервные копии не ниже уровня файловых групп. Файловая группа определяет границы хранения. Когда в ней несколько файлов, то SQL Server распределяет информацию по всем файлам. Однако данные из таблиц, индексов или секций, размещаются в файлах фактически случайно. Поэтому для восстановления базы данных необходимо, чтобы все файлы в группе были в абсолютно одинаковом состоянии.

Резервная копия файловой группы может использоваться совместно с разностной резервной копией и резервной копией журнала транзакций для восстановления части базы данных. Кроме того, если в SQL Server 2008 Enterprise не требуется восстановление первичной файловой группы базы данных, в ходе восстановления файловой группы база данных может продолжать работу и быть доступной для приложений. При этом недоступна только та часть базы данных, которая в данный момент восстанавливается.

ПРИМЕЧАНИЕ На экзамене

Необходимо знать, как создавать резервные копии каждого типа. На экзамене уделяется особое внимание созданию резервных копий и восстановлению баз данных и целой среды SQL Server.

Частичные резервные копии

Файловые группы можно пометить, как доступные только для чтения. Объекты, хранящиеся в таких файловых группах, не могут изменяться. Поскольку целью создания резервных копий является сохранение изменений, так чтобы базу данных можно было восстановить в наиболее актуальном состоянии, то резервное копирование неизменяемых файловых групп напрасно занимает место в резервной копии.

Чтобы уменьшить размер резервной копии, включив туда только изменяемые файловые группы, можно создать частичную резервную копию (partial backup). Это делается при помощи параметра `READ_WRITE_FILEGROUPS`:

```
BACKUP DATABASE имя_базы_данных READ_WRITE_FILEGROUPS
```

```
[, <список_файлов_файловой_группы>] TO <устройство_резервного_копирования>
```

При создании частичной резервной копии, SQL Server включает в нее первичную файловую группу, все файловые группы, доступные для чтения и записи, а также все явно перечисленные файловые группы, доступные только для чтения.

Поврежденные страницы

Надеемся, что вам не придется иметь дела с повреждением базы данных. К сожалению, случаются сбои аппаратных компонентов, особенно контроллеров дисков и дисковых накопителей. Перед окончательным выходом из строя они могут повредить страницы данных, выполнив незавершенные операции записи. SQL Server до версии 2005 мог целиком перейти в автономный режим в случае обнаружения поврежденной страницы. В SQL Server 2005 добавлена возможность помещать поврежденную страницу в карантин, сохраняя работоспособность базы данных. При выполнении следующей команды SQL Server ищет поврежденные страницы и помещает их в карантин:

```
ALTER DATABASE <имя_базы_данных> SET PAGE_VERIFY CHECKSUM
```

Сохраняя страницу на диске, SQL Server вычисляет ее контрольную сумму. Если включена проверка страниц, SQL Server при чтении каждой страницы пересчитывает ее контрольную сумму и сравнивает с сохраненной. Если контрольные суммы не совпадают, SQL Server возвращает ошибку и регистрирует страницу в таблице базы данных *msdb*.

ПРИМЕЧАНИЕ Восстановление поврежденных страниц

Когда база данных участвует в сеансе зеркального отображения, копия поврежденной страницы доставляется с зеркального отображения. Если с полученной страницей все в порядке, она автоматически заменяет поврежденную страницу.

Кроме возможности помещать поврежденные страницы в карантин, SQL Server имеет механизм для защиты базы данных от массивов поврежденных страниц. В одной базе данных может быть до 1000 поврежденных страниц. При достижении предела поврежденных страниц, SQL Server переводит базу данных в автономный режим для защиты ее от дальнейших повреждений, и присваивает ей статус *подозрительной* (Suspect).

Планы обслуживания

План обслуживания позволяет графически создавать последовательность операций по поддержке стандартных административных функций, таких как создание резервных копий, переиндексация и управление пространством хранения данных.

К задачам, выполняемым в рамках плана обслуживания, относятся:

- создание резервных копий баз данных и журналов транзакций;
- сжатие баз данных;
- переиндексация;
- обновление статистики;
- выполнение проверок целостности.

Наиболее часто выполняемой задачей в планах обслуживания является создание резервных копий баз данных. Вместо того чтобы писать код создания резервной копии базы данных, можно настроить план обслуживания на выполнение операций резервного копирования, необходимых для выполнения требований по восстановлению в случае аварии.

ПРИМЕЧАНИЕ Выполнение планов обслуживания

Планы обслуживания основаны на задачах служб SQL Server Integration (SSIS). Поэтому при выполнении плана обслуживания сначала загружается модуль SSIS. Затем .Net Framework интерпретирует задачи в пакете, строит необходимые выражения для создания резервной копии и выполняет сгенерированный код.

Сертификаты и главные ключи

Каждый экземпляр сервера всегда имеет *главный ключ службы* (service master key). Кроме того, главные ключи и сертификаты могут быть у баз данных. Нужно создавать резервные копии сертификатов и главных ключей — это необходимо для обеспечения возможности полного восстановления экземпляра.

Главный ключ службы создается автоматически при первом запуске экземпляра сервера. Он обновляется, когда изменяется учетная запись службы SQL Server или ее пароль. Первое, что следует выполнить после запуска экземпляра, — это создать резервную копию главного ключа службы. Это же необходимо сделать сразу после изменения учетной записи службы или ее пароля. Общий синтаксис создания резервной копии главного ключа службы следующий:

```
BACKUP SERVICE MASTER KEY TO FILE = 'путь_к_файлу'  
    ENCRYPTION BY PASSWORD = 'пароль'
```

Главные ключи базы данных (database master key, DMK) создаются перед созданием сертификата, симметричного или асимметричного ключа. Как объясняется в главе 11, DMK является корнем иерархии шифрования базы данных. Чтобы в будущем гарантировать доступ к сертификатам, симметричным и асимметричным ключам базы данных, необходимо иметь резервную копию DMK. Ее следует создавать сразу после создания DMK. Общий синтаксис создания резервной копии DMK таков:

```
BACKUP MASTER KEY TO FILE = 'путь_к_файлу'  
    ENCRYPTION BY PASSWORD = 'пароль'
```

Перед созданием резервной копии DMK главный ключ БД надо открыть. По умолчанию DMK зашифрован главным ключом службы. Если DMK зашифрован только паролем, то сначала он его надо открыть командой:

```
USE <имя_базы_данных>;  
OPEN MASTER KEY DECRYPTION BY PASSWORD = '<здесь_надо_задать_надежный_пароль>';
```

Сертификаты используются для шифрования данных, а также для цифрового подписания исполняемых модулей. Хотя в случае потери сертификата, его можно создать заново, однако исходный сертификат необходим для обращения к зашифрованной с его помощью информации. Сертификату соответствуют открытый и закрытый ключи. Можно создавать резервную копию только открытого ключа следующей командой:

```
BACKUP CERTIFICATE <имя_сертификата> TO FILE = 'путь_к_файлу'
```

В случае восстановления из резервной копии сертификата, содержащего только открытый ключ, SQL Server генерирует новый закрытый ключ. К сожалению, закрытый ключ является важным элементом сертификата, исполь-

зуемым для шифрования и расшифровки информации в SQL Server. Поэтому необходимо обеспечить резервное копирование как открытого, так и закрытого ключей сертификата. Как и в случае с главным ключом, резервную копию сертификата надо создавать сразу после его создания, используя команду:

```
BACKUP CERTIFICATE <имя_сертификата> TO FILE = 'путь_к_файлу'  
[ WITH PRIVATE KEY  
  ( FILE = 'путь_к_файлу_закрытого_ключа',  
    ENCRYPTION BY PASSWORD = 'пароль_шифрования'  
    [, DECRYPTION BY PASSWORD = 'пароль_расшифровки' ] ) ]
```

Хранилище резервных копий

Чтобы восстановить базу данных после аварии, нужен доступ к ее резервной копии. Поскольку авария может затронуть все предприятие, резервные копии следует хранить вне его. Однако резервные копии, перемещаемые в хранилища вне предприятия, создают риск нарушения безопасности, поскольку информация выходит из-под механизмов управления доступом в Active Directory и сети предприятия. Поэтому необходимо предпринять соответствующие меры физической безопасности, чтобы гарантировать сохранность резервных копий.

Главные ключи и сертификаты накладывают дополнительные ограничения на внешнее хранилище. Как правило, у организации более одного внешнего поставщика услуг сбора и хранения резервных копий предприятия. Если бы кто-нибудь попытался украсть резервную копию базы данных, хранящей зашифрованную информацию, он все равно не смог бы получить доступ к зашифрованным данным без главных ключей или сертификатов. Поэтому, несмотря на необходимость создания резервных копий сертификатов и главных ключей, они никогда не должны храниться вместе с резервными копиями баз данных, к которым относятся.

ПРИМЕЧАНИЕ Главные ключи

Подробнее об использовании и управлении сертификатами и главными ключами см. главу 11.

Проверка резервных копий

Резервные копии являются страховкой на случай повреждения базы данных, поэтому нужно быть уверенным, что созданные резервные копии исправны и пригодны для использования. Для проверки резервной копии служит команда:

```
RESTORE VERIFYONLY FROM <устройство_резервного_копирования>
```

При проверке резервной копии SQL Server выполняет следующие действия:

- вычисляет контрольную сумму резервной копии и сравнивает ее с контрольной суммой, хранящейся в файле резервной копии;
- проверяет правильность и целостность заголовка резервной копии;
- проходит по цепочке страниц базы данных, проверяя их наличие и доступность.

Контрольные вопросы

1. Какие существуют четыре типа резервных копий?
2. Как обнаружить и зарегистрировать поврежденные страницы?

Ответы на контрольные вопросы

1. Могут создаваться полные и разностные резервные копии базы данных, а также резервные копии журнала транзакций и файлов или файловых групп.
2. Выполнить команду `ALTERDATABASE <имя_базы_данных> SET PAGE_VERIFY CHECKSUM.`

Практикум. Создание резервных копий баз данных

На этом практикуме вы создадите полную, разностную резервные копии и резервную копию журнала транзакций.

ПРИМЕЧАНИЕ

Перед выполнением заданий убедитесь, что в базе данных *AdventureWorks* выбрана полная модель восстановления.

Задание 1. Создание сжатой полной зеркальной резервной копии

Вы создадите сжатую резервную копию базы данных *AdventureWorks* и ее зеркальное отражение для избыточности и проверки контрольных сумм страниц.

1. Чтобы создать резервную копию базы данных *AdventureWorks*, выполните следующий код:

```
BACKUP DATABASE AdventureWorks
TO DISK = 'c:\test\AdventureWorks_1.bak'
MIRROR TO DISK = 'c:\test\AdventureWorks_2.bak'
WITH COMPRESSION, INIT, FORMAT, CHECKSUM, STOP_ON_ERROR
GO
```

Задание 2. Создание резервной копии журнала транзакций

Вы создадите пару резервных копий журнала транзакций базы данных *AdventureWorks*.

1. Чтобы изменить данные и создать первую резервную копию журнала транзакций, выполните следующий код:

```
USE AdventureWorks
GO

INSERT INTO HumanResources.Department
(Name, GroupName)
VALUES('Test1', 'Research and Development')
GO

BACKUP LOG AdventureWorks
TO DISK = 'c:\test\AdventureWorks_1.trn'
WITH COMPRESSION, INIT, CHECKSUM, STOP_ON_ERROR
GO
```

2. Чтобы внести в данные дополнительные изменения и создать вторую резервную копию журнала транзакций, выполните следующий код:

```
INSERT INTO HumanResources.Department
(Name, GroupName)
VALUES('Test2', 'Research and Development')
GO

BACKUP LOG AdventureWorks
TO DISK = 'c:\test\AdventureWorks_2.trn'
WITH COMPRESSION, INIT, CHECKSUM, STOP_ON_ERROR
GO
```

Задание 3. Создание разностной резервной копии

Вы создадите разностную резервную копию базы данных *Adventure Works*.

1. Чтобы выполнить еще две транзакции запустите следующий код:

```
USE AdventureWorks
GO

INSERT INTO HumanResources.Department
(Name, GroupName)
VALUES('Test3', 'Research and Development')
GO
```

2. Выполните следующую команду создания разностной резервной копии:

```
BACKUP DATABASE AdventureWorks
TO DISK = 'c:\test\AdventureWorks_1.dif'
MIRROR TO DISK = 'c:\test\AdventureWorks_2.dif'
WITH DIFFERENTIAL, COMPRESSION, INIT, FORMAT, CHECKSUM, STOP_ON_ERROR
GO
```

Резюме занятия

- Полная резервная копия является начальной точкой любой процедуры резервного копирования и процесса восстановления. Полная резервная копия содержит только используемые страницы базы данных.
- Разностная резервная копия включает все страницы, измененные после создания последней полной резервной копии, и используется для уменьшения объема резервных копий журнала транзакций, необходимых для восстановления.
- Резервная копия журнала транзакций содержит все изменения, произошедшие после создания предыдущей резервной копии журнала транзакций.
- Для создания резервной копии журнала транзакций необходимо, чтобы в базе данных была выбрана либо полная модель восстановления, либо модель с неполным протоколированием, была создана полная резервная копия, а журнал транзакций был после этого усечен.
- Параметр *READ_WRITE_FILEGROUPS* команды *BACKUP DATABASE* позволяет включать в резервные копии только изменяемые файловые группы.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Вы администратор баз данных компании Fabrikam. Для деятельности компании критически важна база данных *Orders*. В этой БД выбрана полная модель восстановления. Полная резервная копия создается ежедневно в 1:00, разностные резервные копии — каждые 4 часа, начиная с 5:00, а резервные копии журнала транзакций — каждые 5 минут. Если база данных *Orders* будет повреждена и перестанет работать, что нужно выполнить в первую очередь для ее восстановления?
 - A. Восстановить БД из самой последней полной резервной копии, используя параметр *NORECOVER*.
 - B. Восстановить БД из самой последней разностной резервной копии, используя параметр *NORECOVER*.
 - C. Создать резервную копию журнала транзакций с параметром *NO_TRUNCATE*.
 - D. Создать резервную копию журнала транзакций с параметром *TRUNCATE_ONLY*.

Занятие 2. Восстановление баз данных

В повседневной жизни люди приобретают разнообразные страховки, надеясь, что им никогда не придется ими воспользоваться, но если даже придется воспользоваться страховкой, она обеспечит финансовую защиту. Резервные копии являются страховками сохранности информации. Мы надеемся, что нам никогда не придется их использовать, но в случае аварии, они позволят восстановить данные и продолжить работу компании. На этом занятии вы научитесь пользоваться резервными копиями для восстановления среды SQL Server. Кроме того, поскольку восстановление базы данных зависит от состояния журнала транзакций, здесь будет немного рассказано про его внутреннее устройство.

Изучив материал этого занятия, вы сможете:

- ✓ восстанавливать базы данных.

Продолжительность занятия — 20 минут.

Внутреннее устройство журнала транзакций

Журнал транзакций состоит из одного или нескольких файлов, связанных с базой данных, в которых отслеживаются все изменения, внесенные в данные или объекты базы данных. Журналы транзакций не распространяются далее одной базы данных, поэтому бизнес-операции, выполняемые в нескольких базах данных, реализуются как отдельные транзакции в каждой относящейся к ней базе данных. Журнал транзакций должен хранить достаточно информации, чтобы SQL Server при перезапуске мог восстановить базу данных.

Важной частью информации, хранящейся в журнале транзакций, является номер LSN. Отсчет LSN начинается при создании базы данных и этот номер увеличивается без ограничений. LSN всегда увеличивается, никогда не повторяется и не может быть возвращен в предыдущее значение. Любой оператор, влияющий на состояние базы данных, увеличивает LSN.

Каждая единица хранения базы данных отслеживает LSN последнего изменения, выполненного над структурой хранения. На уровне базы данных номер LSN последнего изменения базы данных хранится в заголовке ее главного файла данных. На уровне файла данных LSN последнего изменения страницы в этом файле хранится в заголовке этого файла. Каждая страница данных в базе данных также хранит LSN последнего изменения в ней.

Все изменения данных происходят в буферах памяти. При выполнении изменения модифицируется соответствующий буфер и добавляется запись в журнал транзакций. Измененная страница помечается в буферном пуле как «грязная». Каждая «грязная» страница отслеживает в журнале транзакций номер LSN, соответствующий изменению, которое модифицировало страницу в буферном пуле. Когда SQL Server фиксирует контрольную точку, все «грязные» страницы в буферном пуле записываются в файл данных.

В процессе сохранения контрольной точки SQL Server сравнивает LSN «грязной» страницы из буферного пула с LSN страницы данных на диске. Если LSN страницы данных на диске меньше или равен LSN «грязной» страницы в буферном пуле, а также меньше или равен LSN файла данных, то страница на диске перезаписывается страницей из буферного пула. Если LSN «грязной» страницы меньше, чем LSN страницы на диске или файла данных, содержащего страницу, страница в буферном пуле перезаписывается страницей с диска. Когда процесс сохранения контрольной точки завершает запись «грязных» страниц в файл данных, в заголовок каждого файла заносится наибольший LSN, записанный в него. Кроме того, наибольший LSN, записанный в процессе сохранения контрольной точки, записывается в заголовок главного файла данных. SQL Server обеспечивает, чтобы LSN всех страниц файла был меньше или равен LSN файла, а LSN всех файлов базы данных — меньше или равен LSN базы данных. Процесс завершается тем, что во всех «грязных» страницах, затронутых контрольной точкой, сбрасывается флаг, информирующий, что страница изменена.

При запуске SQL Server каждая база данных проходит процедуру, называемую «восстановление при перезапуске». Оно выполняется в два этапа — REDO и UNDO. На этапе REDO все зафиксированные транзакции журнала транзакций сбрасываются на диск. На этапе REDO применяется та же логика, что и в процессе сохранения контрольной точки. Если LSN страницы меньше или равен LSN записи журнала, записываемой в эту страницу, то изменение записывается. В противном случае оно отбрасывается как уже зафиксированное на диске. При завершении этапа REDO начинается этап UNDO, на котором просматривается журнал транзакций и аннулируются все открытые транзакции, что предотвращает попадание незафиксированных транзакций на диск. В завершении этапа UNDO база данных проходит процесс, известный, как «накат» (roll forward). При накате базы данных SQL Server считывает последний LSN, сохраненный в журнале транзакций, увеличивает его на единицу и записывает

новый LSN в заголовки всех файлов базы данных, предотвращая запись в файлы данных транзакций, которые «старше» точки наката.

В каждой созданной резервной копии хранятся минимальный и максимальный LSN базы данных на момент создания резервной копии. Поскольку полная резервная копия содержит часть журнала транзакций, сгенерированную за время ее построения, то она соответствует моменту завершения ее создания и хранит только последний используемый в ней LSN. В разностные резервные копии и в резервные копии журнала транзакций записывается LSN базы данных на начало их создания, а также LSN базы данных при завершении их создания.

Поскольку LSN всегда возрастает, то для определения применимости резервной копии к базе данных, SQL Server достаточно сравнить текущий LSN с LSN, записанным(и) в резервной копии. Если резервная копия содержит следующий по порядку LSN, то она может быть восстановлена. Если в резервной копии не содержится следующего по порядку LSN, генерируется ошибка, а процесс восстановления прерывается и никаких изменений не происходит.

Чтобы запустить последовательность восстановления, необходима полная резервная копия или резервная копия файловой группы, после чего могут применяться разностные резервные копии или резервные копии журнала транзакций. Однако для восстановления из дополнительных разностных резервных копий или резервных копий журнала транзакций база данных или файловая группа должны быть в состоянии восстановления. Любая попытка восстановления из разностной резервной копии или резервной копии журнала транзакций в базу данных или файловую группу, не находящиеся в состоянии восстановления, приведет к ошибке.

Долгое время многие люди ошибочно утверждали, что разностная резервная копия или резервная копия журнала транзакций не могут применяться к восстановленной базе данных, поскольку в конце процесса восстановления LSN увеличивается и становится не совместимым с любой разностной резервной копией, либо резервной копией журнала транзакций. На самом деле в этом случае SQL Server не отвергает разностную резервную копию или резервную копию журнала транзакций по причине несоответствия LSN. Разностная резервная копия или резервная копия журнала транзакций относятся к полной резервной копии, либо к резервной копии файловой группы. В восстановленной базе данных могут выполняться транзакции, которые делают состояние базы данных несовместимым с любыми разностными резервными копиями и резервными копиями журнала транзакций. Поскольку транзакции не могут исполняться в базе данных или файловой группе, находящихся в состоянии восстановления, то SQL Server достаточно просто удостовериться, что база данных или файловая группа находятся в этом состоянии, чтобы выполнить последующую проверку совместимости LSN.

ПРИМЕЧАНИЕ

Подробнее о том, как SQL Server обрабатывает транзакции, и о структурах файлов данных и журналов транзакций, см. книгу Калена Делани «Microsoft SQL Server 2008 Internals» (Microsoft Press, 2009, Kalen Delaney).

Восстановление баз данных

Любая процедура восстановления начинается либо с полной резервной копии, либо с резервной копии файловой группы. Процесс восстановления резервных копий можно остановить в любой момент и разрешить в базе данных выполнение транзакций. После перевода восстанавливаемой базы данных или файловой группы в рабочее состояние уже нельзя будет применить к базе данных никакие разностные резервные копии или резервные копии журнала транзакций.

Восстановление из полной резервной копии

Общий синтаксис восстановления из полной резервной копии:

```
RESTORE DATABASE { имя_базы_данных | @переменная_с_именем_базы_данных }
[ FROM <устройство_резервного_копирования> [ ,...n ] ]
[ WITH [ { RECOVERY | NORECOVERY |
  STANDBY = {имя_резервного_файла | @переменная_с_именем_резервного_файла } ]
  | , <параметры_WITH_общие > [ ,...n ]
  | , <параметр_WITH_репликации>
  | , <параметр_WITH_отслеживания_изменения_данных>
  | , <параметры_WITH_компонента_Service_Broker>
  | , <параметры_WITH_на_момент_вермени-RESTORE_DATABASE>
  } [ ,...n ]
]
```

<параметры_WITH_общие> [,...n] ::=

--Параметры операции восстановления

```
MOVE 'логическое_имя_файла_в_резервной_копии' TO 'имя_файла_в_
операционной_системе'
[ ,...n ] | REPLACE | RESTART | RESTRICTED_USER
```

Если база данных еще не существует на экземпляре, то при выполнении команды *RESTORE SQL Server* создает базу данных со всеми ее файлами. В ходе этого процесса создаются все файлы, размеры которых совпадают с размерами исходных файлов на момент создания резервной копии. Завершив создание файлов, SQL Server начинает восстанавливать страницы базы данных из резервной копии.

СОВЕТ Восстановление существующей базы данных

Создание и изменение размеров всех связанных с базой данных файлов требует значительного времени. Если база данных уже существует, то, как будет описано ниже, резервную копию следует развертывать поверх существующей базы данных.

Параметр *REPLACE* применяется для выполнения восстановления поверх существующей базы данных.

Резервное копирование базы данных выполняется гораздо чаще восстановления, поэтому создание резервной копии оптимизировано, чтобы выполняться за минимальное время. Поэтому при создании резервной копии SQL Server помещает страницы в нее, игнорируя их порядок. Однако при восстановлении базы данных страницы должны размещаться в ней последовательно. SQL Server должен последовательно найти в файлах страницу 1, 2, 3 и т. д. Эмпириче-

ское правило таково: восстановление занимает приблизительно на 30% больше времени, чем на создание исходной резервной копии.

В первых страницах базы данных хранится информация о ее структуре, в том числе список страниц базы данных. После восстановления первой страницы все, что находится на диске в тот момент, становится недействительным. Если процесс восстановления поверх существующей базы данных прерывается, то вся она становится недоступной, пока не будет выполнена полная операция восстановления. При восстановлении файла или файловой группы затрагивается только восстанавливаемая часть базы данных.

Чтобы база данных после завершения операции *RESTORE* переходила в рабочий режим и позволяла выполнять транзакции, следует указать параметр *RECOVERY*. Если *RESTORE* запускается с параметром *NORECOVERY*, по завершении восстановления база данных остается в состоянии *RECOVERING*, позволяя применять последующие разностные резервные копии или резервные копии журнала транзакций. Параметр *STANDBY* применяется, чтобы разрешить запуск в базе данных выражения *SELECT*, пока происходит восстановление разностных резервных копий и/или резервных копий журнала транзакций. При восстановлении базы данных с параметром *STANDBY* создается дополнительный файл, что обеспечивает целостность базы данных на момент применения последнего восстановления.

Файловая система компьютера, где восстанавливается база данных, может не совпадать с файловой системой компьютера, на котором была создана резервная копия, либо процессе восстановления может потребоваться изменить расположение файлов базы данных. Для изменения расположения одного или нескольких файлов данных служит параметр *MOVE*.

Пути восстановления

Перед восстановлением базы данных сначала необходимо провести инвентаризацию имеющихся резервных копий и состояние базы данных, к которой применима та или иная копия. В табл. 9-1 представлены основные состояния базы данных для каждой резервной копии, созданной на занятии 1.

Табл. 9-1. Изменение базы данных

Созданная резервная копия	Изменение данных
Полная резервная копия	Добавление Test1
Резервная копия журнала транзакций	Добавление Test2
Резервная копия журнала транзакций	Добавление Test3
Разностная резервная копия	

Независимо от момента времени, на который требуется восстановить базу данных, сначала необходимо применить полную резервную копию. Если нужно восстановить базу данных сразу за моментом добавления отдела Test1, следует применить первую резервную копию журнала транзакций и перевести базу

данных в рабочий режим. При этом отделы Test2 и Test3 будут потеряны. Точно так же, если нужно восстановить базу данных на момент перед добавлением отдела Test3, то следует применить полную резервную копию, а затем — первую и вторую резервную копию журнала транзакций, после чего перевести базу данных в рабочий режим.

Если требуется восстановить базу данных без потери каких-либо данных, то просто нужно применить полную резервную копию, а затем — разностную копию, поскольку последняя содержит все изменения из обеих резервных копий журнала транзакций. Что случится, однако, если после применения полной резервной копии обнаружится, что разностная резервная копия не пригодна для использования из-за повреждения? В этом случае можно будет применить обе резервные копии журнала транзакций, но отдел Test3 будет утерян безвозвратно.

Для обеспечения большей гибкости процесса восстановления первым шагом в нем должно быть создание резервной копии журнала транзакций исходной базы данных. Очевидно, что если вся исходная база данных пропадет, нельзя будет перед началом операции восстановления создать окончательную резервную копию журнала транзакций. Однако пока цел журнал транзакций и главная база данных содержит запись поврежденной базы данных, можно выполнить команду *BACKUP LOG* над базой данных, даже если ее файлы данных повреждены или отсутствуют. Этап процесса восстановления, когда сначала создается окончательная резервная копия журнала транзакций, называется *созданием резервной копии заключительного фрагмента журнала* (backing up the tail of the log).

Восстановление из разностной резервной копии

Синтаксис разностного восстановления такой же, что и при полном восстановлении базы данных. После восстановления из полной резервной копии можно восстановить самую последнюю разностную резервную копию.

Восстановление из резервной копии журнала транзакций

Общий синтаксис восстановления из резервной копии журнала транзакций:

```
RESTORE LOG { имя_базы_данных | @переменная_с_именем_базы_данных }
[ <файл_или_файловая_группа_или_страницы> [ ,...n ] ]
[ FROM <устройство_резервного_копирования> [ ,...n ] ]
[ WITH { [ RECOVERY | NORECOVERY |
    STANDBY = {имя_резервного_файла | @переменная_с_именем_резервного_файла } ]
    | , <параметры_WITH_общие> [ ,...n ]
    | , <параметр_WITH_репликации>
    | , <параметры_WITH_на_определенный_момент_времени-RESTORE_LOG>
< параметры_WITH_на_определенный_момент_времени-RESTORE_LOG>::=
| { STOPAT = { 'дата_и_время' | @переменная_с_датой_и_временем }
| STOPATMARK = { 'имя_метки' | 'lsn:номер_lsn' }
[ AFTER 'дата_и_время' ]
| STOPBEFOREMARK = { 'имя_метки' | 'lsn:номер_lsn' }
[ AFTER 'дата_и_время' ]
```

Иногда требуется восстановить базу данных, но не все выполненные транзакции. Можно сделать так, чтобы при восстановлении журнала транзакций, SQL Server воспроизвел только его часть, выполнив, так называемое, *восста-*

новление на определенный момент времени (point-in-time restore). Команда *STOPAT* позволяет указать дату и время, по состоянию на которое должна быть восстановлена база данных. Параметры *STOPATMARK* и *STOPBEFOREMARK* позволяют задать LSN или метку журнала транзакции в качестве конечной точки операции восстановления.

Оперативное восстановление

Состояние базы данных определяет ее доступность и операции, которые в ней могут выполняться. Например, база данных в состоянии *ONLINE* разрешает транзакции и выполнение любых операций, но в состоянии *EMERGENCY* доступна только операция *SELECT* и только членам роли *db_owner*.

Каждая файловая группа в базе данных может иметь свое состояние. В одно время одна файловая группа может находиться в состоянии *RESTORING* и быть недоступной, а другая — в состоянии *ONLINE* и позволять выполнение транзакций. Состояние базы данных соответствует состоянию главной файловой группы (*PRIMARY*).

SQL Server 2008 Enterprise допускает выполнение операций восстановления работающей и доступной базы данных. Однако, поскольку полная резервная копия влияет на всю базу данных, а состояние базы данных определяется состоянием главной файловой группы, то в этом случае восстанавливаемая база данных недоступна. Для оперативного восстановления подходит восстановление файлов или файловых групп, но не главной файловой группы или ее файлов.

Восстановление файловой группы влияет только на часть базы данных и называется частичным восстановлением (partial restore).

Восстановление поврежденных страниц

Повреждение страниц возникает при нарушении целостности данных, которое имеет место, если контрольная сумма страницы не соответствует ее содержанию или строка записывается на диск не полностью. Повреждения страниц обычно случаются из-за сбоев контроллера диска.

Если повреждение страницы произошло в индексе, то для ее исправления нет необходимости выполнять восстановление. Достаточно удалить индекс и создать заново.

Однако если повреждена страница данных в таблице или первичный ключ, то для исправления потребуется восстановление. Кроме отдельных файловых групп, можно восстанавливать отдельные страницы базы данных.

К восстановлению страницы предъявляются следующие требования:

- база данных должна иметь либо полную модель восстановления, либо модель восстановления с неполным протоколированием;
- необходимо иметь возможность создать резервную копию журнала транзакций;
- восстановление страницы возможно только в файловой группе доступной для чтения и записи;
- требуется наличие полной резервной копии, резервной копии файла или файловой группы;
- восстановление страницы не может выполняться одновременно с другими операциями восстановления.

Любой выпуск SQL Server 2008 поддерживает восстановление одной или нескольких страниц в автономной базе данных. SQL Server 2008 Enterprise позволяет восстанавливать страницу в оперативной базе данных и даже в оперативной файловой группе, содержащей поврежденную страницу. Однако любые операции, в которых выполняется попытка получить доступ к этой странице во время восстановления, завершаться ошибкой и неудачей.

Синтаксис восстановления страницы:

```
RESTORE DATABASE имя_базы_данных  
PAGE = 'file:page [ ,...n ]' [ ,...n ]  
FROM <устройство_резервного_копирования> [ ,...n ]  
WITH NORECOVERY
```

Процедура восстановления поврежденной страницы такова:

1. Получить PageID поврежденной страницы.
2. Используя самую последнюю полную резервную копию, резервную копию файла или файловой группы выполнить команду:

```
RESTORE DATABASE имя_базы_данных  
PAGE = 'файл:страница [ ,...n ]' [ ,...n ]  
FROM <устройство_резервного_копирования> [ ,...n ]  
WITH NORECOVERY
```

3. Восстановить все разностные резервные копии, используя параметр *NORECOVERY*.
4. Восстановить все дополнительные резервные копии журнала транзакций, используя параметр *NORECOVERY*.
5. Создать резервную копию журнала транзакций.
6. Восстановить разностную резервную копию журнала транзакций, созданную на предыдущем шаге, с параметром *WITH RECOVERY*.

ПРИМЕЧАНИЕ На экзамене

Необходимо знать все этапы процедуры восстановления отдельной страницы базы данных.

Восстановление при ошибках носителя резервной копии

Как говорилось ранее, страницы восстанавливаются последовательно и как только в базе данных восстановлена первая страница, все содержимое базы становится недействительным. Если затем обнаружатся проблемы с носителем резервной копии и восстановление будет прервано, то вы останетесь с дефектной базой данных, непригодной для использования. Если только что перезаписанная база данных была единственной или если это была единственная резервная копия базы данных, то результатом будет не только потеря работоспособности базы данных, но и всей информации.

SQL Server способен продолжать операцию восстановления даже в случае повреждения носителя резервной копии. При обнаружении нечитаемого участка файла резервной копии, SQL Server может пропустить источник повреждения и продолжить восстановление, насколько это возможно. Эта функция известна как *максимально возможное восстановление* (best effort restore).

По завершении восстановления база данных переводится в режим *EMERGENCY*. После этого администратор должен подключиться к базе данных и выяснить ее пригодность для использования. Если база данных кажется пригодной, то администратор может перевести ее в состояние *ONLINE*. В противном случае имеется возможность прочитать как можно больше информации из базы данных, пока она находится в режиме *EMERGENCY*.

Для восстановления с поврежденного носителя резервной копии в команде *RESTORE DATABASE* или *RESTORE LOG* следует указать параметр *CONTINUE_AFTER_ERROR*.

Контрольные вопросы

1. При какой модели восстановления всегда можно восстановить базу данных до момента сбоя, если только имеется возможность создать резервную копию заключительного фрагмента журнала?
2. Какое действие должно выполняться первым при любой операции восстановления?

Ответы на контрольные вопросы

1. Модель полного восстановления.
2. Создание резервной копии заключительного фрагмента журнала.

Практикум. Восстановление базы данных

На этом упражнении вы восстановите базу данных *AdventureWorks* из резервных копий, созданных на практикуме 1-го занятия.

Задание 1. Намеренное повреждение базы данных

Вы намеренно повредите базу данных *AdventureWorks*, так чтобы для доступа к данным потребовалось восстановление.

1. Выполните следующий запрос на добавление новой строки в базу данных *AdventureWorks*:

```
USE AdventureWorks
GO

INSERT INTO HumanResources.Department
(Name, GroupName)
VALUES('Test4', 'Research and Development')
GO
```

2. Откройте Диспетчер конфигурации SQL Server (SQL Server Configuration Manager) и остановите службу **SQL Server** своего экземпляра SQL Server 2008.
3. Откройте Проводник Windows и удалите файл AdventureWorks.mdf. При этом файл AdventureWorks.ldf удалять не надо.
4. В Диспетчере конфигурации SQL Server запустите службу **SQL Server** своего экземпляра SQL Server 2008.

5. Подключитесь к экземпляру средствами SSMS.
6. Проверьте, что хотя запись базы данных *AdventureWorks*, еще существует, но сама база данных абсолютно недоступна из-за отсутствия файла данных.

Задание 2. Восстановление из полной резервной копии

Вы восстановите базу данных *AdventureWorks* из полной резервной копии, созданной на практике 1-го занятия.

ПРИМЕЧАНИЕ

Для восстановления базы данных требуется исключительный доступ. Перед выполнением любых действий по восстановлению следует убедиться, что нет открытых подключений к базе данных *AdventureWorks*.

1. Первый шаг процесса восстановления — создание резервной копии заключительного фрагмента журнала. Откройте новое окно запроса и выполните следующий код:

```
BACKUP LOG AdventureWorks
TO DISK = 'c:\test\AdventureWorks_3.trn'
WITH COMPRESSION, INIT, NO_TRUNCATE
GO
```

2. Получив заключительный фрагмент журнала, выполните следующий код для восстановления из полной резервной копии:

```
RESTORE DATABASE AdventureWorks
FROM DISK = 'c:\test\AdventureWorks_1.bak'
WITH STANDBY = 'c:\test\AdventureWorks.stn'
GO
```

3. Проверьте, что данные можно прочесть, но не изменяйте их.
4. Убедитесь, что добавленные отделы отсутствуют в базе данных.

Задание 3. Восстановление из разностной резервной копии

Вы примените разностную резервную копию на базе данных *AdventureWorks* и, выполнив проверку, переведете базу данных в оперативный режим.

1. Примените разностную резервную копию, выполнив следующий код:

```
RESTORE DATABASE AdventureWorks
FROM DISK = 'c:\test\AdventureWorks_1.dif'
WITH STANDBY = 'c:\test\AdventureWorks.stn'
GO
```

3. Убедитесь, что отделы Test1, Test2 и Test3 существуют.
4. Чтобы завершить процесс восстановления и вернуть базу данных в рабочее состояние, выполните команду:

```
RESTORE DATABASE AdventureWorks
WITH RECOVERY
GO
```

Задание 4. Восстановление из резервной копии журнала транзакций

Вы восстановите базу данных *AdventureWorks*, используя три созданные ранее резервные копии журнала транзакций.

1. Разверните полную резервную копию базы данных *AdventureWorks*, выполнив код:

```
RESTORE DATABASE AdventureWorks
FROM DISK = 'c:\test\AdventureWorks_1.bak'
WITH STANDBY = 'c:\test\AdventureWorks.stn',
REPLACE
GO
```

2. Убедитесь, что в таблице *Department* отсутствуют данные.
3. Примените первую резервную копию журнала транзакций, выполнив код:

```
RESTORE DATABASE AdventureWorks
FROM DISK = 'c:\test\AdventureWorks_1.trn'
WITH STANDBY = 'c:\test\AdventureWorks.stn'
GO
```

4. Убедитесь, что появился отдел *Test1*.
5. Примените вторую резервную копию журнала транзакций, выполнив код:

```
RESTORE DATABASE AdventureWorks
FROM DISK = 'c:\test\AdventureWorks_2.trn'
WITH STANDBY = 'c:\test\AdventureWorks.stn'
GO
```

6. Убедитесь, что появился отдел *Test2*.
7. Примените третью резервную копию журнала транзакций, выполнив код:

```
RESTORE DATABASE AdventureWorks
FROM DISK = 'c:\test\AdventureWorks_3.trn'
WITH STANDBY = 'c:\test\AdventureWorks.stn'
GO
```

8. Убедитесь, что появились отделы *Test3* и *Test4*.
9. Выполните следующий код, чтобы разрешить доступ и выполнение транзакций в базе данных:

```
RESTORE DATABASE AdventureWorks
WITH RECOVERY
GO
```

Резюме занятия

- Вначале любой процедуры восстановления создается резервная копия заключительного фрагмента журнала.
- Последовательность восстановления начинается с восстановления из полной резервной копии, резервной копии файла или файловой группы.
- Если восстановление выполнять с параметром *NORECOVERY*, позднее можно применять разностные резервные копии или резервные копии журнала транзакций.

- При выполнении восстановления с параметром *RECOVERY* база данных возвращается в оперативный режим, после чего к ней уже невозможно применить какие-либо дополнительные резервные копии.
- Если содержимое базы данных должно быть доступно для чтения и при этом сохранялась возможность восстановления дополнительных разностных резервных копий или резервных копий журнала транзакций, при восстановлении нужно воспользоваться параметром *STANDBY*.
- Создать резервную копию журнала транзакций можно, пока цел ее файл, даже если повреждены все файлы данных базы данных.
- Восстановление может быть продолжено после обнаружения дефекта носителя резервной копии, если задан параметр *CONTINUE_PAST_ERRORS*. В этом случае после восстановления база данных остается в режиме *EMERGENCY*.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Сервер, где располагалась база данных *Customer*, вышел из строя и должен быть заменен. Вы развернули новый сервер и установили SQL Server 2008. В процессе наладки нового сервера вы решили использовать другую букву диска и структуру папок для файлов данных и журнала. С каким параметром следует восстанавливать базу данных *Customers* на новом сервере?
 - A. *NORECOVERY*
 - B. *CONTINUE_PAST_ERRORS*
 - C. *MOVE*
 - D. *PARTIAL*

Занятие 3. Моментальный снимок базы данных

Моментальные снимки баз данных в SQL Server 2005 позволяли пользователям быстро создавать доступные только для чтения копии данных. На этом занятии вы узнаете, как создавать моментальные снимки и использовать их для возвращения информации или базы данных к состоянию на нужный момент времени в прошлом.

ПРИМЕЧАНИЕ Моментальный снимок базы данных

В SQL Server 2008 моментальные снимки базы данных доступны только в выпуске Enterprise.

ВНИМАНИЕ! Управление данными FILESTREAM

Моментальные снимки базы данных не совместимы с данными *FILESTREAM*. При создании моментального снимка базы данных с данными *FILESTREAM* файловая группа *FILESTREAM* будет отключена и недоступна.

Изучив материал этого занятия, вы сможете:

- ✓ создавать моментальные снимки базы данных;
- ✓ возвращать информацию или базу данных в прежнее состояние, используя моментальный снимок базы данных.

Продолжительность занятия — 20 минут.

Создание моментального снимка базы данных

Создание моментального снимка базы данных очень похоже на создание базы данных. Для этого используют команду *CREATE DATABASE* с предложением *AS SNAPSHOT OF*. Поскольку моментальный снимок является копией базы данных, созданной на определенный момент времени и доступной только для чтения, то указывать журнал транзакций не надо.

Требования для создания моментального снимка базы данных таковы:

- необходимо выбрать все файлы данных исходной базы данных;
- логическое имя каждого файла должно в точности совпадать с именем в исходной базе данных.

Общий синтаксис создания моментального снимка базы данных таков:

```
CREATE DATABASE имя_моментального_снимка_базы_данных
ON
    (NAME = логическое_имя_файла,
     FILENAME = 'имя_файла_в_операционной_системе') [ , ... n ]
AS SNAPSHOT OF имя_исходной_базы_данных
```

Существуют ограничения на моментальный снимок базы данных:

- моментальный снимок нельзя восстановить, отсоединить или создать его резервную копию;
- моментальный снимок должен находиться в том же экземпляре, что и исходная база данных;
- полнотекстовые индексы не поддерживаются;
- данные *FILESTREAM* не поддерживаются и в моментальном снимке недоступны;
- нельзя создать моментальный снимок системной базы данных;
- невозможно удалить, восстановить или отсоединить исходную базу данных, имеющую моментальный снимок;
- нельзя ссылаться на автономные файловые группы, уничтоженные или восстанавливаемые в настоящий момент.

При создании моментального снимка базы данных SQL Server не выделяет для нее место на диске, равное текущему размеру файлов данных исходной базы. Вместо этого SQL Server использует преимущества такого свойства файловой системы, как *разреженные файлы* (sparse files). Разреженный файл является по существу записью в таблице размещения файлов и почти не занимает места на диске. По мере добавления данных к файлу автоматически увеличивается занимаемое им на диске место. Благодаря использованию разреженных файлов время создания моментального снимка базы данных не зависит от размера исходной базы данных.

С точки зрения приложения и выполнения запросов доступ к моментальному снимку базы данных очень прост: снимок базы данных выглядит и работает как обычная доступная только для чтения база данных. Поэтому в моментальном снимке базы данных можно выполнять предложение *SELECT* и использовать его как любую другую базу данных.

На момент создания моментальный снимок базы данных не содержит никакой информации, хотя в нем сразу можно выполнять предложения *SELECT*. SQL Server использует исходную базу данных для получения данных, которые еще не были изменены после создания моментального снимка.

Технология копирования при записи

Поскольку моментальный снимок базы данных должен сохранять состояние информации исходной базы данных на момент своего создания, то для SQL Server необходим способ управления любыми изменениями, происходящими в исходной базе данных. Технология, которую SQL Server применяет для этого, известна как *копирование при записи* (copy-on-write).

Как известно, SQL Server хранит данные в страницах, в каждом экстенсте восемь страниц, а записывает и считывает информацию SQL Server экстенстами. При первом изменении данных на странице в экстенсте SQL Server копирует исходный образ экстенста в моментальный снимок базы данных. В дальнейшем при выполнении в моментальном снимке базы данных предложений *SELECT* SQL Server получает измененные данные из моментального снимка, а для всех неизмененных экстенстов продолжает получать информацию из исходной базы данных.

Записывая оригинальный образ экстенста при первом изменении данных, SQL Server разрешает изменения в исходной базе данных, одновременно гарантируя, что запросы моментального снимка базы данных не будут содержать изменений, произошедших после его создания.

После первого изменения страницы в экстенсте, при котором SQL Server записал экстенст в моментальный снимок базы данных, любые последующие изменения в этом экстенсте игнорируются функцией копирования при записи.

Поскольку в одной исходной базе данных можно создать несколько моментальных снимков, исходный образ экстенста записывается в каждый моментальный снимок базы данных, где нет копии этого снимка.

СОВЕТ Максимальный размер моментальной копии базы данных

Поскольку SQL Server поддерживает состояние моментального снимка базы данных на момент его создания, то его максимальный размер соответствует объему данных, хранящихся в исходной базе данных на время создания моментального снимка.

Возвращение базы данных к прежнему состоянию при помощи моментального снимка

Так как моментальный снимок базы данных содержит всю информацию исходной базы данных на момент своего создания, он позволяет возвращать данные в исходной базе данных к состоянию на момент создания снимка. В исключительных случаях моментальный снимок базы данных может использоваться для возвращения всего содержимого исходной базы данных к состоянию момен-

тального снимка. Например, если необходимо отменить все изменения, произошедшие в базе данных после создания моментального снимка.

Возвращение базы данных к предшествующему состоянию (database revert) — особый вид восстановления данных, которое возможно при наличии моментального снимка базы данных.

Если надо вернуть к прежнему состоянию только одну запись или часть базы данных, можно воспользоваться предложениями *INSERT*, *UPDATE*, *DELETE* или *MERGE*. Кроме этого SQL Server позволяет при необходимости вернуть к прежнему состоянию базу данных целиком, используя ее моментальный снимок. В этом случае исходная база данных возвращается в точности к тому состоянию, в котором она была при создании моментального снимка. Все транзакции, выполненные на ней позднее, теряются.

Синтаксис возвращения базы данных к прежнему состоянию на основе моментального снимка таков:

```
RESTORE DATABASE <имя_базы_данных> FROM DATABASE_SNAPSHOT = <имя_моментального_снимка_базы_данных>
```

Есть несколько ограничений на возвращение исходной базы данных в предшествующее состояние:

- в исходной базе данных может существовать только один моментальный снимок;
- полнотекстовые каталоги в исходной базе данных должны быть удалены и созданы заново, но после завершения восстановления;
- поскольку журнал транзакций перестраивается, то цепочка журнала транзакций будет разорвана;
- как исходная база данных, так и ее моментальный снимок должны быть автономными;
- в исходной базе данных не могут содержаться данные *FILESTREAM*.

ПРИМЕЧАНИЕ На экзамене

Помните, что данные *FILSTREAM* не совместимы с моментальным снимком базы данных. Можно создать моментальный снимок базы данных с данными *FILESTREAM*, но такой снимок нельзя использовать в качестве источника в операции *RESTORE DATABASE*.

Контрольные вопросы

1. Какие две возможности базы данных не совместимы с моментальными снимками?
2. Что необходимо предпринять перед возвращением базы данных к предыдущему состоянию, используя ее моментальный снимок?

Ответы на контрольные вопросы

1. Данные *FILESTREAM* и полнотекстовые индексы.
2. Необходимо удалить все моментальные снимки базы данных за исключением того, который будет использоваться в качестве источника в команде *RESTORE*.

Практикум. Создание моментального снимка базы данных

Вы создадите моментальный снимок базы данных *AdventureWorks*.

1. Выполните следующий код:

```
CREATE DATABASE AdventureWorksSnap ON  
(NAME = N'AdventureWorks2008_Data', FILENAME = N'c:\test\AdventureWorks.  
ds'),  
(NAME = N'S AdventureWorksFT', FILENAME = N'C:\test\AdventureWorks FT.ds')  
AS SNAPSHOT OF AdventureWorks  
GO
```

2. Выполните следующий код, чтобы сравнить структуры исходной базы данных и ее моментального снимка. Обратите внимание на значение столбца *source_database_id* базы данных *master.sys*:

```
SELECT * FROM AdventureWorks.sys.database_files  
SELECT * FROM AdventureWorksSnap.sys.database_files  
SELECT * FROM master.sys.databases  
GO
```

3. В Обозревателе объектов (Object Explorer) раскройте узел моментального снимка базы данных и ознакомьтесь со вновь созданным снимком.
4. Выполните предложение *SELECT* на моментальном снимке и сравните результаты с базой данных *AdventureWorks*.
5. Внесите изменение в данные и сравните результаты между моментальным снимком и базой данных *AdventureWorks*.

Резюме занятия

- Моментальный снимок базы данных — это копия базы данных на определенный момент времени, доступная только для чтения.
- Моментальные снимки не совместимы с данными *FILESTREAM* и с полнотекстовыми индексами.
- Моментальный снимок позволяет вернуть базу данных к предыдущему состоянию.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Для какой базы данных может быть создан моментальный снимок? (Выберите все подходящие ответы.).
 - A. master.
 - B. База данных с полнотекстовым индексом.
 - C. База данных с данными *FILESTREAM*.
 - D. *distribution*.

Закрепление материала главы

Для закрепления знаний и навыков, полученных при изучении этой главы, выполните следующее:

- изучите резюме главы;
- повторите основные термины, использованные в этой главе;
- выполните задания по темам этой главы;
- сделайте предложенные упражнения;
- пройдите пробный экзамен.

Резюме главы

- Резервные копии являются страховкой сохранности данных. Хотя все надеются никогда ими не воспользоваться ими, но в случае аварии, резервные копии позволят восстановить базы данных и продолжить работу компании.
- Приступая к восстановлению базы данных, в первую очередь необходимо создать заключительный фрагмент журнала.
- Каждое восстановление начинается с полной резервной копии, резервной копии файла или файловой группы.
- Резервную копию журнала транзакций можно создать, если база данных находится в режиме полной модели восстановления или модели восстановления с неполным протоколированием.
- Резервная копия журнала транзакций позволяет восстановить базу данных на определенный момент времени, за исключением момента времени, когда выполнялась транзакция с неполным протоколированием.

Основные термины

Понятны ли вам значения приведенных здесь основных терминов? Чтобы проверить свои ответы, загляните в глоссарий на прилагаемом компакт-диске.

- Возвращение базы данных в прежнее состояние ~ database revert.
- Разностная резервная копия ~ differential backup.
- Полная резервная копия ~ full backup.
- Порядковый номер транзакции в журнале (LSN) ~ Log Sequence Number (LSN).
- Оперативное восстановление ~ online restore.
- Повреждение страницы ~ page corruption.
- Частичная резервная копия ~ partial backup.
- Частичное восстановление ~ partial restore.
- Резервная копия заключительного фрагмента журнала ~ tail backup.
- Резервная копия журнала транзакций ~ transaction log backup.
- Цепочка журнала транзакций ~ transaction log chain.

Упражнения

Сейчас вы примените на практике знания и навыки, полученные при изучении этой главы. Ответы на вопросы см. в разделе «Ответы» в конце книги.

Упражнение 1. Разработка стратегии резервного копирования для компании Coho Vineyard

Исходные данные

Обзор компании

Компания Coho Vineyard была основана в 1947 г. как частный семейный винный завод. Вина, производимые Coho Vineyard, удостоивались многих наград и пользовались популярностью, что позволило компании значительно расширить производство. Через несколько лет в рамках стратегии по расширению компании были приобретены несколько существующих винных заводов. Сейчас компания владеет 16 винными заводами — 9 в штатах Вашингтон, Орегон и Калифорния, а остальные 7 в штатах Висконсин и Мичиган. В компании работает 532 человека, из которых 162 — в центральном офисе, где размещаются критические важные для бизнеса серверы. В компании трудится 122 мобильных торговых агента, которым требуется доступ к актуальной информации об имеющихся товарах.

Планируемые изменения

До сих пор у каждого из 16 винных заводов Coho Vineyard был собственный веб-узел, размещенный на площадях завода. Руководство требует объединить сайты всех заводов, так пользователи смогли покупать продукцию всех 16 предприятий в одном интернет-магазине. Вся информация веб-узла должна храниться в базе данных центрального офиса.

После объединения всей информации в центральном офисе, для доставки информации торговым агентам, а также для получения от них заказов, будут использоваться репликация слиянием. Чтобы до завершения проекта по объединению удовлетворить требованиям торговых агентов, информация о наличии товаров на каждом из заводов отправляется в центральный офис ежедневно в конце дня.

Существующая среда

Базы данных

Сейчас каждый винный завод поддерживает собственную базу данных, где хранит всю бизнес-информацию. В конце месяца вся информация переносится в центральный офис и загружается в базы данных, перечисленные в табл. 9-2.

Табл. 9-2. Базы данных Coho Vineyard

База данных	Размер
<i>Customer</i> (клиенты)	180 Мб
<i>Accounting</i> (бухгалтерия)	500 Мб
<i>HR</i> (персонал)	100 Мб
<i>Inventory</i> (склад)	250 Мб
<i>Promotions</i> (реклама)	80 Мб

По завершении объединения баз данных хранилищем информации нового интернет-магазина служит новая база данных *Order*. Кроме того в своей ежедневной работе сотрудники будут обращаться к данным БД *Order* с применением помощи нового разработанного в компании собственными силами веб-приложения.

База данных *HR* содержит конфиденциальную информацию и защищена TDE (Transparent Data Encryption). Кроме этого, данные в таблице *Salary* шифруются с использованием сертификата.

Серверы баз данных

Один сервер под управлением Windows Server 2003 Enterprise в центральном офисе по имени DB1 хранит все БД. На нем установлен SQL Server 2008 Enterprise.

Бизнес-требования

От вас требуется разработать решение для архивирования баз данных *Customer* и *Order*. Стратегия архивирования должна обеспечивать хранение информации о клиентах как минимум на протяжении 6 лет.

В процессе подготовки БД *Order* для архивирования вы создали секционированную таблицу *Order.Sales*, состоящую из двух секций: в первой хранится информация о продажах за текущий, а во второй — за предыдущий месяц. Заказы, размещенные еще раньше, должны перемещаться в другую секционированную таблицу — *Order.Archive*, после чего в секции 1 содержатся все архивные данные, а секция 2 остается пустой. Архивные данные должны находиться в файловой группе, отличной от файловой группы активно используемых данных.

Новый процесс должен ежедневно, не позднее 4 часов утра загружать информацию о складских запасах со всех 16 заводов.

Четыре крупных клиента осуществляют заказы, используя XML-схему *Coho Vineyards* для EDI-транзакций (Electronic Data Interchange). EDI-файлы поступают в 5 часов вечера и должны обрабатываться и загружаться в БД *Customer*, *Accounting* и *Inventory*, каждая из которых содержит таблицы с данными, относящимися к размещаемым заказам. В настоящее время утилита импорта EDI-файлов представляет собой однопоточное приложение на C++, которому требуется от 3 до 6 часов на обработку всех файлов. Чтобы выполнить соглашение с клиентами об условиях обслуживания (Service Level Agreement, SLA), необходимо заканчивать обработку EDI-файлов до 17:30. К моменту завершения проекта по объединению EDI-утилита должна загружать информацию в новую базу данных *Order*.

Требуется обеспечить создание резервных копий всех баз данных всех филиалов. Все производственные базы данных должны работать в режиме полной модели восстановления. В самом плохом случае считается приемлемым потеря данных максимум за последние 5 минут. В случае аварии БД *Customer*, *Inventory*, *Promotions* и *Order* могут быть недоступны максимум 20 минут. В случае аварийного сбоя информация БД *Customer* и *Order* старше полугода может быть недоступна максимум 12 часов.

Ответьте на следующие вопросы.

1. Какие резервные копии нужно создавать для баз данных *Account*, *Inventory* и *Promotions*?
2. Какие резервные копии нужно создавать для баз данных *Customer* и *Order*?
3. Какие резервные копии нужно создавать для базы данных *HR*?

Рекомендуемые упражнения

Чтобы подготовиться к экзамену по темам этой главы, выполните следующие задания.

Создание резервных копий базы данных

- **Задание 1.** Создайте сертификат. Создайте таблицу для хранения данных, зашифрованных этим сертификатом. Создайте резервную копию сертификата и закрытого ключа.
- **Задание 2.** Создайте базу данных с несколькими файловыми группами. Выполните ее полное резервирование, создав нужные резервные копии файловых групп, журнала транзакций и разностные резервные копии.

Восстановление базы данных

- **Задание 1.** Восстановите сертификат и закрытый ключ из резервной копии. Убедитесь, что восстановленный сертификат позволяет расшифровать информацию из таблицы.
- **Задание 2.** Потренируйтесь в восстановлении базы данных на разные моменты времени, используя резервные копии файловых групп, журнала транзакций и разностные резервные копии, созданные ранее.

Пробный экзамен

На прилагаемом компакт-диске содержится пробный экзамен, обеспечивающий широкие возможности самоподготовки. Например, можно проверить, как усвоены темы этой главы либо все темы экзамена 70-432. Можно симитировать настоящий экзамен или выбрать режим обучения, когда после каждого вопроса приводятся правильные ответы с пояснениями.

ПРИМЕЧАНИЕ Пробный экзамен

Подробнее о пробном экзамене см. «Введение».

Автоматизация SQL Server

Занятие 1. Создание заданий **213**

Занятие 2. Создание предупреждений **220**

Чтобы гарантировать защиту данных, необходимо регулярно создавать резервные копии. Кроме этого приходится выполнять различные процедуры обслуживания баз данных, такие как переиндексирование, сжатие файлов или расширение баз данных. В этой главе вы научитесь создавать и планировать задания агента SQL Server, а также настраивать предупреждения об ошибках, требующих внимания или запуска процедур для предупреждения неполадок, пока они не стали причиной более серьезных проблем.

Темы экзамена

- Управление заданиями агента SQL Server.
- Управление предупреждениями агента SQL Server.
- Управление операторами агента SQL Server.
- Выявление неполадок при выполнении заданий агента SQL Server.

Прежде всего

Для изучения материалов этой главы необходимо иметь:

- установленный экземпляр Microsoft SQL Server 2008;
- подключенную базу данных AdventureWorks.

Занятие 1. Создание заданий

Агент SQL Server (SQL Server Agent) — система планирования заданий SQL Server. Без него пришлось бы либо устанавливать отдельную систему планирования, либо администраторам пришлось бы помнить о выполнении различных заданий в определенные моменты времени в течение дня.

Задания представляют собой контейнеры, позволяющие объединить в процесс одну или несколько операций. Хотя большинство создаваемых заданий состоят только из одной задачи, SQL Server позволяет создавать задания, включающие несколько задач, которые можно настроить так, что в зависимости от успешности их завершения выполнялись различные операции. Каждая выполняемая задача или единица работы содержится в шаге задания.

Изучив материал этого занятия, вы сможете:

- ✓ создавать задания;
- ✓ создавать операторы;
- ✓ просматривать состояние и историю заданий.

Продолжительность занятия — 20 минут.

Шаги задания

Задания разбиваются на шаги. Существуют несколько типов шагов:

- Transact-SQL (T-SQL);
- задачи репликации;
- задачи операционной системы или исполняемые файлы;
- задачи служб Analysis Services;
- пакеты служб Integration Services;
- сценарии ActiveX.

Как и любой исполняемый код, каждый шаг задания выполняется в определенном контексте безопасности. По умолчанию контекст безопасности шага соответствует учетной записи владельца задания. Но его можно переопределить, указав учетную запись-посредник, которую агент SQL Server будет использовать, применяя учетные данные этой учетной записи.

Кроме выполняемых команд, на каждом шаге задания может настраиваться:

- ведение журнала;
- оповещение оператора;
- параметры, определяющие разрешенное количество повторных попыток, а также интервал между ними в минутах;
- логика потока управления.

Параметры потока управления позволяют определять действия в зависимости от успешности выполнения шага:

- завершить задание с успехом;
- завершить задание с ошибкой;
- перейти к следующему шагу;
- перейти к шагу с определенным номером.

Поток информации в журнал может быть перенаправлен в файл, который либо будет перезаписываться при каждом выполнении шага, либо информация будет добавляться в него. Также журналирование шага может выполняться в таблицу, хотя обычно это не рекомендуется из-за дополнительной нагрузки, создаваемой при записи в таблицу, в отличие от записи в текстовый файл, которая такой нагрузки не создает.

ПРИМЕЧАНИЕ Журналирование шагов задания

Для каждого создаваемого шага задания должна быть настроена запись информации в файл. Как правило, ведение журнала настраивают так, чтобы на первом шаге задания создавался новый файл журнала, а все последующие шаги добавляли в него информацию.

Расписания заданий

После добавления в задание одного или нескольких шагов, можно задать его расписание. Расписания создаются и хранятся как независимые объекты, благодаря чему можно определить одно расписание для нескольких заданий.

Расписание задания можно создавать либо средствами диалогового окна «Управление расписаниями» (Manage Schedules), либо при создании задания. Вот некоторые из параметров расписания:

- частота, например: ежедневно, еженедельно или ежемесячно;
- повторение в рамках ежедневного, еженедельного или ежемесячного цикла, например каждый третий день каждого второго месяца при ежемесячной частоте;
- повторение в течение дня с периодом, определяемым минутами или часами;
- время начала и завершения;
- даты начала и конца расписания.

К примеру, можно создать расписание, предусматривающее выполнение в первый понедельник каждого третьего месяца, каждые 15 минут между 3:00 и 19:00.

Журнал заданий

Все ошибки или сообщения, возникающие во время выполнения задания, заносятся в файл журнала или в таблицу, которая задается для каждого шага задания. Это позволяет в случае ошибки узнать о ней подробнее, просмотрев файл журнала.

В дополнение к журналам отдельных шагов задания SQL Server заносит информацию о выполнении каждого шага задания в таблицу `dbo.sysjobhistory` базы данных `msdb`. Вот некоторые из записываемых в эту таблицу параметров:

- шаг задания;
- состояние;
- дата и время запуска;
- продолжительность;
- в случае возникновения ошибок номер, серьезность и текст последней сгенерированной ошибки.

ПРИМЕЧАНИЕ На экзамене

Следует знать, где искать информацию в случае ошибок при выполнении шагов заданий.

Операторы

Оператор (operator) — это псевдоним пользователя, группы или устройства. Операторы используются для отправки предупреждений и уведомлений при ошибках в заданиях. Для каждого оператора указывается имя и контактная информация, такая как адрес электронной почты, номер пейджера или адрес для команды *NET SEND*. Кроме этого, можно определить какие операторы будут доступны в определенные дни, а также указать их рабочее время.

ПРИМЕЧАНИЕ Стандартная рабочая неделя

Рабочее время, а точнее начало и конец рабочего дня, определяется на основании стандартной рабочей недели США, с понедельника по пятницу. Его нельзя изменить или приспособить под другие определения рабочей недели.

Контрольные вопросы

1. Если задание завершилось с ошибкой, где стоит искать информацию о ней?
2. Шаги задания какого типа являются исполняемыми?

Ответы на контрольные вопросы

1. В первую очередь следует просмотреть журнал заданий, который можно открыть в Среде SQL Server Management Studio (SQL Server Management Studio), щелкнув правой кнопкой задание и выбрав **Просмотр журнала** (View History). Также можно проверить файлы журналов, заданные для отдельных шагов задания. Иногда дополнительная информация содержится в журналах событий Windows.
2. Можно создать задания, исполняющие T-SQL, сценарии ActiveX, команды операционной системы или исполняемые файлы. Также можно настроить специальные задачи для репликации, служб Analysis Services и Integration Services.

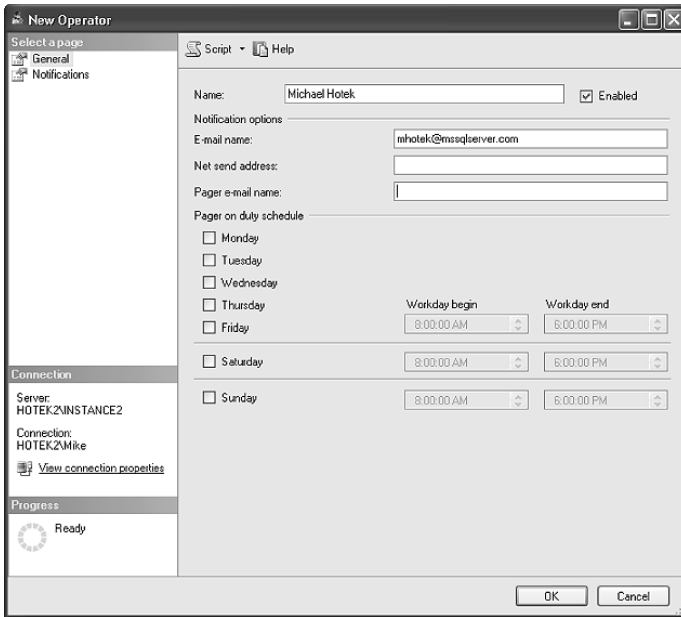
Практикум. Создание заданий и операторов

Вы определите оператор для отправки уведомлений SQL Server. Также вы создадите задание для переиндексирования базы данных *Adventure Works*.

Задание 1. Создание оператора

Вы создадите оператор, который впоследствии будет использоваться для отправки уведомлений заданиями и оповещениями.

1. В среде SQL Server Management Studio откройте узел **Агент SQL Server** (SQL Server Agent), щелкните правой кнопкой узел **Операторы** (Operators) и выберите **Создать оператора** (New Operator).
2. Введите имя оператора и адрес его электронной почты, как показано на рисунке.



3. Щелкните **ОК** и просмотрите только что созданный оператор.

Задание 2. Создание задания

Вы создадите задание для переиндексирования базы данных *AdventureWorks*.

1. Чтобы создать хранимую процедуру для переиндексирования таблиц, выполните в базе данных *AdventureWorks* следующий код:

```
CREATE PROCEDURE dbo.asp_reindex @database SYSNAME, @fragpercent INT
AS
DECLARE @cmd NVARCHAR(max),
        @table SYSNAME,
        @schema SYSNAME
```

--Используем курсор лишь в целях иллюстрации.

--То же самое можно было бы сделать при помощи табличной переменной и цикла WHILE.

```
DECLARE curtable CURSOR FOR
    SELECT DISTINCT OBJECT_SCHEMA_NAME(object_id, database_id) SchemaName,
        OBJECT_NAME(object_id,database_id) TableName
    FROM sys.dm_db_index_physical_stats (DB_ID(@database),NULL,NULL,NULL,'SAMPLED')
    WHERE avg_fragmentation_in_percent >= @fragpercent
FOR READ ONLY

OPEN curtable
FETCH curtable INTO @schema, @table

WHILE @@FETCH_STATUS = 0
BEGIN
    SET @cmd = 'ALTER INDEX ALL ON ' + @database + '.' + @schema + '.' + @table
        + ' REBUILD WITH (ONLINE = ON)'
```

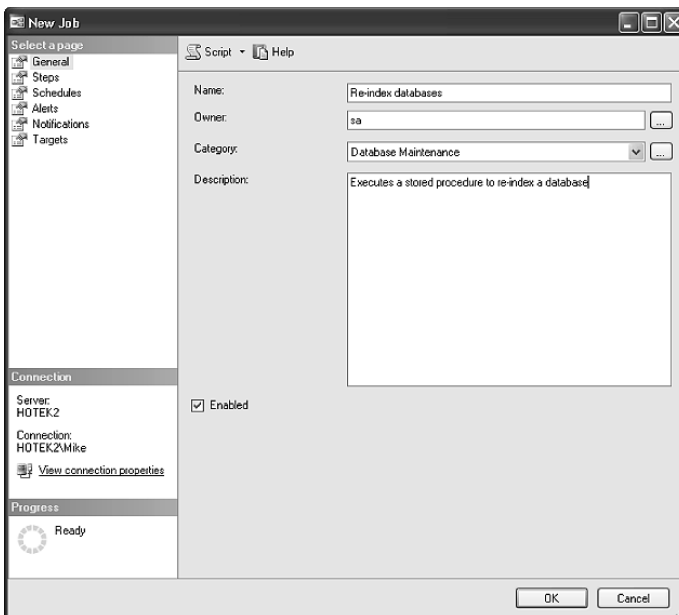


```
--Сначала пытаемся выполнить оперативное (ONLINE) перестроение, если
--это не удастся, то задаем перестроение в автономном режиме (OFFLINE).
BEGIN TRY
    EXEC sp_executesql @cmd
END TRY
BEGIN CATCH
    BEGIN
        SET @cmd = 'ALTER INDEX ALL ON ' + @database + '.' + @schema + '.'
            + @table + ' REBUILD WITH (ONLINE = OFF)'

        EXEC sp_executesql @cmd
    END
END CATCH
FETCH curtable INTO @schema, @table
END

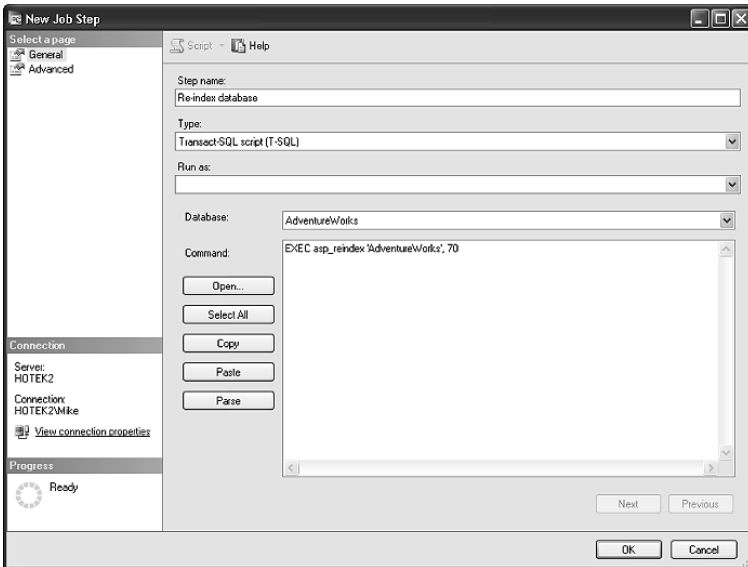
CLOSE curtable
DEALLOCATE curtable
GO
```

- В узле **Агент SQL Server** щелкните правой кнопкой узел **Задания (Jobs)** и выберите **Создать задание (New Job)**.
- Введите имя задания, укажите **sa** в качестве владельца, выберите категорию задания **Обслуживание базы данных (Database Maintenance)** и добавьте описание, как показано на рисунке.

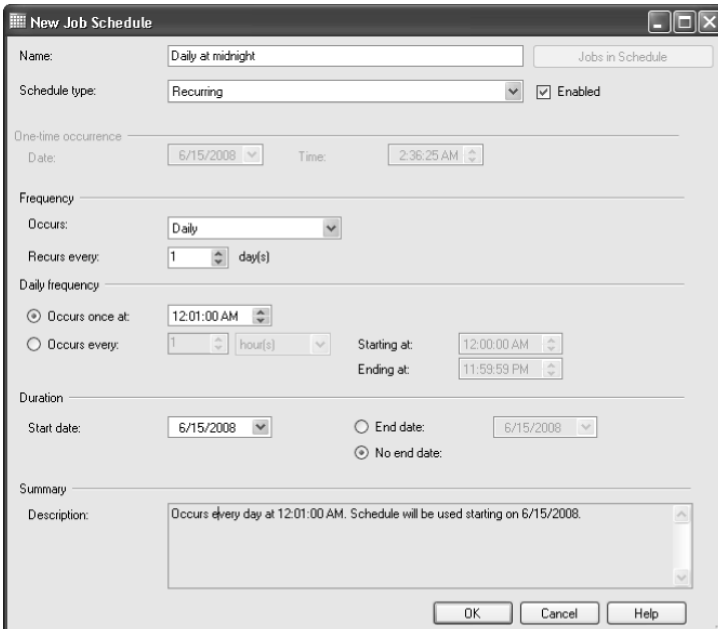


- Перейдите на страницу **Шаги (Steps)** и щелкните кнопку **Создать (New)**. Откроется диалоговое окно **Создание шага задания (New Job Step)**.

5. Введите имя шага, выберите тип шага — **Сценарий Transact-SQL (T-SQL)** [Transact-SQL script (T-SQL)], оставьте пустым поле **Выполнять как** (Run As), укажите имя базы данных **AdventureWorks** и введите показанную на рисунке команду SQL для процедуры переиндексирования, созданной на предыдущем задании.



6. Перейдите на страницу **Дополнительно** (Advanced) и укажите файл для сохранения журнала **C:\Test\Dailymaintenance.txt** и щелкните **OK**.



7. Перейдите на страницу **Расписания** (Schedules) и выберите **Создать** (New), чтобы задать новое ежедневное расписание, как показано на рисунке. Затем щелчком **ОК** закройте диалоговое окно **New Job Schedule** (Создание расписания задания).
8. Щелкните **ОК**, чтобы сохранить новое задание и закрыть диалоговое окно **Создание задания**.
9. Разверните задания, щелкните правой кнопкой задание **Re-index databases** и выберите **Start** (Запустить). После завершения задания изучите историю выполнения задания и файл журнала.

Резюме занятия

- Оператор является псевдонимом пользователя, группы или устройства, которым нужно направлять уведомления.
- В задании можно создать несколько шагов с управлением ходом выполнения, ведением журнала и одним или несколькими расписаниями.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Где следует искать список заданий, завершившихся с ошибкой?
 - A. Журнал событий Windows.
 - B. Журнал заданий в среде SQL Server Management Studio.
 - C. Журнал ошибок агента SQL Server.
 - D. Журнал ошибок SQL Server.

Занятие 2. Создание предупреждений

Предупреждения (alerts) предоставляют возможность отправлять сообщения или выполнять действия на основе событий или условий, возникших либо в экземпляре SQL Server, либо на компьютере, где он работает.

Изучив материал этого занятия, вы сможете:

- ✓ создавать предупреждения.

Продолжительность занятия — 20 минут.

Предупреждения агента SQL Server

Существует три типа предупреждений:

- событие SQL Server;
- предупреждение о наступлении условия производительности;
- событие инструментария управления Windows (WMI).

Предупреждения инициируются событиями SQL Server в зависимости от номера ошибки или уровня серьезности ошибки. Кроме этого можно ограничить

предупреждение рамками определенной базы данных или строкой текста в сообщении об ошибке. При создании предупреждения о событии SQL Server агент SQL Server просматривает журнал событий приложений Windows и ищет совпадения с заданными критериями события. Например, можно инициировать предупреждение об ошибке с уровнем серьезности 22, чтобы уведомить оператора о «подозрительной» таблице.

Предупреждения о наступлении условия производительности определяются на основе счетчиков системного монитора. При определении такого предупреждения указывается объект, счетчик и экземпляр, за которым следует наблюдать, а также условие для значения счетчика и должно ли предупреждение отсылаться, когда счетчик больше, меньше или равен заданному значению. Например, можно создавать предупреждение, когда свободного дискового пространства остается меньше 15%.

Предупреждение о событии WMI позволяет отправлять уведомления о событиях, произошедших на сервере, где работает экземпляр SQL Server. Каждый раз при возникновении на компьютере определенного события (например, отключение сетевого кабеля, создание файла, удаление файла или запись в реестр) в Windows генерируется событие WMI. При создании предупреждения WMI устанавливается слушатель (listener) инфраструктуры WMI, который будет инициировать уведомления при возникновении Windows-событий.

В предупреждении можно задать ответ, причем возможны следующие варианты:

- выполнить задание;
- сообщить оператору.

Настроив выполнение задания при возникновении предупреждения, можно заставить среду автоматически перехватывать и исправлять ошибки, избавив тем самым администратора от необходимости отвлекаться на рутинные события.

ПРИМЕЧАНИЕ На экзамене

Следует знать типы предупреждений, которые можно создавать в SQL Server, а также критерии ответов, которые можно для них определить.

Контрольные вопросы

1. Какие три типа предупреждений можно создавать в SQL Server?
2. Какие два действия ответа можно определить для предупреждения?

Ответы на контрольные вопросы

1. Можно создавать предупреждения на основе счетчиков производительности, ошибок SQL Server и запросов WMI.
2. В ответ на выполнение условия предупреждения можно отправить уведомление или выполнить задание.

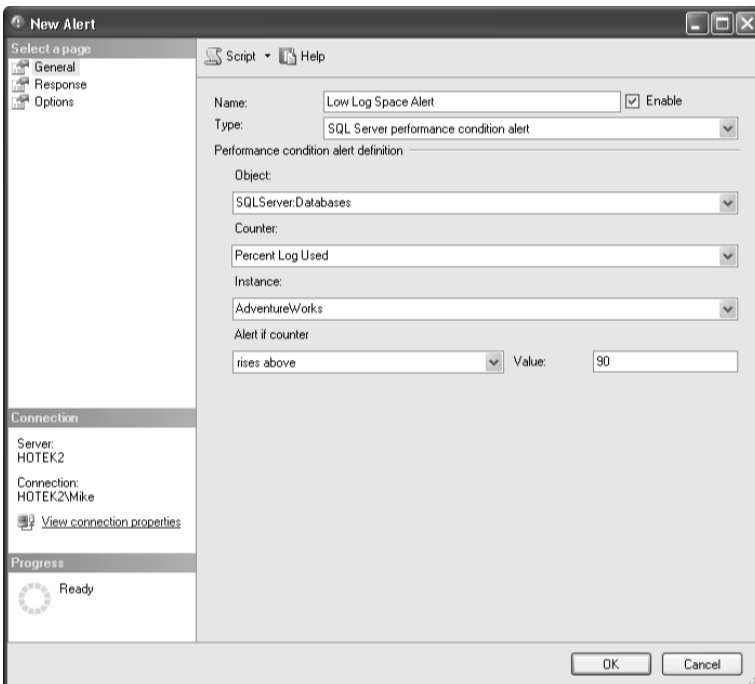
Практикум. Создание предупреждений

На этом упражнении вы создадите предупреждения для отправки уведомлений при нехватке места в журнале транзакций и при возникновении ошибок уровня 22.

Задание 1. Создание предупреждения о выполнении условия производительности

Вы создадите предупреждение, отправляющее уведомление, когда размер использованного пространства журнала транзакций базы данных *AdventureWorks* превысит 90%.

1. В среде SQL Server Management в узле **Агент SQL Server** (SQL Server Agent) щелкните правой кнопкой **Предупреждения** (Alerts) и выберите **Создать предупреждение** (New Alert).
2. Задайте имя нового предупреждения и в поле со списком **Тип** (Type) выберите **Предупреждение о производительности SQL Server** (SQL Server Performance Condition Alert).
3. В поле со списком **Объект** (Object) выберите **SQLServer:Databases**, в поле со списком **Счетчик** (Counter) — **Percent Log Used** и в поле со списком **Экземпляр** (Instance) — **AdventureWorks**. Задайте создание предупреждения, когда счетчик превысит 90, для чего в поле со списком **Создать предупреждение, если счетчик** (Alert If Counter) выберите **Больше** (Rises Above) и в поле **Значение** (Value) введите **90** (см. рисунок).

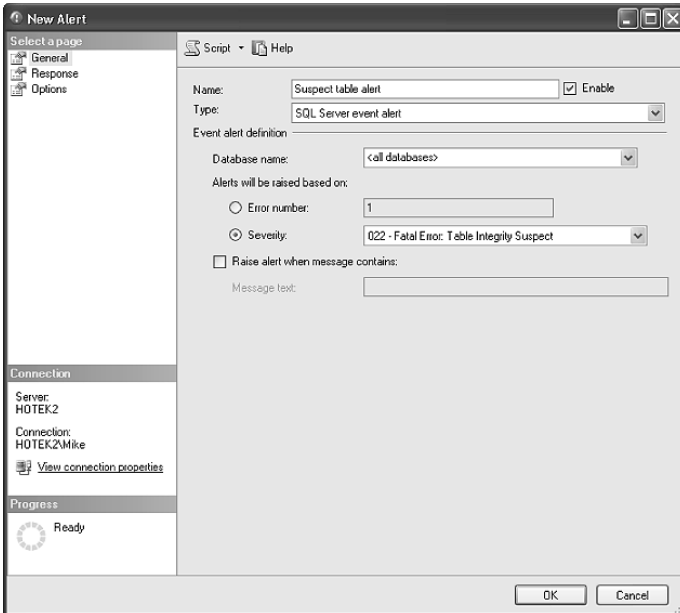


4. Перейдите на страницу **Ответ** (Response). Установите флажок **Уведомить операторов** (Notify Operators) и флажки параметров уведомления операторов.
5. Перейдите на страницу **Параметры** (Options), установите флажок **По электронной почте** (E-mail) и введите текст сообщения об ошибке. Щелкните **ОК**.

Задание 2. Создание предупреждения о событии SQL Server

Вы создадите предупреждение о событии SQL Server.

1. Щелкните правой кнопкой узел **Предупреждения** (Alerts) и выберите **Создать предупреждение** (New Alert).
2. Задайте имя нового предупреждения в поле **Имя** (Name), а в поле со списком **Тип** (Type) выберите **Предупреждение о событии сервера** (Server Event Alert).
3. Выберите **Все базы данных** (All Database) и серьезность ошибки **22**, как показано на рисунке.



4. Перейдите на страницу **Ответ** (Response) и установите флажок **Уведомить операторов** (Notify Operators) и настройте параметры уведомления операторов.
5. Перейдите на страницу **Параметры** (Options) и установите флажок **По электронной почте** (E-mail) и введите текст сообщения об ошибке. Щелкните **ОК**.

Резюме занятия

- Предупреждения позволяют при возникновении событий уведомлять операторов и выполнять задания, устраняющие неполадки.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Ночью произошел сбой в базе данных *Orders* и вы определили, что причина — переполнение файла данных. Какой инструмент следует использовать, чтобы отправлять администратору уведомление и увеличить файл до того, как он переполнится?
 - A. Агент SQL Server (SQL Server Agent).
 - B. Системный монитор (System Monitor).
 - C. Просмотр событий (Event Viewer).
 - D. Сетевой монитор (Network Monitor).

Закрепление материала главы

Для закрепления знаний и навыков, полученных при изучении этой главы, выполните следующее:

- изучите резюме главы;
- повторите основные термины, использованные в этой главе;
- выполните задания по темам этой главы;
- сделайте предложенные упражнения;
- пройдите пробный экзамен.

Резюме главы

- Агент SQL Server предоставляет механизм планирования, который позволяет выполнять задания.
- Каждое задание состоит из одного или нескольких шагов с простым управлением потоком выполнения, журналированием, уведомлениями и одним или несколькими расписаниями выполнения.
- Операторы инкапсулируют параметры отправки уведомлений пользователям, группам или устройствам.

Основные термины

Понятны ли вам значения приведенных здесь основных терминов? Чтобы проверить свои ответы, загляните в глоссарий на прилагаемом компакт-диске.

- Предупреждение ~ alert.
- Шаг задания ~ job step.
- Оператор ~ operator.

Упражнения

Сейчас вы примените на практике знания и навыки, полученные при изучении этой главы. Ответы на вопросы см. в разделе «Ответы» в конце книги.

Упражнение. Разработка стратегии автоматизации для компании Coho Vineyard

Исходные данные

Основные сведения о компании

Компания Coho Vineyard была основана в 1947 г. как частный семейный винный завод. Вина, производимые Coho Vineyard, удостоивались многих наград и пользовались популярностью, что позволило компании значительно расширить производство. Через несколько лет в рамках стратегии по расширению компании были приобретены несколько существующих винных заводов. Сейчас компания владеет 16 винными заводами — 9 в штатах Вашингтон, Орегон и Калифорния, а остальные 7 в штатах Висконсин и Мичиган. В компании работает 532 человека, из которых 162 — в центральном офисе, где размещаются критически важные для бизнеса серверы. В компании трудится 122 мобильных торговых агента, которым требуется доступ к актуальной информации об имеющихся товарах.

Планируемые изменения

До сих пор у каждого из 16 винных заводов Coho Vineyard был собственный веб-узел, размещенный на площадях завода. Руководство требует объединить сайты всех заводов, так чтобы пользователи смогли покупать продукцию всех 16 предприятий в одном интернет-магазине. Вся информация веб-узла должна храниться в базе данных центрального офиса.

После объединения всей информации в центральном офисе для доставки информации торговым агентам, а также для получения от них заказов, будут использоваться репликация слиянием. Чтобы до завершения проекта по объединению удовлетворить требованиям торговых агентов, информация о наличии товаров на каждом из заводов отправляется в центральный офис ежедневно в конце дня.

Существующая среда

Базы данных

Сейчас каждый винный завод поддерживает собственную базу данных, где хранит всю бизнес-информацию. В конце месяца вся информация переносится в центральный офис и загружается в базы данных, перечисленные в табл. 10-1.

Табл. 10-1. Базы данных Coho Vineyard

База данных	Размер
<i>Customer</i> (клиенты)	180 Мб
<i>Accounting</i> (бухгалтерия)	500 Мб
<i>HR</i> (управление персоналом)	100 Мб
<i>Inventory</i> (склад)	250 Мб
<i>Promotions</i> (маркетинг)	80 Мб

По завершении объединения баз данных хранилищем информации нового интернет-магазина будет служить новая база данных *Order*. Кроме того в своей ежедневной работе сотрудники будут обращаться к данным БД *Order* с применением нового разработанного в компании собственными силами веб-приложения.

База данных *HR* содержит конфиденциальную информацию и защищена TDE (Transparent Data Encryption). Кроме этого, данные в таблице *Salary* шифруются с использованием сертификата.

Серверы баз данных

Один сервер под управлением Windows Server 2003 Enterprise в центральном офисе по имени DB1 хранит все БД. На нем установлен SQL Server 2008 Enterprise.

Бизнес-требования

От вас требуется разработать решение для архивирования баз данных *Customer* и *Order*. Стратегия архивирования должна обеспечивать хранение информации о клиентах как минимум на протяжении 6 лет.

В процессе подготовки БД *Order* для архивирования вы создали секционированную таблицу *Order.Sales*, состоящую из двух секций: в первой хранится информация о продажах за текущий, а во второй — за предыдущий месяц. Заказы, размещенные еще раньше, должны перемещаться в другую секционированную таблицу — *Order.Archive*, после чего в секции 1 содержатся все архивные данные, а секция 2 остается пустой. Архивные данные должны находиться в файловой группе, отличной от файловой группы активно используемых данных.

Новый процесс должен ежедневно, не позднее 4 часов утра загружать информацию о складских запасах со всех 16 заводов.

Четыре крупных клиента осуществляют заказы, используя XML-схему *Coho Vineyards* для EDI-транзакций (Electronic Data Interchange). EDI-файлы поступают в 5 часов вечера и должны обрабатываться и загружаться в БД *Customer*, *Accounting* и *Inventory*, каждая из которых содержит таблицы с данными, относящимися к размещаемым заказам. В настоящее время утилита импорта EDI-файлов представляет собой однопоточное приложение на C++, которому требуется от 3 до 6 часов на обработку всех файлов. Чтобы выполнить соглашение с клиентами об условиях обслуживания (Service Level Agreement, SLA), необходимо заканчивать обработку EDI-файлов до 17:30. К моменту завершения проекта по объединению EDI-утилита должна загружать информацию в новую базу данных *Order*.

Требуется обеспечить создание резервных копий всех баз данных всех филиалов. Все производственные базы данных должны работать в режиме полной модели восстановления. В самом плохом случае считается приемлемым потеря данных максимум за последние 5 минут. В случае аварии БД *Customer*, *Inventory*, *Promotions* и *Order* могут быть недоступны максимум 20 минут. В случае аварийного сбоя информация БД *Customer* и *Order* старше полугода может быть недоступна максимум 12 часов.

Ответьте на следующий вопрос.

1. Что нужно настроить, чтобы обеспечить автоматизацию процессов администрирования?

Рекомендуемые упражнения

Чтобы подготовиться к экзамену по темам этой главы, выполните следующие задания.

Создание заданий

- **Задание.** В своей среде создайте задания создания полных, разностных резервных копий и резервных копий журнала транзакций.

Создание предупреждений

- **Задание.** Создайте предупреждение, отправляемое, когда на жестком диске, хранящем файлы данных, остается меньше 15% свободного пространства.

Пробный экзамен

На прилагаемом компакт-диске содержится пробный экзамен, обеспечивающий широкие возможности самоподготовки. Например, можно проверить, как усвоены темы этой главы либо все темы экзамена 70-432. Можно симитировать настоящий экзамен или выбрать режим обучения, когда после каждого вопроса приводятся правильные ответы с пояснениями.

ПРИМЕЧАНИЕ Пробный экзамен

Подробнее о пробном экзамене см. «Введение».

Проектирование безопасности SQL Server

Занятие 1. Конечные точки TCP	229
Занятие 2. Настройка контактной зоны SQL Server	235
Занятие 3. Создание участников	238
Занятие 4. Управление разрешениями	246
Занятие 5. Аудит экземпляров SQL Server	259
Занятие 6. Шифрование данных	265

Проектирование надежной системы безопасности требует реализации многоуровневого подхода, своего рода «глубоко эшелонированной обороны». В этой главе рассказывается, как сконфигурировать каждый из уровней инфраструктуры безопасности Microsoft SQL Server, чтобы защитить данные от несанкционированного доступа.

Темы экзамена

- Управление именами входа и ролями сервера.
- Управление пользователями и ролями базы данных.
- Управление разрешениями экземпляра SQL Server.
- Управление разрешениями базы данных.
- Управление разрешениями схемы и объектов.
- Аудит экземпляров SQL Server.
- Управление «прозрачным» шифрованием данных.
- Настройка контактной зоны.

Прежде всего

Для изучения материалов этой главы необходимо иметь:

- установленный экземпляр Microsoft SQL Server 2008;
- подключенную базу данных AdventureWorks.

Занятие 1. Конечные точки TCP

Конечные точки управляют возможностью подключения к экземпляру SQL Server и определяют допустимые способы коммуникации. Подобно брандмауэрам в сети, конечные точки обеспечивают безопасность на границе между приложениями и экземпляром SQL Server. В этом занятии приведены общие сведения об архитектуре конечных точек в SQL Server 2008.

Изучив материал этого занятия, вы сможете:

- ✓ объяснить роль конечных точек при обеспечении безопасности экземпляров SQL Server 2008.

Продолжительность занятия — 15 минут.

Типы конечных точек и полезные данные

У конечной точки есть два важных компонента: протокол передачи данных (*TCP* или *HTTP*) и полезные данные, определяющие поддерживаемый тип трафика. Существуют следующие типы полезных данных: *SOAP*, *TSQL*, *SERVICE_BROKER* и *DATABASE_MIRRORING*.

В табл. 11-1 перечислены допустимые сочетания протоколов передачи данных и типов полезной нагрузки для конечных точек.

Табл. 11-1. Протоколы передачи данных и типы полезной нагрузки для конечных точек

Протокол передачи данных	Тип полезной нагрузки
TCP	TSQL
TCP	SERVICE_BROKER
TCP	DATABASE_MIRRORING
HTTP	SOAP

С помощью различных комбинаций протоколов передачи данных и типов полезной нагрузки выполняется предварительная фильтрация допустимого трафика еще до поступления команды на экземпляр SQL Server. Допустим, для конечной точки задан протокол *TCP* и тип полезной нагрузки *TSQL*. В этом случае при попытке передачи через данную конечную точку трафика типа *HTTP*, *SERVICE_BROKER* или *DATABASE_MIRRORING* соединение будет отклонено даже без проверки подлинности запроса.

Этот процесс во многом подобен работе брандмауэров в сети. Администраторы сети настраивают брандмауэры так, чтобы разрешить передачу трафика только через заданные UDP- и TCP-порты. Любые попытки передачи запросов через заблокированные порты брандмауэр отклоняет. Точно так же запросы, формат которых не соответствует определению конечной точки, ею отклоняются.

Доступ к конечным точкам

Даже если в поступающем на конечную точку трафике используются «правильные» протокол передачи данных и тип полезной нагрузки, соединение все

равно будет отклонено, если не будет разрешений на доступ к данной конечной точке. Доступ к конечной точке состоит из двух уровней.

На первом уровне управление доступом определяется состоянием конечной точки. Конечная точка может находиться в одном из трех состояний: *STARTED* (запущено), *STOPPED* (остановлено) и *DISABLED* (отключено). В зависимости от состояния конечная точка действует по-разному:

- *STARTED* — конечная точка активно прослушивает соединения и отвечает на запросы приложений;
- *STOPPED* — конечная точка активно прослушивает соединения, но возвращает в приложения ошибку подключения;
- *DISABLED* — конечная точка не прослушивает соединения и не отвечает на какие-либо попытки подключения.

Второй уровень управления доступом — разрешение на подключение к конечной точке. Чтобы подключиться к конечной точке, у используемого приложением имени входа должно быть разрешение *CONNECT* на подключение к этой конечной точке.

Вы спросите: зачем так много усилий нужно приложить, чтобы создать соединение с экземпляром SQL Server еще до того, как начнется проверка подлинности пользователя. В предыдущих версиях SQL Server любое приложение могло подключиться и передать любой запрос на сервер SQL Server. Никто не проверял корректность запросов приложений, поэтому злоумышленникам было проще атаковать серверы SQL Server. В SQL Server 2008 до постановки запроса в очередь на обработку выполняется проверка допустимости каждого запроса и отправившего его пользователя. При попытке компрометации сервера SQL Server администраторы также могут немедленно закрыть доступ путем перевода соответствующей конечной точки в состояние *DISABLED*.

Конечные точки TCP

У конечных точек *TCP* возможен один из трех типов полезной нагрузки: *TSQL*, *DATABASE_MIRRORING* и *SERVICE_BROKER*.

Аргументы конечных точек TCP

Конечные точки *TCP* можно настроить на прослушивание трафика по определенным IP-адресам и портам. Для любых конечных точек *TCP* можно задать следующие два аргумента:

- *LISTENER_PORT*;
- *LISTENER_IP*.

Аргумент *LISTENER_PORT* обязательный. Конечная точка *TCP* для полезной нагрузки *TSQL*, создаваемая для каждого экземпляра в ходе установки, настроена по умолчанию на TCP-порт 1433 или другой порт для данного экземпляра.

ПРИМЕЧАНИЕ Номера портов

Поскольку в конечных точках *DATABASE_MIRRORING* по умолчанию используется TCP-порт 5022, а в конечных точках *TSQL* — TCP-порт 1433, имеет смысл задать в этих конечных точках другие номера портов. Это позволит предотвратить возможные атаки или хотя бы затруднит их, ведь при этом злоумышленникам придется воспользоваться сканером портов, а не просто вслепую подключиться к порту 1433 или 5022 для организации DoS-атаки («отказ в обслуживании») или другой попытки взлома.

Аргумент *LISTENER_IP* является необязательным и позволяет организовать очень мощный уровень безопасности в приложениях некоторых типов. В конечной точке можно задать IP-адрес, который будет прослушиваться. По умолчанию у данного аргумента значение *ALL*, то есть конечная точка прослушивает все соединения, поступающие на все действительные IP-адреса на данном сервере. Задав значение аргумента *LISTENER_IP*, можно ограничить запросы на соединение отдельным сетевым интерфейсом. При этом конечная точка будет прослушивать запросы, отправляемые лишь на указанный IP-адрес.

ПРИМЕЧАНИЕ На экзамене

У конечных точек *TSQL* не предусмотрены дополнительные параметры конфигурации помимо общих параметров TCP.

Общие аргументы для конечных точек *DATABASE_MIRRORING* и *SERVICE_BROKER*

Для конечных точек *DATABASE_MIRRORING* и *SERVICE_BROKER* можно задать метод проверки подлинности и параметр шифрования. В качестве метода проверки подлинности можно использовать сертификаты или проверку подлинности Windows, выбрав один из параметров: *NTLM*, *KERBEROS* или *NEGOTIATE*. Если задан параметр *NEGOTIATE*, экземпляры выбирают метод проверки подлинности динамически. Для проверки подлинности на основе сертификатов можно использовать сертификаты, выпущенные доверенным центром сертификации, или создать собственные сертификаты Windows.

ПРИМЕЧАНИЕ Проверка подлинности

Когда все экземпляры зеркального отображения базы данных и компонента Service Broker находятся в одном домене или в доверенных доменах, следует выбирать проверку подлинности Windows. В противном случае используйте проверку подлинности на основе сертификатов.

SQL Server позволяет шифровать весь обмен данными между конечными точками, причем алгоритм шифрования задается отдельно. По умолчанию используется алгоритм RC4, но можно выбрать и гораздо более надежный AES.

ПРИМЕЧАНИЕ Шифрование

Алгоритм RC4 обеспечивает минимальную надежность шифрования и наилучшую производительность. Если требуется надежный алгоритм шифрования, выбирайте AES, но учтите, что при этом возрастет вычислительная нагрузка и снизится производительность.

Аргументы для конечных точек DATABASE_MIRRORING

Для конечных точек *DATABASE_MIRRORING* предусмотрен третий аргумент, определяющий их роль в сеансе зеркального отображения базы данных.

ПРИМЕЧАНИЕ На экзамене

Для каждого экземпляра SQL Server можно задать лишь одну конечную точку *TCP* с типом полезной нагрузки *DATABASE_MIRRORING*.

Для конечной точки можно задать роль *PARTNER*, *WITNESS* или *ALL*. Конечная точка с ролью *PARTNER*, может участвовать в сеансе зеркального отображения только как основная или зеркальная база данных. Конечная точка с ролью *WITNESS*, может выступать только как сервер-свидетель. Если задано значение *ALL*, конечная точка может выполнять любую роль.

ПРИМЕЧАНИЕ Конечные точки в выпуске Express Edition

Конечные точки *DATABASE_MIRRORING* в SQL Server 2008 выпуска Express Edition поддерживают лишь роль *WITNESS*.

Следующий код на языке T-SQL создает конечную точку *Database_MIRRORING*:

```
CREATE ENDPOINT [Mirroring]
AS TCP (LISTENER_PORT = 5022)
FOR DATA_MIRRORING (ROLE = PARTNER, ENCRYPTION = REQUIRED);
ALTER ENDPOINT [Mirroring] STATE = STARTED;
```

При выполнении этого кода создается конечная точка, обслуживающая сеансы зеркального отображения базы данных через порт 5022 и реагирующая на запросы с любых действительных IP-адресов. Параметр *ROLE = PARTNER* означает, что в сеансах зеркального отображения, обслуживаемых данной конечной точкой, могут принимать участие лишь базы данных на текущем экземпляре SQL Server с ролью основной или зеркальной базы данных и при использовании алгоритма шифрования RC4.

Аргументы для конечных точек SERVICE_BROKER

Помимо режимов проверки подлинности и шифрования для конечных точек *SERVICE_BROKER* предусмотрены аргументы, связанные с пересылкой сообщений.

Параметр *MESSAGE_FORWARDING* позволяет настроить пересылку сообщений, предназначенных для другого экземпляра компонента Service Broker, на другой адрес. Этот параметр принимает два значения: *ENABLED* (включен) и *DISABLED* (отключен). Если параметру *MESSAGE_FORWARDING* присвоено значение *ENABLED*, можно также указать значение параметра *MESSAGE_FORWARDING_SIZE*, определяющего максимальный размер хранилища для пересылаемых сообщений.

Подробное обсуждение компонента Service Broker выходит за рамки данной книги, но мы вкратце объясним принципы его работы. Экземпляры компонента Service Broker обрабатывают сообщения асинхронно, с помощью хранимых процедур. Каждый экземпляр Service Broker настроен на обработку сообщений определенного формата. Вместе с тем в одной среде можно настроить много

экземпляров компонента Service Broker, обрабатывающих разные типы сообщений. Пересылка сообщений позволяет администраторам организовать балансировку нагрузки между различными экземплярами Service Broker без необходимости внесения изменений в приложения.

ПРИМЕЧАНИЕ Шифрование

Шифрование обмена данными в конечных точках реализовано так, что позволяет определить источник и пункт назначения трафика. Если обмен данными не выходит за пределы экземпляра SQL Server, трафик не шифруется, чтобы избежать ненужных издержек. Это особенно важно для компонента Service Broker, когда на одном экземпляре происходит обмен множеством сообщений между очередями. Трафик шифруется лишь в том случае, если он передается за пределы экземпляра SQL Server.

Контрольные вопросы

1. Каковы два компонента конечной точки?
2. Назовите три состояния конечной точки и опишите, чем они различаются.
3. Какое разрешение необходимо предоставить, чтобы конечная точка принимала запросы на подключение?
4. Какие типы проверки подлинности доступны для конечных точек *Service_Broker* и *Database_Mirroring*?
5. Назовите два общих (универсальных) аргумента конечных точек *TCP*.

Ответы на контрольные вопросы

1. В конечной точке определяется протокол передачи данных (*TCP* или *HTTP*) и тип полезной нагрузки (*TSQL*, *SERVICE_BROKER*, *DATABASE_MIRRORING* или *SOAP*).
2. Три состояния конечной точки: *STARTED*, *STOPPED* и *DISABLED*. Конечная точка в состоянии *STARTED* прослушивает и поддерживает подключения, в состоянии *STOPPED* — прослушивает запросы на соединение и возвращает сообщение об ошибке, а в состоянии *DISABLED* — не реагирует ни на какие запросы.
3. Необходимым условием принятия запроса на подключение является наличие у используемого имени входа разрешения *CONNECT* на данную конечную точку.
4. Можно задать проверку подлинности *NTLM* или *Kerberos*. Также можно указать параметр *NEGOTIATE*, который позволяет выбирать метод проверки подлинности путем согласования между приложением и конечной точкой.
5. Для конечной точки необходимо указать порт для прослушивания подключений. При желании можно также указать IP-адрес, чтобы конечная точка реагировала лишь на трафик, поступающий с этого IP-адреса.

Практикум. Сбор данных об имеющихся конечных точках

Вы создадите запросы нескольких динамических административных представлений, чтобы получить информацию о конечных точках, настроенных в текущей среде.

1. Откройте Среду SQL Server Management Studio и подключитесь к своему экземпляру. Откройте новое окно запроса и выполните следующий код:

```
SELECT * FROM sys.endpoints
SELECT * FROM sys.tcp_endpoints
SELECT * FROM sys.http_endpoints
SELECT * FROM sys.database_mirroring_endpoints
SELECT * FROM sys.service_broker_endpoints
```

2. В результатах найдите данные, полученные от каждого динамического административного представления.

Резюме занятия

- Конечные точки в SQL Server по функциональности во многом схожи с брандмауэрами и выполняют фильтрацию трафика в соответствии с заданными критериями.
- В каждой конечной точке задается протокол передачи данных: *TCP* или *HTTP*.
- Другая характеристика конечных точек — полезная нагрузка: *TSQL*, *DATABASE_MIRRORING*, *SERVICE_BROKER* или *SOAP*.
- В процессе установки конечные точки *TSQL* настраиваются на прослушивание подключений через заданный порт.
- Для конечных точек *Service_Broker* и *Database_Mirroring* можно задать метод проверки подлинности, а также выбрать алгоритм шифрования всего трафика.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Вы администратор баз данных в компании A. Datum Corporation. Пользователи жалуются, что приложения не подключаются к SQL Server. Вы проверили правильность всех параметров конфигурации приложений, а также возможность подключения к серверу SQL Server со своего компьютера средствами Среды SQL Server Management Studio, но приложения пользователей по-прежнему возвращают сообщение об отказе в доступе. В чем может быть дело?
 - A. Конечная точка *TCP* с типом полезной нагрузки *TSQL* находится в состоянии *DISABLED*.
 - B. Конечная точка *TCP* с типом полезной нагрузки *TSQL* находится в состоянии *STOPPED*.
 - C. Не включены удаленные подключения.
 - D. У пользователей нет разрешений *CONNECT* для данной конечной точки.

2. Вы настроили в среде сеанс зеркального отображения базы данных. Вы успешно создали конечные точки для основной и зеркальной баз данных, присвоили параметру *ROLE* значение *PARTNER* и затем их запустили. Вы проверили возможность подключиться к каждой конечной точке и пройти проверку подлинности. Но конфигурирование зеркального отображения базы данных завершается ошибкой. В чем может быть дело?
- A. В качестве режима проверки подлинности указан метод *NTLM*.
 - B. В качестве режима проверки подлинности указан метод *NEGOTIATE*.
 - C. В конечных точках по-разному настроено шифрование.
 - D. Во всех конечных точках задан алгоритм шифрования AES.

Занятие 2. Настройка контактной зоны SQL Server

Если иметь достаточно времени, можно взломать любую систему безопасности. Цель защиты состоит в том, чтобы создать столько преград на пути злоумышленников, чтобы затраченные усилия не оправдывали результат. В этом занятии вы научитесь настраивать экземпляры, сводя к минимуму набор включенных функциональных возможностей в SQL Server, что позволяет минимизировать число слабых мест системы.

Изучив материал этого занятия, вы сможете:

- ✓ включать и отключать функциональные возможности SQL Server.

Продолжительность занятия — 15 минут.

Настройка контактной зоны

Чаще всего злоумышленники получают доступ к системе через функциональные возможности, которые активированы, но редко используются. Теперь в SQL Server отключаются все функции, которые не требуются для работы ядра СУБД.

Во время установки можно настроить проверку подлинности пользователей на каждом экземпляре с применением только учетных данных Windows. Если в экземпляре задан режим проверки подлинности Windows, пользователи лишаются возможности применения имен входа SQL Server.

В течение многих лет при работе с SQL Server не возникало никаких проблем с вирусами, поскольку вирусы для SQL Server просто никто не писал. Несколько лет назад была совершена первая атака на SQL Server. Программаторы Slammer всего за несколько часов нанесли серьезный ущерб многим компаниям по всему миру. Разрушительная сила этого вируса объяснялась не столько его нацеленностью на SQL Server, сколько тем, что администраторам было невероятно сложно локализовать вирус из-за тысяч установленных в компаниях экземпляров SQL Server. На многих экземплярах SQL Server была установлена версия Microsoft Database Engine (MSDE), используемая различными приложениями в качестве локального хранилища данных. К сожалению, каждый экземпляр SQL Server не зависимо от версии поддерживал открытые соединения из любого источника, хотя фактически удаленные соединения требовались лишь для нескольких экземпляров.

Начиная с выпуска SQL Server 2005 появилась возможность настройки экземпляров на принятие или отклонение удаленных соединений путем конфигурирования сетевых протоколов удаленного доступа. По умолчанию в версиях, которым обычно не нужна поддержка удаленных соединений, например SQL Server Express, включена только поддержка поставщика сетевых услуг общей памяти. Чтобы разрешить удаленное подключение к экземплярам SQL Server, необходимо включить поддержку поставщика сетевых услуг TCP/IP.

Наибольший риск для экземпляров представляет использование функциональных возможностей с внешним интерфейсом или допускающих выполнение произвольного кода. Наибольшую угрозу представляют функции *OPENROWSET* и *OPENDATASOURCE* и процедуры OLE-автоматизации.

Для включения и отключения функциональных возможностей SQL Server служит процедура *sp_configure*. Необходимо отключить следующие функции, если они не используются:

- *Ad Hoc Distributed Queries* (нерегламентированные распределенные запросы);
- *CLR Enabled* (среда CLR включена);
- межбазовые цепочки владения (CDOC);
- компонент Database Mail;
- управление внешними ключами;
- уровень доступа файлового потока;
- процедуры OLE-автоматизации;
- удаленные административные подключения;
- расширенные хранимые процедуры службы SQL Mail;
- параметр *xp_cmdshell*.

Процедуры OLE-автоматизации в составе SQL Server обеспечивают минимальный уровень совместимости с предыдущими версиями. После включения поддержки общезыковой среды выполнения (CLR) в SQL Server 2005 все приложения, требующие OLE-автоматизации, должны быть переписаны как сборки Visual Basic .NET или C#.NET. Главное преимущество процедур CLR состоит в том, что они выполняются в защищенной памяти и не могут повредить стек памяти SQL Server, что возможно при OLE-автоматизации.

Расширенные хранимые процедуры службы SQL Mail обеспечивают обратную совместимость и не рекомендованы к использованию в версии SQL Server 2005. Необходимо отказаться от использования службы SQL Mail, а приложения, в которых задействована ее функциональность, следует переписать до выхода следующей версии SQL Server.

Функции *OPENROWSET* и *OPENDATASOURCE* делают сервер уязвимым для атак, поскольку позволяют встраивать учетные данные безопасности в код приложений, что делает возможным создание подключения SQL Server к другому экземпляру. Для выполнения запросов на нескольких экземплярах следует использовать связанные серверы, поддерживающие передачу учетных данных Windows между различными машинами.

С помощью межбазовых цепочек владения (CDOC) можно переносить полномочия на выполнение запросов от одной базы данных к другой. Если эта функциональность включена, владелец базы данных, в которой содержится запрашиваемых объект, может передать управление владельцу другой базы дан-

ных. В занятии 4 вы познакомитесь с подписями, которые позволяют усовершенствовать управление безопасностью и в то же время разрешают процедурам, функциям и триггерам доступ к объектам в нескольких базах данных.

ПРИМЕЧАНИЕ На экзамене

Предназначенное для настройки контактной зоны средство Surface Area Configuration Manager входит в состав SQL Server 2005, но отсутствует в SQL Server 2008. Однако его возможности по управлению соединениями теперь доступны в Диспетчере конфигурации SQL Server (SQL Server Configuration Manager), а управление функциональными возможностями осталось без изменений за исключением того, что был отменен графический пользовательский интерфейс для доступа к процедуре *sp_configure*.

Контрольные вопросы

1. Как следует настроить экземпляр, чтобы он поддерживал только локальные соединения?
2. С помощью каких средств можно включать или отключать функциональные возможности экземпляра?

Ответы на контрольные вопросы

1. Поставщик TCP/IP позволяет устанавливать удаленные соединения с экземпляром. Если его отключить, экземпляр будет поддерживать только локальные соединения.
2. Для включения и отключения функциональных возможностей используется системная хранимая процедура *sp_configure*.

Практикум. Настройка контактной зоны

Вы отключите несколько функциональных возможностей и проверите параметры конфигурации экземпляра.

1. Чтобы включить возможность просмотра всех параметров конфигурации экземпляра, выполните код:

```
EXEC sp_configure 'show advanced options',1
GO
RECONFIGURE WITH OVERRIDE
GO
EXEC sp_configure
GO
```

2. Чтобы отключить параметр *Ad Hoc Distributed Queries*, межбазовые цепочки владения, параметр *CLR Enabled*, процедуры OLE-автоматизации, расширенные хранимые процедуры службы SQL Mail и параметр *xp_cmdshell*, выполните код:

```
EXEC sp_configure 'Ad Hoc Distributed Queries',0
EXEC sp_configure 'clr enabled',0
EXEC sp_configure 'cross db ownership chaining',0
```

```
EXEC sp_configure 'Ole Automation Procedures',0
EXEC sp_configure 'SQL Mail XPs',0
EXEC sp_configure 'xp_cmdshell',0
GO
RECONFIGURE WITH OVERRIDE
GO
```

Резюме занятия

- Первое решение при настройке контактной зоны принимается на этапе установки, когда для доступа к экземпляру можно выбрать использование только учетных данных Windows.
- Для всех экземпляров, не требующих удаленных соединений, следует отключить поставщика TCP/IP.
- Для включения и отключения функциональных возможностей используется процедура *sp_configure*.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Какое средство применяется для включения и отключения функциональных возможностей SQL Server?
 - A. Диспетчер конфигурации SQL Server;
 - B. Процедура *sp_configure*;
 - C. Средство SQL Server Surface Area Configuration Manager;
 - D. Центр установки SQL Server.

Занятие 3. Создание участников

Участники (principals) предназначены для прохождения проверки подлинности и идентификации в экземпляре сервера или в базе данных. Участники подразделяются на две основные категории: имена входа/пользователи и группы как на уровне экземпляра, так и на уровне базы данных.

Изучив материал этого занятия, вы сможете:

- ✓ создавать имена входа;
- ✓ управлять составом роли сервера;
- ✓ создавать пользователей и роли базы данных;
- ✓ управлять составом роли базы данных;
- ✓ создавать пользователей, не имеющих имени входа.

Продолжительность занятия — 40 минут.

Имена входа

Для доступа к экземпляру пользователь должен пройти проверку подлинности, предоставив свои учетные данные серверу SQL Server для проверки. Чтобы пользователи могли проходить проверку подлинности, необходимо создать имена входа для соответствующего экземпляра. В SQL Server 2008 различают пять типов имен входа:

- стандартное имя входа SQL Server;
- имя входа Windows;
- группа Windows;
- сертификат;
- асимметричный ключ.

Администраторы баз данных создают стандартные имена входа и настраивают для них имя и пароль, которые и должны предъявлять пользователи при проверке подлинности. Имя входа хранится в главной базе данных и сопоставляется локальному идентификатору безопасности (SID) в SQL Server.

Имя входа SQL Server также может быть сопоставлено имени входа Windows или группе Windows. При добавлении имени входа Windows или группы Windows SQL Server сохраняет как имя имени входа или группы, так и соответствующий идентификатор безопасности Windows. Когда пользователь входит в систему экземпляра, используя учетные данные Windows, SQL Server вызывает API-функции безопасности для проверки учетной записи, получения SID, а затем его сравнения с SID, хранящимся в главной базе данных, чтобы убедиться, что данная учетная запись Windows имеет доступ к текущему экземпляру.

ПРИМЕЧАНИЕ На экзамене

Можно создавать имена входа SQL Server, сопоставленные сертификатам или асимметричным ключам, но такие имена входа не позволяют пройти проверку подлинности для доступа к экземпляру и используются для внутренних целей в качестве контейнеров безопасности.

Для создания имени входа используют следующий синтаксис:

```
CREATE LOGIN loginName { WITH <список_параметров1> | FROM <источники> }
```

```
<список_параметров1> ::=
```

```
PASSWORD = { 'пароль' | hashed_password HASHED } [ MUST_CHANGE ]  
[ , <option_list2> [ ,... ] ]
```

```
<список_параметров2> ::=
```

```
SID = sid  
| DEFAULT_DATABASE = база_данных  
| DEFAULT_LANGUAGE = язык  
| CHECK_EXPIRATION = { ON | OFF}  
| CHECK_POLICY = { ON | OFF}  
| CREDENTIAL = имя_учетных_данных
```

```
<sources> ::=
```

```
WINDOWS [ WITH <параметры_windows> [ ,... ] ]  
| CERTIFICATE имя_сертификата
```

```
| ASYMMETRIC KEY имя_асимметричного_ключа  
<параметры_windows> ::=  
  DEFAULT_DATABASE = база_данных  
  | DEFAULT_LANGUAGE = язык
```

Когда включен параметр *CHECK_POLICY* (используемый по умолчанию и рекомендуемый), при создании имени входа SQL Server в SQL Server 2008 будут применены параметры политики паролей Windows. Параметр *CHECK_EXPIRATION* служит для защиты от атак методом подбора имени входа. Если этот параметр включен, то каждый раз при использовании имени входа для проверки подлинности экземпляра SQL Server будет проверять срок действия пароля и запрашивать у пользователя смену пароля, если его срок действия истек.

Группы Windows обеспечивают наибольшую гибкость при управлении безопасностью доступа. Для управления доступом к экземпляру SQL Server достаточно просто добавить или удалить учетные записи из группы. Кроме того, администратору баз данных не нужно знать о новых и уволенных сотрудниках или о перемещении сотрудников между группами внутри организации. Ему достаточно определить группы на основе разрешений, а добавление и удаление учетных записей пользователей выполняется в рамках стандартных бизнес-процессов компании.

При создании имени входа SQL Server можно явно задать идентификатор безопасности учетной записи. Как правило, эта возможность не используется. Однако указание идентификатора безопасности при копировании имен входа SQL Server с одного экземпляра на другой позволит корректно сопоставить имена входа для любых восстанавливаемых баз данных.

Механизм защиты учетных записей в ОС Windows блокирует учетную запись после заданного количества неудачных попыток ввода пароля. Из-за неудачных попыток входа в Windows может быть заблокирована любая учетная запись кроме учетной записи администратора, которая не блокируется, поскольку в противном случае администратор не сможет войти в систему и устранить неполадки. Подобно учетной записи администратора в Windows учетная запись *sa* не блокируется в случае неудачных попыток входа и поэтому является главной мишенью для атак методом подбора. Чтобы предотвратить подобные атаки системные администраторы переименовывают учетную запись администратора. Вы тоже можете переименовать учетную запись *sa* с этой же целью.

На время обслуживания базы данных, например при развертывании нового кода или изменении структуры базы данных, необходимо запретить пользователям доступ к базе данных. Для этого можно отозвать разрешения у имени входа, но впоследствии потребуется восстановить эти разрешения. Чтобы закрыть доступ без внесения изменений в разрешения для имени входа, можно отключить это имя входа, выполнив такой код:

```
ALTER LOGIN <имя_входа> DISABLE
```

Предопределенные роли сервера

По своей функциональности роли в SQL Server совпадают с группами Windows. С помощью ролей удобно группировать нескольких пользователей с одинаковыми разрешениями. Разрешения назначаются ролям, а не отдельным пользо-

вателям. Затем при добавлении учетных записей пользователей в соответствующую роль они получают необходимый набор разрешений.

В состав SQL Server входит набор ролей на уровне экземпляра. Эти роли на уровне экземпляра называются предопределенными ролями (fixed roles) сервера, поскольку их разрешения нельзя изменить. На уровне экземпляра также нельзя создать другие роли.

В табл. 11-2 перечислены роли сервера, поставляемые в составе SQL Server.

Табл. 11-2. Предопределенные роли сервера

Роль	Возможности членов роли
<i>bulkadmin</i>	Администрирование операций массового копирования и вставки
<i>dbcreator</i>	Создание баз данных
<i>diskadmin</i>	Управление дисковыми ресурсами
<i>processadmin</i>	Управление подключениями и запуск или приостановка экземпляра
<i>securityadmin</i>	Создание, изменение и сброс имен входа, но не изменение паролей
<i>serveradmin</i>	Выполнение тех же действий, что и у ролей <i>diskadmin</i> и <i>processadmin</i> , а также управление конечными точками, изменение параметров экземпляра и отключение экземпляра
<i>setupadmin</i>	Управление связанными серверами
<i>sysadmin</i>	Выполнение любых действий в экземпляре. Участникам этой роли невозможно запретить доступ к каким-либо объектам и выполнение каких-либо действий

Пользователи базы данных

Система безопасности SQL Server работает по принципу отсутствия доступа по умолчанию. Если пользователю явно не предоставлено разрешение, он не сможет выполнить соответствующее действие. Чтобы предоставить доступ к базе данных, нужно добавить имя входа к базе данных как пользователя, выполнив команду *CREATE USER*, у которой следующий синтаксис:

```
CREATE USER имя_пользователя
[ { { FOR | FROM }
  { LOGIN имя_входа
    | CERTIFICATE имя_сертификата
    | ASYMMETRIC KEY имя_асимметричного_ключа}
  | WITHOUT LOGIN }
[ WITH DEFAULT_SCHEMA = имя_схемы ]
```

Идентификатор безопасности имени входа сопоставляется пользователю базы данных, что позволяет создать путь доступа после того, как пользователь пройдет проверку подлинности в экземпляре. Когда пользователь меняет контекст на базу данных, SQL Server находит идентификатор безопасности данного имени входа и если тот добавлен к базе данных, пользователю разрешается доступ к ней. Однако доступ пользователя к базе данных не означает, что ему автоматически доступны какие-либо объекты в этой базе данных. Для этого у пользователя должны еще быть разрешения на объекты базы данных.

Можно создать пользователя базы данных, сопоставленного сертификату или асимметричному ключу. Такие пользователи входят во внутреннюю структуру безопасности базы данных. Одно из применений этой структуры — подписи — рассматривается в занятии 4.

Пользователи без имени входа

В базе данных можно создать пользователя, не связанного с именем входа. Он так и называется — называют *пользователь без имени входа* (loginless user).

До выпуска SQL Server 2005 для того, чтобы предоставить пользователям доступ к базе данных только при выполнении конкретного приложения, использовали роли приложения. Сначала нужно было создать роль приложения с паролем и назначить ей разрешения. Затем для получения доступа к базе данных в контексте безопасности данной роли приложения пользователи должны были указать пароль для данной роли приложения. К сожалению, при подключении пользователя с ролью приложения SQL Server переставал распознавать команды, создаваемые этим пользователем, что затрудняло аудит действий.

Пользователи без имени входа были созданы взамен ролей приложений. Пользователи также проходят проверку подлинности для доступа к экземпляру, используя свои учетные данные. Имени входа пользователя должен быть предоставлен доступ к базе данных. После того как контекст пользователя меняется на контекст базы данных, при получении необходимых разрешений он олицетворяет пользователя без имени входа. Проверка подлинности пользователей на экземпляре выполняется с применением их учетных данных, поэтому возможен аудит действий отдельного имени входа в SQL Server, хотя само имя входа олицетворяет пользователя без имени входа.

ПРИМЕЧАНИЕ На экзамене

Пользователи без имени входа созданы взамен ролей приложений. Наличие пользователей без имени входа позволяет организовать более эффективный аудит, чем роли приложений, так как каждый пользователь должен пройти проверку подлинности на экземпляре с помощью своих учетных данных, а не общей учетной записи.

Предопределенные роли базы данных

Аналогично предопределенным ролям на уровне экземпляра в SQL Server предусмотрен набор предопределенных ролей на уровне базы данных (табл. 11-3).

Табл. 11-3. Предопределенные роли базы данных

Роль	Возможности участников роли
<i>db_accessadmin</i>	Добавление и удаление пользователей базы данных
<i>db_backupoperator</i>	Резервное копирование базы данных, но без возможности восстановления базы данных или просмотра информации в базе данных
<i>db_datareader</i>	Выполнение инструкции <i>SELECT</i> во всех таблицах, представлениях и функциях базы данных

Табл. 11-3. (окончание)

Роль	Возможности участников роли
<i>db_datawriter</i>	Выполнение инструкций <i>INSERT</i> , <i>UPDATE</i> , <i>DELETE</i> и <i>MERGE</i> во всех таблицах базы данных. Члены этой роли также должны быть членами роли <i>db_datareader</i>
<i>db_ddladmin</i>	Выполнение DDL-инструкций
<i>db_denydatareader</i>	Запрет выполнения инструкции <i>SELECT</i> во всех таблицах, представлениях и функциях базы данных
<i>db_denydatawriter</i>	Запрет выполнения инструкций <i>INSERT</i> , <i>UPDATE</i> , <i>DELETE</i> и <i>MERGE</i> во всех таблицах базы данных
<i>db_owner</i>	Владелец базы данных, обладающий полным доступом к базе данных и всем содержащимся в ней объектам
<i>db_securityadmin</i>	Управление составом ролей и связанными разрешениями, но без возможности управления составом роли <i>db_owner</i>
<i>public</i>	Группа по умолчанию, имеющаяся в каждой базе данных. В эту группу входят все пользователи

Роли пользователей базы данных

Вместо управления разрешениями каждой учетной записи во всех современных операционных системах поддерживаются группы пользователей, в рамках каждой группы у всех пользователей одинаковые разрешения. Системным администраторам остается лишь управлять составом групп, а не сотнями и даже тысячами отдельных разрешений.

В SQL Server реализованы те же принципы управления безопасностью, что применяются администраторами в доменах Windows при создании ролей базы данных. Роль базы данных — это участник базы данных, содержащий одного или нескольких пользователей базы данных. Разрешения назначаются ролям базы данных. Разрешения можно назначить напрямую пользователю, но настоятельно рекомендуется создавать роли базы данных, добавлять в них пользователей, и лишь затем назначать этим ролям разрешения.

Контрольные вопросы

- 1. Какие имена входа нельзя использовать для проверки подлинности на экземпляре?
- 2. Какой участник базы данных создан взамен роли приложения?

Ответы на контрольные вопросы

- 1. Для проверки подлинности на экземпляре нельзя использовать имена входа, сопоставленные сертификатам или асимметричным ключам.
- 2. Взамен ролей приложений созданы пользователи без имени входа.

Практикум. Создание имен входа и пользователей базы данных

Вы создадите несколько имен входа, добавите их как пользователей в базу данных *AdventureWorks*, а затем создадите пользователя без имени входа в базе данных *AdventureWorks*.

1. В меню **Пуск (Start)** щелкните правой кнопкой **Мой компьютер (My Computer)** и выберите **Управление (Manage)**
2. Щелкните правой кнопкой узел **Пользователи (Users)** в папке **Локальные пользователи и группы (Local Users and Groups)** и выберите **Новый пользователь (New User)**. Создайте учетную запись Windows и назовите ее **TestAccount**. Закройте консоль **Управление компьютером (Computer Management)**.
4. Выполните следующий код, чтобы добавить созданную учетную запись Windows как имя входа в свой экземпляр, указав вместо *<имя_компьютера>* имя компьютера, на котором работает SQL Server.

--Скобки необходимы по правилам для идентификаторов

```
CREATE LOGIN [<имя_компьютера>\TestAccount] FROM WINDOWS  
GO
```

5. Выполните следующий код, чтобы создать два собственных имени входа SQL Server, указав вместо *<Введите_здесь_надежный_пароль>* надежный пароль, и добавить созданные учетные записи как пользователей в базу данных *AdventureWorks*.

```
CREATE LOGIN Test WITH PASSWORD = '<Введите_здесь_надежный_пароль>'  
CREATE LOGIN Test2 WITH PASSWORD = '<Введите_здесь_надежный_пароль>'  
GO  
USE AdventureWorks  
GO  
CREATE USER Test FOR LOGIN Test  
CREATE USER Test2 FOR LOGIN Test2  
GO
```

6. Выполните следующий код, чтобы создать пользователя без имени входа в базе данных *AdventureWorks*:

```
USE AdventureWorks  
GO  
CREATE USER TestUser WITHOUT LOGIN  
GO
```

6. Выполните следующий код, чтобы просмотреть конечные точки, а также участников экземпляра и базы данных.

--Участники на уровне экземпляра.

```
SELECT * FROM sys.asymmetric_keys  
SELECT * FROM sys.certificates  
SELECT * FROM sys.credentials  
SELECT * FROM sys.linked_logins
```

```
SELECT * FROM sys.remote_logins
SELECT * FROM sys.server_principals
SELECT * FROM sys.server_role_members
SELECT * FROM sys.sql_logins
SELECT * FROM sys.endpoints
GO
```

--Участники на уровне базы данных.

```
SELECT * FROM sys.database_principals
SELECT * FROM sys.database_role_members
GO
```

7. Выполните следующий код, чтобы переименовать учетную запись *sa*.

```
ALTER LOGIN sa WITH NAME = MySaAccount
GO
```

Резюме занятия

- Можно создавать собственные имена входа SQL Server или сопоставлять учетные записи Windows именам входа SQL Server.
- Имена входа могут быть сопоставлены сертификатам и асимметричным ключам, но такие имена входа нельзя использовать для проверки подлинности на экземпляре.
- Поскольку учетная запись *sa* не может быть заблокирована, ее следует переименовать командой *ALTER LOGIN*.
- Участники роли *sysadmin* могут выполнять любые действия на экземпляре, и им невозможно запретить выполнение каких-либо команд. Участники роли *db_owner* могут выполнять любые действия в соответствующей базе данных, и им невозможно запретить выполнение каких-либо команд в этой базе данных.
- Пользователи без имени входа созданы взамен ролей приложений, это пользователи базы данных, которым не сопоставлены имена входа.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. В компании Wide World Importers развернут домен Windows Server 2003, и все серверы, на которых установлен SQL Server, работают под управлением Windows Server 2003 Enterprise Edition. На экземпляре SQL Server настроена только проверка подлинности Windows. Для каждой группы разрешений в базе данных созданы роли базы данных. Имена входа добавлены в роли базы данных. Администраторы баз данных хотят передать назначение разрешений пользователей владельцам отдельных приложений, но при этом не терять возможности управлять учетными записями или разрешениями на экземпляре SQL Server. Как администраторам баз данных решить постав-

ленную задачу? (Выберите два варианта ответа, которые в совокупности образуют правильный ответ.)

- A. Администратор Windows должен разрешить владельцам приложений управлять группами Windows, связанными с их приложениями.
 - B. Добавить имена входа для владельцев приложений в роль *securityadmin*.
 - C. Сопоставить имена входа SQL Server группам Windows, соответствующим отдельным приложениям.
 - D. Добавить имена входа для владельцев приложения в роль *sysadmin*.
2. Тине нужно создавать резервные копии базы данных. При этом ей не нужны разрешения на восстановление базы данных или доступ к ее содержимому. Как реализовать эти бизнес-требования с наименьшими усилиями?
- A. Добавить Тину в роль *diskadmin*.
 - B. Добавить Тину в роль *db_owner*.
 - C. Добавить Тину в роль *db_backupoperator*.
 - D. Добавить Тину в роль *sysadmin*.

Занятие 4. Управление разрешениями

По умолчанию в SQL Server запрещен доступ к объектам и действиям. Поэтому для получения доступа к объектам и выполнения каких-либо действий необходимо получить соответствующее разрешение. На этом занятии вы научитесь управлять разрешениями на так называемые *защищаемые объекты* (securables) на уровне экземпляра или базы данных, олицетворять пользователя для целей проверки правильности установки разрешений, а также создавать главные ключи и управлять ими, что позволит использовать подписи для повышения разрешений только на время выполнения кода.

Изучив материал этого занятия, вы сможете:

- ✓ назначать пользователям разрешения;
- ✓ управлять разрешениями, используя области;
- ✓ объяснить преимущества защиты метаданных;
- ✓ работать с цепочками владения;
- ✓ олицетворять имена входа и пользователей;
- ✓ создавать главные ключи и управлять ими;
- ✓ создавать подписи и модули подписи.

Продолжительность занятия — 20 минут.

Административные учетные записи

Административные учетные записи занимают особое место в структуре безопасности SQL Server и включают в себя:

- членов предопределенной роли сервера *sysadmin*;
- членов предопределенной роли базы данных *db_owner*;
- учетную запись *sa*.

Кроме того, члены роли *sysadmin* являются членами роли *db_owner* в каждой базе данных на экземпляре.

Чтобы запретить учетной записи выполнять определенное действие, нужно удалить соответствующее разрешение. Возможность ограничения разрешений административных учетных записей отсутствует. Разрешения можно попытаться удалить с помощью специальных команд, но это не даст результата, так как в SQL Server разрешения административных учетных записей не проверяются.

Защищаемые объекты

Что толку от разрешений, если их не к чему применить? И зачем нужны разрешения, если их некому использовать? Разрешения работают, если есть защищаемые объекты и участники системы безопасности. Для определения (отмены, запрета) разрешения используются инструкции *GRANT/REVOKE/DENY* *<разрешение> ON <защищаемые объекты> TO <участники>*.

Защищаемыми называются объекты, на которые предоставляют разрешения. Каждый объект в SQL Server, включая весь экземпляр, является защищаемым. Например, экземпляр содержит базы данных, базы данных — схемы, а схемы включают таблицы, представления, процедуры, функции и т. д.

Схемы

Любой создаваемый в базе данных объект не может существовать без владельца. У всех объектов должны быть владельцы, поскольку объекты не возникают самопроизвольно — кто-то их создает. Кроме того, для доступа к объекту учетной записи должны быть назначены разрешения. Поэтому необходим хотя бы один пользователь, который имеет право управлять разрешениями на данный объект.

Поскольку владельцами объектов являются пользователи, при удалении пользователя из базы данных могут возникнуть проблемы с управлением объектами. Если бы пользователи базы данных владели объектами напрямую, пользователя можно было бы удалить лишь после переназначения соответствующего объекта другому владельцу, а при этом изменится имя объекта.

Схемы представляют контейнеры, которые являются владельцами всех объектов базы данных. Схемами, в свою очередь, владеют пользователи базы данных. Благодаря применению схем как посредников между пользователями и объектами можно удалять пользователей из базы данных, не изменяя имен объектов и не влияя на работу приложений, использующих эти объекты. Схемы — единственные объекты, которыми пользователи базы данных владеют напрямую. Поэтому перед удалением пользователя, владеющего схемой, нужно назначить владельцем этой схемы другого пользователя.

Разрешения

Разрешения позволяют участникам выполнять различные действия в пределах экземпляра или базы данных. Некоторые разрешения затрагивают такие инструкции, как *INSERT*, *UPDATE* и *SELECT*, другие применяются к таким дей-

ствиям, как *ALTER TRACE*, а третьи охватывают широкий спектр возможностей управления, например разрешение *CONTROL*.

Для создания новых разрешений объекта применяют инструкцию *GRANT*, а для запрета доступа — инструкцию *DENY*. Для доступа к объекту необходимо явно задать соответствующее разрешение. Всякий раз при выполнении инструкции *GRANT* в таблицу безопасности SQL Server добавляется запись о присвоении соответствующего разрешения, а при выполнении инструкции *DENY* — о запрете на доступ. Поскольку *DENY* отменяет любые разрешения, заданные ею разрешения (а точнее запрещающие разрешения) пользуются приоритетом перед разрешениями, заданными инструкцией *GRANT*.

Инструкция *REVOKE* удаляет записи разрешений указанного объекта. Например, после выполнения инструкции:

```
GRANT SELECT ON Person.Address TO Test
```

можно отменить предоставленный доступ командой:

```
REVOKE SELECT ON Person.Address FROM Test.
```

Аналогично, выполнив инструкцию:

```
DENY SELECT ON Person.Address TO Test
```

можно затем удалить разрешение *DENY* командой:

```
REVOKE SELECT ON Person.Address FROM Test.
```

Разрешения также можно предоставлять на нескольких уровнях. Например, можно назначить разрешение *SELECT* базе данных *AdventureWorks*, схеме *Person* или непосредственно таблице *Person.Address*. Чтобы запретить пользователю доступ к таблице *Person.Address*, можно позже выполнить инструкции *REVOKE* по отношению к базе данных, схеме и таблице. Это удалит разрешение *SELECT* на таблицу *Person.Address*.

Область действия разрешений

До выпуска SQL Server 2005 все разрешения назначались напрямую объектам базы данных. В SQL Server 2005 и более поздних выпусках разрешения могут иметь различные *области* (scope). Защищаемым объектом может быть база данных, схема или объект. Таким образом, разрешения можно назначать любому из этих защищаемых объектов. Определение разрешения базе данных неявным образом распространяет это разрешение на все схемы в этой базе данных и, следовательно, на все объекты в этих схемах. Назначение разрешения схеме означает, что это разрешение неявно назначается всем объектам в данной схеме.

ПРИМЕЧАНИЕ Область действия разрешений

Разрешения можно назначать на любом защищаемом объекте. Используя контейнеры более высокого уровня, например базы данных и схемы, можно очень гибко назначать разрешения. Все разрешения можно назначить напрямую объектам самого низкого уровня, однако если пользователю нужно предоставить одинаковое разрешение на все объекты схемы или базы данных, можно обойтись без десятков и даже тысяч отдельных разрешений, назначив разрешение схеме или базе данных, в которой содержатся эти объекты.

Схема представляет первый уровень безопасности в пределах базы данных, которым следует пользоваться при планировании и назначении разрешений. Схема, как правило, создается как функциональная группировка объектов в приложении, например заказчиков, продуктов, товарных запасов или данных отдела кадров. Затем в рамках схемы создаются объекты. Разрешения предоставляются схемам для обеспечения безопасного доступа приложений.

Например, назначить разрешения *SELECT*, *INSERT*, *UPDATE* и *DELETE* всем таблицам и представлениям базы данных можно одним из трех способов:

- назначить разрешения на каждую таблицу и представление;
- назначить разрешения на каждую схему в базе данных;
- назначить разрешения на базу данных.

Безопасность метаданных

В повседневной жизни мы принимаем как должное, что от нас скрыты многие вещи, на доступ к которым у нас нет прав. Поэтому не удивительно, что в SQL Server реализован сходный принцип: «Меньше знаешь, крепче спишь». В SQL Server все метаданные системы засекречены, поэтому пользователи могут просматривать только объекты экземпляра или базы данных, на которую им предоставлены разрешения.

Чтобы разрешить пользователям просматривать метаданные в базе данных, выполните следующий код:

```
GRANT VIEW DEFINITION TO <пользователь>
```

Если разрешение *VIEW ANY DEFINITION* предоставлено имени входа, используя его, пользователь сможет просматривать метаданные любых объектов экземпляра. Разрешение *VIEW ANY DATABASE* позволяет имени входа проверять наличие баз данных на SQL Server даже при отсутствии прав на доступ к этим базам данных. Имя входа, которому предоставлено разрешение *VIEW SERVER STATE*, может просматривать статистику выполнения, например представление *sys.dm_exec_requests*.

Цепочки владения

Каждому объекту в базе данных соответствует владелец схемы. Также можно создавать объекты, ссылающиеся на другие объекты в базе данных, например хранимые процедуры, вызывающие функции, которые выполняют инструкции *SELECT* над представлениями, основанными на таблицах. Владельцы всех объектов, на которые ссылается стек вызовов, образуют цепочку владения по мере того, как при выполнении кода происходит переход от объекта к объекту в стеке вызова. Если владелец объекта и владельцы всех прочих объектов, на которые ссылается данный объект, совпадают, говорят, что цепочка владения *целостная* (intact). В SQL Server выполняется проверка разрешений на объект наверху стека вызовов, а также при каждом изменении владельца объекта в стеке.

Благодаря использованию цепочек владения хранимые процедуры становятся наиболее мощным механизмом безопасности базы данных. Вызов хранимых процедур можно реализовать в приложениях, при этом хранимые процедуры будут выполнять все действия с данными, необходимые для приложения. Вместе с тем, пользователям никогда не предоставляется прямой доступ к базовым

таблицам, потому как возможно выполнение лишь тех действий, которые разрешены в соответствующей хранимой процедуре.

ПРИМЕЧАНИЕ Прикладные API-интерфейсы

Интересно, что многие разработчики считают нецелесообразным вызывать хранимые процедуры в приложениях и предпочитают встраивать инструкции SQL напрямую в приложение. Но ни одному из разработчиков не придет в голову написать приложение, которое само по себе будет встроенным кодом. Вместо этого они тратят уйму времени на создание объектов, имеющих интерфейсы, а затем создают приложения, соединяя объекты посредством их интерфейсов. Такой подход к разработке позволяет нескольким разработчикам работать над одним сложным приложением, даже если зависимые фрагменты кода еще не завершены. Хранимые процедуры выполняют те же функции, что и API-интерфейсы, используемые разработчиками в каждом приложении. Хранимая процедура представляет не что иное, как API-интерфейс базы данных. Это значит, что разработчикам даже не обязательно знать структуру этой базы данных.

Если в стеке вызовов задействовано несколько владельцев объектов, а пользователю не предоставлено разрешение на все объекты в стеке вызовов, говорят, что цепочка владения *прервана*. Широко распространено заблуждение, что прерванная цепочка владения — результат ошибки при проектировании базы данных. В некоторых случаях, например при организации аудита, цепочку владения прерывают нарочно, чтобы пользователи не смогли получить доступ к коду, который используется для аудита действий, или к любым хранимым данным аудита. Пробелы в прерванных цепочках владения можно «заполнить», задействовав подписи, о которых мы поговорим в конце этого занятия.

ПРИМЕЧАНИЕ Владелец объекта

Хотя все объекты базы данных содержатся в схемах, при определении цепочек владения в SQL Server владелец схемы считается владельцем всех объектов данной схемы.

Олицетворение

Для выполнения команд в контексте другого пользователя можно задать олицетворение другого участника. Для этого учетной записи, выполняющей олицетворение, должно быть предоставлено разрешение *IMPERSONATE* для участника, которого планируется олицетворять. Если разрешение *IMPERSONATE* назначено имени входа, можно олицетворить это имя входа и выполнять действия от имени этого участника в любой базе данных, к которой у этого участника есть доступ. Если же разрешение *IMPERSONATE* назначено пользователю базы данных, действия в этом контексте пользователя можно выполнять только в пределах этой базы данных.

Олицетворение выполняется инструкцией *EXECUTE AS*:

```
{ EXEC | EXECUTE ] AS <context_specification>
<context_specification> ::=
{ LOGIN | USER } = 'name'
[ WITH { NO REVERT | COOKIE INTO @varbinary_variable } ]
| CALLER
```

ПРИМЕЧАНИЕ На экзамене

Чтобы создать схему, владельцем которой является другой участник базы данных, у пользователя, создающего схему, должно быть разрешение *IMPERSONATE* на участника, который будет назначен владельцем схемы.

Если инструкция *EXECUTE AS* была выполнена без предложения *NO REVERT*, можно вернуться к предыдущему контексту выполнения инструкцией *REVERT*.

Главные ключи

Главные ключи лежат в основе иерархии шифрования в SQL Server. Они также необходимы для создания сертификатов и асимметричных ключей. Существует единый главный ключ службы для экземпляра и главные ключи базы данных в каждой базе данных.

Главный ключ службы

У каждого экземпляра SQL Server имеется главный ключ службы, который автоматически создается при первом запуске экземпляра. *Главные ключи службы* (Service master keys) — это симметричные ключи, создаваемые на основе ключа локального компьютера и шифруемые с использованием учетной записи службы SQL Server и API-интерфейса защиты данных Windows (Windows Data Protection API).

Такой процесс создания и шифрования гарантирует, что главный ключ службы может быть расшифрован только учетной записью службы, которой он был создан, или участником, у которого есть доступ к учетным данным этой службы. По умолчанию главный ключ службы используется для шифрования всех главных ключей баз данных, создаваемых в экземпляре.

Главный ключ базы данных

Главный ключ базы данных создается явным образом командой:

```
CREATE MASTER KEY ENCRYPTION BY PASSWORD = '<Надежный_пароль>'
```

У каждой базы данных свой главный ключ. Поэтому пользователь, способный расшифровать данные одной базы данных, не сможет расшифровать данные в другой без соответствующего разрешения.

Главный ключ базы данных служит для защиты всех сертификатов, симметричных и асимметричных ключей, хранящихся в базе данных. Главный ключ базы данных шифруется с применением алгоритма Triple DES и заданного пользователем пароля. Копия главного ключа базы данных также шифруется с использованием главного ключа службы, что делает возможным автоматическую расшифровку в текущем экземпляре.

При создании запроса на расшифровку данных главный ключ службы используется для расшифровки главного ключа базы данных, который, в свою очередь, применяется для расшифровки сертификата, симметричного или асимметричного ключа, а тот, в свою очередь, применяется для расшифровки данных.

Эта иерархия очень важна, и надо соблюдать осторожность при перемещении резервных копий, содержащих зашифрованные данные, между экземплярами SQL Server. Чтобы восстановить и успешно расшифровать данные также по-

требуется создать резервную копию главного ключа базы данных, а затем повторно создать главный ключ базы данных на другом экземпляре. Для этого применяют команды *OPEN MASTER KEY*, *BACKUP MASTER KEY*, *RESTORE MASTER KEY* и *CLOSE MASTER KEY*.

ПРИМЕЧАНИЕ Главные ключи

Подробнее о шифровании данных вы узнаете в занятии 6. Главный ключ базы данных необходим для создания сертификата, который служит для подписывания.

Сертификаты

Сертификаты — это ключи, основанные на стандарте X.509 и служащие для проверки подлинности сущности, предъявляющей сертификат. Можно создавать общие и личные сертификаты. Общий сертификат представляет собой файл, выпущенный центром сертификации и служащий для подтверждения достоверности сущности, использующей данный сертификат. Личные сертификаты создаются организациями для защиты их данных. Например, личный сертификат на веб-сайте банка подтверждает достоверность веб-сайта и применяется для шифрования данных, которыми обмениваются браузер клиента и серверы банка.

Для создания самозаверенного сертификата в SQL Server выполните следующую команду:

```
CREATE CERTIFICATE имя_сертификата [ AUTHORIZATION имя_пользователя ]
    { FROM <existing_keys> | <generate_new_key> }
    [ ACTIVE FOR BEGIN_DIALOG = { ON | OFF } ]
```

```
<existing_keys> ::=
    ASSEMBLY assembly_name | {
        [ EXECUTABLE ] FILE = 'путь_к_файлу'
        [ WITH PRIVATE KEY ( <параметры_закрытого_ключа> ) ] }
```

```
<generate_new_key> ::=
    [ ENCRYPTION BY PASSWORD = 'пароль' ]
    WITH SUBJECT = 'имя_субъекта_сертификата'
    [ , <date_options> [ ,...n ] ]
```

```
<параметры_закрытых_ключей> ::=
    FILE = 'путь_к_личному_ключу'
    [ , DECRYPTION BY PASSWORD = 'пароль' ]
    [ , ENCRYPTION BY PASSWORD = 'пароль' ]
```

```
<параметры_даты> ::=
    START_DATE = 'mm/dd/yyyy' | EXPIRY_DATE = 'mm/dd/yyyy'
```

Подписи

Подпись (signature) позволяют повысить разрешения пользователя, но только при выполнении пользователем определенного фрагмента кода.

Цифровую подпись добавляют к модулю (хранимым процедурам, функциям, триггерам и сборкам) командой *ADD SIGNATURE*. Процесс цифрового подписывания кода для управления разрешениями выглядит так:

1. Создается главный ключ базы данных.
2. Создается сертификат в базе данных.
3. Создается пользователь, сопоставленный сертификату.
4. Пользователю назначаются разрешения на объект или объекты.
5. Выполняется команда *ADD SIGNATURE* по отношению к модулю с применением сертификата.

Одна из наиболее полезных сфер применения подписей — восполнение пробела в прерванной цепочке владения.

Например, можно создать логику для аудита действий пользователя в базе данных и запретить пользователям прямой доступ к данным аудита путем прерывания цепочки владения. Код, протоколирующий действия пользователя, можно затем подписать цифровой подписью. Таким образом, аудит будет выполняться в контексте транзакции пользователя, но у пользователя не будет возможности непосредственно просматривать таблицы аудита.

Контрольные вопросы

1. Как связаны между собой участники, защищаемые объекты и разрешения?
2. Дайте определение цепочки владения. Как возникают прерванные цепочки владения?

Ответы на контрольные вопросы

1. Можно предоставлять (*GRANT*), отзывать (*REVOKE*) и запрещать (*DENY*) разрешения на (*ON*) защищаемый объект для (*TO*) участника.
2. Цепочка владения имеет отношение к объектам, ссылающимся на другие объекты в базе данных. Владелец схемы, содержащей объект, считается владельцем этого объекта. SQL Server проверяет разрешения на первый объект, к которому получает доступ пользователь, а затем всякий раз, когда меняется владелец в стеке вызовов. Цепочка владельцев объектов в стеке вызовов называется цепочкой владения. Прерванная цепочка владения возникает, если при изменении владельца объекта в стеке вызовов и у пользователя не оказывается разрешений для доступа к другим объектам в текущем стеке вызовов.

Практикум. Управление разрешениями

Вы предоставите разрешения на объекты с различной областью действия и посмотрите, как это повлияет на безопасность метаданных в базе данных. Вы также изучите цепочки владения и воспользуетесь подписями, чтобы предоставить доступ к объекту посредством хранимой процедуры при отсутствии прямого доступа к этому объекту.

Задание 1. Определение разрешений объекта

Вы назначите разрешения на объект с различной областью действия и с помощью олицетворения посмотрите, как это повлияет на безопасность метаданных.

1. Выполните следующий код, чтобы проверить контекст пользователя:

```
SELECT SUSER_SNAME(), USER_NAME()  
GO
```

2. Измените контекст на базу данных *AdventureWorks* и просмотрите список объектов:

```
USE AdventureWorks  
GO
```

--Просматриваем список объектов в базе данных.

```
SELECT * FROM sys.objects  
GO
```

3. Выполните олицетворение тестового пользователя и просмотрите список объектов:

```
EXECUTE AS USER = 'Test'  
GO  
SELECT SUSER_SNAME(), USER_NAME()  
GO  
SELECT * FROM sys.objects  
GO  
REVERT  
GO  
SELECT SUSER_SNAME(), USER_NAME()  
GO
```

4. Предоставьте разрешение *SELECT* на таблицу *Production.Document* тестовому пользователю и просмотрите результат:

```
GRANT SELECT ON Production.Document TO Test  
GO  
  
EXECUTE AS USER = 'Test'  
GO  
SELECT * FROM sys.objects  
SELECT DocumentNode, Title, FileName FROM Production.Document  
REVERT  
GO
```

5. Предоставьте разрешение *SELECT* на схему *Production* и просмотрите результаты:

```
--Разрешение на схему  
GRANT SELECT ON SCHEMA::Production TO Test  
GO  
EXECUTE AS USER = 'Test'  
GO
```

```
SELECT * FROM sys.objects  
REVERT  
GO
```

6. Предоставьте разрешение *SELECT* на всю базу данных *AdventureWorks* и просмотрите результаты. Обратите внимание, что хотя теперь у пользователя есть разрешение *SELECT* на всю базу данных, в ней остаются объекты, которые видны только владельцу базы данных.

```
GRANT SELECT ON DATABASE::AdventureWorks TO Test  
GO  
EXECUTE AS USER = 'Test'  
GO  
SELECT * FROM sys.objects  
REVERT  
GO
```

7. Отмените возможность просматривать метаданные объекта и просмотрите результаты:

```
DENY VIEW DEFINITION TO Test  
GO  
EXECUTE AS USER = 'Test'  
GO  
SELECT * FROM sys.objects  
SELECT DocumentNode, Title, FileName FROM Production.Document  
REVERT  
GO
```

8. Восстановите возможность просматривать метаданные объекта:

```
REVOKE VIEW DEFINITION FROM Test  
GO  
EXECUTE AS USER = 'Test'  
GO  
SELECT * FROM sys.objects  
REVERT  
GO
```

9. Удалите разрешение *SELECT* на базу данных. Обратите внимание, что пользователь по-прежнему может просматривать содержимое схемы *Production*.

```
REVOKE SELECT ON DATABASE::AdventureWorks FROM Test  
GO  
EXECUTE AS USER = 'Test'  
GO  
SELECT * FROM sys.objects  
REVERT  
GO
```

10. Удалите разрешение *SELECT* на схему. Обратите внимание, что пользователь по-прежнему может просматривать таблицу *Production.Document* и объекты, напрямую связанные с таблицей.

```
REVOKE SELECT ON SCHEMA::Production FROM Test
GO
EXECUTE AS USER = 'Test'
GO
SELECT * FROM sys.objects
REVERT
GO
```

11. Удалите разрешение *SELECT* на таблицу. Обратите внимание, что доступ тестового пользователя к таблице *Production.Document* наконец отменен.

```
REVOKE SELECT ON Production.Document FROM Test
GO
EXECUTE AS USER = 'Test'
GO
SELECT * FROM sys.objects
REVERT
GO
```

Задание 2. Создание и управление главными ключами

Вы создадите главный ключ базы данных и пользователя базы данных на основе сертификата. Вы также узнаете, как создавать резервную копию сертификата.

1. Создайте главный ключ в базе данных *AdventureWorks*:

```
USE AdventureWorks
GO
CREATE MASTER KEY ENCRYPTION BY PASSWORD = '<Введите_здесь_надежный_пароль>'
GO
```

2. Создайте резервную копию главного ключа базы данных и сохраните ее в надежном месте, отдельно от резервных копий базы данных.

```
OPEN MASTER KEY DECRYPTION BY PASSWORD = '<Введите_здесь_надежный_пароль>'

BACKUP MASTER KEY TO FILE = 'C:\Program Files\
Microsoft SQL Server\MSSQL10.MSSQLSERVER\
MSSQL\Backup\awmasterkey.key'
ENCRYPTION BY PASSWORD = '<Введите_здесь_надежный_пароль>'
GO
```

3. Создайте сертификат:

```
CREATE CERTIFICATE TestCert WITH SUBJECT = 'Test Certificate'
GO
```

4. Создайте резервную копию сертификата и сохраните ее в надежном месте, отдельно от резервных копий базы данных.

```
BACKUP CERTIFICATE TestCert TO FILE = 'C:\Program Files\
Microsoft SQL Server\MSSQL10.MSSQLSERVER\
MSSQL\Backup\testcert.cer'
GO
```

5. Создайте пользователя базы данных, сопоставленного данному сертификату:

```
CREATE USER CertUser FROM CERTIFICATE TestCert
GO
```

Задание 3. Добавление в модуль подписи для восстановления цепочки владения

Вы намеренно создадите прерванную цепочку владения, а затем с помощью подписей получите доступ к объектам, к которым у вашей учетной записи в обычных условиях доступа нет.

1. Создайте схему и тестовые объекты, образующие прерванную цепочку владения:

```
CREATE SCHEMA SignatureTest AUTHORIZATION Test2
GO
CREATE TABLE SignatureTest.TestTable
(ID          INT          IDENTITY(1,1),
Col1        VARCHAR(10) NOT NULL)
GO
INSERT INTO SignatureTest.TestTable
(Col1)
VALUES ('Row1'), ('Row2')
GO
--Создание процедуры для доступа к тестовой таблице
CREATE PROCEDURE SignatureTest.asp_Proc1
AS
    SELECT ID, Col1 FROM SignatureTest.TestTable
GO
--Создание стека вызовов хранимой процедуры
CREATE PROCEDURE dbo.asp_SignatureTest
AS
    EXEC SignatureTest.asp_Proc1
GO
--Предоставление разрешений на выполнение внешней хранимой процедуры
GRANT EXECUTE ON dbo.asp_SignatureTest TO Test
GO
```

2. Протестируйте хранимую процедуру:

```
EXECUTE AS USER = 'Test'
EXEC dbo.asp_SignatureTest
REVERT
GO
```

3. Предоставьте разрешение на выполнение пользователю, сопоставленному сертификату, на внутреннюю хранимую процедуру и добавьте цифровую подпись для внешней процедуры:

```
GRANT EXECUTE ON SignatureTest.asp_Proc1 TO CertUser
GO
```



```
--Подписание процедуры с помощью сертификата  
ADD SIGNATURE TO dbo.asp_SignatureTest BY CERTIFICATE TestCert  
GO
```

4. Протестируйте выполнение процедуры:

```
EXECUTE AS USER = 'Test'  
EXEC dbo.asp_SignatureTest  
REVERT  
GO
```

5. Убедитесь, что пользователь не может непосредственно выполнить внутреннюю хранимую процедуру:

```
EXECUTE AS USER = 'Test'  
EXEC SignatureTest.asp_Proc1  
REVERT  
GO
```

6. Проверьте, что пользователь не может получить доступ к таблице напрямую:

```
EXECUTE AS USER = 'Test'  
SELECT ID, Col1 FROM SignatureTest.TestTable  
REVERT  
GO
```

7. Убедитесь, что вы не можете олицетворить пользователя, сопоставленного сертификату:

```
EXECUTE AS USER = 'CertUser'  
GO
```

Резюме занятия

- Разрешения предоставляются (*GRANT*) на (*ON*) защищаемый объект для (*TO*) участника.
- Экземпляр, база данных и схема являются защищаемыми объектами. Назначение разрешений на базу данных или схему применяется ко всем объектам, содержащимся в этой базе данных или схеме.
- Все метаданные в SQL Server защищены. Если у пользователя нет разрешений на объект, он его даже не увидит.
- Инструкция *EXECUTE AS* позволяет задать олицетворение имени входа или пользователя базы данных. Невозможно олицетворение участника, который сопоставлен сертификату или асимметричному ключу.
- Главный ключ службы создается при первом запуске экземпляра. Главный ключ базы данных должен создаваться явно в каждой базе данных. Он необходим для создания сертификата, асимметричного и симметричного ключа.
- Цифровые подписи применяются к модулям кода с помощью инструкции *ADD SIGNATURE* и служат для повышения разрешений только при выполнении определенного модуля и не разрешают прямой доступ к базовым объектам.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Компания Wide World Importers недавно реализовала новую систему обработки заказов. У всех пользователей, имеющих доступ к базе данных, должна быть возможность выполнения инструкции *SELECT* в любой таблице базы данных. Как реализовать эту функциональность с минимальными усилиями?
 - A. Добавить пользователей в роль базы данных *db_datawriter*.
 - B. Предоставить пользователям разрешение *SELECT* на все таблицы в базе данных.
 - C. Предоставить пользователям разрешение *SELECT* на базу данных.
 - D. Предоставить пользователям разрешение *SELECT* на все схемы в базе данных.
2. Какая инструкция запретит пользователям просматривать метаданные об объектах в отдельной базе данных, даже если у них есть доступ к этим объектам?
 - A. *DENY VIEW DEFINITION*
 - B. *DENY VIEW ANY DEFINITION*
 - C. *DENY VIEW SERVER STATE*
 - D. *REVOKE VIEW DEFINITION*

Занятие 5. Аудит экземпляров SQL Server

После предоставления минимальных разрешений, необходимых для выполнения задачи, рассмотрим второй принцип безопасности: «Доверяй, но проверяй». В этом занятии вы узнаете о возможностях аудита в SQL Server 2008.

Изучив материал этого занятия, вы сможете:

- ✓ создавать триггеры DDL;
- ✓ конфигурировать спецификации аудита экземпляра и базы данных;
- ✓ реализовывать аудит C2.

Продолжительность занятия — 30 минут.

Триггеры DDL

В дополнение к действиям *CREATE*, *DROP* и *ALTER* триггеры DDL позволяют фиксировать и реагировать на события входа в систему. Триггеры DDL можно задавать на уровне экземпляра или базы данных.

Общий синтаксис создания триггера DDL таков:

```
CREATE TRIGGER имя_триггера
ON { ALL SERVER | DATABASE }
[ WITH <ddl_trigger_option> [ ,...n ] ]
{ FOR | AFTER } { event_type | event_group } [ ,...n ]
```

```
AS { sql_statement [ ; ] [ ,...n ] |  
    EXTERNAL NAME < method specifier > [ ; ] }  
  
--Триггер события входа в систему  
CREATE TRIGGER имя_триггера  
ON ALL SERVER  
[ WITH <параметр_триггера_входа> [ ,...n ] ]  
{ FOR | AFTER } LOGON  
AS { sql_statement[ ; ] [ ,...n ] |  
    EXTERNAL NAME < method specifier > [ ; ] }
```

Предложение *ON* служит для задания области действия триггера — экземпляра (*ON ALL SERVER*) или база данных (*ON DATABASE*). В предложении *FOR* нужно указать событие или группу событий DDL, которые запускают триггер.

Триггеры DDL срабатывают в контексте выполняемой инструкции DDL. Помимо получения информации о выполненной команде, триггеры DDL позволяют запретить различные действия DDL. Если в триггере DDL выполнить команду *ROLLBACK TRANSACTION*, произойдет откат выполненной инструкции DDL, поскольку почти все инструкции DDL являются транзакционными и выполняются автоматически в контексте транзакции.

Однако не все инструкции DDL выполняются в контексте транзакции. Инструкция *ALTER DATABASE* может вносить изменения не только в базу данных, но и в ее файловую структуру. Поскольку операционная система Windows не является транзакционной, нельзя откатить операцию в файловой системе. Поэтому команда *ALTER DATABASE* выполняется вне области транзакции. Запуск триггера DDL *ON ALTER DATABASE* по-прежнему возможен, хотя этот триггер поддерживает лишь аудит и не позволяет запретить какие-либо действия.

ПРИМЕЧАНИЕ На экзамене

Важная особенность триггеров DDL — возможность выполнить откат действий. В рамках платформы Policy-Based Management Framework триггеры DDL создаются для всех политик, настраиваемых для предотвращения ситуаций несоответствия.

В SQL Server реализован механизм группирования всех DDL-событий экземпляра. Можно создать триггер DDL, запускаемый при выполнении команд *CREATE*, *DROP* и *ALTER* по отношению к таблице или задать соответствующую группу событий — *DDL_TABLE_EVENTS*.

ПРИМЕЧАНИЕ Группы событий

Подробнее о группах событий см. статью «DDL Event Groups» (группы событий DDL) по адресу: <http://technet.microsoft.com/en-us/library/bb510452.aspx>.

В контексте выполнения триггера DDL доступна специальная функция *EVENTDATA()*, предоставляющая информацию о действии DDL. Функция *EVENTDATA()* возвращает документ XML, структура которого определяется событием.

ПРИМЕЧАНИЕ Схемы EVENTDATA()

Документация по схеме XML для событий доступна по адресу: <http://schemas.microsoft.com/sqlserver/2006/11/eventdata>.

Спецификации аудита

До появления SQL Server 2008 для полного аудита экземпляра требовалось использовать несколько функциональных возможностей сервера. Триггеры DDL отслеживали изменения DDL, триггеры DML — изменения данных, но увеличивали продолжительность транзакций, трассировка SQL использовалась для аудита инструкций *SELECT*.

В SQL Server 2008 все эти возможности аудита включены в спецификацию аудита. Спецификации аудита начинаются с объекта аудита уровня сервера, который определяет место хранения журнала аудита. Затем создаются спецификации аудита сервера и базы данных, привязанные к объекту аудита.

Общий синтаксис создания объекта аудита сервера выглядит так:

```
CREATE SERVER AUDIT имя_аудита
    TO { [ FILE (<параметры файлов> [, ...n]) ] |
        APPLICATION_LOG | SECURITY_LOG }
    [ WITH ( <параметры аудита> [, ...n] ) ] [ ; ]

<параметры файлов> ::=
{ FILEPATH = 'путь_к_файлу_в_ОС'
  [, MAXSIZE = { max_size { MB | GB | TB } | UNLIMITED } ]
  [, MAX_ROLLOVER_FILES = integer ]
  [, RESERVE_DISK_SPACE = { ON | OFF } ] }

<параметры аудита> ::=
{ [ QUEUE_DELAY = integer ]
  [, ON_FAILURE = { CONTINUE | SHUTDOWN } ]
  [, AUDIT_GUID = uniqueidentifier ] }
```

При указании файла для журнала аудита можно задать максимальный размер одного файла журнала, а также количество файлов продолжения, сохраняемых в операционной системе. Кроме того, можно заранее выделить дисковое пространство для журнала аудита, чтобы избежать роста файла по мере добавления строк аудита.

Регистрация сообщений происходит синхронно или асинхронно. Когда *QUEUE_DELAY* равно нулю, записи аудита отправляются в журнал аудита синхронно с транзакцией. Если задать время задержки (в миллисекундах), записи аудита могут накапливаться, но они должны быть записаны в журнал в указанный интервал.

Действие *ON_FAILURE* управляет поведением экземпляра в случае, если записи аудита не могут быть добавлены в журнал. Параметр по умолчанию *CONTINUE* позволяет экземпляру продолжить выполнение и обработку транзакций. При задании параметра *SHUTDOWN* если запись аудита не может быть добавлена в журнал в указанный в параметре *QUEUE_DELAY* интервал времени, работа экземпляра завершается.

После определения объекта аудита сервера можно создать одну или несколько спецификаций аудита. Чтобы реализовать аудит действий на уровне экземпляра, создайте спецификацию аудита сервера, используя такой синтаксис:

```
CREATE SERVER AUDIT SPECIFICATION имя_спецификации_аудита
FOR SERVER AUDIT имя_аудита
```

```
{ { ADD ( { имя_группы_действий_аудита } ) } [, ...n]  
[ WITH ( STATE = { ON | OFF } ) ]}[ ; ]
```

Чтобы выполнить аудит событий в базе данных, создайте спецификацию аудита базы данных, используя такой синтаксис:

```
CREATE DATABASE AUDIT SPECIFICATION имя_спецификации_аудита  
{ [ FOR SERVER AUDIT имя_аудита ]  
  [ { ADD ( { <спецификация действия аудита> |  
    audit_action_group_name } )  
    } [, ...n] ]  
  [ WITH ( STATE = { ON | OFF } ) ]}[ ; ]
```

<спецификация действия аудита>::=

```
{action [ ,...n ]ON [ class :: ] securable BY principal [ ,...n ]}
```

ПРИМЕЧАНИЕ События аудита

Подробнее о списке классов и групп событий, аудит которых поддерживается см. статью «SQL Server Audit Action Groups and Actions» (Действия и группы действий аудита SQL Server) по адресу <http://technet.microsoft.com/en-us/library/cc280663.aspx>.

Аудит C2

Аудит C2 представляет собой спецификацию аудита Министерства обороны США. Эту спецификацию включают следующим кодом:

```
sp_configure 'c2 audit mode', 1
```

На смену аудиту C2 пришла спецификация Common Criteria, разработанная специалистами Европейского Союза. Независимо от выбора спецификации — C2 или Common Criteria, с точки зрения SQL Server, результат аудита в сущности одинаков. Необходимо выполнять аудит каждой успешной и неуспешной попытки доступа к объектам базы данных.

Когда включен аудит C2, файл журнала аудита записывается в каталог данных по умолчанию, при этом размер файлов продолжения равен 200 Мбайт. SQL Server создает файлы продолжения, пока не закончится свободное место на диске, что приведет к прекращению работы экземпляра. При включении аудита C2 записи аудита должны фиксироваться в журнале. Если система слишком занята, запросы пользователей будут отклоняться, чтобы освободить ресурсы для записи журнала аудита.

ВНИМАНИЕ! Влияние аудита

При внедрении аудита C2 нужно быть очень осмотрительным. Убедитесь в том, что более низкие уровни аудита не отвечают вашим требованиям. Включение аудита C2 означает, что аудит более важен, чем транзакции. Если система слишком занята, SQL Server будет прерывать пользовательские транзакции, чтобы записать данные в журнал аудита.

Контрольные вопросы

1. Чем можно воспользоваться для аудита и запрета большинства изменений объектов?
2. Какой объект необходимо создать перед спецификацией аудита сервера или базы данных?

Ответы на контрольные вопросы

1. Триггеры DDL позволяют выполнять аудит любой команды DDL. Если команда DDL выполняется в рамках транзакции, ее можно откатить с помощью триггера DDL и предотвратить изменение объекта.
2. До создания спецификации аудита сервера или базы данных необходимо создать объект аудита сервера.

Практикум. Создание спецификации аудита базы данных

Вы создадите спецификацию аудита базы данных для аудита инструкций *SELECT*, *INSERT*, *UPDATE* и *DELETE*, выполняемых пользователем с разрешением *db_owner* над конфиденциальными данными из таблицы, содержащей сведения о выплате зарплаты.

1. Выполните следующий код, чтобы создать объект аудита сервера:

```
USE MASTER
GO
CREATE SERVER AUDIT RestrictedAccessAudit
    TO APPLICATION_LOG
    WITH ( QUEUE_DELAY = 1000, ON_FAILURE = CONTINUE);
GO
```

2. Выполните следующий код, чтобы создать спецификацию аудита базы данных:

```
USE AdventureWorks
GO
CREATE DATABASE AUDIT SPECIFICATION EmployeePayrollAccess
FOR SERVER AUDIT RestrictedAccessAudit
    ADD (SELECT, INSERT, UPDATE, DELETE
        ON HumanResources.EmployeePayHistory
        BY dbo)
    WITH (STATE = ON);
GO
```

3. Включите аудит таким кодом:

```
USE MASTER
GO
ALTER SERVER AUDIT RestrictedAccessAudit
    WITH (STATE = ON);
GO
```

4. Разверните узел *Security* в Обозревателе объектов (Object Explorer) и найдите объект аудита сервера с именем *RestrictedAccessAudit* в узле *Audits*.
5. В Обозревателе объектов разверните узел **Безопасность** (Security), а затем узел — **Спецификации аудита базы данных** (Database Audit Specifications) в узле базы данных *AdventureWorks* и просмотрите свойства спецификации аудита базы данных с именем *EmployeePayrollAccess*, которую вы создали.
6. Выполните следующий код, чтобы протестировать аудит базы данных.

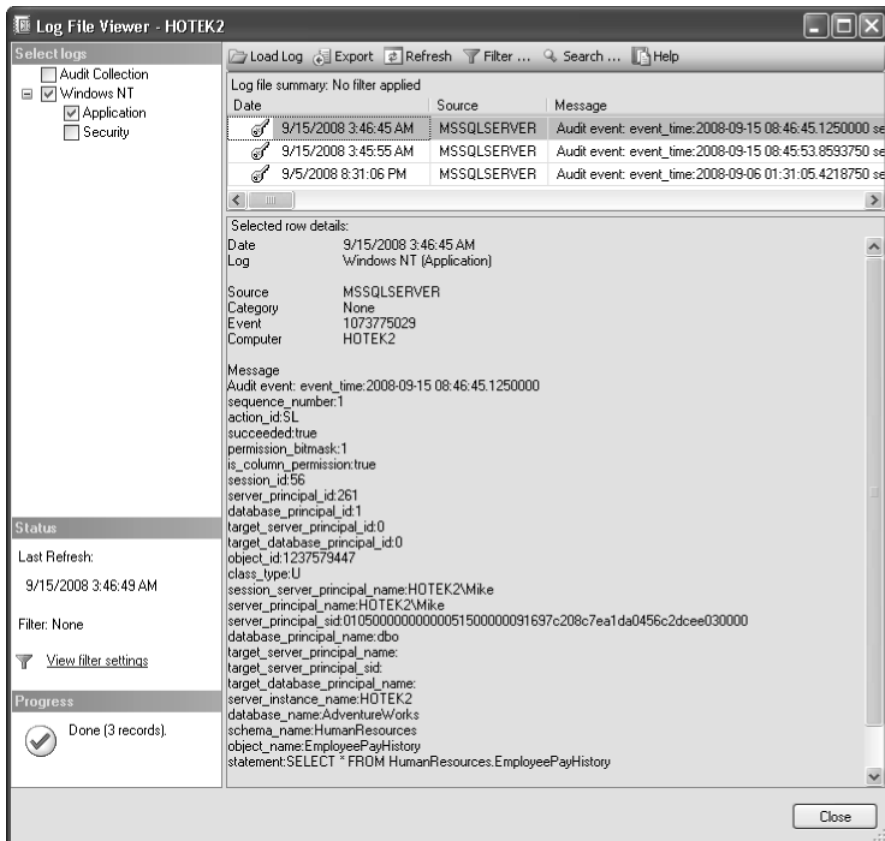
USE AdventureWorks

GO

```
SELECT * FROM HumanResources.EmployeePayHistory
```

GO

- Щелкните правой кнопкой объект аудита сервера и выберите команду **Просмотреть журналы** (View Audit Logs), чтобы просмотреть результаты аудита, как показано на рисунке.



8. Отключите аудит сервера, выполнив такой код:

USE MASTER

GO

```
ALTER SERVER AUDIT RestrictedAccessAudit
```

```
WITH (STATE = OFF);
```

GO

9. Просмотрите журнал аудита и обратите внимание, что отключение аудита также зафиксировано в журнале.

Резюме занятия

- Триггеры DDL запускаются при выполнении заданных событий или групп событий DDL.
- Если событие DDL выполняется в контексте транзакции, можно воспользоваться триггером DDL, чтобы отменить соответствующее действие.
- Команда *CREATE SERVER AUDIT* служит для создания экземпляра объекта аудита.
- После создания объекта аудита к нему можно добавить спецификации аудита сервера или базы данных для централизованного управления аудитом.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Руководитель отдела кадров компании Contoso потребовал, чтобы доступ к информации о зарплатах сотрудников был только у полномочных пользователей. Что нужно создать, чтобы выполнить эти требования по аудиту?
 - A. Спецификацию аудита базы данных.
 - B. Триггер DDL.
 - C. Триггер DML.
 - D. Спецификацию аудита сервера.
2. Администраторы базы данных в компании Fabrikam реализовали доставку журналов в базе данных *Orders*. Для бесперебойной работы доставки журналов необходимо, чтобы никто не смог изменить модель восстановления базы данных на простую. Что нужно применить для выполнения этой задачи?
 - A. Триггер DDL.
 - B. Триггер DML.
 - C. Невозможно защитить модель восстановления от изменений.
 - D. Спецификацию аудита сервера.

Занятие 6. Шифрование данных

Данные, которые должны оставаться конфиденциальными и закрытыми даже для пользователей с разрешением *SELECT* на таблицу, следует зашифровать. В этом занятии вы узнаете об инфраструктуре шифрования SQL Server 2008 и о применении шифрования данных.

Изучив материал этого занятия, вы сможете:

- ✓ шифровать данные с применением алгоритма хеширования;
- ✓ шифровать данные с применением симметричных ключей;
- ✓ шифровать данные с применением асимметричных ключей или сертификатов;
- ✓ включать прозрачное шифрование базы данных.

Продолжительность занятия — 30 минут.

Шифрование данных

Данные базы, которые должны оставаться конфиденциальными (например, номера кредитных карт), следует зашифровать. После зашифровки данных, их нельзя считать без соответствующих учетных данных. Кроме того, зашифрованные столбцы не могут применяться в качестве аргумента поиска или столбца индекса, поскольку это сделало бы шифрование данных бесполезным.

Столбцы можно зашифровать с применением хеша, парольной фразы, сертификата, симметричного или асимметричного ключа. Как правило, используются симметричные ключи, поскольку они обеспечивают оптимальный баланс между безопасностью данных и производительностью. Асимметричные ключи и сертификаты лежат в основе наиболее мощного метода шифрования и расшифровки.

Запрет доступа к объектам и данным

Задача защиты базы данных вовсе не в закрытии доступа к объектам. Если информация обладает достаточной ценностью, а у злоумышленника есть время, он обязательно до нее доберется. Цель защиты — усложнить проникновение в систему настолько, чтобы попытки взлома не оправдывали себя.

Кроме того, администраторы не просто так контролируют всю систему — они управляют ею. Разрешения пользователя с административным доступом не проверяются, поэтому действия администратора ограничить невозможно. У администратора есть доступ к внутренним структурам, что позволяет ему получить любые имеющиеся ключи шифрования. Поэтому доступ администратору закрыть невозможно. В SQL Server реализована мощная многоуровневая инфраструктура безопасности; это не система управления цифровыми правами.

Защита объектов также сужает круг функциональных возможностей, доступных для этих объектов. В частности, невозможно индексирование зашифрованного столбца и поиск по нему. Я видел одну глупейшую статью, в которой объяснялось, как можно искать по зашифрованному столбцу. Для этого предлагалось поместить столбец в таблицу, содержащую незашифрованные данные, чтобы по нему мог выполняться поиск. При этом шифрованный столбец также сохранялся. Если вы планируете хранить данные в незашифрованном формате, бессмысленно вообще шифровать те же данные, особенно в той же таблице. Кроме того, если вы разрешите пользователю искать по зашифрованным данным, вы от-

крываете злоумышленнику возможность с помощью простых атак перебором по словарю реконструировать зашифрованную информацию.

Алгоритмы хеширования

Алгоритмы шифрования бывают односторонними и двусторонними. Двусторонние алгоритмы позволяют зашифровывать и расшифровывать данные. Односторонние алгоритмы служат только для шифрования данных без возможности их расшифровки. Алгоритм хеширования относится к односторонним алгоритмам и поддерживает лишь шифрование данных.

ВНИМАНИЕ! Передача и хранение паролей

Существует распространенное заблуждение о том, что пароли передаются в SQL Server в виде открытого текста и что SQL Server расшифровывает сохраненный пароль, чтобы проверить его совпадение с полученным. Для обработки паролей в SQL Server применяется хеш MD5. Когда для объекта задается пароль, SQL Server вычисляет хеш MD5 пароля и сохраняет именно его. При указании пароля для доступа к объекту пароль хешируется с помощью того же алгоритма MD5, хешированный пароль передается по защищенному каналу и сравнивается с сохраненным значением хеша. Даже администратор, выполняющий трассировку, не может получить доступ к этому хешу.

В SQL Server поддерживает 5 алгоритмов хеширования — SHA, SHA1, MD2, MD4 и MD5. Лучше всего выбрать алгоритм MD5, поскольку он обеспечивает более надежное шифрование по сравнению с остальными алгоритмами. Алгоритмы хеширования также не зависят от платформы. Можно создать хеш в PHP-системе в Linux и получить то же значение, как если бы это значение было хешировано в SQL Server, при условии, что использован одинаковый алгоритм.

Алгоритмы хеширования уязвимы для атак методом подбора. Если диапазон шифруемых значений невелик, атакующий может с легкостью генерировать все возможные хеши для диапазона возможных значений. После того как хеши сгенерированы, злоумышленник должен сравнить значения хеша, чтобы найти совпадения, и таким образом реконструировать данные. Поэтому для шифрования таких данных, как даты рождения, уровень зарплаты и номера кредитных карт, лучше хеширование не использовать.

Хеширование с рассеиванием

Если диапазон возможных значений невелик, система взламывается методом прямого подбора. Однако можно сильно усложнить задачу злоумышленника, используя прием шифрования, называемый *рассеивание* (salting).

Специальная строка рассеивания (salt) состоит из одного или нескольких символов, добавляемых к значению перед хешированием. Добавление строки рассеивания, включающей хотя бы один символ, уже защитит от большинства атак и усложнит задачу злоумышленникам. Например, если в конце значения зарплаты добавить ноль, это не приведет к усложнению задачи злоумышленника и не защитит от атак методом подбора, но использование буквы сильно затруднит подбор. Даже если злоумышленник узнает о том, что добавлена буква, в английском языке это создает 52 возможных варианта (с учетом заглавных и

строчных букв). Кроме того, букву можно добавить в начале или в конце значения, что создает 104 возможных комбинации для каждого возможного значения зарплаты. А если предположить, что букву можно вставить в любое место значения зарплаты, диапазон возможных значений для каждой зарплаты будет настолько велик, что для любого злоумышленника затрачиваемые усилия не будут оправданы.

Симметричные ключи

В шифровании *симметричными ключами* один ключ применяется как для шифрования, так и для расшифровки. Поскольку для шифрования и расшифровки нужен всего один ключ, шифрование с симметричным ключом менее надежно по сравнению с шифрованием асимметричным ключом или с помощью сертификата. Тем не менее, симметричные ключи обеспечивают наилучшую производительность при ежедневной работе с зашифрованными данными.

Сертификаты и асимметричные ключи

Сертификаты и асимметричные ключи основаны на стандарте X.509 и применяются сходным образом. *Асимметричные ключи* создаются сервером ключей в организации. Нельзя создать их резервную копию или переместить между системами. Сертификаты можно копировать и восстанавливать из резервных копий, что позволяет перемещать зашифрованные базы данных и воссоздавать сертификат для доступа к данным.

Прозрачное шифрование данных

Парольные фразы и ключи шифрования могут применяться в приложениях для произвольного шифрования данных. Однако чтобы использовать эти данные, потребуется применить специальные процедуры для их расшифровки. Хотя возможно и выборочное шифрование данных, шифрование всего содержимого базы данных, как правило, невозможно.

Если данные не зашифрованы, злоумышленник может считать их напрямую из файлов базы данных на диске. Информацию считать непросто, но все равно данные хранятся на диске в виде открытого текста и их можно просмотреть в любом текстовом редакторе.

Чтобы предотвратить кражу данных, хранящихся на диске или в резервной копии, в SQL Server 2008 предусмотрено прозрачное шифрование данных (Transparent Data Encryption, TDE). TDE обеспечивает шифрование и расшифровку в режиме реального времени. Таким образом, данные в файлах и резервных копиях зашифрованы. Кроме того, SQL Server прозрачно шифрует и расшифровывает данные, поэтому чтобы воспользоваться преимуществами шифрования, больше не нужно вносить изменения в код приложений.

Практический совет

Майк Хотек

В последние годы участились сообщения о кражах данных из разных организаций. Но такова уж человеческая натура: каждый уверен, что с ним-то такое не случится.

Недавно я работал с крупным банком, в котором бились над повышением уровня безопасности. Они шифровали секретные данные в столбцах. Однако сторонняя аудиторская фирма выявила, что слабое место в системе безопасности — резервные копии, хотя конфиденциальные данные были зашифрованы. В базе данных все еще оставались конфиденциальные данные, а структура таблицы была достаточно информативной и облегчала злоумышленникам их работу. Руководство банка приняло решение о шифровании всего содержимого резервных копий.

Через три месяца поисков они выбрали четырех поставщиков, предлагавших решения, отвечающие их требованиям. К сожалению, лишь один поставщик мог зашифровать данные перед их записью на диск, при этом резервные копии оставались уязвимыми. Кроме того, руководство полагало, что для покупки необходимых лицензий потребуются очень большие вложения. Поскольку лицензии на SQL Server, приобретенные компанией, были включены в программу Software Assurance, мы обновили базу данных до SQL Server 2008 за выходные и настроили прозрачное шифрование данных. Теперь шифровались не только резервные копии, но и все данные и файлы журналов.

Компания сэкономила значительные средства. Спустя три недели после настройки прозрачного шифрования данных исчезла лента с резервной копией. В итоге ее нашли — оказалось, что ее просто неправильно маркировали. Но даже если бы произошла реальная кража данных, компания была бы защищена.

При выполнении прозрачного шифрования данных используется ключ шифрования, хранящийся в загрузочной записи базы данных. Ключ TDE шифруется с помощью сертификата в главной базе данных. Если злоумышленник похитит данные, файлы журнала или (что вероятнее) резервную копию базы данных, он не сможет получить доступ к ее содержимому без сертификата, хранящегося в главной базе данных.

Прозрачное шифрование данных в базе данных внедряется в следующем порядке:

1. Создайте главный ключ базы данных в главной базе данных.
2. Создайте сертификат в главной базе данных.
3. Создайте ключ шифрования базы данных в целевой базе данных с помощью сертификата главной базы данных.
4. Измените базу данных и активируйте шифрование.

ПРИМЕЧАНИЕ На экзамене

Необходимо создать резервную копию сертификата, используемого для прозрачного шифрования данных, и сохранить ее в надежном месте. После шифрования вы не сможете получить доступ к данным без сертификата.

Управление ключами шифрования

В SQL Server поддерживаются различные методы шифрования, но активация и управление отдельными методами выполняется в рамках отдельных экзем-

пляров. В SQL Server 2008 реализована возможность интеграции с системами управления ключами предприятия с применением технологии расширенного управления ключами (Extensible Key Management, EKM). Ключи могут централизованно храниться в сети предприятия и при необходимости экспортироваться на SQL Server. После регистрации поставщика управления ключами в SQL Server на экземпляре могут использоваться все расширенные возможности аппаратных и программных решений для управления ключами, например смена ключей.

Контрольные вопросы

1. Какой объект необходим для реализации прозрачного шифрования данных?
2. Как можно повысить надежность шифрования, если диапазон хешируемых значений невелик?

Ответы на контрольные вопросы

1. Необходимо создать сертификат в главной базе данных, который необходим для шифрования ключа шифрования базы данных.
2. Если диапазон шифруемых значений невелик, необходимо перед созданием хеша добавить в данные строку рассеивания. Это позволит защититься от атак методом подбора.

Практикум. Шифрование данных

Вы воспользуетесь различными ключами шифрования для шифрования данных приложения, а также настроите прозрачное шифрование данных в базе данных *Adventure Works*.

Задание 1. Хеширование данных

Вы зашифруете данные с применением алгоритма хеширования.

1. Выполните следующий код и сравните результаты для разных алгоритмов хеширования:

```
DECLARE @Hash varchar(100)
SELECT @Hash = 'Шифруемый_текст'
SELECT HashBytes('MD5', @Hash)
SELECT @Hash = 'Шифруемый_текст'
SELECT HashBytes('SHA', @Hash)
```

2. Выполните следующий код и обратите внимание, что алгоритм хеширования чувствителен к регистру:

```
DECLARE @Hash varchar(100)
SELECT @Hash = 'шифруемый_текст'
SELECT HashBytes('SHA1', @Hash)
SELECT @Hash = 'ШИФРУЕМЫЙ_ТЕКСТ'
SELECT HashBytes('SHA1', @Hash)
```

Задание 2. Шифрование данных с помощью парольной фразы

Вы зашифруете данные с применением парольной фразы.

1. Выполните следующий код и оцените результаты шифрования с помощью парольной фразы.

```
DECLARE @EncryptedText VARBINARY(80)
SELECT @EncryptedText =
    EncryptByPassphrase('<Введите_надежный_пароль>', 'Шифруемый_текст')
SELECT @EncryptedText,
    CAST(DecryptByPassPhrase('<Введите_надежный_пароль>', @EncryptedText)
    AS VARCHAR(MAX))
```

Задание 3. Шифрование данных с помощью симметричного ключа

Вы создадите симметричный ключ для шифрования данных.

1. Выполните следующий код в базе данных *AdventureWorks*, чтобы создать симметричный ключ:

```
CREATE SYMMETRIC KEY TestSymmetricKey WITH ALGORITHM = RC4
    ENCRYPTION BY PASSWORD = '<Введите_надежный_пароль>'

SELECT * FROM sys.symmetric_keys
```

2. Выполните следующий код, чтобы открыть симметричный ключ:

```
OPEN SYMMETRIC KEY TestSymmetricKey
    DECRYPTION BY PASSWORD = '<Введите_надежный_пароль>'
```

3. Выполните следующий код, чтобы просмотреть данные, зашифрованные с помощью симметричного ключа:

```
DECLARE @EncryptedText VARBINARY(80)
SELECT @EncryptedText =
    EncryptByKey(Key_GUID('TestSymmetricKey'), 'Шифруемый текст')
SELECT @EncryptedText, CAST(DecryptByKey(@EncryptedText) AS VARCHAR(30))
```

4. Закройте симметричный ключ, выполнив код:

```
CLOSE SYMMETRIC KEY TestSymmetricKey
GO
```

Задание 4. Шифрование данных с помощью сертификата

Вы создадите сертификат и с его помощью зашифруете данные, чтобы пользователи, не имеющие разрешения на доступ к данным, не смогли их просмотреть.

1. Выполнив следующий код, создайте тестовую таблицу, двух пользователей и разрешения.

```
CREATE TABLE dbo.CertificateEncryption
(ID INT IDENTITY(1,1),
SalesRep VARCHAR(30) NOT NULL,
SalesLead VARBINARY(500) NOT NULL)
GO
```

```
CREATE USER SalesRep1 WITHOUT LOGIN
GO

CREATE USER SalesRep2 WITHOUT LOGIN
GO

GRANT SELECT, INSERT ON dbo.CertificateEncryption TO SalesRep1
GRANT SELECT, INSERT ON dbo.CertificateEncryption TO SalesRep2
GO
```

2. Создайте сертификат для каждого пользователя:

```
CREATE CERTIFICATE SalesRep1Cert AUTHORIZATION SalesRep1
    WITH SUBJECT = 'SalesRep 1 certificate'
GO

CREATE CERTIFICATE SalesRep2Cert AUTHORIZATION SalesRep2
    WITH SUBJECT = 'SalesRep 2 certificate'
GO

SELECT * FROM sys.certificates
GO
```

3. Вставьте данные для каждого пользователя:

```
EXECUTE AS USER='SalesRep1'
GO
INSERT INTO dbo.CertificateEncryption
(SalesRep, SalesLead)
VALUES('SalesRep1',EncryptByCert(Cert_ID('SalesRep1Cert'), 'Fabrikam'))
REVERT
GO

EXECUTE AS USER='SalesRep2'
GO
INSERT INTO dbo.CertificateEncryption
(SalesRep, SalesLead)
VALUES('SalesRep2',EncryptByCert(Cert_ID('SalesRep2Cert'), 'Contoso'))
REVERT
GO
```

4. Просмотрите содержимое таблицы за каждого пользователя:

```
SELECT ID, SalesRep, SalesLead
FROM dbo.CertificateEncryption
GO

EXECUTE AS USER='SalesRep1'
GO
SELECT ID, SalesRep, SalesLead,
    CAST(DecryptByCert(Cert_Id('SalesRep1Cert'), SalesLead)
    AS VARCHAR(MAX))
FROM dbo.CertificateEncryption
REVERT
GO
```

```
EXECUTE AS USER='SalesRep2'  
GO  
SELECT ID, SalesRep, SalesLead,  
       CAST(DecryptByCert(Cert_Id('SalesRep2Cert'), SalesLead)  
            AS VARCHAR(MAX))  
FROM dbo.CertificateEncryption  
REVERT  
GO
```

Задание 5. Реализация прозрачного шифрования

Вы настроите прозрачное шифрование в базе данных *Adventure Works*.

1. Создайте главный ключ и сертификат в главной базе данных:

```
USE master  
GO  
CREATE MASTER KEY ENCRYPTION BY PASSWORD = '<Введите_здесь_надежный_пароль>'  
GO  
CREATE CERTIFICATE ServerCert WITH SUBJECT = 'My Server Cert for TDE'  
GO
```

2. Создайте резервную копию сертификата и закрытого ключа, чтобы гарантировать возможность восстановления:

```
BACKUP CERTIFICATE ServerCert TO FILE = 'C:\Program Files\Microsoft  
SQL Server\MSSQL10.MSSQLSERVER\MSSQL\Backup\servercert.cer'  
WITH PRIVATE KEY (FILE = 'C:\Program Files\Microsoft SQL Server\  
MSSQL10.MSSQLSERVER\MSSQL\Backup\servercert.key',  
                  ENCRYPTION BY PASSWORD = '<Введите_здесь_надежный_пароль>')
```

3. Создайте ключ шифрования базы данных *Adventure Works*:

```
USE AdventureWorks  
GO  
  
CREATE DATABASE ENCRYPTION KEY  
WITH ALGORITHM = AES_128  
ENCRYPTION BY SERVER CERTIFICATE ServerCert  
GO
```

4. Включите шифрование в базе данных *Adventure Works*:

```
ALTER DATABASE AdventureWorks  
SET ENCRYPTION ON  
GO
```

Резюме занятия

- Данные в таблице можно зашифровать с помощью алгоритма хеширования, парольной фразы, симметричного ключа, асимметричного ключа или сертификата.
- Если диапазон шифруемых данных недостаточно широк, чтобы предотвратить атаки методом подбора, алгоритм хеширования следует использовать вместе со строкой рассеивания.

- Прозрачное шифрование данных применяется к хранящимся данным. Данные, а также содержимое журнала транзакций и всех резервных копий шифруется ядром СУБД.
- Внедряя прозрачное шифрование данных, не забудьте убедиться, что у вас есть резервная копия сертификата и закрытый ключ. Иначе вы не сможете восстановить резервную копию.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Администраторы базы данных в банке Woodgrove управляют несколькими базами данных, содержащими конфиденциальную информацию о кредитных картах и клиентах. Необходимо зашифровать все содержимое этих баз данных, чтобы злоумышленники не смогли считать информацию с диска. Как это сделать с минимальными усилиями?
 - A. Создать сертификат в базе данных, который будет использоваться для шифрования данных.
 - B. Создать ключ шифрования базы данных и включить шифрование базы данных.
 - C. Создать симметричный ключ в базе данных, который будет использоваться для шифрования данных.
 - D. Создать асимметричный ключ в базе данных, который будет использоваться для шифрования данных.
2. Администраторы базы данных в банке Woodgrove управляют несколькими базами данных, содержащими конфиденциальную информацию о кредитных картах и клиентах. Из-за недавних сообщений о краже данных в других банках руководство банка требует зашифровать все данные в резервных копиях. Как выполнить это требование, не внося изменений в приложения?
 - A. Создать сертификат в базе данных, который будет использоваться для шифрования данных.
 - B. Создать симметричный ключ в базе данных, который будет использоваться для шифрования данных.
 - C. Создать ключ шифрования базы данных и включить шифрование базы данных.
 - D. Создать асимметричный ключ в базе данных, который будет использоваться для шифрования данных.

Закрепление материала главы

Для закрепления знаний и навыков, полученных при изучении этой главы, выполните следующее:

- изучите резюме главы;
- изучите список основных терминов из этой главы;

- выполните задания по темам этой главы;
- сделайте предложенные упражнения;
- пройдите пробный экзамен.

Резюме главы

- Конечные точки представляют первый уровень безопасности в SQL Server. Играя роль барьера, очень похожего на брандмауэр, конечные точки обеспечивают доступ к экземпляру SQL Server только через разрешенные соединения и только для трафика разрешенного типа.
- Конечные точки можно создавать для протокола TCP или HTTP. Конечные точки TCP поддерживают полезные данные типа *TSQL*, *DATABASE_MIRRORING* и *SERVICE_BROKER*. Конечные точки HTTP поддерживают полезные данные *SOAP*.
- Конечные точки HTTP позволяют предоставлять хранимые процедуры и функции в виде веб-служб. Таким образом, SQL Server фактически работает как зарегистрированная веб-служба.

Основные термины

Понятны ли вам значения приведенных здесь основных терминов? Чтобы проверить свои ответы, загляните в глоссарий на прилагаемом компакт-диске.

- Асимметричный ключ ~ asymmetric key.
- Сертификат ~ certificate.
- Спецификация аудита базы данных ~ database audit specification.
- Главный ключ базы данных ~ database master key.
- Триггер DDL ~ DDL trigger.
- Предопределенная роль базы данных ~ fixed database role.
- Предопределенная роль сервера ~ fixed server role.
- Алгоритм хеширования ~ hash algorithm.
- Олицетворение ~ impersonation.
- Пользователь без имени входа ~ loginless user.
- Цепочка владения ~ ownership chain.
- Участник ~ principal.
- Привязка ~ salting.
- Защищаемый объект ~ securable.
- Аудит сервера ~ server audit.
- Спецификация аудита сервера ~ server audit specification.
- Главный ключ службы ~ service master key.
- Подпись ~ signature.
- Симметричный ключ ~ symmetric key.
- Конечная точка TCP ~ TCP endpoint.

Упражнения

Сейчас вы примените на практике знания и навыки, полученные при изучении этой главы. Ответы на вопросы см. в разделе «Ответы» в конце книги.

Упражнение 1. Безопасность в компании Coho Vineyard

Исходные данные

Информация о компании

Компания Coho Vineyard основана в 1965 году как семейная винодельня. За последние десятилетия вина Coho Vineyards неоднократно отмечались премиями на выставках, благодаря чему предприятие существенно выросло. Сегодня компания состоит из 12 заводов штатах Калифорния и Вашингтон. В Coho Vineyard работает 400 сотрудников, 74 из них работают в центральном офисе, где расположены критически важные для бизнеса серверы.

Запланированные изменения

В 2008 году в Coho Vineyard наконец интегрировали все 12 виноделен, создав централизованную платформу баз данных, с которой работают различные веб-приложения. Результаты аудита показали, что в базах данных Coho Vineyard фактически не реализована никакая защита, а вся надежда только на правильность реализации приложений, которым предоставлен полный доступ к базам данных под учетной записью *sa*.

Учетная запись *sa* должна использоваться только в экстренных случаях членами группы администраторов базы данных. В любое другое время администраторы базы данных должны при входе на экземпляр использовать свои учетные данные Windows. Таблицы, содержащие информацию о клиентах (особенно номера кредитных карт), требуется зашифровать, чтобы пройти предстоящую аудиторскую проверку на предмет соблюдения Стандартов безопасности данных в сфере платежных карт (Payment Card Industry). Все действия, выполняемые учетной записью *sysadmin* или владельцем базы данных, должны подвергаться аудиту и регистрироваться в журнале событий приложений Windows.

Текущая среда данных

Базы данных

В табл. 11-3 показаны базы данных компании Coho Vineyard.

Табл. 11-4. Базы данных Coho Vineyard

База данных	Объем
<i>Customer</i> (данные о клиентах)	180 MB
<i>Accounting</i> (бухгалтерские данные)	500 MB
<i>HR</i> (кадровый учет)	100 MB
<i>Inventory</i> (складские запасы)	250 MB
<i>Promotions</i> (сведения о поощрениях)	80 MB

Серверы баз данных

Все базы данных в центральном офисе хранятся в SQL Server 2008 Enterprise на одном сервере DB1 под управлением Windows Server 2003 Enterprise.

Бизнес-требования

Общие требования

Веб-приложениям необходимо предоставить возможность считывания и записи данных в соответствующую базу данных. База данных *HR* должна быть доступна только сотрудникам отдела по управлению персоналом. Необходимо обеспечить аудит любых попыток доступа к базе данных *HR* со стороны пользователей с правами администратора.

Технические требования

Безопасность

Необходимо обеспечить шифрование всего входящего и исходящего трафика сервера DB1. Конфигурация SQL Server должна обеспечить минимальную уязвимость сервера для атак при условии соблюдения всех технических и бизнес-требований.

Необходимо обеспечить автоматическое обновление всех клиентских компьютеров в штаб-квартире средствами Центра обновления Microsoft.

1. Как настроить экземпляр, чтобы обеспечить к нему доступ для приложений, не предоставляя этим приложениям полномочий роли *sysadmin*?
2. Как обеспечить шифрование номеров кредитных карт в базе данных?
3. Как реализовать аудит доступа к базе данных *HR*?

Рекомендуемые упражнения

Чтобы подготовиться к экзамену по темам этой главы, выполните следующие задания.

Управление именами входа и ролями сервера

- **Задание 1.** Создайте имя входа SQL Server и имя входа Windows для экземпляра.
- **Задание 2.** Отключите имя входа, чтобы запретить доступ к экземпляру.

Управление пользователями и ролями базы данных

- **Задание 1.** Добавьте пользователя в базу данных, сопоставив ему имя входа.
- **Задание 2.** Создайте роль базы данных и добавьте в нее пользователя.

Управление разрешениями экземпляра SQL Server

- **Задание.** Добавьте имя входа к соответствующей предопределенной роли сервера, чтобы предоставить нужные разрешения.

Управление разрешениями базы данных

- **Задание 1.** Добавьте пользователя в предопределенную роль базы данных.
- **Задание 2.** Предоставьте разрешения на уровне базы данных и просмотрите результаты.

Управление разрешениями схемы и разрешениями объектов

- **Задание 1.** Предоставьте разрешения на уровне схемы и просмотрите результаты.
- **Задание 2.** Предоставьте разрешения на уровне объекта и просмотрите результаты.

Аудит экземпляров SQL Server

- **Задание 1.** Создайте объект аудита сервера.
- **Задание 2.** Добавьте спецификации аудита сервера и аудита базы данных в аудит сервера и протестируйте процесс аудита.

Управление прозрачным шифрованием данных

- **Задание 1.** Сконфигурируйте прозрачное шифрование данных в базе данных.
- **Задание 2.** Восстановите базу данных на другом экземпляре и попрактикуйтесь в восстановлении ключей шифрования, необходимых для доступа к резервной копии.

Пробный экзамен

На прилагаемом компакт-диске содержится пробный экзамен, обеспечивающий широкие возможности самоподготовки. Например, можно проверить, как усвоены темы этой главы либо все темы экзамена 70-432. Можно симитировать настоящий экзамен или выбрать режим обучения, когда после каждого вопроса приводятся правильные ответы с пояснениями.

ПРИМЕЧАНИЕ Пробный экзамен

Подробнее о пробном экзамене см. «Введение».

Мониторинг Microsoft SQL Server

Занятие 1. Использование Системного монитора	280
Занятие 2. Использование SQL Server Profiler	287
Занятие 3. Диагностика сбоев баз данных	303
Занятие 4. Диагностирование сбоев служб	310
Занятие 5. Диагностирование неполадок оборудования	320
Занятие 6. Устранение неполадок, связанных с кратковременной и обычной блокировкой	323

В совершенном мире установка Microsoft SQL Server и развертывание баз данных проходит без сучка и задоринки, приложения работают с оптимальной производительностью и ничто не беспокоит администратора. Но мы живем в реальном мире, где оборудование ломается, производительность приложений падает, транзакции блокируют друг друга, а внесение изменений в среду приводит к отказам. В этой главе вы узнаете, как наблюдать за средой SQL Server и диагностировать неполадки.

Темы экзамена

- Сбор данных производительности средствами Системного монитора.
- Сбор данных трассировки средствами приложения SQL Server Profiler.
- Обнаружение неполадок службы SQL Server.
- Обнаружение неполадок параллелизма.
- Поиск сведений об ошибках.

Прежде всего

Для изучения материалов этой главы необходимо иметь:

- установленный экземпляр Microsoft SQL Server 2008;
- подключенную базу данных AdventureWorks.

Практический совет*Майк Хотек*

За прошедшие годы платформы разработки и серверы баз данных стали сложнее, обогатились новой функциональностью и графическими интерфейсами. Предполагается, что графические интерфейсы позволяют быстрее вести разработку за счет изоляции разработчика от всех низкоуровневых деталей. У такой изоляции есть плюсы и минусы.

В дополнение ко всему в инструментальных средствах разработки есть отладчики, и слишком многие считают отладку единственным способом обнаружения и устранения неполадок кода. Все чаще разработчик оказывается неспособен обнаружить проблему, если отладчик не покажет именно ту строку программы, где произошла ошибка, а также точно не объяснит в чем она состоит.

Это создает серьезные сложности при работе в среде SQL Server. SQL Server самым разным образом взаимодействует со многими компонентами оборудования. У SQL Server также есть много внутренних компонентов, взаимодействие которых не отличается простотой. В эту сложную картину вы добавляете свои приложения со структурами данных, кодом базы данных и многими одновременно работающими пользователями. Один неудачный запрос, выполненный одним из пользователей, может создать проблемы для других пользователей, а неполадки могут распространиться на многие компоненты оборудования и сервера SQL Server. Администратору базы данных очень непросто выполнить мониторинг SQL Server, обнаружить проблемные области и внести корректирующие изменения до того, как проблемы скажутся на работе приложений. В тех случаях, когда администратору базы данных не удастся избавиться от неполадки, справиться с проблемой позволяет только тщательная диагностика, предполагающая знание всех подробностей низкоуровневого взаимодействия SQL Server и оборудования.

Занятие 1. Использование Системного монитора

Системный монитор — утилита Microsoft Windows, служащая для сбора статистической информации об аппаратной среде, операционной системе и всех приложениях, которые поддерживают свойства и счетчики. На этом занятии вы узнаете, как использовать Системный монитор для сбора показаний счетчиков в журналы, которые применяются для устранения неполадок системы и проблем с производительностью.

Изучив материал этого занятия, вы сможете:

- ✓ выбирать счетчики производительности;
- ✓ создавать журнал счетчика.

Продолжительность занятия — 20 минут.

Общие сведения о Системном мониторе

Системный монитор, или PerfMon, использует архитектуру опроса ресурсов для получения и записи числовых данных, предоставляемых приложениями. Приложения ответственны за обновление счетчиков, предоставленных для доступа Системному монитору. Администратор выбирает счетчики, данные которых нужно записывать, и интервал сбора данных. После этого Системный монитор, используя определение счетчиков и интервал опроса, собирает показания только нужных счетчиков и в заданном интервале.

ПРИМЕЧАНИЕ Как найти Системный монитор

Знакомство администратора с каждой новой версией Windows начинается с процедуры поиска знакомых средств. Они либо перенесены в другое место, либо их название изменено. Системный монитор в этом смысле не исключение. Все «айтишники», с которыми мне приходилось иметь дело, называют эту утилиту PerfMon, потому что она запускается путем открытия файла PerfMon.exe. В Windows XP и Windows Server 2003 она называлась Производительность (Performance), Системным монитором (Performance Monitor или System Monitor). В Windows Vista и Windows Server 2008 она была снова переименована, и Windows 7 не станет исключением. Независимо от того, как эта утилита называется в вашей версии Windows, ее официальное название «Системный монитор Microsoft», запускается она командой PerfMon.exe, и все ее называют просто — PerfMon.

Поскольку Системный монитор работает только с числовыми данными и при сборе данных никаких вычислений не выполняется, нагрузка, создаваемая Системным монитором на систему, незначительна и практически не зависит от числа записываемых счетчиков. Естественно желание сократить число записываемых счетчиков до минимума, чтобы не утонуть в данных, но системе совершенно безразлично, сколько этих счетчиков — сто или один, на производительности это никак не сказывается.

Счетчики объединены в иерархию, состоящую из трех уровней: объект производительности, счетчик и экземпляр. Объект — это компонент, приложение или подсистема в приложении. Например, «Процессор», «Сетевой интерфейс» и SQLServer:Databases — объекты. У объекта один или несколько счетчиков, причем у объекта должен быть как минимум один счетчик. Например, у объекта SQLServer:Databases есть счетчики активных транзакций, размера файла данных, числа транзакций в секунды и занятого пространства в журнале транзакций. У счетчика один, несколько или ни одного экземпляра. Например, у объекта Система (System) есть счетчик «Длина очереди процессора» (Processor Queue Length), у которого нет экземпляров, а у счетчика объекта SQLServer:Databases, показывающего занятое пространство в журнале транзакций — Используемый размер файлов журнала (Log File(s) Used Size), есть экземпляр для каждой базы данных, а также сводный экземпляр _Total.

Определяя счетчики, которые нужно записывать, можно задавать критерии на любом уровне иерархии. Если вы решите записывать данные всего объекта, Системный монитор соберет данные всех счетчиков выбранного объекта, а также всех экземпляров каждого счетчика. Если записывать все не надо, можете выбрать часть счетчиков объекта или подмножество экземпляров определенного счетчика.

ПРИМЕЧАНИЕ На экзамене

На экзамене вы должны знать, что представляет собой тот или иной счетчик производительности, а также уметь выбрать подходящий счетчик для диагностики неполадки экземпляра.

Запись журналов счетчиков

Большинство начинающих пользователей Системного монитора добавляют объекты и счетчики и просматривают результаты в графическом интерфейсе. Однако графический дисплей не позволяет сохранить информацию в файл для последующего анализа, а лишь предоставляет двухминутный снимок счетчиков в системе.

Чтобы записывать данные для анализа, нужно сконфигурировать журнал счетчиков, который выполняется в фоновом режиме, когда в систему никто не вошел. В разных ОС журналы счетчиков настраиваются в разных местах. На рис. 12-1 показано, как это выглядит в Windows XP и Windows Server 2003.

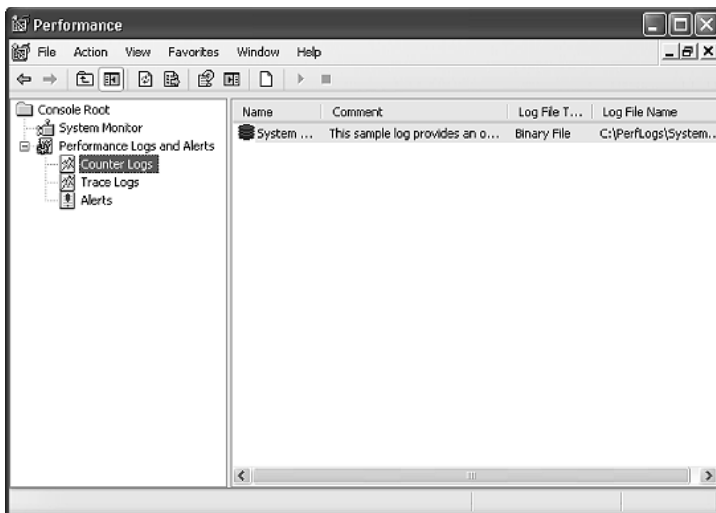


Рис. 12-1. Журналы счетчиков в Системном мониторе

В Системном Мониторе щелкните правой кнопкой **Журналы счетчиков** (Counter Logs), выберите **Новые параметры журнала** (New Log Settings) и присвойте журналу счетчиков имя — откроется окно определения журнала (рис. 12-2).

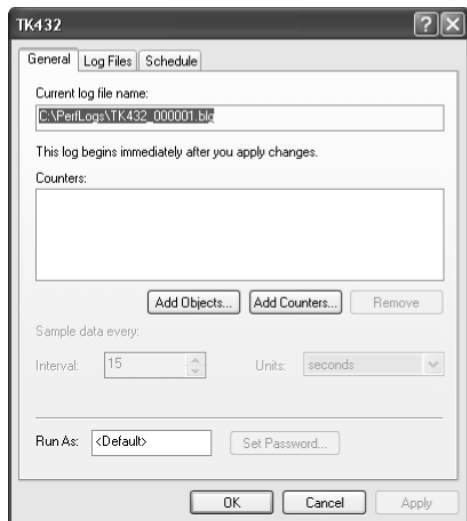


Рис. 12-2. Определение свойств журнала счетчиков

Щелчок **Добавить объекты** (Add Objects) позволяет задать объекты, информацию которых нужно записывать. Щелкните **Добавить счетчики** (Add Counters), чтобы добавить отдельные счетчики объекта, а также отдельные экземпляры счетчиков.

ПРИМЕЧАНИЕ Запись журналов счетчиков

Просмотр счетчиков в графическом интерфейсе Системного монитора полезен только для изучения текущего состояния системы. Намного полезнее записывать показания счетчиков в журнал, который можно использовать для анализа в дальнейшем. При настройке журналов счетчиков, рекомендуется выбирать объекты, а не отдельные счетчики, чтобы записать все необходимое для анализа.

Интервал сбора данных определяет, как часто Windows фиксирует показания счетчиков, указанных в журнале. По умолчанию показания записываются каждые 15 секунд. Это самый популярный интервал для стандартного анализа, определения отправной точки и выявления долгосрочных тенденций. Если нужно проанализировать повторяющуюся неполадку, но имеющую очень короткую продолжительность, разумно уменьшить интервал сбора данных.

ПРИМЕЧАНИЕ Рекомендации по определению учетной записи журнала счетчиков

В нижней части окна определения журнала счетчиков можно определить данные учетной записи, под которой будет записываться журнал счетчиков. Учетную запись надо задавать так, чтобы журнал счетчиков записывался под учетной записью, обладающей достаточными разрешениями для доступа к приложениям, счетчики которых будут записываться. Неправильное определение учетной записи — самая популярная причина неработоспособности журналов счетчиков. Другие популярные причины этой неполадки — устаревание пароля, блокировка или отключение учетной записи.

На вкладке **Файлы журнала** (Log Files) (рис. 12-3) можно задавать название и формат журнала счетчиков.

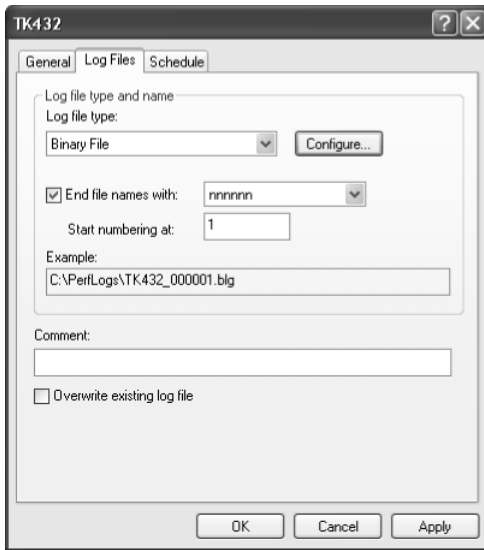


Рис. 12-3. Определение свойств журнала

Счетчики производительности

В принципе, можно записывать показания тысяч счетчиков и десятков тысяч экземпляров, но для диагностики подавляющего большинства неполадок достаточно небольшого числа стандартных счетчиков.

Обычно для диагностики используют несколько счетчиков и информацию о среде. Отдельные счетчики и их группы указывают на часть системы, которая нуждается в дальнейшем исследовании, но, как правило, не указывает прямо на причину неполадки. Однако есть три счетчика, которые могут точно указывать на неполадку:

- Система/Длина очереди процессора (System/Processor Queue Length).
- Сетевой интерфейс/ Длина очереди вывода (Network Interface/Output Queue Length).
- Физический диск/Средняя длина очереди диска (Physical Disk/Avg. Disk Queue Length).

Когда процессор, сетевой интерфейс или диск перегружен, процессам приходится ждать освобождения ресурсов. Каждый ожидающий ресурс поток увеличивает значение соответствующего счетчика длины очереди. Например, значение 8 счетчика «Длина очереди процессора» указывает, что на компьютере не хватает процессорных мощностей и что 8 запросов ожидают в очереди освобождения ядра процессора. Длина очереди может быть не равна нулю очень недолго, а постоянная ненулевая длина очереди указывает на узкое место в системе, сказывающееся на производительности приложений.

ПРИМЕЧАНИЕ Диагностика неполадок с помощью счетчиков производительности

В занятиях 3, 4 и 5 вы найдете информацию о десятках счетчиков, которые используются для диагностики неполадок SQL Server, оборудования и Windows.

Контрольные вопросы

1. Для каких элементов можно записывать данные средствами Системного монитора?
2. Какие типы данных собирает Системный монитор?
3. Какие три счетчика способны прямо указывать на неполадки в системе?

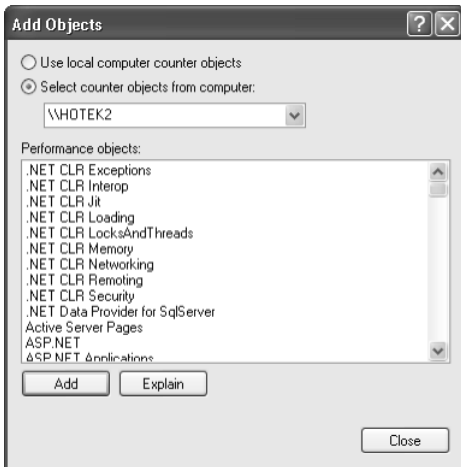
Ответы на контрольные вопросы

1. Записывать данные можно для объектов, счетчиков и экземпляров.
2. Системный монитор собирает числовые данные счетчиков производительности, определенных у аппаратных или программных компонентов.
- 3 Это счетчики: Система/Длина очереди процессора (System/Processor Queue Length), Сетевой интерфейс/ Длина очереди вывода (Network Interface/Output Queue Length), Физический диск/Средняя длина очереди диска (Physical Disk/Avg. Disk Queue Length).

Практикум. Создание журнала счетчиков

Вы создадите журнал счетчиков для определения базовых показателей и диагностики различных неполадок системы.

1. Откройте **Системный монитор** (System Monitor) и создайте новый журнал счетчиков.
2. На вкладке **Общие** (General) окна свойств нового журнала счетчиков щелкните **Добавить объекты** (Add Objects). Откроется окно **Добавить объекты** (Add Objects).

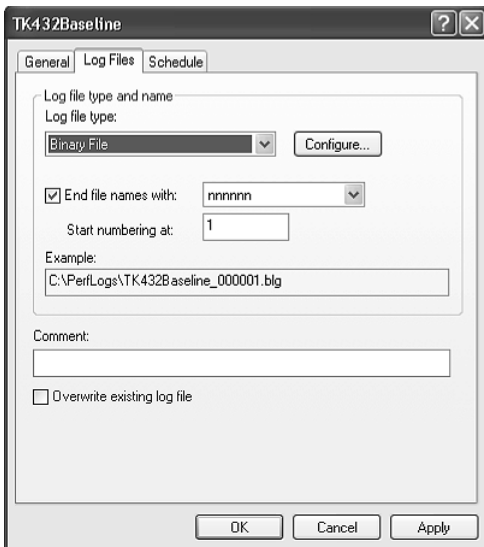


3. Добавьте следующие объекты: Память, Сетевой интерфейс, Физический диск (PhysicalDisk), Процессор, SQLServer:BufferManager, SQLServer:Databases, SQLServer:Exec Statistics, SQLServer:General Statistics, SQLServer:Latches,

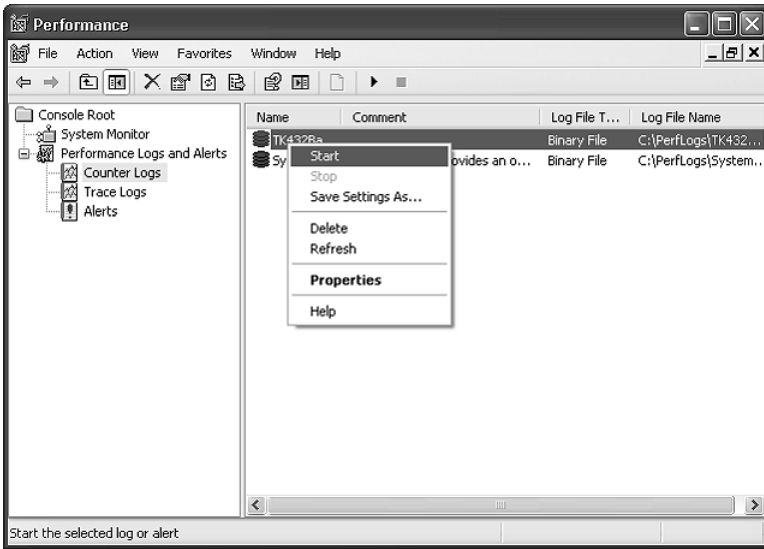
SQLServer:Locks, SQLServer:Memory Manager, SQLServer:PlanCache, SQL-Server:SQL Statistics и Система. Щелкните **Заккрыть** (Close).



4. Задайте интервал 15 секунд, а также учетную запись, под которой будет заполняться журнал счетчиков.
5. Щелкните вкладку **Файлы журнала** (Log Files) и просмотрите настройку по умолчанию.



6. Щелкните **ОК**, чтобы сохранить журнал счетчиков. Запустите журнал счетчиков, щелкнув правой кнопкой журнал и выбрав **Запуск** (Start).



Резюме занятия

- Системный монитор используется для записи числовой статистики работы аппаратных и программных компонентов.
- Счетчики объединены в иерархию, состоящую из трех уровней: объект производительности, счетчик и экземпляр.
- Объект должен содержать как минимум один счетчик.
- Счетчиков может быть несколько, один или ни одного экземпляра.
- В Системном мониторе можно записывать журнал счетчиков для последующего анализа.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Что измеряет счетчик **Система/Длина очереди процессора** (System/Processor Queue Length)?
 - A. Число системных запросов, ожидающих освобождения процессора.
 - B. Число запросов SQL Server, ожидающих освобождения процессора.
 - C. Число активно работающих процессоров.
 - D. Время работы процессора.

Занятие 2. Использование SQL Server Profiler

SQL Server базируется на подсистеме событий SQL Trace, доступной для администраторов, которые могут использовать ее для записи информации, связанной с событиями (их более 200), которые могут произойти в экземпляре. SQL Server

Profiler — графический инструмент, предоставляющий самый общий интерфейс для взаимодействия с подсистемой SQL Trace. Полученные данные можно использовать для мониторинга экземпляра на предмет ошибок и конкуренции за ресурсы. Можно также воспользоваться приложением SQL Server Profiler для записи данных, которые используются для оптимизации производительности запросов. В этом занятии рассказывается, как создавать файлы трассировки приложения SQL Server Profiler, которые можно применять для диагностики неполадок и повышения производительности и стабильности приложений.

Изучив материал этого занятия, вы сможете:

- ✓ создавать файл трассировки приложения SQL Server Profiler;
- ✓ выбирать события для трассировки;
- ✓ фильтровать данные трассировки.

Продолжительность занятия — 20 минут.

Определение трассировки

Трассировку можно определить, используя Transact-SQL (T-SQL), но чаще для этого используют приложение SQL Server Profiler. Приложение SQL Server Profiler можно запустить, используя подменю **Средства обеспечения производительности** (Performance Tools) в меню SQL Server 2008. После запуска приложения SQL Server Profiler выберите File/New Trace и подключитесь к экземпляру, чтобы начать настройку трассировки (рис. 12-4).

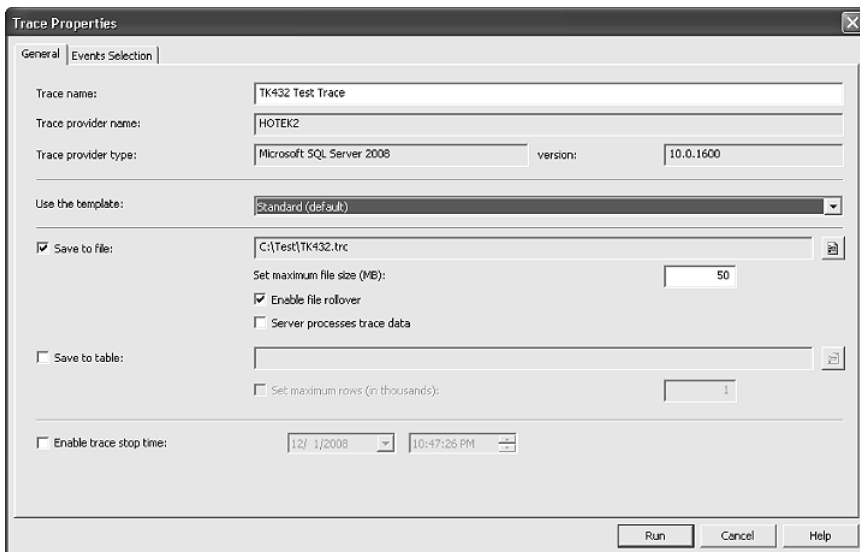


Рис. 12-4. Создание новой трассировки

Далее можно задать несколько общих свойств трассировки, таких как имя, шаблона, время остановки, а также флажок, указывающий, нужно ли сохранять данные трассировки. У каждой трассировки должно быть имя и как минимум одно событие.

В составе SQL Server Profiler поставляется несколько шаблонов, в которых уже определены события, столбцы данных и фильтры. Их можно использовать как отправную точку. В таблице 12-1 перечислены шаблоны и их обычное назначение.

Табл. 12-1. Шаблон трассировки SQL Server Profiler

Шаблон	Назначение
Пустой шаблон	Служит для создания трассировки с нуля
SP_Counts	Собирает информацию обо всех хранимых процедурах, позволяя узнать, сколько процедур было выполнено
Standard	Самый общий шаблон; собирает информацию о хранимых процедурах и нерегламентированных SQL-запросах наряду со статистикой производительности каждой процедуры и пакета. Также регистрируется каждый вход и выход из системы
TSQL	Собирает информацию о хранимых процедурах и нерегламентированных SQL-запросах, но не включает статистику производительности
TSQL_Duration	Собирает информацию о времени выполнения хранимых процедур и нерегламентированных SQL-запросов
TSQL_Grouped	Наряду с информацией об исполняемых хранимых процедурах и нерегламентированных SQL-запросах собирает сведения о всех входах и выходах из системы. Включает информацию, позволяющую идентифицировать приложение и пользователя, выполняющего запрос, но не включает статистику производительности
TSQL_Locks	Собирает информацию об обычных и взаимных блокировках, такую как блокировка процессов, цепочки взаимной блокировки, графы взаимной блокировки, укрупнение блокировок и времена ожидания блокировки. Также регистрируются все хранимые процедуры, все команды хранимых процедур и все нерегламентированные SQL-запросы
TSQL_Replay	Собирает информацию о выполняемых в экземпляре хранимых процедурах и нерегламентированных пакетных файлах SQL в формате, позволяющем воспроизвести трассировку в тестовой системе. Этот шаблон обычно используется для нагрузочного и регрессионного тестирования
TSQL_SPs	Собирает информацию о производительности всех выполняемых в экземпляре нерегламентированных пакетных файлах SQL, хранимых процедурах и всех инструкциях в хранимых процедурах. Также регистрируются все входы и выходы из системы
Tuning	Собирает информацию о базовой производительности нерегламентированных пакетных файлов SQL, хранимых процедур и всех инструкций в хранимых процедурах

По умолчанию при запуске трассировки средствами SQL Server Profiler, все события отображаются в интерфейсе в виде сетки. Данные трассировки также можно сохранить в таблице, файле или и в том, и в другом.

ВНИМАНИЕ! Возвращая результаты в сетку

Многие администраторы устанавливают клиентские инструментальные средства на сервере с SQL Server. Они удобны для выполнения запросов и устранения неполадок экземпляра с серверной консоли, но надо не забывать о дополнительной нагрузке, которую они создают. Приложение SQL Server Profiler за короткое время может собрать очень много событий, которые будучи загружены в сетку интерфейса SQL Server Profiler, могут требовать очень большого объема памяти. Хотя сетки и представляют информацию в удобном для понимания формате, они намного более требовательны к ресурсам, чем простой текстовый файл.

Сохраняя трассировку в файл, можно определить максимальный размер файла, чтобы его размер не рос бесконтрольно. Кроме того, можно включить переключение на файл продолжения. В этом случае, после достижения максимально допустимого размера файл трассировки закрывается и создается новый файл. Если задать максимальный размер, но не включить переключение на файл продолжения, по достижении максимального размера файла приложение SQL Server Profiler прекратит запись событий.

ПРИМЕЧАНИЕ Рекомендации по определению максимального размера файла трассировки

Максимальный размер файла равный 50 Мб — хороший компромисс между размером и числом файлов трассировки. Файл такого размера удобно копировать и перемещать по любой сети, при этом он содержит достаточное число событий, чтобы можно было выполнять анализ.

Если определена таблица для хранения результатов трассировки, приложение SQL Server Profiler создает подключение и перенаправляет все собранные события в заданную таблицу. Если задать максимальный размер как число строк, SQL Server Profiler прекратит сбор событий, достигнув заданного числа строк.

ПРИМЕЧАНИЕ Рекомендации по записи трассировки в файл

Хотя SQL Server оптимизирован для обработки большого числа изменений данных, API-интерфейс SQL Trace способен сгенерировать столько событий, что с ними не сможет совладать даже SQL Server. Никогда не следует записывать события трассировки в экземпляр, события которого собираются. Из-за риска перегрузить сервер SQL Server операциями записи событий рекомендуется регистрировать события трассировки в файл, который в дальнейшем можно загрузить на сервер для анализа.

Время остановки трассировки позволяет запускать трассировку на заданное время. По достижении этого времени (по часам сервера) трассировка останавливается автоматически.

ПРИМЕЧАНИЕ Запись событий служб Analysis Services

Приложение SQL Server Profiler может использоваться для сбора событий служб Analysis Services.

Определение событий для трассировки

SQL Trace предоставляет для записи более 200 событий. Самая важная часть настройки трассировки — выбор событий, которые нужно контролировать.

События разбиты на 21 группу; некоторые группы содержат более 40 событий. Группы событий перечислены в таблице 12-2.

Табл. 12-2. Группы событий SQL Trace

Группа событий	Назначение
Service Broker	13 событий сообщений, очередей и диалогов компонента Service Broker
CLR	1 событие загрузки сборки на языке CLR
Cursors (курсоры)	7 событий создания, удаления и доступа к курсору
Database (база данных)	6 событий изменения данных, увеличения или сжатия файла журнала, а также событий изменения состояния зеркалирования баз данных
Deprecation (устаревшая функциональность)	2 события уведомления об использовании функциональности, которая не будет поддерживаться в последующих версиях SQL Server
Errors and Warnings (ошибки и предупреждения)	16 событий записи ошибок, предупреждений и информационных сообщений. Эти события используются для обнаружения «подозрительных» страниц, заблокированных процессов и отсутствующей статистики столбцов
Full Text (полнотекстовый индекс)	3 события, служащие для отслеживания процесса создания полнотекстового индекса
Locks (блокировки)	9 событий создания, освобождения и укрупнения блокировок, а также создания взаимных блокировок
OLEDDB	5 событий вызова распределенных запросов и удаленных хранимых процедур
Objects (объекты)	3 события создания, изменения и удаления объекта
Performance (производительность)	14 событий, связанных с планами отображения, использования структуры плана и параллелизмом. Эта группа события также служит для сбора информации о полнотекстовых запросах
Progress Report (отчет о ходе выполнения)	1 событие о ходе создания индекса в оперативном режиме
Query Notifications (уведомления о запросах)	4 события, служащие для отслеживания параметров, подписки и шаблонов уведомлений о запросах
Scans (сканирование)	2 события сканирования таблицы или индекса
Security Audit (аудит безопасности)	44 события для отслеживания разрешений, олицетворения, изменений объектов безопасности, выполнения управляющих действия над объектами, запуска или остановки экземпляра, а также резервного копирования и восстановления баз данных

Табл. 12-2. (окончание)

Группа событий	Назначение
Server (сервер)	3 события загрузки ленты, изменений памяти сервера и закрытия файла трассировки
Sessions (сеансы)	3 события существующих подключений, для отслеживания начала трассировки, выполнения триггеров входа в систему и классифицирующих функций регулятора ресурсов
Stored Procedures (хранимые процедуры)	12 событий для отслеживания выполнения хранимых процедур, использования кеша, повторной компиляции и инструкций хранимой процедуры
Transactions (транзакции)	13 таких событий, как начало, сохранение, фиксация и откат транзакций
TSQL	9 событий, связанных с выполнением нерегламентированных запросов T-SQL или XQuery. События всего пакетного файла SQL, а также всех инструкций в пакете
User Configurable (пользовательские события)	10 событий, которые администратор может самостоятельно определить в средствах SQL Trace

Наиболее популярны группы события «Locks», «Performance», «Security Audit», «Stored Procedures» и TSQL. События групп «Stored Procedures» и TSQL обычно регистрируются вместе с событиями группы «Performance» для определения базовых уровней и оценки производительности выполнения запросов. Группа событий «Security Audit» служит для быстрого определения событий аудита среди всего разнообразия событий безопасности. Вместе с тем новая функциональность спецификации аудита предоставляет более безопасные и гибкие возможности аудита. События группы «Locks» обычно используются для анализа неполадок параллелизма.

ПРИМЕЧАНИЕ На экзамене

Вы должны знать, какие события используются для устранения различных неполадок. Например, для устранения взаимных и обычных блокировок или неудовлетворительной производительности хранимых процедур.

Большинство событий возвращает небольшой объем данных, но есть события, способные создавать значительную нагрузку на сильно загруженном экземпляре. Вот события, с которыми надо быть очень осторожным:

- Performance | Showplan *
- Stored Procedures | SP:StmtCompleted
- Stored Procedures | SP:StmtStarting
- TSQL | StmtCompleted
- TSQL | StmtStarting

Эта группа событий должна включаться в трассировку только с очень строгим фильтром, ограничивающим трассировку одним объектом или строкой запроса.

События Showplan Statistics Profile, Showplan Text и Showplan XML возвращают разное количество данных в зависимости от сложности запроса. Слож-

ные запросы могут вернуть объемный план запроса, а функции и хранимые процедуры способны возвращать много планов запроса, соответствующих инструкциям в функции или процедуре. На события Showplan XML приходится большая часть данных всех событий в группе «Performance», потому что помимо сведений о плане отображения и статистической информации возвращаются еще все XML-теги разметки плана отображения в соответствии с его XML-схемой.

События StmtCompleted и StmtStarting обеспечивают самое большое число событий в любой трассировке. Пакеты хранимых процедур и T-SQL могут содержать большое количество выполняемых инструкций. Можно регистрировать данные производительности полной хранимой процедуры, используя событие RPC:Completed, и всего пакета, — применяя событие SQL:BatchCompleted. Однако если надо отладить отдельную инструкцию, можно использовать события StmtCompleted и StmtStarting, которые генерируют события для каждой инструкции, выполняемой в хранимой процедуре или пакете T-SQL.

Выбор столбцов данных

Определив регистрируемые события, надо определить возвращаемые столбцы данных. Можно выбрать все 64 возможных столбца данных, но разумнее собирать только информацию, непосредственно необходимую для решения задачи. Например, глупо регистрировать порядковый номер транзакции в событии RPC:Completed, в условиях, когда анализируемые хранимые процедуры не изменяют никаких данных, — порядковый номер транзакции будет попросту занимать место, не неся никакой информации. Кроме того, не все столбцы данных присутствуют во всех событиях трассировки, например поля Reads, Writes, CPU и Duration неприменимы к таким событиям, как RPC:Starting или SQL:BatchStarting.

Определяя базовый уровень производительности, следует собрать информацию следующих событий:

- RPC:Starting;
- RPC:Completed;
- SQL:BatchStarting;
- SQL:BatchCompleted.

Однако легко обнаружить, что события семейства *Starting предоставляют практически такую же информацию, что и события семейства *Completed. Вместе с тем столбцы Reads, Writes, CPU и Duration события *Completed дополнительно позволяют собирать статистику производительности. Также бессмысленно регистрировать события SQL:BatchStarting или RPC:Starting, потому что они не предоставляют никаких данных производительности, необходимых для определения базового уровня или устранения неполадок производительности.

Столбец DatabaseID события можно записывать вместе со столбцом DatabaseName. Но DatabaseName намного более информативный и понятный, чем DatabaseID, поэтому лучше не включать DatabaseID в определение трассировки.

Столбцы ApplicationName, NTUserName, LoginName, ClientProcessID, SPID, HostName, LoginSID, NTDomainName и SessionLoginName предоставляют информацию о контексте: кто выполняет команду и где она выполняется. Session-

LoginName всегда отображает имя входа в систему, использованный для подключения к экземпляру SQL Server, а столбец LoginName сообщает о всех инструкциях EXECUTE AS и отображает текущий пользовательский контекст команды. Столбец ApplicationName пуст, если только в свойствах приложения не была задана строка подключения на момент попытки подключения к экземпляру. NTUserName и NTDomainName информируют о заданной в подключении учетной записи Windows независимо от того, какая это учетная запись — Windows или SQL Server. HostName исключительно полезен для обнаружения подозрительных процессов в среде, так как содержит имя машины, с которой поступила команда. Например, столбец HostName можно использовать, чтобы найти команды, выполняемые с машины разработки на производственном экземпляре, что происходит из-за неправильной настройки пула приложений.

Столбцы StartTime и EndTime указывают временные границы события. Столбцы StartTime и EndTime оказываются очень кстати, когда нужно сопоставить данные трассировки с информацией из других систем.

Столбцы ObjectName, ServerName, и DatabaseName хороши для анализа. Например, столбец ObjectName события RPC:Completed перечисляет имена выполненных хранимых процедур, что позволяет легко найти хранимую процедуру, поведение которой вызывает неполадки в среде. Данные трассировки можно сохранять в таблице, поэтому очень часто в среде выделяют один экземпляр, куда импортируются все трассировки. Включение столбцов ServerName и DatabaseName позволяет легко разбить данные трассировки по экземплярам и/или базам данных, не разбивая единую таблицу на несколько более мелких.

Использование фильтров

Чтобы более тонко отбирать данные трассировки, можно добавить в трассировку один или несколько фильтров. Фильтр — это по существу предложение WHERE, применяемое к данным события, возвращенным API-интерфейсом SQL Trace. Фильтры закрепляются за столбцами данных и позволяют задавать многие критерии (рис. 12-5).

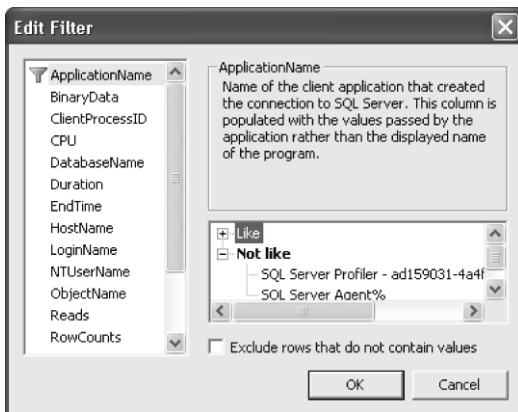


Рис. 12-5. Определение фильтров трассировки

В столбцах данных, содержащих символьные данные, можно задавать текстовые фильтры с предложением LIKE или NOT LIKE, содержащим символы

подстановки. В столбцах данных времени можно задавать условия «больше» или «меньше», в числовых столбцах — «равно», «не равно», «больше или равно» или «меньше или равно». Столбцы двоичных данных фильтрации не поддаются.

Несколько фильтров, примененных к одному столбцу данных, интерпретируются как объединенные условием «ИЛИ». Фильтры, применяемые к нескольким столбцам, интерпретируются как объединенные условием «И».

Управление трассировками

Трассировку можно запустить, остановить или приостановить. После запуска трассировки API-интерфейс SQL Trace возвращает события, удовлетворяющие определению трассировки, и отбрасывает все остальные. При остановке трассировки сбор информации о событиях завершается, а при перезапуске трассировки все предыдущие данные трассировки в окне SQL Server Profiler удаляются. Можно также временно приостановить сбор данных. После возобновления трассировки события из окна SQL Server Profiler не удаляются, а новые добавляются в конец списка.

SQL Server Profiler — приложение, предоставляющее графический интерфейс настройки параметров, которые в дальнейшем преобразуются в вызовы хранимых процедур создания и управления трассировками. Модули трассировки, поставляемые в составе SQL Server 2008, перечислены в таблице 12-3.

Табл. 12-3. Группы событий SQL Trace

Модуль	Назначение
sp_trace_create	Хранимая процедура, создающая новый объект трассировки. Соответствует определению на вкладке General диалогового окна New Trace в приложении SQL Server Profilere
sp_trace_generateevent	Хранимая процедура, позволяющая определять пользовательские события трассировки
sp_trace_setevent	Хранимая процедура, добавляющая столбец данных события, которое будет записываться. Процедуру sp_trace_setevent надо вызывать при добавлении каждого столбца данных регистрируемого события. Соответствует сетке выбора события и столбца данных на вкладке Events Selection диалогового окна New Trace в приложении SQL Server Profiler
sp_trace_setfilter	Хранимая процедура, добавляющая фильтр в трассировку. Соответствует диалоговому окну Edit Filter в приложении SQL Server Profilere
sp_trace_setstatus	Хранимая процедура, которая запускает, останавливает и закрывает трассировку. Состояние 0 означает остановку трассировки. Состояние 1 — запуск трассировки. Состояние 2 — завершение трассировки и удаление определения трассировки из экземпляра
fn_trace_geteventinfo	Функция, возвращающая события и столбцы данных, регистрируемые трассировкой
fn_trace_getfilterinfo	Функция, возвращающая фильтры указанной трассировки

Табл. 12-3. (окончание)

Модуль	Назначение
fn_trace_getinfo	Функция, возвращающая состояние и свойства всех определенных в экземпляре трассировок
fn_trace_gettable	Функция, считывающая один или более файлов трассировки. Возвращает содержимое в виде набора результатов. Обычно используется для импорта файлов трассировки в таблицу

ПРИМЕЧАНИЕ **Рекомендации по автоматическому выполнению трассировки**
SQL Server Profiler можно использовать для запуска вручную кратковременных трассировок экземпляра, но намного чаще создают и управляют трассировками, используя код. Выполнение трассировки с применением кода позволяет избежать от нагрузки, создаваемой инструментом с графическим интерфейсом, а также позволяет выполнять трассировку автоматически с применением заданий Агента SQL Server.

Связь между производительностью и мониторингом данных

SQL Trace служит для регистрации информации о происходящем в экземпляре SQL Server. Системный монитор применяется для записи показаний счетчиков производительности, информирующих о том, что происходит с оборудованием и другими компонентами сервера. SQL Server не может работать без оборудования. Состояние аппаратных ресурсов определяет, насколько успешно SQL Server выполняет свои функции. Например, запрос может выполняться медленно, но приложение SQL Server Profiler сообщит только, насколько медленно. Однако воспользовавшись счетчиками производительности, можно обнаружить, что причина медленного выполнения запросов — недостаток вычислительных ресурсов.

Хотя диагностировать неполадку можно, используя только Системный монитор или SQL Trace, использование обоих инструментов существенно обогащает возможности анализа. Основная сложность — правильно согласовать полученные два набора данных, чтобы анализ оказался действительно эффективным. Данные трассировки можно сохранить в таблице, и то же можно сделать с журналом счетчиков. Объединив оба набора данных в единой таблице, можно выполнять различные запросы, сопоставляя информацию.

Если создавать запросы сопоставления данных не хочется, можно выполнить сопоставление намного более простым способом. Приложение SQL Server Profiler позволяет просматривать файл трассировки совместно с журналом счетчиков. Приложение SQL Server Profiler позволяет не только отображать эти два набора данных в одном окне, но и синхронизировать их по времени записи. При просмотре трассировки специальный индикатор показывает значения счетчиков в момент выполнения запроса.

ВНИМАНИЕ! **Сопоставление журнала счетчиков и файла трассировки**
Такое сопоставление возможно, только если в трассировке записывался столбец данных StartTime.

Контрольные вопросы

1. Какие три элемента нужно определить в трассировке?
2. Какие события обычно используются для определения базового уровня производительности?

Ответы на контрольные вопросы

1. В трассировке надо задать события, столбцы данных и фильтры.
2. Для определения базового уровня производительности используются события RPC:Completed и SQL:BatchCompleted.

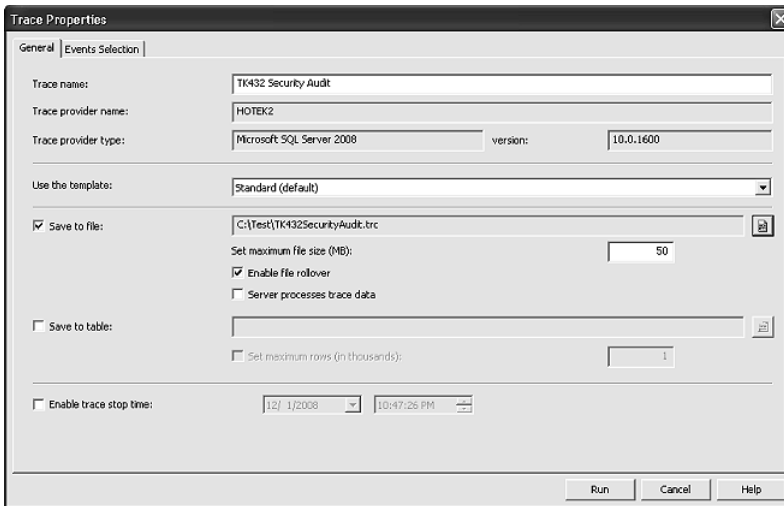
Практикум. Создание трассировки

Вы создадите трассировки для аудита безопасности, определения базового уровня производительности и устранения взаимной блокировки. Вы также импортируете трассировку в таблицу.

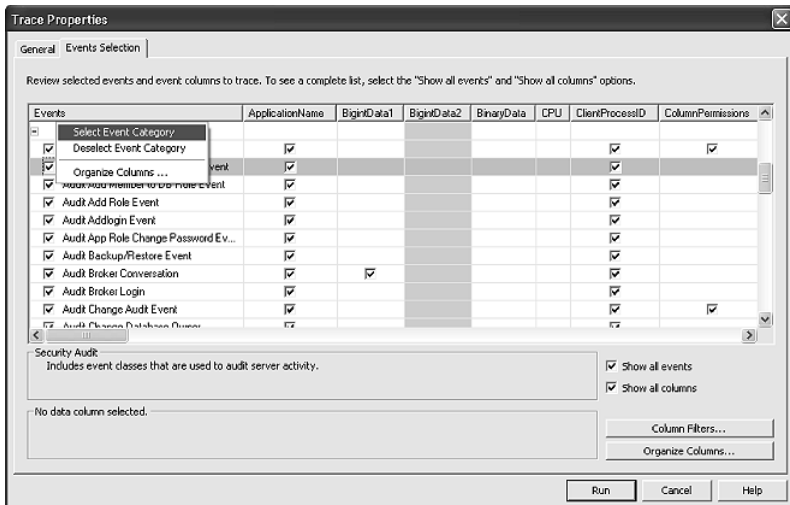
Задание 1. Создание трассировки аудита безопасности

Вы создадите трассировки для аудита безопасности.

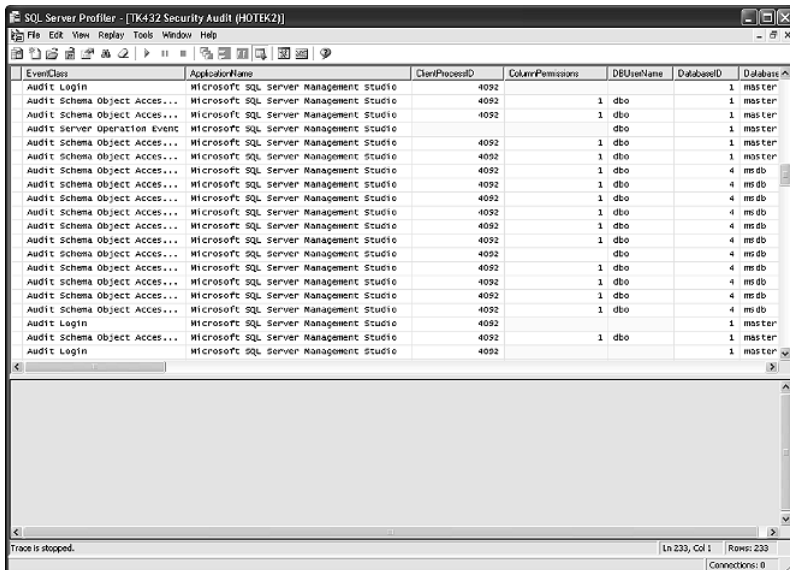
1. Откройте приложение SQL Server Profiler. Выберите **File/New Trace** и подключитесь к своему экземпляру SQL Server.
2. Задайте имя трассировки, шаблон и параметры сохранения в файле, как показано ниже.



3. Выберите вкладку **Events Selection**.
4. Установите флажки **Show All Events** и **Show All Columns**.
5. Щелкните правой кнопкой категорию событий **Security Audit** и выберите **Select Event Category**, как показано ниже.



- Щелкните **Run**.
- Откройте Среду SQL Server Management Studio (SQL Server Management Studio), подключитесь к своему экземпляру SQL Server и выполните связанные с безопасностью операции, такие как создание имени входа в систему, создание пользователя базы данных и объекта.
- Посмотрите результаты своих действий в сетке событий SQL Server Profiler, как показано ниже.

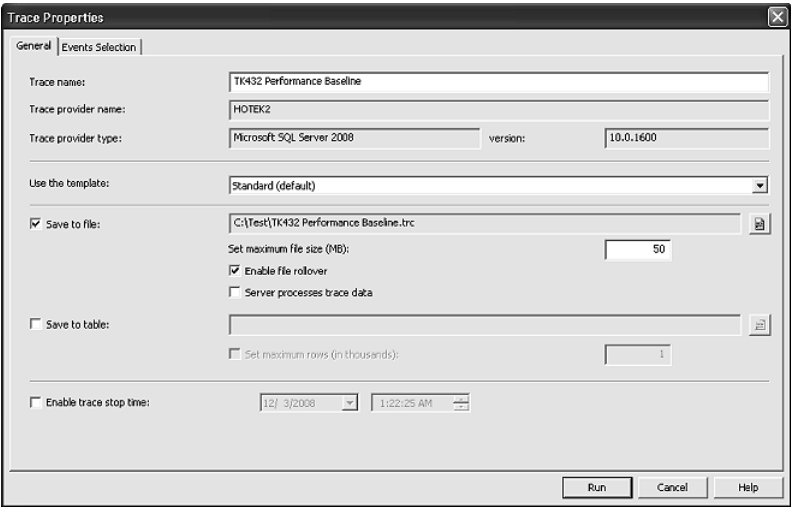


- Остановите и закройте трассировку.

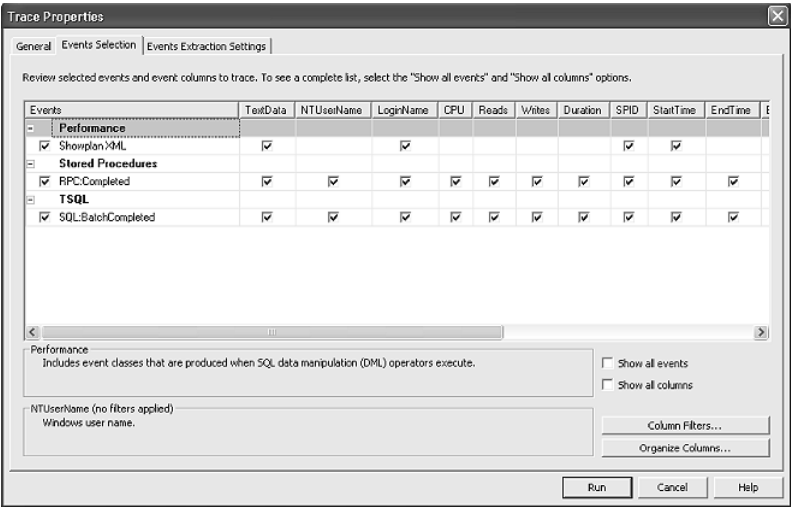
Задание 2. Определение базового уровня производительности

Вы сконфигурируете трассировку для определения базового уровня производительности сохраненных процедур и нерегламентированных SQL-запросов.

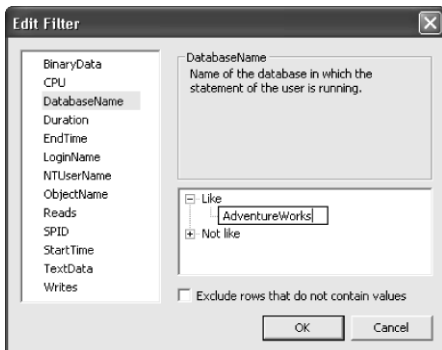
1. Откройте приложение SQL Server Profiler. Выберите **File/New Trace** и подключитесь к своему экземпляру SQL Server.
2. Задайте имя трассировки, шаблон и параметры сохранения в файле, как показано ниже.



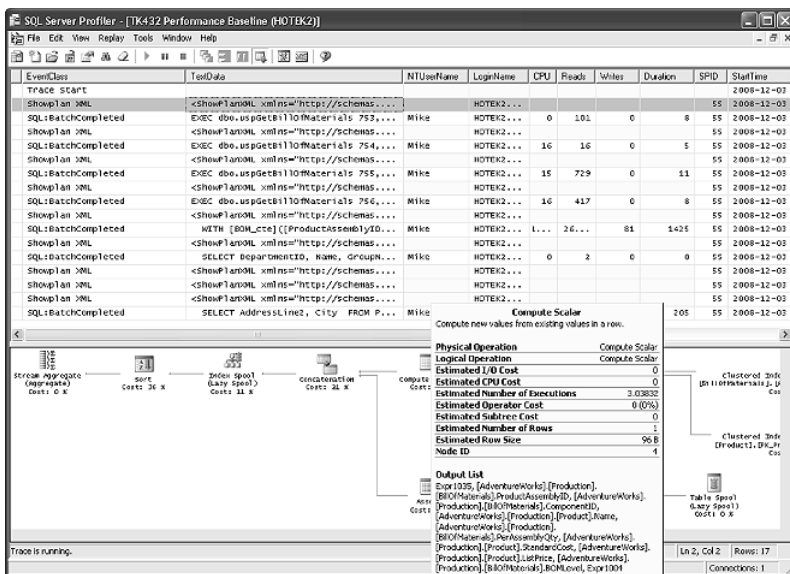
3. Выберите вкладку **Events Selection**.
4. В разделе Security Audit снимите флажки событий **Audit Login**, **Audit Logout**, **ExistingConnection** и **SQL:BatchStarting**.
5. Отметьте флажком событие **Showplan XML**. Этот флажок добавляем только для иллюстративных целей; обычно при трассировке базового уровня производительности план отображения XML не регистрируется.
6. Выберите столбцы **TextData**, **NTUserName**, **LoginName**, **CPU**, **Reads**, **Writes**, **Duration**, **SPID**, **StartTime**, **EndTime**, **BinaryData**, **DatabaseName**, **ServerName** и **ObjectName**, как показано ниже.



7. Щелкните **Column Filters** и определите фильтр базы данных Adventure Works, как показано ниже.



8. Щелкните **Run**.
9. Выполните несколько запросов и сохраненных процедур в базе данных AdventureWorks и посмотрите результаты в приложение SQL Server Profiler, как показано ниже.



Задание 3. Импорт файла трассировки

Вы выполните импорт файла трассировки в таблицу для дальнейшего анализа.

1. Откройте окно запросов и выполните команду:

```
SELECT * INTO dbo.TK432BaselineTrace
FROM fn_trace_gettable('c:\test\TK432 Performance Baseline.trc', default);
GO
```

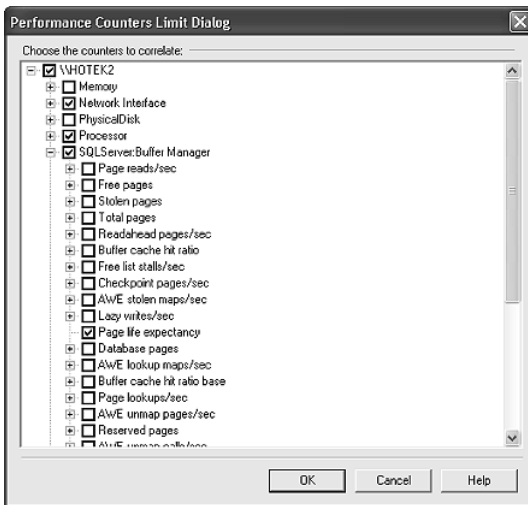
2. Выполните следующий запрос и просмотрите результаты:

```
SELECT * FROM dbo.TK432BaselineTrace  
GO
```

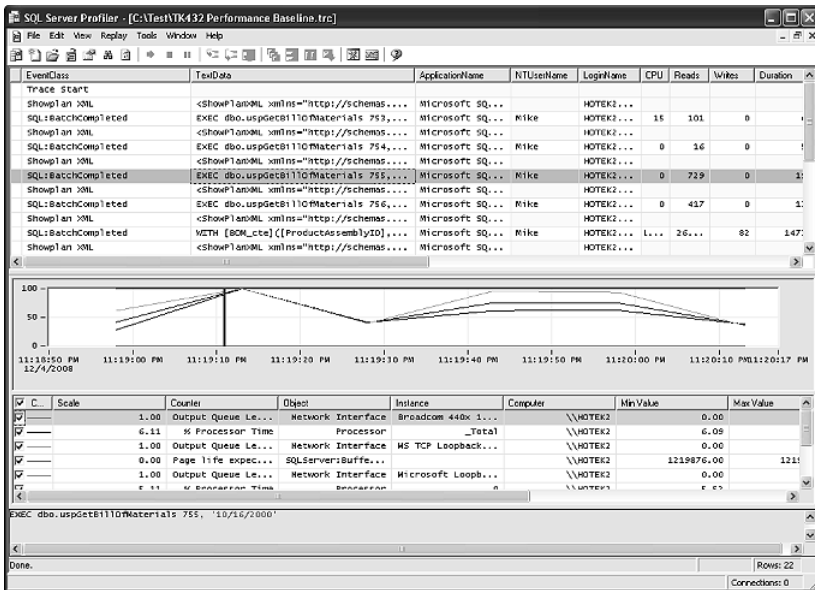
Задание 4. Сопоставление данных SQL Trace и Системного монитора

Вы сопоставите данные журнала счетчиков Системного монитора, созданного на занятии 1 с трассировкой SQL Server Profiler, созданной в задании 2 данного занятия.

1. Остановите журнал счетчиков, созданный на занятии 1.
2. Откройте приложение SQL Server Profiler. Выберите **Select File/Open/Trace File**. Выберите файл трассировки базового уровня производительности, созданного в задании 2 этого занятия.
3. Выберите **File/Import Performance Data**. Выберите журнал счетчиков, созданный в занятии 1.
4. В окне Performance Counters Limit Dialog (см. рисунок) выберите **Network Interface:Output Queue Length**, **Processor:% Processor Time**, **System:Processor Queue Length** и **SQLServer:Buffer Manager:Page life expectancy**. Щелкните **OK**.



5. Прокрутите до конца трассировку в верхней панели окна SQL Server Profiler и просмотрите изменения, происходящие в графике счетчика производительности и сетке, как показано ниже.



Резюме занятия

- Приложение SQL Server Profiler — утилита, предоставляющая графический интерфейс для взаимодействия с SQL Trace API.
- SQL Trace предоставляет события, которые можно регистрировать для выполнения аудита, мониторинга состояния экземпляра, запросов определения базовых уровней, а также для устранения неполадок производительности.
- В каждом событии можно выбрать столбцы данных, которые будут регистрироваться.
- Выходную информацию трассировки можно ограничивать, применяя фильтры.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Вы пытаетесь устранить неполадку производительности в компании Fabrikam. Приблизительно каждые 15 минут по истечении каждого часа наблюдается снижение производительности, длящееся примерно одну минуту, после чего производительность приложения снижается до обычного уровня. Какие инструментальные средства можно задействовать для выявления причины этой проблемы с производительностью? (Выберите все подходящие ответы.)
 - A. Системный монитор.
 - B. Помощник по настройке ядра СУБД (Database Engine Tuning Advisor).
 - C. Регулятор ресурсов (Resource Governor).
 - D. Приложение SQL Server Profiler.

Занятие 3. Диагностика сбоев баз данных

Здесь рассказывается, как просматривать и фильтровать журналы, используемые для диагностики неполадок. Вы также узнаете, что делать с неполадками, связанными с нехваткой пространства баз данных — самыми частыми причинами ошибок.

Изучив материал этого занятия, вы сможете:

- ✓ работать с журналами SQL Server;
- ✓ диагностировать неполадки, связанные с нехваткой пространства.

Продолжительность занятия — 20 минут.

Журналы SQL Server

Информационные сообщения и сообщения об ошибках сервера SQL Server можно найти в следующих журналах:

- журнал событий Windows;
- журнал ошибок SQL Server;
- журнал Агента SQL Server;
- журналы почты базы данных.

Журналы событий Windows обычно просматривают с помощью консоли **Просмотр событий** (Event Viewer), где информация разбита на три журнала:

- Журнал **Система (System)** содержит системные информационные сообщения и сообщения об ошибках, прежде всего относящихся к компонентам ОС и оборудования.
- Журнал **Приложение (Application)** — основной источник информации о SQL Server, содержащий все информационные сообщения и сообщения об ошибках экземпляра, включая сообщения о запуске, остановке или приостановке служб.
- Журнал **Безопасность (Security)** — если включен аудит событий входа и выхода из системы, каждая успешная попытка подключения и отключения от экземпляра фиксируется в этом журнале.

Журнал ошибок SQL Server — это текстовый файл на диске, который можно открывать и просматривать любым текстовым редактором, таким как Блокнот. Текущий журнал ошибок SQL Server называется errorlog (без расширения) и находится в папке `MSSQL10.<имя_экземпляра>\MSSQL\LOG`. Содержимое текущего журнала ошибок можно получить в виде набора результатов, выполнив дополнительную системную хранимую процедуру `xp_readerrorlog`.

Журнал ошибок SQL Server содержит начальную информацию, такую как версия (включая пакет исправлений) SQL Server и Windows, идентификатор процесса Windows, режим проверки подлинности, число процессоров, параметры конфигурации экземпляра и сообщения всех баз данных, которые открывались, восстанавливались и успешно запускались. Журнал ошибок также содержит информационные сообщения об основных событиях в экземпляре, таких как запуск и остановка трассировок, резервное копирование и восстановление

баз данных. Однако основная задача журнала ошибок состоит в регистрации сообщений об ошибках, таких как повреждение базы данных, дамп стека, недостаток ресурсов и отказ оборудования.

Журнал ошибок SQL Server также содержит все сообщения, созданные командой *RAISERROR* с параметром *WITH LOG*. Сообщения регистрируются в журнале ошибок SQL Server и журнале событий **Приложение** (Application) Windows, если *RAISERROR* выполнена с уровнем серьезности ошибок 16 и выше.

Ошибки и информационные сообщения, относящиеся к Агенту SQL Server, попадают в журнал Агента SQL Server, который называется *Sqlogent.out*. Журнал почты базы данных хранится в таблице *dbo.sysmail_log* базы данных *msdb*.

Консоль **Просмотр событий** (Event Viewer) можно использовать для просмотра журналов событий, Блокнот (Notepad) — для изучения журналов SQL Server и Агента SQL Server, а T-SQL — для просмотра журналов почты базы данных. Есть альтернатива множеству инструментальных средств — Средство просмотра журнала (Log File Viewer) в SSMS, позволяющее изучать различные журналы ошибок и событий в едином интерфейсе, показанном в рис. 12-6.

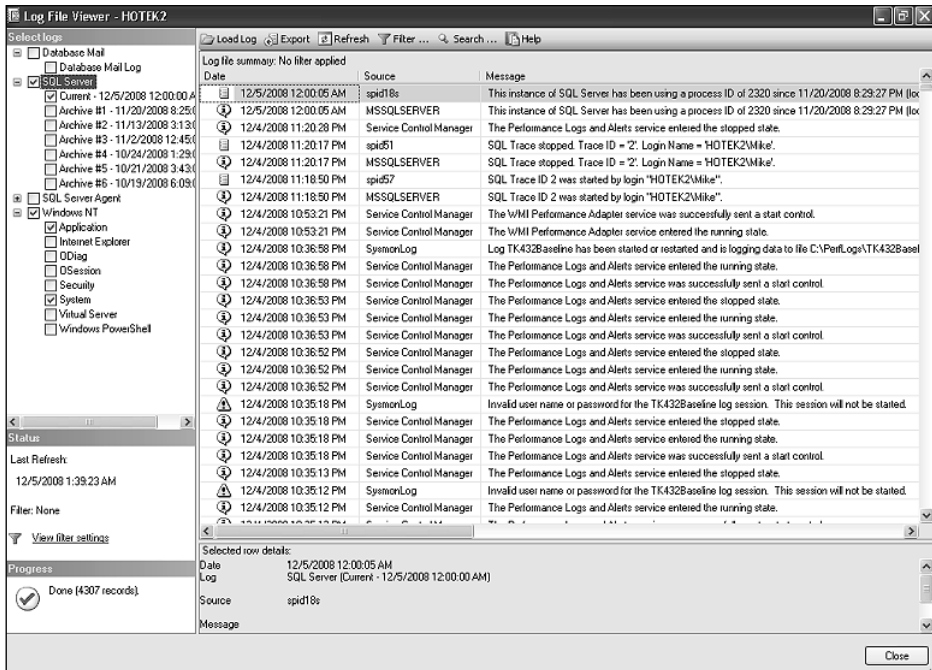


Рис. 12-6. Просмотр журналов ошибок и событий

Средство просмотра журнала обеспечивает интегрированное представление всех выбранных журналов по дате и времени событий. Объединение всех журналов позволяет прямо сопоставлять информацию многих журналов в процессе диагностики неполадок, например увидеть все записи в системных или прикладных журналах, выполненные в момент возникновения неполадки SQL Server.

Помимо возможности одновременного просмотра многих файлов в едином интерфейсе Средство просмотра журнала также позволяет искать, фильтровать и экспортировать информацию в файлы журналов.

Проблемы с нехваткой пространства в базе данных

Самые часто встречающиеся ошибки — исчерпание свободного пространства для хранения данных или журналов.

При исчерпании свободного пространства для журнала транзакций все операции записи в базу данных останавливаются. Возможность считывать данные из базы сохраняется, но любая попытка записи откатывается и возвращает приложению ошибку 9002, которая также фиксируется в журнале ошибок SQL Server и журнале **Приложение** (Application). При переполнении журнала транзакций допустимы следующие действия:

- резервное копирование журнала транзакций;
- увеличение дискового пространства на томе, где расположен файл журнала транзакций;
- перемещение журнала на том большего объема;
- увеличение размера файла журнала транзакций;
- создание другого журнала на томе, где есть свободное пространство.

Первым делом надо выполнить резервное копирование журнала транзакций. Однако это может не освободить достаточно пространства в повторно используемом журнале, если хотя бы одно из следующих условий верно:

- база данных участвует в репликации, а база данных распространителя недоступна;
- имеется открытая транзакция;
- приостановлено зеркальное отображение базы данных;
- зеркальная база данных отстает от основной БД;
- выполняется сканирование журнала;
- SQL Server не может достаточно быстро увеличить размер файла.

ПРИМЕЧАНИЕ Увеличение свободного пространства в журнале транзакций

Если резервное копирование журнала транзакций не освобождает достаточно пространства в повторно используемом журнале, нужно увеличить пространство, доступное журналу транзакций, путем расширения доступного дискового пространства. Для этого обычно добавляют в базу данных второй журнал на дисковом томе, где есть достаточно свободного пространства.

Если база данных заполняет все дисковое пространство, возникает ошибка 1101 или 1105. Пока базе данных не хватает пространства, в нее нельзя добавить никаких новых данных. Увеличить пространство, доступное базе данных, можно, присоединив файл в соответствующую файловую группу на дисковом томе, на котором есть дополнительное свободное пространство.

Казалось бы можно добавить новую файловую группу с файлами на дисковый том со свободным пространством, но это не решит проблему нехватки пространства. Все данные записываются в таблицы базы данных. Таблица хранится в одной файловой группе или сопоставляется схеме секционирования,

которая распределена среди многих файловых групп. При добавлении новой файловой группы ни одна из существующих таблиц или схемы секционирования не в состоянии использовать новую файловую группу для размещения данных, записываемых в существующие объекты базы данных. Чтобы увеличить доступное пространство и устранить ошибки 1101 и 1105, файлы надо добавлять в существующие в базе данных файловые группы.

ПРИМЕЧАНИЕ Увеличение файла

Перед добавлением файлов в файловые группы надо проверять, не включена ли в файлах данных функция автоувеличения размера файла. Если автоувеличение не включено, а на дисковом томе еще есть пространство, можно расширить доступное пространство, увеличив размер существующих файлов.

База данных tempdb требует особого внимания и отношения. Нехватка пространства в tempdb создает серьезные проблемы в работе многих процессов. Помимо хранения локальных и глобальных временных таблиц, которые может создавать любой пользователь, tempdb используется для хранения:

- рабочих таблиц запросов GROUP BY, ORDER BY и UNION;
- рабочих таблиц курсоров и операций пула;
- рабочих таблиц для создания и перестроения индексов, если задан параметр SORT_IN_TEMPDB;
- рабочие файлы для операций хеширования.

База данных Tempdb также используется как хранилище версий, которое необходимо для хранения версий строк в следующих операциях:

- создание индексов в оперативном режиме;
- перестроение индексов в оперативном режиме;
- транзакции, выполняющиеся на уровне изоляции моментального снимка;
- транзакции, выполняющиеся в базе данных, когда включено свойство чтения зафиксированного моментального снимка;
- выполнение операций в режиме MARS (Multiple Active Result Sets).

Нехватка свободного пространства в tempdb может повлиять на все базы данных экземпляра. В особо серьезных случаях могут прекратить работу все приложения, использующие экземпляр. В случае проблем со свободным пространством помимо ошибок 1101, 1105 и 9002, tempdb может инициировать ошибки 3958, 3959, 3966 и 3967.

ПРИМЕЧАНИЕ На экзамене

На экзамене вы должны знать коды стандартных ошибок, связанных с нехваткой, свободного пространства, что означает тот или иной код и как устранить причину ошибки.

ПРИМЕЧАНИЕ Рекомендации по устранению ошибок, связанных с нехваткой, свободного пространства

Исчерпание свободного пространства — серьезная неполадка, которую нужно избегать любой ценой. Большинство администраторов базы данных занимают пассивную позицию, предпринимая какие-либо действия только при возникновении неполадок. А надо действовать с упреждением, чтобы не допускать не-

хватки пространства. Для этого можно создать предупреждения, информирующие администратора о снижении объема свободного пространства ниже заданного порога — обычно задают порог в 10–15%. Можно также настроить немедленное уведомление при возникновении проблем со свободным пространством, создавая предупреждения в Агенте SQL Server.

Контрольные вопросы

1. Где обычно можно найти информационные сообщения и сообщения об ошибках в работе ядра базы данных?
2. Каковы три кода ошибок, возникающих при нехватке пространства в базе данных?

Ответы на контрольные вопросы

1. Информацию об ошибках ядра базы данных обычно ищут в журнале ошибок SQL Server и журнале событий **Приложение** (Application). Для аудита входа и выхода из системы экземпляра обычно применяют журнал ошибок SQL Server и журнал «Безопасность». Кроме того в журнале Windows «Система» можно найти информацию об оборудовании и операционной системе, которые могут быть полезными для устранения неполадок ядра базы данных.
2. Ошибка 9002 возникает при исчерпании свободного пространства для журнала транзакций, а ошибки 1101 или 1105 — при нехватке пространства для файлов данных.

Практикум. Создание предупреждений о неполадках

Вы создадите предупреждения Агента SQL Server о различных ошибках базы данных.

Задание 1. Предупреждение о хранилище версий

Вы создадите предупреждения об ошибках хранилища версий в tempdb.

1. В панели **Обозреватель объектов** (Object Explorer) откройте узел **Агент SQL Server** (SQL Server Agent).
2. Щелкните правой кнопкой узел **Предупреждения** (Alerts) и выберите **Создать предупреждение** (New Alert).
3. Присвойте предупреждению имя Full Version Store.
4. В качестве базы данных выберите tempdb.
5. Выберите **Номер ошибки** (Error Number) и задайте номер 3959, как показано на рис. 12-7.
6. Перейдите на страницу **Ответ** (Response) и выберите операторов, которых нужно уведомлять по электронной почте.
7. Перейдите на страницу **Параметры** (Options), установите флажок **По электронной почте** (E-mail) и задайте параметр **Задержка между ответами** (Delay Between Responses) равным 1 минуте (рис. 12-8).

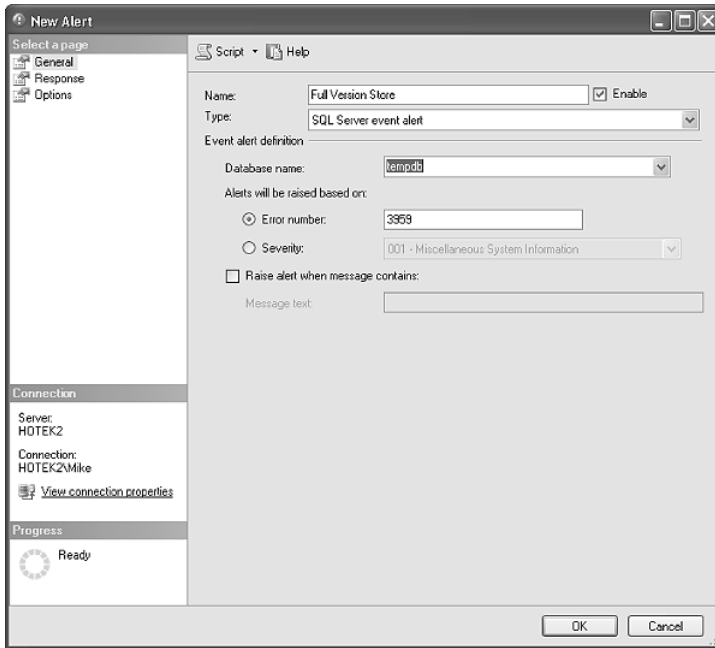


Рис. 12-7. Определение номера ошибки

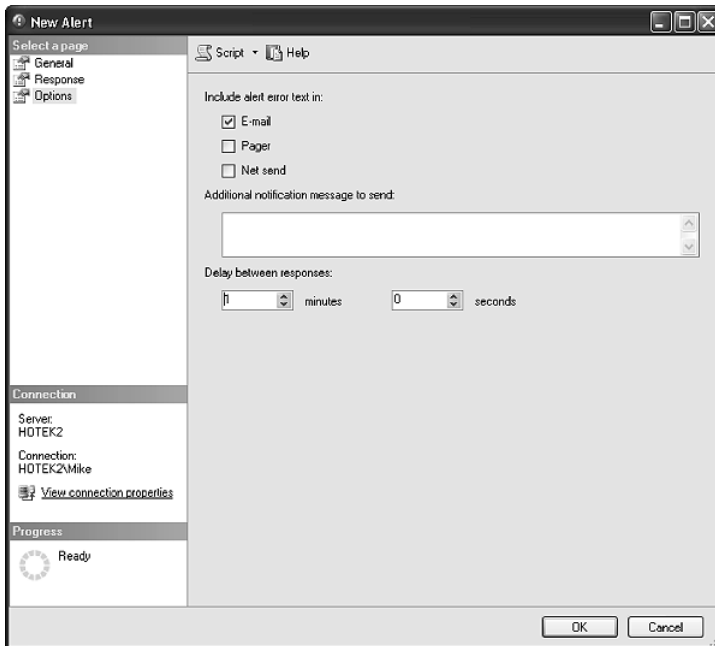


Рис. 12-8. Определение задержки между ответами

8. Щелкните **ОК**.
9. Повторив описанные выше операции, создайте предупреждение Forced Version Store Shrink об ошибке 3967.

10. Повторив описанные выше операции, создайте предупреждение Row Version Not Found об ошибке 3958.
11. Повторив описанные выше операции, создайте предупреждение об ошибке 3966. Поскольку названия предупреждений должны быть уникальны, а у нас два номера ошибки, указывающие на одну неполадку, во второе предупреждение добавим порядковый номер: Row Version Not Found2.

Задание 2. Предупреждения журнала

Вы создадите предупреждение о переполнении журнала транзакций.

1. Щелкните правой кнопкой узел **Предупреждения (Alerts)** и выберите **Создать предупреждение (New Alert)**.
2. Присвойте предупреждению имя Transaction Log Full.
3. Выберите **<все базы данных> (<All Databases>)**.
4. Выберите **Номер ошибки (Error Number)** и задайте номер 9002.
5. Перейдите на страницу **Ответ (Response)** и выберите операторов, которых нужно уведомлять по электронной почте.
6. Перейдите на страницу **Параметры (Options)**, установите флажок **По электронной почте (E-mail)** и задайте параметр **Задержка между ответами (Delay Between Responses)** равным 1 минуте.
7. Щелкните **ОК**.

Задание 3. Предупреждения о файлах данных

Вы создадите предупреждение о нехватке пространства для базы данных.

1. Щелкните правой кнопкой узел **Предупреждения (Alerts)** и выберите **Создать предупреждение (New Alert)**.
2. Присвойте предупреждению имя Database Full.
3. Выберите **<все базы данных> (<All Databases>)**.
4. Выберите **Номер ошибки (Error Number)** и задайте номер 1101.
5. Перейдите на страницу **Ответ (Response)** и выберите операторов, которых нужно уведомлять по электронной почте.
6. Перейдите на страницу **Параметры (Options)**, установите флажок **По электронной почте (E-mail)** и задайте параметр **Задержка между ответами (Delay Between Responses)** равным 1 минуте.
7. Щелкните **ОК**.
8. Повторив описанные выше процедуры, создайте предупреждение по имени Database Full2 об ошибке 1105.

Резюме занятия

- Журнал ошибок SQL Server содержит конфигурационную информацию о запуске экземпляра, ошибках, дампах стека, а также информационные сообщения об экземпляре.
- Журнал событий Windows **Приложение (Application)** содержит сообщения о запуске и остановке служб, сообщения о важных событиях, ошибки, а также всю информацию, возвращаемую командой RAISERROR, выполняемой с параметром WITH LOG или с уровнем серьезности 16 или выше.

- Средство просмотра журнала (Log File Viewer) позволяет просматривать журналы ошибок и событий, объединенные в единый список в хронологическом порядке. Средство просмотра журнала также позволяет фильтровать и выполнять поиск в журналах.
- Ошибка 1101 или 1105 случается при нехватке свободного пространства в базе данных.
- Ошибка 9002 наблюдается при переполнении журнала транзакций.
- При нехватке свободного пространства в хранилище версий могут наблюдаться следующие ошибки: 3958, 3959, 3966 и 3967.
- В Агенте SQL Server можно сконфигурировать предупреждения о любых событиях связанных с наличием или отсутствием свободного пространства.
- В Агенте SQL Server можно сконфигурировать условное предупреждение о производительности, информирующее о низком объеме свободного пространства.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Какие типы событий SQL Server регистрируются в журнале событий «Приложение»? (Выберите все подходящие варианты.)
 - A. Создание дампа стека.
 - B. Сообщения о настройке запуска.
 - C. Сбои заданий.
 - D. Уничтожение процессов.

Занятие 4. Диагностирование сбоев служб

SQL Server и Агент SQL Server выполняются как службы Windows. Помимо сбоев SQL Server из-за проблем с оборудованием или программным обеспечением могут наблюдаться сбои запуска. Здесь рассказывается, как диагностировать и устранять самые типичные неполадки, приводящие к невозможности запуска службы.

Изучив материал этого занятия, вы сможете:

- ✓ находить информацию о сбое запуска службы;
- ✓ диагностировать причину сбоя запуска службы;
- ✓ устранять наиболее популярные причины сбоя запуска службы.

Продолжительность занятия — 20 минут.

Обнаружение сбоев запуска служб

Для просмотра состояния и конфигурации служб SQL Server вы можете использовать две утилиты — консоль **Службы** (Services) и **Диспетчер конфигурации SQL Server** (SQL Server Configuration Manager).

Как уже говорилось в главе 11, у каждого экземпляра в корне иерархии шифрования есть главный ключ службы, который шифруется с использованием имени и пароля учетной записи службы SQL Server. В консоли **Службы** нет кода шифрования главного ключа службы, поэтому никогда не следует использовать консоль **Службы** для изменения учетной записи или пароля учетной записи для службы.

Диспетчер конфигурации

Диспетчер конфигурации SQL Server устанавливается на каждой машине, где работает экземпляр SQL Server 2008, и его нельзя удалить. В отличие от других утилит SQL Server, Диспетчер конфигурации SQL Server не поддерживает управление экземплярами на многих машинах. Диспетчер конфигурации SQL Server применяется для настройки и управления службами SQL Server, сетевой конфигурацией и собственным клиентом. В главной консоли Диспетчера (рис. 12-9) можно просматривать состояние, режим запуска и учетную запись службы, а также запустить, остановить, приостановить и перезапустить службу.

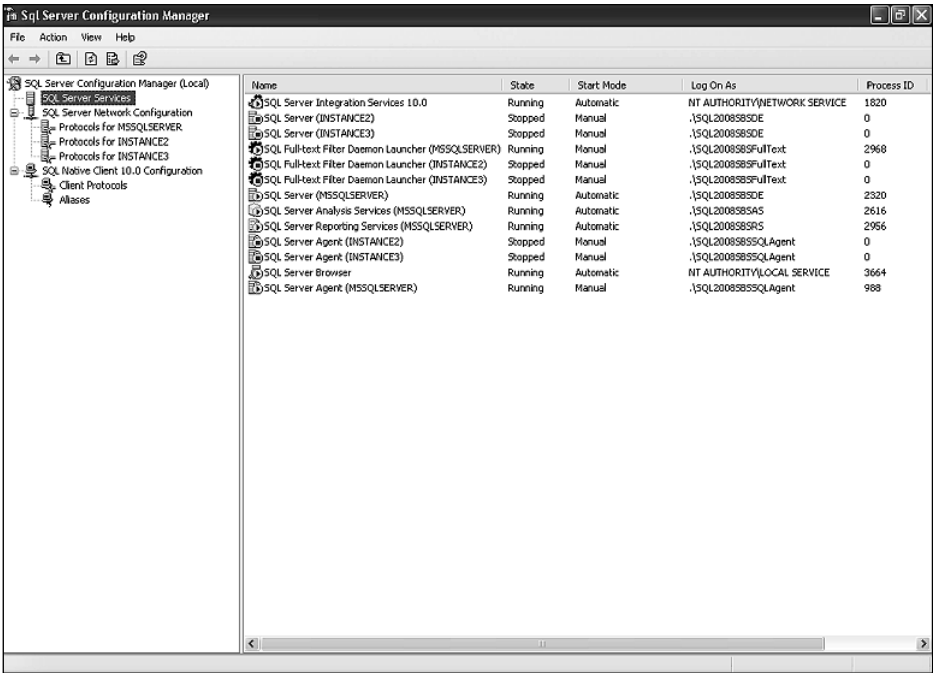


Рис. 12-9. Диспетчер конфигурации SQL Server

ПРИМЕЧАНИЕ Рекомендации по управлению службами SQL Server

Всегда намного проще помнить и применять политику администрирования, в которой немного исключений. Средствами консоли «Службы» можно безопасно изменять некоторые, но не все параметры служб SQL Server. Поэтому в большинстве сред, с которыми я работал, существует четкое предписание, что все изменения службы SQL Server должны выполняться только средствами Диспетчера конфигурации SQL Server.

Удаленный рабочий стол

Много организаций теперь устанавливают SQL Server на выделенных машинах, причем, к сожалению, во многих из таких организаций также реализована политика так называемого «разделения обязанностей». Такая политика обычно подразумевает, что администраторы баз данных SQL Server должны управлять только SQL Server, а администраторы Windows — ОС Windows и оборудованием, на котором работает SQL Server.

Но такой подход не учитывает, что SQL Server глубоко связан с ОС и оборудованием. Когда машина с SQL Server работает нормально, администраторам баз данных не нужен доступ к операционной системе или аппаратным средствам, кроме как для проверки содержимого и наличия свободного пространства на дисках. Когда работа машины с SQL Server нарушается, администраторам баз данных нужен доступ к ОС, аппаратным средствам и диагностическим утилитам.

При устранении неполадок запуска службы нужно иметь возможность использовать удаленный рабочий стол для подключения к консоли машины с SQL Server, чтобы получить доступ к Диспетчеру конфигурации SQL Server. Также требуется доступ к дисковым томам, консоли **Просмотр событий** (Event Viewer) и файлам ошибок. Если у администратора базы данных нет удаленного доступа к серверу с SQL Server, ему приходится искать того, у кого такой доступ есть, и только затем пытаться устранять сбой запуска службы по телефону, надеясь, что человек на том конце линии делает все правильно и выполняет те действия, о которых его просит администратор базы данных. Во многих случаях устранение сбоя запуска службы, когда у администратора базы данных нет удаленного доступа, превращает неполадку, которую можно было бы устранить за пару минут, в серьезную проблему, на решение которой уходят часы.

Если нельзя подключиться к экземпляру SQL Server, надо первым делом убедиться, что SQL Server работает. Если служба находится в состоянии запуска, значит SQL Server запускается и вскоре станет доступным.

Если служба остановлена, SQL Server отключен по одной из следующих причин:

- в службе настроен режим запуска «Вручную» или «Отключена» и машина была перезагружена;
- служба кем-то остановлена;
- в SQL Server случилась критическая ошибка, и служба была остановлена.

Когда служба SQL Server находится в остановленном состоянии, первым делом надо посмотреть журнал ошибок SQL Server или журнал событий **Приложение** (Application). Если экземпляр SQL Server был остановлен из-за критических ошибок, в обоих журналах будут содержаться дополнительные диагностические сообщения, которые можно использовать для устранения неполадок оборудования. Подробнее об этом говорится в занятии 5.

Если машина была перезагружена, в обоих журналах содержится сообщение «Работа SQL Server завершается вследствие отключения системы». (SQL Server is terminating because of a system shutdown). Если кто-то остановил службу,

тогда в журналах будет сообщение: «Работа SQL Server завершается в ответ на запрос “stop” от диспетчера управления службами». (SQL Server is terminating in response to a ‘stop’ request from Service Control Manager). Если в журналах нет никаких дополнительных сообщений об ошибках оборудования, нужно просто перезапустить службу.

После перезапуска службы надо подключиться и убедиться, что все базы данных находятся в оперативном режиме и доступны, а приложения успешно подключаются. Восстановив работоспособность сервера, надо узнать, кто остановил службу или перезагрузил машину, чтобы неполадка не повторялась в будущем.

Нормальный режим запуска SQL Server и служб Агента SQL Server на автономной машине — «Авто». Если службы настроены на запуск вручную или их запуск отключен, нужно узнать, кто изменил режим запуска и почему, и позаботиться, чтобы эти изменения не влияли на работоспособность сервера. Режим запуска можно изменить на вкладке **Служба** (Service) страницы свойств службы (рис. 12-10), щелкнув правой кнопкой службу и выбрав **Свойства** (Properties).



Рис. 12-10. Режим запуска службы

ПРИМЕЧАНИЕ Режим запуска

Служба с типом запуска «Отключена» (Disabled) содержит запись «Другое» (Other) в столбце «Режим запуска» (Start Mode) в окне Диспетчера конфигурации SQL Server.

ВНИМАНИЕ! Кластеризованные экземпляры SQL Server

Режим запуска всех служб SQL Server в кластере должен быть «Вручную» (Manual). Не изменяйте режим запуска каких-либо служб в кластере. Службы управляются службой кластера, и настройка автоматического запуска любой из служб создаст существенные проблемы в работе кластера при перегрузке узлов.

Учетные записи служб

Для нормальной работы учетной записи службы SQL Server у нее должно быть несколько разрешений:

- доступ на чтение и запись к папке (или папкам), где хранятся файлы данных и журналы системных баз данных;
- доступ на чтение и запись к относящимся к SQL Server разделам реестра;
- привилегия «Вход в качестве службы» (Log on as a service);
- права системного администратора в экземпляре SQL Server.

Если служба SQL Server закрывается сразу после запуска, скорее всего причина в проблемах с учетной записью службы. Первым делом нужно проверить, верно ли хотя бы одно из утверждений относительно учетной записи службы:

- учетная запись удалена;
- учетная запись заблокирована;
- учетная запись отключена;
- истек срок пароля.

Если учетная запись службы была удалена, надо попросить системного администратора воссоздать учетную запись, а затем сбросить учетную запись для запуска службы на вкладке **Вход** (Log On) окна свойств службы SQL Server (рис. 12-11).



Рис. 12-11. Учетная запись службы

Если учетная запись отключена, надо, чтобы системный администратор разблокировал ее, после чего следует перезапустить службу SQL Server. После перезапуска надо совместно с системным администратором выяснить, почему учетная запись службы была удалена или отключена и предусмотреть процедуры, предотвращающие возникновение такой ситуации в будущем.

Если истек срок пароля, системный администратор должен отключить срок замены пароля у этой учетной записи. После этого надо изменить пароль, обновить пароль на вкладке **Вход** (Log On) окна свойств службы SQL Server и перезапустить службу.

Если учетная запись службы была заблокирована, надо немедленно изменить пароль. Перед разблокировкой учетной записи службы, сетевой и системный администраторы должны выполнить тщательную проверку защиты сети. Поскольку учетная запись службы никогда не используется ни для чего кроме

работы службы, блокировка учетной записи службы означает, что кто-то пытался использовать учетную запись для других целей в нарушение политики безопасности. Нужно локализовать человека или приложение, которое пыталось использовать учетную запись службы и позаботиться об устранении бреши в защите, прежде чем разблокировать учетную запись и запустить резервную копию службы SQL Server. Хотя может показаться странным оставлять SQL Server недоступным, пока не изолирована причина блокировки учетной записи службы, но если человек или приложение смогли сознательно или случайно «взломать» пароль, то запуск резервной копии SQL Server до устранения бреши в защите предоставит злоумышленнику доступ ко всем данным экземпляра.

Если учетная запись службы проходит все проверки, тогда, скорее всего, проблема в разрешениях.

При запуске SQL Server первой в оперативный режим переводится главная база данных. После этого пересоздается база данных tempdb. Создание базы данных tempdb иницирует запись в папку или папки, где хранятся данные и журналы базы данных tempdb. Если SQL Server не удается обновить базу данных tempdb, служба закрывается. Ниже приведены стандартные причины неспособности SQL Server обновить базу данных tempdb:

- у учетной записи службы SQL Server недостаточно разрешений;
- нужная папка не существует или недоступна.

Если нужная папка не существует, перед перезапуском экземпляра SQL Server надо ее создать или вернуть дисковый том базы данных tempdb в оперативный режим. Если дисковый том вернуть в оперативный режим нельзя или причина неполадок tempdb заключается в ошибке конфигурации, можно запустить SQL Server в однопользовательском режиме, используя параметр запуска *-m*, изменить местоположение tempdb и перезапустить экземпляр в обычном режиме.

Если папка доступна, нужно поискать следующую последовательность событий в журнале событий **Приложение**:

- запуск службы;
- ошибка активации устройства или ошибка «файл не найден»;
- завершение работы службы.

Эта последовательность событий указывает, что у учетной записи службы SQL Server нет достаточных разрешений для доступа к данным или журналу главной базы данных или tempdb. Переименуйте файлы данных и журналов tempdb и перезапустите экземпляр — если в папке базы данных tempdb появится файл по имени Tempdb.mdf нулевого размера, то проблема в файлах данных или журналов tempdb. После устранения проблем с разрешениями, SQL Server запускается, как обычно.

ПРИМЕЧАНИЕ Ошибки активации устройств

Ошибка активации устройства означает, что SQL Server не смог обратиться к данным или журналу базы данных. Ошибки активации устройства по отношению к главной базе данных и tempdb делают невозможным запуск экземпляра, тогда как ошибки активации устройства для любой другой базы данных делают недоступной только эту базу данных. Нужно внимательно диагностировать любые ошибки активации устройств, потому что причина либо в сбое

дисковой подсистемы, либо в том, что администратор некорректно завершает работу системы хранения, в то время, когда она используется.

Параметры запуска

Ошибка активации устройства, связанная с главной базой данных, инициирует следующую последовательность событий в журнале событий **Приложение**:

- запуск службы;
- ошибка активации устройства или ошибка «файл не найден»;
- завершение работы службы.

Стандартные причины ошибки активации устройства во время загрузки SQL Server:

- у учетной записи службы нет достаточных разрешений;
- хранилище недоступно;
- неверно изменены параметры запуска.

Прежде всего надо убедиться, что хранилище находится в оперативном режиме, существуют папки и файлы главной базы данных, а учетная запись службы имеет разрешения на доступ к этим папкам. Если с разрешениями, файлами и папками все нормально, надо проверить параметры запуска службы (рис. 12-12).



Рис. 12-12. Параметры запуска

Параметры запуска службы можно задать непосредственно, щелкнув раскрывающийся список и задав параметры настройки (рис. 12-13).

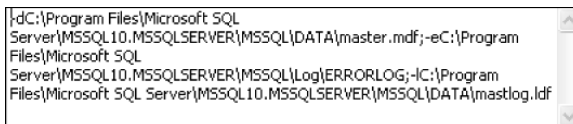


Рис. 12-13. Редактирование параметров запуска

Параметр *-d* перечисляет местоположение и имя главного файла данных. Параметр *-l* перечисляет местоположение и имя файла главного файла журна-

ла транзакций. Параметр *-e* перечисляет имя и местоположение журнала ошибок SQL Server. Чаще всего параметры запуска изменяют, добавляя флажки трассировки с помощью параметра *-T*. Имейте в виду, что, не соблюдая исключительную осторожность и аккуратность при изменении параметров запуска, можно легко нарушить нормальную загрузку экземпляра. Внимательно посмотрев на рис. 12-11, вы увидите, что параметры запуска отделяются точкой с запятой (;). Естественно желание ввести пробел после точки с запятой. Если это сделать, SQL Server будет интерпретировать точку с запятой и все, что следует после нее, как часть предыдущего параметра. В результате SQL Server посчитает, что имя главного журнала транзакций не *<имя_каталога\mastlog.ldf>*, а *<имя_каталога\masterlog.ldf>; -T<флажок_трассировки>*. Поэтому если после изменения параметров запуска экземпляр не запускается, надо проверить, не появился ли в параметрах запуска неуместный пробел.

Если папка или папки, содержащие данные и журналы главной базы данных, доступны и файлы существуют, прежде всего надо проверить, есть ли у учетной записи службы SQL Server разрешения на чтение и запись в эту папку или папки. Если разрешения есть, но ошибка активации устройства в файлах главной базы данных не исчезает, значит главная база данных повреждена и восстановить ее можно только, воспользовавшись установщиком SQL Server. Чтобы восстановить поврежденный экземпляр SQL Server, надо запустить установщик SQL Server, перейти на страницу **Обслуживание** (Maintenance) и запустить Мастер восстановления (Repair Wizard).

ПРИМЕЧАНИЕ На экзамене

При подготовке к экзамену надо сосредоточиться на самых популярных ситуациях, в которых возникают ошибки; большинство из них связаны с разрешениями безопасности. Вы должны знать, какие утилиты использовать для диагностирования ошибки, как восстановить главную базу данных SQL Server 2008, используя новый Центр установки SQL Server (SQL Server Installation Center), хотя в нем и используется привычный установщик SQL Server. На экзамене очень часто встречаются вопросы, проверяющие знание «нового» или «улучшенного» способа выполнения операции в текущей версии SQL Server по сравнению с предыдущей версией.

ВНИМАНИЕ! Системные базы данных

В каждом экземпляре SQL Server есть четыре системных базы данных: master, model, msdb и tempdb. Если настроена репликация, есть еще база данных distribution. В SQL Server есть еще шестая системная база данных, которая впервые появилась в SQL Server 2005 и называется mssqlsystemresource. Она содержит большую часть кода хранимых процедур, функций, динамических административных представлений, а также другой код, поставляемый в составе SQL Server. База данных mssqlsystemresource критически важна для работы SQL Server, и если ее нет или она недоступна SQL Server не запускается. К сожалению, эта база данных скрыта, искать ошибки активации устройства, связанные с базой данных mssqlsystemresource следует так же, как для баз tempdb и master.

Контрольные вопросы

1. Что означает ошибка активации устройства?
2. Ошибки в каких трех базах данных не позволяют запустить SQL Server?

Ответы на контрольные вопросы

1. Ошибка активации устройства означает, что SQL Server не может найти или получить доступ к файлам данных или журнала.
2. Ошибки в базах данных master, tempdb и mssqlsystemresource могут не позволить запустить экземпляра. Ошибки во всех других базах данных делают недоступной до устранения неполадки только проблемную базу данных.

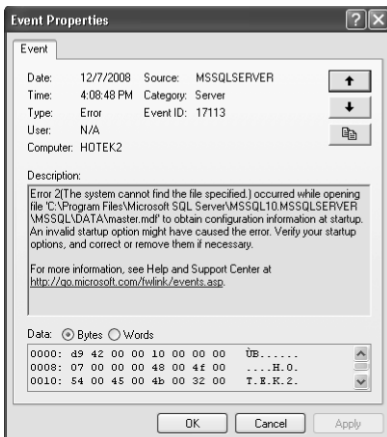
Практикум. Устранение неполадок запуска служб

Вы устраните проблему с запуском SQL Server, связанную с неспособностью SQL Server найти файлы главной базы данных.

Задание 1. Изменение папки

Вы измените папку для хранения файлов главной базы данных.

1. Откройте **Диспетчер конфигурации SQL Server** (SQL Server Configuration Manager) и выберите узел **Службы SQL Server** (SQL Server Services).
2. Остановите службу **SQL Server**.
3. Откройте Проводник и перейдите к папке, содержащей файлы master.mdf и mastlog.ldf (обычно это папка *Microsoft SQL Server\MSSQL10.<имя_экземпляра>\MSSQL\Data*).
4. Переименуйте файл **master.mdf** на **master.mdf2**.
5. Переименуйте файл **mastlog.ldf** на **mast-log.ldf2**.
6. Попытайтесь запустить экземпляр SQL Server.
7. Просмотрите журнал **Приложение** (Application) и журнал ошибок SQL Server, как показано ниже.



Задание 2. Устранение ошибки активации устройств

Вы устраните ошибку активации устройства, созданную в задании 1.

1. Щелкните правой кнопкой службу **SQL Server** в Диспетчере конфигурации SQL Server и выберите **Свойства** (Properties).
2. Перейдите на вкладку **Дополнительно** (Advanced).
3. Измените имена файлов в параметрах журнала и главной базы данных.
4. Щелкните **ОК**.
5. Запустите экземпляр.
6. Просмотрите в журнале «Приложение» и журнале ошибок SQL Server сообщения о нормальном запуске SQL Server.

Резюме занятия

- Диспетчер конфигурации SQL Server служит для настройки и управления службами, протоколами и собственным клиентом SQL.
- Стандартная причина сбоев при запуске служб — отсутствие необходимых разрешений.
- Ошибка активации устройства означает, что SQL Server не может найти или выполнить запись в файл данных или журнала.
- При изменении параметров запуска службы SQL Server нужно следить, чтобы не вставлять пробел после точки с запятой, которая служит разделителем параметров.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Вы администратор базы данных в компании Blue Yonder Airlines и вам сообщили, что основное приложение заказа билетов перестало работать и его не удастся переподключить к базе данных. Вы пытаетесь подключиться к серверу SQL Server, но он недоступен. Обнаружилось, что служба остановилась, а в журналах ошибок большое число сообщений об ошибках активации устройства. Какова наиболее вероятная причина неполадки?
 - A. Кто-то удалил файлы базы данных с информацией о заказе билетов.
 - B. Потерпело сбой дисковое хранилище, где располагаются базы данных master или tempdb.
 - C. Потерпело сбой дисковое хранилище, где располагается база данных о заказе билетов.
 - D. Заблокирована учетная запись службы SQL Server.

Занятие 5. Диагностирование неполадок оборудования

Большинство неполадок SQL Server связано с организационными процессами или системой безопасности, но бывают и аппаратные сбои. Вы узнаете, как диагностировать причины проблем с оборудованием, чтобы знать, какие компоненты нужно заменить.

Изучив материал этого занятия, вы сможете:

- ✓ диагностировать неполадки, связанные со сбоями оборудования.

Продолжительность занятия — 20 минут.

Дисководы

Дисковод — один из последних компонентов оборудования, в котором остались движущиеся части. Дисководы содержат очень мелкие детали с очень строгими допусками, причем эти детали движутся с очень высокими скоростями и испытывают огромные механические нагрузки напряжениям, поэтому неудивительно, что дисковод — самая частая причина сбоя оборудования.

Отказ одного диска обычно не создает проблем, так как обычно все базы данных располагаются на дисковой системе с RAID-массивом, обеспечивающим избыточность. Однако отказ нескольких дисков может сделать недоступным целый том, а также базы данных или весь экземпляр SQL Server.

Первый признак сбоя дисковой подсистемы — ошибки в журнале **Система** (System) или в системе журналов сети хранилищ SAN или NAS. Если ошибки настолько серьезные, что приводят к недоступности целого тома или массива, вы немедленно увидите ошибки активации устройства в журнале ошибок SQL Server, а также в журнале **Приложение**.

Обнаружив ошибки активации устройства и определив, что хранилище, в котором располагаются данные или журналы, недоступно, нужно немедленно уведомить администратора корпоративного хранилища.

Если хранилище баз данных подключено непосредственно, можно использовать узел **Управление дисками** (Disk Management) консоли **Управление компьютером** (Computer Management), чтобы определить состояние дисковых томов (рис. 12-14). Ошибки непосредственно подключенного хранилища можно диагностировать и даже устранять, используя утилиту командной строки CHKDSK.

ВНИМАНИЕ! Массивы NAS и SAN

Если базы данных размещаются в массивах SAN или NAS, для диагностики и устранения любых неполадок дисков нужно всегда использовать специализированные утилиты для управления массивом.

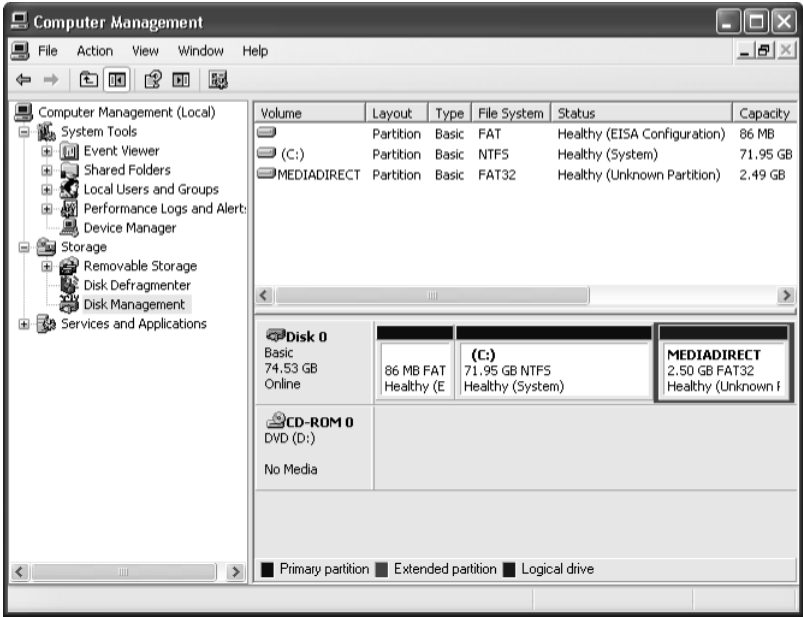


Рис. 12-14. Управление непосредственно подключенными дисками

Память и процессоры

Полные отказы оперативной памяти или процессора весьма редки. Чаще возникают спорадические ошибки, которые происходят случайно, и экземпляр SQL Server вроде бы продолжает работу, не испытывая ошибок. Иногда появляются дампы стека в журнале ошибок SQL Server или папке, где хранится журнал ошибок SQL Server.

Когда в SQL Server наблюдается серьезная ошибка, генерируется дамп стека. Если есть возможность восстановиться после ошибки, SQL Server продолжает работу, в противном случае экземпляр останавливается. Поскольку очень редко какой-либо пользователь работает с консоли машины, на которой работает SQL Server, вряд ли кто-то видит синий экран останова системы. Поэтому первый признак возможных неполадок памяти или процессора — появление дампов стека.

При первом же появлении дампа стека в журнале ошибок SQL Server, журнале **Приложение** или **Система**, или, если в папке, где хранится журнал ошибок SQL Server, создается файл с расширением .mdmp, нужно немедленно уведомить оператора.

Для диагностики неполадок памяти или процессора нужно использовать диагностические утилиты, поставляемые производителем этого оборудования.

ПРИМЕЧАНИЕ На экзамене

На экзамене вы должны знать самые стандартные ошибки, связанные со сбоем аппаратных компонентов.

Контрольные вопросы

1. Какие ошибки наблюдаются, когда дисковое хранилище с базой данных становится недоступным?
2. Какие ошибки наблюдаются в случае неполадок памяти или процессора?

Ответы на контрольные вопросы

1. Когда дисковый том, на котором размещаются базы данных, становится недоступным, на SQL Server начинают регистрироваться ошибки активации устройства.
2. В случае неполадок памяти или процессора возникают ошибки типа «останов системы» (STOP). Если ошибка памяти возникает при загрузке компьютера, происходит ошибка POST. Обе эти ошибки сопровождаются появлением синего экрана с дополнительной диагностической информацией.

Резюме занятия

- Серьезный отказ дисковой системы, приводящий к ее недоступности, вызывает ошибку активации устройств хранения журнала, а соответствующие базы данных становятся недоступными.
- Ошибки памяти и процессора в большинстве случаев возникают спонтанно, и обычно после них выполняется дамп стека.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Руководство компании Humongous Insurance пригласило вас, чтобы вы оценили инфраструктуру SQL Server, дали свои рекомендации и управляли проектами по совершенствованию промышленной среды. В процессе оценки выяснилось, что файлы баз данных хранятся на многих дисках и точках подключения. Файлы данных и журналов совмещены с файлами резервных копий. Системные базы данных размещаются на тех же дисках, что и пользовательские базы данных. Первый запланированный вами проект состоит в переносе всех системных баз данных, чтобы они хранились отдельно от пользовательских баз данных. Вы также хотите на каждом экземпляре переместить базу данных tempdb в выделенное хранилище, потому что очень много плохо написанных запросов выполняют перенос огромных объемов данных через tempdb. Поддержку осуществляет приходящий администратор базы данных; он проверил процедуры несколько раз в тестовой среде. Вам сообщают, что после переноса баз данных первый экземпляр не запускается. Вы убедились, что разрешения учетной записи службы на папке, содержащей файлы базы данных master, настроены правильно, файлы базы данных mas-

тер находятся в правильном месте, а параметры запуска правильные. Какова наиболее вероятная причина неполадки?

- A. Повреждена главная база данных.
- B. Повреждена база данных mssqlsystem.
- C. У учетной записи службы нет разрешений на доступ к папке, содержащей файлы базы данных tempdb.
- D. На сервере сбоит модуль памяти.

Занятие 6. Устранение неполадок, связанных с кратковременной и обычной блокировкой

Как шутят администраторы, «SQL Server работал бы идеально, если бы не было пользователей». Однако если б не пользователи, работающие с данными, база данных была бы ненужной. Также, если бы нужно было только читать данные из базы, не требовалось бы волноваться об одновременной работе многих пользователей с одними и теми же данными. Поскольку данные в базе данных должны быть доступны для чтения и для изменения многими пользователями, нужен механизм управления параллельным доступом, который бы обеспечивал целостность данных. В этом занятии рассказывается, как механизм блокировки управляет доступом в SQL Server и как устранять неполадки пересекающихся процессов, в которых возникают обычные блокировки и взаимоблокировки.

ПРИМЕЧАНИЕ Блокировки

Подробнее о разных видах блокировки и уровнях изоляции см. книгу Калена Делани (Kalen Delaney) «Microsoft SQL Server 2008 Internals» (Microsoft Press, 2009).

Изучив материал этого занятия, вы сможете:

- ✓ обнаруживать заблокированные процессы;
- ✓ уничтожать процессы;
- ✓ просматривать граф взаимоблокировки.

Продолжительность занятия — 20 минут.

Блокировки

SQL Server использует механизм блокировки для обеспечения целостности данных при доступе к ним многих пользователей. Внутренний процесс, носящий название Диспетчер блокировок (Lock Manager) определяет, какую блокировку применить в том или ином случае, как долго сохранять блокировку, и определяет, когда позволять процессам изменять данные так, чтобы результаты операций чтения были непротиворечивыми.

В SQL Server семь режимов блокировки и три типа блокировки. В большинстве ситуаций приходится иметь дело только со следующими тремя режимами блокировки:

- общий;
- монопольный;
- обновление.

Существуют три типа блокировки — блокировка строки, страницы и таблицы. Область действия блокировки можно ограничить сеансом, транзакцией или курсором.

Общая блокировка создается операциями чтения, чтобы запретить изменение данных во время чтения. Поскольку операции чтения не могут нарушить целостность данных, у одного ресурса может одновременно быть много общих блокировок. Монопольная блокировка применяется к ресурсу, который изменяется, на все время изменения. Как понятно из названия, в каждый момент времени у ресурса может быть лишь одна монопольная блокировка, и все остальные процессы, которым нужен ресурс, должны ожидать завершения монопольной блокировки. Блокировка обновления — это смесь общей и монопольной блокировок. Хотя блокировка обновления применяется при любом обновлении, такие блокировки могут применяться при любой операции, требующей сначала найти подлежащие обновлению данные. Процедура блокировки обновления начинается с применения общей блокировки к ресурсу, пока не будет найдена порция данных, подлежащая обновлению, после чего общая блокировка меняется на монопольную на все время обновления данных.

Каждый режим блокировки может применяться к строке, странице или таблице. Диспетчер блокировок определяет тип блокировки на основании очень жесткого ограничения ресурсов, которое обычно называется правилом двух процентов, цель которого — сокращение до минимума числа блокировок, которые нужно создавать и управлять, потому что каждая блокировка потребляет память. Если SQL Server решает, что потребуется доступ к больше чем двум процентам строк на странице, такая страница блокируется. Аналогично, если требуется доступ к более чем двум процентам страниц в таблице, такая таблица блокируется.

Определяя тип блокировки, Диспетчер блокировок использует статистику распределения, такую же, что применяется Оптимизатором запросов. Поскольку статистика распределения не всегда точна или не всегда доступна, в Диспетчере блокировок есть механизм, который называется укрупнение блокировок (lock escalation) и позволяет повышать уровень блокировки. SQL Server может укрупнить блокировку строки до блокировки таблицы или блокировку страницы — до блокировки таблицы.

ПРИМЕЧАНИЕ Укрупнение блокировки

Очень часто неправильно считают, что SQL Server укрупняет блокировку строки до блокировки страницы — блокировка строки укрупняется только до блокировки таблицы.

Уровни изоляции транзакции

Уровни изоляции определяют, как SQL Server обрабатывает транзакции, а также длительность блокировок. В SQL Server пять уровней изоляции, они перечислены в таблице 12-4.

Табл. 12-4. Уровни изоляции транзакций

Уровень изоляции	Описание
READ UNCOMMITTED	Разрешено чтение не зафиксированных данных. Хотя монополюсная блокировка блокирует другую монополюсную блокировку, все операции чтения игнорируют монополюсную блокировку
READ COMMITTED	Это уровень изоляции по умолчанию в SQL Server. Монополюсная блокировка блокирует как общие, так и монополюсные блокировки. Общая блокировка блокирует монополюсную блокировку. Общая блокировка освобождается сразу по завершении чтения
REPEATABLE READ	Монополюсные блокировки блокируют и общие и монополюсные блокировки. Общие блокировки блокируют монополюсные блокировки. Общие блокировки сохраняются на все время выполнения транзакции
READ SERIALIZABLE	Применяются такие же ограничения, что и в уровень изоляции REPEATABLE READ. Кроме того, нельзя вставить новую строку в набор ключей, в текущий момент заблокированный транзакцией. Блокировки сохраняются на все время выполнения транзакции
SNAPSHOT	В этом уровне изоляции используется управление версиями строки для предотвращения взаимной блокировки общих и монополюсных блокировок в процессе обеспечения целостности данных. Операция чтения извлекает сведения о версии строки до начала операции изменения данных

Блокировка процессов

В своей работе Диспетчер блокировок руководствуется правилом ФИФО (FIFO), то есть первый попавший в очередь обслуживается первым. Каждый выполняющий команду процесс должен применить блокировку. Затребованные блокировки Диспетчер блокировок ставит в очередь в порядке их поступления. Если запрошенный ресурс не заблокирован или если его блокировка не противоречит запрашиваемой блокировке, Диспетчер блокировок удовлетворяет запрос на блокировку. В случае конфликта блокировок, например наличия запроса на общую блокировку строки, которая монополюсно заблокирована другим ресурсом, запрос не удовлетворяется, а Диспетчер блокировок размещает запрос на блокировку в очереди наряду со всеми остальными запросами того же ресурса до освобождения ресурса.

Блокирование — это наличие конкурирующих блокировок одного ресурса. Второй процесс не получает блокировки, пока первый ресурс не снимет конкурирующую блокировку. Выполнение заблокированного процесса останавливается до получения нужных блокировок.

Блокирование — нормальное явление в любой базе данных, поддерживающей одновременную работу многих пользователей; проблема возникает, когда блокирование вообще или долгое время не прекращается.

Любой живущий в большом городе знаком с блокированием. В каждый день в часы пик масса автомобилей пытается воспользоваться единственным ресурсом

сом — дорогой. Пока конфликтов нет, все автомобили быстро перемещаются по дороге и добираются к цели маршрута. Однако в случае ДТП, перекрывающего все полосы в одном направлении, движение останавливается и люди раздражаются. Движение невозможно восстановить, пока поврежденные в ДТП автомобили не уберут с проезжей части; со временем нормальное движение восстанавливается. Такая же ситуация наблюдается в базах данных. Если блокирование длится долго или если процессы постоянно блокируют ресурс, падает производительность приложения.

Узнать, заблокирован ли процесс, можно с помощью представления `sys.dm_exec_requests`. У заблокированных процессов в столбце `blocking_session_id` содержится ненулевое значение.

Решив, что процесс вызывает конфликты в базе данных, член фиксированной роли `sysadmin` может принудительно завершить процесс следующей командой (где `SPID` — системный идентификатор процесса блокирующего сеанса):

```
KILL <spid>
```

При принудительном завершении процесса:

- все открытые транзакции откатываются;
- клиенту возвращается сообщение;
- вносится запись в журнал ошибок SQL Server;
- вносится запись в журнал событий Windows **Приложение** (Application).

Взаимные блокировки

SQL Server обеспечивает корректное исполнение потока транзакций даже в условиях блокирования операций. Заблокированный процесс ожидает, пока не освободится конкурирующая блокировка, после чего возобновляет выполнение. Однако можно создать ситуацию, когда блокировку вообще невозможно освободить. Когда два процесса блокируют друг друга так, что никакой процесс не может освободить ресурс, возникает взаимная блокировка.

Для образования взаимной блокировки нужно не менее двух процессов, каждый из которых должен выполнять операцию изменения данных. Так как допускается много одновременных общих блокировок одного ресурса, невозможно создать взаимную блокировку, используя процесс, который только читает данные.

Взаимная блокировка — временная неполадка, возникающая при следующей последовательности событий:

1. `SPID1` получает монопольную блокировку строки `RowA`.
2. `SPID2` получает монопольную блокировку строки `RowC`.
3. `SPID1` пытается получить общую блокировку строки `RowC` и блокируется монопольной блокировкой.
4. `SPID2` пытается получить общую блокировку строки `RowA` и также блокируется монопольной блокировкой.

Поскольку каждый из процессов ожидает, пока другой процесс не освободит монопольную блокировку, Диспетчер блокировок оказывается перед неразрешимой задачей. В такой ситуации Диспетчер блокировок обнаруживает взаимную блокировку и автоматически завершает один из процессов.

К сожалению, в качестве «жертвы» выбирается процесс с наименьшим суммарным временем работы на SQL Server. Вполне возможно, что принудительно будет завершен критически важный процесс, потому что вы не поддерживаете подключение, выполняя несколько запросов в рамках сеанса. Алгоритм выбора завершаемого процесса изменить нельзя.

При обнаружении взаимной блокировки клиенту возвращается сообщение об ошибке 1205, а взаимная блокировка регистрируется в журнале ошибок SQL Server. Кроме того, можно задействовать приложение SQL Server Profiler для записи трассировки взаимной блокировки, что позволяет исследовать причину взаимной блокировки в графическом интерфейсе.

ПРИМЕЧАНИЕ Рекомендации по устранению взаимных блокировок

Взаимные блокировки в основном вызываются неудачной синхронизацией. Если бы запросы, выполняемые в двух сеансах, завершались быстрее, взаимной блокировки можно было бы избежать. С другой стороны, если бы один из процессов начался чуть позже, возможно взаимная блокировка вообще не возникла. Поскольку взаимная блокировка — переходное состояние конфликта блокировок, приложения надо писать так, чтобы они «умели» обнаруживать ошибки 1205 и немедленно повторять транзакцию, потому что очень маловероятна взаимная блокировка процесса при повторной попытке выполнения команды.

ПРИМЕЧАНИЕ На экзамене

Вы должны знать, какие блокировки возможны и как укрупнение блокировок может приводить к блокированию или взаимной блокировке. Вы также должны знать, как устранять блокирование и взаимные блокировки.

Контрольные вопросы

1. Каковы самые популярные режимы и типы блокировок?
2. Как возникает взаимная блокировка?

Ответы на контрольные вопросы

1. Три стандартных режима блокировки — общий, монополярный и режим обновления. Существует три типа блокировки — блокировка строки, страницы и таблицы.
2. Для возникновения взаимной блокировки требуется не менее двух процессов, пытающихся изменить данные. Каждый из процессов получает монополярную блокировку ресурса, после чего пытается получить общую блокировку этого же ресурса, который в свою очередь монополярно заблокирован другим процессом.

Практикум. Устранение взаимной блокировки

Вы настроите трассировку, чтобы собрать информацию о взаимной блокировке.

1. В окне приложения SQL Server Profiler выберите **File/New Trace** и подключитесь к экземпляру.

2. Задайте имя трассировки, шаблон и выберите вариант сохранения результатов в файле, как показано ниже.

3. Перейдите на вкладку **Events Selection**.
4. Отметьте флажком событие **Deadlock graph** в категории **Locks**, как показано ниже.

Events	EventSequence	IsSystem	LoginName	LoginSid	SPID	ServerName	SessionLoginName	StartTime
Locks								
☒ Deadlock graph	☒	☒	☒	☒	☒	☒	☒	☒

5. Щелкните **Run**.
6. Откройте два окна запросов и измените контекст на базу данных AdventureWorks.
7. В первом окне выполните код:

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE
GO
```

```
BEGIN TRANSACTION
UPDATE Production.Product
SET ReorderPoint = 600
WHERE ProductID = 316
```

8. Во втором окне запроса выполните код:

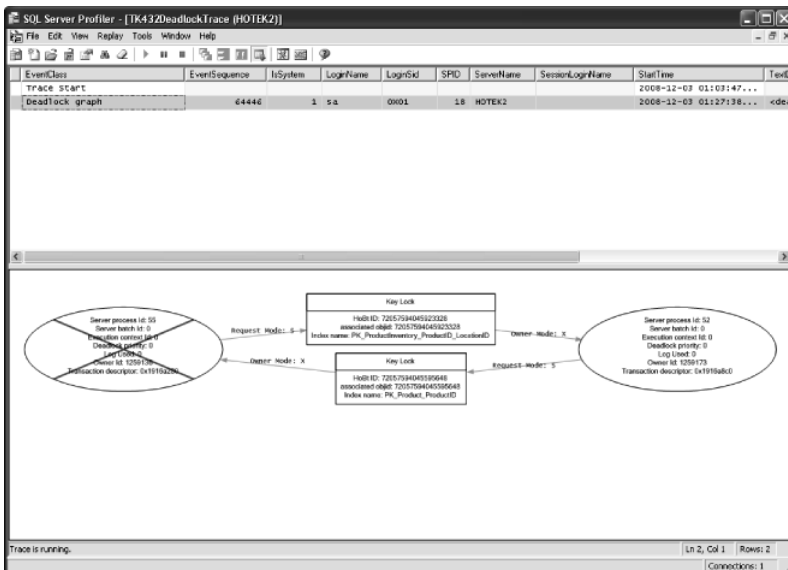
```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE
GO
BEGIN TRANSACTION
UPDATE Production.ProductInventory
SET Quantity = 532
WHERE ProductID = 316
AND LocationID = 5
```

```
SELECT Name, ReorderPoint
FROM Production.Product
WHERE ProductID = 316
```

9. Вернитесь в первое окно запроса и выполните следующий код, но следите, чтобы не выполнить транзакцию фиксации:

```
SELECT ProductID, LocationID, Shelf, Bin, Quantity, ModifiedDate
FROM Production.ProductInventory
WHERE ProductID = 316
AND LocationID = 5
```

10. Ознакомьтесь с результатами в приложении SQL Server Profiler. Пример показан ниже.



Резюме занятия

- Диспетчер блокировок управляет блокировками, которые в SQL Server используются для обеспечения целостности данных при изменении данных многими пользователями одновременно.
- Если один из процессов получил монопольную блокировку, никакому другому процессу не предоставляется общая блокировка для чтения или монопольная блокировка для изменения данных, пока не будет снята исходная монопольная блокировка. Если процесс выполняется с уровнем изоляции READ UNCOMMITTED, при операции чтения монопольные блокировки игнорируются.
- Для принудительного завершения процесса можно использовать команду KILL.
- Взаимная блокировка — промежуточное состояние, в котором два процесса получают конкурирующие блокировки так, что ни один из них не в состоянии продолжить выполнение. Диспетчер блокировок выдает ошибку 1205 и принудительно завершает один из взаимно заблокированных процессов.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Какие из следующих средств используются для нахождения заблокированных процессов? (Выберите все подходящие варианты.)
 - A. Представление sys.dm_exec_sessions view.
 - B. Представление sys.dm_exec_requests view.
 - C. Представление sys.dm_os_waiting_tasks view.
 - D. Хранимая процедура sp_who2 system.

Закрепление материала главы

Для закрепления знаний и навыков, полученных при изучении этой главы, выполните следующее:

- изучите резюме главы;
- повторите основные термины, использованные в этой главе;
- выполните задания по темам этой главы;
- сделайте предложенные упражнения;
- пройдите пробный экзамен.

Резюме главы

- Приложение SQL Server Profiler — утилита, предоставляющая графический интерфейс для взаимодействия с SQL Trace API.
- Системный монитор позволяет записывать показания счетчиков производительности, которые можно сопоставлять с результатами SQL Trace в ин-

терфейсе приложения SQL Server Profiler, которое обеспечивает контекст состояния оборудования для зафиксированных событий.

- Сбои возможны на многих уровнях: оборудование, учетные записи служб и конфигурация. Самая стандартная причина сбоя запуска службы — неполадки с разрешениями. Самая популярная причина отказа активных баз данных — нехватка дискового пространства.
- Взаимные блокировки — временные неполадки, которые обусловлены наличием конкурирующих блокировок. Приложение должно «уметь» обнаруживать их и повторно инициировать потерпевшую сбой команду.

Основные термины

Понятны ли вам значения приведенных здесь основных терминов? Чтобы проверить свои ответы, загляните в глоссарий на прилагаемом компакт-диске.

- Журнал счетчиков ~ counter log.
- Взаимная блокировка ~ deadlock.
- Уровень изоляции ~ isolation level.
- Увеличение блокировки ~ lock escalation.
- SQL Trace.
- Событие трассировки ~ trace event.

Упражнения

Сейчас вы примените на практике знания и навыки, полученные при изучении этой главы. Ответы на вопросы см. в разделе «Ответы» в конце книги.

Упражнение. Разработка стратегии автоматизации для компании Coho Vineyard

Исходные данные

Основные сведения о компании

Компания Coho Vineyard была основана в 1947 г. как частный семейный винный завод. Вина, производимые Coho Vineyard, удостоивались многих наград и пользовались популярностью, что позволило компании значительно расширить производство. Через несколько лет в рамках стратегии по расширению компании были приобретены несколько существующих винных заводов. Сейчас компания владеет 16 винными заводами — 9 в штатах Вашингтон, Орегон и Калифорния, а остальные 7 в штатах Висконсин и Мичиган. В компании работает 532 человека, из которых 162 — в штаб-квартире, где размещаются критически важные для бизнеса серверы. В компании трудится 122 мобильных торговых агента, которым требуется доступ к актуальной информации об имеющихся товарах.

Планируемые изменения

До настоящего времени у каждого из 16 заводов Coho Vineyard был собственный веб-сайт, размещенный на локальном веб-сервере. Компания Coho Vineyard планирует объединить веб-ресурсы всех заводов, чтобы клиенты могли поку-

пать продукцию любого из 16 заводов в едином онлайн-магазине. Все данные этого веб-сайта будут храниться в базе данных в штаб-квартире.

Пока проект консолидации данных не завершен, для обслуживания торговых агентов в конце рабочего дня данные о запасах с каждого завода пересылаются в штаб-квартиру. Репликация слиянием настроена так, чтобы торговые агенты получали локальные копии данных по клиентам, складским запасам и заказам.

Существующая инфраструктура данных

Базы данных

Сейчас каждый винный завод поддерживает собственную базу данных, где хранит всю бизнес-информацию. В конце месяца вся информация переносится в штаб-квартиру и загружается в базы данных, перечисленные в табл. 12-5.

Табл. 12-5. Базы данных Coho Vineyard

База данных	Объем
<i>Customer</i> (данные о клиентах)	180 Мб
<i>Accounting</i> (бухгалтерские данные)	500 Мб
<i>HR</i> (данные о кадрах)	100 Мб
<i>Inventory</i> (складские запасы)	250 Мб
<i>Promotions</i> (сведения о поощрениях)	80 Мб

В результате выполнения работ по консолидации баз данных будет создана новая база данных Order, которая будет работать с хранилищем данных нового веб-магазина. В процессе своей ежедневной работы сотрудники будут периодически обращаться к базе данных Order, используя новое разработанное собственными силами веб-приложение.

База данных HR содержит конфиденциальную информацию и защищена с использованием TDE (Transparent Data Encryption). Кроме того, данные в таблице Salary (данные о зарплатах) зашифрованы с использованием сертификатов.

Серверы баз данных

Все базы данных в центральном офисе хранятся в SQL Server 2008 Enterprise на одном сервере DB1 под управлением Windows Server 2003 Enterprise.

Руководитель ИТ-отдела планирует приобрести новый компьютер для замены сервера DB1, поскольку пользователи жалуются на эпизодические проблемы с производительностью. Кроме этого несколько заводов сообщали об ошибке активации устройства и даже о нескольких случаях синих экранов при запуске на сервере их локальных экземпляров SQL Server.

Бизнес-требования

Вам необходимо разработать решение архивирования баз данных *Customer* и *Order*. Стратегия архивирования должна обеспечивать хранение данных базы *Customer* в течение 6 лет.

В процессе подготовки БД *Order* для архивирования вы создали секционированную таблицу *Order.Sales*, состоящую из двух секций: в первой хранится информация о продажах за текущий, а во второй — за предыдущий месяц. За-

казы, размещенные еще раньше, должны перемещаться в другую секционированную таблицу — *Order.Archive*, после чего в секции 1 содержатся все архивные данные, а секция 2 остается пустой.

Новый процесс должен ежедневно, не позднее 4 часов утра загружать информацию о складских запасах со всех 16 заводов.

Четыре крупных клиента осуществляют заказы, используя XML-схему «Coho Vineyards» для EDI-транзакций (Electronic Data Interchange). EDI-файлы поступают в 5 часов вечера и должны обрабатываться и загружаться в БД *Customer*, *Accounting* и *Inventory*, каждая из которых содержит таблицы с данными, относящимися к размещаемым заказам. В настоящее время утилита импорта EDI-файлов представляет собой однопоточное приложение на C++, которому требуется от 3 до 6 часов на обработку всех файлов. Чтобы выполнить соглашение с клиентами об условиях обслуживания (Service Level Agreement, SLA), необходимо заканчивать обработку EDI-файлов до 17:30. К моменту завершения проекта по объединению EDI-утилиты должна загружать информацию в новую базу данных *Order*.

Сообщалось о многочисленных конфликтах на центральном сервере SQL Server во время импорта данных в процессе ночной консолидации. Требуется уменьшить количество конфликтов, либо совсем исключить их.

Требуется обеспечить создание резервных копий всех баз данных всех филиалов. В самом плохом случае считается приемлемым потеря данных за последние 5 минут. В случае аварии БД *Customer*, *Inventory*, *Promotions* и *Order* могут быть недоступны максимум 20 минут. В случае аварийного сбоя информация БД *Customer* и *Order* старше полугода может быть недоступна максимум 12 часов.

Ответьте на следующие вопросы:

1. Как выяснить причину проблем с производительностью?
2. Как снизить или вообще избавиться от проблем, связанных с блокированием в процессе консолидации данных в ночное время?
3. Как устранять неполадки, наблюдающиеся на винных заводах?

Рекомендуемые упражнения

Чтобы подготовиться к экзамену по темам этой главы, выполните следующие задания.

Настройка баз данных

Создание трассировки с помощью приложения SQL Server Profiler для диагностики проблем с производительностью и взаимными блокировками

- **Задание 1.** Создайте трассировку для получения графов взаимных блокировок и настройте автоматический запуск трассировки при запуске SQL Server.
- **Задание 2.** Создайте трассировку для записи статистики производительности запросов, которая может использоваться для определения базовых уровней нагруженности экземпляра.

Создание журнала счетчиков средствами Системного монитора для диагностики проблем с производительностью и взаимными блокировками, а также системных неполадок

- **Задание 1.** Создайте журнал счетчиков для записи показаний счетчиков оборудования и SQL Server, которые далее могут использоваться для определения базового уровня производительности машины.
- **Задание 2.** Создайте журнал счетчиков для записи событий, позволяющих обнаруживать ошибки, связанные с неполадками оборудования и нехваткой дискового пространства.

Пробный экзамен

На прилагаемом компакт-диске содержится пробный экзамен, обеспечивающий широкие возможности самоподготовки. Например, можно проверить, как усвоены темы этой главы либо все темы экзамена 70-432. Можно симитировать настоящий экзамен или выбрать режим обучения, когда после каждого вопроса приводятся правильные ответы с пояснениями.

ПРИМЕЧАНИЕ Пробный экзамен

Подробнее о пробном экзамене см. «Введение».

Оптимизация производительности

Занятие 1. Применение помощника по настройке ядра СУБД	336
Занятие 2. Использование Регулятора ресурсов	343
Занятие 3. Динамические административные представления и функции	352
Занятие 4. Использование хранилища данных о производительности	360

Целые книги, сотни интернет-трансляций, тысячи секций на конференциях, сотни семинаров, масса курсов и десятки тысяч страниц на сотнях веб-сайтов призваны помочь оптимизировать производительность Microsoft SQL Server. Авторы большинства этих ресурсов предполагают, что причины низкой производительности уже известны, и предлагают рецепты решения выявленных проблем. Но каждый, имевший дело с оптимизацией производительности SQL Server, знает, что самое сложное — найти причины низкой производительности. В главе 12 уже рассказывалось о двух инструментах — SQL Server Profiler и Системном мониторе, которые незаменимы при оптимизации производительности. В этой главе рассказывается об остальных инструментах SQL Server 2008, служащих для устранения неполадок производительности.

Темы экзамена

- Применение Регулятора ресурсов (Resource Governor).
- Использование помощника по настройке ядра СУБД.
- Сбор данных о производительности при помощи динамических административных представлений (Dynamic Management View, DMV).
- Использование приложения Performance Studio.

Прежде всего

Для изучения материалов этой главы необходимо иметь:

- установленный экземпляр Microsoft SQL Server 2008;
- подключенную базу данных AdventureWorks.

Практический совет*Майк Хотек*

Я часто слышу, что настройка производительности — это «искусство». Те, кто это говорят, обычно пытаются убедить вас, что настройка производительности является искусством, лишь для того, чтобы продать вам приложения или консалтинговые услуги. В конечном счете, всех нас учили, что у «творческих личностей» особый талант, которым обладают лишь немногие. Взять чистый лист холста, краски и кисть и нарисовать картину — это искусство. Взять молоток и долото, чтобы обтесать камень и создать статую — это тоже искусство. Но настройка производительности имеет столько же общего с искусством, сколько Остап Бендер с турецким султаном.

SQL Server — это набор команд, исполняемых компьютером. Код SQL Server определяет результаты, выдаваемые компьютером на основе получаемых данных. Все компьютеры основаны на двоичной логике, поэтому один и тот же набор исходных данных будет давать строго один и тот же результат. Программный код также контролирует входные данные, реагируя только на те, что считает верными. Пока ваши запросы подчиняются ограничениям компьютерной программы, они всегда выполняются. Способность настраивать производительность основывается только на знаниях — чем лучше вы понимаете правила и устройство компьютерного кода, лежащего в основе SQL Server, тем лучше будут ваши запросы и тем меньше они будут требовать различных ресурсов.

Из всей математики в настройке производительности используется только один принцип: самый короткий путь между двумя точками — прямая линия.

Если программный код считывает больше данных, чем требуется, приложение будет работать медленнее. Если коду для получения результата требуется два прохода по данным, он выполняется медленнее, чем код, которому достаточно одного прохода.

Оптимизация производительности всегда начинается с поиска программного кода, который заставляет SQL Server отклоняться от прямой линии к цели, требует более одного прохода по данным, либо использует больше данных, чем нужно для выполнения задачи. Обнаружив «неправильный» код, следует использовать свои знания SQL Server и изменить запрос так, чтобы он стал прямолинейным, использовал как можно меньше данных и выполнялся за один проход.

Занятие 1. Применение помощника по настройке ядра СУБД

Помощник по настройке ядра СУБД (Database Engine Tuning Advisor, DTA) служит для оценки степени соответствия запросов правилам оптимизатора запросов (Query Optimizer) и предоставляет рекомендации по повышению производительности. Здесь рассказывается, как создавать файл рабочей нагрузки,

а затем применять DTA для анализа рабочей нагрузки запроса, чтобы выяснить, какие есть возможности повышения производительности.

Изучив материал этого занятия, вы сможете:

- ✓ настраивать DTA для анализа рабочей нагрузки;
- ✓ сохранять рекомендации DTA.

Продолжительность занятия — 20 минут.

Помощник по настройке ядра СУБД

Помощник по настройке ядра СУБД, или DTA, использует выходные данные трассировки SQL. Сначала собирается информация трассировки запроса, которая будет анализировать DTA. Затем DTA считывает результаты трассировки и анализирует ее, сверяясь с базой данных. DTA предоставляет следующие рекомендации:

- добавить индексы;
- удалить индексы;
- секционировать таблицы;
- выровнять хранение таблиц.

Источником рабочей нагрузки DTA может быть файл трассировки, сценарий T-SQL или таблица с командами T-SQL. SQL Server Profiler собирает широкий круг событий, но DTA учитывает только следующие:

- RPC:Starting
- RPC:Completed
- SQL:Batch Starting
- SQL:Batch Completed

Анализ выполняется в четыре этапа:

1. Генерация рабочей нагрузки для анализа.
2. Запуск DTA и подключение к серверу SQL Server, содержащему базу данных, на которой будет анализироваться рабочая нагрузка.
3. Выбор рабочей нагрузки для использования.
4. Определение параметров оптимизации.

Познакомимся поближе с этапами анализа DTA и возможными вариантами. Запустите DTA, чтобы настроить сеанс анализа (рис. 13-1).

DTA выполняет анализ в рамках сеанса. Каждому сеансу нужно присвоить имя и сохранить, чтобы позже можно было изучить результаты. Сеансам следует давать уникальные имена, которые помогут впоследствии вспомнить, когда и для чего выполнялся анализ.

После присвоения имени следует выбрать параметры рабочей нагрузки. Чаще всего анализ выполняется на основе файла, содержащего либо результаты трассировки, либо команды T-SQL.

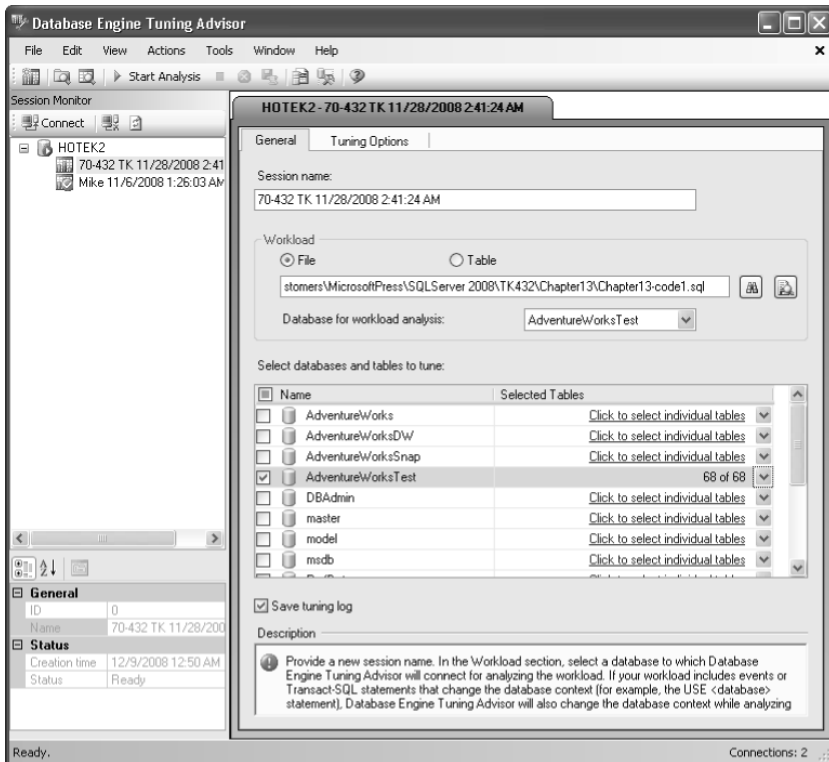


Рис. 13-1. Создание сеанса анализа DTA

ПРИМЕЧАНИЕ Автоматический анализ

Если вас не устраивает графический интерфейс DTA, можете воспользоваться утилитой командной строки `dta.exe`. Можно настроить трассировку, используя код, который может выполняться заданием Агента SQL Server. Задание может по окончании трассировки импортировать файл трассировки в таблицу. Поскольку DTA может использовать таблицу в качестве источника рабочей нагрузки, можно создать дополнительным шагом задания запустить DTA для анализа только что импортированных данных трассировки. Можно не тратить время на работу в интерфейсе DTA, а целиком поручить анализ компьютеру.

В разделе «Рабочая нагрузка» (Workload) выбирают базу данных для анализа рабочей нагрузки. Она служит базой для выработки рекомендаций по оптимизации.

Раздел внизу окна позволяет задать базу данных и таблицы для выполнения оптимизации. Запросы, которые содержат не подлежащие оптимизации объекты, DTA в ходе анализа игнорирует.

После определения общих параметров, переходят на вкладку «Параметры настройки» (Tuning Options) (рис. 13-2).

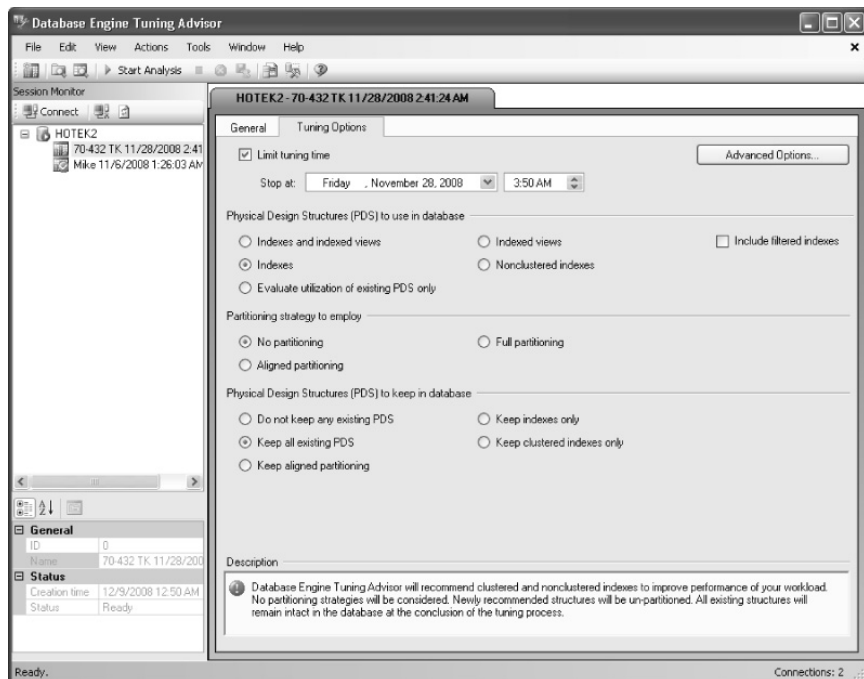


Рис. 13-2. Определение параметров настройки

Параметры настройки разбиты на четыре группы:

- ограничения по времени и интерактивные действия;
- существующие в базе данных структуры;
- параметры секционирования;
- сохранение структур, существующих в базе данных.

Если ограничить время настройки, DTA проанализирует столько запросов, сколько успеет за отведенное время. По истечении этого времени все оставшиеся запросы из рабочей нагрузки остаются необработанными. Диалоговое окно «Дополнительные параметры настройки» (Advanced Tuning Options) позволяет ограничить пространство, занимаемое рекомендациями, а также определить, как вносить рекомендованные изменения — в оперативном или автономном режиме (рис. 13-3).

DTA предлагает рекомендации по изменению индексов и индексируемых представлений на основе параметров раздела «Структуры физического проектирования (PDS) для использования в базе данных» [Physical Design Structures (PDS) To Use In Database]. Чаще всего используется параметр ограничения рекомендаций только индексами, включая как кластерные, так и некластерные индексы. Параметр «Оценивать использование только существующих структур PDS» (Evaluate Utilization Of Existing PDS only) применяется для выявления неиспользуемых индексов и индексируемых представлений, которые можно удалить.

Рекомендации по выбору стратегии секционирования предусматривают либо полное секционирование, либо выровненное секционирование. Раздел «Структуры физического проектирования (PDS), которые следует сохранять в базе данных» (Physical Design Structures (PDS) To Keep In Database) позволяет

указать, должны ли рекомендации учитывать существующую в базе данных структуру индексов и секционирования и могут ли рекомендации предусматривать их удаление.

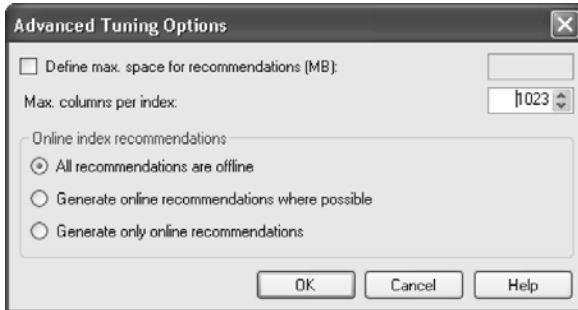


Рис. 13-3. Дополнительные параметры настройки

ВНИМАНИЕ! ДТА и производительность

ДТА анализирует стоимость указанного запроса, сравнивая со всеми возможными рекомендациями. Стоимость запроса определяется оптимизатором запросов на основе статистики распределения. Для оценки точной стоимости ДТА перед передачей запроса оптимизатору обновляет статистику в базе данных, где выполняется анализ рабочей нагрузки. Создание и удаление статистики может серьезно увеличить нагрузку на анализируемую базу данных. Поэтому следует быть очень осторожными при запуске ДТА на рабочей базе данных, но практически всегда для этого лучше использовать тестовую систему.

После определения всех желаемых параметров, можно начать анализ, щелкнув **Начать анализ** (Start Analysis) на панели инструментов. Завершив анализ, ДТА отображает рекомендации вместе с командами, реализующими ту или иную рекомендацию (рис. 13-4).

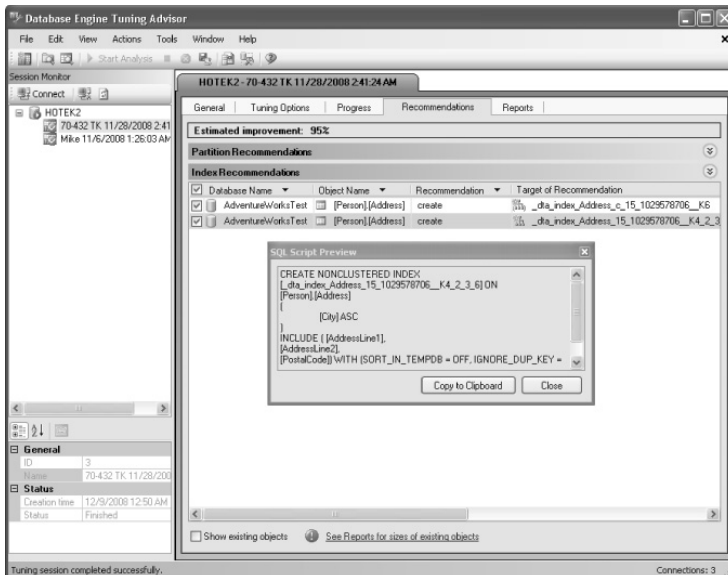


Рис. 13-4. Рекомендации по настройке

Можно просмотреть ряд отчетов о настраиваемых запросах:

- ожидаемое повышение производительности в процентах от текущей;
- частота появления каждого запроса в рабочей нагрузке;
- статистика стоимости запросов;
- подробный отчет об имеющихся индексах в анализируемой базе данных.

ПРИМЕЧАНИЕ На экзамене

Следует знать, как каждый из параметров настройки влияет на рекомендации, предлагаемые DTA.

Контрольные вопросы

1. Какие исходные данные может анализировать DTA?

Ответы на контрольные вопросы

1. DTA может анализировать запросы и хранимые процедуры, находящиеся в файле или таблице. В качестве входных данных для DTA чаще всего используется файл трассировки.

Практикум. Анализ рабочей нагрузки запроса

Вы создадите файл рабочей нагрузки, который можно передать DTA для выработки рекомендаций по улучшению производительности.

1. Создайте базу данных *AdventureWorksTest*, если она еще не существует.
2. Создайте тестовую таблицу командой:

```
USE AdventureWorksTest
GO
CREATE SCHEMA Person AUTHORIZATION dbo
GO
SELECT * INTO AdventureWorksTest.Person.Address
FROM AdventureWorks.Person.Address
```

3. Сохраните следующий код в файл:

```
USE AdventureWorksTest
GO
SELECT AddressLine1, AddressLine2, City, PostalCode
FROM Person.Address
WHERE City = 'Dallas'
GO
SELECT AddressLine1, AddressLine2, City, PostalCode
FROM Person.Address
WHERE City LIKE 'S%'
GO
SELECT AddressLine1, AddressLine2, City, PostalCode
FROM Person.Address
WHERE PostalCode = '75201'
GO
```

4. Запустите DTA и подключитесь к экземпляру сервера, содержащего базу данных *AdventureWorksTest*.
5. Задайте имя сеанса настройки.
6. Выберите файл, созданный в п. 3 и укажите базу данных *AdventureWorksTest* для выполнения анализа рабочей нагрузки.
7. Выберите для оптимизации базу данных *AdventureWorksTest* и таблицу *Person.Address*.
8. Перейдите на вкладку **Параметры настройки** (Tuning Options).
9. В разделе **Структуры физического проектирования (PDS) для использования в базе данных** (Physical Design Structures (PDS) To Use In Database) выберите, если нужно, параметр **Индексы** (Indexes).
10. В разделе **Выбранная стратегия секционирования** (Partitioning Strategy To Employ) выберите параметр **Без секционирования** (No Partitioning).
11. В разделе **Структуры физического проектирования (PDS), которые следует сохранять в базе данных** (Physical Design Structures (PDS) To Keep In Database) выберите, если нужно, параметр **Сохранять все существующие PDS** (Keep All Existing PDS).
12. Запустите анализ.
13. Просмотрите рекомендации, а также все отчеты анализа.

Резюме занятия

- DTA применяется для анализа рабочей нагрузки запросов в базе данных и получения рекомендации по созданию или удалению структур с целью повышения производительности.
- В качестве источника рабочей нагрузки можно использовать файл или таблицу.
- DTA создает статистику анализируемой базы данных, а затем передает оптимизатору запросов поручение оценить стоимость запросов и определить, возможны ли улучшения.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Какие типы рабочих нагрузок могут использоваться в DTA для анализа? (Выберите все подходящие варианты).
 - A. Сценарий T-SQL.
 - B. Файл трассировки, содержащий планы отображения (showplan) в формате XML.
 - C. Файл трассировки, содержащий события RPC:Completed.
 - D. Файл трассировки, содержащий события SP:StmtCompleted.

Занятие 2. Использование Регулятора ресурсов

Регулятор ресурсов (Resource Governor) позволяет ограничить использование процессора и памяти, определенными подключениями или группами пользователей. На этом занятии рассказывается, как настраивать Регулятор ресурсов, чтобы ресурсы распределялись в соответствии с бизнес-приоритетами.

Изучив материал этого занятия, вы сможете:

- ✓ создавать пулы ресурсов;
- ✓ создавать группы рабочей нагрузки;
- ✓ создавать функции-классификаторы;
- ✓ оценивать использование ресурсов пулом ресурсов;
- ✓ оценивать использование ресурсов группами рабочей нагрузки.

Продолжительность занятия — 20 минут.

Регулятор ресурсов

Регулятор ресурсов работает с компонентами трех типов:

- пулы ресурсов;
- группы рабочей нагрузки;
- функции-классификаторы.

При активизации Регулятора ресурсов процессы в SQL Server выполняются по схеме, показанной на рис. 13-5.

ПРИМЕЧАНИЕ Классификация подключений

Классификация происходит в момент создания подключения. Поэтому единственный критерий, на основании которого можно ограничить использование ресурсов, — свойства подключений. Нет возможности ограничивать использование ресурсов отдельными запросами или даже типами запросов.

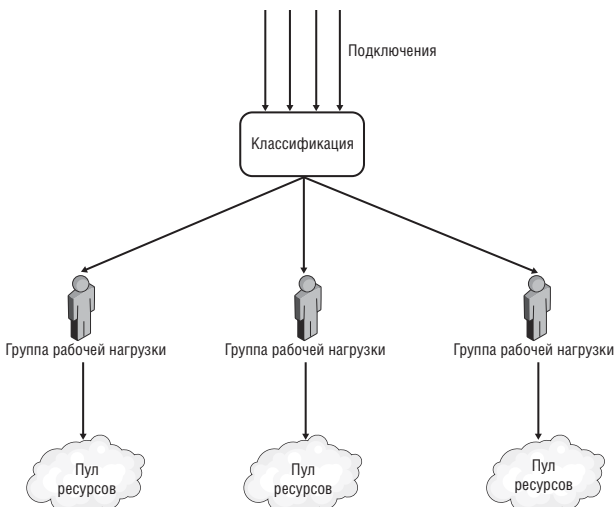


Рис. 13-5. Обработка подключения при включенном Регуляторе ресурсов

Ресурсы, которыми управляет Регулятор ресурсов, — это процессор и память. Хотя можно ограничить ресурсы, доступные рабочей группе, все выполняющиеся запросы не учитывают ограничения, также невозможно наложить ограничения на внутренние операции SQL Server.

Группа рабочей нагрузки (workload group) — это всего лишь имя, связанное с пользовательским сеансом. Группы рабочей нагрузки определяют не рабочую нагрузку запросов, а учетную запись, которая исполняет эти запросы. Группы рабочей нагрузки — это всего лишь метки, сопоставляемые подключениям при их создании, чтобы Регулятор ресурсов мог назначать подключению соответствующий пул ресурсов.

Функция-классификатор (classifier function) — функция, создаваемая в базе данных *master*. У Регулятора ресурсов в каждый момент времени может быть активна лишь одна функция-классификатор. Эта функция не может иметь входных параметров и должна возвращать скалярное значение. Возвращаемое значение содержит имя группы рабочей нагрузки, которой должен быть отнесен сеанс. Функция может содержать любой удовлетворяющий требованиям код, но следует минимизировать количество кода функции-классификатора. Поскольку функция-классификатор выполняется после проверки подлинности, но перед возвратом пользователскому приложению описателя сеанса, любые проблемы с производительностью функции-классификатора могут увеличивать время подключения к серверу SQL Server и, возможно, приводить в приложениях к ошибкам превышения времени ожидания подключения.

ПРИМЕЧАНИЕ Пулы ресурсов по умолчанию

Если Регулятору ресурсов не назначена функция-классификатор или она не существует, возвращает NULL или несуществующую группу рабочей нагрузки, то пользовательский сеанс назначается в пул ресурсов по умолчанию.

Подключение может принадлежать только одной группе рабочей нагрузки, но к одной группе рабочей нагрузки могут относиться несколько подключений. Каждая группа рабочей нагрузки связана только с одним пулом ресурсов, но с одним пулом ресурсов может быть связано несколько групп рабочих нагрузок.

Пул ресурсов (resource pool) определяет минимальные и максимальные значения использования процессора и памяти по отдельности либо одновременно, выделяемых этому пулу ресурсов. Минимальное значение определяет минимально гарантированный объем ресурсов, доступных пулу ресурсов. Для каждого пула ресурсов можно указать минимальное значение, но сумма минимальных значений по всем группам ресурсов не может превышать 100. Максимальное значение определяет максимальное количество ресурсов, которые могут выделяться группе рабочей нагрузки, связанной с пулом ресурсов.

ПРИМЕЧАНИЕ Выделение ресурсов

Считается, что все подключения в пуле ресурсов имеют одинаковый вес и SQL Server выполняет балансировку ресурсов, доступных пулу, среди всех выполняемых на сервере запросов.

Функцию-классификатор можно создавать в любое время, но рекомендуется делать это на последнем этапе настройки Регулятора ресурсов. После при-

вязки функции-классификатора к Регулятору ресурсов она выполняется при создании каждого нового подключения к экземпляру сервера.

Регулятор ресурсов настраивается в следующем порядке:

1. включить Регулятор ресурсов;
2. создать один или несколько пулов ресурсов;
3. создать одну или несколько групп рабочих нагрузок;
4. связать каждую группу рабочей нагрузки с пулом ресурсов;
5. создать и протестировать функцию-классификатор;
6. связать функцию-классификатор с Регулятором ресурсов.

ПРИМЕЧАНИЕ На экзамене

Следует знать ресурсы, которыми может управлять Регулятор ресурсов, а также как тестировать и устранять неполадки в функциях-классификаторах.

Для получения информации о настройке Регулятора ресурсов можно использовать следующие представления:

```
SELECT * FROM sys.resource_governor_resource_pools
SELECT * FROM sys.resource_governor_workload_groups
SELECT * FROM sys.resource_governor_configuration
GO
```

При активном Регуляторе ресурсов столбец `group_id` таблицы `sys.dm_exec_sessions` содержит идентификатор группы рабочей нагрузки, связанной с сеансом.

Контрольные вопросы

1. Какие объекты применяются при настройке Регулятора ресурсов?
2. Какие ресурсы может контролировать Регулятор ресурсов?

Ответы на контрольные вопросы

1. Присоединяя подключение к группе рабочей нагрузки, Регулятор ресурсов руководствуется пользовательской функцией-классификатором, хранящейся в базе данных `master`. Каждая группа рабочей нагрузки назначается в пул, управляющий ресурсами процессора и памяти.
2. Регулятор ресурсов применяется для управления ресурсами процессора и памяти.

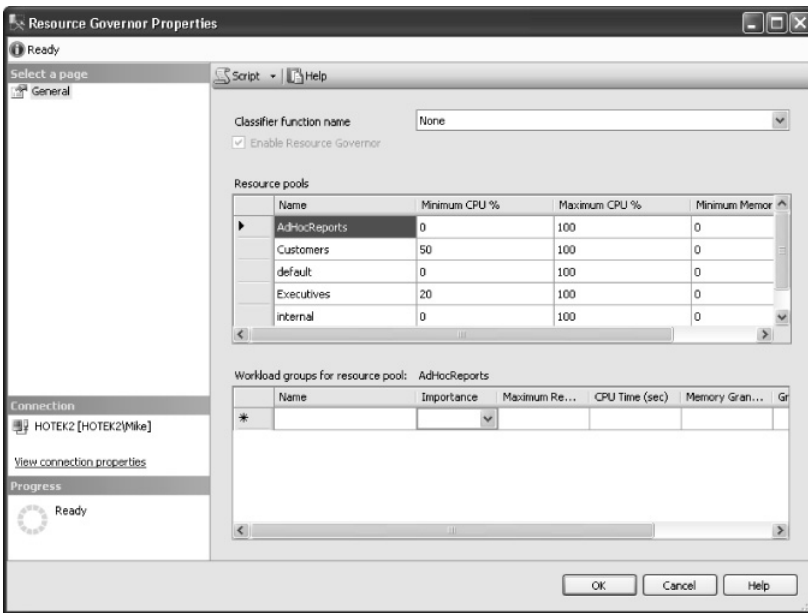
Практикум. Настройка Регулятора ресурсов

Вы настроите Регулятор ресурсов и проверите его влияние на пользовательский сеанс.

Задание 1. Создание пула ресурсов

Вы создадите два пула ресурсов, которые будут обеспечивать доступность процессора группам пользователей.

1. Откройте **Обозреватель объектов** (Object Explorer) в среде SSMS и подключитесь к своему экземпляру сервера.
2. Раскройте узлы **Управление** (Management) и **Регулятор ресурсов** (Resource Governor), щелкните правой кнопкой **Пулы ресурсов** (Resource Pools) и выберите **Создать пул ресурсов** (New Resource Pool).
3. Отметьте флажок **Включить регулятор ресурсов** (Enable Resource Governor).
4. Создайте пул под именем **Executives** (руководство) и задайте минимальное значение процессора равным 20%.
5. Создайте пул под именем **Customers** (пользователи) и задайте минимальное значение процессора равным 50%.
6. Создайте пул под именем **AdHocReports** (нерегламентированные отчеты) и задайте минимальное значение процессора равным 0%.

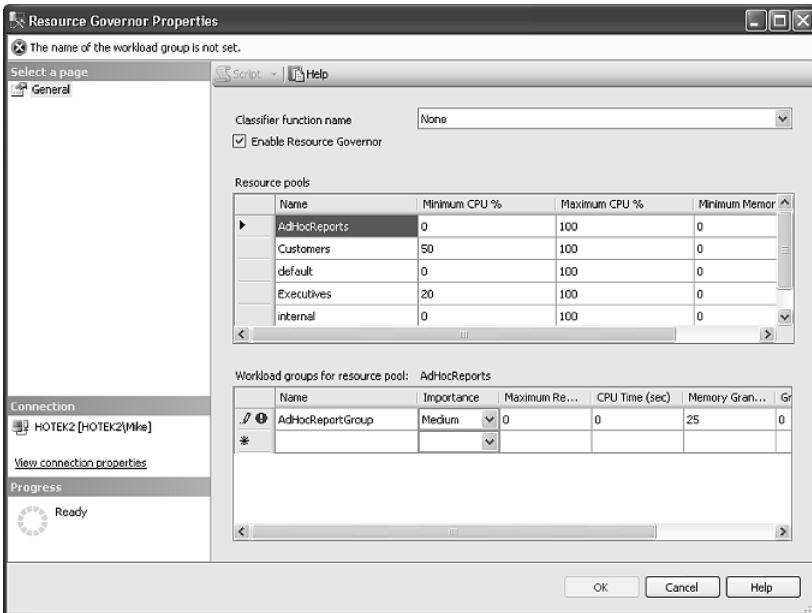


7. Щелкните **ОК**.

Задание 2. Назначение группы рабочей нагрузки

Вы создадите группы рабочей нагрузки, которые будут использоваться для разбиения пользовательских запросов по соответствующим пулам ресурсов.

1. В Обозревателе объектов раскройте узел **AdHocReports**, щелкните правой кнопкой **Группы рабочей нагрузки** (Workload Groups) и выберите **Создать группу рабочей нагрузки** (New Workload Group).
2. Создайте группу **AdHocReportGroup** для пула ресурсов AdHocReports, как показано ниже.



3. Создайте группу **CustomerGroup** для пула ресурсов Customers.
4. Создайте группу **ExecutiveGroup** для пула ресурсов Executives.
5. Щелкните **OK**.

Задание 3. Создание функции-классификатора

Вы создадите функцию-классификатор, а затем проверите классификацию рабочей нагрузки и назначение к пулам ресурсов.

1. Выполните следующий код в базе данных master:

```
CREATE FUNCTION dbo.fn_ResourceGovernorClassifier()
RETURNS sysname
WITH SCHEMABINDING
AS
BEGIN
    DECLARE @group sysname
    -- Имя группы рабочей нагрузки учитывает регистр,
    -- независимо от настройки сервера
    IF SUSER_SNAME() = 'Executive'
        SET @group = 'ExecutiveGroup'
    ELSE IF SUSER_SNAME() = 'Customer'
        SET @group = 'CustomerGroup'
    ELSE IF SUSER_SNAME() = 'AdHocReport'
        SET @group = 'AdHocReportGroup'
    ELSE
        SET @group = 'default'

    RETURN @group
END
```

```
GO
```

2. Свяжите функцию-классификатор с регулятором ресурсов следующим кодом:

```
ALTER RESOURCE GOVERNOR WITH (CLASSIFIER_FUNCTION = dbo.fn_  
    ResourceGovernorClassifier)
```

```
GO
```

3. Активизируйте функцию-классификатор следующим кодом:

```
ALTER RESOURCE GOVERNOR RECONFIGURE
```

```
GO
```

Задание 4. Проверка регулятора ресурсов

Вы создадите простой тестовый сценарий и будете отслеживать работу пулов ресурсов средствами Системного монитора.

1. Откройте новое окно запросов, войдите в систему как администратор и выполните следующий код, чтобы создать три учетные записи для тестирования:

```
CREATE LOGIN Executive WITH PASSWORD = '<надежный_пароль>'
```

```
GO
```

```
CREATE LOGIN Customer WITH PASSWORD = '<надежный_пароль>'
```

```
GO
```

```
CREATE LOGIN AdHocReport WITH PASSWORD = '<надежный_пароль>'
```

```
GO
```

2. Откройте новое окно запросов и войдите систему под учетной записью Customer.
3. Откройте новое окно запросов и войдите систему под учетной записью Executive.
4. Откройте новое окно запросов и войдите систему под учетной записью AdHocReport.
5. В окне запросов, где вы вошли в систему как администратор, выполните следующий запрос, чтобы проверить группы рабочей нагрузки, назначенные каждому подключению:

```
SELECT b.name WorkloadGroup, a.login_name, a.session_id  
FROM sys.dm_exec_sessions a INNER JOIN sys.dm_resource_governor_workload_  
groups b  
    ON a.group_id = b.group_id  
WHERE b.name != 'internal'
```

6. В каждом окне запросов введите следующий запрос:

```
SET NOCOUNT ON
```

```
DECLARE @var INT
```

```
SET @var = 1
```

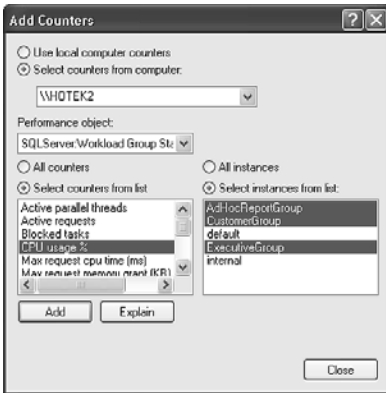
```
WHILE @var < 10000000
```

```
BEGIN
```

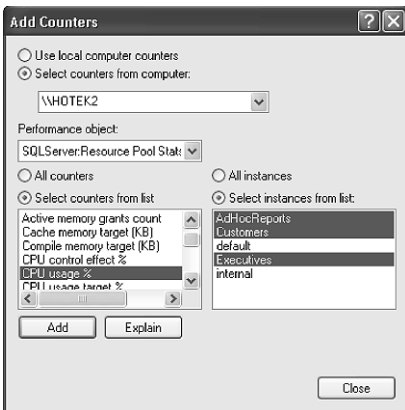
```
    SELECT @@VERSION
```

```
SET @var = @var + 1
END
GO
```

7. Поскольку нас не интересуют реальные результаты запроса, в окнах запросов Customer, Executive и AdHocReports выберите **Запрос (Query)**, **Параметры запроса (Query Options)**. В левой панели диалогового окна **Параметры запроса** выберите узел **Сетка (Grid)**, а затем отметьте флажок **Отбросить результаты после выполнения (Discard Results After Execution)**.
8. Запустите Системный монитор, удалите все счетчики по умолчанию и добавьте счетчик **SQLServer:Workload Group Status: CPU Usage %** для экземпляров **AdHocReportGroup**, **CustomerGroup** и **ExecutiveGroup**, как показано ниже.

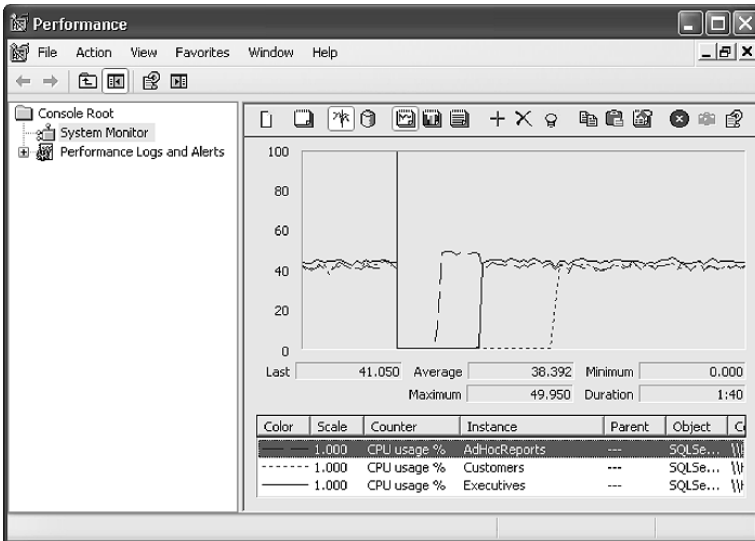
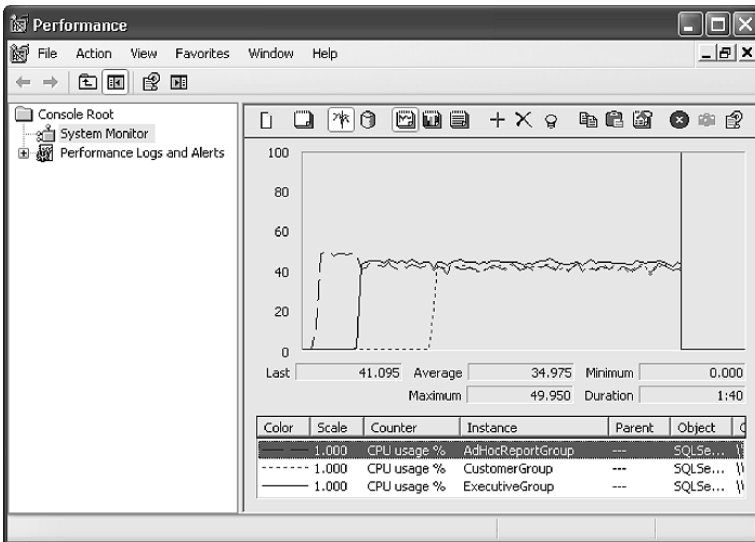


9. Запустите второй экземпляр Системного монитора, удалите все счетчики по умолчанию и добавьте счетчик **SQLServer:Resource Pool Status: CPU Usage%** для экземпляров **AdHocReportGroup**, **CustomerGroup** и **ExecutiveGroup**, как показано ниже.



10. Выполните сценарий в окне запросов Customer и изучите графики в системном мониторе.
11. Выполните сценарий в окне запросов Executive и изучите графики в системном мониторе.

12. Выполните сценарий в окне запросов AdHocReport и изучите графики в системном мониторе, как показано ниже.

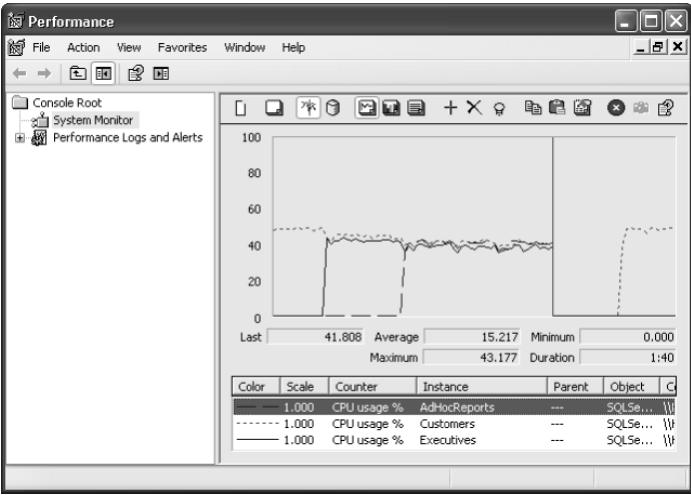
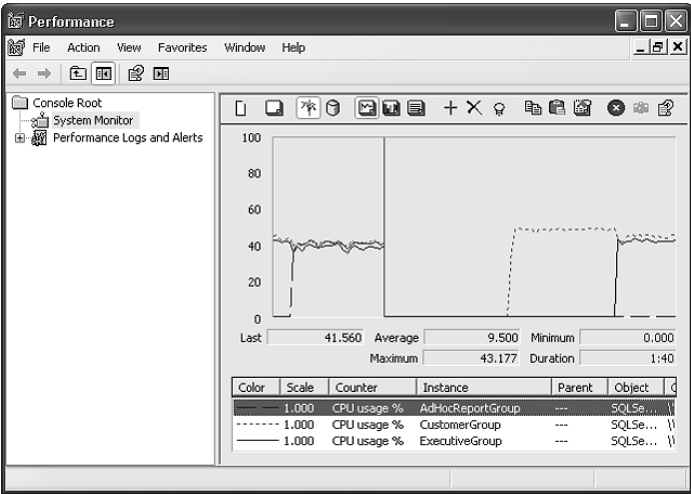


13. В окнах запросов Customer, Executive и AdHocReport остановите выполнение запросов.
14. Ограничьте использование ресурсов пулом AdHocReports, выполнив следующий код:

```
ALTER RESOURCE POOL AdHocReports
    WITH (MAX_CPU_PERCENT = 5)
ALTER RESOURCE POOL AdHocReports
    WITH (MIN_CPU_PERCENT = 0)
ALTER RESOURCE POOL Executives
```

```
WITH (MAX_CPU_PERCENT = 20)
ALTER RESOURCE POOL Executives
WITH (MIN_CPU_PERCENT = 20)
ALTER RESOURCE POOL Customers
WITH (MAX_CPU_PERCENT = 75)
ALTER RESOURCE POOL Customers
WITH (MIN_CPU_PERCENT = 50)
ALTER RESOURCE GOVERNOR RECONFIGURE
GO
```

- 15. Выполните сценарий в окне запросов Customer и изучите графики в системном мониторе.
- 16. Выполните сценарий в окне запросов Executive и изучите графики в системном мониторе.
- 17. Выполните сценарий в окне запросов AdHocReport и изучите графики в системном мониторе, как показано ниже.



ВНИМАНИЕ! Ограничение ресурсов

Даже при включенном регуляторе ресурсов SQL Server старается выделять как можно больше ресурсов для одновременно выполняемых запросов. Если для пула ресурсов задан максимальный предел, его подключения могут использовать больше ресурсов, чем указанный предел. Если другим сеансам не требуются все ресурсы, свободные ресурсы могут использоваться любым другим сеансом, даже если это превысит установленное ограничение ресурсов на связанном с сеансом пуле ресурсов.

Резюме занятия

- Регулятор ресурсов применяется для ограничения использования процессора и памяти порознь, либо одновременно, одним или несколькими подключениями.
- Подключения назначаются группе рабочей нагрузки функцией-классификатором.
- Группа рабочей нагрузки назначается пулу ресурсов.
- Пулы ресурсов настраиваются для управления ресурсами процессора и памяти.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Вы администратор баз данных компании Coho Vineyards. После объединения баз данных всей продукции винных заводов, клиентов и заказов в отделе маркетинга запросили обеспечить возможность выполнять нерегламентированные запросы для анализа. Пользователям позволяется выполнять любые запросы, несмотря на то, что это может негативное сказаться на производительности базы данных. Руководство не ограничивает запросы, которые могут выполняться сотрудниками отдела маркетинга, но требует обеспечить своевременное создание и обработку заказов от клиентов. Как ограничить негативное влияние запросов маркетинга и обеспечить гарантированную обработку клиентских заказов?
 - A. Настроить параметр «максимальная степень параллелизма».
 - B. Настроить Регулятор ресурсов.
 - C. Настроить стоимостный порог Регулятора запросов.
 - D. Ограничить использование памяти пользователями из группы маркетинга.

Занятие 3. Динамические административные представления и функции

Динамические административные представления (dynamic management view, DMV) и *динамические административные функции* (dynamic management func-

tion, DMF) лежат в основе инструментальной инфраструктуры, позволяющую администраторам баз данных получать информацию о системе, а также контролировать, диагностировать и устранять неполадки. На этом занятии вы узнаете об основных динамических административных представлениях и функциях, используемых для оптимизации производительности.

ПРИМЕЧАНИЕ Соглашение о терминах

Для простоты весь набор инструментального кода SQL Server называют обобщенно динамическими административными представлениями, не важно идет ли речь о представлениях или о функциях.

Изучив материал этого занятия, вы сможете:

- ✓ разбираться в категориях динамических административных представлений и функций;
- ✓ определять важные для мониторинга и контроля производительности динамические административные представления и функции.

Продолжительность занятия — 20 минут.

Категории динамических административных представлений

Все динамические административные представления хранятся в схеме *sys* и разбиты на несколько десятков категорий. Поскольку в них используются стандартные схемы наименования и имена говорят сами за себя, то разбиение по категориям вполне очевидно. В табл. 13-1 перечислены наиболее важные категории динамических административных представлений, применяемых при настройке производительности.

Табл. 13-1. Категории динамических административных представлений

Префикс имени	Назначение
<i>dm_db_*</i>	Общая информация об использовании места и индексов в базе данных
<i>dm_exec_*</i>	Статистика по исполняемым в настоящий момент запросам, а также по выполненным запросам, планы которых еще хранятся в кеше запросов
<i>dm_io_*</i>	Статистика дисковой подсистемы
<i>dm_os_*</i>	Статистика использования аппаратных ресурсов

Статистика базы данных

Вот наиболее популярные динамические административные представления для сбора статистики базы данных:

- *sys.dm_db_index_usage_stats*
- *sys.dm_db_index_operational_stats*
- *sys.dm_db_index_physical_stats*
- *sys.dm_db_missing_index_groups*
- *sys.dm_db_missing_index_group_stats*
- *sys.dm_db_missing_index_details*

Индексы создаются для повышения производительности определенных запросов. Но обновление индексов при изменении данных создает дополнительную нагрузку. Недостаточное число индексов может вызвать проблемы с производительностью, так же как наличие неиспользуемых или редко используемых индексов. Представление *sys.dm_db_index_usage_stats* показывает сколько раз (и когда в последний раз) каждый индекс использовался для поиска, сканирования или просмотра, а также сколько раз каждый индекс обновлялся и время последнего обновления. Если по индексу не проводился поиск, сканирование или просмотр, значит, он не используется. Если это происходит постоянно, SQL Server перестает использовать этот индекс.

Функция *sys.dm_db_index_operational_stats* принимает четыре необязательных параметра — *database_id*, *object_id*, *index_id* и *partition_id*. Для каждого индекса она возвращает статистику по доступу, блокировкам и кратковременным блокировкам, что показывает, насколько активно используются индексы. Эта функция помогает также выявить причину конфликтов из-за блокировок или кратковременных блокировок.

Функция *Sys.dm_db_index_physical_stats* принимает 5 необязательных параметров — *database_id*, *object_id*, *index_id*, *partition_id* и *mode*. Она возвращает размер и статистику фрагментации всех индексов и является основным источником информации для определения момента, когда следует дефрагментировать индексы.

Получив запрос, SQL Server анализирует и оптимизирует его, чтобы определить наиболее эффективный способ его выполнения. Созданный в результате план выполнения используется затем для выполнения запроса. Даже если в таблице нет индексов, SQL Server все равно хранит статистику распределения для каждого столбца таблицы. Таким образом даже при отсутствии индексов оптимизатор в состоянии определить, будет ли индекс полезен для обработки запроса. Если включен параметр *AUTO CREATE STATISTICS* и оптимизатор решит, что от статистики будет польза, она будет генерироваться автоматически и затем может использоваться в запросах для повышения производительности.

Когда оптимизатор решает, что индекс был бы полезен, но отсутствует, он создает ситуацию, известную как нехватка индекса (*index miss*). И хотя при нехватке индекса автоматически генерируется статистика, индекс все же эффективнее для выполнения запросов, если он создается автоматически.

Одни из наиболее интересных категорий в SQL Server 2008 — представления семейства *sys.dm_db_missing_index_**. Когда образуется нехватка индекса, SQL Server записывает об этом информацию в журнал, который можно просматривать при помощи представлений *sys.dm_db_missing_index_**. Представление *sys.dm_db_missing_index_details* содержит сводную статистику о том, сколько раз генерировалась нехватка индекса для каждого возможного индекса, — эта информация полезна для определения, нужно ли создавать индекс.

ПРИМЕЧАНИЕ Рекомендации по созданию индексов

Динамические административные представления о нехватке индексов могут содержать много комбинаций групп столбцов, на основе которых можно создавать индексы. Каждая уникальная комбинация столбцов приводит к записи о нехватке индекса. Выполнив простое агрегирование данных представления об отсутствующих индексах, можно выявить наиболее полезные индексы. Имейте

в виду, что стоимость поддержки индексов не включена в агрегацию. Для вычисления «степени полезности» можно использовать такой запрос:

```
SELECT *
FROM
(SELECT user_seeks * avg_total_user_cost * (avg_user_impact * 0.01) AS
index_advantage, migs.* FROM sys.dm_db_missing_index_group_stats migs) AS
migs_adv
INNER JOIN sys.dm_db_missing_index_groups AS mig
    ON migs_adv.group_handle = mig.index_group_handle
INNER JOIN sys.dm_db_missing_index_details AS mid
    ON mig.index_handle = mid.index_handle
ORDER BY migs_adv.index_advantage
```

Если выигрыш от индекса (*index_advantage*) превышает 10 000, наличие такого индекса может значительно поднять производительность запросов, но при принятии решения надо учитывать затраты на его поддержку. Если же польза от индекса превышает 50 000, то выгода от его создания значительно превышает затраты на его обновление при изменении данных.

Статистика запросов

Динамические административные представления *sys.dm_exec_** возвращают информацию о подключениях к экземпляру, а также о выполнении запросов.

Следующие динамические административные представления возвращают информацию о подключениях и часто выполняемых запросах:

- *Sys.dm_exec_connections*
- *Sys.dm_exec_sessions*
- *Sys.dm_exec_requests*

Sys.dm_exec_connections содержит по одной записи на каждое подключение к экземпляру. В этом представлении можно узнать о времени создания подключения, а также его свойства и параметры шифрования. Оно также сообщает количество операций записи и чтения через подключение, а также последнее время чтения или записи.

Sys.dm_exec_sessions содержит по одной записи на каждый прошедший проверку подлинности сеанс. Кроме данных о входе, это представление также отслеживает состояние всех возможных параметров запроса и текущее состояние выполнения. Дополнительно оно возвращает для сеанса суммарное время операций чтения и записи данных, использования процессорного времени и продолжительности выполнения запросов

Sys.dm_exec_requests содержит по одной записи на каждый текущий запрос, выполняемый в экземпляре. Для выявления проблем подключения можно использовать столбец *blocking_session_id*. Это представление содержит время запуска, длительность, ожидаемое время завершения, прочитанные и записанные данные, а также использование запросом процессора. Кроме этого, можно получить базу данных, выполняемую команду, дескрипторы инструкции SQL и план запроса.

Каждый выполняемый запрос должен быть скомпилирован, разобран и сравнен с кешем запросов. При совпадении для выполнения запроса оптимизатор

использует его кешированный план, в противном случае генерируется план запроса и помещается в кеш.

Для всех запросов в кеше представление *sys.dm_exec_query_stats* содержит подробную статистику о выполнении и использованных ресурсах. Оно показывает время последнего выполнения запроса, сколько раз он выполнялся, а также минимальную и максимальную продолжительность выполнения, логические и физические операции чтения, записи и использования процессора, а также дескриптор плана запроса, созданного оптимизатором.

SQL Server хранит в кеше план запросов и текст всех выполненных запросов, которые идентифицируются уникальным значением, называемым дескриптором. Функция *sys.dm_exec_sql_text* возвращает текст инструкции SQL, связанной с переданным дескриптором. Функция *sys.dm_exec_query_plan* принимает дескриптор плана и возвращает его XML Showplan.

Если требуется получить запрос и XML Showplan всех выполняемых в настоящее время запросов, можно воспользоваться следующей инструкцией:

```
SELECT * FROM sys.dm_exec_requests CROSS APPLY sys.dm_exec_query_plan(plan_handle)
      CROSS APPLY sys.dm_exec_sql_text(sql_handle)
```

Следующую команду можно использовать для получения инструкций SQL и XML Showplan всех запросов из кеша:

```
SELECT * FROM sys.dm_exec_query_stats CROSS APPLY sys.dm_exec_query_plan(plan_handle)
      CROSS APPLY sys.dm_exec_sql_text(sql_handle)
```

Статистика дисковой подсистемы

Функция *sys.dm_io_virtual_file_stats* для каждого файла базы данных возвращает статистику операций чтения и записи данных. Она возвращает агрегированное количество операций чтения и записи, а также сколько байт было прочитано и записано в каждый файл с момента запуска экземпляра. Кроме этого можно получить информацию о задержке ввода/вывода, или IOStall. Если при попытке чтения или записи SQL Server вынужден ожидать освобождения дисковой подсистемы, возникает задержка IOStall. Время IOStall измеряется в миллисекундах и записывается в журнал каждого файла базы данных.

Информация, возвращаемая функцией *sys.dm_io_virtual_stats*, используется для выяснения, сказывается ли конкуренция за дисковые ресурсы на производительности. Также эти сведения полезны для определения сбалансированности дискового ввода/вывода по файлам базы данных и выявления перегруженности.

Представление *sys.dm_io_pending_requests* содержит записи о запросах, ожидающих завершения дисковой подсистемой ввода/вывода. В загруженной системе в этом представлении всегда есть записи. Однако если какой-то запрос появляется в нем очень часто или надолго остается в представлении, скорее всего дисковая подсистема стала узким местом и с этим надо разбираться.

Аппаратные ресурсы

Одна из ошибок, которую многие совершают при попытке выявить причину низкой производительности — вера в то, что медленно выполняющийся запрос можно ускорить, добавив индексы или переписав код. Отчасти это верно — за-

прос действительно может медленно выполняться из-за неэффективного кода или отсутствия индексов. Однако причина также может быть в конкуренции за ресурсы, что заставляет запрос ожидать их освобождения.

При выполнении запросов SQL Server применяет модель совместной обработки. Каждый выполняемый запрос присваивается планировщику пользовательского режима (User Mode Scheduler, UMS). Если не изменена стандартная настройка параметра привязки к процессорам (affinity mask), SQL Server для выполнения запросов поддерживает по одному планировщику UMS на каждый процессор. Поскольку на процессоре в каждый момент времени может выполняться только одна команда, максимальное число запросов, которые могут обрабатываться одновременно, равно количеству планировщиков UMS, выполняющихся на SQL Server. Любой запрос свыше этого количества добавляется в очередь готовых к запуску запросов в порядке поступления. После того, как запрос оказывается в начале очереди готовых к запуску запросов и освобождается планировщик UMS, он передается в очередь выполняемых запросов планировщика и начинает исполняться. Если запрос вынужден ожидать какие-либо ресурсы, такие как освобождение блокировки, доступ к операциям ввода/вывода или выделение памяти, он освобождает процессор и возвращается в очередь ожидания, чтобы освободить место для выполнения следующего запроса из очереди запросов, готовых к выполнению. Запрос остается в очереди ожидания пока нужный ресурс не станет доступным, а затем переводится в конец очереди запросов, готовых к выполнению, где он ждет все остальные запросы, прежде чем снова попадает в планировщик UMS для продолжения обработки.

В условиях конкуренции за ресурсы запрос может совершить несколько циклов между очередью запросов, готовых к выполнению, очередью выполнения и очередью ожидания, прежде чем он будет выполнен. Ликвидировав узкое место в обработке запросов, можно повысить производительность.

При переводе запроса в очередь ожидания SQL Server задает значение, называемое *wait type* (тип ожидания), которое обозначает тип ресурса, который запрос ожидает. Сразу после определения типа ожидания SQL Server запускает отсчет времени. После освобождения ресурса SQL Server останавливает отсчет времени и записывает время ожидания освобождения ресурса. SQL Server также запускает отсчет времени, когда запрос попадает в очередь запросов, готовых к выполнению, это значение называется *signal wait* (ожидание команды) и показывает, сколько времени потребовалось запросу, чтобы подняться до начала очереди и начать выполняться.

Процедура *sys.dm_os_wait_stats* показывает, сколько раз возникало ожидание команды и суммарное время такого ожидания по каждому типу. Ожидание команды и время ожидания — это агрегированные значения с момента последней очистки статистики. В отличие от большинства динамических административных представлений, которые очищаются только перезагрузкой экземпляра, время ожидания и ожидание команды можно очистить следующим кодом:

```
DBCC SQLPERF(WAITSTATS,CLEAR)
```

ПРИМЕЧАНИЕ Типы ожидания

Несмотря на то, что статистика ожиданий чрезвычайно важная часть данных, необходимых для диагностики проблем производительности, и прошло уже

почти 10 лет после того, как стала доступна подробная информация об этом, Майкрософт до сих пор не выпустила документ с описанием типов ожиданий, которые отличаются исключительно непонятными именами. Самый лучший ресурс, позволяющий разобраться в типах ожидания и том, как устранять неполадки, выявленные с их применением, — веб-сайт Герта Драпера (Gert Draper) по адресу <http://www.sqldev.net>.

ПРИМЕЧАНИЕ На экзамене

Следует знать предназначение основных динамических административных представлений, и как применять каждое из них для диагностирования и решения проблем производительности.

Контрольные вопросы

1. В чем разница между *sys.dm_db_index_operational_stats* и *sys.dm_db_index_physical_stats*?
2. Какое динамическое административное представление можно использовать, чтобы получить статистику выполнения всех подключений, выполняющих в настоящий момент команды?

Ответы на контрольные вопросы

1. *sys.dm_db_index_physical_stats* возвращает статистику фрагментации всех индексов, а *sys.dm_db_index_operational_stats* возвращает информацию о блокировках, кратковременных блокировках и статистику доступа ко всем индексам.
2. Представление *sys.dm_exec_requests* возвращает по одной записи на каждую выполняемую в настоящий момент команду.

Практикум. Оценка нехватки индексов

Используя динамические административные представления, вы найдете и оцените нехватку индексов.

1. Откройте окно запросов и выполните в базе данных *AdventureWorksTest* следующий запрос:

```
SELECT City, PostalCode, AddressLine1
FROM Person.Address
WHERE City = 'Seattle'
GO
SELECT City, PostalCode, AddressLine1
FROM Person.Address
WHERE City = 'Seattle' AND AdressLine2 IS NOT NULL
GO
SELECT City, PostalCode, AddressLine1
FROM Person.Address
WHERE City LIKE 'D%'
GO
```

2. Выполните оценивающий индексы запрос, чтобы узнать, какие индексы предлагает оптимизатор:

```
SELECT *
FROM
(SELECT user_seeks * avg_total_user_cost * (avg_user_impact * 0.01) AS
index_advantage, migs.* FROM sys.dm_db_missing_index_group_stats migs) AS
migs_adv
INNER JOIN sys.dm_db_missing_index_groups AS mig
ON migs_adv.group_handle = mig.index_group_handle
INNER JOIN sys.dm_db_missing_index_details AS mid
ON mig.index_handle = mid.index_handle
ORDER BY migs_adv.index_advantage
```

3. Выполните следующий код:

```
SELECT City, PostalCode, AddressLine1
FROM Person.Address
WHERE City LIKE 'Atlan%'
go 100
```

4. Выполните следующий запрос, оценивающий индексы, для того, чтобы увидеть, как изменятся значения при увеличении числа выполняемых запросов:

```
SELECT *
FROM
(SELECT user_seeks * avg_total_user_cost * (avg_user_impact * 0.01) AS
index_advantage, migs.* FROM sys.dm_db_missing_index_group_stats migs) AS
migs_adv
INNER JOIN sys.dm_db_missing_index_groups AS mig
ON migs_adv.group_handle = mig.index_group_handle
INNER JOIN sys.dm_db_missing_index_details AS mid
ON mig.index_handle = mid.index_handle
ORDER BY migs_adv.index_advantage
```

Резюме занятия

- Динамические административные представления *sys.dm_db_** предоставляют общую информацию об использовании пространства и индексов.
- Динамические административные представления *sys.dm_exec_** используются для получения информации о выполняемых в настоящий момент запросах и запросах, находящихся в кеше. Этот набор представлений можно применять также для решения проблем с блокировками и просматривать последний тип ожидания, присвоенный запросу.
- Динамические административные представления *sys.dm_io_** применяются для оценки производительности дисковой подсистемы и поиска в ней узких мест.
- Динамическое административное представление *sys.dm_os_wait_status* предоставляет информацию о внутренней обработке запросов, а также сколько времени запрос ожидает освобождения ресурсов.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Какое административное представление следует применять для поиска неиспользуемых индексов?
 - A. *sys.dm_db_index_operational_stat*
 - B. *sys.dm_db_index_physical_stats*
 - C. *sys.dm_db_index_usage_stats*
 - D. *sys.dm_db_missing_index_details*

Занятие 4. Использование хранилища данных о производительности

Хранилище данных о производительности (Performance Data Warehouse), или приложение Performance Studio, — новинка среды SQL Server Management Studio (SSMS), позволяющая собирать информацию о производительности экземпляров для последующего анализа во всей среде. На этом занятии рассказывается о компонентах хранилища данных о производительности, а также как настраивать наборы сбора данных, сбор данных и анализировать результаты.

Изучив материал этого занятия, вы сможете:

- ✓ создавать хранилище данных о производительности;
- ✓ создавать элементы и наборы сбора данных;
- ✓ определять целевой объект сбора данных;
- ✓ настраивать сбор данных;
- ✓ анализировать результаты сбора данных.

Продолжительность занятия — 20 минут.

Хранилище данных о производительности

Хранилище данных о производительности базируется на новой функциональности SQL Server 2008 — сборщике данных (Data Collector). Сборщик данных основан на пакетах службы Integration Services (SSIS) и заданиях Агента SQL Server, как показано на рис. 13-6.

Настройка сбора информации для хранилища данных о производительности осуществляется через сборщик одного из следующих типов:

- запросы T-SQL;
- трассировки SQL;
- счетчики производительности;
- операций запросов.

Сборщик запросов T-SQL позволяет определить инструкцию *SELECT* для выполнения, а также базу данных, где она будет выполняться. Результаты за-

проса помещаются в таблицу хранилища данных о производительности, имя которой задается параметром при определении сборщика данных *OutputTable*. Поскольку сборщик данных динамически создает таблицу на основе переданного запроса, следует обеспечить следующее:

- результирующий набор не должен содержать столбцы с такими именами, как *snapshot_time*, *snapshot_id* или *database_name*, так как они зарезервированы для сборщика данных;
- у всех столбцов результирующего набора должны быть имена;
- не должно быть столбцов типа *image*, *text*, *ntext* или *XML*;
- должен возвращаться только один результирующий набор.

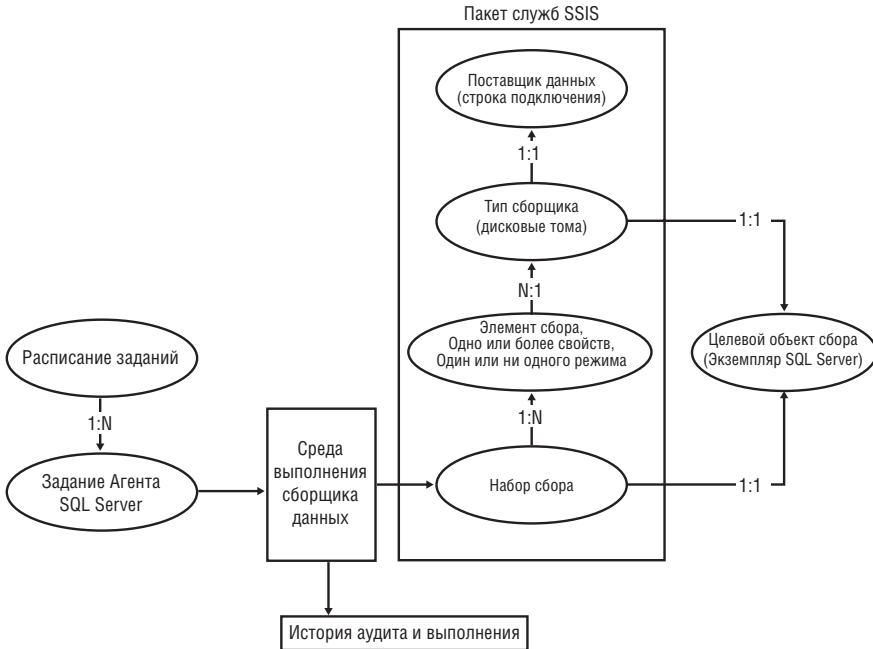


Рис. 13-6. Сборщик данных

Сборщик трассировки SQL поддерживает как стандартную трассировку, так и определенную пользователем. Результаты трассировки помещаются в файл, а сборщик данных использует функцию *fn_trace_gettable* для получения содержимого файла в виде результирующего набора и последующей загрузки его в хранилище данных о производительности. Для сбора данных могут быть указаны любые возможные параметры трассировки (например, файлы продолжения и фильтры). Результаты сборки данных сохраняются в таблицах хранилища данных о производительности *snapshots.trace* и *snapshots.trace_data*.

Сборщик счетчика производительности позволяет определять любой набор объектов, счетчиков и экземпляров счетчиков. Результаты сбора данных могут сохраняться в таблице хранилища данных о производительности *snapshots.performance_counters*.

Сборщик операций запроса получает информацию из динамических административных представлений *sys.dm_exec_requests*, *sys.dm_exec_sessions* и *sys.dm_exec_query_stats*.

ПРИМЕЧАНИЕ На экзамене

Необходимо знать, для чего нужно Хранилище данных о производительности, на каких компонентах основана сборка данных и какую информацию можно собирать.

Контрольные вопросы

1. На каких элементах основано хранилище данных о производительности?
2. Какие типы сборщиков существуют в SQL Server 2008?

Ответы на контрольные вопросы

1. Хранилище данных о производительности построено на основе инфраструктуры сборщика данных. Сборка базируется на пакетах SSIS и заданиях Агента SQL Server.
2. SQL Server 2008 поставляется со сборщиками запросов T-SQL, трассировки T-SQL, операций запросов и счетчиков производительности.

Практикум. Настройка хранилища данных о производительности

Вы настроите хранилище данных о производительности, сконфигурируете встроенные наборы сбора данных и проанализируете результаты.

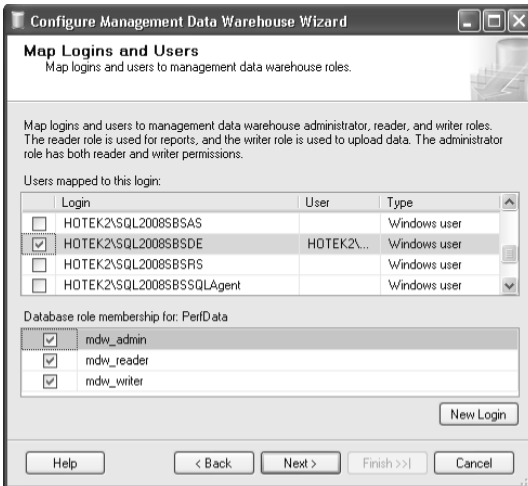
Задание 1. Настройка хранилища данных о производительности

Вы настроите хранилище данных о производительности.

1. Запустите SSMS, подключитесь к своему экземпляру, раскройте узел **Управление** (Management), щелкните правой кнопкой **Сбор данных** (Data Collection) и выберите **Настройка хранилища управляющих данных** (Configure Management Data Warehouse).
2. Щелкните **Далее** (Next), выберите **Создать или обновить хранилище управляющих данных** (Create Or Upgrade A Management Data Warehouse), как показано ниже, и снова щелкните **Далее**.



3. Создайте новую базу данных с именем PerfData и параметрами по умолчанию. Щелкните **Далее**.
4. Выберите имя входа, соответствующее учетной записи службы SQL Server (на этом занятии он называется SQL2008SBSDE) и роль mdw_admin, как показано ниже. Щелкните **Далее**.



5. Щелкните **Готово** (Finish), чтобы создать структуру базы данных *PerfData*.
6. Ознакомьтесь с объектами, созданными в базе данных *PerfData*.

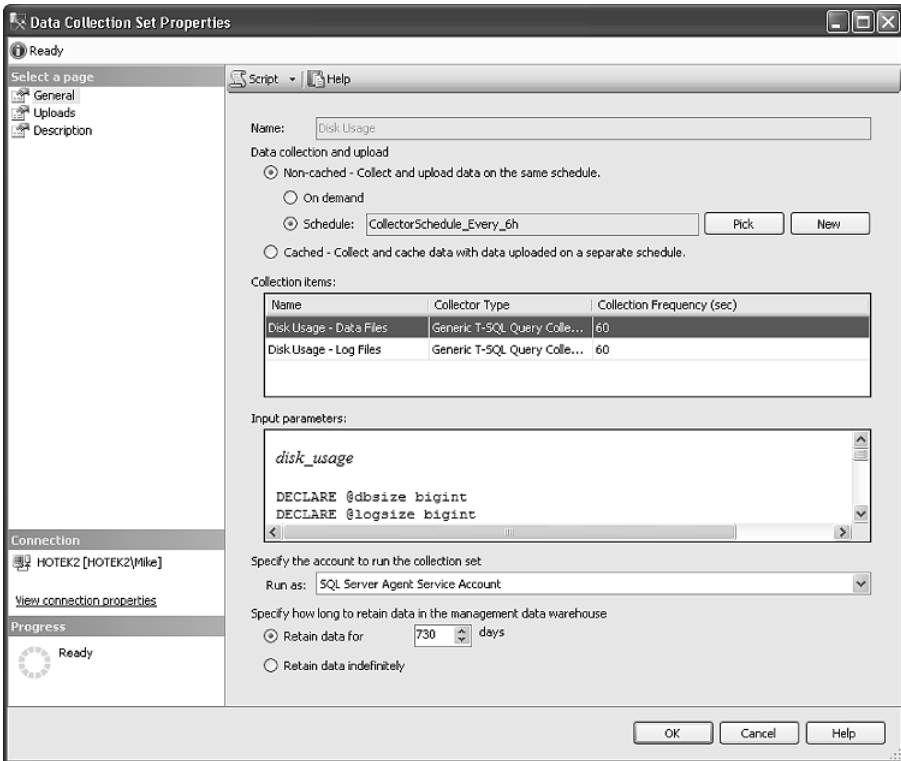
Задание 2. Настройка сбора данных

Вы настроите сбор данных для только что созданного хранилища управляющих данных.

1. Щелкните правой кнопкой узел **Сбор данных** (Data Collection) и выберите **Настройка хранилища управляющих данных** (Configure Management Data Warehouse).
2. Щелкните **Далее** и выберите **Настроить сбор данных** (Set Up Data Collection), как показано ниже, и снова щелкните **Далее**.



3. Выберите базу данных *PerfData* и оставьте пустым поле **Каталог кеша** (Cache directory).
4. Щелкните **Готово**.
5. Раскройте папку **Системные наборы сбора данных** (System Data Collection Sets), щелкните правой кнопкой сборщик **Использование места на диске** (Disk Usage) и выберите **Свойства** (Properties). Изучите параметры набора сбора данных, как показано ниже.



Задание 3. Изучение результатов сбора данных

Вы изучите стандартные отчеты, созданные системными наборами сбора данных.

1. Щелкните правой кнопкой узел **Сбор данных** (Data Collection), выберите **Отчеты** (Reports), а затем — **Management Data Warehouse** и **Disk Usage Summary**.
2. Изучите результаты, которые должны выглядеть, как показано ниже.

Disk Usage Su...24 AM - HOTEK2

Disk Usage Su...20 AM - HOTEK2

Object Explorer Details

Disk Usage Collection Set

on HOTEK2 at 11/6/2008 2:24:28 AM

Microsoft

SQL Server 2008

This report provides an overview of the disk space used for all databases on the server and growth trends for the data file and the log file for each database for the last 3 collection points between 11/6/2008 2:01:32 AM and 11/6/2008 2:05:52 AM.

Database Name	Database				Log			
	Start Size (MB)	Trend	Current Size (MB)	Average Growth (MB/Day)	Start Size (MB)	Trend	Current Size (MB)	Average Growth (MB/Day)
AdventureWorks	194.75		194.75	0	2.00		2.00	0
AdventureWorksDw	83.50		83.50	0	2.00		2.00	0
DBAdmin	2.25		2.25	0	1.00		1.00	0
master	4.00		4.00	0	1.00		1.00	0
model	1.25		1.25	0	0.50		0.50	0
msdb	14.06		14.06	0	1.75		1.75	0
PerfData	100.00		100.00	0	10.00		10.00	0
ReportServer	3.25		3.25	0	6.13		6.13	0
ReportServerTempDB	3.25		3.25	0	1.25		1.25	0
SQL2008SBS	16.00		16.00	0	9.50		9.50	0
SQL2008SBSDw	16.25		16.25	0	52.00		52.00	0

Резюме занятия

- Сборщик данных — это новый элемент инфраструктуры SQL Server 2008, основанный на пакетах SSIS и заданиях Агента SQL Server.
- Можно создавать сборщики различных типов: сборщики запросов T-SQL, трассировки SQL, счетчиков производительности и операций запросов.
- Сборщик запросов T-SQL наиболее гибкий и позволяет выбрать инструкцию *SELECT*, а также базу данных, где она будет выполняться.
- Вся информация, полученная сборщиком данных, направляется в хранилище данных о производительности вместе с параметрами всех наборов сбора данных.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. В результате недавних приобретений в компании Humongous Insurance сейчас есть экземпляры разных версий SQL Server — от 6.5 до 9.0. Для управления нагрузкой в среде SQL Server применялись разнообразные продукты сторонних производителей и пользовательский код. Руководство требует объединить все на одной платформе, способной выполнять задачи по управлению нагрузкой и анализу производительности путем сравнения с базовым уровнем. Требуется реализовать наиболее дешевое решение, требующее минимума усилий по настройке и сопровождению. Какое решение вы предложите?

- A. Установить экземпляр SQL Server 2008 и внедрить управление на основе политик.
- B. Установить экземпляр SQL Server 2008 и развернуть хранилище данных о производительности.
- C. Установить экземпляр SQL Server 2008 и переписать все, используя SSIS.
- D. Внедрить Microsoft System Center Operations Manager 2007.

Закрепление материала главы

Для закрепления знаний и навыков, полученных при изучении этой главы, выполните следующее:

- изучите резюме главы;
- повторите основные термины, использованные в этой главе;
- выполните задания по темам этой главы;
- сделайте предложенные упражнения;
- пройдите пробный экзамен.

Резюме главы

- Регулятор ресурсов позволяет объединять подключения в группы рабочей нагрузки, которые могут присваиваться пулам ресурсов, ограничивающим доступные для подключений ресурсы процессора и памяти.
- DTA оценивает одну или несколько инструкций SQL и предоставляет рекомендации по созданию или удалению индексов для увеличения производительности.
- Динамические административные представления (DMV) — это набор представлений и функций, поставляемых с SQL Server, которые предоставляют системную и диагностическую информацию в формате, удобном для использования и обработки.
- Хранилище данных о производительности использует инфраструктуру сборщика данных для объединения информации, полезной для управления нагрузкой и анализа тенденций производительности.

Основные термины

Понятны ли вам значения приведенных здесь основных терминов? Чтобы проверить свои ответы, загляните в глоссарий на прилагаемом компакт-диске.

- Функция-классификатор ~ classification function.
- Элемент сбора данных ~ collection item.
- Набор сбора данных ~ collection set.
- Целевой объект сбора данных ~ collection target.
- Сборщик данных ~ Data collector.
- Поставщик данных ~ data provider.
- Динамическая административная функция ~ Dynamic Management Function (DMF).

- Динамическое административное представление ~ Dynamic Management View (DMV).
- Пул ресурсов ~ resource pool.
- Файл рабочей нагрузки ~ workload file.
- Группа рабочей нагрузки ~ workload group.

Упражнения

Сейчас вы примените на практике знания и навыки, полученные при изучении этой главы. Ответы на вопросы см. в разделе «Ответы» в конце книги.

Упражнение. Разработка стратегии автоматизации для компании Coho Vineyard

Исходные данные

Основные сведения о компании

Компания Coho Vineyard была основана в 1947 г. как частный семейный винный завод. Вина, производимые Coho Vineyard, удостоивались многих наград и пользовались популярностью, что позволило компании значительно расширить производство. Через несколько лет в рамках стратегии по расширению компании были приобретены несколько существующих винных заводов. Сейчас компания владеет 16 винными заводами — 9 в штатах Вашингтон, Орегон и Калифорния, а остальные 7 в штатах Висконсин и Мичиган. В компании работает 532 человека, из которых 162 — в штаб-квартире, где размещаются критически важные для бизнеса серверы. В компании трудится 122 мобильных торговых агента, которым требуется доступ к актуальной информации об имеющихся товарах.

Планируемые изменения

До настоящего времени у каждого из 16 заводов Coho Vineyard был собственный локальный веб-сайт. Компания Coho Vineyard планирует объединить веб-ресурсы всех заводов, чтобы клиенты могли покупать продукцию любого из 16 заводов в едином онлайн-магазине. Все данные этого веб-сайта будут храниться в базе данных в штаб-квартире.

Пока проект консолидации данных не завершен, в конце рабочего дня данные о запасах с каждого завода пересылаются в штаб-квартиру для обслуживания торговых агентов. Для обеспечения торговых представителей локальными копиями данных о клиентах, товарах и заказах налажена репликация слиянием.

Существующая инфраструктура данных

Базы данных

Сейчас каждый винный завод поддерживает собственную базу данных, где хранит всю бизнес-информацию. В конце месяца вся информация переносится в штаб-квартиру и загружается в базы данных, перечисленные в табл. 13-2.

Табл. 13-2. Базы данных Coho Vineyard

База данных	Объем
<i>Customer</i> (данные о клиентах)	180 Мб
<i>Accounting</i> (бухгалтерские данные)	500 Мб
<i>HR</i> (данные о кадрах)	100 Мб
<i>Inventory</i> (складские запасы)	250 Мб
<i>Promotions</i> (сведения о поощрениях)	80 Мб

В результате консолидации баз данных будет создана новая база данных *Order*, которая будет работать хранилищем данных нового онлайн-магазина. В процессе своей ежедневной работы сотрудники будут периодически обращаться к базе данных *Order*, используя новое разработанное собственными силами веб-приложение.

База данных *HR* содержит конфиденциальную информацию и защищена с использованием TDE (Transparent Data Encryption). Кроме того, данные в таблице *Salary* (данные о зарплатах) зашифрованы с использованием сертификатов.

Серверы баз данных

Все базы данных в штаб-квартире хранятся в SQL Server 2008 Enterprise на одном сервере DB1 под управлением Windows Server 2003 Enterprise.

Руководитель ИТ-отдела изучает возможность покупки нового компьютера для замены сервера DB1, поскольку пользователи жалуются на эпизодические проблемы с производительностью. Кроме этого несколько заводов сообщали об ошибке активации устройства и даже о нескольких случаях синих экранов при запуске на сервере их локальных экземпляров SQL Server.

Бизнес-требования

Вам необходимо разработать решение архивирования баз данных *Customer* и *Order*. Стратегия архивирования должна обеспечивать хранение данные базы *Customer* в течение 6 лет.

В процессе подготовки БД *Order* к архивированию создана секционированная таблица *Order.Sales*, состоящая из двух секций: в первой хранится информация о продажах за текущий месяц, а во второй — за предыдущий. Заказы, размещенные еще раньше, будут перемещены в другую секционированную таблицу — *Order.Archive*, после чего в секции 1 будут содержаться все архивные данные, а секция 2 останется пустой.

Нужно создать новый процесс, который ежедневно не позднее 4 часов утра будет загружать информацию о складских запасах со всех 16 заводов.

Четыре крупных клиента осуществляют заказы, используя XML-схему Coho Vineyards для EDI-транзакций (Electronic Data Interchange). EDI-файлы поступают в 5 часов вечера и должны обрабатываться и загружаться в БД *Customer*, *Accounting* и *Inventory*, каждая из которых содержит таблицы с данными, относящимися к размещаемым заказам. В настоящее время утилита импорта EDI-файлов представляет собой однопоточное C++-приложение, которому требуется от 3—6 часов на обработку всех файлов. Чтобы выполнить соглаше-

ние с клиентами об условиях обслуживания (Service Level Agreement, SLA), необходимо заканчивать обработку EDI-файлов до 17:30. К моменту завершения проекта по объединению EDI-утилиты должна загружать информацию в новую базу данных *Order*.

Сообщалось о многочисленных конфликтах на центральном сервере SQL Server во время импорта данных в процессе ночной консолидации. Требуется уменьшить количество конфликтов, либо совсем исключить их.

Требуется обеспечить создание резервных копий всех баз данных всех филиалов. Все производственные базы данных должны работать в режиме полной модели восстановления. В самом плохом случае считается приемлемым потеря данных за последние 5 минут. В случае аварии БД *Customer*, *Inventory*, *Promotions* и *Order* могут быть недоступны максимум 20 минут. В случае аварийного сбоя информация БД *Customer* и *Order* старше полугода может быть недоступна максимум 12 часов.

Ответьте на следующие вопросы:

1. Как определить причину низкой производительности?
2. Как устранить неполадки, возникающие на заводах?

Рекомендуемые упражнения

Чтобы подготовиться к экзамену по темам этой главы, выполните следующие задания.

Сбор информации средствами хранилища данных о производительности для оптимизации производительности

- **Задание 1.** Во всех экземплярах SQL Server версии 2005 или выше, настройте набор сбора информации об операциях запросов.
- **Задание 2.** Чтобы определить базовый уровень производительности, настройте набор сбора данных счетчика производительности для всех экземпляров, собирающих информацию о System, Processor, Network, Physical Disk, а также обо всех объектах производительности SQL Server.
- **Задание 3.** Настройте набор сбора трассировки SQL, включающий события *RPC:Completed* и *SQL:BatchCompleted*, для получения базовых показателей производительности при выполнении запросов.
- **Задание 4.** Настройте набор сбора данных о запросах T-SQL, чтобы получить информацию о приложении для последующего анализа.
- **Задание 5.** Создайте единое хранилище данных о производительности и объедините в нем информацию, полученную с нескольких целевых объектов сбора данных.
- **Задание 6.** Используя службу отчетов, определите пользовательский отчет на основании собираемых данных.

Сбор информации средствами Помощника по настройке ядра СУБД для оптимизации производительности

- **Задание 1.** Используя результаты набора сбора данных о трассировке SQL, размещенные в хранилище данных о производительности, выполните анализ при помощи DTA.

Сбор информации средствами динамических административных представлений для оптимизации производительности

- **Задание 1.** Используя сборщик данных о запросах T-SQL, настройте сбор информации для динамических административных представлений *sys.dm_db_** и *sys.dm_io_**.

Пробный экзамен

На прилагаемом компакт-диске содержится пробный экзамен, обеспечивающий широкие возможности самоподготовки. Например, можно проверить, как усвоены темы этой главы либо все темы экзамена 70-432. Можно симитировать настоящий экзамен или выбрать режим обучения, когда после каждого вопроса приводятся правильные ответы с пояснениями.

ПРИМЕЧАНИЕ Пробный экзамен

Подробнее о пробном экзамене см. «Введение».

Отказоустойчивые кластеры

Занятие 1. Проектирование кластеров Windows **373**

Занятие 2. Проектирование отказоустойчивых кластеризованных экземпляров SQL Server 2008 **391**

Отказоустойчивые кластеры Microsoft SQL Server создаются на основе кластеров Microsoft Windows и предназначены для защиты систем от аппаратных сбоев. В этой главе рассматриваются конфигурации кластеров Windows и отказоустойчивых кластеров SQL Server.

Темы экзамена

- Создание экземпляра кластера SQL Server.

Прежде всего

Для изучения материалов этой главы необходимо иметь:

- поддерживающее кластеризацию оборудование или сервер Microsoft Virtual Server 2005 R2;
- сервер под управлением ОС Windows Server 2003 SP2 или более позднего выпуска или Windows Server 2008.

ПРИМЕЧАНИЕ Виртуальный сервер

Для моделирования аппаратных конфигураций можно использовать Virtual Server или Microsoft Virtual PC. В отличие от Virtual PC, Virtual Server поддерживает кластеры Windows и пригоден для построения отказоустойчивых кластеров SQL Server.

ВНИМАНИЕ! Упражнения к занятию

Используйте Virtual Server при выполнении всех упражнений этой главы. Для этого потребуется создать три виртуальных машины на основе Virtual Server и установить на каждой из них ОС Windows Server 2003 Standard Edition SP2 или более позднего выпуска или Windows Server 2008 Standard или более позднего выпуска. Одну из виртуальных машин следует настроить как контроллер домена, а две другие — как рядовые серверы в этом домене. Рядовым серверам выделите по 512 Мб, а контроллеру домена — 192 Мб оперативной памяти. Чтобы обеспечить приемлемую производительность такой конфигурации, Vir-

tual Server должен оснащаться как минимум 1,5 Гб оперативной памяти и жесткими дисками со скоростью вращения шпинделя не менее 7200 об/мин.

Перед выполнением упражнений в этих занятиях позаботьтесь о следующем:

- создайте три виртуальные машины;
- установите на каждую виртуальную машину ОС Windows Server 2003 Standard Edition или более позднего выпуска или Windows Server 2008 Standard Edition или более позднего выпуска;
- настройте одну виртуальную машину как контроллер домена;
- настройте две виртуальные машины как рядовые серверы в домене;
- настройте на контроллере домена один сетевой адаптер, как показано в табл. 14-1;
- настройте на рядовых серверах два сетевых адаптера, как показано в табл. 14-1;
- настройте все сети на режим Guest Only.

Табл. 14-1. Конфигурация адресов TCP/IP в сетях

Компьютер	Подключение	Конфигурация IP
Контроллер домена	Подключение по локальной сети	IP-адрес: 10.1.1.1 Маска подсети: 255.255.255.0 Шлюз: 10.1.1.1 DNS-сервер: 10.1.1.1
Рядовой сервер (Node1)	Подключение по локальной сети	IP-адрес: 10.1.1.2 Маска подсети: 255.255.255.0 Шлюз: 10.1.1.1 DNS-сервер: 10.1.1.1
Рядовой сервер (Node1)	Подключение по локальной сети 2	Назначается динамически
Рядовой сервер (Node2)	Подключение по локальной сети	IP-адрес: 10.1.1.3 Маска подсети: 255.255.255.0 Шлюз: 10.1.1.1 DNS-сервер: 10.1.1.1
Рядовой сервер (Node2)	Подключение по локальной сети 2	Назначается динамически

ВНИМАНИЕ!

Подробное рассмотрение технологии Virtual Server выходит за рамки этой книги. Пошаговые инструкции о выполнении всех операций по конфигурированию базовой среды см. в документации к Virtual Server. При наличии физического оборудования, на котором можно разместить кластер, упражнения можно выполнять на нем, пропустив шаги, касающиеся настройки среды Virtual Server.

Занятие 1. Проектирование кластеров Windows

Кластеры Windows служат основой для построения отказоустойчивых кластеров SQL Server. В этом занятии приводятся инструкции и рекомендации по конфигурированию кластеров Windows.

ВНИМАНИЕ! Совместимое оборудование

Чаще всего причиной сбоев кластеров становится использование оборудования, не сертифицированного для создания кластеров. Следует убедиться, что кластер создается на сертифицированном для этого оборудовании, которое указано в каталоге Windows Catalog. Используемое аппаратное решение должно быть целиком сертифицировано для кластеризации, поэтому обязательно проверьте категории кластеризации в каталоге Windows (www.microsoft.com/whdc/hcl/default.msp).

ПРИМЕЧАНИЕ Кластеры Windows

Технические документы, веб-трансляции, блоги и другие ресурсы, посвященные кластерам Windows, см. на веб-странице www.microsoft.com/windowsserver2003/community/centers/clustering.

Изучив материал этого занятия, вы сможете:

- ✓ проектировать реализацию службы кластеров Microsoft (MSCS).

Продолжительность занятия — 45 минут.

Компоненты кластеров Windows

Кластер Windows представляет собой единую платформу, состоящую из нескольких серверов, на которой выполняются приложения. Каждый сервер в кластере называется *узлом кластера* (cluster node).

ПРИМЕЧАНИЕ Версии ОС Windows Server

На момент написания данной книги ОС Windows Server 2008 только появилась на рынке. Упражнения в этой главе, а также подробная информация о кластерах Windows в основном относятся к ОС Windows Server 2003, а о Windows Server 2008 приведены только те сведения, которые удалось получить на этот момент. Об особенностях кластеров на основе ОС Windows Server 2008 см. документацию к данной ОС.

Прежде всего устанавливают узлы кластера под управлением ОС Windows Server 2003 или Windows Server 2008. Возможное количество узлов зависит от выбранного выпуска (см. табл. 14-2).

Табл. 14-2. Количество узлов кластера

Версия	Выпуск	Максимальное число узлов
Windows Server 2003	Standard	2
Windows Server 2003	Enterprise	4
Windows Server 2003	Datacenter	8
Windows Server 2008	Standard	2
Windows Server 2008	Enterprise	16

Каждому кластеру Windows назначается уникальное имя и IP-адрес. *Имя кластера* регистрируется в службе DNS и разрешается в сети.

Создается *база данных кворума*, которая содержит все сведения о конфигурации кластера.

Все узлы кластера должны быть членами одного домена Windows. Нужно создать доменную учетную запись, которая будет использоваться в качестве учетной записи администратора кластера.

Наиболее сложные элементы кластера — группы и ресурсы. *Группа кластера* — это логическое имя, которое назначается контейнеру, содержащему один или несколько ресурсов кластера. *Ресурсы кластера* — это любые элементы, которые можно настроить на сервере, например IP-адрес, сетевые имена, дисковые накопители, службы Windows и общие папки.

На рис. 14-1 представлена простая схема кластера из двух узлов.

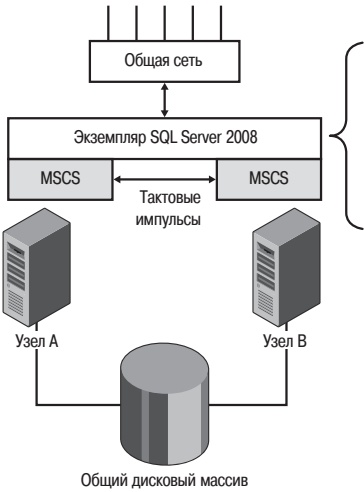


Рис. 14-1. Windows-кластер с двумя узлами

Типы кластеров

Windows Server 2003 и Windows Server 2008 поддерживают стандартные кластеры и кластеры с набором узлов большинства.

Стандартные кластеры Windows

Стандартный кластер (рис. 14-1) содержит одну базу данных кворума, которая хранится на общем дисковом массиве. В каждый момент времени лишь один узел кластера имеет доступ к диску кворума, у остальных узлов доступа к этому диску нет. В случае сбоя другой узел становится владельцем дискового ресурса, содержащего базу данных кворума, и работа кластера продолжается.

Кластеры с набором узлов большинства

Основное отличие *кластеров с набором узлов большинства* (majority node set cluster) состоит в том, что копия базы данных кворума хранится локально на каждом узле кластера.

ПРИМЕЧАНИЕ Локальный кворум

База данных кворума хранится в папке `%SystemRoot%\Cluster\QoN.%ResourceGUID%$\%GUID ресурса%\MSCS`. На каждом узле кластера создается общая папка с именем `\\%Имя_узла%\%GUIDРесурса%$`. Не изменяйте эту папку и ее разрешения, а также не предоставляйте общего доступа к ней.

Кластер с набором узлов большинства назван так потому, что для его работы необходима доступность большинства его узлов. Поэтому кластеры с набором узлов большинства можно создавать лишь при наличии трех и более узлов в кластере. В табл. 14-3 показано, при каком количестве недоступных узлов сохраняют работоспособность стандартные кластеры и кластеры с набором узлов большинства.

Табл. 14-3. Отказоустойчивость кластеров

Количество узлов	Кластер с набором узлов большинства: допустимое число нерабочих узлов	Стандартный кластер: допустимое число нерабочих узлов
1	0	0
2	0	1
3	1	2
4	1	3
5	2	4
6	2	5
7	3	6
8	3	7

При изучении табл. 14-3 возникает законный вопрос: зачем вообще нужны кластеры с набором узлов большинства, ведь они на первый взгляд менее отказоустойчивы, чем стандартные кластеры.

База данных кворума содержит сведения о конфигурации кластера и управляет работой кластера. В случае ее недоступности весь кластер выходит из строя. В стандартном кластере есть единственная база данных кворума, размещенная на единственном общем дисковом массиве. В результате сбоя общего

дискового массива или повреждения базы данных кворума весь кластер становится недоступным. В кластере с набором узлов большинства одинаковые копии базы данных кворума хранятся на каждом узле и синхронизируются друг с другом, поэтому база данных кворума не становится единственной точкой отказа в кластере.

Конфигурация безопасности

При настройке каждого узла в кластере необходимо учитывать все рекомендации по безопасности Windows. Отключите все ненужные службы.

Создайте в домене, где находится кластер, учетную запись, которая должна использоваться в качестве учетной записи администратора кластера. Прежде чем переходить к конфигурированию кластера, добавьте эту учетную запись в группу локальных администраторов на каждом узле кластера.

ВНИМАНИЕ! Операционная система с шифрованием

Для поддержки шифрования файловой системы в конфигурации кластера следует включить Kerberos и настроить доверие по отношению к учетным записям компьютера и службы кластера. При шифровании файловой системы также необходимо учитывать, что снизится производительность операций чтения и записи из-за шифрования и расшифровки.

Для службы кластеров нельзя использовать учетную запись рядового пользователя. Эта служба должна подключать и отключать диски, останавливать и запускать службы, выполнять операции чтения и записи в реестр и другие задачи, которые доступны только локальным администраторам.

Дисковая конфигурация

Для создания кластеров можно использовать диски SCSI (iSCSI) или Fibre Channel. Диски IDE для этой цели не подходят. Кластер, содержащий более двух узлов под управлением ОС Windows Datacenter или 64-разрядной версии Windows, можно создать только с применением дисков Fibre Channel.

Кластеры не поддерживают динамические диски — для них подойдут только базовые диски и точки подключения. Буквы диска A, B, C и D уже заняты для обозначения локальных ресурсов на каждом узле, поэтому в общей сложности можно использовать 22 буквы диска.

ПРИМЕЧАНИЕ Операционная система

Узнайте у производителя SAN, возможна ли загрузка узлов с диска SAN. Если это невозможно или используется напрямую подключенное хранилище, операционную систему следует установить на внутреннем жестком диске, с которого выполняется загрузка узла. Наиболее распространена конфигурация, в которой ОС размещена на внутреннем жестком диске на каждом узле.

При конфигурировании дисков отдельный диск следует выделить для кворума.

Во всех кластерах настройте координатор распределенных транзакций Microsoft (MS DTC). После создания кластера в него необходимо вручную добавить ресурс MS DTC и выделить ему место на диске, сконфигурированном как зависимый от этого ресурса.

Выбор диска для ресурса MS DTC — нелегкая задача для большинства администраторов. С одной стороны, требуется обеспечить как можно больше букв диска для работы с базами данных, а с другой — учесть рекомендации при определении разумного соотношения между производительностью и стабильностью. Опыт показывает, что лучше сначала выделить отдельный диск ресурсу MS DTC, а затем настроить координатор MS DTC и связанный с ним жесткий диск в отдельной группе кластера.

Если координатор MS DTC не задействован в приложениях, эффективнее использовать этот жесткий диск для баз данных. Итак, при недостатке жестких дисков для создания конфигурации диска, поддерживающей работу баз данных, или в случае, если координатор MS DTC не используется ни для каких приложений, ресурс MS DTC можно поместить в группу кластера и настроить зависимость от физического диска, выделенного под кворум. Такая конфигурация противоречит рекомендациям, но она позволяет без ущерба для работы кластера высвободить один лишний диск, при условии что тот не используется координатором MS DTC.

ВНИМАНИЕ! Антивирусное программное обеспечение

В последнее время антивирусные программы очень часто устанавливают на серверы баз данных. В кластере антивирус следует настроить так, чтобы он не мешал работе кластера. Из сканирования необходимо исключить каталог MSCS и все папки, содержащие файлы данных. При сбое диски подключаются к тому узлу, на который происходит переключение, а затем запускается сканирование диска антивирусом. Если антивирус начнет сканировать файл базы данных до того, как он будет открыт сервером SQL Server, восстановление базы данных будет отложено до завершения полной проверки файла. Как правило, файлы баз данных очень большие, поэтому их сканирование может существенно замедлить процесс перехода при отказе.

Конфигурация сети

На каждом узле в кластере Windows должны быть как минимум две сетевые карты — для общей и частной сети. Общая сеть служит точкой доступа для всего трафика приложений и внешних систем, запрашивающих данные у кластера, а частная сеть используется для взаимодействия узлов внутри кластера.

В кластере Windows периодически выполняются проверки работоспособности для контроля доступности узлов и возможности запуска на них приложений. Чаще всего для этого используется тест LooksAlive, заключающийся в обмене эхо-запросами (ping) узлами кластера. Если узел не проходит этот тест, он считается недоступным и кластер переключается на резервный узел.

При высокой нагрузке на частную сеть тест LooksAlive может дать сбой, что приведет к ошибочному переключению. Чтобы избежать подобной ситуации, частную и общую сеть следует размещать в различных подсетях.

Практический совет. Подключение к частной сети

При настройке подключения к частной сети выполните следующее:

- отключите все службы за исключением службы TCP/IP;

- удалите адрес шлюза по умолчанию;
 - удалите все адреса DNS-серверов;
 - отключите регистрацию в DNS;
 - отключите NetBIOS через TCP/IP;
 - отключите просмотр LMHOSTS.

В такой конфигурации сеть будет обрабатывать только трафик TCP/IP и будут использоваться только известные IP-адреса.

ПРИМЕЧАНИЕ Удаленный вызов процедур (RPC)

Для всех проверок работоспособности кластера применяется служба RPC. Если она недоступна или была отключена, любые проверки работоспособности кластера не пройдут. Убедитесь, что служба RPC включена и настройте ее автоматический запуск на всех узлах кластера.

Ресурсы кластера

Ресурсы кластера, наиболее низкоуровневые конфигурируемые в кластере объекты, можно разбить на несколько общих категорий, перечисленных в табл. 14-4.

Табл. 14-4. Ресурсы кластера

Категория	Примеры
Сетевые подключения	IP-адрес, сетевое имя
Оборудование	Дисковые накопители
Программное обеспечение	Службы, исполняемые файлы, общие папки, MS DTC

Физически подключенные к компьютеру ресурсы не могут настраиваться средствами кластера. Так могут ли дисковые накопители быть ресурсами кластера? Как говорилось ранее, все данные в кластере должны храниться на внешнем дисковом массиве. Этот массив может представлять собой дисковую стойку с интерфейсом Fibre Channel, подключаемую к каждому узлу кластера или дисковое хранилище SAN, подключенное ко всем узлам кластера. Локальные жесткие диски на узлах кластера нельзя настроить в качестве ресурса кластера.

Физические дисковые накопители в дисковом массиве сами по себе не являются ресурсами кластера. Определение подключения диска в среде Windows создается и находится под управлением службы кластеризации Windows. И хотя дисковый ресурс определяется на всех узлах, диск подключается и доступен лишь на узле, который настроен как владелец данного дискового ресурса. На всех остальных узлах хранится определение подключения диска, но сами диски не подключены. Таким образом, несколько компьютеров не могут одновременно записывать данные на один носитель.

Основной ресурс, настраиваемый в кластере, это службы, например служба SQL Server или агент SQL Server. Хотя на каждом узле кластера есть записи соответствующей службы, сама служба запускается только на одном узле кластера.

Один из наиболее важных аспектов работы кластера — порядок работы с IP-адресами и сетевыми именами. Хотя у каждого узла кластера есть IP-адрес и определение сетевого имени, но лишь на узле, настроенном как владелец

данного IP-адреса и имени, они привязаны к физической сетевой карте. При переходе на другой узел в случае сбоя служба кластеризации выполняет следующие операции в сетевом стеке:

- 1. Отменяет регистрацию сетевого имени в DNS.
- 2. Связывает IP-адрес с физической сетевой картой на рабочем узле.
- 3. Регистрирует сетевое имя в DNS.

Такой процесс гарантирует, что во всех приложениях используются одинаковые IP-адрес и сетевое имя независимо от того, на каком узле они в данный момент работают. Благодаря сохранению IP-адреса и сетевого имени после перехода на другой узел не требуется изменять настройку приложений для повторного подключения после сбоя.

Группы кластера

Группы кластера служат для объединения одного или нескольких ресурсов кластера в логическую управляемую структуру (рис. 14-2). Группа является единицей перехода на резервный узел в случае сбоя кластера. Группу кластера также можно рассматривать как приложение. Каждый создаваемый отказоустойчивый кластеризованный экземпляр SQL Server выступает в качестве отдельной группы в кластере Windows.

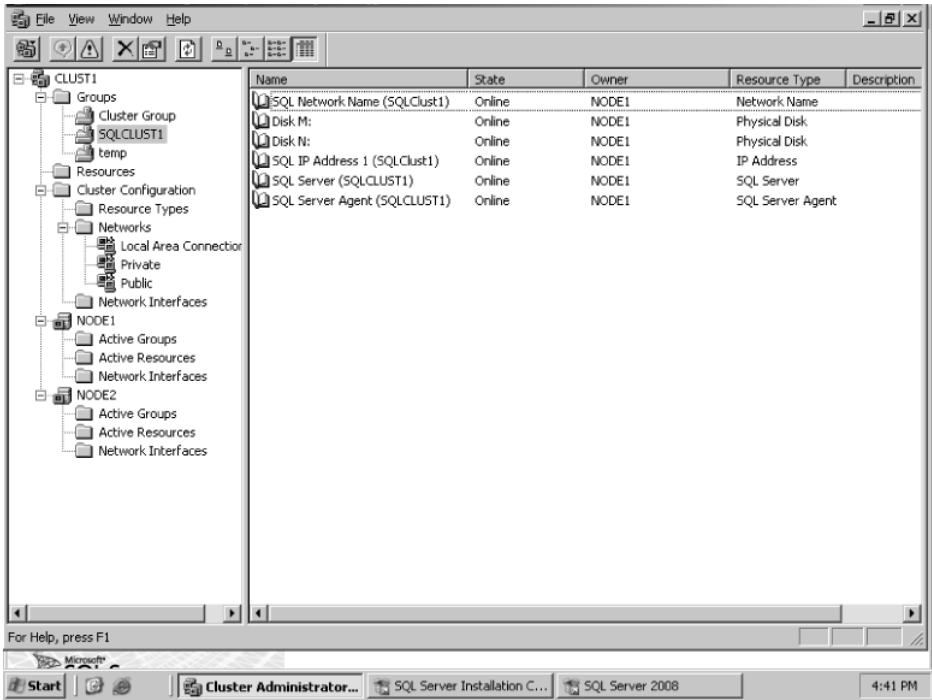


Рис. 14-2. Группа кластера и ее ресурсы

Контрольные вопросы

1. Каково основное отличие стандартного кластера от кластера с набором узлов большинства.
2. Приведите примеры ресурсов кластера.
3. Сколько сетевых подключений необходимо для узла кластера? Почему?
4. Как выполняется проверка работоспособности в кластере Windows?
5. Какие конфигурации диска поддерживаются кластером?

Ответы на контрольные вопросы

1. В стандартном кластере используется общая база данных кворума. В кластере с набором узлов большинства на каждом узле поддерживается отдельная база данных кворума, которая синхронизируется с базами данных кворума на остальных узлах. В кластере с набором узлов большинства должны быть доступны большинство узлов (более половины).
2. К ресурсам кластера относятся оборудование, программное обеспечение и сетевые подключения, например IP-адреса, сетевые имена, подключаемые диски и службы Windows.
3. Для каждого узла необходимо минимум два подключения: общее для взаимодействия с приложениями в сети и частное для взаимодействия внутри кластера.
4. Для базовой проверки работоспособности используют тест LooksAlive, заключающийся в обмене узлов эхо-запросам.
5. Кластеры поддерживают базовые диски, а динамические нет. Диски также должны быть внешними по отношению к узлам кластера, поэтому диски, подключенные локально к узлам кластера, не видны другим ресурсам кластера.

Практикум. Создание кластера Windows

Вы создадите кластер Windows, который будет использован в занятии 2 для установки отказоустойчивого кластеризованного экземпляра SQL Server.

1. Откройте веб-сайт **Virtual Server Administration**.
2. Запустите клиент для удаленного управления виртуальной машиной **Virtual Machine Remote Control Client** и подключитесь к своему экземпляру Virtual Server.
3. Убедитесь, что узлы Node1 и Node2 отключены. Запустите контроллер домена.
4. В разделе **Virtual Disks** веб-сайта Virtual Server Administration последовательно выберите команды **Create** и **Fixed Size Virtual Hard Disk**.
5. Назовите этот диск Quorum.vhd и выделите ему 500 Мб.
6. Повторив пп.4 и 5, создайте еще два диска: **Qqldata.vhd** — размером 1 Гб и **Sqllog.vhd** — размером 500 Мб.
7. На веб-сайте Virtual Server Administration щелкните **Edit Configuration** первого узла в кластере (далее именуемого Node1).

8. Позаботьтесь о настройке двух сетевых адаптеров. Если они не сконфигурированы, добавьте второй сетевой адаптер.
9. Щелкните ссылку **SCSI Adapters**.
10. Добавьте три адаптера SCSI, в качестве идентификатора адаптера SCSI укажите 6 и установите флажок **Share SCSI Bus For Clustering**, как показано в табл. 14-6.

Табл. 14-5. Конфигурация адаптера SCSI для узла Node1

Виртуальный адаптер	Идентификатор SCSI
Virtual SCSI Adapter 1	6 (Share SCSI Bus for Clustering)
Virtual SCSI Adapter 2	6 (Share SCSI Bus For Clustering)
Virtual SCSI Adapter 3	6 (Share SCSI Bus for Clustering)

11. Щелкните ссылку **Hard Disks**.
12. Щелкните **Add Disk** и добавьте диски Quorum.vhd, Sqldata.vhd и Sqllog.vhd. Подключите диски, как показано в табл. 14-6.

Табл. 14-6. Конфигурация дисков кластера для узла Node1

Диск	Имя	Подключение
Virtual Hard Disk 1	<i>(Имя диска на основной машине)</i>	Основной канал (0)
Virtual Hard Disk 2	Quorum.vhd	SCSI 0 ID 0 (общая шина)
Virtual Hard Disk 3	Sqldata.vhd	SCSI 1 ID 0 (общая шина)
Virtual Hard Disk 4	Sqllog.vhd	SCSI 2 ID 0 (общая шина)

13. Убедитесь, что заданная конфигурация соответствует табл. 14-6. Пример показан на рис. 14-3.

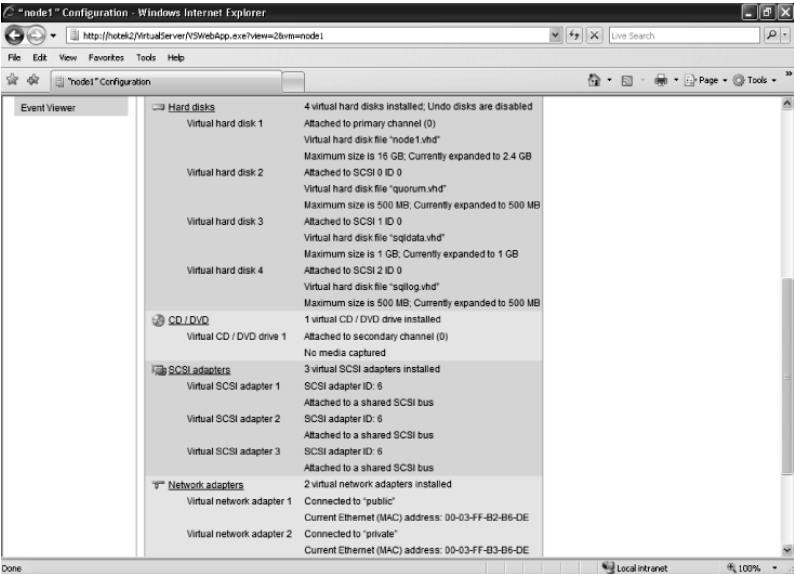


Рис. 14-3. Конфигурация Node1

14. Щелкните ссылку **Master Status** в разделе **Navigation**.
15. Повторите пп.7–13 для второго узла кластера (далее именуемого Node2).

ПРИМЕЧАНИЕ Идентификатор SCSI-адаптера узла Node2

Узлам следует задавать уникальные, отличные от других идентификаторы SCSI-адаптеров. В узле Node1 в качестве идентификатора SCSI-адаптера задано значение 6 для каждого адаптера SCSI, поэтому для узла Node2 нужно задать идентификатор 7 на всех SCSI-адаптерах.

16. Убедитесь, что заданные конфигурации соответствуют табл. 14-7 и 14-8. Пример см на рис. 14-4.

Табл. 14-7. Конфигурация SCSI-адаптера узла Node2

Виртуальный адаптер	Идентификатор SCSI
Virtual SCSI Adapter 1	7 (Share SCSI Bus For Clustering)
Virtual SCSI Adapter 2	7 (Share SCSI Bus For Clustering)
Virtual SCSI Adapter 3	7 (Share SCSI Bus For Clustering)

Табл. 14-8. Конфигурация диска кластера для узла Node2

Диск	Имя	Подключение
Virtual Hard Disk 1	(Имя диска на основной машине)	Основной канал (0)
Virtual Hard Disk 2	Quorum.vhd	SCSI 0 ID 0 (общая шина)
Virtual Hard Disk 3	Sqldata.vhd	SCSI 1 ID 0 (общая шина)
Virtual Hard Disk 4	Sqllog.vhd	SCSI 2 ID 0 (общая шина)

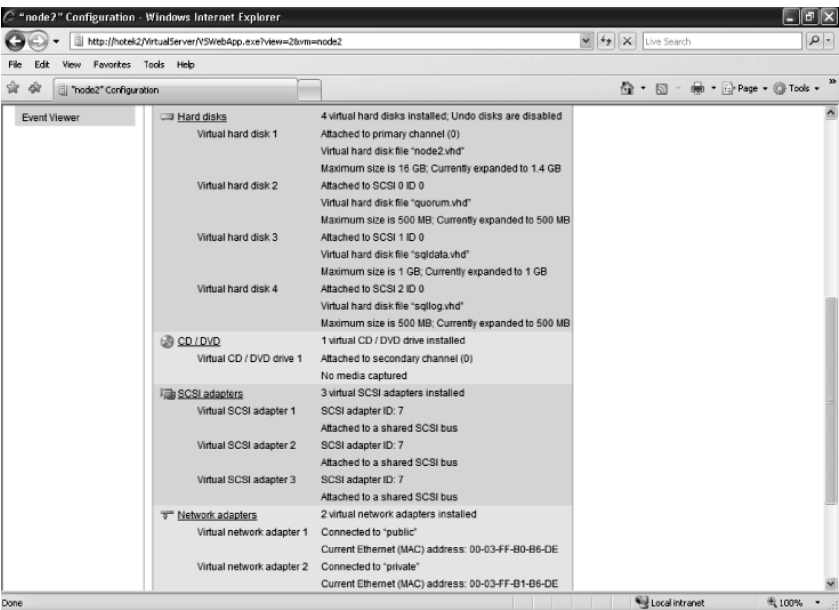


Рис. 14-4. Конфигурация узла Node2

- 17. Щелкните ссылку **Master Status** в разделе **Navigation**.
- 18. Перейдите в клиент *Virtual Machine Remote Control Client* и войдите в контроллер домена.
- 19. Откройте оснастку **Пользователи и компьютеры Active Directory** (Active Directory Users And Computers).
- 20. Создайте нового пользователя с именем Clusteradmin, не являющегося членом никакой группы (рис. 14-5).

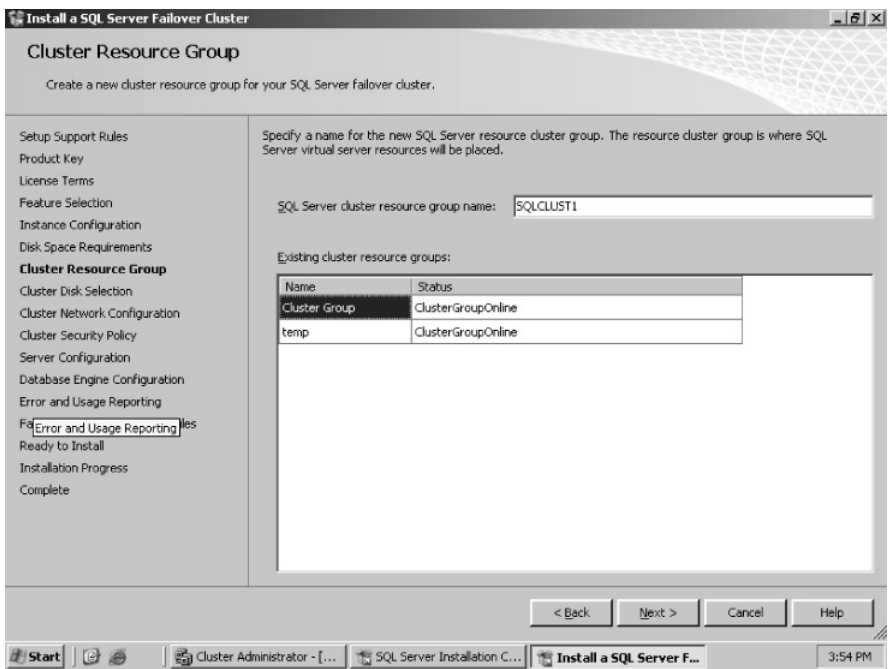


Рис. 14-5. Учетная запись администратора кластера

ВНИМАНИЕ! Начальная конфигурация

Необходимо очень внимательно отнестись к порядку запуска и остановки узлов Node1 и Node2 при выполнении этого упражнения. Одновременный запуск обоих узлов до завершения настройки кластера приведет к повреждению дисков и невозможности выполнения дальнейших шагов. Прежде чем запускать или останавливать любой из узлов внимательно проверьте их состояние.

- 21. Убедитесь, что узел Node2 отключен, затем запустите узел Node1.
- 22. Войдите в систему узла Node1 и откройте консоль **Управление дисками** (Disk Management), щелкнув правой кнопкой пункт **Мой компьютер** (My Computer) в меню **Пуск** (Start) и выбрав **Управление** (Manage). В дереве консоли **Управление компьютером** выберите **Управление дисками** (Disk Management).
- 23. Поскольку все три диска не сконфигурированы, откроется Мастер инициализации и преобразования дисков (Initialize And Convert Disk Wizard).

24. Щелкните **Далее** (Next), убедитесь, что выбраны все три диска, и еще раз щелкните **Далее**.
25. Убедитесь, что все три диска не выбраны (потому что динамические диски не совместимы с кластером), щелкните **Далее**, а затем **Готово** (Finish).

ВНИМАНИЕ! Базовые диски

При настройке дисков следуйте инструкциям в диалоговом окне. Обязательно убедитесь, что диски не динамические. Кластеры поддерживают только базовые диски. Если диски преобразованы в динамические, это сделает невозможным конфигурирование кластера и весь процесс придется начинать сначала, используя новые диски.

27. На каждом диске создайте охватывающий весь диск новый основной раздел NTFS с нераспределенным дисковым пространством.
28. Настройте буквы диска согласно табл. 14-9 и как показано на рис. 14-6.

Табл. 14-9. Дисковая конфигурация узла Node1

Диск	Буква диска
Disk 0	C
Disk 1	Q
Disk 2	M
Disk 3	N

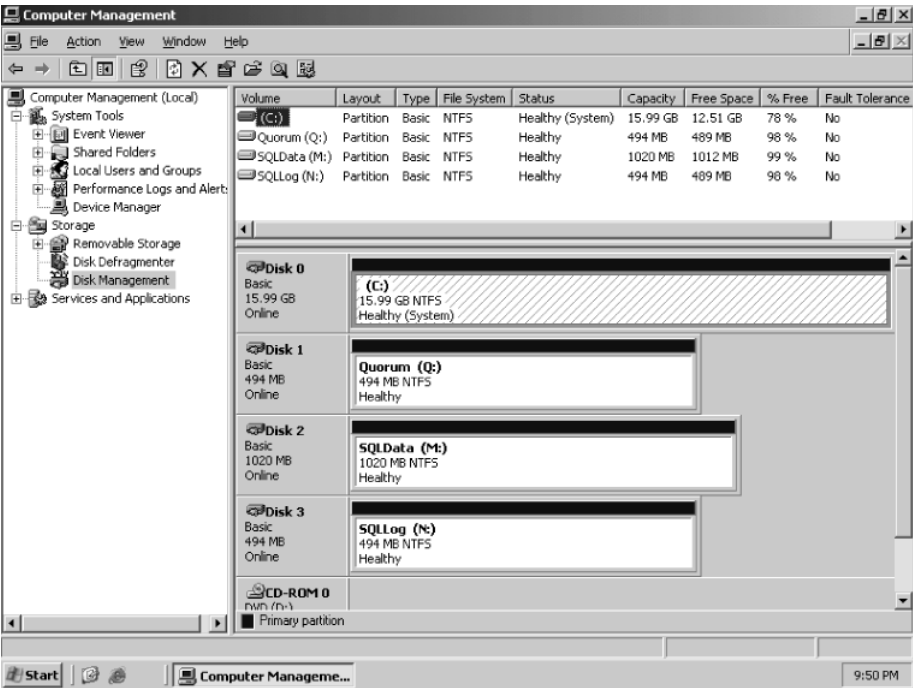


Рис. 14-6. Дисковая конфигурация узла Node1

29. В консоли **Управление компьютером** раскройте узел **Локальные пользователи и группы** (Local Users And Groups) и выберите **Группы** (Groups).
30. Дважды щелкните группу **Администраторы** (Administrators) и добавьте в группу учетную запись Clusteradmin, созданную в домене в п.20.
31. Закройте консоль **Управление компьютером**.
32. Откройте утилиту **Сетевые подключения** (Network Connections).
33. Переименуйте **Подключение по локальной сети** (Local Area Connection) в **Public**.
34. Переименуйте **Подключение по локальной сети 2** (Local Area Connection 2) в **Private**.
35. Щелкните правой кнопкой подключение **Private** и в контекстном меню выберите **Свойства** (Properties).
36. Сбросьте флажки **Клиент для сетей Microsoft** (Client For Microsoft Networks) и **Служба доступа к файлам и принтерам сетей Microsoft** (File And Printer Sharing For Microsoft Networks) (рис. 14-7).

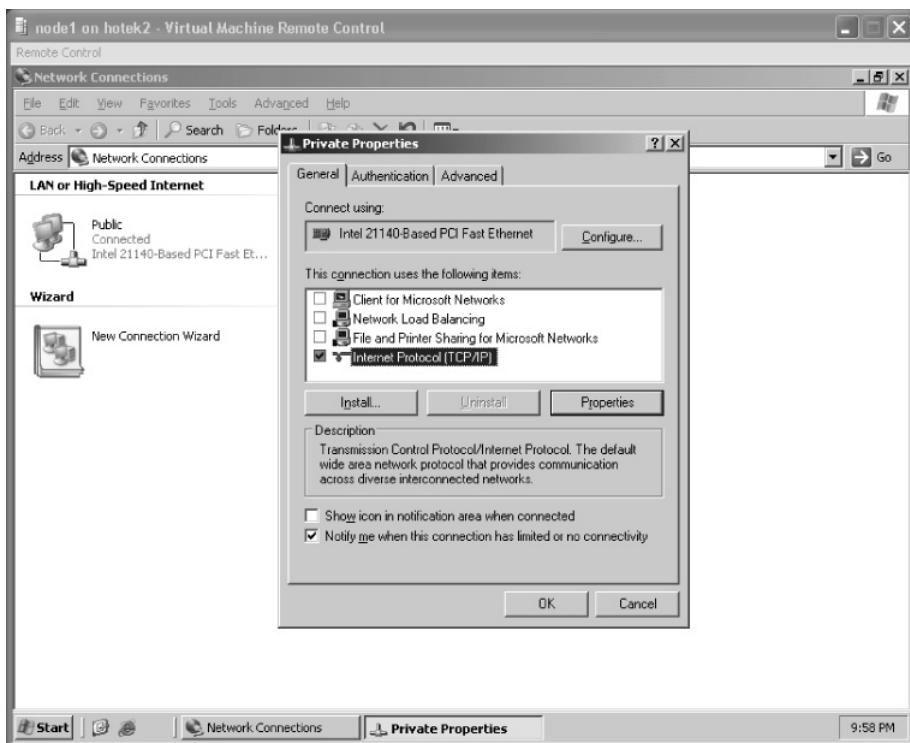


Рис. 14-7. Свойства сетевого адаптера для частной сети

37. Выберите **Протокол Интернета (TCP/IP)** [Internet Protocol (TCP/IP)] и щелкните **Свойства**.
38. Задайте IP-адрес 10.10.213.1 и маску подсети 255.255.255.0. Не указывайте адреса DNS-сервера и основного шлюза (рис. 14-8).

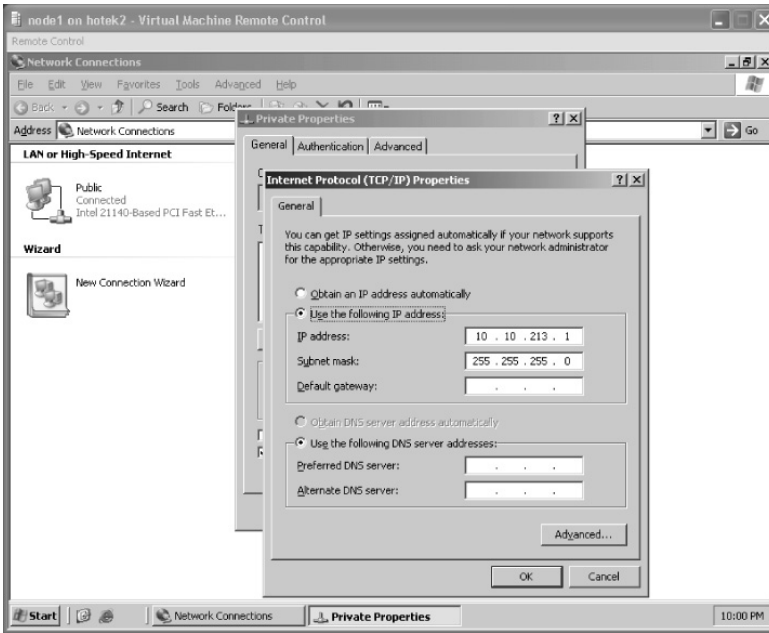


Рис. 14-8. Параметры IP и DNS для частной сети

39. Щелкните **Дополнительно** (Advanced).

40. На вкладке **DNS** сбросьте флажок **Зарегистрировать адреса этого подключения в DNS** (Register This Connection's Addresses In DNS) (рис. 14-9).

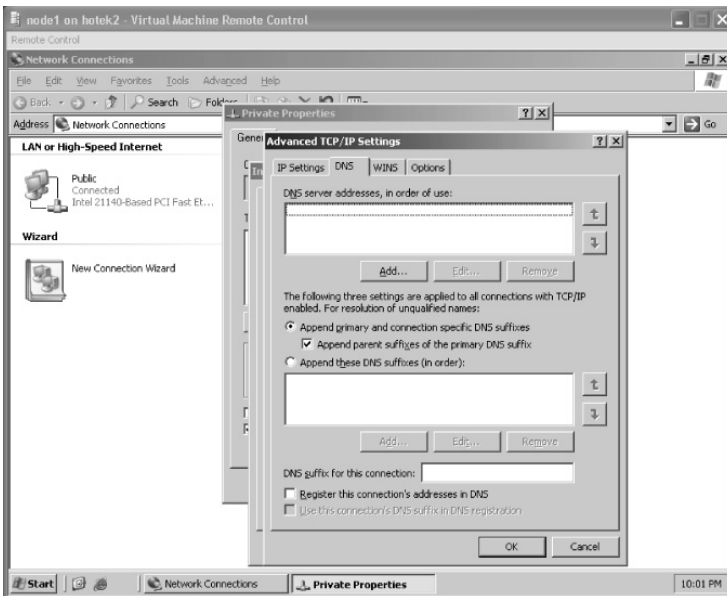


Рис. 14-9. Конфигурация DNS для частной сети

41. На вкладке **WINS** сбросьте флажок **Включить просмотр LMHOSTS** (Enable LMHOSTS Lookup), а затем выберите **Отключить NetBIOS через TCP/IP** (Disable NetBIOS Over TCP/IP) (рис. 14-10).

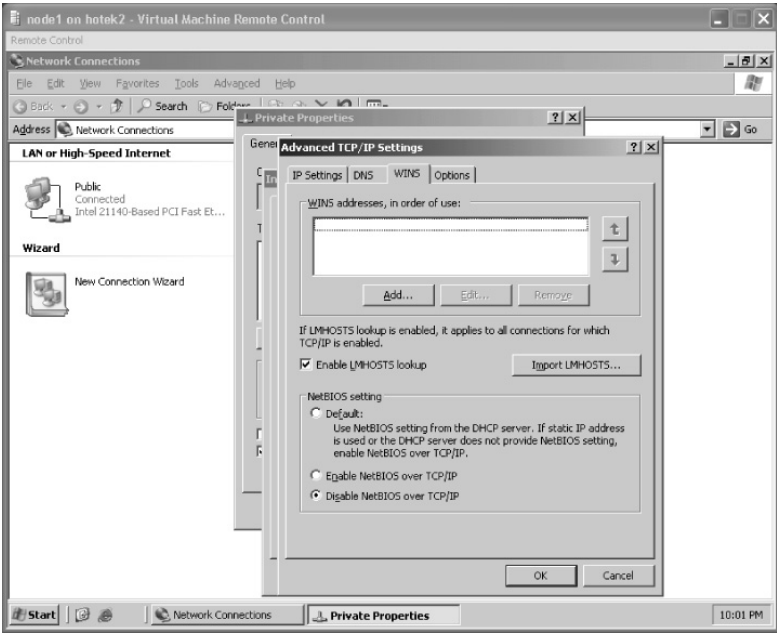


Рис. 14-10. Конфигурация WINS для частной сети

41. Дважды щелкните **ОК**, а затем щелкните **Закреть** (Close), чтобы закрыть диалоговое окно свойств подключения по частной сети.
42. Закройте диалоговое окно **Сетевые подключения** и отключите узел Node1.
43. Убедитесь, что узел Node1 выключен, и запустите узел Node2.
44. Повторите пп.21–42 для узла Node2. Конфигурации диска и сети для Node2 см. в табл. 14-10 и 14-11.

ПРИМЕЧАНИЕ Инициализация диска

При выборе команды «Управление диском» на узле Node2 мастер инициализации и преобразования дисков не откроется, так как у этих дисков уже есть подпись. Эти диски не нужно форматировать, поскольку вы это уже сделали в процессе конфигурирования узла Node1. Для этих дисков также не требуется указывать буквы диска, поскольку для узла Node2 они будут взяты из кластера после завершения его настройки.

Табл. 14-10. Конфигурация диска для узла Node2

Диск	Буква диска
Disk 0	C
Disk 1	Q
Disk 2	M
Disk 3	N

Табл. 14-11. Конфигурация сети для узла Node2

Параметр	Значение
Клиент для сетей Microsoft	Отключен
Служба доступа к файлам и принтерам для сетей Microsoft	Отключен
IP-адрес	10.10.213.2
Маска подсети	255.255.255.0
Основной шлюз	Пусто
DNS-сервер	Пусто
Зарегистрировать адреса этого подключения в DNS	Отключен
Включить просмотр LMHOSTS	Отключен
Отключить NETBIOS через TCP/IP	Установлен

45. Убедитесь, что узлы Node1 и Node2 отключены, после чего включите узел Node1.
46. Войдите в систему узла Node1 и запустите консоль **Администратор кластера** (Cluster Administrator).
47. В раскрывающемся списке **Действие** (Action) выберите **Создать новый кластер** (Create New Cluster) и щелкните **ОК**.
48. Откроется мастер создания кластера. Щелкните **Далее**, убедитесь, что в раскрывающемся списке **Домен** (Domain) правильно указано имя домена, и в поле **Имя кластера** (Cluster Name) введите Clust1. Щелкните **Далее**.
49. В качестве имени компьютера по умолчанию должен быть указан узел Node1. Щелкните **Далее**.
50. Мастер выполнит проверку совместимости узла Node1 для работы в кластере. После завершения проверки щелкните **Далее**.
51. Введите IP-адрес кластера в разделе **Public**. Следуя предложенным в начале этой главы параметрам IP-адреса, укажите 10.1.1.5. Щелкните **Далее**.
52. В качестве имени пользователя укажите clusteradmin, введите пароль для этой учетной записи, убедитесь в правильности имени домена и щелкните **Далее**.
53. На странице **Предполагаемая конфигурация кластера** (Proposed Cluster Configuration) щелкните **Кворум** (Quorum) и убедитесь, что в качестве кворума задано Disk Q. Если это не так, внесите изменения и щелкните **ОК**.

ПРИМЕЧАНИЕ **Определение кворума**

При настройке кластера на физическом оборудовании мастер создания кластера по умолчанию выберет в качестве кворума первый созданный в узле Node1 диск, который не является локальным. Virtual Server выбирает первую букву диска в алфавитном порядке. В диалоговом окне «Кворум настройки кластера» (Cluster Configuration Quorum) можно указать локальный кворум, используемый при создании одноузлового кластера в целях тестирования. В этом окне также можно изменить тип кластера со стандартного, создаваемого в этом упражнении, на

кластер с набором узлов большинства, выбрав **Набор узлов большинства** (Majority Node Set) в раскрывающемся списке — в этом случае мастер создания кластера создаст базу данных кворума на каждом узле кластера.

54. Убедитесь, что все параметры заданы правильно, и щелкните **Далее**.
 55. Следующая операция — создание кластера по заданным параметрам — займет несколько минут. По ее завершении щелкните **Далее**, а затем **Готово** (Finish).
 56. Поздравляем — вы создали кластер Windows!
 57. Убедитесь, что созданы три группы. В группу Cluster Group входят имя кластера Cluster Name, IP-адрес кластера Cluster IP Address и Disk Q. Группа Group 0 содержит Disk M, а группа Group 1 — Disk N.
 58. При работающем узле Node1 запустите также узел Node2.
 59. В консоли **Администратор кластера** в узле Node1 щелкните правой кнопкой **Clust1** и выберите **Создать** (New), а затем **Узел** (Node). Откроется окно Мастера добавления узлов (Add Nodes Wizard). Щелкните **Далее**.
 60. В качестве имени компьютера укажите **Node2**, щелкните **Создать** (Add), а затем — **Далее**.
 61. После завершения проверки щелкните **Далее**.
 62. Введите пароль для учетной записи Clusteradmin и щелкните **Далее**.
 63. Проверьте настройку кластера и щелкните **Далее**.
 64. Теперь узел Node2 настроен для работы в кластере и добавлен в кластер Clust1.
 65. Щелкните **Далее**, а затем **Готово**.
-

ПРИМЕЧАНИЕ Предупреждения в ходе анализа кластера

Из-за особенностей обращения Virtual Server с дисковыми ресурсами при конфигурировании кластера могут появляться предупреждения. Это нормальная ситуация, не влияющая на работоспособность кластера. Если ошибок (когда индикатор выполнения становится красным) нет, значит, настройка успешно завершена и кластер полностью функционален.

66. Убедитесь, что оба узла Node1 и Node2 настроены как часть кластера Clust1.
67. Выберите группу **Cluster Group**. В области сведений щелкните правой кнопкой **Имя кластера** (Cluster Name) и выберите **Перевести в автономный режим** (Take Offline). Щелкните правой кнопкой **IP-адрес** (IP Address) и выберите **Перевести в автономный режим** (Take Offline).
68. В меню **Файл** (File) выберите **Создать** (New), а затем выберите **Ресурс** (Resource).
69. В качестве имени укажите MSDTC, в поле **Тип ресурса** выберите **Координатор распределенных транзакций** (Distributed Transaction Coordinator) и убедитесь, что в качестве группы выбрана Cluster Group. Щелкните **Далее**.
70. Убедитесь, что оба узла Node1 и Node2 указаны в качестве возможных владельцев, и щелкните **Далее**.
71. Добавьте Cluster Name и Disk Q в раздел **Зависимости ресурса** (Resource Dependencies). Щелкните **Готово**. Щелчком **ОК** закройте диалоговое окно, подтверждающее создание ресурса кластера.

72. Щелкните правой кнопкой **Cluster Group** и выберите **Перевести в оперативный режим (Bring Online)** (рис. 14-11).

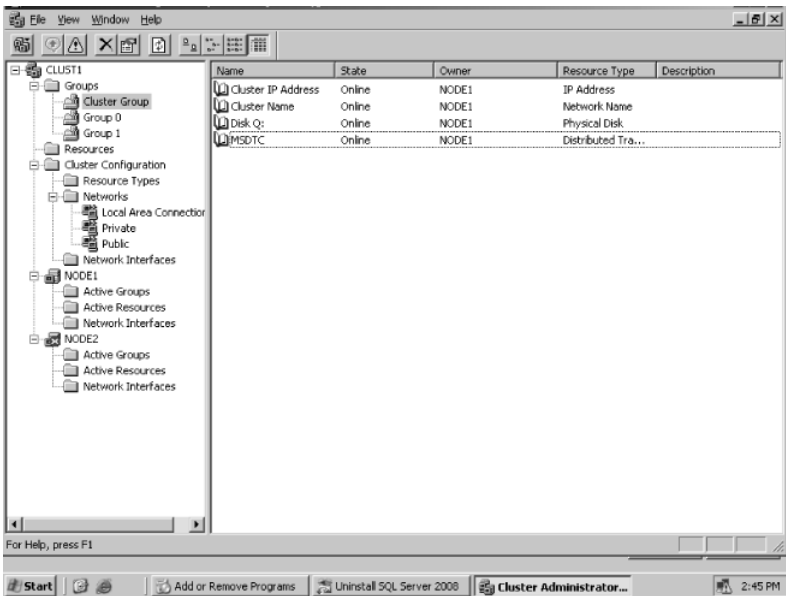


Рис. 14-11. Готовый кластер с двумя узлами

Практический совет. Координатор распределенных транзакций Microsoft (MS DTC)

MS DTC, который должен присутствовать в каждом создаваемом кластере Windows, гарантирует выполнение операций, требующих связывания ресурсов, например COM+. При этом всегда рекомендуют настраивать MS DTC на использование диска, отличного от диска кворума или любых других дисков, занятых под SQL Server и других приложений. Мы считаем, что такой подход приводит к трате и без того сильно ограниченных дисковых ресурсов. Если в кластере выполняются приложения, очень активно использующие MS DTC, для работы MS DTC нужно выделить отдельный диск. Если же в кластере не выполняются приложения, требующие COM+, средство MS DTC можно без опаски настроить в группе кластера и зависимым от диска кворума.

Резюме

- Стандартный кластер использует единую базу данных кворума на общем дисковом массиве. В кластере с набором узлов большинства копия базы данных кворума хранится на каждом узле кластера.
- Кластеры Windows поддерживают только базовые диски. Диски можно шифровать, но шифрование и расшифровка снижают производительность. Кластеры не поддерживают сжатие дисков.

- Для кластера необходимы две отдельные сети: общая сеть используется кластером для взаимодействия с приложениями и клиентами, а частная сеть — для взаимодействий внутри кластера.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Недавно в компании Coho Vineyards были отмечены проблемы в работе системы сбыта. Причиной опозданий в графике отправки и доставки грузов поставщикам стали аппаратные сбои. Руководство компании поручило техническому директору приобрести аппаратное решение, устойчивое к сбою серверов. Было решено закупить оборудование для создания кластера. Какую операционную систему следует на нем установить, чтобы удовлетворить эти бизнес-требования с наименьшими затратами?
 - A. Windows 2000 Server Standard Edition.
 - B. Windows 2000 Advanced Server.
 - C. Windows Server 2003 Standard Edition.
 - D. Windows Server 2003 Enterprise Edition.
2. Технический директор компании Coho Vineyards принял решение приобрести два сервера для создания кластера, на котором планируется разместить систему сбыта. Какое сочетание версии ОС и типа кластера обеспечит наибольшую отказоустойчивость с наименьшими затратами?
 - A. Windows Server 2003 Standard Edition и стандартный кластер.
 - B. Windows Server 2003 Standard Edition и кластер с набором узлов большинства.
 - C. Windows Server 2003 Enterprise Edition и стандартный кластер.
 - D. Windows Server 2003 Enterprise Edition и кластер с набором узлов большинства.
3. Какая служба должна работать, чтобы было возможно выполнять проверку работоспособности кластера?
 - A. Служба сервера (Server service).
 - B. Служба удаленного вызова процедур (RPC).
 - C. Служба сетевого входа в систему (Net Logon).
 - D. Служба служб терминалов (Terminal Services).

Занятие 2. Проектирование отказоустойчивых кластеризованных экземпляров SQL Server 2008

На готовом и настроенном кластере Windows можно устанавливать экземпляры SQL Server. Кластеризованные экземпляры обеспечивают отказоустойчивость SQL Server за счет того, что в них сбой оборудования не приводит к длительной недоступности приложений. В этом занятии рассказывается о выполнении

установки и настройки отказоустойчивых кластеризованных экземпляров SQL Server 2008 для достижения оптимальной избыточности в кластере.

Изучив материал этого занятия, вы сможете:

- ✓ устанавливать кластеризованные экземпляры SQL Server.

Продолжительность занятия — 45 минут.

Практический совет

Майк Хотек

Чуть более двух лет тому назад я занимался созданием кластера в среде заказчика. Вместо того, чтобы сразу приступить к развертыванию и настройке кластера, мне пришлось объяснить заказчику суть кластеризации. Кто-то из консультантов сказал сотрудникам этой компании, что кластер будет использоваться для устранения простоев при установке пакетов обновления и для балансировки нагрузки на аппаратные ресурсы. Им также сказали, что благодаря кластеризации транзакция, запущенная на одном узле, может быть завершена после того, как в кластере произойдет переключение на резервный узел.

Кластеры не пригодны для выполнения ни одной из этих задач. Отказоустойчивые кластеры SQL Server обеспечивают защиту от аппаратных сбоев. В случае сбоя одного узла другой узел автоматически берет управление на себя и запускает SQL Server.

Установка пакетов обновления будет приводить к простоям кластера, поскольку экземпляр SQL Server может одновременно выполняться только на одном узле. Любые транзакции, не завершенные на момент сбоя кластера, обязательно откатываются. Балансировка нагрузки тоже невозможна, так как SQL Server не поддерживает одновременный доступ нескольких процессов к файлам базы данных.

После того как я разъяснил заказчику, что кластер защищает только от аппаратных сбоев, я все же создал кластер. Заказчик получил возможность эффективно управлять базой данных кластера, понимая, что сбои будут по-прежнему приводить к простоям, но время недоступности в результате аппаратных сбоев будет сведено к минимуму.

Терминология

Для обозначения экземпляров SQL Server, установленных в кластере, используют разные термины, многие из которых не точны. Поэтому перед тем, как перейти к объяснению настройки SQL Server в кластере, необходимо разобраться с терминологией.

Кластеры SQL Server могут состоять из одного или нескольких экземпляров. Кластер с одним экземпляром — это кластер Windows, в котором установлен ровно один экземпляр SQL Server, а кластер несколькими экземплярами содержит более одного экземпляра SQL Server. Не важно, на каком узле выполняются экземпляры, терминология это не отражает.

Кластеры в режимах «активный-активный» и «активный-пассивный» существуют на уровне Windows. Режим «активный-активный» означает, что приложения выполняются на всех узлах кластера. В режиме «активный-пассивный» приложения работают только на одном узле. С точки зрения SQL Server это не имеет значения, поскольку экземпляр SQL Server может быть либо работающим, либо остановленным. Отдельные экземпляры SQL Server не имеют информации о других экземплярах. В SQL Server невозможна балансировка нагрузки. Таким образом, SQL Server выполняется на одном из узлов, определяемом администратором базы данных, управляющего этим кластером.

Экземпляры SQL Server, устанавливаемые в кластере, обычно называют виртуальными серверами (virtual server). Этот термин создает серьезную путаницу, так как у Microsoft есть отдельный продукт под названием Virtual Server. Экземпляры SQL Server в кластере мы будем называть кластеризованными экземплярами SQL Server или отказоустойчивыми кластеризованными экземплярами SQL Server.

Компоненты отказоустойчивого кластеризованного экземпляра

При установке изолированного экземпляра SQL Server администраторам баз данных не обязательно настраивать IP-адреса, сетевые имена и даже учитывать наличие дисковых накопителей. Однако об этих компонентах придется подумать при установке экземпляра SQL Server в кластере.

В отказоустойчивом кластеризованном экземпляре SQL Server нужно настроить следующие компоненты:

- IP-адреса;
- сетевые имена;
- дисковые накопители в общем дисковом массиве;
- службы SQL Server;
- учетные записи служб.

Сетевая конфигурация

Каждому экземпляру SQL Server, установленному в кластере, должен назначаться уникальный IP-адрес сегмента общей сети, настроенного в кластере. Каждому IP-адресу должно сопоставляться уникальное сетевое имя, зарегистрированное в DNS, что позволяет разрешать SQL Server по имени.

Практический совет. Служба обозревателя SQL

Вместе с SQL Server 2008 устанавливается служба «Обозреватель SQL» (SQL Browser). Если в кластере установлены именованные экземпляры, обязательно наличие работоспособной службы обозревателя SQL, которая будет разрешать эти имена. Если в кластере нет именованных экземпляров, обозреватель SQL следует отключить.

Дисковая конфигурация

В каждом кластеризованном экземпляре SQL Server следует настроить выделенный набор букв диска. На изолированном сервере несколько экземпляров SQL Server могут использовать для хранения баз данных один диск и даже каталог. В кластере диски в каждый момент времени подключены только к одному узлу; у остальных узлов нет доступа к этим дискам. Экземпляр SQL Server может работать на любом узле. Если же базы данных нескольких кластеризованных экземпляров SQL Server хранятся на одном диске, возможна конфигурация, в которой экземпляр работает на одном узле, а владельцем дисков является другой узел, что приводит к неработоспособности экземпляра SQL Server.

Понятие дисковой конфигурации в кластере SQL Server заключается в соотношении экземпляров и дисков. Кластеризованный экземпляр SQL Server может ассоциироваться с несколькими буквами дисков, но буква диска может привязываться только к одному кластеризованному экземпляру SQL Server. Кроме того, буквы диска следует настроить как зависимости для службы SQL Server и разрешить хранение на них баз данных.

Настройка безопасности

Для каждой службы SQL Server нужно настроить учетную запись службы. Как правило, это различные учетные записи для разных служб SQL Server, например для службы SQL Server, агента SQL Server и компонента Full Text Search.

Этим учетным записям не требуются специальные привилегии, но это должны быть учетные записи домена, поскольку идентификатор безопасности (SID) локальной учетной записи нельзя разрешить на другой машине.

Для SQL Server 2008 не нужны учетные записи служб с правами администратора Windows. Учетной записи Windows могут быть назначены десятки индивидуальных разрешений, например на доступ к реестру, каталогам и файлам. Изменение учетной записи службы раньше было очень сложной задачей, поскольку для нормальной работы службы ее новой учетной записи требовалось назначить все эти индивидуальные разрешения.

Благодаря изменениям в инфраструктуре безопасности учетные записи Windows для служб SQL Server 2008 разработаны с учетом отраслевых рекомендаций по управлению учетными записями Windows. Разрешение на доступ к различным ресурсам предоставляются группам Windows. Затем учетные записи Windows добавляются в соответствующие группы, что дает им доступ к нужным ресурсам.

На изолированном сервере эти группы создаются по умолчанию в виде записей `SQLServerMSSQLUser$<имя_компьютера>$<имя_экземпляра>` и `SQLServerSQLAgentUser$<имя_компьютера>$<имя_экземпляра>`. Программа установки SQL Server автоматически назначает группам разрешения на каталоги, разделы реестра и другие ресурсы, необходимые для нормальной работы SQL Server. Затем учетные записи служб добавляются в соответствующие группы.

Этот процесс выполняется на изолированном сервере, но в кластере все не так просто. Отказоустойчивый кластеризованный экземпляр SQL Server может выполняться на любом физическом сервере в кластере. Локальные группы Windows не поддерживают единый контекст безопасности на нескольких ма-

шинах, поэтому группы для учетных записей служб SQL Server необходимо создавать на уровне домена.

В программу установки не передаются сведения о создании групп в домене. Поэтому следующие три группы домена необходимо создавать до установки отказоустойчивого кластеризованного экземпляра SQL Server:

- учетные записи службы SQL Server;
- учетные записи службы агента SQL Server;
- учетные записи компонента Full Text Search в SQL Server.

Созданные группы указываются на заключительных этапах работы программы установки.

Практический совет. Равновесие между безопасностью и управляемостью

С точки зрения безопасности рекомендуется создавать доменную группу для каждого типа службы и для каждого устанавливаемого кластеризованного экземпляра SQL Server. Для удобства управления имеет смысл создать по доменной группе для каждой из трех служб, а для всех отказоустойчивых кластеризованных экземпляров SQL Server указать единый набор доменных групп. Определитесь, где провести грань между очень сложной моделью групп домена с высокой степенью защиты и менее сложной, но в то же время менее безопасной моделью.

Проверка работоспособности

В кластере выполняется два типа проверки работоспособности отказоустойчивых кластеризованных экземпляров SQL Server. В ходе теста LooksAlive узлы направляют эхо-запросы (ping) по IP-адресу экземпляра SQL Server. Однако тест проверки связи не показывает, доступен ли экземпляр: экземпляр может отвечать на запросы, но при этом оставаться недоступным.

Для выявления проблем с доступностью SQL Server используют другой тип проверки — тест IsAlive, в котором создается соединение с экземпляром SQL Server и выполняется команда *SELECT @@SERVERNAME*. Для прохождения этой проверки работоспособности SQL Server должен вернуть правильный результирующий набор.

Переход на резервный ресурс кластера при сбое

Если хотя бы одна из проверок работоспособности терпит сбой, в кластере инициируется переключение экземпляра SQL Server на другой ресурс.

Первый шаг в процессе перехода на другой ресурс — перезапуск SQL Server на том же узле. Предполагается, что сбой проверки работоспособности возможен из-за временных неполадок.

Если после перезапуска экземпляр не дает немедленного отклика, группа SQL Server переключается на другой узел кластера (вторичный узел). Отменяется регистрация в DNS сетевого имени сервера, на котором выполняется SQL Server и выполняется привязка IP-адреса экземпляра SQL Server к сетевой интерфейсной карте на вторичном узле; связанные с экземпляром диски также подключа-

ются ко вторичному узлу. После привязки IP-адреса к сетевой интерфейсной карте вторичного узла сетевое имя экземпляра SQL Server регистрируется в DNS. Сетевое имя и диски готовы к работе, затем запускается служба SQL Server, а вслед за ней запускаются агент SQL Server и компонент Full Text Search.

Экземпляр SQL Server останавливается и перезапускается, причем эта операция выполняется независимо от того, на каком узле перезапускается экземпляр — на том же или вторичном. При перезапуске SQL Server происходит откат всех транзакций, не завершенных на момент переключения на другой ресурс кластера. После перезапуска следует обычный процесс восстановления.

В среднем переход на другой ресурс кластера занимает 10–15 секунд. На это время может повлиять регистрация в DNS; оно также может возрасти, если экземпляр обслуживает большое число баз данных. В SQL Server 2000 время перехода на другой ресурс зависело от скорости завершения этапов отката и повтора транзакций, что в свою очередь зависело от приложений, выполняющих транзакции с базами данных. Теперь базы данных становятся доступными сразу после завершения этапа повтора, поэтому кластеризованный экземпляр SQL Server 2008 гораздо быстрее переключается на другой ресурс, а базы данных быстрее становятся доступными.

Контрольные вопросы

1. Какие типы учетных записей и групп Windows можно использовать в кластеризованном экземпляре SQL Server?
2. Сколько кластеризованных экземпляров могут привязываться к одной букве диска?
3. Какие два типа проверки работоспособности выполняются в кластере? Какие операции в них выполняются?

Ответы на контрольные вопросы

1. В отказоустойчивых кластеризованных экземплярах SQL Server можно использовать доменные группы и пользователи. Идентификаторы безопасности для учетных записей и групп должны разрешаться на всех узлах кластера. Идентификаторы безопасности локальных учетных записей или групп не годятся, так как их контекст безопасности ограничивается только локальной машиной.
2. Хотя кластеризованный экземпляр может работать с несколькими буквами диска, каждая буква диска может сопоставляться только одному экземпляру. Это предотвращает ситуацию, когда SQL Server выполняется на одном узле, а владельцем необходимых ему дисков является другой узел.
3. Тест LooksAlive по умолчанию выполняется каждые 5 секунд и заключается в отправке узлами эхо-запроса по IP-адресу кластеризованного экземпляра SQL Server. Тест IsAlive по умолчанию выполняется раз в минуту, при этом создается подключение к кластеризованному экземпляру SQL Server, выполняется команда `SELECT @@SERVER-NAME` и проверяется корректность ответа.

Практикум. Установка отказоустойчивого кластеризованного экземпляра SQL Server

Вы установите отказоустойчивый кластеризованный экземпляр SQL Server в кластере Windows, созданном в практикуме занятия 1.

ПРИМЕЧАНИЕ Системные требования

Перед выполнением этого упражнения необходимо установить на обоих узлах кластера .NET Framework 2.0 с пакетом обновления 1 (SP1). Для кластера Windows Server 2003 также следует загрузить и установить исправление KB937444.

1. Откройте консоль **Администратор кластера** (Cluster Administrator) и подключитесь к **Clust1**.
2. Щелкните правой кнопкой группу **Group 0** и переименуйте ее в **Temp**.

ВНИМАНИЕ! Определение группы кластера

В отличие от трех предыдущих версий SQL Server в программе установки SQL Server 2008 в качестве места установки служб нельзя указать существующую группу кластера, даже если в ней нет экземпляра SQL Server. Поэтому убедитесь в том, что у вас нет ранее созданной группы с таким же именем, как нужно назвать группу, в которую будут установлены службы.

3. Выберите группу **Group 1**. Перетащите диск из группы **Group 0** в группу **Temp**. Дважды щелкните **ОК**, чтобы подтвердить перемещение.
4. Убедитесь, что в группе **Temp** есть оба диска: **Disk M** и **Disk N**.
5. Щелкните правой кнопкой **Group 1** и выберите **Удалить** (Delete). Щелчком **Да** (Yes) подтвердит удаление. Проверьте, чтобы в группе Temp содержались диски Disk M и Disk N (рис. 14-12).

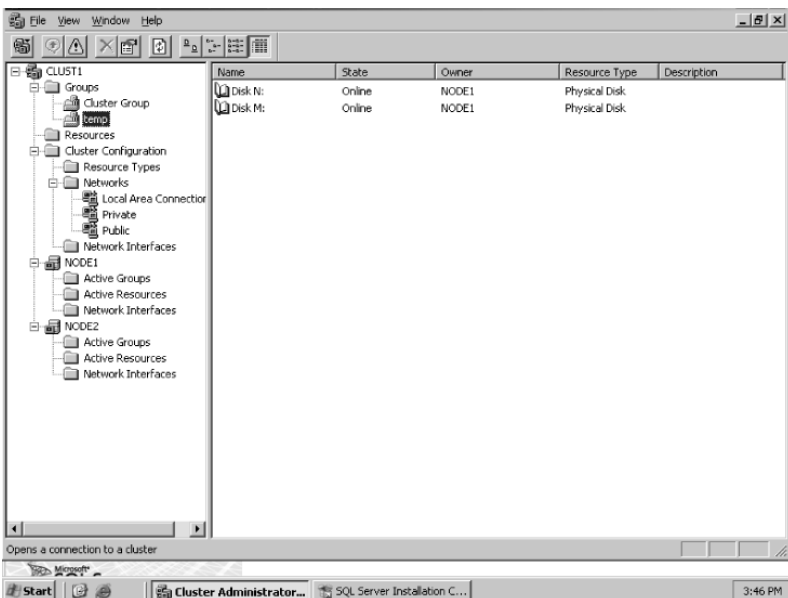


Рис. 14-12. Начальная настройка группы кластера

6. Переключитесь на контроллер домена.
7. Откройте оснастку **Пользователи и компьютеры Active Directory** (Active Directory Users And Computers).
8. Создайте три глобальные группы безопасности: **SQLServerService**, **SQL-ServerAgentService** и **SQLServerFullTextService**.
9. Создайте учетную запись пользователя с именем **SQLAdmin** (рис. 14-13). Также создайте учетную запись с именем **SQLServerFullText**.

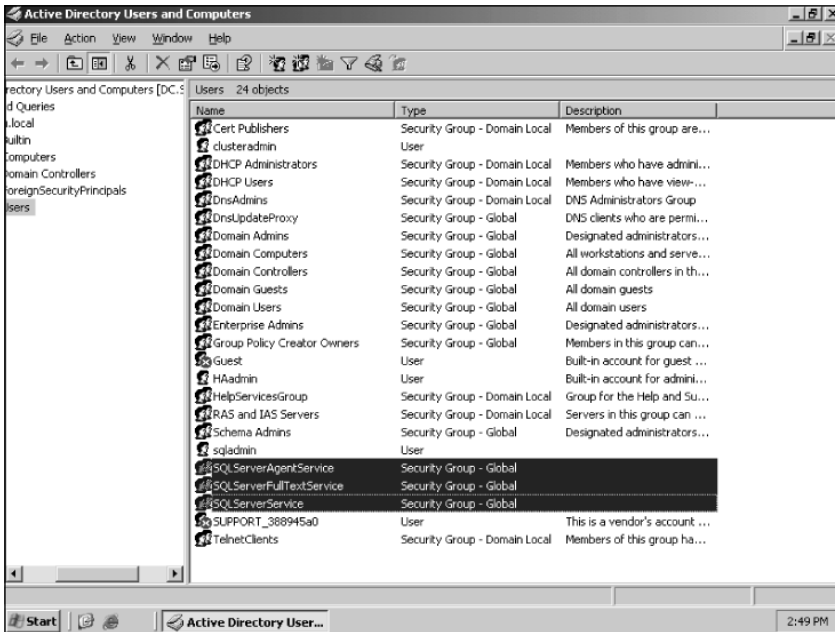


Рис. 14-13. Группы и пользователи домена

10. Добавьте учетную запись SQLAdmin в группы SQLServerService и SQL-ServerAgentService. Добавьте учетную запись SQLServerFullText в группу SQLServerFullTextService.
11. Переключитесь на узел Node1 и запустите установку SQL Server.
12. Примите условия лицензионного соглашения и щелкните **Далее**.
13. Щелкните **Установить** (Install), чтобы установить необходимое ПО. После завершения установки щелкните **Далее**.
14. В центре установки SQL Server щелкните ссылку **Установка** (Installation), а затем — ссылку **Новая установка отказоустойчивого кластера SQL Server** (New SQL Server Failover Cluster).
15. После выполнения правил установки щелкните **ОК**.
16. На странице **Файлы поддержки программы установки** (Setup Support Files) щелкните **Установить** (Install), чтобы установить файлы поддержки.
17. После завершения проверки конфигурации системы щелкните **Далее**.
18. На странице **Ключ продукта** (Product Key) укажите ключ продукта или выберите **Specify A Free Edition** (Specify A Free Edition.). Щелкните **Далее**.

- 19. На странице **Условия лицензионного соглашения** (License Terms) выберите **Я принимаю условия лицензионного соглашения** (I Accept The License Terms). Щелкните **Далее**.
- 20. Выберите **Службы SQL Server Database Services** (SQL Server Database Services) и **SQL Server Replication** (SQL Server Replication). Щелкните **Далее**.
- 21. На странице **Конфигурация экземпляра** (Instance Configuration) в полях **Сетевое имя отказоустойчивого кластера SQL Server** (SQL Server Failover Cluster Network Name), **Имя экземпляра** (Instance Name) и **Идентификатор экземпляра** (Instance ID) введите SQLClust1 (рис. 14-14). Щелкните **Далее**.

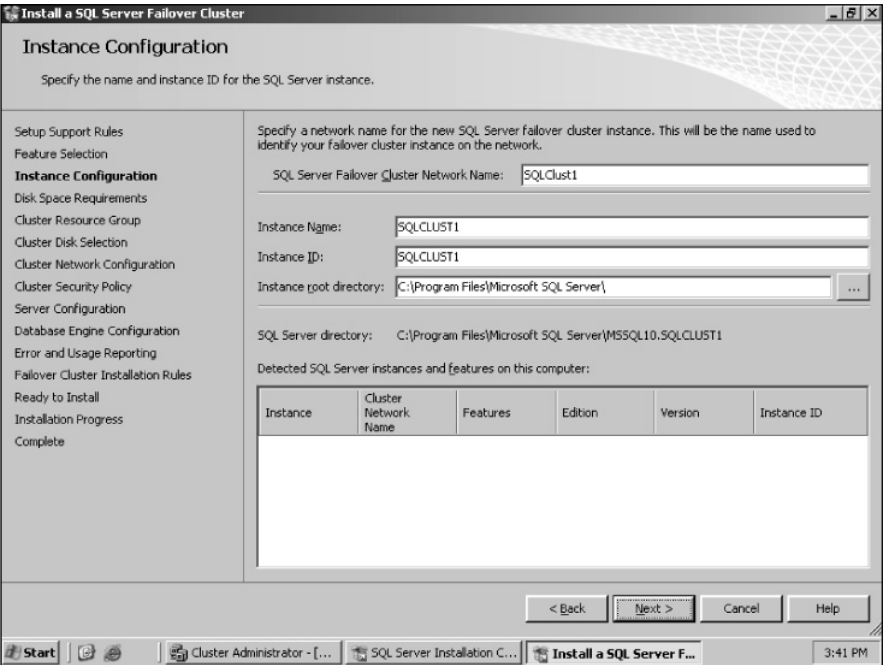


Рис. 14-14. Указание имени кластера

- 22. Убедитесь, что выполнены требования к наличию свободного места на диске. Щелкните **Далее**.
- 23. В качестве имени группы ресурсов кластера SQL Server укажите SQLClust1 и щелкните **Далее** (рис. 14-15).

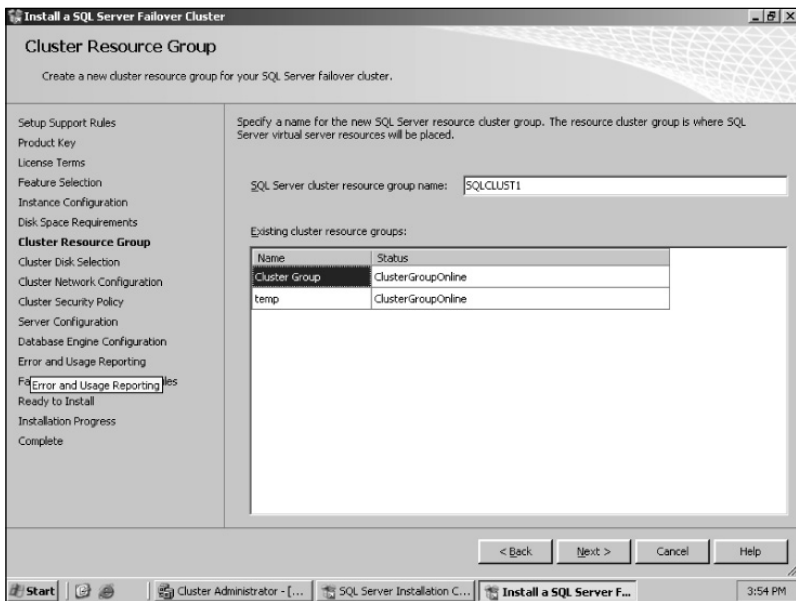


Рис. 14-15. Указание группы ресурсов кластера

24. На странице **Выбор диска кластера** (Cluster Disk Selection) выберите диски **Disk M** и **Disk N** (рис. 14-16). Щелкните **Далее**.

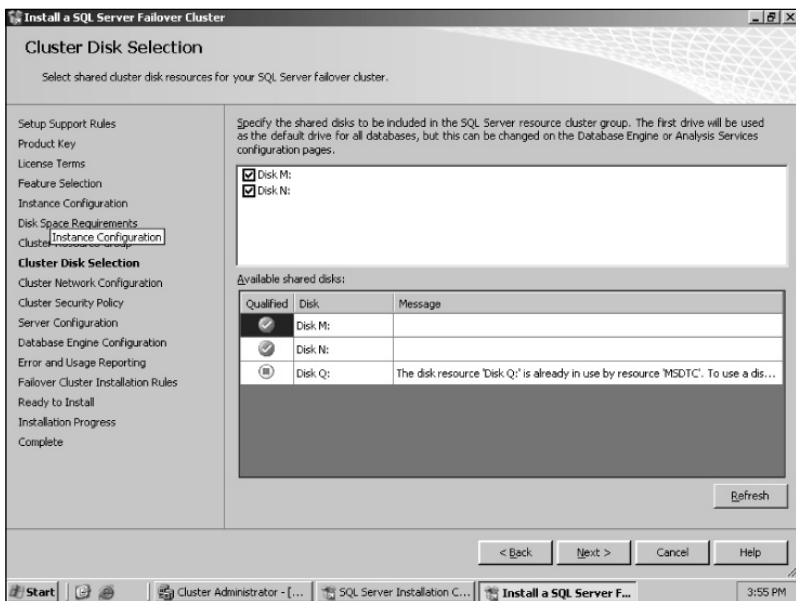


Рис. 14-16. Выбор диска кластера

25. Установите флажок **IPv4**, в качестве IP-адреса укажите 10.1.1.6. Щелкните **Далее** (рис. 14-17).

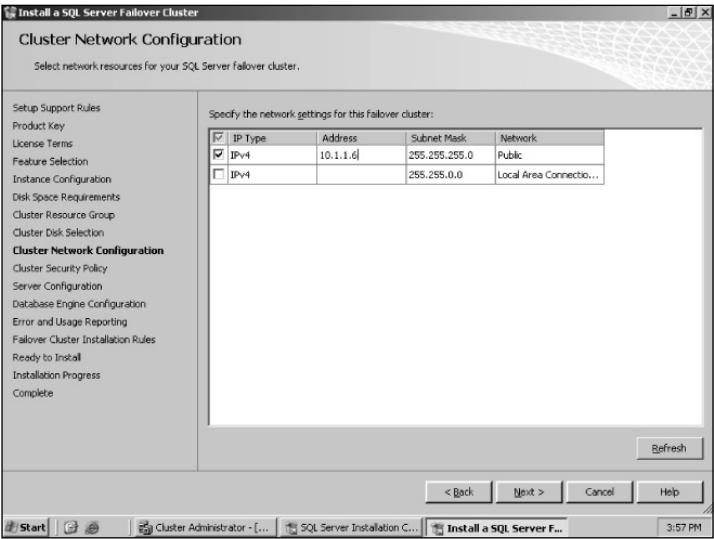


Рис. 14-17. Выбор сети кластера

26. На странице **Политика безопасности кластера** (Cluster Security Policy) в качестве доменной группы компонента Database Engine укажите **SQLServerService**, в поле **Группа домена агента SQL Server** (SQL Server Agent domain group) введите **SQLServerAgentService** и щелкните **Далее** (рис. 14-18).

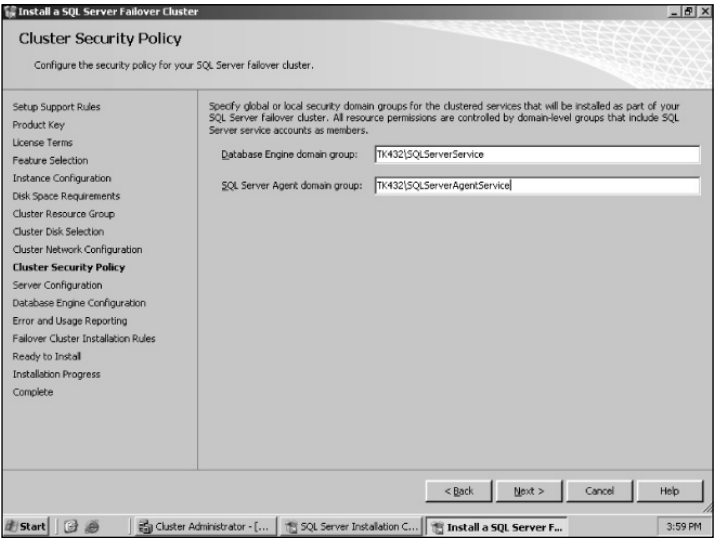


Рис. 14-18. Политика безопасности кластера

27. Укажите учетные записи службы для агента SQL Server, компонента SQL Server Database Engine и компонента Full Text Search, а также их пароли (рис. 14-19), и щелкните **Далее**.

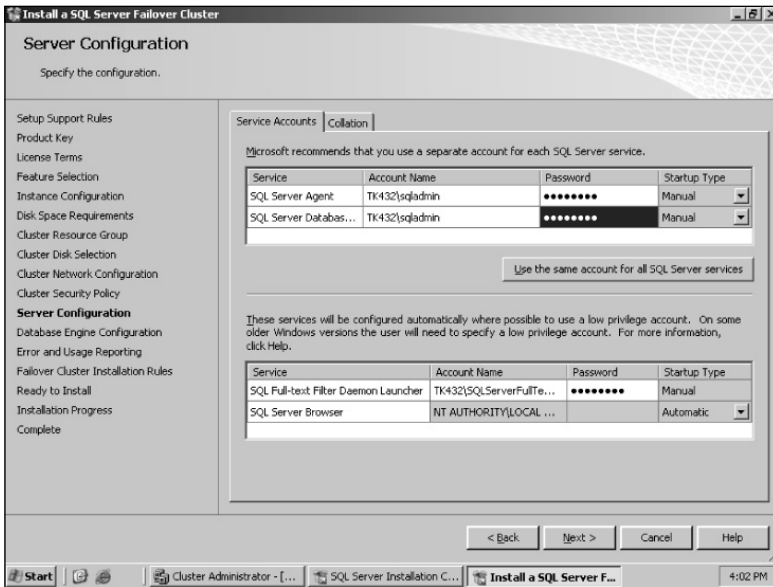


Рис. 14-19. Определение учетных записей служб

ВНИМАНИЕ! Запуск служб

Для служб кластера SQL Server следует задать тип запуска вручную. В кластере Windows должна быть возможность управления службами. Выбор автоматического типа запуска приведет к ошибкам в работе кластера.

28. Выберите **Смешанный режим (проверка подлинности SQL Server и Windows)** [Mixed Mode (SQL Server Authentication And Windows Authentication)] и щелкните **Добавить текущего пользователя (Add Current User)** (рис. 14-20).

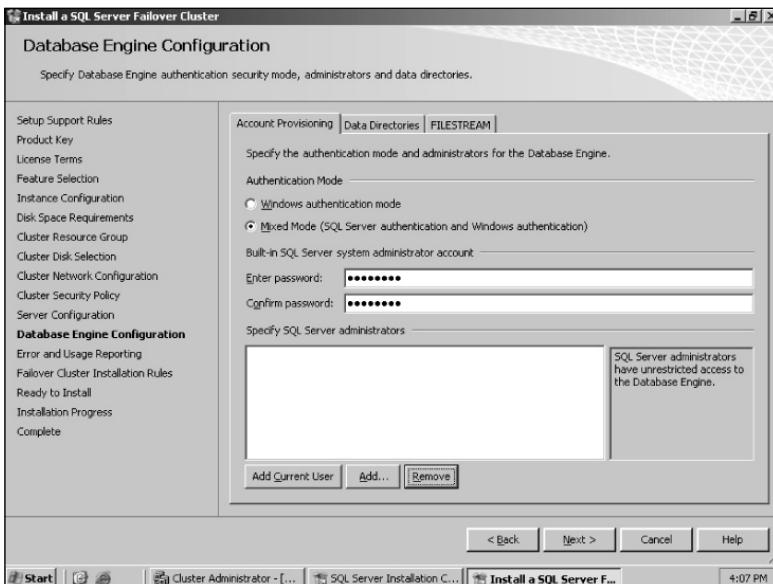


Рис. 14-20. Определение режима проверки подлинности

29. Перейдите на вкладку **Каталог данных** (Data Directories) и в качестве пути к файлам журнала укажите диск N (рис. 14-21). Щелкните **Далее**.

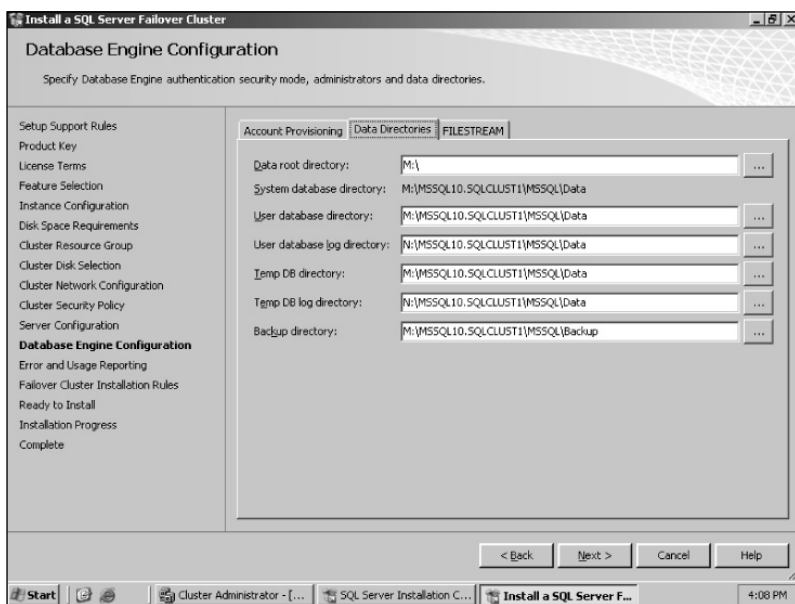


Рис. 14-21. Определение каталога данных

30. Установите соответствующие флажки на странице **Отчеты об ошибках и использовании** (Error And Usage Reporting), чтобы отправлять отчеты об ошибках и сведения об использовании в Microsoft. Щелкните **Далее**.
31. Проверьте данные на странице **Правила установки кластера** (Cluster Installation Rules). Щелкните **Показать подробности** (Show Details), чтобы просмотреть правила установки и состояние конфигурации. Щелкните **Далее**.
32. Проверьте конфигурацию на странице **Все готово для установки** (Ready To Install) и щелкните **Установить** (Install).
33. По завершении установки щелкните **Заккрыть** (Close).
34. Просмотрите список ресурсов, настроенных в группе SQLClust1, в консоли **Администратор кластера** (рис. 14-22).
35. С одной машины нельзя установить все узлы в кластере, поэтому переключитесь на узел Node2, сопоставьте каталог на узле Node1, содержащий файлы установки SQL Server, и запустите установку SQL Server с узла Node2.
36. Установите необходимый выпуск .NET Framework, все недостающие исправления и, если потребуется, перезагрузите компьютер.
37. Перейдя в Центр установки SQL Server (SQL Server Installation Center), щелкните ссылку **Установка** (Installation), а затем **Добавление узла в отказоустойчивый кластер SQL Server** (Add Node To A SQL Server Failover Cluster).
38. По завершении проверки правил поддержки установки щелкните **ОК**.
39. Установите файлы поддержки программы установки.

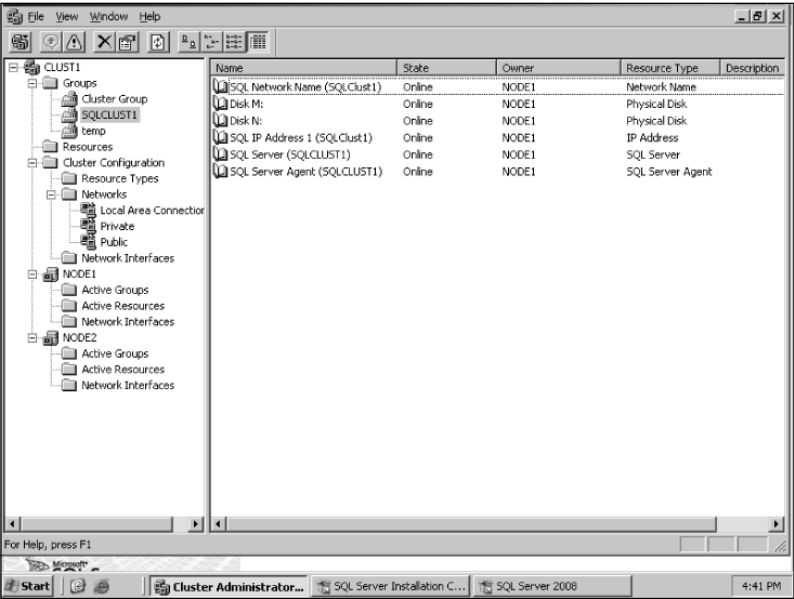


Рис. 14-22. Настроенный кластер с одним узлом

40. После завершения второй проверки правил поддержки установки щелкните **Далее**.
41. На странице **Конфигурация узла кластера** (Cluster Node Configuration) выберите **SQLCLUST1** в качестве имени экземпляра (рис. 14-23) и щелкните **Далее**.

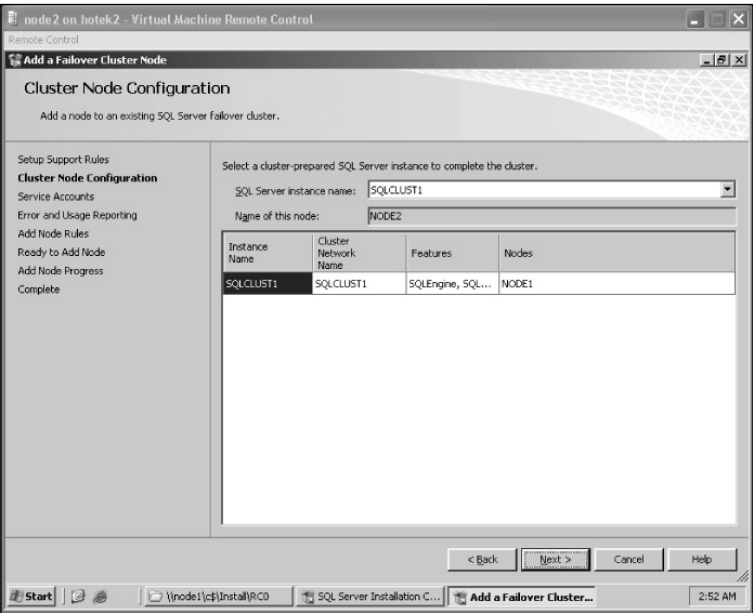


Рис. 14-23. Выбор кластеризованного экземпляра для настройки

42. Укажите пароли для учетных записей служб. Щелкните **Далее**.
43. При необходимости установите флажки на странице **Отчеты об ошибках и использовании** (Error And Usage Reporting) и щелкните **Далее**.
44. Убедитесь, чтобы сервер соответствует всем правилам узла, и щелкните **Далее**.
45. Щелчком **Установить** (Install) начните установку (рис. 14-24).

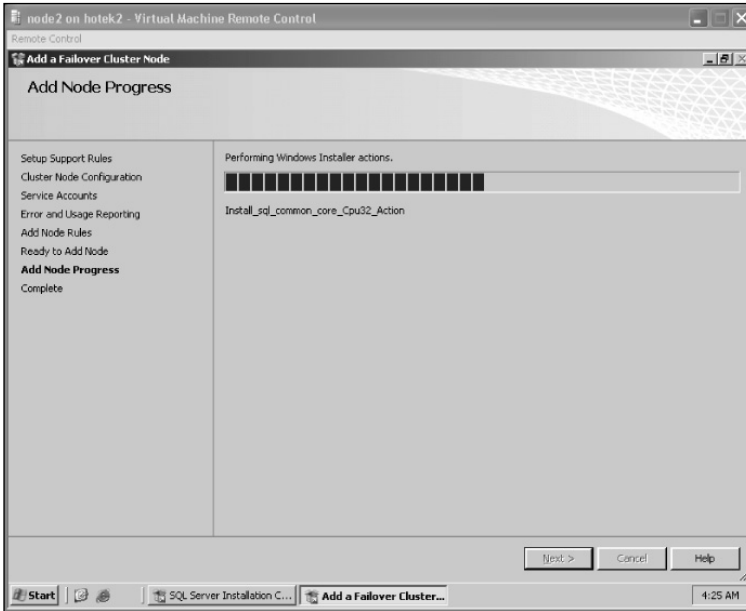


Рис. 14-24. Установка на втором узле

Резюме

- При размещении на кластере можно установить один или несколько экземпляров SQL Server.
- Тесты для проверки работоспособности LooksAlive и IsAlive позволяют обнаруживать сбои и выполнять автоматический переход на другой ресурс кластера.
- Экземпляр может использовать несколько дисков, но отдельный диск может быть связан только с одним кластеризованным экземпляром SQL Server.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Из-за аппаратных сбоев в компании Consolidated Messenger были отмечены перерывы в работе систем. В бизнес-системах компании используются базы данных SQL Server. Необходимо реализовать решение, которое поможет свести к минимуму время простоя. Руководство компании также требует,

чтобы системы восстанавливались после сбоя без вмешательства ИТ-специалистов. Какая технология позволит выполнить эти требования?

- A. Доставка журналов.
 - B. Репликация.
 - C. Отказоустойчивый кластер.
 - D. Моментальные снимки базы данных.
2. В настоящее время в компании Trey Research установлено четыре экземпляра SQL Server со множеством баз данных, используемых при выполнении медицинских исследований. Экземпляр Instance1 требует 200 Гб места на диске для баз данных и одновременно обслуживает более 500 пользователей. Экземпляр Instance2 требует около 1 Тб дискового пространства для небольшой группы из 25 специалистов, исследующих генотип человека. На экземплярах Instance3 и Instance4 хранятся базы данных меньшего размера, которые служат для управления всей деятельностью компании (в частности, база данных *HumanResources* для отдела кадров, *Payroll* для бухгалтерии и база данных *Contacts*, содержащая контактную информацию). База данных *Genetrak* на экземпляре Instance1 в среднем потребляет более 60% процессорной мощности. Экземпляр Instance2 в среднем использует 45% мощности процессора. Какую версию и выпуск Windows следует выбрать для создания кластера SQL Server за минимальные деньги?
- A. Windows 2000 Advanced Server.
 - B. Windows 2000 Datacenter Edition.
 - C. Windows Server 2003 Standard Edition.
 - D. Windows Server 2003 Enterprise Edition.

Закрепление материала главы

Для закрепления знаний и навыков, полученных при изучении этой главы, выполните следующее:

- изучите резюме главы;
- повторите основные термины, использованные в этой главе;
- выполните задания по темам этой главы;
- сделайте предложенные упражнения;
- пройдите пробный экзамен.

Резюме главы

- Кластеры SQL Server создаются на основе кластеров Windows и обеспечивают автоматическое обнаружение сбоев и переход на резервный ресурс.
- Существуют стандартные кластеры с общей базой данных кворума и кластеры с набором узлов большинства, в которых копия базы данных кворума хранится на каждом узле.
- Тесты проверки работоспособности LooksAlive и IsAlive предназначены для обнаружения аппаратных сбоев и недоступности SQL Server.

- Отказоустойчивый кластер SQL Server обеспечивает защиту только от аппаратных сбоев.

Основные термины

Понятны ли вам значения приведенных здесь основных терминов? Чтобы проверить свои ответы, загляните в глоссарий на прилагаемом компакт-диске.

- Группа кластера ~ cluster group.
- Имя кластера ~ cluster name.
- Узел кластера ~ cluster node.
- Ресурс кластера ~ cluster resource.
- Кластер с набором узлов большинства ~ Majority node set cluster.
- База данных кворума ~ quorum database.
- Стандартный кластер ~ standard cluster.

Упражнения

Сейчас вы примените на практике знания и навыки, полученные при изучении этой главы. Ответы на вопросы см. в разделе «Ответы» в конце книги.

Упражнение. Планирование высокой доступности

Вы примените на практике знания об отказоустойчивых кластерах, полученные в этой главе. Ответы на вопросы можно найти в разделе «Ответы» в конце книги.

Исходные данные

О компании

Офис туристической компании Margie's Travel находится в г. Сан-Диего. Клиенты могут встретиться с агентом в офисе или разместить заявку на веб-сайте компании.

Ситуация

После добавления нового каталога продуктов веб-сайт компании стал работать нестабильно. Время от времени клиенты не могли совершать покупки на сайте, когда в данные вносили изменения.

Компания недавно отказалась от услуг консалтинговой фирмы, которая занималась разработкой и поддержкой ее веб-сайта и всех остальных приложений и не смогла обеспечить доступность критически важных бизнес-систем.

Планируемые изменения

Перед новым техническим директором была поставлена задача обеспечения высокой доступности всех критически важных бизнес-систем. Технический директор взял на работу нового администратора баз данных и системного администратора для решения этой задачи и помощи в повседневной работе.

Существующая среда

В компании используется 11 баз данных, приведенных в табл. 14-12.

Табл. 14-12. Базы данных компании Margie’s Travel

База данных	Описание	Размер
<i>Orders</i>	Сведения о заказах, полученных от клиентов	50 Гб
<i>Customers</i>	Личная информация о клиентах	15 Гб
<i>CreditCards</i>	Данных о кредитных картах клиентов	200 Мб
<i>Employees</i>	Информация о всех сотрудниках	50 Мб
<i>Human Resources</i>	Все документы отдела кадров и данные о зарплате сотрудников	300 Мб
<i>Products</i>	Данные о товарах, предлагаемых для продажи на веб-сайте компании	25 Гб
<i>Flights</i>	Сведения о билетах на рейсы, забронированных клиентами	2 Гб
<i>Cruises</i>	Данные о путевках, забронированных клиентами	1 Гб
<i>CarRental</i>	Данные об аренде автомобилей клиентами	1 Гб
<i>Excursions</i>	Данные об экскурсиях, забронированных клиентами (К экскурсиям относятся все услуги, кроме полетов, путевок, товаров и аренды автомобилей.)	2 Гб
<i>Admin</i>	Служебная база данных для работы администратора баз данных, временно пустая	12 Гб

В среде компании имеется один веб-сервер с именем WEB1 и один сервер баз данных с именем SQL1. Все серверы работают под управлением ОС Windows Server 2003, а на сервере SQL1 установлено ПО SQL Server 2008.

На сервере SQL1 имеется внешняя дисковая стойка, подключенная к контроллеру RAID-массива с резервной батареей, который может настраиваться на режим RAID 0, RAID 1 и RAID 5. В настоящее время весь массив дисков настроен как единый набор RAID 0. Хранилище используется всего на 10% его мощности.

К серверам WEB1 и SQL1 подключен ленточный накопитель, который ни разу не использовался.

В настоящее время серверы SQL1 и WEB1 находятся в помещении рядом с рабочим местом уволенного консультанта. Все приложения на сервере WEB1 написаны в использовании ColdFusion или ASP.

Предлагаемая среда

Технический директор выделил часть бюджета на приобретение четырех дополнительных серверов с установленными Windows Server 2003 и SQL Server 2008. Все закупаемое оборудование должно поддерживать работу в кластере.

Данные в базах данных *Products*, *Customers*, *Orders*, *Flights*, *Cruises*, *Excursions* и *CarRental* будут доступны через Интернет посредством приложений, выполняемых на сервере Web1. Все остальные базы данных должны быть защищены брандмауэром и доступны только для пользователей, прошедших проверку подлинности в корпоративном домене.

Для хранения баз данных развернута новая сеть SAN, имеющая достаточно дискового пространства для размещения всех баз данных. Каждый из 20 номеров логических устройств (LUN) в сети SAN сконфигурирован в виде зеркальных наборов с чередованием (stripe of mirrors), с четырьмя дисками в каждом зеркальном наборе.

Бизнес-требования

В компании развернуто временное решение, обеспечивающее полное восстановление системы после любых сбоев в течение двух рабочих дней при максимальной потере данных только за последний час перед сбоем. В случае крупной аварии компания может выдержать потерю данных не более чем за два дня.

Для внесения изменений в систему предусмотрено время обслуживания в промежутке от полуночи до 8:00 утра воскресенья.

Необходимо создать долгосрочное решение для защиты компании от аппаратных сбоев с максимальным временем простоя не более одной минуты.

Технические требования

Базы данных *Orders* и *Customers* связаны между собой, поэтому должны размещаться на одном экземпляре SQL Server и вместе переключаться на резервный ресурс кластера при сбое.

Нужно обеспечить максимальную безопасность всех баз данных отдела кадров, а доступ к ним должен предоставляться только директору отдела кадров. Любые данные из отдела кадров должны быть зашифрованы при хранении как в базе данных, так и в любом другом месте в сети.

Для разработки новых маркетинговых кампаний и товарных предложений отделу маркетинга необходимо создавать отчеты на основе всех данных о клиентах и заказах, а также о заказанных товарах и услугах. Для проведения анализа нужны данные, получаемые почти в режиме реального времени.

Все базы данных должны быть доступны 99,92% времени в течение всего года. Для восстановления после сбоев должно требоваться минимальное вмешательство администратора. Для клиентов, использующих веб-сайт, переходы при сбое должны происходить незаметно.

1. Какую технологию (или технологии) нужно использовать для выполнения всех требований к доступности и бизнес-требований? (Выберите все подходящие ответы.)
 - A. Двухузловой кластер с набором узлов большинства.
 - B. Двухузловой стандартный кластер.
 - C. Зеркальное отображение баз данных.
 - D. Репликация.
2. Какую технологию необходимо задействовать, чтобы выполнить требования отдела маркетинга?
 - A. Отказоустойчивый кластер.
 - B. Зеркальное отображение баз данных.
 - C. Доставка журналов.
 - D. Репликация.

3. Какие комбинации ОС Windows и SQL Server удовлетворяют требованиям компании Margie's Travel, обеспечивая максимальную экономию денег?
- A. Windows Server 2003 Standard Edition и SQL Server 2008 Standard.
 - B. Windows Server 2003 Enterprise Edition и SQL Server 2008 Standard.
 - C. Windows Server 2003 Enterprise Edition и SQL Server 2008 Enterprise.
 - D. Windows Server 2003 Datacenter Edition и SQL Server 2008 Datacenter.

Рекомендуемые упражнения

Чтобы подготовиться к экзамену по темам этой главы, выполните следующие задания.

Кластеры Windows

При выполнении следующих заданий используйте кластер Windows, созданный в практикуме занятия 1.

- **Задание 1.** Переключите кластер с узла Node1 на Node2 и проследите состояние каждого ресурса в цепочке зависимостей.
- **Задание 2.** Переключите все группы на узел Node1. Исключите узел Node2 из кластера.
- **Задание 3.** Верните узел Node2 в кластер.
- **Задание 4.** Измените IP-адрес кластера.
- **Задание 5.** Создайте рекомендуемую конфигурацию кластера Windows, присвоив параметру **Публичная сеть** (Public network) значение **Все сообщения** (All Communications), а параметру **Частная сеть** (Private network) — значение **Только для внутрикластерных сообщений** (Internal Clustering Communications Only).

Отказоустойчивые кластеры SQL Server

При выполнении следующих заданий используйте отказоустойчивый кластеризованный экземпляр SQL Server, созданный в практикуме занятия 2.

- **Задание 1.** Переключите экземпляр SQL Server с узла Node1 на Node2 и проследите состояние каждого ресурса в цепочке зависимостей.
- **Задание 2.** Установите второй отказоустойчивый кластеризованный экземпляр в кластер Windows.
- **Задание 3.** Измените IP-адрес сервера, на котором работает SQL Server.
- **Задание 4.** Создайте общую папку, добавьте ее в кластер и настройте ее так, чтобы при ссылке на нее использовалось одно и то же имя независимо от того, на каком узле она в данный момент работает.
- **Задание 5.** Настройте общую папку так, чтобы в случае ее недоступности после переключения кластера на резервный ресурс, это не приводило к отключению всей группы.

Пробный экзамен

На прилагаемом компакт-диске содержится пробный экзамен, обеспечивающий широкие возможности самоподготовки. Например, можно проверить, как усвоены темы этой главы либо все темы экзамена 70-432. Можно симитировать настоящий экзамен или выбрать режим обучения, когда после каждого вопроса приводятся правильные ответы с пояснениями.

ПРИМЕЧАНИЕ Пробный экзамен

Подробнее о пробном экзамене см. «Введение».

Зеркальное отображение базы данных

Занятие 1. Обзор зеркального отображения базы данных	413
Занятие 2. Инициализация зеркального отображения базы данных	424
Занятие 3. Проектирование перехода на резервный ресурс и возврата на основной ресурс	430

Зеркальное отображение базы данных — альтернатива кластеру в плане обеспечения отказоустойчивости SQL Server, причем оно способно обеспечивать защиту от сбоев на уровне одной или нескольких баз данных, а не всего экземпляра.

В этой главе рассказывается о проектировании и реализации зеркального отображения базы данных.

Темы экзамена

- Реализация зеркального отображения базы данных.

Прежде всего

Для изучения материалов этой главы необходимо иметь:

- три установленных экземпляра SQL Server:
 - два под управлением SQL Server 2008 выпуска Standard, Enterprise или Developer;
 - третий может быть под управлением любой версии SQL Server, в том числе SQL Server 2008 Express;
- базу данных *AdventureWorks* версии SQL Server 2005, установленную хотя бы на одном из экземпляров.

ВНИМАНИЕ! Данные файлового потока (FILESTREAM)

Вам потребуется база данных *AdventureWorks* из предыдущего выпуска SQL Server, поскольку в версии базы данных *AdventureWorks* для SQL Server 2008 содержатся данные файлового потока (FILESTREAM), которые не совместимы с зеркальным отображением базы данных.

Занятие 1. Обзор зеркального отображения базы данных

Новая технология зеркального отображения базы данных привнесла с собой новые термины и возможности. В этом занятии рассматривается терминология зеркального отображения базы данных (Database Mirroring), а также принципы работы этой технологии.

Изучив материал этого занятия, вы сможете:

- ✓ спроектировать роли зеркального отображения базы данных.

Продолжительность занятия — 45 минут.

Роли зеркального отображения базы данных

Существует две обязательные *роли зеркального отображения базы данных* и третья дополнительная. Одной базе данных нужно назначить роль основной, а другой — зеркальной базы данных. Также при желании одному экземпляру SQL Server можно назначить роль следящего сервера, который будет управлять автоматическим переключением с основной базы данных на зеркальную. На рис. 15-1 показана примерная схема конфигурации зеркального отображения базы данных.

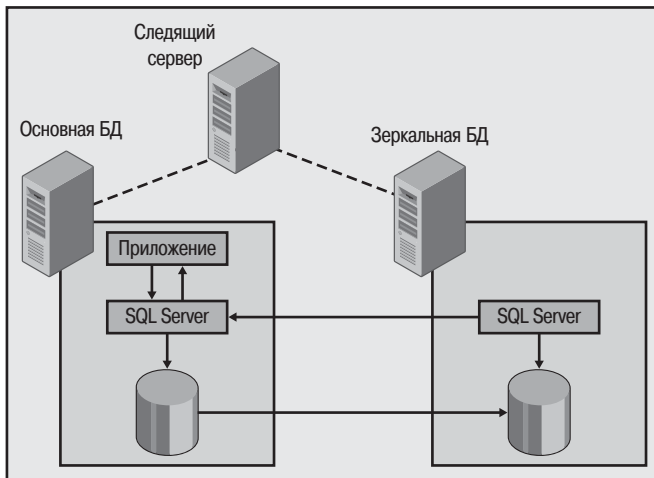


Рис. 15-1. Основные составляющие схемы зеркального отображения базы данных

Базы данных с ролями основной и зеркальной участвуют в *сеансе зеркального отображения базы данных*. Для каждого сеанса можно настроить дополнительный следящий сервер. Один следящий сервер может управлять несколькими сеансами зеркального отображения баз данных.

Роль основной базы данных

База данных, настроенная как *основная* (principal), становится источником всех транзакций в сеансе зеркального отображения базы данных. Эта основная (или

главная) база данных поддерживает восстановление и подключения; приложения получают и записывают в нее данные.

ПРИМЕЧАНИЕ Обслуживание базы данных

Когда на экземпляре размещается база данных, поддерживающая обработку запросов, говорят, что экземпляр предоставляет эту базу данных.

Роль зеркальной базы данных

База данных, которой назначена роль *зеркальной*, является *партнером* основной базы данных и непрерывно принимает транзакции. В процессе зеркального отображения происходит постоянное воспроизведение транзакций из основной базы данных в журнале транзакций и сброс журнала транзакций в файлы данных зеркальной базы. Таким образом, зеркальная БД содержит те же данные, что и основная. Зеркальная база данных находится в состоянии восстановления, поэтому она не поддерживает никакие подключения и в нее не могут быть напрямую записаны транзакции. Однако можно создать моментальный снимок зеркальной базы данных и предоставить доступ только для чтения к ее данным на определенный момент времени.

ПРИМЕЧАНИЕ Временные рабочие состояния

Роли основной и зеркальной базы данных являются временными рабочими состояниями в сеансе зеркального отображения. Поскольку эти базы данных являются точными копиями и синхронизированы между собой, каждая из них может в любой момент взять на себя роль основной или зеркальной.

Следящий сервер

Следящий сервер (witness server) — третья, дополнительная, роль, которую можно определить в механизме зеркального отображения баз данных. Следящий сервер выступает лишь в качестве арбитра в режиме высокой доступности, обеспечивая в каждый момент времени обслуживание базы данных только одним экземпляром SQL Server. Если основная БД вышла из строя, а следящий сервер это подтверждает, зеркальная БД может взять на себя роль основной и начать обслуживать запросы на доступ к данным.

Хотя при зеркальном отображении основная и зеркальная базы данных существуют только в парах (например, у основной базы данных не может быть несколько зеркальных и наоборот), один следящий сервер может обслуживать несколько таких пар. В представлении каталога *sys.database_mirroring_witnesses* каждой участвующей в зеркальном отображении паре баз данных, которая обслуживается данным следящим сервером, отводится отдельная строка.

ВНИМАНИЕ! Роли уровня базы данных и сервера

Роли основной и зеркальной базы данных задаются на уровне базы данных, и их следует определять на экземплярах под управлением SQL Server 2008 Standard или Enterprise. Роль следящего сервера определяют на уровне экземпляра. В качестве следящего сервера можно использовать экземпляр под управлением любой версии SQL Server 2008, в том числе SQL Server 2008 Express. Поэтому говорят об основной или зеркальной *базе данных*, но о следящем *сервере*.

Конечные точки зеркального отображения базы данных

При зеркальном отображении базы данных весь трафик передается через *конечные точки TCP* (TCP endpoint) с полезными данными *DATABASE_MIRRORING*. В экземпляре SQL Server можно создать всего одну конечную точку зеркального отображения базы данных.

ПРИМЕЧАНИЕ Конечные точки

Подробнее об определении конечных точек см. главу 8.

По умолчанию для определения конечной точки зеркального отображения базы данных выбирают порт 5022. Хотя этот порт можно использовать для зеркального отображения базы данных, мы рекомендуем выбрать другой номер порта, чтобы избежать атак хакеров, пытающихся найти бреши, предполагая, что конфигурация системы стандартная.

На одном сервере можно настроить несколько экземпляров SQL Server, у каждого из которых может быть своя конечная точка зеркального отображения базы данных. Однако конечным точкам на различных экземплярах, расположенных на одном сервере, должны быть назначены разные номера портов. Если для зеркального отображения базы данных задействован всего один экземпляр на сервере, порт для него следует выбрать в соответствии со стандартами текущей среды.

Каждой создаваемой конечной точке можно присвоить имя. Имя конечной точки зеркального отображения БД используется только при изменении состояния или выполнении инструкции *GRANT/REVOKE*. Поскольку имя конечной точки применяется только администратором БД для выполнения внутренних операций, рекомендуется оставить имя *Mirroring*, заданное по умолчанию.

Наиболее важным аспектом при конфигурировании зеркального отображения базы данных является безопасность. Вы можете настроить зашифрованный или незашифрованный обмен данными через конечную точку зеркального отображения. Рекомендуется оставить стандартную настройку конечной точки, предусматривающую шифрование всего трафика между конечными точками. Если экземпляры, участвующие в зеркальном отображении, используют разные учетные записи службы SQL Server, надо обеспечить, чтобы каждой учетной записи службы был предоставлен доступ к SQL Server, а также разрешение *CONNECT TO* для конечной точки зеркального отображения БД.

ПРИМЕЧАНИЕ Безопасность конечных точек

Подробнее об определении безопасности конечной точки зеркального отображения БД см. главу 8.

Режимы работы

Зеркальное отображение базы данных можно настроить в одном из трех *режимов работы*: высокая доступность, высокая производительность и высокая безопасность. Режим работы определяет, как SQL Server передает транзакции между основной и зеркальной базами данных, а также процессы переключения на резервный ресурс, доступными в сеансе зеркального отображения БД. В этом занятии рассказывается о каждом из режимов и их преимуществах. Кроме того, вы узнаете об улучшенных возможностях зеркального отображения БД по

сравнению с другими технологиями обеспечения доступности благодаря таким функциям, как кеширование и прозрачное перенаправление клиента.

Режим высокой доступности

Режим *высокой доступности* (High Availability) обеспечивает надежную синхронную передачу данных между основной и зеркальной базами данных, а также автоматическое обнаружение сбоев и переключение на резервный ресурс.

Сначала SQL Server записывает все транзакции в буфер памяти в пространстве памяти SQL Server. Система записывает содержимое этих буферов памяти в журнал транзакций. При записи транзакции в журнал вступает в действие зеркальное отображение БД, инициирующее передачу строк этой транзакции из журнала транзакций в зеркальную базу данных. При фиксации транзакции приложением она сначала фиксируется в зеркальной БД. Подтверждение фиксации направляется в основную базу данных, после чего транзакция фиксируется в основной БД. После этого подтверждение передается в приложение, что позволяет ему продолжить свою работу. Такой порядок гарантирует, что все транзакции фиксируются и заносятся в журнал транзакций в основной и зеркальной БД до передачи подтверждения фиксации в приложение.

Порядок обработки транзакций при зеркальном отображении БД отличает эту технологию от других технологий обеспечения избыточности, например доставки журналов и репликации, при которых транзакция должна быть завершена до ее передачи на другую машину. При зеркальном отображении передача записей журнала происходит по мере их записи в основную БД. Благодаря этому зеркальное отображение базы данных обеспечивает обработку транзакций, состоящих из огромного числа строк, причем это практически не влияет на работу приложения. В сущности, по мере увеличения среднего размера транзакций снижается влияние синхронной передачи данных, необходимой для работы механизма зеркального отображения БД. Такой эффект обусловлен тем, что относительная доля подтверждения, необходимого в режиме высокой доступности, в общем времени выполнения транзакции уменьшается по мере роста размера транзакции.

При синхронной передаче данных перед приложениями встает вопрос планирования. Поскольку транзакция считается зафиксированной лишь после ее фиксации в журнале транзакций основной и зеркальной баз данных, режим высокой доступности создает дополнительную нагрузку на приложения. Чем дальше друг от друга находятся основной и зеркальный экземпляры, тем ниже производительность.

Помимо основной и зеркальной баз данных в режиме высокой доступности также необходим следящий сервер для обнаружения сбоев основной БД и автоматического переключения на зеркальную. Для обнаружения сбоев в режиме высокой доступности используется простая проверка связи с помощью эхо-запросов (ping) между всеми экземплярами, участвующими в сеансе зеркального отображения.

ВНИМАНИЕ! Ограниченные возможности эхо-запросов (ping)

База данных может стать недоступной из-за сбоя транзакции или других операций. Однако при зеркальном отображении такие ситуации не считаются сбоями. Как сбой определяются лишь ошибки при проверке связи.

Также следует учитывать количество настроенных на отдельном экземпляре сеансов зеркального отображения. Каждая база данных в сеансе зеркального отображения отправляет эхо-запросы (ping) другой базе данных раз в несколько секунд для проверки связи и доступности. Раз в несколько секунд обе базы данных также должны отправлять эхо-запросы на следящий сервер, чтобы убедиться в его доступности. Сбой в ходе проверки связи в режиме высокой доступности приведет к переключению на зеркальную базу данных. Когда в сеансах зеркального отображения участвует большое количество баз данных, расположенных на одном сервере, существует вероятность ошибок при переключении из-за чрезмерной нагрузки на сеть.

При переключении на резервный ресурс в ходе сеанса зеркального отображения SQL Server меняет местами роли основной и зеркальной баз данных. Также SQL Server автоматически меняет направление потока транзакций. Этот процесс значительно эффективнее других механизмов обеспечения доступности, таких, к примеру, как репликация или доставка журналов, в которых для изменения потока транзакций требуется вмешательство оператора и даже изменение конфигурации.

При автоматическом переключении на резервный ресурс зеркальная БД берет на себя роль и функции основной базы данных. Но прежде следящий сервер должен разрешить переключение и обмен ролями, заручившись согласием двух из трех ролей в зеркальном отображении БД (т. е. кворума). Кворум необходим, чтобы исключить обслуживание базы данных на нескольких экземплярах в сеансе зеркального отображения. Если основная база данных выйдет из строя, а зеркальная не сможет соединиться со следящим сервером, кворум будет невозможен и SQL Server не сможет перевести зеркальную БД на роль основной.

ПРИМЕЧАНИЕ Феномен «раздвоения личности»

Если бы зеркальная база данных могла по собственному решению брать на себя роль основной, могла бы сложиться ситуация, когда база данных доступна для транзакций более чем на одном сервере. Такой феномен называется «раздвоением личности» (split-brain).

Автоматическое обнаружение сбоев и переключение в режиме высокой доступности выполняются следующим образом:

1. Основная и зеркальная базы данных постоянно обмениваются эхо-запросами.
2. Следящий сервер периодически отправляет эхо-запросы основной и зеркальной базам данных.
3. Основная база данных выходит из строя.
4. Зеркальная база данных обнаруживает сбой и запрашивает у следящего сервера разрешение взять на себя роль основной.
5. Следящий сервер не может связаться с основной, но убеждается в наличии связи с зеркальной базой данных, поэтому соглашается на обмен ролями и переводит зеркальную базу данных на роль основной.
6. Основной сервер восстанавливается после сбоя и обнаруживает, что роль основной базы данных выполняет зеркальная.

7. SQL Server переводит исходную основную базу данных на роль зеркальной и направляет к ней поток транзакций для ее синхронизации с новой основной базой данных.

ВНИМАНИЕ! Доступность следящего сервера при горячей замене

Автоматическое переключение возможно лишь в случае доступности следящего сервера. Если следящий сервер недоступен, автоматическое переключение не произойдет. Это означает, что режим высокой доступности можно использовать для создания сервера горячей замены лишь в том случае, если доступен следящий сервер. Иначе получается режим горячего резервирования.

Режим высокой производительности

В режиме *высокой доступности* (High Performance) используются основная и зеркальная база данных и не требуется следящий сервер. Этот режим обеспечивает горячее резервирование, не поддерживающее автоматическое обнаружение сбоев и переключение на резервный ресурс.

В режиме высокой доступности невозможен автоматический переход с основной базы данных на зеркальную из-за асинхронной отправки транзакций в зеркальную базу данных. Транзакции фиксируются в основной базе данных, а приложение получает подтверждение о фиксации транзакций. Отдельный процесс постоянно направляет эти транзакции в зеркальную базу данных, что создает задержку в выполнении этого процесса и делает невозможным автоматическое переключение в сеансе зеркального отображения из-за того, что до сбоя зеркальная база данных могла не получить все транзакции.

Поскольку передача транзакций выполняется асинхронно, режим высокой производительности не оказывает влияния на быстродействие приложений и допускает большую удаленность основной и зеркальной баз данных. Однако из-за асинхронности передачи возможна потеря транзакций при сбое основной базы данных или при принудительном переходе с основной базы данных на зеркальную в сеансе зеркального отображения.

Режим высокой безопасности

В режиме *высокой безопасности* (High Safety) транзакции передаются синхронно, но отсутствует следящий сервер. Синхронная передача гарантирует, что все транзакции, зафиксированные в основной базе данных, предварительно фиксируются в зеркальной базе данных. При анализе производительности в этом режиме следует руководствоваться теми же соображениями, что и для режима высокой доступности. Однако отсутствие следящего сервера делает невозможным автоматический переход на зеркальную базу данных в случае сбоя основной. При выходе из строя основной базы данных в режиме высокой безопасности необходимо вручную перевести зеркальную базу данных на роль основной.

Переключение вручную без потери транзакций возможно только в режиме высокой безопасности. При этом SQL Server 2008 разрывает все соединения с основной базой данных и переводит ее в состояние синхронизации, чтобы исключить установку новых соединений. Последние записи из журнала транзакций направляются в зеркальную базу данных, а затем основная и зеркальная базы меняются ролями. После этого клиенты смогут повторно подключиться к базе данных и продолжить обработку транзакций.

ПРИМЕЧАНИЕ Режим высокой безопасности

Синхронная передача транзакций в режиме высокой безопасности обычно снижает производительность работы приложений, но, вместе с тем, в этом режиме невозможно автоматическое переключение в случае сбоя. Поэтому данный режим не рекомендуется использовать в обычных условиях. Режим высокой безопасности рекомендуется только для замены текущего следящего сервера. После замены или восстановления следящего сервера необходимо вернуться в режим высокой доступности.

Кеширование

Любая технология обеспечения высокой доступности в SQL Server 2008 оказывает влияние на производительность и, зачастую, на работу приложений при переключении на резервный ресурс в случае сбоя. Кластеризация позволяет избежать подобных ситуаций, поскольку при ней задействуется лишь один экземпляр, но при этом требуется перезапуск экземпляра на другом узле, что приводит к необходимости перезаполнения кешей данных и запросов. При доставке журналов требуется особая архитектура приложения, предусматривающая переключение на сервер-получатель (secondary server) и перезаполнение кешей данных и процедур. При репликации также требуется изменить приложение, предусмотрев возможность повторного подключения к подписчику, что отчасти отрицательно сказывается на производительности, поскольку требуется перезаполнение кеша запросов и частично кеша данных.

При зеркальном отображении БД отсутствуют проблемы с кешированием. Помимо отправки транзакций в зеркальную БД в ходе зеркального отображения выполняется периодическая передача метаданных, при которой зеркальная БД считывает страницы в кеш данных. Этот процесс позволяет сохранять кеш зеркальной базы данных в «полугорячем» состоянии. Кеш зеркальной БД не является точным отражением кеша основной, но содержит большинство страниц, поэтому при переходе на резервный ресурс в ходе зеркального отображения SQL Server не перестраивает кеш полностью и не оказывает столь сильного влияния на производительность, как другие технологии обеспечения доступности.

Прозрачное перенаправление клиента

При использовании механизма доставки журналов или репликации одна из самых больших сложностей переключения на резервный ресурс связана с соединениями с приложениями. Для продолжения нормальной работы приложений их необходимо перенаправить на сервер-получатель. Особенности конфигурации при зеркальном отображении базы данных позволяют избежать таких сложностей.

Новая версия компонентов доступа к данным MDAC (Microsoft Data Access Components), поставляемая в составе Microsoft Visual Studio 2008, содержит функциональную возможность, относящуюся к зеркальному отображению базы данных, в объекте подключения под названием *Transparent Client Redirect* (прозрачное перенаправление клиента). При подключении клиента к основной базе данных в этом объекте кешируются основная и зеркальная базы данных. Это кеширование прозрачно для приложений, и для реализации этой функциональности разработчикам не нужно писать дополнительный код.

Если в ходе сеанса зеркального отображения базы данных происходит переключение на резервный ресурс, когда установлено соединение приложения с базой данных, это соединение разрывается, а объект подключения возвращает клиенту ошибку. Затем клиенту нужно лишь повторно подключиться к базе данных, а кеш подключений MDAC автоматически перенаправит это подключение к зеркальному серверу. Приложение как будто подключается к исходному серверу, хотя на самом деле это уже другой сервер.

Организация потоков при зеркальном отображении базы данных

Зеркальное отображение базы данных — это многопоточный процесс, при котором возможно параллельное выполнение различных операций между основной и зеркальной базами данных и следящим сервером.

При инициации сеанса зеркального отображения компонент Service Broker назначается ответственным за управление глобальным потоком обмена данными. Этот поток служит для обнаружения ситуаций, когда один из участников сеанса зеркального отображения базы данных недоступен. При этом в режиме высокой доступности выполняется переключение на резервный ресурс, а участники сеанса могут обмениваться сообщениями о состоянии.

Помимо глобального потока обмена данными на экземпляре создается по одному потоку для каждой базы данных, участвующей в сеансе зеркального отображения. Потоки баз данных предназначены для обмена сообщениями между основной и зеркальной базами данных, например транзакциями и подтверждениями. На зеркальном сервере для каждой базы данных создается отдельный поток, управляющий записью в журнал и кешами запросов и данных.

На следящем сервере имеется дополнительный поток, который служит для управления обменом сообщениями между следящим сервером и основными и зеркальными базами данных во всех связанных с ним сеансах зеркального отображения. Прежде всего, через этот поток направляются сообщения об изменении состояния основной или зеркальной баз данных и запросы на переключение на резервный ресурс.

Моментальные снимки баз данных

Зеркальная база данных в сеансе зеркального отображения постоянно находится в состоянии восстановления и недоступна для пользователей. Однако для нее можно создать моментальный снимок, поддерживающий доступ только для чтения на определенный момент времени.

Вот пример создания моментального снимка базы данных:

```
CREATE DATABASE AdventureWorksDS ON  
( NAME = AdventureWorks_Data, FILENAME =  
'C:\Program Files\Microsoft SQL  
Server\MSSQL10.MSSQLSERVER\MSSQL\Data\AdventureWorks_data.ds' )  
AS SNAPSHOT OF AdventureWorks;  
GO
```

Благодаря моментальным снимкам зеркальной базы данных можно обеспечить масштабирование отчетности средствами зеркального отображения базы данных.

Контрольные вопросы

- 1. Назовите три роли зеркального отображения базы данных. Каковы их функции?
- 2. Назовите три режима работы зеркального отображения базы данных.
- 3. Какой режим работы не рекомендуется использовать в обычных условиях?

Ответы на контрольные вопросы

- 1. Основная база данных обеспечивает работу с приложениями. Зеркальная база данных находится в состоянии восстановления и не поддерживает соединения. Следящий сервер (необязательная роль) — это экземпляр SQL Server, который служит для автоматического обнаружения сбоев и переключения с основной базы данных на зеркальную.
- 2. Режим высокой доступности, режим высокой производительности и режим высокой безопасности.
- 3. Режим высокой безопасности не рекомендуется в обычных условиях, поскольку синхронная передача, выполняемая в этом режиме, существенно влияет на производительность, в нем также не поддерживается автоматический переход на резервный ресурс.

Практикум. Создание конечных точек для зеркального отображения базы данных

Вы создадите конечные точки, необходимые для сеанса зеркального отображения базы данных. Вы настроите конечные точки для основной и зеркальной баз данных и следящего сервера, обеспечив создание сеанса зеркального отображения базы данных в любом режиме. В табл. 15-1 приведены соглашения об именовании, используемые в упражнениях этой главы.

Табл. 15-1. Соглашения об именовании экземпляров

Имя экземпляра	Роль
INSTP	Основная база данных
INSTM	Зеркальная база данных
INSTW	Следящий сервер

Экземпляры в этом упражнении настроены только в режиме проверки подлинности Windows. Для запуска служб SQL Server используется учетная запись SQLAdmin. Для выполнения этого упражнения вам потребуется три различных сервера, на каждом из которых установлен экземпляр SQL Server. Для экземпляров, обслуживающих основную и зеркальную базы данных, необходим SQL Server 2008 Standard или более поздний выпуск. Экземпляр, выполняющий роль следящего сервера, может работать под управлением любого выпуска SQL Server 2008. Вместо имени экземпляра укажите имя сервера, а вместо учетной записи SQLAdmin — имя вашей учетной записи Windows.

ВНИМАНИЕ! Номера портов

Если экземпляры установлены на одном сервере, у конечных точек должны быть разные номера портов во избежание конфликта в стеке TCP/IP.

1. Подключитесь к экземпляру INSTP в среде SQL Server Management Studio (SSMS), откройте новое окно запроса и выполните следующую команду:

```
CREATE ENDPOINT [Mirroring]
AUTHORIZATION [INSTP\SQLAdmin]
STATE=STARTED
AS TCP (LISTENER_PORT = 5024, LISTENER_IP = ALL)
FOR DATA_MIRRORING (ROLE = PARTNER, AUTHENTICATION = WINDOWS NEGOTIATE
, ENCRYPTION = REQUIRED ALGORITHM RC4)
```

ПРИМЕЧАНИЕ Получение информации о конечной точке

Чтобы получить информацию о конечной точке, создайте запрос динамического представления *sys.database_mirroring_endpoints*.

2. Подключитесь к экземпляру INSTM, откройте новое окно запроса и выполните следующую команду:

```
CREATE ENDPOINT [Mirroring]
AUTHORIZATION [INSTM\SQLAdmin]
STATE=STARTED
AS TCP (LISTENER_PORT = 5024, LISTENER_IP = ALL)
FOR DATA_MIRRORING (ROLE = PARTNER, AUTHENTICATION = WINDOWS NEGOTIATE
, ENCRYPTION = REQUIRED ALGORITHM RC4)
```

ПРИМЕЧАНИЕ Выбор имени конечной точки

В целях стандартизации наименования этого типа конечных точек в качестве имени конечной точки укажите *Mirroring*. Тем самым вы сможете с легкостью отличить ее от конечных точек других типов.

3. Подключитесь к экземпляру INSTW, откройте новое окно запроса и выполните следующую команду:

```
CREATE ENDPOINT [Mirroring]
AUTHORIZATION [INSTW\SQLAdmin]
STATE=STARTED
AS TCP (LISTENER_PORT = 5024, LISTENER_IP = ALL)
FOR DATA_MIRRORING (ROLE = WITNESS, AUTHENTICATION = WINDOWS NEGOTIATE
, ENCRYPTION = REQUIRED ALGORITHM RC4)
```

ПРИМЕЧАНИЕ На экзамене

Для зеркального отображения базы данных конечные точки основной и зеркальной баз данных и следящего сервера должны быть настроены на разных экземплярах SQL Server.

4. Подключитесь к каждому экземпляру и проверьте только что созданные конечные точки, выполнив следующие команды:

```
SELECT * FROM sys.database_mirroring
```

```
SELECT * FROM sys.database_mirroring_endpoints  
SELECT * FROM sys.database_mirroring_witnesses
```

ПРИМЕЧАНИЕ Альтернативный графический пользовательский интерфейс

В среде SSMS предусмотрен специальный графический пользовательский интерфейс для настройки конечных точек зеркального отображения базы данных и для конфигурирования, переключения, приостановки, возобновления и изменения параметров режима работы сеанса зеркального отображения. Чтобы открыть этот интерфейс, щелкните правой кнопкой базу данных, выберите **Свойства** (Properties) и в диалоговом окне **Свойства** (Properties) выберите страницу **Зеркальное отображение** (Mirroring).

Резюме занятия

- В зеркальном отображении базы данных участвует три роли. База данных, выполняющая роль основной, доступна для подключений и поддерживает изменение данных. База данных, выполняющая роль зеркальной, недоступна для подключений и получает транзакции, выполняемые в основной базе данных. Роль следящего сервера выполняет экземпляр SQL Server; эта роль применяется в режиме высокой доступности для принятия решения о переходе на резервный ресурс.
- Зеркальное отображение базы данных можно настроить в одном из трех режимов. В режиме высокой доступности участвуют основная и зеркальная базы данных и следящий сервер и поддерживается автоматический переход на резервный ресурс. В режиме высокой производительности участвуют основная и зеркальная базы данных и поддерживается переключение на резервный ресурс вручную. В режиме высокой безопасности также участвуют основная и зеркальная базы данных и поддерживается переключение на резервный ресурс вручную. В режимах высокой доступности и высокой безопасности передача данных выполняется синхронно, а в режиме высокой производительности — асинхронно.
- Весь трафик зеркального отображения базы данных передается между конечными точками TSP, заданными на каждом экземпляре, обслуживающем базу данных в сеансе зеркального отображения.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Какая роль участвует в зеркальном отображении базы данных?
 - A. Издатель.
 - B. Основная база данных.
 - C. База данных-источник.
 - D. Монитор.
2. Какие из нижеперечисленных действий выполняет следящий сервер? (Выберите все правильные ответы.)

- A. Принятие решения о переходе на резервный ресурс в режиме высокой безопасности.
 - B. Принятие решения о переходе на резервный ресурс в режиме высокой доступности.
 - C. Обслуживание базы данных, когда основная и зеркальная базы данных недоступны.
 - D. Обслуживание нескольких сеансов зеркального отображения базы данных.
3. Какие из следующих значений параметров конечных точек необходимы для обмена транзакциями между основной и зеркальной базами данных? (Выберите 2 варианта — в совокупности они представляют правильный ответ.)
- A. Параметр *STATE* со значением по умолчанию.
 - B. Порт 6083, настроенный для обмена данными.
 - C. Параметр *COMPRESSSION* со значением *ENABLED*.
 - D. Параметр *ROLE* со значением *PARTNER*.

Занятие 2. Инициализация зеркального отображения базы данных

Зеркальное отображение базы данных настраивается на уровне базы данных. Все базы, участвующие в сеансе зеркального отображения, должны настраиваться с моделью полного восстановления. До начала сеанса зеркального отображения каждую зеркальную базу данных нужно синхронизировать с основной с помощью резервной копии. В этом занятии рассматриваются четыре основных этапа подготовки к зеркальному отображению базы данных.

- 1. Проверка того, что базы данных настроены с моделью полного восстановления.
- 2. Резервное копирование основной базы данных.
- 3. Восстановление базы данных на экземпляре, где находится зеркальная база данных, с параметром *NORECOVERY*.
- 4. Копирование всех необходимых системных объектов на экземпляр, где находится зеркальная база данных.

ПОДГОТОВКА К ЭКЗАМЕНУ

Зеркальное отображение нельзя настроить в базах данных, содержащих файловую группу *FILESTREAM*.

Изучив материал этого занятия, вы сможете:

- ✓ инициализировать зеркальное отображение базы данных.

Продолжительность занятия — 45 минут.

Практический совет

Майк Хотек

Мой клиент, в компании которого тысячи баз данных расположены на сотнях экземпляров SQL Server, пытался найти сравнительно простую стратегию обеспечения доступности. В текущем решении применялись доставка журналов и репликация. Репликация изначально была настроена для нескольких систем, поскольку заказчику было необходимо снять с них нагрузку по созданию отчетов.

Однако по мере роста числа баз данных и экземпляров нагрузка на распространителя возросла настолько, что заказчику пришлось поддерживать почти дюжину различных распространителей. Кроме того, существенно усложнилось управление архитектурой, что потребовало принять в штат новых администраторов баз данных.

После тщательного анализа стало ясно, что заказчику необходимо обеспечить автоматический переход на резервный ресурс для нескольких баз данных. Однако эту задачу не решали ни репликация, ни доставка журналов. Все операции по созданию отчетов на подписчиках можно было выполнить с помощью копий данных на определенный момент времени. Для остальных баз данных автоматическое переключение не требовалось.

Вместо того, чтобы расширить архитектуру и попытаться создать средства для более простого развертывания и управления тысячами сеансов доставки журналов и репликации, мы решили заменить всю текущую архитектуру на зеркальное отображение баз данных.

Начиная с версии CTP 16 сервера SQL Server 2005 мы начали заменять сеансы доставки журналов на зеркальное отображение базы данных в режиме высокой производительности. После реализации первой группы и стабилизации архитектуры мы заменили остальные сеансы доставки журналов в базах данных, для которых требовалось настроить автоматический переход на резервный ресурс, на зеркальное отображение в режиме высокой доступности.

Наконец мы заменили сеансы репликации на зеркальное отображение базы данных в режиме высокой производительности или высокой доступности в зависимости от требуемых возможностей перехода на резервный ресурс. Мы запланировали задания для периодического создания моментальных снимков зеркальной БД для баз данных, используемых для распределения нагрузки по созданию отчетов.

Модели восстановления

В SQL Server предусмотрены три модели восстановления баз данных: простая, модель восстановления с неполным протоколированием и модель полного восстановления. В простой модели все транзакции фиксируются в журнале, но неактивная часть журнала транзакций удаляется после каждой контрольной точки. В модели восстановления с неполным протоколированием некоторые операции фиксируются в журнале транзакций не полностью (например, *BULK INSERT*, *BCP*, *CREATE INDEX* и т. п.). При зеркальном отображении основная

и зеркальная базы данных являются точными копиями друг друга, включая синхронизацию всех внутренних структур, например номеров последовательности файлов журнала (LSN), поэтому простая модель восстановления и модель восстановления с неполным протоколированием не совместимы с зеркальным отображением. Таким образом в зеркальном отображении могут участвовать базы данных только с моделью полного восстановления.

ПРИМЕЧАНИЕ Модель полного восстановления как обязательное условие

Для баз данных, не имеющих модели полного восстановления, невозможно настроить зеркальное отображение. Кроме того, для баз данных, участвующих в зеркальном отображении, нельзя изменить модель восстановления.

Резервное копирование и восстановление

Поскольку основная и зеркальная базы данных являются копиями друг друга, необходим специальный механизм, гарантирующий, что обе базы данных будут инициализированы в одинаковом состоянии. Процесс инициализации для зеркального отображения включает резервное копирование основной базы данных и восстановление ее как зеркальной БД. Резервное копирование — единственный механизм, используемый для инициализации зеркальной базы данных, поскольку необходима синхронизация всех внутренних структур, в том числе номеров последовательности файлов журнала (LSN), а также данных.

При восстановлении базы данных в качестве зеркальной очень важно указать для команды *RESTORE* параметр *NORECOVERY*. Благодаря этому начальное состояние зеркальной базы данных будет отражать состояние основной базы данных, включая номера LSN.

При конфигурировании зеркального отображения базы данных вы наверняка заметите, что больше всего времени занимает процесс резервного копирования и восстановления. Однако вам скорее всего не удастся перевести основную базу данных в автономный режим для того, чтобы инициализировать зеркальное отображение базы данных. Вместо этого, учитывая что зеркальная база данных находится в невосстановленном состоянии, для ее актуализации можно воспользоваться цепочкой журналов транзакций.

ПРИМЕЧАНИЕ Инициализация зеркальной базы данных

Вместо создания резервной копии для инициализации зеркальной базы данных я всегда использую последнюю полную резервную копию основной базы данных и применяю к ней все последующие журналы транзакций. После создания резервных копий всех журналов я создаю резервную копию последнего журнала транзакций для того, чтобы загрузить все оставшиеся транзакции, а затем инициализирую зеркальное отображение базы данных. Можно воспользоваться и другим методом — функцией доставки журналов для синхронизации двух баз данных и для инициализации зеркального отображения базы данных. В этом случае до начала сеанса зеркального отображения скорее всего потребуется применить резервную копию хотя бы одного журнала транзакций. Подробнее о резервном копировании и восстановлении и доставке журналов см. главы 9 и 16 соответственно.

Копирование системных объектов

Зеркальное отображение выполняется на уровне базы данных, поэтому оно не управляет никакими другими объектами на сервере. Вы можете настроить автоматический переход на зеркальную базу данных в случае сбоя, но если после перехода нужно обеспечить работоспособность приложений, для этого потребуется перенести все остальные объекты на экземпляр, обслуживающий зеркальную БД.

Так, необходимо перенести такие объекты, как учетные данные для проверки подлинности приложений при доступе к базе данных. На сервере, обслуживающем основную базу данных, также могут быть заданы связанные серверы, пакеты служб SQL Server Integration Services (SSIS), задания агента SQL Server, настраиваемые сообщения об ошибках и другие объекты. Копирование всех этих объектов на экземпляр, обслуживающий зеркальную базу данных, является заключительным этапом в процессе инициализации.

ПРИМЕЧАНИЕ Перенос объектов с помощью заданий служб SSIS

Для переноса объектов на экземпляр, обслуживающий зеркальную базу данных, можно воспользоваться службами SSIS, включающими задачу «Передача имен входа» (Transfer Logins) для переноса учетных записей с одного экземпляра SQL Server на другой, причем все пароли остаются в зашифрованном виде. Службы SSIS также содержат задачи для переноса заданий агента SQL Server, сообщений об ошибках и объектов других типов.

Контрольный вопрос

- Опишите процесс подготовки базы данных к сеансу зеркального отображения.

Ответ на контрольный вопрос

- Нужно изменить модель восстановления на полную, создать резервную копию основной базы данных, восстановить ее на экземпляре, обслуживающем зеркальную базу данных, с параметром *NORECOVERY*, а затем скопировать все системные объекты, такие как имена входа, связанные серверы и т. д.

Практикум. Конфигурирование зеркального отображения базы данных

Вы инициализируете зеркальное отображение базы данных, настроите его режим работы и создадите моментальный снимок базы данных.

ПРИМЕЧАНИЕ Несовместимость с файловыми группами FILESTREAM

Поскольку зеркальное отображение базы данных нельзя настроить для базы данных, содержащей файловые группы FILESTREAM, настоятельно рекомендуется сохранить все данные FILESTREAM в отдельной базе данных.

Задание 1. Инициализация зеркального отображения базы данных

Вы подготовите базу данных к зеркальному отображению на примере базы данных *AdventureWorks*.

1. Подключитесь к экземпляру INSTP в среде SSMS, откройте новое окно запроса и выполните следующий код:

```
ALTER DATABASE AdventureWorks SET RECOVERY FULL;
```

2. На сервере, где размещен экземпляр INSTP, создайте каталог с именем C:\TK432.
3. На сервере, где размещен экземпляр INSTM, создайте каталог с именем C:\TK432.
4. Создайте резервную копию базы данных *AdventureWorks* на экземпляре INSTP, выполнив следующий код:

```
BACKUP DATABASE AdventureWorks TO DISK = 'C:\TK432\AdventureWorks.bak';  
BACKUP LOG AdventureWorks TO DISK = 'C:\TK432\AdventureWorks.trn';
```

5. Скопируйте резервные копии, созданные в п. 4, на сервер, обслуживающий экземпляр INSTM.
6. Восстановите базу данных *AdventureWorks* на сервере INSTM в режиме *NORECOVERY*, выполнив следующий код:

```
RESTORE DATABASE AdventureWorks FROM DISK = 'C:\TK432\AdventureWorks.bak'  
WITH NORECOVERY;  
RESTORE LOG AdventureWorks FROM DISK = 'C:\TK432\AdventureWorks.trn' WITH  
NORECOVERY;
```

7. Создайте пакет SSIS, чтобы скопировать все имена входа, задания, связанные серверы и настраиваемые сообщения об ошибках с экземпляра INSTP на INSTM.
8. Подключитесь к экземпляру INSTM и отключите все задания, перенесенные с экземпляра INSTP.

Задание 2. Настройка режима работы

Вы настроите базу данных *AdventureWorks* в режиме высокой доступности.

1. Подключитесь к экземпляру INSTM и выполните следующий код:

```
ALTER DATABASE AdventureWorks SET PARTNER = 'TCP://<имя_сервера>:5024';
```

2. Подключитесь к экземпляру INSTP и выполните следующий код:

```
ALTER DATABASE AdventureWorks SET PARTNER = 'TCP://<имя_сервера>:5024';  
ALTER DATABASE AdventureWorks SET WITNESS = 'TCP://<имя_сервера>:5024';
```

ПРИМЕЧАНИЕ Поиск конечных точек зеркального отображения базы данных

Если вы не помните адреса конечных точек, их можно узнать одним из следующих способов.

- Чтобы узнать адрес конечной точки на экземпляре, выполните запрос представления *sys.database_mirroring_endpoints*.
- Запустите мастер настройки безопасности зеркального отображения баз данных, щелкнув правой кнопкой основную базу данных и выбрав **Зеркаль-**

ное отображение (Mirroring) в меню **Задачи** (Tasks). Затем в диалоговом окне **Свойства базы данных** (Database Properties) щелкните ссылку **Настроить безопасность** (Configure Security) и выполните последующие шаги мастера. Поскольку конечные точки уже были созданы, мастер автоматически извлечет их данные. После завершения работы мастера адреса конечных точек будут автоматически указаны в соответствующих полях настройки зеркального отображения базы данных.

3. Убедитесь в том, что зеркальное отображение базы данных выполняется. Для этого проверьте, что база данных *AdventureWorks* на экземпляре INSTP находится в состоянии *Основная база данных, Синхронизовано* (Principal, Synchronized) и что база данных *AdventureWorks* на экземпляре INSTM находится в состоянии *Зеркальная база данных, Синхронизовано* (Mirror, Synchronized).

ПРИМЕЧАНИЕ Альтернативный графический пользовательский интерфейс для настройки зеркального отображения базы данных

Для настройки зеркального отображения базы данных с помощью графического пользовательского интерфейса щелкните правой кнопкой базу данных *AdventureWorks* на экземпляре INSTP и выберите вкладку **Зеркальное отображение** (Mirroring).

Задание 3. Создание моментального снимка зеркальной базы данных

Вы создадите моментальный снимок зеркальной базы данных.

1. Подключитесь к экземпляру INSTM и выполните следующий код:

```
CREATE DATABASE AdventureWorksDS ON  
( NAME = AdventureWorks_Data, FILENAME =  
'C:\Program Files\Microsoft SQL  
Server\MSSQL10.MSSQLSERVER\Data\AdventureWorks_data.ds' )  
AS SNAPSHOT OF AdventureWorks;
```

2. Проверьте корректность создания моментального снимка, выполнив следующий код:

```
USE AdventureWorksDS  
GO  
SELECT DepartmentID, Name, GroupName, ModifiedDate  
FROM HumanResources.Department;
```

Резюме занятия

- Во всех БД, участвующих в зеркальном отображении, должна быть указана модель полного восстановления, что обеспечить применение всех транзакций к зеркальной базе данных.
- Затем необходимо инициализировать зеркальную базу данных, восстановив резервную копию с параметром *NORECOVERY*.
- Поскольку при зеркальном отображении базы данных выполняется лишь копирование содержимого базы данных на сервер, обслуживающий зеркаль-

ную базу данных, следует отдельно скопировать все остальные объекты сервера, в частности имена входа, связанные серверы и задания.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Что нужно предпринять для подготовки базы данных к сеансу зеркального отображения? (Выберите все правильные ответы.)
 - A. Настроить распространение.
 - B. Создать резервную копию базы данных.
 - C. Восстановить базу данных с параметром *RECOVERY*.
 - D. Восстановить базу данных с параметром *NORECOVERY*.
2. Какой параметр настройки используется при зеркальном отображении?
 - A. Модель полного восстановления.
 - B. Уровень совместимости 80.
 - C. Только для чтения.
 - D. Модель восстановления с неполным протоколированием.
3. Укажите характеристики режима высокой доступности. (Выберите все правильные ответы.)
 - A. Асинхронная передача данных.
 - B. Синхронная передача данных.
 - C. Автоматический переход на резервный ресурс.
 - D. Переход на резервный ресурс вручную.

Занятие 3. Проектирование перехода на резервный ресурс и возврата на основной ресурс

Решения по обеспечению отказоустойчивости не предназначены для предотвращения простоев. Их цель — как можно быстрее вернуть систему в оперативный режим. Чтобы свести к минимуму время простоя, необходимо задокументировать и отладить процессы перехода на резервный ресурс и возврата на основной ресурс.

Изучив материал этого занятия, вы сможете:

- ✓ спроектировать стратегию запланированной и незапланированной смены ролей.

Продолжительность занятия — 20 минут.

Проектирование перехода на резервный ресурс в сеансе зеркального отображения

Сеансы зеркального отображения выполняются между базами данных. Поэтому они не затрагивают межбазовые транзакции или любые объекты сервера вне баз данных, участвующие в зеркальном отображении.

Приложения, выполняющие транзакции в нескольких базах данных на одном экземпляре, следует учитывать только в том случае, если данные в одной базе данных зависят от данных в другой (например, база данных о клиентах логически связана с отдельной базой данных с заказами этих клиентов). Для логически связанных баз данных необходимо, чтобы все они использовали одинаковый режим работы при зеркальном отображении. Также следует создать политику, согласно которой все эти базы данных должны рассматриваться как группа и участвовать в переходе на резервный ресурс как группа. Благодаря объединению баз данных в одну группу можно свести к минимуму вероятность наличия логически противоречивых данных во всех базах данных.

Перенос имен входа и связанных серверов — это наиболее важный этап, гарантирующий, что приложения будут работать после перехода на резервный ресурс в случае сбоя. В службах SSIS есть задача по переносу имен входа между экземплярами. При переносе имен входа SQL Server службы SSIS сохраняют их пароли в зашифрованном виде, предотвращая нарушение безопасности. Связанные серверы приходится создавать повторно.

Если для доступа используются учетные записи Windows, после перехода на резервный ресурс ничего дополнительно делать не потребуется. Если же для доступа применяются имена входа SQL Server, то после перехода на резервный ресурс могут потребоваться дополнительные действия. При создании учетных данных SQL Server генерируется пустой идентификатор безопасности (security identifier, SID), который затем служит для связи пользователя в базе данных с именем входа на сервере. Если не создать учетные данные SQL Server в том же порядке, что и в основной базе данных, имя входа и пользователь в базе данных могут не совпасть, что может привести к предоставлению имени входа разрешений более высокого уровня. В этом случае необходимо выполнить команду *ALTER LOGIN*, чтобы повторно сопоставить имена входа. Однако поскольку зеркальная база данных не доступна, этот шаг можно выполнить только после перехода на резервный ресурс.

Для правильного и полного перехода приложений на резервный ресурс нужно заново создать в зеркальной базе данных другие объекты, такие как задания агента SQL Server, пакеты SSIS и настраиваемые сообщения об ошибках. Задания агента SQL Server создавать необходимо, но они будут отключены, т. к. у них нет доступа к зеркальной базе данных.

Проектирование возврата на основной ресурс в сеансе зеркального отображения

Наиболее сложным аспектом любого решения по обеспечению высокой доступности является разработка стратегии возврата на основной ресурс. Когда происходит переключение на дополнительный сервер, все транзакции выполняются на этом сервере. Чтобы разрешить приложениям обратно подключаться

к основной базе данных, до перевода базы данных обратно в рабочий режим к ней потребуются применить текущую копию данных.

Практический совет. Переключение на основной ресурс

Допустим, что для достижения максимальной доступности среды вы реализовали несколько решений по обеспечению высокой доступности. Дополнительный сервер взял на себя роль основного вследствие сбоя или действий администратора. Если не нужно переключить систему обратно на вышедший из строя сервер после того, как он вернется в оперативный режим, можно этого не делать. Возврат на основной ресурс может потребоваться лишь по следующим причинам:

- таково требование политики компании;
- снижение производительности.

Политика компании может требовать, чтобы сервер, назначенный на роль основного, был таковым всегда, когда он доступен. Настоятельно рекомендуем отказаться от таких политик, поскольку они налагают излишние искусственные ограничения на сотрудников ИТ-отдела и порядок управления системами. Если основной и дополнительный серверы во всех отношениях равноценны (с точки зрения безопасности, производительности, мощности и т. д.), неважно, какой из них обрабатывает запросы. Поэтому следует по возможности отказаться от «серверов аварийного восстановления» и называть их «дополнительными серверами», дабы не вводить руководство компании в искушение создавать искусственные построения.

Иногда основной и дополнительный сервер невозможно сделать одинаковыми. На дополнительном сервере могут быть менее мощные аппаратные ресурсы или ограниченный объем памяти, что сильно сказывается на его способности обслуживать организацию, но его мощностей может быть достаточно для поддержки работы компании на время неработоспособности основного сервера. В случае падения производительности на дополнительном сервере нужно будет срочно вернуть «к жизни» основной сервер, а затем как можно скорее переключить на него все приложения, таким образом вернув производительность в нормальные рамки.

В зеркальном отображении базы данных были устранены почти все проблемы с возвратом на основной ресурс, свойственные другим технологиям. Рассмотрим два пути возврата на основной ресурс: после мягкого и принудительного переключения.

Возврат на основной ресурс после мягкого переключения

При выходе из строя основной БД зеркальная база данных берет на себя ее роль — при переключении вручную или автоматически. После возврата основной базы данных в оперативный режим она автоматически назначается на роль зеркальной. Такой автоматический перевод основной БД не позволяет приложениям подключиться к базе данных в ее более раннем состоянии.

Поскольку в сеансе зеркального отображения каждая база данных синхронизируется с другими, существует возможность постепенной (накопительной)

синхронизации вышедшей из строя базы данных. Вышедшая из строя основная база данных не только автоматически переводится на роль зеркальной — при этом также меняется направление потока транзакций с тем, чтобы в вышедшую из строя БД попали все недостающие транзакции.

Если вышедшая из строя база данных какое-то время находилась в автономном режиме, могли быть созданы резервные копии журнала транзакций, удалившие записи из журнала транзакций. Поскольку вышедшая из строя база данных теперь находится в состоянии восстановления, к ней можно применить резервные копии журнала транзакций для выполнения ее наката (roll forward). Как только накат достигнет определенной точки во времени, когда номер LSN вышедшей из строя базы данных будет перекрыт журналом транзакций основной базы данных, зеркальное отображение сможет продолжить отправку оставшихся транзакций для завершения синхронизации вышедшей из строя БД.

Чтобы такая накопительная синхронизация прошла гладко и без ошибок, выполните следующее:

1. Приостановите резервное копирование журнала транзакций основной БД.
2. Переведите вышедшую из строя БД в оперативный режим.
3. Восстановите все резервные копии журнала транзакций, созданные с момента сбоя и до наиболее поздней копии, обязательно указав параметр *NORECOVERY*.
4. После того как основная база данных начнет передачу транзакций в зеркальную БД, возобновите резервное копирование журнала транзакций основной БД.
5. По завершении синхронизации основной и зеркальной баз данных выполните мягкое переключение сеанса зеркального отображения и подключите приложения к основной базе данных.

Возврат на основной ресурс после принудительного переключения

Принудительное переключение происходит, когда основная база данных выходит из строя, не будучи синхронизированной с зеркальной базой данных. Это приводит к потере транзакций, зафиксированных в основной базе данных. Такая ситуация возможна лишь в режиме высокой производительности.

Переключение на резервный ресурс в режиме высокой производительности происходит вручную путем выполнения следующей команды в зеркальной БД:

```
ALTER DATABASE AdventureWorks SET PARTNER FORCE_SERVICE_ALLOW_DATA_LOSS;
```

Наличие параметра *ALTER DATABASE* означает, что принудительное переключение может привести к потере транзакций. В этой ситуации возможен разрыв последовательности номеров LSN между двумя базами данных, участвующих в сеансе зеркального отображения. Если в вышедшей из строя базе данных имеются транзакции, отсутствующие в основной (бывшей зеркальной) базе данных, синхронизация вышедшей из строя базы данных будет невозможна. В этом случае необходимо удалить текущий сеанс зеркального отображения и повторно инициализировать зеркальное отражение БД. Удаляют сеанс зеркального отображения командой:

```
ALTER DATABASE AdventureWorks SET PARTNER OFF;
```


Контрольные вопросы

1. Как вернуться на основной ресурс после принудительного переключения на резервный ресурс?
2. На какие объекты сервера следует обратить внимание в первую очередь в случае переключения на резервный ресурс?

Ответы на контрольные вопросы

1. Если на момент переключения на резервный ресурс обе базы данных были синхронизированы, можно выполнить накат вышедшей из строя базы данных, используя резервные копии журнала транзакций. Затем, в ходе зеркального отображения базы данных процесс синхронизации будет завершен. Если же базы данных не были синхронизированы на момент переключения на резервный ресурс, сеанс зеркального отображения потребуется отменить и инициализировать заново.
2. На экземпляре, обслуживающем зеркальную базу данных, должны присутствовать все имена входа, необходимые для подключения приложений и пользователей к базе данных. Если в приложениях используются связанные серверы, они должны существовать, чтобы избежать сбоев в работе этих приложений. Другие объекты сервера, например пакеты и задания служб SSIS, не столь важны при переключении на резервный ресурс. Ими можно заняться вскоре после переключения, когда будет восстановлена работа всех приложений.

Практикум. Переход на резервный ресурс в сеансе зеркального отображения базы данных

Вы выполните переход на резервный ресурс в сеансе зеркального отображения базы данных, созданном в предыдущем практикуме.

1. Подключитесь к экземпляру INSTP, откройте новое окно запроса и выполните следующий код:

```
ALTER DATABASE AdventureWorks SET PARTNER FAILOVER;
```

2. Убедитесь, что база данных *AdventureWorks* на экземпляре INSTM теперь выполняет роль основной базы данных.

ПРИМЕЧАНИЕ Принудительный переход на резервный ресурс

Параметр *SET PARTNER FAILOVER* доступен, только если сеанс зеркального отображения синхронизирован, настроен в режиме высокой доступности и выполняется в основной базе данных. Чтобы принудительно перейти на резервный ресурс в режиме высокой производительности или при недоступности основной базы данных нужно выполнить команду *ALTER DATABASE AdventureWorks SET PARTNER FORCE_SERVICE_ALLOW_DATA_LOSS*.

Резюме

- При зеркальном отображении базы данных может выполняться переключение с основной базы данных на зеркальную, автоматическое присвоение зеркальной базе данных роли основной, а вышедшей из строя базе данных — роли зеркальной и в некоторых случаях даже автоматическая накопительная синхронизация. Однако эти возможности бесполезны, если на зеркальной базе данных на момент переключения отсутствуют имена входа, связанные серверы и другие необходимые объекты.
- Возврат на основной ресурс следует выполнять, только если этого требует политика компании или если после переключения отмечено ухудшение производительности.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Перечислите характеристики режима высокой производительности. (Выберите все подходящие ответы, в совокупности образующие правильный ответ.)
 - А. Асинхронная передача данных.
 - В. Синхронная передача данных.
 - С. Автоматический переход на резервный ресурс.
 - Д. Переход на резервный ресурс вручную.
2. Перечислите характеристики режима высокой безопасности. (Выберите все правильные ответы. Каждый из них представляет часть правильного ответа.)
 - А. Асинхронная передача данных.
 - В. Синхронная передача данных.
 - С. Автоматический переход на резервный ресурс.
 - Д. Переход на резервный ресурс вручную.

Закрепление материала главы

Для закрепления знаний и навыков, полученных при изучении этой главы, выполните следующее:

- изучите резюме главы;
- повторите основные термины, использованные в этой главе;
- выполните задания по темам этой главы;
- сделайте предложенные упражнения;
- пройдите пробный экзамен.

Резюме главы

- В режиме высокой доступности транзакции дополнительно фиксируются на всех серверах и поддерживается возможность автоматического перехода на резервный ресурс.

- В режиме высокой безопасности транзакции дополнительно фиксируются на всех серверах, но переход на резервный ресурс выполняется вручную.
- В режиме высокой производительности не гарантируется дополнительная фиксация транзакций и переход на резервный ресурс выполняется вручную.

Основные термины

Понятны ли вам значения приведенных здесь основных терминов? Чтобы проверить свои ответы, загляните в глоссарий на прилагаемом компакт-диске.

- Зеркальное отображение ~ database mirroring.
- Роль зеркального отображения ~ database mirroring role.
- Сеанс зеркального отображения ~ database mirroring session.
- База данных-партнер ~ database partner.
- Конечная точка ~ endpoint.
- Режим высокой доступности ~ high availability operating mode.
- Режим высокой производительности ~ high performance operating mode.
- Режим высокой безопасности ~ high safety operating mode.
- Зеркальная база данных ~ mirror.
- Переключение (переход) на зеркальную базу данных ~ mirror failover.
- Режим работы ~ operating mode.
- Основная база данных ~ principal.
- Прозрачное перенаправление клиента ~ transparent client redirect.
- Следящий сервер ~ witness server.

Упражнения

Сейчас вы примените на практике знания и навыки, полученные при изучении этой главы. Ответы на вопросы см. в разделе «Ответы» в конце книги.

Упражнение. Планирование высокой доступности

Исходные данные

О компании

Офис туристической компании Margie's Travel находится в г. Сан-Диего. Приобрести тур клиенты могут, обратившись лично к агенту в отделении компании или разместив заявку на веб-сайте.

Формулировка задачи

После добавления нового каталога продуктов веб-сайт компании стал работать нестабильно. Иногда во время внесения данных доходило до того, что клиенты в это время не могли приобретать туры на сайте.

Компания недавно отказалась от услуг консалтинговой фирмы, сотрудник которой отвечал за разработку и поддержку веб-сайта и всех остальных приложений и не смог обеспечить доступность критически важных бизнес-систем.

Планируемые изменения

Перед новым техническим директором поставлена задача обеспечения высокой доступности всех критически важных бизнес-систем. Для решения этой задачи, а также для повседневной поддержки систем технический директор нанял нового администратора баз данных и системного администратора.

Существующая среда

В компании используется 11 баз данных, которые перечислены в табл. 15-2.

Табл. 15-2. Базы данных компании Margie's Travel

База данных	Описание	Размер
<i>Orders</i>	Сведения о всех размещенных клиентами заказах	50 Гб
<i>Customers</i>	Личная информация о клиентах	15 Гб
<i>CreditCards</i>	Данных о кредитных картах клиентов	200 Мб
<i>Employees</i>	Информация о всех сотрудниках	50 Мб
<i>Human Resources</i>	Все документы отдела управления персоналом и данные о зарплате сотрудников	300 Мб
<i>Products</i>	Данные о товарах, предлагаемых для продажи на веб-сайте компании	25 Гб
<i>Flights</i>	Сведения о забронированных клиентами местах на авиарейсы	2 Гб
<i>Cruises</i>	Данные о турах, забронированных клиентами	1 Гб
<i>CarRental</i>	Данные о бронировании арендуемых клиентами автомобилей	1 Гб
<i>Excursions</i>	Данные об экскурсиях, забронированных клиентами (К экскурсиям относятся все услуги, кроме мест на авиарейсы, туров, товаров и аренды автомобилей.)	2 Гб
<i>Admin</i>	Служебная база данных для работы администратора баз данных, временно пустая	12 Гб

В среде компании имеется один веб-сервер с именем WEB1 и один сервер баз данных по имени SQL1. Все серверы работают под управлением ОС Windows Server 2003, а на SQL1 установлен SQL Server 2008.

На сервере SQL1 имеется внешняя дисковая стойка, подключенная к контроллеру RAID-массива с резервной батареей, на котором можно настроить режим RAID 0, RAID 1 и RAID 5. В настоящее время весь массив дисков настроен как единый набор RAID 0. Хранилище используется всего на 10% его мощности.

К серверам WEB1 и SQL1 подключен ленточный накопитель, который ни разу не использовался.

В настоящее время серверы SQL1 и WEB1 находятся в помещении рядом с рабочим местом уволенного консультанта. Все приложения на сервере WEB1 созданы с использованием ColdFusion или ASP.

Предлагаемая среда

Технический директор выделил часть бюджета на приобретение четырех дополнительных серверов с установленными Windows Server 2003 и SQL Server 2008. Все закупаемое оборудование должно поддерживать работу в кластере.

Данные в базах данных *Products*, *Customers*, *Orders*, *Flights*, *Cruises*, *Excursions* и *CarRental* будут доступны через Интернет посредством приложений, выполняемых на сервере WEB1. Все остальные базы данных должны быть защищены брандмауэром и доступны только для пользователей, прошедших проверку подлинности в корпоративном домене.

Для хранения баз данных развернута новая сеть SAN, имеющая достаточно дискового пространства для размещения всех баз данных. Каждый из 20 номеров логических устройств (LUN) в сети SAN сконфигурирован в виде зеркальных наборов с чередованием (*stripe of mirrors*), с четырьмя дисками в каждом зеркальном наборе.

Бизнес-требования

В компании развернуто временное решение, обеспечивающее полное восстановление системы после любых сбоев в течение двух рабочих дней при максимальной потере данных только за последний час перед сбоем. В случае крупной аварии допускается некритичная для бизнеса потеря данных, но не более чем за два дня.

Для внесения изменений в систему предусмотрено время обслуживания в промежутке от полуночи до 8:00 утра воскресенья.

Необходимо создать долгосрочное решение для защиты компании от аппаратных сбоев с максимальным временем простоя не более одной минуты.

Технические требования

Базы данных *Orders*, *Customers* и *CreditCards* связаны между собой, поэтому должны размещаться на одном экземпляре SQL Server и при сбое вместе переключаться на резервный ресурс кластера.

Нужно обеспечить максимальную безопасность всех баз данных отдела управления персоналом, а доступ к ним должен предоставляться только директору этого отдела. Все данные отдела управления персоналом должны храниться в зашифрованном виде как в базе данных, так и в любом другом месте в сети.

Для разработки новых маркетинговых кампаний и товарных предложений отделу маркетинга необходимо создавать отчеты на основе всех данных о клиентах и заказах, а также о заказанных товарах и услугах. Для проведения анализа нужные данные, получаемые практически в режиме реального времени.

Все базы данных должны быть доступны 99,92% времени в течение всего года. Для восстановления после сбоев должно требоваться минимальное вмешательство администратора. Для клиентов веб-сайта переходы при сбое должны происходить незаметно.

1. Технический директор принял решение о размещении дополнительного сервера на расстоянии 100 км от серверов, обслуживающих основные базы данных. Этот дополнительный сервер будет работать под управлением выпуска SQL Server 2008 Standard. Какая технология обеспечит выполнение бизнес-требований в случае выхода из строя основного сервера?

- A. Отказоустойчивый кластер.
 - B. Зеркальное отображение базы данных в режиме высокой безопасности.
 - C. Зеркальное отображение базы данных в режиме высокой производительности.
 - D. Репликация.
2. Изменения в каталог продуктов вносятся раз в день в запланированное для обслуживания время. Технический директор требует обеспечить дублирование каталога продуктов на дополнительном сервере. Какую технологию можно использовать в качестве решения по обеспечению доступности?
- A. Зеркальное отображение базы данных в режиме высокой производительности.
 - B. Зеркальное отображение базы данных в режиме высокой доступности.
 - C. Репликация.
 - D. Доставка журналов.
3. Какое решение позволит удовлетворить требования отдела маркетинга?
- A. Доставка журналов в режиме ожидания.
 - B. Отказоустойчивый кластер.
 - C. Репликация транзакций с поочередным обновлением подписчиков.
 - D. Репликация моментальных снимков.
4. Какая технология оптимальна для поддержки баз данных *Orders*, *Customers*, *Products* и *CreditCards*?
- A. Отказоустойчивый кластер.
 - B. Зеркальное отображение в режиме высокой доступности.
 - C. Репликация транзакций с поочередным обновлением подписчиков.
 - D. Доставка журналов.

Рекомендуемые упражнения

Чтобы подготовиться к экзамену по темам этой главы, выполните следующие задания.

Настройка зеркального отображения базы данных

Чтобы на практике ознакомиться с зеркальным отображением базы данных, создайте конечные точки и настройте зеркальное отображение с различными режимами работы. Переводя различные компоненты зеркального отображения в автономный режим, сравните состояние сеанса зеркального отображения. Выполните переключение на резервный ресурс автоматически и вручную.

- **Задание 1.** Создайте конечные точки зеркального отображения базы данных для основной и зеркальной баз данных и следящего сервера двумя различными способами: с помощью мастера настройки безопасности зеркального отображения баз данных и применяя команды языка Transact-SQL (T-SQL) *CREATE ENDPOINT* и *ALTER ENDPOINT*.

- **Задание 2.** Настройте зеркальное отображение в режиме высокой доступности для базы данных *Adventure Works*.
- **Задание 3.** Переведите следящий сервер в автономный режим и посмотрите, как изменится состояние зеркальной базы данных. Переведите в автономный режим зеркальную базу данных и посмотрите, как это отразится на основной базе данных. Верните зеркальную базу данных и следящий сервер в оперативный режим и посмотрите, как будут меняться состояния системы.
- **Задание 4.** Установите режим высокой производительности и повторите задание 3.
- **Задание 5.** Установите режим высокой безопасности и повторите задание 3.
- **Задание 6.** Выполните автоматическое переключение на резервный ресурс в режиме высокой доступности, выключив экземпляр, обслуживающий основную базу данных, когда зеркальная база данных и следящий сервер находятся в оперативном режиме.
- **Задание 7.** Выполните переключение на резервный ресурс вручную в каждом режиме работы, используя два метода: среду SSMS и команды языка T-SQL.

Создание моментального снимка зеркальной базы данных

Создайте моментальный снимок базы данных, который затем можно использовать для отчетности.

- **Задание 1.** Создайте моментальный снимок зеркальной базы данных. Удалите и повторно создайте моментальный снимок базы данных или создайте последовательность снимков базы данных, чтобы увидеть, как изменились данные и насколько быстро они менялись в зеркальной базе данных в зависимости от режима работы.

Пробный экзамен

На прилагаемом компакт-диске содержится пробный экзамен, обеспечивающий широкие возможности самоподготовки. Например, можно проверить, как усвоены темы этой главы либо все темы экзамена 70-432. Можно симитировать настоящий экзамен или выбрать режим обучения, когда после каждого вопроса приводятся правильные ответы с пояснениями.

ПРИМЕЧАНИЕ Пробный экзамен

Подробнее о пробном экзамене см. «Введение».

Доставка журналов

Занятие 1. Знакомство с доставкой журналов	442
Занятие 2. Запуск доставки журналов	446
Занятие 3. Разработка стратегии перехода на другой ресурс и возврата на исходный ресурс	457

Доставка журналов — механизм автоматического дублирования данных на резервном сервере (так называемом сервере-получателе), основанный на последовательной передаче резервных копий журнала транзакций. В этой главе вы ознакомитесь с типовой конфигурацией доставки журналов, а также с соображениями по настройке перехода на другой ресурс и возврата на исходный в среде, где применяется доставка журналов.

ПРИМЕЧАНИЕ Резервное копирование и восстановление
Подробнее о резервном копировании и восстановлении см. главу 9.

Темы экзамена

- Реализация доставки журналов.

Прежде всего

Для изучения материалов этой главы необходимо иметь:

- как минимум один компьютер с Microsoft SQL Server 2008 выпуска Standard, Enterprise или Developer;
- установленную базу данных *AdventureWorks*.

ПРИМЕЧАНИЕ
Можно реализовать доставку журналов между двумя базами данных одного экземпляра SQL Server, однако это не рекомендуется применять в производственных средах, поскольку при сбое экземпляра вы потеряете и базу данных-источник, и резервную базу данных. Доставку журналов на одном экземпляре следует использовать только для обучения и тестирования.

Занятие 1. Знакомство с доставкой журналов

Доставка журналов осуществляется на уровне баз данных и основана на передаче данных от базы-источника в базу-получатель. Кроме того, в конфигурацию доставки журналов можно включить сервер мониторинга, который будет выявлять неполадки в ходе сеанса доставки журналов и рассылать уведомления об ошибках. В занятии дается общий обзор компонентов доставки журналов и их ролей в архитектуре доставки журналов.

Изучив материал этого занятия, вы сможете:

- ✓ определять сервер-источник и сервер-получатель.

Продолжительность занятия — 25 минут.

Ситуации, в которых применяется доставка журналов

Доставка журналов находит самые разные применения в корпоративной среде.

Перенос генерации отчетов на другой сервер

Доставку журналов можно использовать для поддержки сервера отчетов. Вместо восстановления резервных копий журнала транзакций в режиме без восстановления (No Recovery Mode) можно выбрать режим ожидания (Standby Mode). В последнем случае допускается наличие подключений и выполнение инструкций *SELECT*.

Восстановление невозможно, пока есть подключения к базе данных. Чтобы время простоя было минимальным, база-получатель должна получать данные из базы-источника с очень небольшой задержкой. Такие взаимоисключающие требования не позволяют использовать режим ожидания для обеспечения высокой доступности.

Инициализация зеркального отображения баз данных

Для реализации зеркального отображения баз данных нужны резервные копии основной базы. После восстановления всех резервных копий основной базы данных и синхронизации основной и зеркальной баз данных инициирование сеанса зеркального отображения базы данных занимает считанные секунды. Весьма типична ситуация, когда доставку журналов применяют для инициализации сеанса зеркального отображения базы данных, чтобы основная и зеркальная базы данных были максимально близки к полной синхронизации и для перевода сеанса зеркального отображения базы данных в оперативный режим требовалось лишь несколько секунд. Это позволяет тратить минимум времени на инициирование зеркального отображения баз данных (после чего приложения снова могут выполнять транзакции).

Обновление версий или миграция на новую платформу

Параллельное обновление и миграция на новую платформу во многом аналогичны. В обоих случаях создают новый экземпляр сервера, а затем переносят базы данных. Если необходимо свести к минимуму время простоя приложений, можно воспользоваться доставкой журналов, последовательно восстанавливая

копии журналов транзакций на новом экземпляре. Тогда для переключения приложений на новый экземпляр требуется лишь короткий перерыв, необходимый для применения оставшихся резервных копий журналов транзакций.

Поддержка переключения между основной и резервной базами данных

Доставка журналов может использоваться как основной механизм поддержки одной или нескольких резервных баз данных, на которые приложения могут переключаться в случае отказа основной базы данных.

Но чаще бывает, что конфигурацию доставки журналов развертывают в отказоустойчивых кластерах или средах с поддержкой зеркального отображения баз данных, чтобы обеспечить дополнительный механизм автоматического перехода на другой ресурс.

ВНИМАНИЕ! Область применения доставки журналов

Область применения доставки журналов ограничена базой данных. Поэтому при доставке журналов не учитываются никакие возможные зависимости между базами данных. Например, если есть база данных о клиентах и база данных о заказах этих клиентов, целостность данных обеспечивается приложениями, транзакции которых должны учитывать эту зависимость. Однако доставка журналов нарушит эту целостность, поскольку работает с резервными копиями журнала транзакций, а SQL Server в этом случае не поддерживает транзакции, охватывающие несколько баз данных. Даже если в сеансах доставки журналов участвуют и база данных о клиентах, и база данных о заказах, а резервное копирование и восстановление выполняется в один временной интервал, после восстановления может оказаться, что существуют строки с данными о заказах, для которых нет соответствующей строки с данными о клиенте или что исчезли все данные о заказах каких-либо клиентов. Как правило, если среда не позволяет обеспечивать ссылочную целостность с помощью внешних ключей (как в случае, когда данные о клиентах и об их заказах хранятся в разных базах данных, а не в разных таблицах одной и той же базы данных), SQL Server не может гарантировать сохранность связей между данными.

Компоненты механизма доставки журналов

На рис. 16-1 показаны основные компоненты механизма доставки журналов.

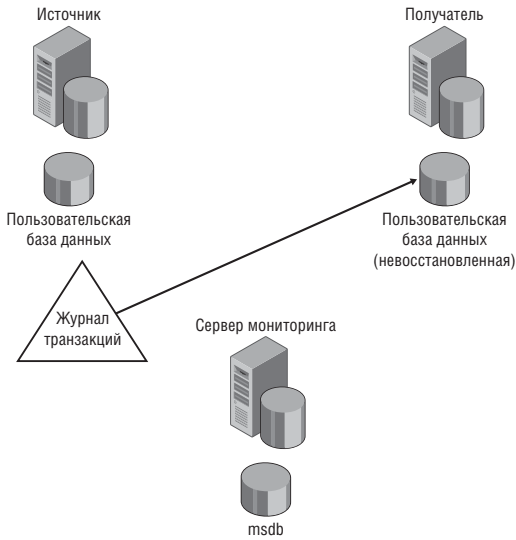


Рис. 16-1. Компоненты механизма доставки журналов

База данных-источник

База данных-источник доступна приложениям и обрабатывает транзакции. Механизм доставки журналов периодически создает резервные копии ее журнала транзакций и передает их на сервер, на котором хранится база данных-получатель.

База данных-получатель

База данных-получатель, которую еще называют *резервной базой данных* (standby), обычно недоступна. На ней постоянно восстанавливаются поступающие резервные копии журнала транзакций. База данных-получатель может функционировать в одном из двух режимов — режиме ожидания (Standby Mode) или режиме без восстановления (No Recovery Mode). Если база данных-получатель находится в режиме ожидания, пользователи могут подключаться к ней и выполнять инструкции *SELECT*. Если база данных находится в режиме без восстановления, пользователи не могут к ней подключиться. В обоих режимах поддерживается восстановление резервных копий журнала транзакций в базу данных-получатель. Поскольку восстановление из журнала транзакций невозможно, если к базе данных подключены пользователи, режим ожидания не следует использовать в архитектурах, обеспечивающих высокую доступность.

Сервер мониторинга

Это необязательный компонент архитектуры доставки журналов, на котором выполняется ряд заданий, отправляющих предупреждения, если выявлено нарушение синхронизации при доставке журналов.

ПРИМЕЧАНИЕ Терминологические соглашения

Доставка журналов выполняется между базами данных, поэтому, вообще говоря, в ней нет сервера-источника и сервера-получателя. Вполне возможно выполнить

доставку журналов одной базы данных с сервера А на сервер В, а затем — доставку журналов другой базы данных с сервера В на сервер А. Когда мы говорим об источнике или о *сервере-источнике*, мы имеем в виду, что на этом сервере работает экземпляр SQL Server, содержащий базу данных, которая в данном сеансе доставки журналов является источником. Когда мы говорим о получателе или о *сервере-получателе*, мы подразумеваем, что на нем работает экземпляр SQL Server, содержащий базу данных, которая в данном сеансе доставки журналов является получателем. Когда мы говорим о *резервном сервере* (standby), мы также подразумеваем, что на нем установлен экземпляр SQL Server, содержащий базу данных, которая в данном сеансе доставки журналов является получателем. Далее в этой главе мы для краткости будем использовать термины *источник* и *получатель/резервный*. Имейте в виду, что эти термины относятся к конкретным базам данных, а термины «сервер-источник» или «сервер-получатель» относятся к экземплярам, содержащим ту или иную базу данных.

Контрольные вопросы

1. Какие существуют два режима работы резервного сервера?
2. Какие три компонента используются при доставке журналов и что они делают?

Ответы на контрольные вопросы

1. Режим ожидания и режим без восстановления.
2. База данных-источник доступна и обрабатывает транзакции. На базе данных-получателе последовательно восстанавливаются копии журнала транзакций, что обеспечивает синхронизацию с базой данных-источником. *Сервер мониторинга* рассылает предупреждения, когда обнаруживает нарушение синхронизации в ходе сеанса доставки журналов.

Типы доставки журналов

Доставку журналов можно реализовать либо с помощью встроенных компонентов SQL Server, либо написав свой код, определяющий логику доставки. Оба подхода широко применяются на практике.

Встроенные компоненты доставки журналов представляют собой набор хранимых процедур и таблиц. Хранимые процедуры служат для настройки заданий, выполняющих всю работу по доставке журналов, и для обработки данных, хранящихся в таблицах доставки журналов. Таблицы доставки журналов — это просто хранилище данных, где находится информация о конфигурации и оперативном состоянии среды доставки журналов.

Решения на основе самостоятельно разрабатываемого кода во многом аналогичны принципам, реализованным в поставляемом Microsoft встроенном коде. Короче говоря, доставка журналов — это механизм автоматизации постоянного создания и восстановления из резервных копий журнала транзакций. Решения, основанные на собственном коде, как правило, содержат хранимые процедуры, реализующие процесс резервного копирования и восстановления. Затем вручную создают задания Агента SQL Server (SQL Server Agent), периодически

формирующие резервные копии журналов транзакций, передающие резервные копии на один или несколько серверов-получателей и выполняющие хранимые процедуры, которые восстанавливают резервные копии журналов транзакций на получателях в порядке создания резервных копий.

ПРИМЕЧАНИЕ На экзамене

Доставка журналов основана на работе с резервными копиями журнала транзакций. При изменении модели восстановления на простую доставка журналов перестанет функционировать. Аналогично, выполнение команды *BACKUP LOG ... WITH TRUNCATE_ONLY* также нарушит доставку журналов.

Резюме занятия

- Доставка журналов — это автоматическое последовательное восстановление резервных копий журнала транзакций на одном или нескольких резервных серверах.
- База данных-источник находится в оперативном режиме и обрабатывает транзакции.
- На базе данных-получателе происходит постоянное последовательное восстановление из резервных копий журналов транзакций.
- Сервер мониторинга — необязательный компонент, рассылающий предупреждения при значительной рассинхронизации базы данных-получателя и базы данных-источника.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Какие из нижеперечисленных терминов являются компонентами механизма доставки журналов?
 - A. Участник.
 - B. Источник.
 - C. Распространитель.
 - D. Резервная база данных.

Занятие 2. Запуск доставки журналов

Доставка журналов основана на постоянном применении резервных копий журнала транзакций базы данных-источника к базе данных-получателю. Перед применением таких резервных копий необходимо синхронизировать источник и получатель. В этом занятии рассказывается о порядке запуска сеанса доставки журналов.

Изучив материал этого занятия, вы сможете:

- ✓ инициализировать доставку журналов.

Продолжительность занятия — 25 минут.

Порядок запуска доставки журналов

Запуск доставки журналов — довольно очевидный процесс, не прерывающий работу базы данных-источника.

ПРИМЕЧАНИЕ Встроенные средства и собственный код

Единственное отличие между доставкой журналов, основанной на собственном коде и использованием встроенных компонентов, поставляемых в составе SQL Server, обусловлено особенностями конкретных бизнес-требований. Доставка журналов, основанная на собственном коде, как правило учитывает особые требования, обусловленные спецификой бизнес-среды. В обоих случаях применяется одинаковый подход: постоянное восстановление из копий журнала транзакций на базе данных-получателя. Чтобы упростить ознакомление с доставкой журналов, мы посвятим оставшиеся два занятия этой главы исключительно встроенным компонентам, поставляемым в составе SQL Server 2008.

Процесс запуска доставки журналов состоит из нескольких операций:

1. Создание общей папки для источника и получателя, необходимой для обмена резервными копиями между серверами.
2. Создание заданий резервного копирования журнала транзакций, копирования резервных копий на сервер-получатель и восстановления из резервных копий.
3. Восстановление полной резервной копии в базе данных-получателя.
4. Восстановление всех последующих копий журнала транзакций.
5. Запуск заданий по автоматическому восстановлению копий журнала транзакций.
6. Копирование всех объектов уровня экземпляра, необходимых базе данных-получателю для обслуживания приложений.

ВНИМАНИЕ! Агент SQL Server

Работоспособность доставки журналов зависит от заданий, выполняемых Агентом SQL Server (SQL Server Agent). Если он не включен, доставка журналов не работает. Если задания, используемые при доставке журналов, отключены или не запущены, доставка журналов также невозможна.

Создание заданий

Есть два средства создания заданий, используемых при доставке журналов: Среда Microsoft SQL Server Management Studio (SSMS) и сценарии на Transact-SQL.

ПРИМЕЧАНИЕ Использование SSMS

SSMS предоставляет единый интерфейс для настройки всех компонентов доставки журналов. Хотя интерфейс позволяет полностью настроить конфигурацию, большинство администраторов баз данных предпочитает настраивать доставку журналов, применяя сценарии на Transact-SQL. К счастью, SSMS позволяет настроить конфигурацию доставки журналов и сгенерировать сценарий для ее внедрения.

При настройке доставки журналов создают три задания:

- задание резервного копирования;
- задание копирования;
- задание восстановления.

Задание резервного копирования всегда выполняется на источнике. *Задание восстановления* всегда выполняется на получателе. *Задание копирования* может выполняться либо на источнике, либо на получателе. Обычно задание резервного копирования также выполняют на получателе.

ПРИМЕЧАНИЕ Объем данных, подвергаемых риску потери

Общее эмпирическое правило таково: риску потери подвергаются данные за период, в два раза превышающий интервал между запусками заданий резервного копирования журнала транзакций. Например, если задания резервного копирования выполняются каждые 5 минут, есть риск потерять данные за последние 10 минут. При определении этого периода учитывается время, необходимое для создания резервной копии журнала транзакций и ее копирования на получатель. В системах, обрабатывающих очень большие объемы транзакций, может показаться, что резервные копии журнала транзакций создаются почти постоянно, поскольку время создания резервной копии почти совпадает с интервалом, через который планируется создание следующей резервной копии.

Восстановление из резервных копий

Поскольку доставка журналов основана на восстановлении резервных копий журнала транзакций, прежде всего, необходимо восстановить на получателе полную копию базы данных. Без этого будет невозможно восстановить дополнительные резервные копии журнала транзакций.

При настройке доставки журналов можно сразу создать резервную копию, скопировать ее на получатель и восстановить, а затем механизм доставки журналов будет восстанавливать дополнительные резервные копии журнала транзакций.

ПРИМЕЧАНИЕ Восстановление из полной резервной копии во время настройки

Вообще говоря, не рекомендуется создавать полную резервную копию при настройке сеанса доставки журналов. Из-за этого может очень сильно замедлиться работа существующей среды, особенно если размер базы данных превышает 10 Гб. На момент настройки доставки журналов у вас уже должны быть резервные копии. (Заметьте: *всегда* необходимо иметь резервные копии своих баз данных.) Чтобы запустить доставку журналов в типичной производственной среде, как правило достаточно действий, описанных в данном разделе.

Запуск доставки журналов выполняется в следующей последовательности:

1. Скопируйте на получатель последнюю полную резервную копию и восстановите ее, оставив базу данных в режиме ожидания или режиме без восстановления.
2. Скопируйте на получатель и восстановите последнюю разностную резервную копию, оставив базу данных в режиме ожидания или режиме без восстановления.

3. Скопируйте и восстановите все резервные копии журнала транзакций, созданные после создания последней разностной резервной копии, оставив базу данных в режиме ожидания или режиме без восстановления.
4. Отключите существующее задание, чтобы создать резервные копии журнала транзакций.
5. Скопируйте на получатель и восстановите дополнительные резервные копии журнала транзакций.
6. Запустите сеанс доставки журналов.

ПРИМЕЧАНИЕ

Такая последовательность действий позволяет затратить минимум времени на инициирование доставки журналов, не ожидая, пока создастся еще одна полная резервная копия. Кроме того, затрачивается минимум дискового пространства.

Доставка журналов основана на постоянном создании резервных копий журнала транзакций, которые последовательно восстанавливаются на получателе. Нужно быть очень осторожным при настройке задания резервного копирования журнала транзакций. Задание резервного копирования, используемое при доставке журналов, замещает все существующие задания по резервному копированию журнала транзакций источника. Если на источнике более одного задания по резервному копированию журнала транзакций, доставка журналов очень быстро перестанет функционировать.

ВНИМАНИЕ! Планы обслуживания

Как правило в планах обслуживания предусматривают резервное копирование баз данных по определенному графику, поэтому администратору не приходится беспокоиться о перенастройке заданий обслуживания при удалении или добавлении баз данных. Поэтому в большинстве случаев при организации доставки журналов база данных-источник уже обслуживается планом резервного копирования журнала транзакций. А это означает, что появляется несколько графиков резервного копирования журнала транзакций: один предусмотрен планом обслуживания, а другой — заданием доставки журналов. Каждый график содержит только часть данных, которые требуется восстановить. Чтобы восстановить актуальное состояние базы данных, необходимы оба набора, поскольку каждая резервная копия журнала транзакций содержит только изменения данных, сделанные после создания последней резервной копии журнала транзакций. Перед настройкой доставки журналов следует изменить все планы обслуживания, создающие резервные копии журнала транзакций, исключив из них базу данных-источник. Создание полных и разностных резервных копий не влияет на доставку журналов.

Копирование объектов уровня экземпляра

При доставке журналов работа ведется только с содержимым базы данных. Однако одной базы данных недостаточно для того, чтобы приложения могли обращаться к данным. Чтобы приложения могли функционировать при переходе на получатель, необходимы объекты, существующие на уровне экземпляра.

Ниже перечислены основные объекты уровня экземпляра, которые необходимо скопировать на получатель:

- объекты безопасности;
- связанные серверы;
- службы Integration Services;
- конечные точки;
- объекты Агента SQL Server (SQL Server Agent);
- DDL-триггеры (Data Definition Language) уровня экземпляра;
- параметры репликации.

Если данные в базе данных зашифрованы, необходимо скопировать главный ключ источника и восстановить его на получателе.

Для шифрования данных и предоставления доступа к приложениям, а также для поддержки шифрования (в сочетании с главным ключом базы данных) могут использоваться сертификаты. Все сертификаты, используемые при работе с базой данных, необходимо создать на сервере-получателе.

Кроме того, для управления доступом к базе данных используются имена входа — имена входа SQL Server или имена входа или группы Microsoft Windows. Каждое создаваемое имя входа имеет идентификатор безопасности (SID), хранящийся в экземпляре. Для предоставления доступа к данным и объектам этот идентификатор безопасности в дальнейшем связывается с пользователем базы данных. Идентификатор безопасности, хранящийся для имени входа Windows или группы Windows — это «настоящий» идентификатор безопасности, назначаемый Windows, глобально уникальный для пользователя и передаваемый внутри инфраструктуры безопасности Windows, чтобы по имени входа предоставлялся доступ к серверу, на котором работает SQL Server. Идентификатор безопасности, связанный с именем входа SQL Server, генерируется SQL Server на локальном уровне в соответствии с правилами создания имен входа SQL Server; его глобальная уникальность не гарантируется. Хотя все имена входа, используемые для предоставления доступа к базе данных-получателю, должны копироваться на получатель, нужно иметь в виду, что сопоставление идентификаторов безопасности для имен входа SQL Server и пользователей базы данных может оказаться некорректным.

ПРИМЕЧАНИЕ На экзамене

В SSIS имеется задание, которое можно использовать для переноса имен входа между экземплярами.

ПРИМЕЧАНИЕ Повторное сопоставление пользователей

Чтобы заново корректно сопоставить пользователя базы данных имени входа SQL Server, после перевода базы данных получателя в оперативный режим выполните команду *ALTER LOGIN*. Для имен входа Windows повторное сопоставление не требуется, поскольку их идентификаторы безопасности глобально уникальны и всегда сопоставляются корректно.

У связанных серверов имеется два компонента, требующие внимания. Связанным серверам могут потребоваться ODBC-драйверы или поставщики OLE DB, не загруженные на получателе. Кроме того, может оказаться, что имя входа и пароль встроены в определение связанного сервера. Нужно установить все драйверы ODBC и поставщики OLE DB, необходимые связанному серверу.

Кроме того, требуется повторно создать связанный сервер с корректными учетными данными входа.

Пакеты служб SSIS обычно используют для загрузки или извлечения данных из базы. Все пакеты служб SSIS, зависящие от основной базы данных, необходимо повторно создать на получателе. Нужно внимательно следить, чтобы в этих пакетах не было перекрестных зависимостей. Допустим, что имеется пакет служб SSIS, извлекающий или загружающий данные в базы Database1, Database2 и Database3. Если в доставке журналов участвует только Database2, пакет служб SSIS, скопированный на получатель, может оказаться неработоспособным. В таких случаях может потребоваться модифицировать пакеты служб SSIS так, чтобы имя сервера в соединениях с SQL Server выбиралось на основании параметров.

Конечные точки служат для управления доступом к различным ресурсам экземпляра или базы данных. Все конечные точки, используемые при работе с источником, необходимо повторно создать на получателе. Чтобы ресурсы получателя не начали использоваться, для каждой из этих конечных точек следует задать STATE=STOPPED.

У Агента SQL Server также могут быть объекты, которые потребуются повторно создать на получателе. На получателе необходимо повторно создать все задания, зависящие от источника. Каждое из этих заданий требуется отключить, чтобы они не выполнялись. Если имеются категории заданий, расписания заданий, операторы или предупреждения, связанные с базой данных, нужно повторно создать каждый из этих объектов на получателе.

Триггеры DDL могут создаваться на уровне как экземпляра, так и базы данных. Триггеры DDL уровня базы данных уже есть на получателе, поскольку являются частью базы данных. А триггеры DDL уровня экземпляра придется создать на получателе повторно.

База данных-источник может участвовать в репликации как издатель или как подписчик. В конфигурации репликации напрямую указаны имена экземпляров и баз данных издателя, подписчика и распространителя. Обычно имя базы данных на получателе такое же, но имя экземпляра отличается. Все объекты репликации уже существуют в базе данных-получателе, но пока не используются. После перехода на получатель необходимо запланировать перенастройку архитектуры репликации, после того как получатель перейдет в оперативный режим.

ПРИМЕЧАНИЕ Репликация

Подробнее о репликации см. главу 17.

Все скопированные на получатель объекты уровня экземпляра необходимо обслуживать. Например, если внести изменения в пакет служб SSIS на источнике, нужно так же изменить конфигурацию получателя.

Контрольные вопросы

1. Переход на какую модель восстановления базы данных-источника нарушает доставку журналов?
2. На каком типе резервного копирования основана доставка журналов?

3. Какую операцию нельзя выполнять в базе данных-получателе, чтобы не прервать восстановление резервных копий журнала транзакций?

Ответы на контрольные вопросы

1. Изменение модели восстановления на простую (simple) прерывает последовательность копий журнала транзакций и не позволяет выполнять резервное копирование журнала транзакций.
2. Резервное копирование журнала транзакций.
3. Базу данных нельзя переводить в оперативный режим, поэтому нельзя выполнять инструкцию *RESTORE DATABASE ... WITH RECOVERY*.

Практикум. Настройка доставки журналов

Вы сконфигурируете сеанс доставки журналов между двумя экземплярами SQL Server для базы данных *Adventure Works*.

ПРИМЕЧАНИЕ Соглашение об именовании

Будем именовать экземпляр с источником INSTP, а экземпляр с получателем — INSTS.

1. Откройте Проводник Windows (Windows Explorer) на источнике и создайте общую папку LSBackup.
2. Предоставьте учетной записи службы SQL Server на источнике разрешение на полный доступ (Full Control) к этой папке, а учетной записи службы Агента SQL Server (SQL Server Agent) на получателе — разрешение на чтение (Read).
3. Откройте Проводник Windows (Windows Explorer) на получателе и создайте общую папку LSCopy.
4. Предоставьте учетной записи службы SQL Server на источнике и учетной записи службы Агента SQL Server на получателе разрешение на полный доступ (Full Control) к этой папке.
5. Проверьте доступ, чтобы убедиться в правильности назначения разрешений.
6. Убедитесь, что для базы данных *Adventure Works* задана полная модель восстановления (Full) или модель восстановления с неполным протоколированием (Bulk-logged). Если это не так, измените модель восстановления на полную (Full).
7. Запустите SSMS, подключитесь к INSTP средствами Обозревателя объектов (Object Explorer), щелкните базу данных *Adventure Works* правой кнопкой и выберите **Свойства** (Properties). В окне **Выберите страницу** (Select a Page) выберите **Доставка журналов транзакций** (Transaction Log Shipping).
8. Установите флажок **Включить эту базу данных в качестве источника в конфигурации доставки журналов** (Enable this as a primary database in a log shipping configuration) и щелкните **Параметры копирования** (Backup Settings).
9. Введите в текстовое поле **Сетевой путь к папке резервного копирования** (Network path to backup folder) UNC-путь к общей папке, созданной в п.1.

10. Введите в текстовое поле **Если папка резервного копирования находится на сервере-источнике** (If The Backup Folder Is Located On The Primary Server) физический путь к каталогу в котором будут храниться резервные копии.
11. Задайте интервал предупреждений — 6 минут и интервал резервного копирования — 2 минуты (рис. 16-2).

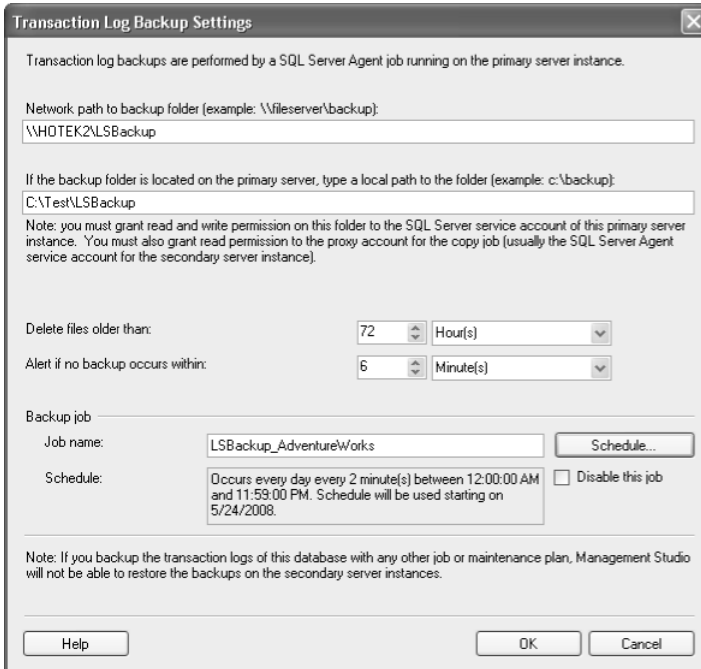


Рис. 16-2. Настройка параметров резервного копирования

12. Щелчком **ОК** закройте диалоговое окно **Параметры резервного копирования журналов транзакций** (Transaction Log Backup Settings).
13. Щелкните **Добавить** (Add), чтобы добавить получателя.
14. Щелкните **Соединить** (Connect) и подключитесь к INSTS; оставьте имя базы данных-получателя AdventureWorks.
15. Механизм доставки журналов должен создавать полную резервную копию, поэтому убедитесь, что выбран вариант **Да, создать полную резервную копию базы данных-источника и выполнить восстановление из нее в базу данных-получатель** (Yes, Generate A Full Backup Of The Primary Database And Restore It Into The Secondary Database), как показано на рис. 16-3.

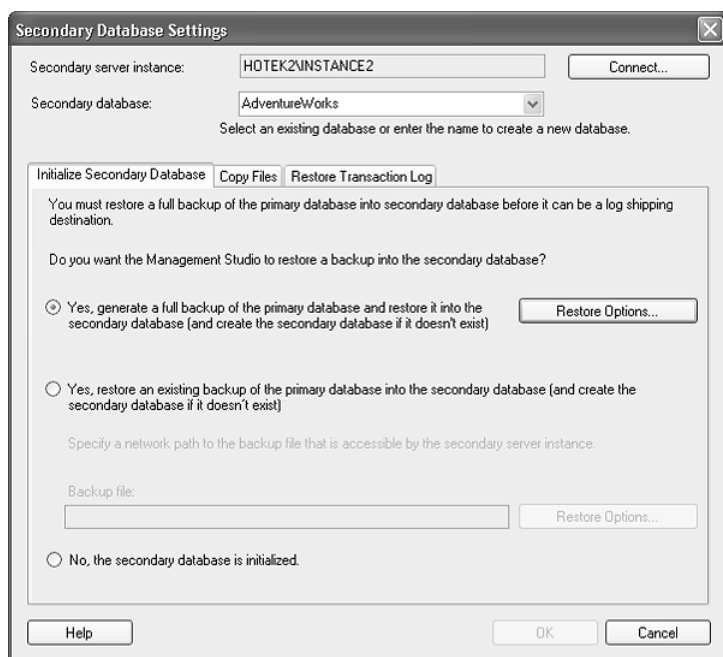


Рис. 16-3. Настройка получателя

16. Щелкните **Параметры восстановления** (Restore Options) и введите путь к каталогу, в котором нужно разместить файлы данных и журнала (рис. 16-4). Щелкните **ОК**.

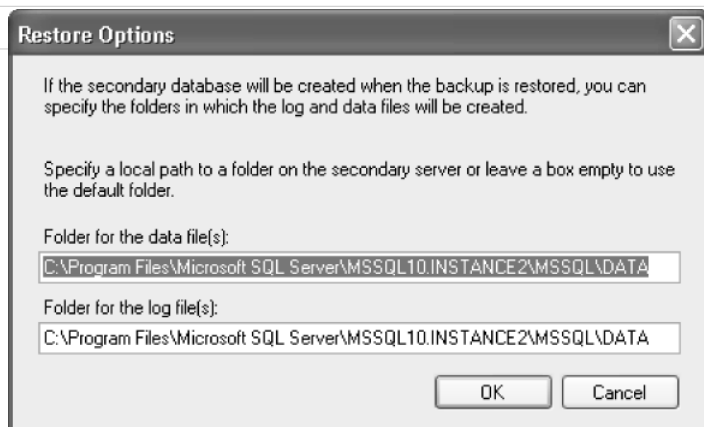


Рис. 16-4. Настройка местонахождения базы данных на получателе

17. Щелкните вкладку **Копирование файлов** (Copy Files).
18. Задайте целевую папку, в которую будут копироваться резервные копии журнала транзакций, и определите интервал копирования — 2 минуты (рис. 16-5).

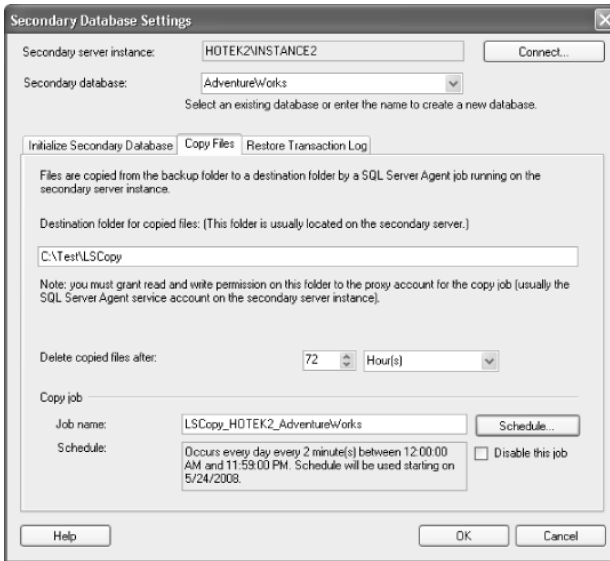


Рис. 16-5. Настройка задания копирования

19. Щелкните вкладку **Восстановление журнала транзакций** (Restore Transaction Log).
20. Убедитесь, что выбран **Режим без восстановления** (No Recovery Mode), установите задержку — 0 минут, интервал предупреждений — 6 минут и запланируйте восстановление каждые 2 минуты (рис. 17-6). Щелчком **ОК** закройте диалоговое окно **Настройки базы данных-получателя** (Secondary Database Settings).

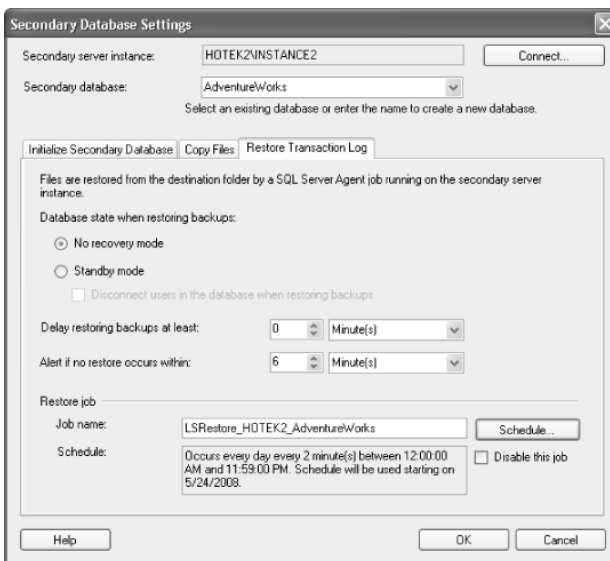


Рис. 16-6. Настройка задания восстановления

ПРИМЕЧАНИЕ Сервер мониторинга

Настройка сервера мониторинга будет дополнительным заданием, которое вы выполните в конце этой главы.

21. Щелкните **Сценарий конфигурации** (Script Configuration) и выберите вариант **Вывести конфигурацию в новое окно запроса** (Script Configuration To New Query Window). SSMS сгенерирует файл сценария настройки доставки журналов, который откроется в новом окне запроса.
 22. При необходимости вернитесь в диалоговое окно **Свойства базы данных** (Database Properties), в котором вы настраиваете доставку журналов.
-

ВНИМАНИЕ! Файловая группа FILESTREAM

База данных *AdventureWorks* содержит файловую группу *FILESTREAM*. Перед восстановлением базы данных *AdventureWorks* на экземпляре-получателе убедитесь, что группа *FILESTREAM* активна.

23. Щелкните **ОК**, чтобы сгенерировать конфигурацию доставки журналов, создать резервную копию базы данных *AdventureWorks* на INSTP, восстановить *AdventureWorks* на INSTS, создать задания и предупреждения доставки журналов и запустить доставку журналов.
 24. Убедитесь, что резервные копии попадают в нужную папку, корректно копируются с INSTP на INSTS и восстанавливаются на INSTS.
 25. Изучите сгенерированный сценарий, чтобы увидеть, какие операции выполняются при настройке доставки журналов в графическом пользовательском интерфейсе SSMS.
-

ПРИМЕЧАНИЕ Настройка интервалов

Интервалы выполнения заданий резервного копирования и копирования определяют, сколько данных можно потерять при сбое сервера-источника. Например, если настроить систему так, чтобы резервное копирование выполнялось каждые 15 минут, а копирование резервных копий — каждые 45 минут, то при сбое сервера-источника будут потеряны резервные копии за один последний час: резервное копирование может завершиться сразу же после завершения задания копирования, что означает, что задание копирования не получило данные о транзакциях за последние 15 минут и не будет их копировать в течение еще 45 минут.

Минимально допустимый интервал резервного копирования при доставке журналов — 2 минуты. Как правило, следует настраивать задания резервного копирования, копирования и восстановления так, чтобы они выполнялись с одним интервалом. Настройте рассылку предупреждений, если резервное копирование или восстановление не выполнилось в течение времени, в три раза превышающего интервал запуска задания резервного копирования или восстановления.

Резюме занятия

- Чтобы база данных могла участвовать в доставке журналов, в ней должна быть настроена полная модель восстановления или модель восстановления с неполным протоколированием.

- Чтобы инициировать доставку журналов, необходимо восстановить полную резервную копию. Затем на получателе постоянно восстанавливают резервные копии журнала транзакций в порядке их поступления с отправителя.
- Последующие операции создания полных и разностных резервных копий источника не влияют на доставку журналов.
- Все объекты уровня экземпляра, такие как конечные точки, имена входа, сертификаты и пакеты служб SSIS, необходимо вручную скопировать с источника на получатель; все изменения этих объектов на источнике необходимо вручную дублировать на получателе.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Какие из нижеперечисленных объектов уровня экземпляра должны существовать на получателе, чтобы можно было обращаться к базе данных после ее перехода в оперативный режим?
 - A. Главный ключ базы данных.
 - B. Имена входа.
 - C. Задания Агента SQL Server (SQL Server Agent).
 - D. Триггеры DDL.

Занятие 3. Разработка стратегии перехода на другой ресурс и возврата на исходный ресурс

Поскольку доставка журналов работает на уровне баз данных, на получателе нужно предусмотреть все объекты уровня экземпляра, необходимые для того, чтобы база данных-получатель могла обслуживать приложения после перехода на нее. В этом занятии рассказывается, как проектировать переход на другой ресурс и готовить к нему сеанс доставки журналов, а также о ситуациях, в которых требуется вернуться на исходный ресурс, и операциях, которые необходимо для этого выполнить.

Изучив материал этого занятия, вы сможете:

- ✓ переключать роли серверов;
- ✓ разрабатывать стратегию перехода приложений на другой ресурс;
- ✓ разрабатывать стратегию переключения клиентских приложений на резервную базу данных.

Продолжительность занятия — 45 минут.

Практический совет

Майк Хотек

Примерно два года назад я консультировал крупную компанию, занимающуюся розничной торговлей. Ее сеть складов охватывала всю территорию США, а головной офис располагался в верхней части Среднего Запада. На каждом складе имелся локальный сервер, который обслуживал операторов вилочных погрузчиков, готовящих отгрузку товаров в магазины сети в соответствии с заказами. Простой такого сервера создал бы массу неприятностей, так как SQL Server управлял всеми операциями склада. Оператор погрузчика принимал данные о заказе очередного товара, размещенном на складе, через наушники. Он получал информацию о ряде и стеллаже, на котором находится товар, а также о том, сколько товара требуется положить на грузовой поддон. Оператор подтверждал выполнение заказа и запрашивал следующий маршрут с помощью голосовых команд. В случае остановки системы операторы не смогли бы узнать, какие товары заказаны, и отгрузка товаров в магазины прекратилась бы. Такие простои легко могли превратиться в миллионные убытки для компании.

Чтобы предотвратить простои систем, было принято решение реализовать доставку журналов. Все базы данных каждого склада копировались в офис корпорации, в котором находился резервный сервер. Доставка журналов могла бы снизить риск простоев складов, но архитекторы не учли ряд критически важных моментов.

Склады подключались к офису корпорации по коммутируемым каналам пропускной способностью 56 Кб/с. Коммутируемые соединения достаточно хорошо подходили для периодического приема заказов и отправки подтверждений их выполнения, но в случае, когда оператор обслуживал заказ и на сервер, находящийся в 2000 миль от склада, направлялись соответствующие запросы, производительность падала до неприемлемого уровня.

Резервное копирование журнала транзакций выполнялось каждые 5 минут. Поскольку на сильно загруженных складах выполнялось множество операций отгрузки, некоторые резервные копии могли иметь размер 500 Мб и более. Потребовалось бы несколько минут на то, чтобы передать их по коммутируемому каналу, и очень часто накапливались копии журнала транзакций за два или три часа.

Поскольку базы данных активно изменялись, этот клиент каждый день выполнял переиндексацию каждой из четырех баз данных каждого склада. Наличие даже одного сервера, который вечером доставляет копии журнала транзакций объемом 10 Гб и более, было проблемой. А у клиента было 47 складов, отправлявших копии своих журналов транзакций на единственный резервный сервер в головном офисе.

Вскоре стало естественным начинать дневную смену в 7:00 (когда данные доставляемых журналов отставали на 3–4 часа). Задержка быстро увеличивалась до 12 – 14 часов и только после этого начинала снижаться. К моменту, когда задержка уменьшалась до менее 1 часа, начинало выполняться ночное задание переиндексации и все повторялось заново.

Техническое решение, не учитывающее, в какой бизнес-среде оно будет применяться на практике, почти наверняка обречено на неудачу!

До появления SQL Server 2008 количество изменяемых данных крайне сильно влияло на задержку доставки журналов. В SQL Server 2008 стало можно сжимать резервные копии, чтобы сократить задержку, являющуюся самым медленным звеном доставки журналов — сетью.

Переход на другой ресурс

В доставке журналов не предусмотрен механизм ни автоматического обнаружения сбоя на источнике, ни автоматического перехода на другой ресурс. Обнаружение сбоя и переход на другой ресурс выполняются вручную.

Хотя процесс не автоматизирован, он довольно прост. В случае сбоя необходимо восстановить базу данных-получатель и подключить к ней приложения.

В целом, процесс перехода на получатель выполняется так:

1. Восстановите все имеющиеся копии журнала транзакций, чтобы привести получатель в как можно более актуальное состояние.
2. Восстановите последнюю копию журнала транзакций с параметром *WITH RECOVERY*.
3. Выполните инструкцию *ALTER LOGIN* для каждого имени входа SQL Server, чтобы повторно сопоставить пользователей базы данных и имена входа, обеспечив их корректное соответствие.
4. При необходимости запустите все конечные точки.
5. Проверьте разрешения безопасности.
6. Измените строки соединения приложений, чтобы они указывали на получатель.
7. Запустите необходимые задания, например задание резервного копирования базы данных.

При переходе на другой ресурс прежде всего необходимо свести к минимуму потерю данных. Если имеются журналы транзакций, которые еще не были применены на получателе в процессе доставки журналов, необходимо их восстановить. При восстановлении последнего журнала транзакций следует указать параметр *WITH RECOVERY*. Если все журналы транзакций уже восстановлены в процессе доставки журналов, необходимо выполнить команду *RESTORE DATABASE ... WITH RECOVERY*. Эта команда переведет базу данных в оперативный режим, после чего восстановление других резервных копий журнала транзакций станет невозможным.

После перевода базы данных получателя в оперативный режим следует проверить работоспособность всех объектов, необходимых для функционирования приложений. Следует выполнить инструкцию *ALTER LOGIN* для каждого имени входа SQL Server, чтобы обеспечить корректное сопоставление имен входа и пользователей базы данных-получателя. Кроме того, следует проверить все конечные точки, связанные серверы и другие объекты, необходимые приложениям. После проверки, что все объекты существуют, что разрешения корректны и база данных доступна, нужно подключить приложения к базе данных, чтобы пользователи могли продолжить работу.

Оставшиеся операции по администрированию следует выполнять только после возобновления работы пользователей. Удалите все элементы конфигурации доставки журналов (как правило, это задания копирования и восстановления). Кроме того, следует создать задание, выполняющее резервное копирование базы данных.

База данных, которая была получателем, теперь стала источником, обрабатывает транзакции, и ее надо защитить как можно быстрее. Если вы скопировали все необходимые объекты уровня экземпляра на получатель и как можно тщательнее проверили каждый объект, время перехода приложений с источника на получатель составляет 1–2 минуты.

После восстановления работоспособности первоначального источника и его перевода в оперативный режим необходимо заново активизировать доставку журналов, понизив ранг первоначального источника до получателя. В результате снова будет обеспечиваться дублирование данных, причем для этого не придется останавливать приложения.

Возврат на исходный ресурс

После перехода на получатель база данных-получатель начинает отличаться от первоначального источника. После того, как база данных-получатель начинает обрабатывать транзакции, она становится источником и приложения начинают использовать ее данные. После восстановления работоспособности сервера, на котором произошел отказ, и его перевода в оперативный режим следует отвести этому серверу роль резервного сервера.

Есть только две причины возврата на первоначальный источник:

- руководство требует, чтобы приложения работали с определенным сервером;
- производительность или отказоустойчивость снижается, когда приложения используют получатель.

Ничего хорошего из жестких инструкций руководства использовать только определенный экземпляр SQL Server ожидать не следует. Решения такого рода — исключительная прерогатива технических специалистов. Если приложения работают и обслуживают бизнес-задачи, руководство не должно интересоваться деталями технической реализации.

Иногда бывает невозможно выделить резервный сервер с мощностью, сопоставимой с главным сервером. Если у резервного сервера меньше ресурсов, производительность приложений может снизиться. В таком случае следует как можно скорее переключить приложения на сервер-источник, на котором произошел сбой. Поскольку транзакции выполняются в базе данных, находящейся на резервном сервере, вы должны передать текущее состояние базы данных обратно на первоначальный источник. Чтобы это сделать, лишь на короткое время прервав работу приложений, выполните следующее:

1. Повторно инициализируйте источник, восстановив резервную копию базы данных-получателя в режиме без восстановления.
2. Восстановите на источнике самую свежую разностную копию базы данных и резервные копии журнала транзакций, также указав во всех случаях режим без восстановления.

3. Скопируйте обратно на источник все объекты уровня экземпляра, созданные или измененные на резервном сервере.
4. Повторяйте п.2 до тех пор, пока не будете готовы переключить приложения обратно на базу данных-источник.
5. Остановите задание резервного копирования журнала транзакций на источнике.
6. Отсоедините все приложения от резервного сервера и запретите доступ к нему, отключив имена входа командой:

```
ALTER LOGIN <имя_входа> DISABLE;
```
7. Создайте последнюю резервную копию журнала транзакций резервной базы данных.
8. Восстановите последнюю резервную копию журнала транзакций на источнике с параметром *WITH RECOVERY*.
9. Перенастройте приложения на подключение к источнику.
10. Удалите конфигурацию доставки журналов в резервной базе данных.
11. Повторно создайте конфигурацию доставки журналов на источнике и заново инициализируйте резервную базу данных.
12. Сделайте активными имена входа на резервном сервере.
13. Первые четыре операции этого процесса можно выполнить автоматически, настроив доставку журналов из резервной базы данных в основную. Механизм доставки журналов повторно инициализирует отправитель и будет последовательно восстанавливать копии журнала транзакций до тех пор, пока вы не будете готовы сделать короткий перерыв в работе приложений.

ПРИМЕЧАНИЕ Возврат на основной ресурс с резервного сервера

Всегда, когда это возможно, следует следить за тем, чтобы производительность и отказоустойчивость не снижались при переходе на получатель. Тогда не будет никаких технических причин для возврата на первоначальный источник. Если возвращаться на первоначальный источник не нужно, после возврата потерпевшего сбой сервера в оперативный режим можно настроить доставку журналов в обратном направлении. Поскольку возвращаться на исходный ресурс не требуется, в работе приложений не будет дополнительных перерывов.

Контрольные вопросы

1. Какой параметр восстановления указывают для перевода базы данных в оперативный режим?
2. По каким двум причинам может потребоваться снова остановить приложения и вернуться на первоначальный источник после восстановления его работоспособности и перевода в оперативный режим?

Ответы на контрольные вопросы

1. При наличии параметра *WITH RECOVERY* база данных переходит в оперативный режим и становится доступной для соединения.

2. Единственные причины, по которым может понадобиться вернуться на первоначальный источник, — требование руководства вернуться на первоначальный источник и снижение производительности приложений при использовании сервера-получателя.

Практикум. Переход на другой ресурс при использовании доставки журналов

Вы перейдете на базу данных-получатель, сконфигурированную в предыдущем упражнении.

1. Откройте Диспетчер конфигурации SQL Server (SQL Server Configuration Manager) и остановите экземпляр службы SQL Server INSTP, чтобы симулировать сбой источника.
2. Средствами Обозревателя объектов (Object Explorer) в SSMS подключитесь к INSTS.
3. Откройте журнал ошибок SQL Server и убедитесь, что последняя резервная копия базы данных *AdventureWorks* восстановлена.
4. Откройте новое окно запроса и измените контекст базы данных на *master*.
5. Если имеются резервные копии журнала транзакций, которые еще не применены к базе данных *AdventureWorks*, восстановите их. Это необходимо, чтобы состояние базы данных стало как можно более актуальным.
6. После восстановления всех резервных копий выполните команду:
`RESTORE DATABASE AdventureWorks WITH RECOVERY;`
7. Убедитесь, что база данных находится в оперативном режиме и доступна.
8. Отключите задание копирования (Copy job) и задание восстановления (Restore job), заданные в конфигурации доставки журналов.
9. Переключитесь в Диспетчер конфигурации SQL Server (SQL Server Configuration Manager) и перезапустите INSTP.
10. После перезапуска INSTP подключитесь к INSTS средствами Обозревателя объектов (Object Explorer) SSMS.
11. Повторите операции, выполненные в занятии 2, чтобы перенастроить доставку журналов, изменив ее направление: с INSTS на INSTP.

Резюме занятия

- При использовании доставки журналов выявление сбоя и переход на другой ресурс осуществляются вручную.
- После восстановления базы данных-получателя приложения могут переключиться на нее и возобновить работу.
- Для восстановления дублирования данных после устранения сбоя сервера-источника лучше всего понизить его ранг до сервера-получателя и изменить направление доставки журналов на противоположное.
- Если требуется вернуться к работе с первоначальным источником, придется снова останавливать приложения.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Какую команду выполняют, чтобы база данных-получатель восстановилась и начала обрабатывать транзакции?
 - A. ALTER DATABASE
 - B. RESTORE DATABASE ... WITH RECOVERY
 - C. RESTORE DATABASE ... WITH STANDBY
 - D. EXECUTE ALTER LOGIN

Закрепление материала главы

Для закрепления знаний и навыков, полученных при изучении этой главы, выполните следующее:

- изучите резюме главы;
- повторите основные термины, использованные в этой главе;
- выполните задания по темам этой главы;
- сделайте предложенные упражнения;
- пройдите пробный экзамен.

Резюме главы

- Доставка журналов основана на постоянном создании резервных копий журнала транзакций базы данных-получателя и их последовательном восстановлении в базе данных-получателе.
- Доставка журналов — технология «горячего» резервирования. Администратор должен вручную выявлять сбой источника и осуществлять переход на получатель.
- Доставка журналов работает на уровне базы данных, и не обеспечивает создание объектов уровня экземпляра, которые могут понадобиться приложению, например имена входа, конечные точки и связанные серверы. Кроме того, не гарантируется целостность данных, хранящихся в нескольких взаимосвязанных базах.

Основные термины

Понятны ли вам значения приведенных здесь основных терминов? Чтобы проверить свои ответы, загляните в глоссарий на прилагаемом компакт-диске.

- Задание резервного копирования ~ backup job.
- Задание копирования ~ copy job.
- Сеанс доставки журналов ~ log shipping session.
- Экземпляр мониторинга ~ monitor instance.
- База данных-источник ~ primary database.

- Сервер-источник ~ primary server.
- Задание восстановления ~ restore job.
- База данных-получатель ~ secondary database.
- Сервер-получатель ~ secondary server.
- Резервная база данных ~ standby database.
- Резервный сервер ~ standby server.
- Последовательность резервных копий журнала транзакций ~ transaction log chain.

Упражнения

Сейчас вы примените на практике знания и навыки, полученные при изучении этой главы. Ответы на вопросы см. в разделе «Ответы» в конце книги.

Упражнение 1. Планирование среды с высокой доступностью

Исходные данные

О компании

Офис туристической компании Margie’s Travel находится в г. Сан-Диего. Приобрести тур клиенты могут, обратившись лично к агенту в отделении компании или разместив заявку на веб-сайте.

Формулировка задачи

После добавления нового каталога продуктов веб-сайт компании стал работать нестабильно. Иногда во время внесения данных доходило до того, что клиенты в это время не могли приобретать туры на сайте.

Компания недавно отказалась от услуг консалтинговой фирмы, сотрудник которой отвечал за разработку и поддержку веб-сайта и всех остальных приложений и не смог обеспечить доступность критически важных бизнес-систем.

Планируемые изменения

Перед новым техническим директором поставлена задача обеспечения высокой доступности всех критически важных бизнес-систем. Для решения этой задачи, а также для повседневной поддержки систем технический директор нанял нового администратора баз данных и системного администратора.

Существующая среда

В компании используется 11 баз данных, которые перечислены в табл. 16-1.

Табл. 16-1. Базы данных компании Margie’s Travel

База данных	Описание	Размер
Orders	Сведения о заказах, полученных от клиентов	50 Гб
Customers	Личная информация о клиентах	15 Гб
CreditCards	Данных о кредитных картах клиентов	200 Мб

Табл. 16-1. (окончание)

База данных	Описание	Размер
<i>Employees</i>	Информация о всех сотрудниках	50 Мб
<i>Human Resources</i>	Все документы отдела кадров и данные о зарплате сотрудников	300 Мб
<i>Products</i>	Данные о товарах, предлагаемых для продажи на веб-сайте компании	25 Гб
<i>Flights</i>	Сведения о билетах на рейсы, забронированных клиентами	2 Гб
<i>Cruises</i>	Данные о путевках, забронированных клиентами	1 Гб
<i>CarRental</i>	Данные об аренде автомобилей клиентами	1 Гб
<i>Excursions</i>	Данные об экскурсиях, забронированных клиентами (К экскурсиям относятся все услуги, кроме полетов, путевок, товаров и аренды автомобилей.)	2 Гб
<i>Admin</i>	Служебная база данных для работы администратора баз данных, временно пустая	12 Гб

В среде компании имеется один веб-сервер с именем WEB1 и один сервер баз данных с именем SQL1. Все серверы работают под управлением ОС Windows Server 2003, а на сервере SQL1 установлено ПО SQL Server 2008.

На сервере SQL1 имеется внешняя дисковая стойка, подключенная к контроллеру RAID-массива с резервной батареей, которым можно настроить для работы в режиме RAID 0, RAID 1 или RAID 5. В настоящее время весь массив дисков настроен как единый набор RAID 0. Хранилище используется всего на 10% его мощности.

К серверам WEB1 и SQL1 подключен ленточный накопитель, который ни разу не использовался.

В настоящее время серверы SQL1 и WEB1 находятся в помещении рядом с рабочим местом уволенного консультанта. Все приложения на сервере WEB1 написаны в использовании ColdFusion или ASP.

Предлагаемая среда

Технический директор выделил часть бюджета на приобретение четырех дополнительных серверов с установленными Windows Server 2003 и SQL Server 2008. Все закупаемое оборудование должно поддерживать работу в кластере.

Данные в базах данных *Products*, *Customers*, *Orders*, *Flights*, *Cruises*, *Excursions* и *CarRental* будут доступны через Интернет посредством приложений, выполняемых на сервере WEB1. Все остальные базы данных должны быть защищены брандмауэром и доступны только для пользователей, прошедших проверку подлинности в корпоративном домене.

Для хранения баз данных развернута новая сеть SAN, имеющая достаточно дискового пространства для размещения всех баз данных. Каждый из 20 номеров логических устройств (LUN) в сети SAN сконфигурирован в виде зеркальных наборов с чередованием (*stripe of mirrors*), с четырьмя дисками в каждом зеркальном наборе.

Чтобы ускорить выполнение операций, в Margie’s Travel собираются объединить базы данных (табл. 16-2).

Табл. 16-2. Планируемая структура баз данных Margie’s Travel

База данных	Исходные базы данных	Размер
Bookings	Orders, Flights, Cruises, Excursions, CarRental	55 Гб
Customers	Customers, CreditCards	15 Гб
Employees		50 Мб
HumanResources		300 Мб
Products		25 Гб
Admin		12 Гб

Бизнес-требования

В компании развернуто временное решение, обеспечивающее полное восстановление системы после любых сбоев в течение двух рабочих дней при максимальной потере данных только за последний час перед сбоем. В случае крупной аварии компания может выдержать потерю данных не более чем за два дня.

Для внесения изменений в систему предусмотрено время обслуживания в промежутке от полуночи до 8:00 утра воскресенья.

Необходимо создать долгосрочное решение для защиты компании от аппаратных сбоев с максимальным временем простоя не более одной минуты.

Технические требования

Базы данных *Orders* и *Customers* связаны между собой, поэтому должны размещаться на одном экземпляре SQL Server и вместе переходить на другой ресурс.

Для базы данных *Products* должна использоваться модель восстановления с неполным протоколированием, чтобы можно было каждую ночь загружать в нее данные о продуктах и исследовательские статьи.

Нужно обеспечить максимальную безопасность всех баз данных отдела кадров, а доступ к ним должен предоставляться только директору отдела кадров. Любые данные из отдела кадров должны быть зашифрованы при хранении как в базе данных, так и в любом другом месте в сети.

Для разработки новых маркетинговых кампаний и товарных предложений отделу маркетинга необходимо создавать отчеты на основе всех данных о клиентах и заказах, а также о заказанных товарах и услугах. Для проведения анализа нужны данные, получаемые почти в режиме реального времени. Изменения производственной среды не должны влиять на операции построения отчетов.

Все базы данных должны быть доступны 99,92% времени в течение всего года. Для восстановления после сбоев должно требоваться минимальное вмешательство администратора. Для клиентов, использующих веб-сайт, переходы при сбое должны происходить незаметно.

1. Какую технологию обеспечения высокой доступности применить, чтобы обеспечить доступность баз данных *Customers* и *Bookings* даже при сбое

сервера-источника? Предполагается, что все диски исправны и транзакции обрабатываются корректно.

- A. Кластеризация с поддержкой перехода на другой ресурс.
 - B. Доставка журналов.
 - C. Репликация.
 - D. Зеркальное отображение баз данных.
2. Разработан новый набор отчетов о клиентах и заказах для руководства. Эти отчеты более статичны и могут отражать состояние производственной системы с запаздыванием до 6 часов, но не более. Какую технологию применить, чтобы выполнить это требование?
- A. Зеркальное отображение баз данных.
 - B. Зеркальное отображение баз данных в сочетании с моментальными снимками баз данных.
 - C. Доставка журналов.
 - D. Репликация.
3. Необходимо, чтобы база данных *Products* поддерживала полное резервирование, позволяющее восстановить данные после крупномасштабной аварии, охватывающей несколько филиалов компании. Какую технологию обеспечения высокой доступности задействовать для базы данных *Products*?
- A. Зеркальное отображение баз данных.
 - B. Кластеризация с поддержкой перехода на другой ресурс.
 - C. Репликация.
 - D. Доставка журналов.
4. Технический директор решил, что в базе данных *Products* будет использоваться доставка журналов. Как запретить пользователям с правами роли sysadmin изменять модель восстановления данных на простую?
- A. Отобрать права sysadmin.
 - B. Создать триггер DDL уровня базы данных.
 - C. Создать триггер DDL уровня экземпляра.
 - D. Рассылать уведомления о событиях.
5. Какие объекты необходимо передать на сервер-получатель, чтобы можно было использовать резервную базу данных *Customers*?
- A. Главный ключ базы данных.
 - B. Главный ключ экземпляра.
 - C. Сертификат.
 - D. Задания Агента SQL Server (SQL Server Agent).
6. Какие механизмы задействовать, чтобы гарантировать безопасность данных кредитных карт? (Выберите все подходящие механизмы.)
- A. Создавать резервные копии в шифрующей файловой системе.
 - B. Выполнять команду *BACKUP* с предложением *PASSWORD*.
 - C. Хранить резервную копию главного ключа базы данных в закрытом помещении, отличном от помещения, в котором хранятся резервные копии.

- D. Хранить резервную копию главного ключа базы данных в каталоге, отличном от каталога, в котором хранятся резервные копии базы данных.
7. При обращении к базе данных *Bookings* используется несколько имен входа SQL Server. База данных *Bookings* дублируется на сервере-получателе. Какую команду надо выполнить после того как когда база данных-получатель станет доступной, но до того как пользователи начнут к ней подключаться, чтобы гарантировать корректность разрешений каждому имени входа на доступ к базе данных *Bookings*?
- A. *sp_resolve_logins*
 - B. *ALTER LOGIN...*
 - C. *GRANT...*
 - D. *sp_change_users_login*
8. Технический директор хочет обеспечить многоуровневое дублирование баз данных *Customers*, *Bookings* и *Products*. Основной механизм перехода на другой ресурс должен позволять перевести все базы данных в оперативный режим за очень короткое время, сохранив целостность баз данных *Customers* и *Bookings*. При полном отказе основного решения должно вступать в действие резервное решение; в этом случае допускается потеря данных за последние 30 минут, а также нарушение целостности связей между базами данных *Customers* и *Bookings*. Какое сочетание технологий применить для выполнения этих бизнес-требований?
- A. Основная БД — кластеризация с поддержкой перехода на другой ресурс; резервная БД — зеркальное отображение баз данных.
 - B. Основная БД — кластеризация с поддержкой перехода на другой ресурс; резервная БД — репликация.
 - C. Основная БД — доставка журналов; резервная БД — кластеризация с поддержкой перехода на другой ресурс.
 - D. Основная БД — кластеризация с поддержкой перехода на другой ресурс; резервная БД — доставка журналов.
 - E. Основная БД — зеркальное отображение баз данных; резервная БД — доставка журналов.

Рекомендуемые упражнения

Чтобы подготовиться к экзамену по темам этой главы, выполните следующие задания.

Запуск доставки журналов

Поэкспериментируйте с различными элементами настройки архитектуры доставки журналов.

- **Задание 1.** Добавьте сервер мониторинга в конфигурацию, созданную при выполнении практикума занятия 2.
- **Задание 2.** Измените имя базы данных на сервере-получателе.

- **Задание 3.** Измените структуру каталогов на получателе. Добавьте в базу данных *AdventureWorks* файловую группу и файл. Убедитесь, что восстановление на получателе потерпело неудачу. Выполните команду *RESTORE LOG ... WITH MOVE ...* вручную, чтобы применить копию журнала транзакций, содержащую новые файловую группу и файл.
- **Задание 4.** Создайте на источнике два имени входа SQL Server и предоставьте им доступ к базе данных *AdventureWorks*. Дайте этим именам входа разные права доступа. Повторно создайте эти два имени входа на получателе в обратном порядке.
- **Задание 5.** Создайте связанные серверы, пакеты служб SSIS, объекты Агента SQL Server и другие объекты и поупражняйтесь в их переносе на получатель.

Переход на другой ресурс и возврат на исходный ресурс при использовании доставки журналов

- **Задание 1.** После выполнения задания 4 из предыдущего упражнения перейдите на базу данных-получатель и войдите под каждым из имен, созданных в этом упражнении. Убедитесь, что их права поменялись местами (то есть, что первое имя для входа имеет права доступа к базе данных, которые должны были бы быть у второго имени, а второе имя для входа имеет права доступа к базе данных, которые должны были бы быть у первого). Приведите права доступа в корректное состояние.
- **Задание 2.** После выполнения задания 5 предыдущего упражнения перейдите на получатель и убедитесь, что объекты уровня экземпляра функционируют, как ожидается.
- **Задание 3.** Переведите первоначальный источник в оперативный режим и сконфигурируйте его заново как сервер-получатель.
- **Задание 4.** Вернитесь на первоначальный источник, применив два метода, описанные в занятии 3.

Пробный экзамен

На прилагаемом компакт-диске содержится пробный экзамен, обеспечивающий широкие возможности самоподготовки. Например, можно проверить, как усвоены темы этой главы либо все темы экзамена 70-432. Можно симитировать настоящий экзамен или выбрать режим обучения, когда после каждого вопроса приводятся правильные ответы с пояснениями.

ПРИМЕЧАНИЕ Пробный экзамен

Подробнее о пробном экзамене см. «Введение».

Репликация

Занятие 1. Общие сведения о репликации	470
Занятие 2. Репликация транзакций	482
Занятие 3. Репликация слиянием	493

Основная задача репликации — распространение данных из главной базы данных в одну или несколько баз данных-получателей. Поскольку репликация обеспечивает дублирование данных, репликация может использоваться для поддержки доступности баз данных для приложений.

Эта глава содержит основные сведения о механизме репликации, а также о доступных вариантах распространения данных. Мы также объясним, как механизмы распределения данных можно использовать для создания решения доступности с исключительно низкой задержкой, которое также позволяет сократить время простоя в случае перехода при сбое.

Темы экзамена

- Реализация репликации.

Прежде всего

Для изучения материалов этой главы необходимо иметь:

- установленный экземпляр Microsoft SQL Server 2008;
- подключенную базу данных AdventureWorks;
- две подключенных базы данных (AWTransactional и AWMerge), которые являются копиями базы данных AdventureWorks.

ПРИМЕЧАНИЕ Учетные записи служб

Из соображений безопасности механизм репликации использует именованные учетные записи. Поэтому учетные записи служб, под которыми работают SQL Server и Агент SQL Server, должны быть либо локальными, либо доменными. В качестве учетной записи этих служб нельзя использовать localsystem.

Занятие 1. Общие сведения о репликации

Репликация — это механизм распространения данных. Базовый принцип репликации простой: изменения одной базы данных распространяются среди

одной или нескольких баз данных-получателей. Архитектура основного механизма репликации рассчитана на очень гибкую реализацию, вместе эта архитектура может использоваться для обеспечения доступности базы данных за счет наличия избыточной копии данных, синхронизируемой с главной базой.

На этом занятии рассказывается о возможностях настройки различных компонентов репликации, а также о базовой архитектуре механизма репликации.

Изучив материал этого занятия, вы сможете:

- ✓ спроектировать топологию репликации для архивации данных;
- ✓ определить издателя публикации;
- ✓ определить подписчика публикации.

Продолжительность занятия — 90 минут.

Компоненты репликации

При определении подлежащих репликации данных задают три основных компонента.

Статья

Статья (article) — основной блок репликации, представляющий наиболее детализированный уровень распространения данных. Статью можно задавать на основе таблицы, представления, хранимой процедуры или функции.

Наиболее подходит для обеспечения высокой доступности статья, определенная на основе таблицы. Статья определяет набор данных в таблице, которую SQL Server реплицирует в одну или несколько баз данных.

Публикации

Публикация (publication) — наиболее детализированный уровень в архитектуре репликации. Публикация — это совокупность статей, определяющих набор репликации.

Фильтры

Уникальность репликации в сравнении с другими технологиями высокой доступности заключается в ее способности дублировать только часть базы данных. К статье можно применить один или более фильтров, ограничив таким образом набор данных, подлежащих репликации.

Статьи можно фильтровать по строкам или по столбцам.

ПРИМЕЧАНИЕ

Данные, перемещаемые в процессе репликации, называются публикацией, или набором данных. Они всегда определяются в контексте подмножества данных, выбранных для репликации на основе статей и с учетом фильтров строк или столбцов данных. Эта уникальная способность репликации позволяет сделать высокодоступной не всю, а лишь часть базы данных.

Фильтр столбцов позволяет отобрать подмножество столбцов таблицы. Фильтр столбца позволяет исключить из репликации часть данных, например конфиденциальную информацию.

Фильтр строк ограничивает набор реплицируемых строк. Существуют три вида фильтров строк:

- **Статический фильтр строк** определяется при создании статьи и ограничивает статью тем же подмножеством данных независимо от подписчика. Вот пример статического фильтра:

```
WHERE State = 'TX'
```

- **Динамический фильтр строк** доступен только в репликации слиянием и не ограничен определенной статьей. В процессе синхронизации фильтр рассчитывается на основе информации подписчика, что позволяет с помощью одной публикации распространять различные наборы данных разным подписчикам. Вот пример динамического фильтра:

```
WHERE UserName = suser_sname()
```

- **Фильтр соединения** доступен только в репликации слиянием и позволяет определять порядок фильтрации таблицы в зависимости от ее отношения к родительской таблице. Допустим, у нас есть таблица с данными клиентов и другая — с заказами и детальной информацией о них. Если в таблице клиентов есть фильтр, отбирающий набор данных по состоянию, разумно так же отфильтровывать заказы и информацию о них. Однако столбца состояния нет ни в одной из этих таблиц. Используя фильтр соединения, можно фильтровать клиентов на основе состояния, а затем отфильтровывать заказы и их подробности на основе выбранного подмножества клиентов.

ПРИМЕЧАНИЕ На экзамене

Хотя в процессе репликации можно применять фильтры к статьям, в архитектуре высокой доступности эта возможность не используется. Основная задача архитектуры высокой доступности — поддержка полной и непротиворечивой копии данных конкретного экземпляра SQL Server.

Роли репликации

В процессе репликации база данных, и, соответственно, экземпляр, на котором она располагается, может играть одну из трех ролей:

- **Издатель** (publisher) хранит главную копию данных. В базе данных издателя конфигурируется публикация, определяющая набор реплицируемых данных.
- **Подписчик** (subscriber) — база данных, получающая в рамках механизма репликации изменения, определенные в публикации, на которую подписчик подписан. Подписчик может получать изменения нескольких публикаций.
- **Распространитель** (distributor) — основной механизм в рамках архитектуры репликации. База данных распространения размещается в экземпляре, сконфигурированном как распространитель. В любой архитектуре репликации агенты репликации по умолчанию работают на распространителе.

Экземпляр SQL Server может быть сконфигурирован как распространитель. База данных может конфигурироваться как издатель, подписчик, или и тот, и другой одновременно.

Топология репликации

Топология репликации определяет схему, описывающую, как перемещаются данные в рамках архитектуры репликации.

Топология с центральным издателем

Такая топология состоит из одного издателя и одного или более подписчиков. Центральный издатель хранит главную копию данных и поддерживает архитектуру репликации. В этой топологии изменения данных обычно происходят на издателе и распространяются вниз к подписчикам (рис. 17-1). Топология с центральным издателем — самая популярная среди всех возможных топологий.

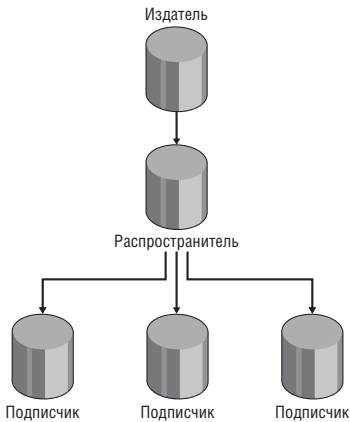


Рис. 17-1. Топология репликации с центральным издателем

Топология с центральным подписчиком

Эта топология состоит из одного подписчика, подписанного на несколько издателей. Изменения издателей объединяются на одном подписчике (рис. 17-2). Топология с центральным подписчиком обычно используется для объединения данных многих баз или для создания централизованной базы данных для создания отчетов.

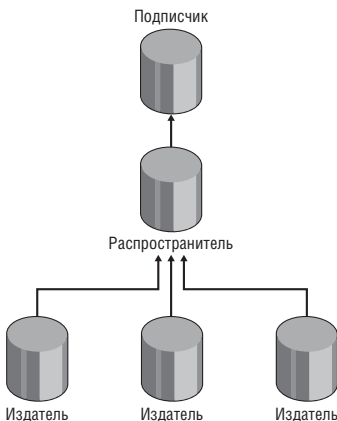


Рис. 17-2. Топология репликации с центральным подписчиком

Другие топологии

В разных источниках описано много разных топологий репликации, но все они представляют собой всего лишь комбинацию топологий с центральным издателем или подписчиком. Один из таких вариантов — «двунаправленная топология», которая есть не что иное, как объединение двух топологий с центральным издателем. По сути это не топология, а архитектурная реализация репликации транзакций. Две другие «топологии» включают центрального издателя с удаленным распространителем и центрального подписчика с удаленным распространителем. Это все равно топологии с центральным издателем или центральным подписчиком, соответственно; местоположение распространителя не имеет значения — это всего лишь детали физической реализации, которые не имеют отношения к топологии.

Агенты репликации

Многих начинающих пользователей репликации смущает то, что механизм репликации реагирует на различные сбои. В конце концов, SQL Server «не понимает» как отменять транзакцию по таймауту или повторять попытку выполнения операции.

Важно понять, что репликация никак не связана с ядром SQL Server и не является его частью. Репликация работает параллельно с SQL Server и реализована как набор исполняемых файлов, которые называются Агенты репликации (Replication Agent), то есть механизм репликации — просто еще одно приложение, подключающееся к SQL Server и обрабатывающее данные. Поэтому механизм репликации подключается и реагирует точно так же, как любое другое приложение, создающее подключение OLE DB к SQL Server.

Агент моментальных снимков

Агент моментальных снимков — это исполняемый файл snapshot.exe. Этот агент отвечает за получение схемы и данных, которые нужно переслать от издателя подписчику. Snapshot.exe используется в репликации моментальных снимков, транзакций и в репликации слиянием.

Агент чтения журнала

Агент чтения журнала, logread.exe, используется только в репликации транзакций и получает подлежащие репликации зафиксированные транзакции из журнала транзакций. Получив зафиксированные транзакции, Агент чтения журнала обеспечивает, чтобы все транзакции были повторно упакованы и записаны в базу данных распространителя в точно той же последовательности, в какой транзакции создавались на издателе. Обеспечение порядка транзакций Агентом чтения журнала критически важно для того, чтобы транзакции не применялись к подписчику в произвольном порядке.

Агент распространителя

Агент распространителя, distrib.exe, используется совместно с репликацией моментальных снимков и транзакций. У него две функции: применение моментальных снимков и отправка транзакций. Агент распространителя отвечает за применение всех моментальных снимков, созданных репликацией момен-

тальных снимков или репликацией транзакций, на всех подписчиках. Он также отвечает за применение всех транзакций, записанных в базу данных распространения Агентом чтения журнала, на всех подписчиках.

Агент слияния

Агент слияния, `herlmerg.exe`, используется в репликации слиянием. Агент слияния применяет снимок, созданный при инициализации подписчика. Агент слияния также отвечает за обмен транзакциями между издателем и подписчиком.

Агент чтения очереди

Агент чтения очереди, `qdrsvc.exe`, используется только, когда включены обновляемые посредством очередей подписки для репликации транзакций и моментальных снимков. Агент чтения очереди отвечает за передачу очереди от подписчика к издателю.

Профили агентов

У каждого агента репликации много параметров конфигурации, которые определяют его поведение. Двенадцать самых общих параметров объединяются в единый модуль, называемый профилем агента. Вот некоторые из этих параметров, которые можно настраивать:

- **Интервал опроса** (*polling interval*) определяет, как часто агент проверяет наличие новых транзакций, подлежащих репликации. Значение по умолчанию — 5 секунд.
- **Время ожидания запроса** (*query timeout*) определяет, сколько времени агент ожидает завершения запроса. Значение по умолчанию — 1800 секунд.
- **Время ожидания входа в систему** (*Login timeout*) определяет, как долго агент ожидает создания подключения. Значение по умолчанию — 15 секунд.

Методы репликации

Существуют три метода репликации: репликация моментальных снимков, репликация транзакций и репликация слиянием.

Репликация моментальных снимков

При репликации моментальных снимков в каждом цикле механизма репликации пересылается полный набор данных. На подписчике применяется полная копия данных. Все транзакции, выполненные на издателе, записываются и пересылаются на подписчик только при следующем выполнении моментального снимка.

В репликации моментальных снимков используются Агент моментальных снимков и Агент распространителя. При инициализации создания моментального снимка Агент моментальных снимков извлекает схему и, используя программу массового копирования, получает данные из папки моментальных снимков.

ПРИМЕЧАНИЕ Папка моментальных снимков

Папка моментальных снимков — место в каталоге, определяемое в процессе настройки репликации и используемое по умолчанию для хранения моментальных снимков. При создании публикации можно переопределить папку моментальных снимков только для этой публикации.

После извлечения схемы и всех данных Агент моментальных снимков завершает работу. Далее начинает работу Агент распространителя, который собирает и применяет снимок в каждом подписчике. В этом процессе существующие таблицы уничтожаются и воссоздаются из сценариев схемы, хранящихся в папке моментальных снимков; затем данные копируются в таблицы на каждом подписчике, используя программу массового копирования.

ПРИМЕЧАНИЕ Применение моментальных снимков

По умолчанию моментальный снимок применяет на подписчике структуру таблицы, первичный ключ, кластеризованные индексы, уникальные ограничения и данные. Все другие связанные с таблицей объекты, такие как проверочные ограничения и ограничения внешних ключей, не пересылаются. Можно переопределить поведение моментальных снимков по умолчанию, изменяя свойства статьи.

Существуют четыре варианта применения моментальных снимков. Вариант по умолчанию — уничтожить существующий объект и воссоздать его. Можно также оставить существующую таблицу без изменений, удалить данные, совпадающие с моментальным снимком или оставить структуру таблицы без изменений, но урезать ее, чтобы в нее можно было добавить только данные моментального снимка.

В этой главе предполагается, что все свойства статьи заданы в конфигурации по умолчанию.

Блок-схема процесса переноса данных при репликации моментальных снимков показана на рис. 17-3.

При репликации моментальных снимков выполняется полная замена данных на подписчике. Этот механизм обычно не используется для обеспечения высокой доступности, потому что все транзакции, выполненные между процедурами применения моментальных снимков, на подписчик не попадают.

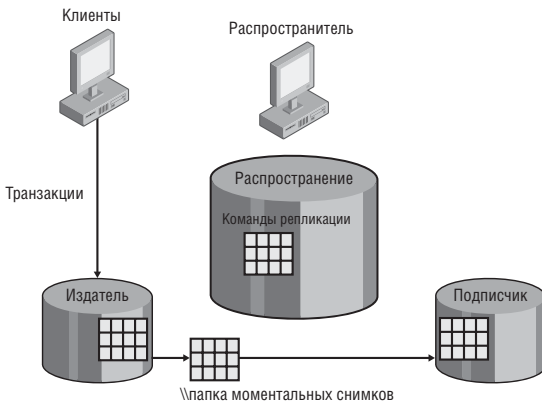


Рис. 17-3. Репликация моментальных снимков

Репликация транзакций

Репликация транзакций, или транзакционная репликация, начинается с применения на подписчике начального моментального снимка, чтобы обеспечить синхронизацию двух баз данных. По мере выполнения последующих транзакций на издатель механизме репликации выполняет их на подписчике. За счет инкрементального потока транзакций от издателя к подписчику репликация транзакций отлич-

но подходит для поддержки вторичной копии базы данных, используемой для обеспечения высокой доступности или поддержки генерации отчетов. Наиболее популярная конфигурация для репликации транзакций — «сервер-сервер».

В репликации транзакций возможны два дополнительных режима: немедленное обновление подписчиков или обновление через очередь — это позволяет выполнять транзакции на подписчике. Помимо пересылки транзакций с издателя на подписчик репликацию транзакций можно также реализовать в двух альтернативных архитектурах: двунаправленная и одноранговая репликация транзакций.

Репликация слиянием

Репликация слиянием предназначена прежде всего для обслуживания мобильных пользователей, непостоянно подключенных к корпоративной сети. В этом методе репликации связь между издателем и подписчиком не обязана быть постоянной, да этого и не требуется.

Точно так же, как в репликации транзакций, на подписчике применяется начальный моментальный снимок, чтобы обеспечить синхронизацию, а затем последующие изменения пересылаются на подписчик. В отличие от репликации транзакций репликация слиянием по умолчанию разрешает изменения как на издателе, так и на подписчике. В течение каждого цикла агента механизм слияния выполняет обмен всеми изменениями между издателем и подписчиком.

ПРИМЕЧАНИЕ Цикл агента

Репликацию можно настроить на работу в непрерывном режиме или по расписанию. В последнем случае Агент репликации активизируется с заданной периодичностью. Будучи сконфигурированным на работу в непрерывном режиме, механизм репликации работает постоянно. В любом случае цикл механизма репликации одинаков: выявление подлежащих репликации изменений, перемещение изменений на подписчик и подтверждение получения изменений. Этот процесс называется *циклом агента репликации*. В режиме по расписанию цикл агента более очевиден, потому что агент запускается, выполняет некоторую работу и затем отключается. В непрерывном режиме цикл менее очевиден, потому что агент никогда не отключается, а сразу по завершении цикла начинает новый цикл. Цикл агента — очень важное понятие, которое позволяет понять происходящее в очень многих ситуациях, наиболее важная из которых — конфликт данных.

Конфликты данных

Конфликты данных возможны в любой среде, где разрешена распределенная обработка транзакций. Когда изменения в один и тот же набор данных могут вноситься во многих местах, должен быть механизм, обеспечивающий обработку изменений.

В приложениях, вносящих изменения в единственную базу данных, уже есть метод обработки противоречивых изменений — новые изменения записываются поверх любых предыдущих изменений или отклоняются, а пользователю сообщается, что с момента получения данных из базы они изменились. Наличие этих процессов на прикладном уровне никак не помогает в распределенной среде, потому что приложения всегда работают с локальной копией данных.

Конфликт изменений происходит только, когда механизм репликации пытается синхронизировать все изменения.

Конфликты данных случаются только между циклами агента репликации, поэтому их число сводится к минимуму. По завершении цикла агента репликации могут применяться стандартные механизмы обнаружения конфликтов в приложениях, потому весь набор данных будет считаться локальным по отношению к приложению.

Конфликты данных происходят только при репликации слиянием, транзакционной репликации с подписчиками, обновляемыми посредством очередей и при двунаправленной и одноранговой репликацией транзакций, потому что в этих случаях изменения могут вноситься как на издателя, так и на подписчика.

Типы конфликтов

Возможны три типа конфликтов:

- **Вставка дублирующего первичного ключа** происходит, когда два пользователя создают одинаковый ключ на издателя и на подписчике.
- **Конфликтующее обновление** возникает, когда два пользователя изменяют одну и ту же строку на издателя и на подписчике.
- **Обновление несуществующей строки** случается, когда один пользователь изменяет строку в своей локальной копии, а второй пользователь в своей копии удаляет ту же строку.

Арбитры конфликтов

Механизм репликации обязан поддерживать единую непротиворечивую копию на издателя и подписчике. Конфликты данных создают серьезную угрозу непротиворечивости, поэтому в репликации слиянием и в транзакционной репликации с применением очередей предусмотрен механизм обнаружения и разрешения конфликтов. Обнаружение и разрешение в совокупности называется разрешением конфликтов.

Компонент, выполняющий разрешение конфликтов называется *арбитром конфликтов* (conflict resolver). В составе SQL Server есть несколько встроенных арбитров конфликтов. Два наиболее популярных используются в репликации слиянием и репликации транзакций с использованием очередей подписчиков:

- издатель всегда прав;
- подписчик всегда прав.

Если сконфигурировать разрешение конфликтов так, чтобы издатель всегда был прав, выполненные на издателя изменения переопределяют изменения на подписчике. В этом случае издатель отбрасывает поступившее от подписчика изменение и вносит его в таблицу конфликтов, а изменение на издателя отправляется подписчику. Это заставляет подписчика записать полученное изменение взамен собственного.

Если разрешение конфликтов настроено так, что прав подписчик, изменение, сделанное на подписчике, заменяют изменение, выполненное на издателя.

В любом случае наличие такого разрешения конфликтов гарантирует, что во всей архитектуре репликации поддерживается единая непротиворечивая копия данных. Однако это создает серьезные проблемы с точки зрения бизнеса. Из-

менения и на издателя, и на подписчике — вполне полномочные транзакции, которые к тому же были зафиксированы. Вполне может случиться, что другой пользователь на основании информации одной из этих транзакций принял бизнес-решение. После этого механизм репликации при попытке обмена данных обнаруживает конфликт и перезаписывает эту информацию. С точки зрения бизнеса принятое решение неправомерно.

Конфликты данных должны обнаруживаться и разрешаться механизмом репликации, чтобы обеспечивать единственность и непротиворечивость данных во всей архитектуре репликации. Вместе с тем только проектировщик приложения может обеспечить, чтобы в распределенных средах обработки не случались конфликты данных для обеспечения незыблемости бизнес-решений. Конфликты данных должны быть аномалией.

ВНИМАНИЕ! Транзакции с минимальным протоколированием

Если база данных участвует в репликации, нужно быть чрезвычайно осторожным с простой моделью восстановления и моделью с неполным протоколированием. В этом случае могут выполняться транзакции с минимальным протоколированием, то есть такие, в которых в журнале транзакций протоколируется только выделение и освобождение страниц, триггеры такие транзакции не запускают.

Вот пять транзакций с минимальным протоколированием:

- CREATE INDEX
- TRUNCATE TABLE
- BULK INSERT
- BCP
- SELECT...INTO

С репликацией связаны только три из них — TRUNCATE TABLE, BULK INSERT и BCP, потому что они затрагивают данные в таблице. Если в базе данных применена модель восстановления и модель с неполным протоколированием и выполняется любая из этих операций, механизм репликации не заметит этих изменений, потому что транзакционная репликация использует транзакции из журнала транзакций, а репликация слиянием полагается на триггеры.

Контрольные вопросы

1. Какие существуют три метода репликации?
2. Какие пять агентов используются в репликации?
3. Какие существуют три типа конфликтов данных?
4. Какие существуют две роли баз данных в архитектуре репликации?
5. Каковы основные компоненты репликации?

Ответы на контрольные вопросы

1. Репликация моментальных снимков, транзакционная репликация и репликация слиянием.
2. Агент моментальных снимков (snapshot.exe), Агент чтения журнала (logread.exe), Агент распространителя (distrib.exe), Агент слияния (replmerg.exe) и Агент чтения очереди (qrdsvc.exe).

3. Вставка дублирующего первичного ключа, конфликтующее обновление и обновление несуществующей строки.
4. Издатель или подписчик.
5. Одна или несколько статей объединены в публикацию, которая используется в сеансе репликации. При необходимости у статей могут быть фильтры.

Практикум. Настройка публикации

Вы настроите публикацию в своем экземпляре SQL Server.

ПРИМЕЧАНИЕ Изучение репликации

В практикумах этой главы среда репликации конфигурируется так, как я всегда рекомендую делать при изучении репликации. Вы сконфигурируете издателя, подписчика и распространителя на одном экземпляре, в результате репликация будет происходить между двумя базами данных экземпляра. Это самый простой способ изучения репликации, потому что в нем отсутствуют две самых популярных причины проблем с конфигурацией репликации: безопасность и подключение.

1. Откройте **Обозреватель объектов** (Object Explorer) в среде SSMS и подключитесь к своему экземпляру сервера.
2. Щелкните правой кнопкой узел **Репликация** (Replication) и выберите **Настроить распространение** (Configure Distribution). Щелкните **Далее**.
3. Выберите первый вариант в своем экземпляре, чтобы он действовал как распространитель и щелкните **Далее**.
4. Оставьте заданную по умолчанию папку моментальных снимков и щелкните **Далее**.
5. Оставьте заданные по умолчанию имя и местоположение базы данных распространителя и щелкните **Далее** (рис. 17-4).



Рис. 17-4. Настройка базы данных распространителя

6. Убедитесь, что текущий экземпляр выбран в качестве издателя и щелкните **Далее**.

7. Убедитесь, что установлен флажок **Настройка распространения** (Configure Distribution), и щелкните **Далее**.
8. Щелкните **Готово**, чтобы включить распространение и щелкните **Заккрыть** (Close).
9. Убедитесь, что в экземпляре создана база данных distribution.

Резюме занятия

- Одна или более статей объединяются в публикацию и образуют определение набора данных, перемещаемого механизмом репликации.
- База данных может выступать в роли издателя, подписчика или и того, и другого.
- Есть три метода репликации: репликация моментальных снимков, транзакционная репликация и репликация слиянием.
- Репликацию реализуют пять агентов: агент моментальных снимков, агент чтения журнала, агент распространителя, агент слияния и агент чтения очереди.
- Когда изменения могут выполняться как на издателе, так и на подписчике, возможны конфликты данных.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. В компании Contoso Limited требуется реализовать архитектуру репликации с сервером отчетов, а также обеспечить, чтобы любые изменения на сервере отчетов попадали обратно на издатель. Какой вариант репликации лучше всего выбрать в такой ситуации?
 - A. Репликация транзакций.
 - B. Репликация моментальных снимков.
 - C. Репликация слиянием.
 - D. Одноранговая репликация.
2. В компании Contoso Limited требуется реализовать архитектуру репликации с сервером отчетов. Небольшие изменения могут выполняться на сервере отчетов, в основном это важные изменения отчетов, которые нужно пересылать обратно на издатель. Решение по созданию отчетов должно быть всегда доступно, даже если недоступен издатель, а также должна обеспечиваться оптимальная производительность. Какой вариант репликации больше всего подходит в такой ситуации?
 - A. Репликация транзакций.
 - B. Репликация транзакций с немедленным обновлением подписчиков
 - C. Репликация слиянием.
 - D. Репликация транзакций с использованием очередей подписчиков.

Занятие 2. Репликация транзакций

Репликация транзакций, или транзакционная репликация, обеспечивает прежде всего однонаправленное распространение транзакций от издателя к подписчику. Репликацию транзакций можно разворачивать в самых разнообразных конфигурациях и вариантах.

В этом занятии объясняется, в каких архитектурах можно разворачивать репликацию транзакций, а также рассказывается о некоторых внутренних операциях репликации транзакций.

Изучив материал этого занятия, вы сможете:

- ✓ спроектировать решение высокой доступности, основанное на репликации;
- ✓ выбрать соответствующий ситуации вариант репликации;
- ✓ выбирать серверы для одноранговой репликации;
- ✓ определять стратегию разрешения конфликтов данных;
- ✓ проектировать топологию репликации для архивации данных;
- ✓ выбирать публикации и статьи для публикации;
- ✓ выбирать издателя для публикации;
- ✓ выбирать подписчика для публикации.

Продолжительность занятия — 45 минут.

Отслеживание изменений

Репликация транзакций управляется двумя агентами репликации: Агентом чтения журнала и Агентом распространителя. Агент чтения журнала отвечает за перенос изменений в журнале транзакций на издателя в базу данных распространителя. Агент распространителя выполняет перенос изменений из базы данных распространителя на каждый подписчик.

Агент чтения журнала

В каждом цикле репликации Агент чтения журнала выполняет следующие операции:

1. Подключается к базе данных распространителя и получает из таблицы MSlogreader_history информацию о поколении репликации (replication watermark) и последний регистрационный номер транзакции (Log Sequence Number, LSN).
2. Подключается к журналу транзакций издателя и находит последний LSN.
3. Начинает чтение транзакций с номерами, следующими за последним LSN, вплоть до самой старой открытой транзакции.
4. Записывает эти транзакции в базу данных распространителя в таблицы MSrepl_commands и MSrepl_transactions, обеспечивая такой же порядок транзакций, в котором они были выполнены на издателе.
5. Изменяет поколение репликации в таблице MSlogreader_history.

6. Отмечает в журнале транзакций флажком «реплицировано» все успешно записанные на распространитель транзакции.
7. Записывает информацию об ошибках и историю в базу данных распространителя.

Агент распространителя

В каждом цикле репликации Агент распространителя выполняет следующие операции:

1. Подключается к базе данных распространителя и находит в таблице `MSdistribution_history` последнюю успешно записанную на подписчик транзакцию.
2. Собирает все транзакции, ожидающие применение на подписчике.
3. Объединяет транзакции в пакеты.
4. Подключается к подписчику и применяет все пакеты транзакций.
5. Обновляет в таблице `MSdistribution_history` порядковый номер последней примененной транзакции.
6. Записывает информацию об ошибках и историю в базу данных распространителя

ПРИМЕЧАНИЕ Очистка базы данных распространителя

Агент распространителя сам не очищает записи в базе данных распространителя, успешно записанные на подписчики. Эту работу делает регулярно выполняемое специальное задание (именуемое Агентом очистки), которое удаляет транзакции, отправленные всем подписчикам. Это задание отделено по соображениям производительности и позволяет Агенту распространителя пересылать транзакции подписчикам с намного большей скоростью, чем Агент чтения журнала успевает записывать транзакции в базу данных распространителя. Такая архитектура гарантирует, что Агент распространителя не станет узким местом этого процесса, даже когда ему приходится отправлять транзакции многим подписчикам одновременно.

Влияние на базу данных

Поскольку механизм репликации гарантирует, что выполненные на издателе транзакции попадают на подписчик, это создает существенную нагрузку на опубликованную базу данных, и это надо учитывать.

Если у базы простая модель восстановления, в каждой контрольной точке неактивная часть журнала удаляется.

Обычно в конце процесса создания резервной копии журнала транзакций неактивная часть журнала удаляется. Причем процесс удаления неактивной части начинается с начала журнала и продолжается до достижения самой старой открытой транзакции — на этом процесс резервного копирования завершается. Чтобы определить, какие транзакции открыты и какие зафиксированы, процесс резервного копирования считывает битовый флаг в каждой записи журнала транзакций, указывающий, зафиксирована транзакция или нет.

Механизм репликации должен гарантировать, чтобы все выполненные на издателе транзакции попадали на подписчик. Однако этому могут помешать

процесс резервного копирования журнала транзакций или процесс создания контрольных точек в простой модели восстановления. Если случится так, что SQL Server удалит неактивную часть журнала до того, как Агент чтения журнала запишет транзакции в базу данных распространителя, транзакции могут потеряться. По этой причине, когда база данных участвует в репликации транзакций, в записях журнала транзакций устанавливается второй флаг. Процесс резервного копирования журнала транзакций так же обрабатывает неактивную часть журнала, но ему не позволено удалить из журнала запись, пока у нее не установлены флаги «зафиксировано» и «реплицировано». Поэтому, если Агент чтения журнала не записывает транзакции в базу данных распространителя, журнал транзакций на издателе продолжает увеличиваться, несмотря на то, что создаются резервные копии журнала транзакций (или контрольные точки базы данных в простой модели восстановления). Это также означает, что репликация транзакций совместима с любой моделью восстановления.

В целом такой же процесс имеет место в базе данных распространителя. Транзакция не удаляется из базы данных распространителя, пока она успешно не записана на подписчик. База данных распространителя увеличивается, когда подписчик отключен или недоступен.

Варианты транзакционной репликации

В репликации транзакций доступны два варианта: обновление подписчиков с применение очередей и немедленное обновление подписчика (рис. 17-5).

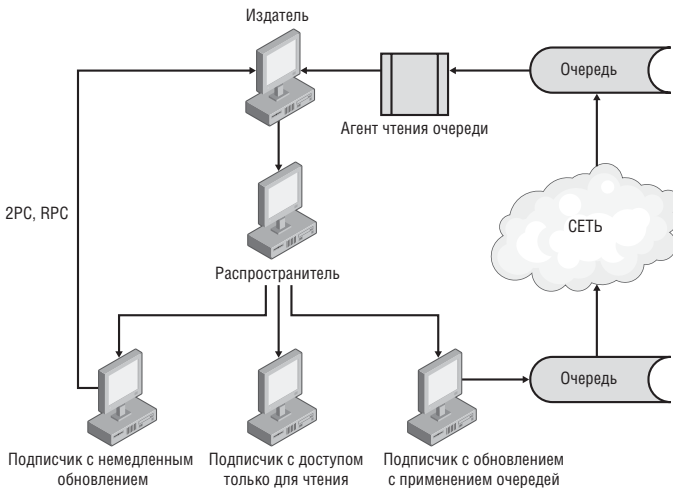


Рис. 17-5. Репликация транзакций

Немедленное обновление подписчиков

Это вариант позволяет выполнять изменения на подписчике, которые затем обратно переносятся на издатель. Далее изменение подхватывается механизмом транзакций и переправляется всем остальным подписчикам, кроме того, на котором произошло изменение. Чтобы изменение не попало на исходный подписчик, в каждой участвующей в репликации таблице должен быть столбец со штампом времени.

Процесс применения изменений на подписчике с немедленным обновлением выполняется в следующей последовательности:

1. Приложение инициирует транзакцию в подписчике.
2. Запускается триггер.
3. Триггер требует у координатора распределенных транзакций Microsoft (Microsoft Distributed Transaction Coordinator, MS DTC) подключиться к издателю и повторить на нем транзакцию.
4. Транзакция фиксируется на издателе.
5. Триггер фиксируется на подписчике.
6. Транзакция фиксируется на подписчике.

Основная проблема в архитектуре высокой доступности с немедленным обновлением подписчиков заключается в том, что изменения должны применяться на издателе. Если издатель недоступен, распределенная транзакция терпит сбой. Поскольку распределенная транзакция выполняется триггером, исходная транзакция также терпит сбой и откатывается. Поэтому репликация с немедленным обновлением подписчиков несовместима с высокой доступностью.

Обновление подписчиков посредством очередей

Этот вариант позволяет выполнить изменение на подписчике и перенести его на издатель, но при это используется совсем другой, отличный от немедленно-го обновления подписчиков механизм.

Процесс применения изменений на подписчике посредством очередей выполняется в следующей последовательности:

1. Приложение инициирует транзакцию.
2. Запускается триггер.
3. Триггер вносит транзакцию в очередь (фактически, это таблица в базе данных).
4. Фиксируется триггер.
5. Фиксируется транзакция.

Агент чтения очереди периодически переносит очередь на издатель и инициирует на нем все транзакции. Переданное на издатель изменение не применяется к подписчику, на котором оно инициировано. Для этого в таблице должен быть столбец штампа времени.

Из-за асинхронности обработки обновления, возможны конфликты данных. Для их разрешения в архитектурах высокой доступности изменения применяются к исходному подписчику только в случае сбоя. Это гарантирует, что изменения всегда выполняются в единственной копии данных, а это позволяет задействовать стандартные механизмы разрешения конфликтов в приложении.

Преимущество обновления подписчиков посредством очередей в том, что при возвращении ранее недоступного издателя в оперативный режим все изменения, произошедшие на подписчике во время недоступности издателя переносятся на издатель автоматически. Становится ненужным планирование и все процессы, обеспечивающие переход после сбоя в других архитектурах, потому что очереди являются встроенным механизмом возвращения издателя в актуальное состояние при переходе после сбоя. Единственное, что придется сделать после перехода издателя после сбоя — указать в приложениях новый издатель.

ПРИМЕЧАНИЕ На экзамене

Транзакционную репликацию можно настроить на одновременную поддержку немедленного обновления подписчиков и обновления посредством очередей. В этом случае обновления с применением очередей используется как механизм перехода при сбое на случай недоступности издателя.

Архитектуры репликации транзакций

Помимо стандартной архитектуры однонаправленного обновления репликация транзакций поддерживает две другие стандартные архитектуры. Надо не путать эти архитектуры с любым другим механизмом репликации. Эта архитектура так же реализует обычную одностороннюю репликацию транзакций без каких-либо вариантов обновления. Реализация предусматривает прямой и обратный перенос изменений между издателем и подписчиком, но это все равно транзакционный механизм.

Транзакционный механизм не обнаруживает и не разрешает конфликты данных. Поэтому если в процессе внесения изменений на издателе или подписчике возможны конфликты данных, нельзя реализовать транзакционную репликацию в двунаправленной или одноранговой архитектуре.

Одноранговая репликация

Архитектура одноранговой репликации появилась в SQL Server 2005 и доступна только в редакции Enterprise. В основе этой архитектуры лежит возможность транзакционной репликации переносить данные между двумя или больше равноправными узлами. Схема одноранговой архитектуры показана на рис. 17-6.

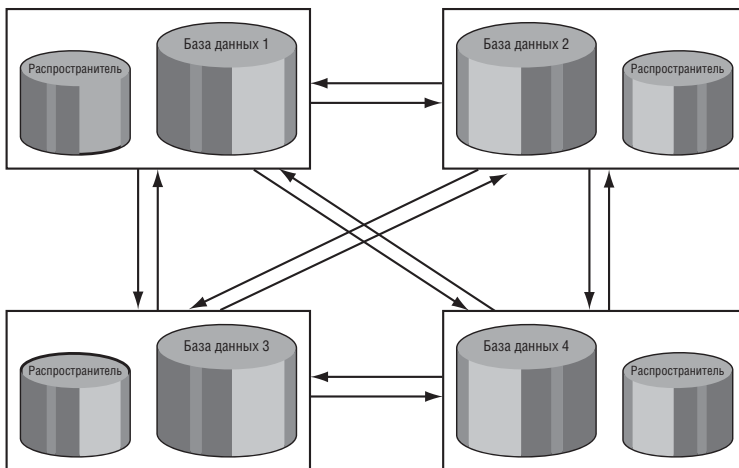


Рис. 17-6. Одноранговая репликация транзакций

Одноранговая репликация — один из вариантов реализации репликации транзакций. Основная идея в том, что берется набор таблиц и реплицируется с Базы данных 1 в Базу данных 2 по механизму репликации транзакций. Далее создается публикация того же набора таблиц в Базе данных 2 и реплицируется обратно в Базу данных 1 по механизму репликации транзакций.

В результате каждая база данных, член одноранговой архитектуры, реплицирует все изменения во все остальные базы данных. Чтобы предотвратить бесконечный цикл переноса транзакций, в каждой из таблиц, участвующих в репликации, должен быть столбец `rowguid`, который позволяет механизму репликации идентифицировать базу данных, в которой была инициирована транзакция. Одноранговая репликация предъявляет ряд требований, которые обязательны к выполнению:

- в каждом узле должен быть собственный издатель;
- структура таблицы должна быть абсолютно одинакова на всех узлах;
- режимы немедленного обновления или обновления посредством очередей недоступны;
- конфликты данных невозможны.

Двунаправленная репликация

Конфигурирование двунаправленной репликации немного отличается от настройки одноранговой. Это также архитектура репликации транзакций. Набор таблиц, реплицируемых из Базы данных 1 в Базу данных 2, тот же, что и реплицируемых из Базы данных 2 в Базу данных 1 (рис. 17-7).

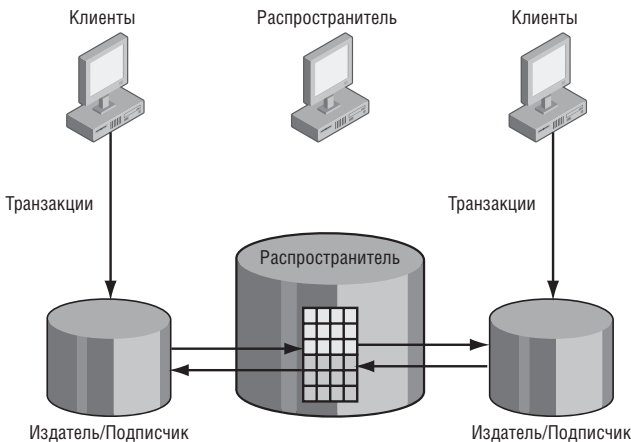


Рис. 17-7. Двунаправленная репликация транзакций

Для предотвращения бесконечного цикла обмена транзакцией между этими двумя базами данных, в каждую подписку нужно добавить параметр `@loopback_detection`.

Двунаправленную репликацию можно реализовать как подмножество одноранговой репликации, но у нее есть ряд преимуществ, связанных с производительностью. Не нужен отдельный распространитель для каждого издателя, а также не требуется добавлять в таблицы, участвующие в репликации, столбец `rowguid`. Структуры таблицы могут отличаться, хотя такой вариант мне никогда не встречался.

Конфликты данных не обрабатываются, а реализуется двунаправленная архитектура путем написания кода. Эту архитектуру нельзя реализовать, используя графический пользовательский интерфейс.

Мониторинг

Монитор репликации обычно используют для наблюдения за архитектурой репликации. Монитор репликации позволяет получить статистику работы всех публикаций, подписок и агентов. Можно также просмотреть информацию об ошибках и истории состояния, что необходимо для устранения неполадок.

Один из самых трудных моментов, связанных со средой репликации в предыдущих версиях SQL Server, были узкие места. Сравнительно просто было определить, на сколько транзакций отставал механизм репликации, выполнив хранимую процедуру `sp_browsereplcmds` в базе данных распространителя. Однако было невозможно определить, сколько механизму репликации потребуется, чтобы «догнать» обновление, потому не поддерживалась информация о длительности операций.

В процессе переноса изменений Агентом чтения журнала и Агентом распространителя механизм репликации собирает статистику по скорости переноса данных и длительности такого переноса. Используя эти статистические данные, Монитор репликации отображает сведения о том, сколько транзакций подлежат отправке подписчикам, а также сколько примерно времени на это требуется.

Хотя статистические данные в Мониторе репликации хорошо информируют о состоянии дел, позволяя узнать, насколько отстают подписчики, инструмент не предоставляет более детальных сведений. Монитор репликации отображает общую статистическую информацию о задержке, но администратор не может определить, где узкое место — в Агенте чтения журнала или Агенте распространителя. Для решения этой задачи были созданы трассировочные маркеры.

Трассировочный маркер — специализированная транзакция, инициируемая механизмом репликации. Как и любая другая транзакция, она попадает в журнал транзакций. Агенты репликации переносят трассировочный маркер в архитектуре репликации так же, как любую другую транзакцию. Трассировочный маркер отличается тем, что механизм репликации распознает эту специальную транзакцию и регистрирует временную статистику, по мере движения маркера в архитектуре репликации. Трассировочный маркер позволяет получить точную информацию о том, сколько времени ушло на перенос маркера в базу данных распространителя и сколько — на перенос на подписчик; можно также получить сводную информацию о задержке переноса от издателя к каждому из подписчиков.

Эта информация позволяет локализовать и обнаружить узкие места в архитектуре репликации.

Проверка синхронизации

Механизм репликации гарантирует, что транзакции переносятся от издателя на подписчик в том же порядке, в каком они изначально были совершены. Поскольку базы данных издателя и подписчика обычно находятся в разных экземплярах SQL Server, которые могут находиться в разных городах, всегда возникает вопрос: как узнать, синхронизированы ли эти две базы данных?

Механизм репликации предоставляет механизм проверки синхронизации. Для этого существуют две хранимые процедуры: `sp_publication_validation`

и `sp_article_validation`. Процедура `Sp_publication_validation` просто выполняет `sp_article_validation` по отношению ко всем статьям публикации.

Каждая из этих процедур может выполнять проверку, используя два разных метода:

- проверка только числа строк;
- проверка числа строк и бинарной контрольной суммы.

По умолчанию выполняется только проверка числа строк, то есть проверяется равенство числа строк на издателя и подписчика. Содержимое строк может полностью различаться; такой факт этот метод проверки обнаружить не позволит. Однако, поскольку базы данных участвуют в репликации, очень маловероятно, что таблицы с одинаковым числом строк не содержат те же данные.

Самая полная проверка включает контроль числа строк и бинарной контрольной суммы. Сравняется число строк в каждой таблице издателя и подписчика, но помимо этого рассчитывается и сравнивается бинарная контрольная сумма. Этот метод проверки позволяет обнаружить любые различия не только в числе строк, но и в их содержимом. Однако, это очень дорогая в плане ресурсов операция, которая создает дополнительную вычислительную нагрузку. По этой причине такую проверку рекомендуется делать нечасто.

ПРИМЕЧАНИЕ Рекомендации по проверке синхронизации

Проверка синхронизации может создавать значительную нагрузку, особенно в средах с большими числом подписчиков на одну публикацию или с публикациями, содержащими много статей. Рекомендуется выполнять проверку количества строк раз в день, а полную проверку и числа строк, и их содержимого — раз в неделю.

Контрольные вопросы

1. Назовите два варианта выполнения репликации транзакций.
2. Назвать две архитектуры репликации транзакций.
3. Какая ставшая недавно доступной специальная транзакция репликации транзакций используется для сбора статистики синхронизации?
4. Какие две процедуры используются для проверки синхронизации издателя и подписчика?

Ответы на контрольные вопросы

1. Немедленное обновление подписчиков и обновление посредством очередей.
2. Одноранговый и двунаправленный.
3. Трассировочные маркеры.
4. Хранимые процедуры `sp_publication_validation` и `sp_article_validation`.

Практикум. Реализация репликации транзакций

Вы сконфигурируете репликацию транзакций в базе данных `AWTransactional`.

Задание 1. Реализация транзакционной репликации

Вы создадите публикацию.

1. Откройте среду SSMS и подключитесь к экземпляру сервера, используемому для репликации.
2. Создайте базу данных по имени AWTranSubscriber на этом же экземпляре, где находится база данных AWTransactional.
3. При необходимости разверните узел **Репликация** (Replication), щелкните правой кнопкой **Локальные публикации** (Local Publications) и выберите **Создать публикацию** (New Publication). Щелкните **Далее**.
4. Выберите базу данных AWTransactional и щелкните **Далее**.
5. Выберите **Публикация транзакций** (Transactional Publication) и щелкните **Далее**.
6. Выберите все таблицы и все определяемые пользователем функции, как показано на рис. 17-8 и щелкните **Далее**.

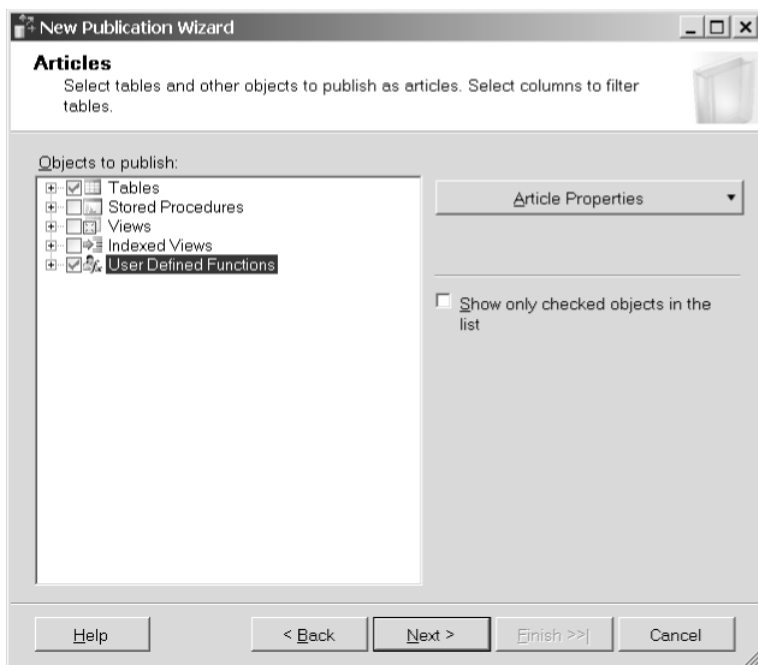


Рис. 17-8. Выбор статей

ПРИМЕЧАНИЕ Определяемые пользователем функции

Мы выбираем все определяемые пользователем функции в публикации, потому что в базе данных AdventureWorks базы есть таблицы, в которых ограничения определены как такие функции. Если их не добавить в публикацию, при применении моментального снимка произойдет сбой.

7. Нам не нужно применять никаких фильтров, поэтому щелкните **Далее**.
8. Установите флажки **Создать моментальный снимок немедленно и обеспечить доступ к нему для инициализации подписок** (Create a snapshot immediately and ensure access to it for subscription initialization).

diately and keep the snapshot available to initialize subscriptions) и щелкните **Далее**.

9. Щелкните **Настройки безопасности** (Security Settings).
10. Выберите **Выполнять с учетной записью службы «SQL Server, агент»** (Run under the SQL Server agent service account), а затем — **Путем олицетворения учетной записи процесса** (By impersonating the process account) и щелкните **ОК**.

ВНИМАНИЕ! Параметры безопасности агента

Не рекомендуется выполнять агентов репликации под учетной записью службы «Агент SQL Server». У учетной записи агента репликации должны быть минимальные разрешения в среде, а у службы агента SQL Server этих разрешений слишком много. В онлайн-справочной системе по SQL Server есть ряд очень хороших статей о безопасности репликации. В некоторых примерах в этой главе явно нарушаются рекомендации по безопасности, и на это есть серьезная причина. Эта глава призвана научить основам репликации и как она может вписаться в вашу среду. Цель этой главы познакомить с механизмом репликации до того, как вы приступите к развертыванию во многих экземплярах с разными требованиями по безопасности и заблокируете доступ. Если вы не поймете основные принципы работы механизма до начала применения защиты к агентам, вам очень трудно придется разбираться, отчего происходят ошибки: из-за особенностей ситуации, неправильной настройки или блокирующей доступ безопасности. Вы не должны выполнять агентов репликации в такой конфигурации в промышленной среде.

-
11. Щелкните **Далее**. Убедитесь, что установлен флажок **Создать публикацию** (Create the publication). Щелкните **Далее**.
 12. Задайте имя публикации и щелкните **Готово** (Finish).
 13. Когда мастер создания публикации закончит работу, щелкните **Заккрыть** (Close).

Задание 2. Создание подписки

Вы создадите подписку к публикации, созданной в предыдущем задании.

1. Разверните узел **Локальные публикации** (Local Publications), щелкните правой кнопкой только что созданную публикацию и выберите **Создать подписку** (New Subscription). В Мастере создания подписки щелкните **Далее**.
2. Убедитесь, что выбрана правильная публикация, и щелкните **Далее**.
3. Убедитесь, что установлен флажок **Выполнять все агенты на распространителе** (Run all agents at the distributor) и щелкните **Далее**.
4. Отметьте флажком свой экземпляр и выберите базу данных AWTranSubscriber в поле со списком **База данных подписки** (Subscription Database). Щелкните **Далее**.
5. На странице Безопасность агента распространителя (Distribution Agent Security) щелкните кнопку рядом с подписчиком в окне **Свойства подписки** (Subscription Properties).

6. Выберите **Выполнять с учетной записью службы «SQL Server, агент»** (Run Under The SQL Server Agent Service Account), а также вариант **Путем олицетворения учетной записи процесса** (By Impersonating The Process Account) в разделах **Соединиться с распространителем** (Connect To The Distributor) и **Соединиться с подписчиком** (Connect To The Subscriber). Щелкните **ОК**, а затем щелкните **Далее**.
7. На странице **Расписание синхронизации** (Synchronization Schedule) убедитесь, что в поле **Расписание агента** (Agent Schedule) выбрано **Выполнять постоянно** (Run Continuously) (это значение по умолчанию) и щелкните **Далее**.
8. Убедитесь, что в разделе **Свойства подписки** (Subscription Properties) установлен флажок **Инициализировать** (Initialize Immediately) и щелкните **Далее**.
9. Убедитесь, что установлен флажок **Создать подписки** (Create Subscription). Щелкните **Далее**, а затем — **Готово** (Finish).
10. По завершении создания подписки щелкните **Заккрыть** (Close).

Задание 3. Использование монитора репликации

Вы воспользуетесь Монитором репликации для просмотра информации о своей публикации и подписке и попрактикуетесь в использовании трассировочных маркеров.

1. В **Обозревателе объектов** (Object Explorer) щелкните правой кнопкой узел **Репликация** и выберите **Запустить монитор репликации** (Launch Replication Monitor).
2. В Мониторе репликации просмотрите различные вкладки и окна, чтобы увидеть ставшую доступной информацию о публикации и подписке, которые вы создали. Щелчок правой кнопкой позволяет увидеть свойства объекта или более подробную информацию.
3. Попрактикуйтесь в создании трассировочных маркеров и наблюдайте за результатами.

Резюме занятия

- В репликации транзакций используется Агент чтения журнала, который переносит транзакции от журнала транзакций на издатель в базу данных распространителя. Далее Агент распространителя переносит транзакции из базы данных распространителя на каждый подписчик.
- Репликация транзакций переносит данные в одном направлении — от издателя к подписчику. Существуют два варианта настройки репликации транзакций — немедленное обновление подписчиков и обновление посредством очередей. Эти варианты позволяют выполнять транзакции на подписчике и переносить их обратно на издатель.
- Существует три варианта архитектуры репликации транзакций. По умолчанию используется архитектура с одним издателем одним или несколькими подписчиками. Репликацию транзакций также можно сконфигурировать на использование двунаправленной или одноранговой архитектуры.

- Трассировочные маркеры позволяют администратору собирать статистику синхронизации при репликации от издателя к распространителю и от издателя к подписчику, а также наблюдать за задержкой между отдельными узлами репликации.

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Корпорация Datum Corporation предоставляет услуги по обработке данных кредитных карт, и ее системы должны обладать доступностью не ниже 99,999% (пять девяток). В то же время объем подлежащих записи данных превышает возможности текущего оборудования и оборудования, которое предполагается приобрести. Сотрудники компании выяснили, что если как-то организовать запись пакетами, одновременно перенеся операции чтения на другой сервер, то предлагаемое к приобретению оборудование обеспечило бы потребности компании в течение следующих двух-трех лет. Чтобы достичь необходимых мощностей для записи и избыточности, в компании надо развернуть 5 серверов, размещенных в разных географических точках, и еще 2 сервера, один из которых может использоваться как резервный, а второй — основной для обслуживания операций чтения. Какую технологию лучше предпочесть для удовлетворения указанных бизнес-требований?
 - A. Зеркальное отображение базы данных.
 - B. Репликация транзакций в конфигурации с одним издателем.
 - C. Репликация транзакций в одноранговой конфигурации.
 - D. Репликация транзакций с использованием очередей для обновления подписчиков.

Занятие 3. Репликация слиянием

Репликация слиянием — еще один вариант, который можно задействовать в системах высокой доступности. Репликация слиянием прежде всего предназначена для обслуживания мобильных, редко подключающихся к корпоративной сети пользователей. По определению в системе уже есть встроенные механизмы для внесения изменений в любом узле и их синхронизации, а также способность противостоять сбоям и продолжать обработку. Здесь рассказывается о внутренних механизмах репликации слиянием и о том, как их применять для повышения доступности систем.

Изучив материал этого занятия, вы сможете:

- ✓ спроектировать решение высокой доступности, основанное на репликации;
- ✓ выбрать соответствующее ситуации решение репликации;
- ✓ создать стратегию разрешения конфликтов данных.

Продолжительность занятия — 45 минут.

Отслеживание изменений

Процесс отслеживания изменений в репликации слиянием выполняется в следующей последовательности:

1. Приложение инициирует транзакцию.
2. Иницируется триггер таблицы.
3. Вставка или обновление протоколируется в таблице MSmerge_contents, а удаления — в таблице MSmerge_tombstone.
4. Фиксируется триггер.
5. Фиксируется транзакция.

В отличие от репликации транзакций репликация слиянием не перемещает копии данных в базу данных распространителя. Запись в таблицы MSmerge_contents и MSmerge_tombstone указывает только на то, что внесено изменение в определенную строку. Данные этого изменения не копируются; они хранятся только в таблице. Это создает интересную ситуацию.

Если между циклами механизма репликации по отношению к одной и той же строке инициировано 10 разных транзакций, а в таблице настроена репликация транзакций, то все 10 транзакций будут перенесены в базу данных распространителя Агентом чтения журнала, после чего все 10 транзакций последовательно будут применены к подписчику. В сухом остатке: 9 транзакций применены только для того, чтобы быть замененной десятой транзакцией в описываемой строке. С другой стороны, если в таблице настроена репликация слиянием, на подписчик будет перенесено только последнее состояние строки, после применения десятой транзакции. Это достигается за счет использования следующего процесса синхронизации изменений:

1. Механизм слияния подключается к подписчику.
2. Механизм слияния подключается к издателю.
3. Механизм слияния опрашивает таблицу MSmerge_genhistory на подписчике и получает список поколений, отсутствующих на издателе.
4. Затем механизм слияния извлекает список таблиц и строк, содержащихся в MSmerge_contents и MSmerge_tombstone, для поколений, отсутствующих на издателе.
5. Механизм слияния создает инструкции DELETE из строк, извлеченных из MSmerge_tombstone.
6. Механизм слияния извлекает содержимое строк из основных таблиц в базе данных и объединяет изменения в пакеты.
7. Упакованные изменения применяются к издателю.
8. Поколения, перенесенные на издатель записываются в MSmerge_contents.
9. Проверяется наличие конфликтов. Обнаруженные конфликты данных разрешаются.
10. Механизм слияния опрашивает таблицу MSmerge_genhistory для получения списка поколений, отсутствующих на подписчике.
11. Механизм слияния извлекает список таблиц и строк, содержащихся в MSmerge_contents и MSmerge_tombstone, для поколений, отсутствующих на подписчике.

12. Механизм слияния создает инструкции DELETE из строк, извлеченных из MSmerge_tombstone.
13. Механизм слияния получает содержимое строк из основных таблиц в базе данных и объединяет изменения в пакеты.
14. Упакованные изменения применяются на подписчике.
15. Поколения, перенесенные на подписчик протоколируются в MSmerge_contents.

Схема репликации слиянием с процессом синхронизации показана на рис. 17-9.

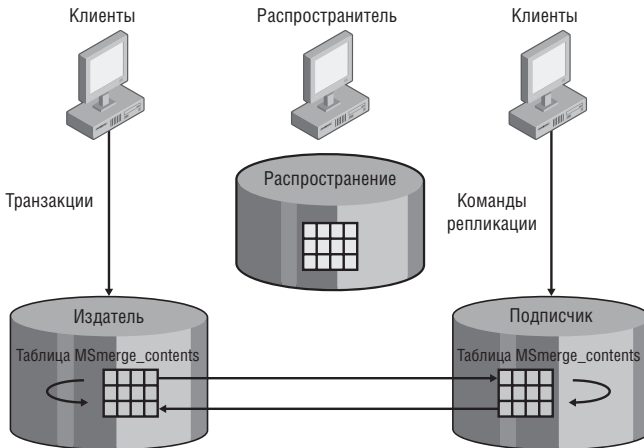


Рис. 17-9. Репликация слиянием

У процесса слияния очень простой алгоритм. При каждом запуске механизма слияния, делается один простой запрос издателя и подписчика: «пришлите мне все, что у меня еще нет». У этого, казалось бы, простого запроса очень мощный эффект.

Механизм репликации транзакций хранит изменения, которые необходимо выполнить и пересылает их на подписчик. Примененные изменения удаляются из механизма репликации. Это заставляет репликацию транзакций работать только с изменениями, уже находящимися в ее очереди, то есть механизм репликации понимает только отчет времени с настоящего момента и вперед. Механизм слияния просто не различает, когда и где было сделано изменение, — он «умеет» понимать все, что случилось в прошлом и переносит только те изменения, которых еще нет. В сущности, таблицы метаданных, лежащие в основе механизма слияния похожи на регистратор в черном ящике самолета, который содержит информацию о всех и каждом изменении, когда-либо имевшем место в базе данных, начиная с момента включения регистратора (запуска механизма слияния).

Именно это процесс делает механизм слияния таким гибким. Каждое изменение протоколируется в MSmerge_contents, MSmerge_tombstone и MSmerge_genhistory. Не имеет значение, было ли изменение применено механизмом репликации или инициировано приложением; в таблицах метаданных слияния регистрируется просто все.

Один из недостатков механизма репликации транзакций заключается в том, что изменения распределены среди трех независимых компонентов: в журнале транзакций на издатель, в базе данных распространителя и на подписчике. Поэтому создание резервных копий и восстановление состояния системы так, чтобы сохранялась функциональность репликации, чрезвычайно сложная задача.

Репликация слиянием устраняет и эту проблему. Таблицы метаданных слияния размещаются в той же базе данных, что и публикуемые статьи. Поэтому вместе с резервной копией базы данных копируются метаданные слияния, которые останутся в состоянии, синхронизированном с содержимым базы данных. В дальнейшем можно буквально восстановить базу данных из любой резервной копии с момента, когда репликация слиянием начала копировать изменения, и предоставить механизму слияния самому решать, что сделать. Независимо, где изначально произошли изменения — механизм слияния просто определить, каких поколений нет на издатель и на подписчике, и пришлет все, чего не хватает. Вам не нужно волноваться о восстановлении системы до какого-то определенного состояния — механизм слияния определит состояние самостоятельно.

В процессе обсуждения отслеживания изменений и синхронизации мы неоднократно упоминали поколения. Таблицы метаданных слияния содержат полную историю всех строк, которые были изменены с момента инициирования репликации слиянием. Опрос строки за строкой метаданных для получения изменений очень неэффективен. Для снижения нагрузки механизм слияния объединяет изменения в пакеты, которые называются поколениями и применяются на издатель или подписчике целиком или не применяются вообще. В процессе поиска изменений механизм репликации ищет поколения, которых еще нет на издатель или на подписчике.

Проверка синхронизации

Проверка публикации слиянием очень похожа на проверку в репликации транзакций. Проверка может выполняться в одном из двух режимов: проверка только числа строк и проверка числа строк и контрольной суммы. Процесс и издержки те же, что описанные ранее для репликации транзакций.

Хранимая процедура `sp_validatemergepublication` позволяет проверить правильность всей публикации. Процедура `sp_validatemergesubscription` проверяет отдельного подписчика, а при наличии параметра `-Validate` — Агента слияния.

Контрольные вопросы

1. Какие две таблицы используются механизмом репликации слиянием для выявления изменений, подлежащих синхронизации между издателем и подписчиком?
2. Какая особенность делает репликацию слиянием более устойчивой к сбоям?

Ответы на контрольные вопросы

1. Таблицы `MSmerge_contents` и `MSmerge_genhistory`.

2. Для обнаружения подлежащих репликации изменений механизм слияния использует метаданные. Поскольку и издатель, и подписчик хранят полную историю всех изменений, а метаданные хранятся в той же базе данных, что участвует в репликации, в процессе резервного копирования и восстановления метаданные хранятся в состоянии, синхронизированном с реплицируемыми данными. Это гарантирует, что после операции восстановления механизм слияния восстанавливает нормальную работу и обеспечивает синхронизацию.

Практикум. Реализация репликации слиянием

Вы настроите репликацию слиянием в базе данных AWMerge.

Задание 1. Создание публикации слиянием

Вы создадите публикацию слиянием.

1. Создайте базу данных по имени AWMergeSubscriber на том же экземпляре, на котором находится база данных AWMerge.
2. В **Обозревателе объектов** (Object Explorer) щелкните правой кнопкой узел **Репликация** (Replication) и выберите **Создать/Публикация** (New/Publication). Щелкните **Далее**.
3. Выберите базу данных AWMerge и щелкните **Далее**.
4. Выберите **Публикация слиянием** (Merge Publication) и щелкните **Далее**.
5. Убедитесь, что установлен флажок **SQL Server 2008** и щелкните **Далее**.
6. Выберите все таблицы и все определяемые пользователем функции и щелкните **Далее**.
7. Снова щелкните **Далее**. Нам не нужна фильтрация строк, поэтому щелкните **Далее** еще раз.
8. Убедитесь, что установлен флажок **Создать моментальный снимок немедленно** (Create a snapshot immediately) и сбросьте флажок **Запланировать запуск агента моментальных снимков в следующее время** (Schedule the Snapshot Agent to run at the following times). Щелкните **Далее**.
9. Задайте параметры безопасности точно так же, как это делалось при создании публикации транзакций и щелкните **Далее**.
10. Убедитесь, что установлен флажок **Создать публикацию** (Create the publication) и щелкните **Далее**.
11. Задайте имя публикации и щелкните **Готово** (Finish). По завершении создания публикации щелкните **Заккрыть** (Close).
12. Просмотрите в базе данных AWMerge все изменения триггеров, хранимых процедур, структур таблиц, индексов и представлений, которые необходимы для отслеживания изменений в репликации слиянием.

Задание 2. Создание подписки слиянием

Вы создадите подписку к публикации слиянием, созданной в предыдущем задании.

1. Разверните узел **Локальные публикации** (Local Publications), щелкните правой кнопкой только что созданную публикацию и выберите **Создать подписку** (New Subscription). В Мастере создания подписки щелкните **Далее**.
2. Выберите свою публикацию слиянием и щелкните **Далее**.
3. Убедитесь, что установлен флажок **Выполнять все агенты на распространителе** (Run all agents at the distributor) и щелкните **Далее**.
4. Отметьте флажком свой экземпляр и выберите базу данных AWMergeSubscriber в поле со списком **База данных подписки** (Subscription Database). Щелкните **Далее**.
5. На странице **Безопасность агента распространителя** (Distribution Agent Security) щелкните кнопку рядом с подписчиком в окне **Свойства подписки** (Subscription Properties).
6. Выберите **Выполнять с учетной записью службы «SQL Server, агент»** (Run Under The SQL Server Agent Service Account), а также вариант **Путем олицетворения учетной записи процесса** (By Impersonating The Process Account) в разделах **Соединиться с распространителем** (Connect To The Distributor) и **Соединиться с подписчиком** (Connect To The Subscriber). Щелкните **ОК**, а затем щелкните **Далее**.
7. На странице **Расписание синхронизации** (Synchronization Schedule) убедитесь, что в поле **Расписание агента** (Agent Schedule) выбрано **Выполнять постоянно** (Run Continuously) (это значение по умолчанию) и щелкните **Далее**.
8. Убедитесь, что в разделе **Свойства подписки** (Subscription Properties) установлен флажок **Инициализировать** (Initialize Immediately) и щелкните **Далее**.
9. Убедитесь, что параметру **Тип подписки** (Subscription type of server) присвоен приоритет разрешения конфликтов 75.00 (это значение по умолчанию). Щелкните **Далее**.
10. Убедитесь, что установлен флажок **Создать подписки** (Create Subscription). Щелкните **Далее**, а затем — **Готово** (Finish).
11. По завершении создания подписки щелкните **Закреть** (Close).
12. В Мониторе репликации ознакомьтесь с агентами, историей и, если есть, с сообщениями об ошибках.
13. Изучите AWMergeSubscriber базу данных на предмет объектов, созданных при применении моментального снимка.
14. Внесите изменения на издатель и подписчике и наблюдайте, как механизм репликации переносит данные.
15. Создайте конфликт данных и наблюдайте, как они обнаруживаются и разрешаются.

Резюме занятия

- Репликация слиянием предоставляет очень мощный и гибкий механизм распространения изменений, потому что она предназначена для обслуживания мобильных, редко подключающихся к корпоративной сети пользователей.

- Особенности архитектуры репликации слиянием, предназначенной для обслуживания мобильных, редко подключающихся к корпоративной сети пользователей, позволяют использовать ее для репликации между серверами, потому что механизм слияния по определению поддерживает ситуации с длительными простоями и непредсказуемым графиком синхронизации данных.
- Отслеживание изменений происходит в наборе таблиц метаданных, которые хранятся в той же базе данных, которая публикуется или на которую создаются подписки.
- Поскольку и издатель, и подписчик хранят полную копию всех внесенных изменений, в каждом цикле механизм слияния можно выполнять один простой запрос: «Пришлите мне все, чего у меня еще нет».

Закрепление материала

Приведенные ниже вопросы помогут вам лучше усвоить основные темы данного занятия. Если вы не сумеете ответить на вопрос, повторите материал занятия. Ответы для самопроверки — в приложении «Ответы» в конце книги.

1. Компания Consolidated Messenger занимается доставкой посылок компаниям в центральной части города AbcCity. Заказы принимаются в центральном офисе, а курьеры получают инструкции по получению посылок на складе и по их доставке на свои коммуникаторы с Microsoft Pocket PC, на которых для синхронизации данных используется репликация слиянием. При получении посылки клиент расписывается в получении и курьер вносит эту информацию в свой коммуникатор, что вызывает обновление записи, относящейся к заказу. GPS-устройства, находящиеся в рюкзаках курьеров непрерывно передают данные о местоположении в центральный офис, что позволяет отслеживать поставку. Как нужно спроектировать базу данных, репликацию и разрешение конфликтов, чтобы учесть тот факт, что изменения данных о заказе происходят как в центральном офисе, так и на коммуникаторе курьера?
 - A. Одна таблица, которая содержит информацию о заказе, доставке и получении, и одна публикация из центрального офиса на коммуникаторы курьеров. При конфликтах данных приоритет всегда отдается издателю.
 - B. Одна таблица, которая содержит информацию о заказе, доставке и получении, и одна публикация из центрального офиса на коммуникаторы курьеров. При конфликтах данных приоритет всегда отдается подписчику.
 - C. Одна таблица, которая содержит информацию о заказе, доставке и получении, и одна публикация с коммуникаторов курьеров в центральный офис. При конфликтах данных приоритет всегда отдается издателю.
 - D. Одна таблица, которая содержит информацию о заказе, доставке и получении, и одна публикация из центрального офиса на коммуникаторы курьеров. При конфликтах данных приоритет всегда отдается издателю.
 - E. Одна таблица содержит информацию о заказе, а вторая — о доставке и получении. Одна публикация из центрального офиса на коммуникаторы курьеров информации о заказе, вторая публикация из центрального офиса на коммуникаторы курьеров информации о доставке и получении.

При конфликтах данных о заказах и о доставке и получении приоритет всегда отдается издателю.

- Г. Одна таблица содержит информацию о заказе, а вторая — о доставке и получении. Одна публикация из центрального офиса на коммуникаторы курьеров информации о заказе, и вторая публикация из центрального офиса на коммуникаторы курьеров информации о доставке и получении. При конфликтах данных о заказах приоритет всегда отдается издателю, а при конфликтах данных о доставке и получении приоритет всегда отдается подписчику.

Закрепление материала главы

Для закрепления знаний и навыков, полученных при изучении этой главы, выполните следующее:

- изучите резюме главы;
- повторите основные термины, использованные в этой главе;
- выполните задания по темам этой главы;
- сделайте предложенные упражнения;
- пройдите пробный экзамен.

Резюме главы

- Репликация транзакций с немедленным обновлением подписчиков не годится для решения высокой доступности, потому что в случае недоступности издателя, на подписчике нельзя вносить изменения.
- Репликация транзакций с использованием очередей для обновления подписчиков и репликацией слиянием прекрасно подходят для решений высокой доступности, потому что в обоих видах репликации поддерживается возможность выполнения транзакций как на издателе, так и на подписчике.
- Репликация транзакций в двунаправленной или одноранговой конфигурации может использоваться в решениях высокой доступности, но нужно обеспечить невозможность возникновения конфликтов данных.

Основные термины

Понятны ли вам значения приведенных здесь основных терминов? Чтобы проверить свои ответы, загляните в глоссарий на прилагаемом компакт-диске.

- Профиль агента ~ agent profile.
- Статья ~ article.
- Фильтр столбцов ~ column filter.
- Арбитр конфликтов ~ conflict resolver.
- Конфликт данных ~ data conflict.
- Агент распространителя ~ distribution agent.
- Распространитель ~ distributor.
- Динамический фильтр ~ dynamic filter.

- Поколение ~ generation.
- Фильтр соединения ~ join filter.
- Агент чтения журнала ~ log reader agent.
- Агент слияния ~ merge agent.
- Репликация слиянием ~ merge replication.
- Публикация ~ publication.
- Издатель ~ publisher.
- Агент чтения очереди ~ queue reader agent.
- Топология репликации ~ replication topology.
- Признак поколения репликации ~ replication watermark.
- Фильтр строк ~ row filter.
- Агент моментальных снимков ~ snapshot agent.
- Папка моментальных снимков ~ snapshot folder.
- Репликация моментальных снимков ~ snapshot replication.
- Подписчик ~ subscriber.
- Подписка ~ subscription.
- Трассировочный маркер ~ tracer token.
- Репликация транзакций, *или* транзакционная репликация ~ transactional replication.
- Проверка синхронизации ~ validation.

Упражнения

Сейчас вы примените на практике знания и навыки, полученные при изучении этой главы. Ответы на вопросы см. в разделе «Ответы» в конце книги.

Упражнение. Планирование высокой доступности

Исходные данные

О компании

Офис туристической компании Margie's Travel находится в г. Сан-Диего. Клиенты могут встретиться с агентом в офисе или разместить заявку на веб-сайте компании.

Ситуация

После добавления нового каталога продуктов веб-сайт компании стал работать нестабильно. Время от времени клиенты не могли совершать покупки на сайте, когда в данные вносили изменения.

Компания недавно отказалась от услуг консалтинговой фирмы, которая занималась разработкой и поддержкой ее веб-сайта и всех остальных приложений и не смогла обеспечить доступность критически важных бизнес-систем.

Планируемые изменения

Перед новым техническим директором была поставлена задача обеспечения высокой доступности всех критически важных бизнес-систем. Технический

директор взял на работу нового администратора баз данных и системного администратора для решения этой задачи и помощи в повседневной работе.

Существующая среда

В компании используется 11 баз данных, приведенных в табл. 17-1.

Табл. 17-1. Базы данных компании Margie's Travel

База данных	Описание	Размер
Orders	Сведения о заказах, полученных от клиентов	50 Гб
Customers	Личная информация о клиентах	15 Гб
CreditCards	Данные о кредитных картах клиентов	200 Мб
Employees	Информация о всех сотрудниках	50 Мб
Human Resources	Все документы отдела кадров и данные о зарплате сотрудников	300 Мб
Products	Данные о товарах, предлагаемых для продажи на веб-сайте компании	25 Гб
Flights	Сведения о билетах на рейсы, забронированных клиентами	2 Гб
Cruises	Данные о путевках, забронированных клиентами	1 Гб
CarRental	Данные об аренде автомобилей клиентами	1 Гб
Excursions	Данные об экскурсиях, забронированных клиентами (К экскурсиям относятся все услуги, кроме полетов, путевок, товаров и аренды автомобилей.)	2 Гб
Admin	Служебная база данных для работы администратора баз данных, временно пустая	12 Гб

В среде компании имеется один веб-сервер WEB1 и один сервер баз данных SQL1. Все серверы работают под управлением ОС Windows Server 2003, а на сервере SQL1 установлена SQL Server 2008.

На сервере SQL1 имеется внешняя дисковая стойка, подключенная к контроллеру RAID-массива с резервной батареей, на котором можно настроить режим RAID 0, RAID 1 и RAID 5. В настоящее время весь массив дисков настроен как единый набор RAID 0. Хранилище используется всего на 10% его мощности.

К серверам WEB1 и SQL1 подключен ленточный накопитель, который ни разу не использовался.

В настоящее время серверы SQL1 и WEB1 находятся в помещении рядом с рабочим местом уволенного консультанта. Все приложения на сервере WEB1 написаны в использовании ColdFusion или ASP.

Предлагаемая среда

Технический директор выделил часть бюджета на приобретение четырех дополнительных серверов с установленными Windows Server 2003 и SQL Server 2008. Все закупаемое оборудование должно поддерживать работу в кластере.

Данные в базах данных *Products*, *Customers*, *Orders*, *Flights*, *Cruises*, *Excursions* и *CarRental* будут доступны через Интернет посредством приложений, выполняемых на сервере WEB1. Вся информация о кредитных картах должна быть перенесена в базу данных *Customers* и защищена посредством ассиметричного шифрования. Все остальные базы данных должны быть защищены брандмауэром и доступны только для пользователей, прошедших проверку подлинности в корпоративном домене.

Для хранения баз данных развернута новая сеть SAN, имеющая достаточно дискового пространства для размещения всех баз данных. Каждый из 20 номеров логических устройств (LUN) в сети SAN сконфигурирован в виде зеркальных наборов с чередованием (*stripe of mirrors*), с четырьмя дисками в каждом зеркальном наборе.

Для целей унификации планируется объединить базы данных, как указано в таблице табл. 17-2.

Табл. 17-2. План объединения баз данных компании Margie's Travel

База данных	Описание	Размер
<i>Bookings</i>	<i>Orders, Flights, Cruises, Excursions, CarRental</i>	55 Гб
<i>Customers</i>	<i>Customers, CreditCards</i>	15 Гб

Бизнес-требования

В компании развернуто временное решение, обеспечивающее полное восстановление системы после любых сбоев в течение двух рабочих дней при максимальной потере данных только за последний час перед сбоем. В случае крупной аварии компания может выдержать потерю данных не более чем за два дня.

Для внесения изменений в систему предусмотрено время обслуживания в промежутке от полуночи до 8:00 утра воскресенья.

Необходимо создать долгосрочное решение для защиты компании от аппаратных сбоев с максимальным временем простоя не более одной минуты.

Технические требования

Базы данных *Orders* и *Customers* связаны между собой, поэтому должны размещаться на одном экземпляре SQL Server и вместе переключаться на резервный ресурс кластера при сбое. Нужно обеспечить безопасность данных кредитных карт во всех форматах, в том числе и в резервных копиях.

В базе данных *Products* каждую ночь нужно задействовать модель восстановления с неполным протоколированием, чтобы облегчить добавочную загрузку данных о продуктах и исследовательских статей.

Нужно обеспечить максимальную безопасность всех баз данных отдела кадров, а доступ к ним должен предоставляться только директору отдела кадров. Любые данные из отдела кадров должны быть зашифрованы при хранении как в базе данных, так и в любом другом месте в сети.

Для разработки новых маркетинговых кампаний и товарных предложений отделу маркетинга необходимо создавать отчеты на основе всех данных о клиентах и заказах, а также о заказанных товарах и услугах. Для проведения анализа нужны данные, получаемые почти в режиме реального времени. На процесс

создания отчетов не должно влиять обновление данных в производственной среде.

Все базы данных должны быть доступны 99,92% времени в течение всего года. Для восстановления после сбоев должно требоваться минимальное вмешательство администратора. Для клиентов, использующих веб-сайт, переходы при сбое должны происходить незаметно.

1. Чтобы воспользоваться оптовыми скидками, в компании Margie's Travel начали размещать в хранилище портфель с большим числом лучших 25 продуктов. Хранилище содержит копию базы данных Products. Для обеспечения актуальности состояния портфеля на веб-сайте, имеющееся количество уменьшается при заказе и увеличивается при увеличении пополнении хранилища. Какова оптимальная технология управления числом предлагаемых продуктов?
 - A. Репликация транзакций с обновлением подписчиков посредством очереди.
 - B. Доставка журналов.
 - C. Репликация слиянием.
 - D. Зеркальное отображение базы данных.
2. Базу данных *Products* надо сделать устойчивой к сбоям с минимальными усилиями. В случае выхода из строя основной базы данных *Products*, переход должен произойти в течение трех-пяти минут. Для обеспечения непротиворечивости, приложения, поддерживающие базу данных *Products*, инициализируют переходы вместо использования SQL Server. Отказоустойчивая база данных не должна быть доступна для любых других операций. Какая технология будет оптимальной в описанной ситуации?
 - A. Доставка журналов.
 - B. Репликация транзакций.
 - C. Репликация транзакций с обновлением подписчиков посредством очереди.
 - D. Репликация слиянием.
3. Базы данных *Bookings* и *Customers* критически важны для работы. Какая конфигурация RAID обеспечит оптимальное соотношение «производительность/доступность»?
 - A. RAID 5.
 - B. RAID 0.
 - C. RAID 1.
 - D. RAID 1 + RAID 0 (зеркало с чередованием).
4. Какое решение лучше всего подходит для реализации базы данных для отчетов, которую будет использовать отдел маркетинга?
 - A. Репликация слиянием.
 - B. Репликация транзакций.
 - C. Репликация моментальных снимков.
 - D. Репликация транзакций с обновлением подписчиков посредством очереди.

5. В последнее время сильно возросло число заказов из-за новых маркетинговых кампаний и размещения информации о компании в поисковой машине. Совокупная нагрузка обработки запросов на чтение и запись превышает возможности на сервера. Какая технология позволит компании Margie's Travel расширить возможности по обработке запросов на чтение и запись, одновременно обеспечив согласованность данных о заказах? (Решение должно предусматривать возможность расширения в будущем.)
- A. Репликация транзакций в одноранговой конфигурации.
 - B. Репликация транзакций в двунаправленной конфигурации.
 - C. Репликация транзакций.
 - D. Репликация транзакций с обновлением подписчиков посредством очереди.

Рекомендуемые упражнения

Чтобы подготовиться к экзамену по темам этой главы, выполните следующие задания.

Репликация транзакций

- **Задание 1.** Настройте в базе данных AdventureWorks репликацию всех данных с использованием репликации транзакций. Внесите изменения на издатель и проверьте результаты на подписчике.
- **Задание 2.** Настройте в базе данных AdventureWorks репликацию данных с использованием репликации транзакций. Примените фильтр строк к одной или нескольким таблицам. Внесите изменения на издатель и проверьте результаты на подписчике.
- **Задание 3.** Настройте в базе данных AdventureWorks репликацию данных с использованием репликации транзакций. Примените фильтр столбцов к одной или более таблицам. Внесите изменения на издатель и проверьте результаты на подписчике.
- **Задание 4.** Настройте в базе данных AdventureWorks репликацию данных с использованием репликации транзакций с обновлением подписчиков посредством очередей. Внесите изменения на издатель и проверьте результаты на подписчике. Внесите изменения на подписчике и проверьте результаты на издатель. Внесите противоречивые данные и посмотрите, что произойдет.
- **Задание 5.** Настройте в базе данных AdventureWorks репликацию данных с использованием репликации транзакций с немедленным обновлением подписчиков. Внесите изменения на издатель и проверьте результаты на подписчике. Внесите изменения на подписчике и проверьте результаты на издатель. Отключите издателя, измените данные на подписчике и посмотрите, что произойдет.
- **Задание 6.** Настройте в базе данных AdventureWorks репликацию данных с использованием репликации транзакций в двунаправленной архитектуре.

Внесите изменения на издателя и на подписчике и посмотрите, что произойдет. Внесите противоречивые данные и посмотрите, что произойдет.

- **Задание 7.** Настройте в базе данных AdventureWorks репликацию данных с использованием репликации транзакций в одноранговой архитектуре. Внесите изменения в каждую базу данных, участвующую в репликации, и посмотрите, что произойдет. Внесите противоречивые данные и посмотрите, что произойдет.
- **Задание 8.** Настройте в базе данных AdventureWorks репликацию данных с использованием репликации транзакций с обновлением подписчиков посредством очередей и примените фильтр строк к одной или нескольким статьям. Внесите изменения на издателя, не соответствующие фильтру строк, и посмотрите, что произойдет. Внесите не соответствующие фильтру строк изменения на подписчике и посмотрите, что произойдет.
- **Задание 9.** Настройте в базе данных AdventureWorks репликацию данных с использованием репликации транзакций с немедленным обновлением подписчиков и примените фильтр строк к одной или нескольким статьям. Внесите изменения на издателя, не соответствующие фильтру строк, и посмотрите, что произойдет. Внесите не соответствующие фильтру строк изменения на подписчике и посмотрите, что произойдет.
- **Задание 10.** Исследуйте, что происходит при определении хранимых процедур, представлений и функций как статей репликации транзакций.

Репликация слиянием

- **Задание 1.** Настройте в базе данных AdventureWorks репликацию слиянием. Внесите изменения на издателя и проверьте результаты на подписчике. Внесите изменения на подписчике и проверьте результаты на издателя. Внесите противоречивые данные и посмотрите, что произойдет.
- **Задание 2.** Настройте в базе данных AdventureWorks репликацию слиянием со статическим фильтром строк. Внесите изменения на издателя, не соответствующие фильтру строк, и посмотрите, что произойдет. Внесите не соответствующие фильтру строк изменения на подписчике и посмотрите, что произойдет.
- **Задание 3.** Настройте в базе данных AdventureWorks репликацию слиянием с динамическим фильтром строк. Внесите изменения на издателя, не соответствующие фильтру строк, и посмотрите, что произойдет. Внесите не соответствующие фильтру строк изменения на подписчике и посмотрите, что произойдет.
- **Задание 4.** Настройте в базе данных AdventureWorks репликацию слиянием с фильтром соединения. Внесите изменения на издателя, не соответствующие фильтру строк, и посмотрите, что произойдет. Внесите не соответствующие фильтру строк изменения на подписчике и посмотрите, что произойдет.

Пробный экзамен

На прилагаемом компакт-диске содержится пробный экзамен, обеспечивающий широкие возможности самоподготовки. Например, можно проверить, как усвоены темы этой главы либо все темы экзамена 70-432. Можно симитировать настоящий экзамен или выбрать режим обучения, когда после каждого вопроса приводятся правильные ответы с пояснениями.

ПРИМЕЧАНИЕ Пробный экзамен

Подробнее о пробном экзамене см. «Введение».

Ответы

Глава 1

Занятие 1

Закрепление материала

1. **Правильные ответы: С и D.**

- A. **Неправильно:** SQL Server 2008 не работает в Windows 2000.
- B. **Неправильно:** SQL Server 2008 поддерживает работу в Windows 2003 Enterprise, но только при условии установки пакета исправлений 2 (SP2) или более позднего.
- C. **Правильно:** SQL Server 2008 поддерживает работу в Windows 2003 Enterprise с SP2 или более поздним или в Windows 2008 Enterprise.
- D. **Правильно:** SQL Server 2008 поддерживает работу в Windows 2003 Enterprise с SP2 или более поздним или в Windows 2008 Enterprise.

2. **Правильный ответ: В.**

- A. **Неправильно:** SQL Server 2008 Express поддерживает работу в Windows XP Home Edition с SP2.
- B. **Правильно:** SQL Server 2008 Express не поддерживает работу на ядре сервера Windows 2008.
- C. **Неправильно:** SQL Server 2008 Express поддерживает работу в Windows 2003 Enterprise с SP2.
- D. **Неправильно:** SQL Server 2008 Express поддерживает работу в Windows XP Tablet Edition с SP2, но это не оптимальное использование операционной системы.

Занятие 2

Закрепление материала

1. **Правильные ответы: В и С.**

- A. **Неправильно:** Хотя редакция Express и в состоянии справиться с нагрузкой в компании Margie's Travel, она не поддерживает работу в режиме издателя для синхронизации множественных копий базы данных заказов.

- В. **Правильно:** Редакция Standard в состоянии справиться с нагрузкой в компании Margie's Travel и поддерживает функциональность репликации, необходимую для синхронизации баз данных.
 - С. **Правильно:** Редакция Enterprise в состоянии справиться с любой нагрузкой, а также предоставить любую необходимую компании функциональность Margie's Travel.
 - Д. **Неправильно:** Редакция Compact не предназначена для работы в режиме СУБД для веб-приложений.
2. **Правильный ответ: D.**
- А. **Неправильно:** Все необходимые аналитические функции, такие как OLAP и Data Mining, поддерживает только SQL Server 2008 Enterprise.
 - В. **Неправильно:** Все необходимые аналитические функции, такие как OLAP и Data Mining, поддерживает только SQL Server 2008 Enterprise.
 - С. **Неправильно:** Все необходимые аналитические функции, такие как OLAP и Data Mining, поддерживает только SQL Server 2008 Enterprise.
 - Д. **Правильно:** Все необходимые аналитические функции, такие как OLAP и Data Mining, поддерживает только SQL Server 2008 Enterprise.

Занятие 3

Закрепление материала

1. **Правильный ответ: D.**
- А. **Неправильно:** Команда *ALTER DATABASE* не включает поддержку *FILESTREAM*.
 - В. **Неправильно:** Команда *DBCC* не включает поддержку *FILESTREAM*.
 - С. **Неправильно:** Хотя хранимая процедура *sp_configure* и может изменять уровень поддержки типа данных *FILESTREAM*, она не поддерживает всех вариантов настройки, в частности определение имени общего ресурса Windows.
 - Д. **Правильно:** Хранимая процедура *sp_filestream_configure* позволяет задать уровень поддержки *FILESTREAM*, а также сконфигурировать общий ресурс Windows.
2. **Правильный ответ: С.**
- А. **Неправильно:** Хотя изменять пароли учетных записей служб можно средствами Диспетчера служб Windows (Windows Service Control), этого делать не следует. Только Диспетчер конфигурации SQL Server (SQL Server Configuration Manager) умеет правильно шифровать и расшифровывать главный ключ, при изменении учетной записи или пароля службы.
 - В. **Неправильно:** SQL Server Management Studio нельзя использовать для изменения паролей учетных записей служб.
 - С. **Правильно:** Только в SQL Server Configuration Manager есть код для правильного шифрования и расшифровки главного ключа службы, используемого SQL Server, при изменении учетной записи службы или пароля.

- D. **Неправильно:** В SQL Server 2005 в SQL Server Surface Area Configuration Manager была соответствующая утилита, но ее удалили в версии SQL Server 2008, а вместо нее изменения в конфигурации предлагается делать непосредственно средствами системной хранимой процедуры `sp_configure`.

Занятие 4

Закрепление материала

1. **Правильные ответы: В и С.**

- A. **Неправильно:** Notification Services были в SQL Server 2005, но в SQL Server 2008 их больше нет.
- B. **Правильно:** Компонент Database Mail может использоваться для отправки сообщений клиентам.
- C. **Правильно:** Можно создать приложение на Visual Studio .NET, использующее почтовые библиотеки .NET Framework.
- D. **Неправильно:** Монитор активности — функция SSMS, позволяющая увидеть деятельность запросов на сервере, но она не поддерживает отправку сообщений.

2. **Правильные ответы: С и D.**

- A. **Неправильно:** Открытый профиль доступен всем пользователям, имеющим право отправлять почту, а это нарушает требования по безопасности.
- B. **Неправильно:** Хотя конфигурирование профиля как закрытого ограничивает доступ, профиль нельзя использовать, если у пользователя нет доступа к нему.
- C. **Правильно:** Надо сконфигурировать профиль как закрытый, но предоставить к нему доступ выделенным пользователям.
- D. **Правильно:** Настройка профиля как профиля по умолчанию позволит выделенным пользователям отправлять почту, используя этот профиль, но не указывая профиль явно.

Упражнения

Упражнение 1. Проектирование инфраструктуры SQL Server

1. Поскольку менеджерам по продажам нужно регулярно отключаться, но при этом требуется синхронизация данных, следует использовать редакция Express или Compact сервера SQL Server 2008. Однако приложениям, установленным на ноутбуках менеджеров по продажам нужна поддержка хранимых процедур. Только редакция Express обеспечит поддержку репликации и хранимых процедур за минимальные деньги.
2. SQL Server 2008 Enterprise нужно установить на корпоративном сервере баз данных. Редакция Enterprise может масштабироваться для поддержки любой нагрузки, обеспечивая при этом аналитические службы.

3. Нужно установить как минимум службы SSAS и SSRS. SSAS обеспечит поддержку аналитических приложений, необходимых для анализа тенденций, а SSRS предоставит инфраструктуру создания отчетов, необходимую для генерации всех оперативных отчетов.
4. Надо установить Windows 2003 Server Enterprise SP2 или более позднюю или Windows Server 2008 Enterprise или более позднюю, чтобы обеспечить поддержку SQL Sever 2008 Enterprise.

Глава 2

Занятие 1

Закрепление материала

1. **Правильный ответ: С.**
 - А. **Неправильно:** У членов роли *db_owner* остается право изменять данные в базе.
 - В. **Неправильно:** У членов роли *db_owner* остается право изменять данные в базе.
 - С. **Правильно:** У членов роли *db_owner* остается право изменять данные в базе, если только не установлен атрибут READ_ONLY.
 - Д. **Неправильно:** У членов роли *db_owner* остается право изменять данные в базе.

Занятие 2

Закрепление материала

1. **Правильный ответ: D.**
 - А. **Неправильно:** Параметр AUTO_SHRINK не гарантирует восстановление базы данных на любой момент времени.
 - В. **Неправильно:** Принудительная параметризация не гарантирует восстановление базы данных на любой момент времени.
 - С. **Неправильно:** Хотя модель восстановления с неполным протоколированием обеспечивает максимальную производительность и позволяет создавать резервные копии журналов транзакций, базу данных нельзя восстановить на момент времени, в течение которого выполнялась минимально протоколируемая операция.
 - Д. **Правильно:** Полная модель восстановления гарантирует восстановление базы данных на любой момент времени.

Занятие 3

Закрепление материала

1. **Правильные ответы: А и С.**

- А. **Правильно:** В процессе работы команды *DBCC CHECKDB* выполняются команды *DBCC CHECKTABLE*, *DBCC CHECKALLOC* и *DBCC CHECKCATALOG*.
- В. **Неправильно:** *DBCC CHECKIDENT* используется для проверки, исправления и повторной генерации значений идентификаторов.
- С. **Правильно:** В процессе работы команды *DBCC CHECKDB* выполняются команды *DBCC CHECKTABLE*, *DBCC CHECKALLOC* и *DBCC CHECKCATALOG*.
- Д. **Неправильно:** Команда *DBCC FREEPROCCACHE* очищает кеш запросов.

Упражнения

Упражнение. Управление данными

- 1. Для обеспечения максимальной производительности базы данных Orders надо во время импорта EDI-транзакций использовать модель восстановления с неполным протоколированием. Так как нельзя восстановить базу данных на момент времени, в течении которого выполнялась минимально протоколируемая операция, модель восстановления с неполным протоколированием надо использовать на очень короткое время, когда это необходимо.
- 2. Чтобы гарантировать предотвращение потери данных более, чем за последние 5 минут, нужно создавать резервные копии журналов транзакций всех баз данных. Такие резервные копии можно создавать только для баз данных с полной моделью восстановления или моделью с неполным протоколированием. Все базы данных всегда должны работать в модели полного восстановления за единственным исключением: на время загрузки данных база Orders должна работать в модели с неполным протоколированием.

Глава 3

Занятие 1

Закрепление материала

1. **Правильные ответы: В и С.**

- А. **Неправильно:** Сжатие страниц можно применить в таблице с типом данных *VARCHAR(MAX)*, просто столбец с данными такого типа сжиматься не будет.
- В. **Правильно:** Нельзя применить сжатие страниц в таблице со столбцами с разреженными данными.
- С. **Правильно:** Наборы столбцов создаются из групп разреженных столбцов,

поэтому из-за того, что сжатие страниц несовместимо с разреженными столбцами, это же верно в отношении наборов столбцов.

- D. **Неправильно:** Сжатие страниц можно применить в таблице с типом данных FILESTREAM, просто столбец с данными такого типа сжиматься не будет.

Занятие 2

Закрепление материала

1. Правильные ответы: А и С.

- A. **Правильно:** Типы ROWGUIDCOL, IDENTITY и FILESTREAM нельзя использовать в разреженных столбцах. Так как в столбце идентификаторов включен параметр NOT FOR REPLICATION, недопустим разреженный столбец с параметром NOT FOR REPLICATION. Кроме того разреженные столбцы должны поддерживать значения NULL.
- B. **Неправильно:** В разреженных столбцах должны быть допустимы значения NULL.
- C. **Правильно:** Типы ROWGUIDCOL, IDENTITY и FILESTREAM нельзя использовать в разреженных столбцах. Так как в столбце идентификаторов включен параметр NOT FOR REPLICATION, недопустим разреженный столбец с параметром NOT FOR REPLICATION. Кроме того, разреженные столбцы должны поддерживать значения NULL.
- D. **Неправильно:** В разреженных столбцах разрешена сортировка данных.

Упражнения

Упражнение 1. Управление данными

1. Руководства по продуктам должны храниться в таблице со столбцом VARBINARY(MAX), определенным как FILESTREAM. Кроме того, надо добавить столбец, в котором будет указываться тип файла, хранимого в том столбце, чтобы приложения корректно интерпретировали эти данные. В таблице с руководствами по продуктам должен быть внешний ключ, указывающий на таблицу продуктов, — это будет гарантировать, что будут загружаться только руководства для продуктов, существующих в базе данных.
2. Можно задать XML в качестве типа данных столбца с описанием продукта, а затем создать по XML-документу для каждого перевода описания, загружаемого в XML-столбец. Вместе с тем, придется анализировать этот XML-документ, чтобы найти описание на нужном языке. Хотя для этого потребуются дополнительная поддержка, самый простой способ хранения описаний продуктов на многих языках — создать отдельный столбец для каждого языка, затем задействовать свойство COLLATE, чтобы задать нужную сортировку в столбце.
3. Можно добавить в таблицу Customers столбец SalesRep с внешним ключом, указывающим на таблицу менеджеров по продажам, чтобы обеспечить целостность. Можно также создать новую таблицу, содержащую столбцы CustomerID и SalesRepID, создать первичный ключ на основе этих столбцов,

а затем создать внешние ключи, указывающие на таблицы клиентов и менеджеров по продажам.

Глава 4

Занятие 1

Закрепление материала

1. **Правильный ответ: С.**

- A. **Неправильно:** Поскольку при определении плана выполнения запроса оптимизатор запросов учитывает только первый столбец индекса (за исключением ситуации, когда запрос возвращает только ProductID, ProductName и SKU), очень маловероятно, что SQL Server будет использовать этот индекс вместо первичного ключа таблицы.
- B. **Неправильно:** Хотя такой индекс позволяет SQL Server находить запросы, выполняющие поиск по ProductName, серверу SQL Server все равно придется обращаться к таблице, чтобы вернуть остальные данные возвращаемого набора.
- C. **Правильно:** Этот индекс позволяет SQL Server выполнять поиск по ProductName и возвращать весь набор для запроса на основании индекса, а не таблицы. Кроме того, индекс остается небольшим по объему, так как только столбец ProductName определяет сбалансированное дерево и максимальный уровень индекса.
- D. **Неправильно:** Хотя SQL Server и может использовать этот индекс для выполнения запросов только на основании индекса, каждый уровень индекса будет основан на ключе размером 250 Кб, что приведет к чтению намного большего числа страниц.

Занятие 2

Закрепление материала

1. **Правильный ответ: D.**

- A. **Неправильно:** Так как столбец Color может содержать значения NULL, фильтруемый индекс будет более эффективен.
- B. **Неправильно:** Так как столбец Color может содержать значения NULL, фильтруемый индекс будет более эффективен.
- C. **Неправильно:** Хотя фильтруемый индекс столбца Color будет более эффективен, чем нефилтруемый, SQL Server придется выполнять дополнительные операции чтения, чтобы считать значения столбцов ProductType и WoodSpecies.
- D. **Правильно:** Так как столбец Color может содержать значения NULL, самым эффективным индексом для этого запроса будет тот, который включает только значения, не поддерживающие NULL. Кроме того, включив в запрос столбцы ProductType и WoodSpecies, можно добиться того, что запрос будет выполняться исключительно на основе индекса.

Занятие 3

Закрепление материала

1. Правильный ответ: С.

- А. **Неправильно:** Команда BULK INSERT позволяет быстро загружать данные, если на момент загрузки данных в таблице есть индексы, но это приводит к большому объему дополнительных операций при записи в индексы.
- В. **Неправильно:** Загрузка данных в таблицу, в которой нет никаких индексов, с последующим созданием индексов после загрузки эффективнее, чем загрузка в таблицу, в которой уже есть индексы. Однако удаление с последующим воссозданием индексов требует больше ресурсов, чем отключение, а затем включение индексов.
- С. **Правильно:** Отключение индексов перед загрузкой позволяет избавиться от дополнительных операций, связанных с поддержкой индексов. Метод отключения с последующим включением индексов избавляет от необходимости создавать сценарии воссоздания индексов после загрузки.
- Д. **Неправильно:** Эффективнее загружать данные в таблицы, чем отказаться от индексов. Службы Integration Services не избавляют от дополнительной нагрузки, связанной с поддержкой индексов во время операции загрузки.

Упражнения

Упражнение 1. Управление данными

- 1. Создать в таблице CustomerOrder фильтруемый индекс на основе столбцов с информацией о еще не доставленных заказах.
- 2. Создать в таблице CustomerOrder пространственный индекс на основе широты и долготы.
- 3. Создать некластеризованный индекс на основе столбцов имени, города и менеджера по продажам.
- 4. Создать в таблице CustomerOrderDetail некластеризованный индекс по имени продукта.

Глава 5

Занятие 1

Закрепление материала

1. Правильные ответы: В и D

- А. **Неправильно:** В SQL Server 2005 полнотекстовые индексы хранились в файловой системе. SQL Server 2008 хранит полнотекстовые индексы в файловой группе в базе данных.
- В. **Правильно:** Полнотекстовые каталоги содержат полнотекстовые индексы, а содержимое этих индексов хранится в базе данных SQL Server 2008.

- C. **Неправильно:** Параметр *NO POPULATION* включает на SQL Server поддержку полнотекстовых индексов, но не заполняет индекс. Поэтому поиски не дадут никаких результатов.
- D. **Правильно:** Параметр *CHANGE_TRACKING AUTO* заставляет SQL Server заполнять полнотекстовый индекс после его создания и автоматически отражать в индексе любые изменения в базе данных.
2. **Правильный ответ: С.**
- A. **Неправильно:** Аргумент *START {FULL | INCREMENTAL | UPDATE} POPULATION* выполняет заполнение полнотекстового индекса, но он должен заполняться вручную или в рамках задания Агента SQL Server.
- B. **Неправильно:** Аргумент *START {FULL | INCREMENTAL | UPDATE} POPULATION* выполняет заполнение полнотекстового индекса, но он должен заполняться вручную или в рамках задания Агента SQL Server.
- C. **Правильно:** Когда аргументу *CHANGE_TRACKING* присваивается значение *AUTO*, SQL Server автоматически обновляет полнотекстовый индекс при изменении данных, на основе которых тот создается. В режиме *AUTO* никакого вмешательства администратора — ручного или с применением запланированного задания — не требуется.
- D. **Неправильно:** Аргумент *START {FULL | INCREMENTAL | UPDATE} POPULATION* выполняет заполнение полнотекстового индекса, но он должен заполняться вручную или в рамках задания Агента SQL Server.

Занятие 2

Закрепление материала

1. **Правильные ответы: А и С.**
- A. **Правильно:** *CONTAINS* поддерживает поиск с учетом взаимного положения слов с строках, если есть ключевое слово *NEAR*.
- B. **Неправильно:** *FREETEXT* не поддерживает поиск с учетом взаимного положения слов.
- C. **Правильно:** *CONTAINSTABLE* поддерживает поиск с учетом взаимного положения слов с строках, если есть ключевое слово *NEAR*.
- D. **Неправильно:** *FREETEXTTABLE* не поддерживает поиск с учетом взаимного положения слов.
2. **Правильный ответ: D.**
- A. **Неправильно:** Аргумент *FORMSOF* поддерживает поиск с учетом тезауруса и падежных окончаний, но не поддерживает поиск с учетом взаимного положения слов.
- B. **Неправильно:** Аргумент *FORMSOF* поддерживает поиск с учетом тезауруса и падежных окончаний, но не поддерживает поиск с учетом взаимного положения слов.
- C. **Неправильно:** *ISABOUT* поддерживает поиск с учетом взаимного положения слов, но не поддерживает поиск с учетом весов — для этого нужно указать ключевое слово *WEIGHT* и значение веса.

D. Правильно: *ISABOUT* поддерживает поиск с учетом взаимного положения слов, а также поиск с учетом весов, если указано ключевое слово *WEIGHT* и значение веса.

Занятие 3

Закрепление материала

1. Правильный ответ: А.

- A. Правильно:** Стоп-списки создаются в SQL Server 2008, чтобы исключить слова из полнотекстового индекса, а также аргументов поиска. После связывания стоп-списка с полнотекстовым индексом все запросы, использующие этот индекс автоматически удаляют любые стоп-слова из этого списка из аргументов поиска.
- B. Неправильно:** Файлы неучитываемых слов использовались в SQL Server 2005. В SQL Server 2008 для исключения слов из аргументов поиска используются стоп-списки.
- C. Неправильно:** Тезаурус позволяет заменять слова, например общепринятые аббревиатуры или орфографические ошибки, но не удалять слова из аргументов поиска.
- D. Неправильно:** Хотя удаление слов из аргументов поиска можно реализовать в приложении, это потребует больше усилий, чем создание, заполнение и управление стоп-списками.

Упражнения

Упражнение 1. Установка и настройка экземпляра SQL Server 2008

- 1. Руководства по продуктам должны храниться в таблице со столбцом VAR-BINARY(MAX), определенным как FILESTREAM. Кроме того, надо добавить столбец, в котором будет указываться тип файла, хранимого в этом столбце, чтобы можно было корректно построить полнотекстовый индекс таблицы.
- 2. В таблице, где хранятся руководства по продуктам, надо создать полнотекстовый каталог, а также полнотекстовый индекс. Если задействовать интерпретирующие форматы документов фильтры полнофункционального индексатора, можно не вносить никаких изменений в индексацию документов и немедленно предоставить пользователям доступ к руководствам.
- 3. Поскольку пользователи не всегда будут использовать точно те термины, которые есть в документации, в тезаурусе нужно создать списки расширения и замены, чтобы предоставить пользователям синонимы, а также исправлять стандартные опечатки и аббревиатуры. Если требуется поддержка руководств на многих языках, надо создать отдельный столбец для каждого языка, который будет содержать локализованные документы для отдельных языков. Для каждого локализованного столбца надо создать полнотекстовый индекс, используя особенности конкретного языка, чтобы обеспечить максимальную эффективность работы средства разбиения по словам, а также

позволить парадигматическому модулю создавать производные формы слов в каждом языке.

Глава 6

Занятие 1

Закрепление материала

1. **Правильный ответ: С.**

- A. **Неправильно:** Хотя доставка журналов и позволяет поддерживать дополнительные копии базы данных, в Contoso нет средств на дополнительное оборудование.
- B. **Неправильно:** Хотя репликация и позволяет поддерживать дополнительные копии базы данных, в Contoso нет средств на дополнительное оборудование.
- C. **Правильно:** Можно секционировать самые активно используемые таблицы, что позволит распределить данные на несколько файлов, а это повысит производительность.
- D. **Неправильно:** Хотя зеркальное отображение баз данных создает дополнительную копию базы данных, в Contoso нет средств на дополнительное оборудование.

Занятие 2

Закрепление материала

1. **Правильные ответы: А и С.**

- A. **Правильно:** Оператор *SWITCH* удаляет заказы старше 30 дней, не вызывая никакой блокировки.
- B. **Неправильно:** Можно выполнить операцию *DELETE*, но при этом будет создана монополярная блокировка, что может помешать пользователям размещать заказы.
- C. **Правильно:** Секционирование столбца OrderDate позволит использовать оператор *SWITCH* для удаления заказов старше 30 дней, не вызывая никакой блокировки.
- D. **Неправильно:** Можно выполнить операцию *DELETE*, но при этом будет создана монополярная блокировка, что может помешать пользователям размещать заказы.

Занятие 3

Закрепление материала

1. **Правильный ответ: В.**

- A. **Неправильно:** Репликация транзакций позволяет перемещать данные в таблицы хранилища данных, но при этом могут создаваться монополярные блокировки, которые могут помешать работе пользователей.

- В. **Правильно:** Оператор *SWITCH* предназначен для перемещения секций данных в таблицу, не создавая блокировок.
- С. **Неправильно:** Механизм зеркального отображения баз данных требует, чтобы зеркальная база данных находилась в автономном режиме, и не подходит для данной ситуации.
- Д. **Неправильно:** Моментальные снимки баз данных представляют собой копию для чтения базы данных на определенный момент времени, поэтому они не содержат никаких новых добавленных позднее данных.

Занятие 4

Закрепление материала

1. Правильный ответ: D.

- А. **Неправильно:** Механизм зеркального отображения баз данных поддерживает зеркальную базу данных в синхронизированном состоянии, но эта база данных недоступна. По этой причине этот вариант не годится для архивирования.
- В. **Неправильно:** Репликация транзакций может применяться для переноса данных в хранилище данных в другой системе, но все равно нужно удалять данные из системы ввода заказов. Это отрицательно сказывается на параллелизме и производительности системы ввода заказов.
- С. **Неправильно:** Моментальные снимки баз данных представляют собой копию для чтения базы данных на определенный момент времени и не годятся для архивирования.
- Д. **Правильно:** Создав секционированную таблицу, вы сможете удалять данные заказов из системы ввода заказов, не влияя на параллелизм и производительность. После удаления секции из таблицы можно загружать данные в хранилище.

Упражнения

Упражнение 1. Проектирование инфраструктуры SQL Server

1. Правильный ответ: В.

- А. **Неправильно:** Данные базы *Customer* должны архивироваться как минимум за последние 6 лет. Даже если удастся перенести все данные в электронные таблицы, потребуются значительные дополнительные усилия от администратора, что значительно сложнее создания простой полной резервной копии данных.
- В. **Правильно:** Данные базы *Customer* должны архивироваться как минимум за последние 6 лет. Ежемесячное создание резервной копии на ленте позволяет удовлетворить этому требованию с минимальными усилиями со стороны администрирования.
- С. **Неправильно:** Чтобы архивировать базу клиентов, не нужно создавать новую базу данных или использовать репликацию баз данных. Проще

всего удовлетворить требованию, регулярно создавая полные резервные копии и храня резервные копии за последние 6 лет.

D. Неправильно: Этот вариант не обеспечивает архивирование. Кроме того, он не обеспечивает сохранение всех данных базы *Customer*. Например, изменения, внесенные в эту базу данных, удалят записи о значениях, существовавших до этих изменений.

2. Правильный ответ: D.

A. Неправильно: Для переноса секций между таблицами следует использовать операцию *SWITCH*.

B. Неправильно: Для переноса секций между таблицами следует использовать операцию *SWITCH*.

C. Неправильно: При выполнении операции *SWITCH* только одна секция может содержать данные. Поэтому возможно только перемещение данных в Секцию 2 таблицы *Order.Archive*.

D. Правильно: При выполнении операции *SWITCH* только одна секция может содержать данные. Поэтому возможно только перемещение данных в Секцию 2 таблицы *Order.Archive*.

3. Правильный ответ: C.

A. Неправильно: При выполнении операции *SWITCH* нельзя перемещать данные между файловыми группами. Поэтому Секция 2 таблицы *Order.Sales* и Секция 2 таблицы *Order.Archive* должны находиться в одной файловой группе.

B. Неправильно: При выполнении операции *SWITCH* нельзя перемещать данные между файловыми группами. Поэтому Секция 2 таблицы *Order.Sales* и Секция 2 таблицы *Order.Archive* должны находиться в одной файловой группе.

C. Правильно: При выполнении операции *SWITCH* нельзя перемещать данные между файловыми группами. Поэтому Секция 2 таблицы *Order.Sales* и Секция 2 таблицы *Order.Archive* должны находиться в одной файловой группе.

D. Неправильно: При выполнении операции *SWITCH* нельзя перемещать данные между файловыми группами. Поэтому Секция 2 таблицы *Order.Sales* и Секция 2 таблицы *Order.Archive* должны находиться в одной файловой группе.

Глава 7

Занятие 1

Закрепление материала

1. Правильные ответы: A и C.

A. Правильно: При импорте данных нужны разрешения *SELECT*, *INSERT* и *ALTER TABLE* в целевой таблице. Однако если не отключать триггеры и проверки ограничений, разрешение *ALTER TABLE* не нужно.

- B. **Неправильно:** При импорте данных нужны разрешения *SELECT*, *INSERT* и *ALTER TABLE* в целевой таблице. Однако если не отключать триггеры и проверки ограничений, разрешение *ALTER TABLE* не нужно.
 - C. **Правильно:** При импорте данных нужны разрешения *SELECT*, *INSERT* и *ALTER TABLE* в целевой таблице. Однако если не отключать триггеры и проверки ограничений, разрешение *ALTER TABLE* не нужно.
 - D. **Неправильно:** При импорте данных нужны разрешения *SELECT*, *INSERT* и *ALTER TABLE* в целевой таблице. Однако если не отключать триггеры и проверки ограничений, разрешение *ALTER TABLE* не нужно.
2. **Правильный ответ: C.**
- A. **Неправильно:** Хотя это и позволяет опубликовать таблицы из Oracle в SQL Server, это существенно более сложная операция, чем использование для переноса данных мастера импорта и экспорта.
 - B. **Неправильно:** Придется написать команды BCP для каждой таблицы, в которую импортируются данные, что намного менее эффективно, чем использование для переноса данных мастера импорта и экспорта.
 - C. **Правильно:** Мастер импорта и экспорта позволяет в единой операции подключиться к Oracle, выбрать несколько таблиц и переместить их данные в SQL Server.
 - D. **Неправильно:** Функция *OPENROWSET* позволяет переносить по одной таблице за раз, что менее эффективно, чем использование мастера импорта и экспорта.

Упражнения

Упражнение 1. Разработка стратегии импорта для компании Coho Vineyard

1. Можно создать процедуру экспорта, которая будет переносить данные с каждого завода в центральную базу данных средствами мастера импорта и экспорта. Однако хорошо бы выполнять уведомления, проверку данных и другие бизнес-проверки данных перед их загрузкой в центральную базу данных. Поэтому лучше задействовать все возможности SSIS.
2. Можно было бы использовать BCP или BULK INSERT для переноса EDI-данных в таблицу до выполнения хранимой процедуры, которая разбивает XML и выполняет все бизнес-вычисления. Используя все возможности SSIS можно создать надежный процесс импорта, который будет параллельно обрабатывать много файлов, проверять структуры XML и даже разбивать XML во время импорта данных.

Глава 8

Занятие 1

Закрепление материала

1. Правильный ответ: С.

- А. **Неправильно:** Даже если создать условие, проверяющее все базы данных, собственник базы данных может предпочесть не подписываться на эту политику — исключение составляет ситуация, когда вы потребуете соблюдать политику.
- В. **Неправильно:** Хотя и можно создать категорию политики с включенным свойством *Mandate* и добавить политику в эту категорию, это потребует больше усилий, чем просто добавление политики в категорию политик по умолчанию.
- С. **Правильно:** В категории политик по умолчанию включено свойство *Mandate*, которое к тому же нельзя отключить.
- Д. **Неправильно:** Можно попытаться вручную контролировать политики на экземпляре, но это потребует больше усилий, чем просто добавление политики в категорию политик по умолчанию.

Упражнения

Упражнение 1. Разработка стратегии управления для компании Coho Vineyard

1. Базу данных *HR* надо защитить средствами TDE, поэтому надо создать политики, гарантирующие создание ключей шифрования, включение шифрование базы данных *HR* и создание используемого TDE сертификата в главной базе данных. Так как база данных *Salary* зашифрована с использованием сертификата, надо создать политику, гарантирующую наличие обеспечивающего шифрование данных сертификата в базе *HR*. Надо также создать политику, обеспечивающую шифрование данных в базе данных *Salary*. Руководство требует, чтобы нельзя было выполнять запросы среды CLR и нерегламентированные удаленные запросы, поэтому надо создать политику, проверяющую применение конфигурации контактной зоны. Наконец надо обеспечить создание резервных копий, чтобы гарантировать возможность восстановления, поэтому надо создать одну или несколько политик проверки последней даты успешного создания резервной копии.

Глава 9

Занятие 1

Закрепление материала

1. **Правильный ответ: С.**

- A. **Неправильно:** Если начать восстановление с последней полной резервной копии, будут потеряны все не успевшие попасть в эту копию транзакции.
- B. **Неправильно:** Процедуру восстановления нельзя начинать с разностной резервной копии.
- C. **Правильно:** Первый шаг любой операции восстановления заключается в резервном копировании «хвоста» журнала. Однако команда *BACKUP LOG* выполняет запись как в журнал транзакций, так и главный файл данных. Параметр *NO_TRUNCATE* позволяет создавать резервную копию журнала транзакций, не выполняя запись в главный файл данных.
- D. **Неправильно:** Параметр *TRUNCATE_ONLY* команды *BACKUP LOG* в SQL Server 2008 не существует.

Занятие 2

Закрепление материала

1. **Правильный ответ: С.**

- A. **Неправильно:** Хотя параметр *NORECOVERY* и позволяет восстанавливать разностные резервные копии и резервные копии журналов транзакций, восстановление потерпит неудачу, так как структура каталогов в SQL Server не подходит для восстановления данной базы данных.
- B. **Неправильно:** Параметр *CONTINUE_AFTER_ERROR* служит для работы с поврежденными носителями с резервными копиями, а не для переноса файлов во время восстановления.
- C. **Правильно:** Поскольку структура папок и файлов отличается, надо переместить файлы данных и журналов в новое место. Операция *MOVE* позволяет указать новое местоположение файлов во время восстановления из полной резервной копии.
- D. **Неправильно:** Параметр *PARTIAL* позволяет выполнить частичное восстановление, но не поддерживает перенос файлов данных.

Занятие 3

Закрепление материала

1. **Правильные ответы: В и С.**

- A. **Неправильно:** Нельзя создавать моментальные снимки системных баз данных.

- В. **Правильно:** Хотя в моментальных снимках баз данных и нельзя выполнять полнотекстовые запросы, создавать моментальные снимки базы данных с полнотекстовым индексом вполне возможно.
- С. **Правильно:** Хотя данные FILESTREAM и недоступны в моментальных снимках баз данных, вполне допустимо создавать моментальные снимки базы данных с данными FILESTREAM.
- Д. **Неправильно:** Нельзя создавать моментальные снимки системных баз данных.

Упражнения

Упражнение 1. Разработка стратегии резервного копирования для компании Coho Vineyard

1. Поскольку разрешается потерять данные максимум за последние 5 минут, резервные копии журнала транзакций базы данных надо создавать каждые 5 минут. Чтобы обеспечить не более чем 20-минутную недоступность указанных баз данных, надо уменьшить до минимума число восстанавливаемых резервных копий журналов транзакций, поэтому разностные резервные копии должны создаваться несколько раз в день, например каждые 4 или 6 часов. Если есть достаточно места на устройствах хранения, желательно создавать полную резервную копию каждый день.
2. В базах данных *Customer* и *Order* надо реализовать секционирование. Данные старше двух месяцев должны храниться в архивной таблице, находящейся в отдельной файловой группе, что позволит воспользоваться возможностями резервного копирования файловых групп. Благодаря этому можно будет отдельно восстанавливать архивные части баз данных *Customer* и *Order*. Так как архивные данные могут длительное время находиться в автономном режиме, резервное копирование файловых групп позволит быстрее перевести приложения в оперативный режим, при этом восстановление архивных можно выполнять в фоновом режиме. Также надо создать такой же график создания резервных копий журнала транзакций и разностных резервных копий, как и для баз данных *Account*, *Inventory* и *Promotions*.
3. Так как в базе данных *HR* реализовано шифрование TDE, надо иметь резервную копию открытого и закрытого ключей сертификата базы данных *master*, на основании которой создан ключ шифрования базы данных *HR*. Надо также иметь резервные копии главного ключа службы, базы данных *master* и базы данных *HR*. Также надо создать такой же график создания резервных копий базы данных *HR*, как и для остальных баз данных. Резервные копии главных ключей и сертификата должны храниться в месте, отличном от места хранения резервных копий баз данных.

Глава 10

Занятие 1

Закрепление материала

1. Правильный ответ: В.

- A. **Неправильно:** Журнал событий Windows может содержать информацию об ошибке задания, но из журнала событий нельзя получить список потерпевших сбой событий.
- B. **Правильно:** В истории заданий в SSMS перечислены все выполненные задания вместе с информацией об их исполнении.
- C. **Неправильно:** Журнал событий Агента SQL Server может содержать информацию о потерпевшем сбой задании, но из него нельзя получить список потерпевших сбой событий.
- D. **Неправильно:** Журнал ошибок SQL Server может содержать информацию о потерпевшем сбой задании, но из него нельзя получить список потерпевших сбой событий.

Занятие 2

Закрепление материала

1. Правильный ответ: А.

- A. **Правильно:** Можно создать уведомление Агента SQL Server, информирующее оператора, а также выполняющее задание, увеличивающее размер файла данных.
- B. **Неправильно:** Системный монитор не может запустить процесс увеличения размера файла данных.
- C. **Неправильно:** Оснастка «Просмотр событий» не может запустить процесс увеличения размера файла данных.
- D. **Неправильно:** Сетевой монитор не может запустить процесс увеличения размера файла данных.

Упражнения

Упражнение 1. Разработка стратегии автоматизации для компании Coho Vineyard

- 1. Создание резервных копий должно основываться на трех разных заданиях: одно для создания полных резервных копий, второе — для создания разностных резервных копий, и третье — для создания резервных копий журналов транзакций. Можно было бы настроить постоянную репликацию слиянием из центрального офиса, но лучше разрешить ручной запуск процесса синхронизации. Загрузку информации о складских запасах из винных заводов надо построить на основе пакетов служб Integration Services, выполняемых в рамках заданий Агента SQL Server. Так как невозможно спрог-

нозировать точное время загрузки EDI-файлов от партнеров, надо создать WMI-уведомление, инициируемое при создании файла в папке для EDI-файлов. Далее это уведомление может запустить задание, выполняющее SSIS-пакет, импортирующий файл для дальнейшей обработки. Наконец, можно создать задание, выполняющееся в конце каждого месяца и архивирующее данные баз *Customer* и *Order*.

Глава 11

Занятие 1

Закрепление материала

1. **Правильный ответ: D.**

- A. **Неправильно:** Если отключить конечную точку *TCP* с типом полезной нагрузки *TSQL*, не удастся подключиться к SQL Server средствами SSMS и пользователи не будут получать никаких сообщений об ошибках.
- B. **Неправильно:** Если остановить конечную точку *TCP* с типом полезной нагрузки *TSQL*, не удастся подключиться к SQL Server средствами SSMS.
- C. **Неправильно:** Удаленные подключения включены, так как вы можете подключиться к SQL Server с рабочего компьютера.
- D. **Правильно:** Если у пользователей нет разрешения *CONNECT* для конечной точки *TCP* с типом полезной нагрузки *TSQL*, они будут получать сообщение об отказе в доступе.

2. **Правильный ответ: C.**

- A. **Неправильно:** Если на всех конечных точках задан одинаковый режим проверки подлинности, это не мешает конфигурированию зеркального отображения.
- B. **Неправильно:** Если на всех конечных точках задан одинаковый режим проверки подлинности, это не мешает конфигурированию зеркального отображения.
- C. **Правильно:** Если проверены подключение и проверка подлинности, только несоответствие настройки шифрования может мешать конфигурированию зеркального отображения.
- D. **Неправильно:** Если на всех конечных точках задан одинаковый режим проверки подлинности, это не мешает конфигурированию зеркального отображения.

Занятие 2

Закрепление материала

1. **Правильный ответ: B.**

- A. **Неправильно:** Диспетчер конфигурации SQL Server позволяет включать и отключать удаленные подключения, но не функциональные возможности SQL Server.

- В. **Правильно:** Процедура *sp_configure* служит для включения и отключения функциональных возможностей SQL Server.
- С. **Неправильно:** Средство SQL Server Surface Area Configuration Manager служило для включения и отключения функциональных возможностей SQL Server 2005, но в SQL Server 2008 ее нет.
- Д. **Неправильно:** Центр установки SQL Server используется для установки, удаления и управления установленными экземплярами SQL Server.

Занятие 3

Закрепление материала

1. Правильные ответы: А и С.

- А. **Правильно:** Надо сопоставить имена входа SQL Server каждой Windows-группе, соответствующей приложению, добавить имя входа как пользователя в соответствующую базу данных, затем добавить этого пользователя в соответствующую роль. После выполнения этих операций все, что нужно владельцам приложений для решения поставленной задачи — это возможность управлять соответствующими Windows-группами.
- В. **Неправильно:** Члены роли *securityadmin* могут выполнять команды *CREATE LOGIN*, *ALTER LOGIN* и *DROP LOGIN*. Однако они могут управлять любыми именами входа (за исключением членов роли *sysadmin*), а не только теми, что связаны с конкретными приложениями.
- С. **Правильно:** Надо сопоставить имена входа SQL Server каждой Windows-группе, соответствующей приложению, добавить имя входа как пользователя в соответствующую базу данных, затем добавить этого пользователя в соответствующую роль. После выполнения этих операций все, что нужно владельцам приложений для решения поставленной задачи — это возможность управлять соответствующими Windows-группами.
- Д. **Неправильно:** Если добавить владельцев приложений в роль *sysadmin*, вы потеряете контроль над экземпляром и излишне повысите привилегии.

2. Правильный ответ: С.

- А. **Неправильно:** Члены роли *diskadmin* могут управлять дисковыми ресурсами, но не могут выполнять резервное копирование баз данных.
- В. **Неправильно:** Члены роли *db_owner* могут выполнять резервное копирование базы данных, но у них есть права выполнять много других операций в базе данных.
- С. **Правильно:** Члены роли *db_backupoperator* могут выполнять резервное копирование базы данных, но им не разрешено выполнять любые другие операции в экземпляре.
- Д. **Неправильно:** Члены роли *sysadmin* могут выполнять резервное копирование базы данных, но у них есть права выполнять много других операций в базе данных.

Занятие 4

Закрепление материала

1. **Правильный ответ: С.**

- А. **Неправильно:** Роль *db_datawriter* не обладает разрешением *SELECT*.
- В. **Неправильно:** Хотя у пользователей и будет разрешение на все таблицы, индивидуальное предоставление разрешений требует больше усилий, чем назначение разрешения *SELECT* на базу данных.
- С. **Правильно:** Если предоставить разрешение *SELECT* на базу данных, пользователь выполнять предложение *SELECT* по отношению к любой таблице этой базы данных.
- Д. **Неправильно:** Хотя у пользователей и будет разрешение на все таблицы, предоставление разрешений на каждую схему требует больше усилий, чем назначение разрешения *SELECT* на базу данных.

2. **Правильный ответ: А.**

- А. **Правильно:** Инструкция *DENY VIEW DEFINITION* предотвращает просмотр пользователями метаданных об объектах, к которым у этого пользователя есть доступ.
- В. **Неправильно:** Инструкция *DENY VIEW ANY DEFINITION* определяет разрешение на уровне экземпляра, то есть ко всем его база данных.
- С. **Неправильно:** Инструкция *VIEW SERVER STATE* разрешает подключению просматривать статистику выполнения и никак не связана с метаданными объектов.
- Д. **Неправильно:** Инструкция *REVOKE* удаляет разрешения *GRANT* или *DENY*, но явно не предотвращает доступ к метаданным объектов, к которым у пользователя есть доступ.

Занятие 5

Закрепление материала

1. **Правильный ответ: А.**

- А. **Правильно:** Можно создать спецификацию аудита для регистрации любых предложений *SELECT*, *INSERT*, *UPDATE*, *DELETE*, *BCP* или *BULK INSERT*, выполняемых в таблице с информацией о зарплатах сотрудников, для группы пользователей, имеющих доступ к этой таблице.
- В. **Неправильно:** Триггер DDL выполняется при возникновении события DDL и не может регистрировать события доступа к данным.
- С. **Неправильно:** Триггер DML поддерживает регистрацию выполнения инструкций *INSERT*, *UPDATE* и *DELETE*, но не инструкции *SELECT*.
- Д. **Неправильно:** Спецификации аудита сервера не могут назначаться отдельным объектам базы данных.

2. **Правильный ответ: С.**

- A. **Неправильно:** Хотя триггер DDL и может выполнять аудит предложения *ALTER DATABASE*, но поскольку она не выполняется в рамках транзакции, нельзя предотвратить изменение модели восстановления.
- B. **Неправильно:** Триггер DML инициируется при изменении данных, а не модели восстановления.
- C. **Правильно:** Поскольку предложение *ALTER DATABASE* может вносить изменения в файловую систему, которые не являются транзактными, выполнение этого предложения нельзя предотвратить. Поэтому требование предотвратить изменение реализовать невозможно.
- D. **Неправильно:** Спецификация аудита сервера позволяет регистрировать, но не предотвращать изменения модели восстановления.

Занятие 6

Закрепление материала

1. Правильный ответ: B.

- A. **Неправильно:** Можно использовать сертификат для шифрования каждого столбца в базе данных, но это потребует изменения кода шифрования и расшифровки в приложениях.
- B. **Правильно:** В TDE используется сертификат базы данных *master*, который применяется для шифрования ключа шифрования базы данных. После включения этого шифрования SQL Server зашифрует данные на диске и их не сможет считать злоумышленник. Причем при этом не потребуется вносить изменения в приложения.
- C. **Неправильно:** Можно использовать симметричный ключ для шифрования каждого столбца в базе данных, но это потребует изменения кода шифрования и расшифровки в приложениях.
- D. **Неправильно:** Можно использовать асимметричный ключ для шифрования каждого столбца в базе данных, но это потребует изменения кода шифрования и расшифровки в приложениях.

2. Правильный ответ: C.

- A. **Неправильно:** Можно использовать сертификат для шифрования каждого столбца в базе данных, но это потребует изменения кода шифрования и расшифровки в приложениях.
- B. **Неправильно:** Можно использовать симметричный ключ для шифрования каждого столбца в базе данных, но это потребует изменения кода шифрования и расшифровки в приложениях.
- C. **Правильно:** В TDE используется сертификат базы данных *master*, который применяется для шифрования ключа шифрования базы данных. После включения этого шифрования SQL Server зашифрует данные на диске и их не сможет считать злоумышленник. Причем при этом не потребуется вносить изменения в приложения.
- D. **Неправильно:** Можно использовать асимметричный ключ для шифрования каждого столбца в базе данных, но это потребует изменения кода шифрования и расшифровки в приложениях.

Упражнения

Упражнение 1. Безопасность в компании Coho Vineyard

1. По возможности можно создать имя входа для каждого пользователя веб-приложений, а лучше всего передавать регистрационные данные Windows пользователя на сервер базы данных. Имена входа должны создаваться как пользователи соответствующих баз данных. Пользователей нужно добавить в роли базы данных, обладающие необходимые разрешения для доступа к нужным объектам.
2. Надо создать главный ключ в базе данных *Customers*. После этого можно приступить к созданию симметричного ключа или сертификата, которые можно использовать для шифрования данных кредитных карт в таблице.
3. Чтобы организовать аудит базы данных *HR*, надо создать спецификации аудита сервера, а затем спецификации аудита базы данных и связать их с объектом аудита сервера.

Глава 12

Занятие 1

Закрепление материала

1. **Правильный ответ: А.**
 - А. **Правильно:** Счетчик «Система/Длина очереди процессора» показывает число системных запросов, ожидающих освобождения процессора.
 - В. **Неправильно:** Счетчик «Система/Длина очереди процессора» показывает все запросы SQL Server, ожидающие освобождения процессора. Он также учитывает запросы других приложений и операционной системы, ожидающие освобождения процессора.
 - С. **Неправильно:** Число активно работающих процессоров можно определить с помощью счетчика «Процессор/% загрузки процессора» (Processor:% Processor Time).
 - Д. **Неправильно:** Время работы процессора можно определить с применением экземпляра счетчика «Процессор/% загрузки процессора».

Занятие 2

Закрепление материала

1. **Правильный ответ: А и D.**
 - А. **Правильно:** Системный монитор позволяет получать показания счетчиков производительности, информирующих о состоянии оборудования, ОС и различных подсистем SQL Server, что можно сопоставить с трассировкой приложения SQL Server Profiler для диагностики проблемы с производительностью.

- В. Неправильно:** Помощник по настройке ядра СУБД служит для разработки рекомендаций по созданию индексов и секций так, чтобы обеспечивалась оптимальная производительность запросов.
- С. Неправильно:** Регулятор ресурсов позволяет ограничивать ресурсы, выделяемые подключениям.
- Д. Правильно:** Приложение SQL Server Profiler позволяет собрать статистику работы запроса в экземпляре, а затем сопоставить с данными счетчиков производительности в Системном мониторе для диагностики проблемы с производительностью.

Занятие 3

Закрепление материала

1. **Правильный ответ: А и Д.**
 - А. Правильно:** Создание дампа стека — критически важная ошибка, которая регистрируется как в журнале ошибок SQL Server, так и в журнале «Приложение» событий Windows.
 - В. Неправильно:** Сообщения о конфигурировании при загрузке регистрируются только в журнале ошибок SQL Server.
 - С. Неправильно:** Ошибки заданий регистрируются в журнале Агента SQL Server.
 - Д. Правильно:** Уничтожение процесса — критически важная ошибка, которая регистрируется как в журнале ошибок SQL Server, так и в журнале «Приложение» событий Windows.

Занятие 4

Закрепление материала

1. **Правильный ответ: В.**
 - А. Неправильно:** Когда база данных находится в оперативном режиме и подключена, SQL Server держит файлы этой базы данных в открытом состоянии, а открытые файлы удалить невозможно.
 - В. Правильно:** Если терпит сбой дисковое хранилище, все базы данных, файлы которых размещаются в этом хранилище, регистрируют в журнале ошибку активации устройства. Если ошибка активации устройства происходит в базе данных master, tempdb или mssqlsystemresource, весь экземпляр завершается.
 - С. Неправильно:** Ошибка активации устройства в базе данных о заказе билетов приведет к отключению только этой базы данных, но не к остановке всего экземпляра SQL Server.
 - Д. Неправильно:** При блокировке учетной записи службы SQL Server любые попытки использовать эту учетную запись Windows отвергает. Однако все приложения, работающие на тот момент под этой учетной записью, продолжают работу. Поэтому блокировка учетной записи не позволит

запустить SQL Server, но не приведет к отключению SQL Server или инициированию сервером SQL Server ошибок активации устройств.

Занятие 5

Закрепление материала

1. Правильный ответ: С.

- A. **Неправильно:** Вполне возможно, что повреждена база данных *master*, но наиболее вероятная причина — одно из недавних изменений. Файлы базы данных *master* вроде бы перенесены правильно, разрешения и параметры запуска настроены верно, поэтому маловероятно, что файлы базы данных повредились только из-за перемещения в другое место.
- B. **Неправильно:** Вполне возможно, что база данных *mssqlsystem* повреждена, но наиболее вероятная причина — одно из недавних изменений. Маловероятно, что файлы этой базы данных повредились только из-за перемещения в другое место.
- C. **Правильно:** Одно из недавних изменений — перенос базы данных *tempdb* в новое место. Первый шаг этой операции — изменение базы данных *tempdb* и изменение местоположения файлов данных и журналов. После остановки экземпляра надо перенести эти файлы в новое место и только после этого начать резервное копирование экземпляра. Учетной записи SQL Server также требуются разрешения на чтение и запись в папку, где находятся файлы базы данных *tempdb*. Надо проверить, нет ли ошибок доступа к файлам базы данных *tempdb*. Намного вероятнее, что SQL Server не может найти или получить доступ к папке базы данных *tempdb* из-за одного из изменений конфигурации.
- D. **Неправильно:** Неисправный модуль памяти на сервере не позволил бы запустить экземпляр, но это очень маловероятно.

Занятие 6

Закрепление материала

1. Правильные ответы: В и D.

- A. **Неправильно:** Представление *sys.dm_exec_sessions* view предоставляет сведения о каждом подключении к экземпляру, но не содержит информацию об активных запросах.
- B. **Правильно:** В столбце *blocking_session_id* перечислены системные идентификаторы (SPID) процессов, блокирующих сеансы.
- C. **Неправильно:** Хотя представление *sys.dm_os_waiting_tasks* view и перечисляет процессы, ожидающие освобождения ресурса, оно не позволяет найти заблокированные процессы.
- D. **Правильно:** Хранимая процедура *sp_who2* system, которая входит в состав SQL Server с верии 6.5, показывает системный идентификатор (SPID) блокирующего процесса.

Упражнения

Упражнение 1. Разработка стратегии автоматизации для компании Coho Vineyard

1. Так как проблемы с производительностью наблюдаются нерегулярно, наиболее возможная причина — блокировка. Надо создать процесс, который периодически получает представление *sys.dm_exec_requests* и выясняет, не наблюдается ли блокировка. После этого можно использовать содержимое столбца *sql_handle* для получения сведений об исполняемых командах, имеющих отношение к блокировке.
2. В процессе консолидации данных в ночное время в данные вносится много изменений. Одновременно механизм репликации слиянием пытается считать изменения и отправить их подключенному подписчику. Можно исключить конфликт с репликацией, запретив процессу слияния синхронизацию изменений. Если слияние выполняется как задание (или задания) Агента SQL Server, надо остановить Агент. Если слияние инициируется подписчиком, значит надо реализовать триггер входа, который запретит подключение процесса слияния во время консолидации данных в ночное время.
3. Надо поискать сообщения об ошибках в журнале ошибок SQL Server и журналах событий Windows. Ошибки активации устройства происходят обычно из-за того, что SQL Server внезапно не может получить доступ к файлам данных или журналу транзакций базы данных по причине либо недостаточных разрешений, либо недоступности дискового тома. «Синие экраны» обычно свидетельствуют о проблемах с процессором или памятью и должны диагностироваться средствами, предоставляемыми поставщиками этого оборудования.

Глава 13

Занятие 1

Закрепление материала

1. **Правильный ответ: А, С и D.**
 - Правильно:** DTA поддерживает в качестве источника рабочей нагрузки файл или таблицу. DTA должна проанализировать в рабочей нагрузке хранимую процедуру, а также предложения BULK INSERT, SELECT, INSERT, UPDATE, DELETE или MERGE.
 - Неправильно:** Хотя план отображения в формате XML полезен для настройки запроса, DTA не поддерживает настройку запросов.
 - Правильно:** DTA поддерживает в качестве источника рабочей нагрузки файл или таблицу. DTA должна проанализировать в рабочей нагрузке хранимую процедуру, а также предложения BULK INSERT, SELECT, INSERT, UPDATE, DELETE или MERGE.

- D. **Правильно:** DTA поддерживает в качестве источника рабочей нагрузки файл или таблицу. DTA должна проанализировать в рабочей нагрузке хранимую процедуру, а также предложения BULK INSERT, SELECT, INSERT, UPDATE, DELETE или MERGE.

Занятие 2

Закрепление материала

1. **Правильный ответ: В.**

- A. **Неправильно:** Параметр «максимальная степень параллелизма» определяет число процессоров, которые может одновременно использовать запрос, но никак не ограничивает объем используемых ресурсов.
- B. **Правильно:** Можно создать группу рабочей нагрузки, в которую включить всех пользователей отдела маркетинга, и назначить эту группу пулу ресурсов, который ограничивает доступные ресурсы процессора и памяти.
- C. **Неправильно:** Регулятор запросов ограничивает время (в секундах) выполнения запроса в зависимости от конкретной конфигурации оборудования. Хотя он может использоваться для управления бесконтрольных запросов, запросы, превысившие порог ожидания, закрываются вместо того, чтобы снизить влияние на других пользователей.
- D. **Неправильно:** Хотя ограничение памяти, выделяемой для выполнения запросов пользователей из отдела маркетинга, и позволяет сохранить больше ресурсов для других запросов, пользователи отдела маркетинга все равно могут потребить все процессорные ресурсы и нарушить обработку заказов от клиентов.

Занятие 3

Закрепление материала

1. **Правильный ответ: С.**

- A. **Неправильно:** Представление *sys.dm_dm_index_operational_stats* возвращает статистику по доступу, блокировкам и кратковременным блокировкам.
- B. **Неправильно:** Представление *Sys.dm_db_index_physical_stats* о возвращает размер и статистику фрагментации всех индексов.
- C. **Правильно:** Представление *sys.dm_db_index_usage_stats* показывает сколько раз (и когда в последний раз) каждый индекс использовался для поиска, сканирования или просмотра, а также сколько раз каждый индекс обновлялся и время последнего обновления.
- D. **Неправильно:** Представление *sys.dm_db_missing_index_details* содержит сводную статистику о том, сколько раз генерировалась нехватка индекса для каждого возможного индекса.

Занятие 4

Закрепление материала

1. **Правильный ответ: В.**

- А. **Неправильно:** Управление на основе политик используется для проверки выполнения правил в экземплярах, а не для сбора информации о свободном пространстве и производительности.
- В. **Правильно:** Хранилище данных о производительности в SQL Server 2008 позволяет быстро настроить сбор данных в версиях SQL Server от 6.5 до 9.0, позволяющих консолидировать управление ресурсами, а также определять базовые уровни.
- С. **Неправильно:** Можно переписать весь код, используя SSIS, но это потребует больше усилий, чем использование хранилища данных о производительности.
- Д. **Неправильно:** Microsoft System Center Operations Manager 2007 не поддерживает сбор данных о производительности, которые могли бы использоваться для определения базовых уровней.

Упражнения

Упражнение 1. Разработка стратегии автоматизации для компании Coho Vineyard

- 1. Все проблемы, с которым сталкивается компания, требуют данные трассировки SQL, а также журналов счетчиков. Хотя ошибки и проблемы с производительностью и могли произойти в недавнее время, требуется более долгосрочная стратегия. Вместо сбора данных, когда проблема уже наблюдается, надо развернуть хранилище данных о производительности и сконфигурировать сбор данных счетчиков производительности, операций запросов, а также трассировки SQL.
- 2. Диагностировать ошибки и неполадки с производительностью можно, используя хранилище данных о производительности. При наличии такого хранилища можно на основании хранящихся в нем данных определить базовые уровни и построить отчеты, сигнализирующие об отклонении системы от заданных базовых уровней.

Глава 14

Занятие 1

Закрепление материала

1. **Правильный ответ: С.**

- А. **Неправильно:** SQL Server 2008 нельзя установить в ОС Windows 2000 Server.

- B. **Неправильно:** SQL Server 2008 нельзя установить в ОС Windows 2000 Server.
 - C. **Правильно:** На основе Windows Server 2003 Standard Edition можно создать кластер из двух узлов, затратив минимум средств.
 - D. **Неправильно:** На основе Windows Server 2003 Enterprise Edition можно создать кластер из двух узлов, но это дороже, чем при использовании Windows Server 2003 Standard Edition.
2. **Правильный ответ: А.**
- A. **Правильно:** На основе Windows Server 2003 Standard Edition можно создать кластер из двух узлов, в котором даже при сбое одного из узлов системы сбоя сохранит работоспособность. Это самый экономичный вариант.
 - B. **Неправильно:** На основе Windows Server 2003 Standard Edition можно создать двухузловый кластер с набором узлов большинства, но при сбое одного из узлов системы сбоя станет неработоспособной.
 - C. **Неправильно:** На основе Windows Server 2003 Enterprise Edition можно создать кластер из двух узлов, но это дороже, чем при использовании Windows Server 2003 Standard Edition.
 - D. **Неправильно:** На основе Windows Server 2003 Enterprise Edition можно создать двухузловый кластер с набором узлов большинства, но при сбое одного из узлов системы сбоя станет неработоспособной.
3. **Правильный ответ: В.**
- A. **Неправильно:** Служба сервера управляет общим доступом к файлам, печати и именованным каналам.
 - B. **Правильно:** Проверка работоспособности кластера выполняется с помощью службы удаленного вызова процедур.
 - C. **Неправильно:** Служба сетевого входа в систему обеспечивает сквозную проверку подлинности.
 - D. **Неправильно:** Службы терминалов позволяют многим пользователям удаленно подключаться к рабочему столу сервера.

Занятие 2

Закрепление материала

1. **Правильный ответ: С.**
- A. **Неправильно:** Доставка журналов не обеспечивает автоматический переход при сбое.
 - B. **Неправильно:** Репликация не обеспечивает автоматический переход при сбое.
 - C. **Правильно:** Отказоустойчивые кластеры защищают от сбоя оборудования, а также обеспечивают обнаружение сбоя активного узла и автоматический переход на другой ресурс.
 - D. **Неправильно:** Моментальные снимки баз данных не обеспечивают отказоустойчивость оборудования.

2. **Правильный ответ: D.**

- A. **Неправильно:** SQL Server 2008 нельзя установить в ОС Windows 2000.
- B. **Неправильно:** SQL Server 2008 нельзя установить в ОС Windows 2000.
- C. **Неправильно:** Из-за большой нагрузки компании Trey Research нужен кластер, состоящий как минимум из трех узлов, — только так можно обеспечить производительность не хуже, чем до этого. Windows Server 2003 Standard Edition поддерживает только два узла в кластере.
- D. **Правильно:** Из-за большой нагрузки компании Trey Research нужен кластер, состоящий как минимум из трех узлов, — только так можно обеспечить производительность не хуже, чем до этого. Windows Server 2003 Enterprise Edition поддерживает до четырех узлов в кластере.

Упражнения

Упражнение 1. Планирование высокой доступности

1. **Правильные ответы: B и D.**

- A. **Неправильно:** Кластер с набором узлов большинства, состоящий всего лишь из двух узлов, не обеспечивает никакой отказоустойчивости.
- B. **Правильно:** Можно создать стандартный двухузловой кластер для ориентированных на клиентов баз данных, а также отдельный двухузловой кластер для всех внутренних баз данных. Такой вариант обеспечивает автоматический переход при сбое, а также удовлетворяет требованиям по доступности.
- C. **Неправильно:** Можно организовать зеркальное отображение баз данных для автоматического перехода при сбое, но приложения ASP и ColdFusion не смогут воспользоваться функцией прозрачного перенаправления клиентских запросов, что потребует перенаправить их на зеркальную базу данных.
- D. **Правильно:** Репликация потребуется для переноса данных из внешних (ориентированных на клиентов) баз данных в базу данных за брандмауэром, чтобы сотрудники отдела маркетинга могли выполнять свой анализ.

2. **Правильный ответ: D.**

- A. **Неправильно:** Отказоустойчивый кластер поддерживает только одну копию данных и не может перемещать данные из внешних баз в базу данных за брандмауэром, обслуживающую отчеты для отдела маркетинга.
- B. **Неправильно:** Механизм зеркального отображения баз данных поддерживает пересылку данных о клиентах и заказах из внешних баз данных на экземпляр SQL Server за брандмауэром, но зеркальные базы данных будут недоступны для пользователей.
- C. **Неправильно:** Доставка журналов могла бы использоваться для поддержки копии данных, доступной для отдела маркетинга, но данные в нее попадают с большой задержкой. Перед выполнением операции восстановления пришлось бы удалять всех пользователей из базы данных.

D. **Правильно:** Репликация вполне может использоваться для переноса данных из внешних баз данных в базу данных за брандмауэром почти в реальном масштабе времени, при этом перед применением изменений не понадобится удалять пользователей из базы данных.

2. **Правильный ответ: А.**

A. **Правильно:** Нужно сконфигурировать два сервера для обеспечения автоматического перехода при сбое баз данных, поддерживающих веб-приложения. Еще два сервера нужны для организации автоматического перехода при сбое для баз данных, обслуживающих внутренние приложения. Windows Server 2003 Standard Edition с SQL Server 2008 Standard позволит организовать двухузловые кластеры, а также зеркальное отображение базы данных за минимальные деньги.

B. **Неправильно:** Нужно сконфигурировать два сервера для обеспечения автоматического перехода при сбое баз данных, поддерживающих веб-приложения. Еще два сервера нужны для организации автоматического перехода при сбое для баз данных, обслуживающих внутренние приложения. Windows Server 2003 Enterprise Edition с SQL Server 2008 Standard позволит организовать двухузловые кластеры, а также зеркальное отображение базы данных, но стоит это будет больше, чем в случае использования Windows Server 2003 Standard Edition.

C. **Неправильно:** Нужно сконфигурировать два сервера для обеспечения автоматического перехода при сбое баз данных, поддерживающих веб-приложения. Еще два сервера нужны для организации автоматического перехода при сбое для баз данных, обслуживающих внутренние приложения. Windows Server 2003 Enterprise Edition с SQL Server 2008 Enterprise позволит организовать двухузловые кластеры, а также зеркальное отображение базы данных, но стоит это будет больше, чем в случае использования Windows Server 2003 Standard Edition.

D. **Неправильно:** Нужно сконфигурировать два сервера для обеспечения автоматического перехода при сбое баз данных, поддерживающих веб-приложения. Еще два сервера нужны для организации автоматического перехода при сбое для баз данных, обслуживающих внутренние приложения. Windows Server 2003 Datacenter Edition с SQL Server 2008 Datacenter позволит организовать двухузловые кластеры, а также зеркальное отображение базы данных, но стоит это будет больше, чем в случае использования Windows Server 2003 Standard Edition.

Глава 15

Занятие 1

Закрепление материала

1. Правильный ответ: В.

- А. **Неправильно:** Издатель — это роль базы данных, участвующей в репликации.
- В. **Правильно:** Основная база данных, или одна из ролей в зеркальном отображении баз данных, является базой данных, которая принимает подключения и обрабатывает транзакции.
- С. **Неправильно:** База данных-источник — общая роль в архитектуре высокой доступности.
- Д. **Неправильно:** Монитор — элемент механизма доставки журналов.

2. Правильные ответы: В и D.

- А. **Неправильно:** В режиме высокой безопасности следящий сервер не используется.
- В. **Правильно:** В режиме высокой доступности есть следящий сервер, который используется для арбитража автоматического перехода при сбое.
- С. **Неправильно:** Следящий сервер не может обслуживать базу данных.
- Д. **Правильно:** Один следящий сервер может обслуживать несколько сеансов зеркального отображения баз данных.

3. Правильные ответы: В и D.

- А. **Неправильно:** Состояние по умолчанию — STOPPED — не разрешает создание подключений.
- В. **Правильно:** Надо задать порт для связи.
- С. **Неправильно:** Этот параметр доступен только в конечных точках HTTP полезными данными SOAP.
- Д. **Правильно:** Чтобы был возможен обмен данными между основной и зеркальной базами данных, на экземпляре, хранящем эти базы данных, нужно создать конечную точку с одной из ролей: *PARTNER* или *ALL*.

Занятие 2

Закрепление материала

1. Правильный ответ: В и D.

- А. **Неправильно:** Распространение настраивается в процессе реализации репликации.
- В. **Правильно:** Резервная копия основной базы данных восстанавливается на зеркале.
- С. **Неправильно:** Восстановленная база данных не может участвовать в сеансе зеркального отображения.

- D. **Правильно:** Чтобы участвовать в сеансе зеркального отображения база данных не должна быть восстановленной.
2. **Правильный ответ: А.**
- A. **Правильно:** База должна быть в модели полного восстановления.
- B. **Неправильно:** Зеркального отображения баз данных в SQL Server 2000 не было.
- C. **Неправильно:** Если основная база данных находится в режиме только для чтения, в ней не могут выполняться транзакции, а это значит, что она не может участвовать в сеансе зеркального отображения
- D. **Неправильно:** Участвующую в сеансе зеркального отображения базу данных нельзя перевести на модель восстановления с неполным протоколированием.
2. **Правильные ответы: В и С.**
- A. **Неправильно:** Асинхронная передача данных используется в режиме высокой производительности.
- B. **Правильно:** Синхронная передача данных используется в режимах высокой доступности и высокой безопасности.
- C. **Правильно:** Автоматический переход на резервный ресурс доступен только в режиме высокой доступности и только, когда следящий сервер находится в оперативном режиме.
- D. **Неправильно:** В режимах высокой безопасности и высокой производительности переход на резервный ресурс выполняется вручную.

Занятие 3

Закрепление материала

1. **Правильные ответы: А и D.**
- A. **Правильно:** Асинхронная передача данных используется в режиме высокой производительности.
- B. **Неправильно:** Синхронная передача данных используется в режимах высокой доступности и высокой безопасности.
- C. **Неправильно:** Автоматический переход на резервный ресурс доступен только в режиме высокой доступности и только, когда следящий сервер находится в оперативном режиме.
- D. **Правильно:** В режимах высокой безопасности и высокой производительности переход на резервный ресурс выполняется вручную.
2. **Правильный ответ: В и D.**
- A. **Неправильно:** Асинхронная передача данных используется в режиме высокой производительности.
- B. **Правильно:** Синхронная передача данных используется в режимах высокой доступности и высокой безопасности.
- C. **Неправильно:** Автоматический переход на резервный ресурс доступен только в режиме высокой доступности и только, когда следящий сервер находится в оперативном режиме.

- D. **Неправильно:** В режимах высокой безопасности и высокой производительности переход на резервный ресурс выполняется вручную.

Упражнения

Упражнение 1. Планирование высокой доступности

1. **Правильный ответ: А.**

- A. **Правильно:** Отказоустойчивые кластеры обеспечивают обнаружение сбоя активного узла и автоматический переход на другой ресурс.
- B. **Неправильно:** Режим высокой производительности не поддерживает автоматический переход на другой ресурс.
- C. **Неправильно:** Режим высокой безопасности не поддерживает автоматический переход на другой ресурс.
- D. **Неправильно:** Репликация не обеспечивает автоматический переход на другой ресурс.

2. **Правильный ответ: В.**

- A. **Неправильно:** Режим высокой производительности не поддерживает автоматический переход на другой ресурс.
- B. **Правильно:** Зеркальное отображение в режиме высокой доступности поддерживает автоматический переход на другой ресурс и обеспечивает дублирование каталога продуктов на дополнительном сайте.
- C. **Неправильно:** Репликация не обеспечивает автоматический переход на другой ресурс.
- D. **Неправильно:** Доставка журналов не обеспечивает автоматический переход на другой ресурс.

3. **Правильный ответ: С.**

- A. **Неправильно:** Доставка журналов не подойдет, так как потребуются отключать всех пользователей отдела маркетинга перед созданием резервной копии журнала транзакций.
- B. **Неправильно:** Отказоустойчивый кластер поддерживает только одну копию базы данных, поэтому он не удовлетворяет требованию по созданию отчетов.
- C. **Правильно:** Репликация транзакций с обновлением подписчиков посредством очереди и не является оптимальным решением из-за того, что данные могут меняться на подписчике, тем не менее, она отвечает требованиям по созданию отчетов.
- D. **Неправильно:** Доставка журналов не удовлетворяет требованию по созданию отчетов почти в реальном масштабе времени.

4. **Правильный ответ: А.**

- A. **Правильно:** Так как базы данных *Orders*, *Customers*, *CreditCards* и *Products* взаимозависимы, отказоустойчивый кластер гарантирует, что в случае сбоя все они вместе перейдут на другой ресурс.
- B. **Неправильно:** Зеркальное отображение в режиме высокой доступности поддерживает автоматический переход на другой ресурс, но доступность

реализована на уровне баз данных. По этой причине при переходе на другой ресурс может привести к рассогласованию этих четырех баз данных. Транзакции, охватывающие несколько баз данных, механизм зеркального отображения разбивает на транзакции в отдельных базах данных и может произойти так, что в зеркальной базе данных могут быть зафиксированы строки, у которых нет соответствующих значений в другой базе данных.

- C. **Неправильно:** Репликация транзакций с обновлением подписчиков посредством очередей не удовлетворяет требованиям по переходу на другой ресурс.
- D. **Неправильно:** Доставка журналов не удовлетворяет требованиям по переходу на другой ресурс.

Глава 16

Занятие 1

Закрепление материала

1. **Правильный ответ: В.**

- A. **Неправильно:** Основная база данных — одна из ролей в зеркальном отображении баз данных.
- B. **Правильно:** В механизме доставки журналов базы данных могут обладать ролью источника или приемника.
- C. **Неправильно:** Распространитель — это экземпляр, хранящий базу данных распространителя, используемую в механизме репликации.
- D. **Неправильно:** Режим ожидания (STANDBY) — это режим сервера-получателя, не роль в механизме доставки журналов.

Занятие 2

Закрепление материала

1. **Правильный ответ: В.**

- A. **Неправильно:** Главный ключ базы данных нужен только для расшифровки данных и не используется для предотвращения доступа к базе данных.
- B. **Правильно:** Без имен входа невозможно получить доступ к базе данных.
- C. **Неправильно:** Задания Агента SQL Server не нужны для того, чтобы база данных стала доступной.
- D. **Неправильно:** Триггеры DDL не нужны для того, чтобы база данных стала доступной.

Занятие 3

Закрепление материала

1. Правильный ответ: В.

- А. **Неправильно:** Команду *ALTER DATABASE* нельзя использовать для восстановления базы данных.
- В. **Правильно:** Восстановление базы данных с параметром *RECOVERY* переводит базу данных в оперативный режим.
- С. **Неправильно:** Параметр *STANDBY* служит для включения доступа только для чтения к базе данных, а также разрешает восстановление дополнительных журналов транзакций. В базе данных в режиме *STANDBY* нельзя выполнять транзакции.
- Д. **Неправильно:** Команда *ALTER LOGIN* служит для изменения имени входа.

Упражнения

Упражнение 1. Планирование высокой доступности

1. Правильный ответ: А.

- А. **Правильно:** Базы данных *Bookings* и *Customers* взаимозависимы. Чтобы обеспечивалась их перекрестная целостность, они должны располагаться на одном экземпляре SQL Server и переходить при сбое на другой ресурс вместе. Отказоустойчивый кластер — единственный способ обеспечить целостность и одновременный переход баз данных на другой ресурс.
- В. **Неправильно:** Доставка журналов оперирует резервными копиями журналов транзакций, в которых каждая транзакция относится к одной базе данных. Поэтому возможна ситуация, когда база данных *Bookings* будет восстановлена до состояния, отличного от состояния базы данных *Customers*.
- С. **Неправильно:** Репликация работает на уровне базы данных, поэтому транзакции могут применяться к разным базам данных независимо. Репликация не гарантирует (и не проверяет) перекрестную целостность баз данных.
- Д. **Неправильно:** Зеркальное отображение работает на уровне базы данных. Так как транзакции выполняются на уровне отдельных баз данных, перекрестная целостность баз данных не гарантируется.

2. Правильный ответ: D.

- А. **Неправильно:** Механизм зеркального отображения может обеспечивать синхронизацию зеркальной копии с основной копией, но зеркальная копия при этом недоступна для работы.
- В. **Неправильно:** Можно было бы создать моментальный снимок базы данных, обновляемый каждые 6 часов, и использовать его для генерации отчетов. Однако на время создания моментального снимка с тем же именем потребуются закрыть все подключения к нему. Можно было бы создать моментальный снимок с другим именем, но тогда все операции по созданию отчетов пришлось бы перенаправлять на другой моментальный снимок.

С. Неправильно: Доставка журналов могла бы использоваться для поддержки копии в режиме ожидания, поддерживающей операции чтения. Однако на время восстановления очередной резервной копии журнала транзакций потребуется закрыть все подключения к копии.

Д. Правильно: Только репликация позволяет пересылать изменения из базы данных-приемника в базу данных-получатель, обеспечивая непрерывную возможность чтения.

3. Правильный ответ: D.

A. Неправильно: Зеркальное отображение не позволяет переводить базу данных в модель восстановления с неполным протоколированием.

B. Неправильно: Географическая область действия отказоустойчивых кластеров обычно ограничена, и эта технология не годится для защиты от стихийных бедствий, охватывающих большие территории, например от землетрясения.

С. Неправильно: Базу данных, участвующую в репликации, можно перевести в модель восстановления с неполным протоколированием. Однако механизм репликации будет отбирать только полностью заprotoколированные транзакции. Любые транзакции с неполным протоколированием, такие как *BCP* или *BULK INSERT*, механизмом репликации распространяться не будут.

Д. Правильно: Можно создавать резервные копии журнала транзакций базы данных *Products*, что позволит защитить ее средствами механизма доставки журналов. Это также позволит разместить базу данных-источник достаточно далеко, чтобы защититься от стихийных бедствий, охватывающих большие территории.

4. Правильный ответ: С.

A. Неправильно: Отзыв прав *sysadmin* не позволит никому изменять модель восстановления базы данных *Products*, но и не позволит пользователям выполнять другие задачи.

B. Неправильно: Триггер DDL уровня базы данных срабатывает при выполнении определенных операций в базе данных. Изменение модели восстановления выполняется командой *ALTER DATABASE*, на которую триггер DDL уровня базы данных не реагирует.

С. Правильно: Триггер DDL экземпляра срабатывает при выполнении команды *ALTER DATABASE*. Позже можно проверить команду, вызвав функцию *Eventdata()* для отката любых команд, изменяющих модель восстановления на простую.

Д. Неправильно: Отправка уведомлений предоставляет информацию о том, что операция произошла, но механизм отправки сообщений не в состоянии предотвратить выполнение операции.

5. Правильный ответ: В.

A. Неправильно: Главный ключ экземпляра не может передаваться на другие экземпляры.

- В. **Правильно:** Главный ключ базы данных используется для шифрования асимметричного ключа, поэтому этот ключ надо восстановить на получателе, чтобы получить возможность расшифровывать данные о кредитных картах.
- С. **Неправильно:** Сертификаты используются с асимметричными ключами.
- Д. **Неправильно:** Вообще говоря, копирование нужных заданий Агента SQL Server на приемник идея неплохая, но задания не нужны для создания избыточной копии, если база данных недоступна.

6. **Правильные ответы: А и С.**

- А. **Правильно:** Шифрующая файловая система создает дополнительную нагрузку, что приведет к увеличению времени резервного копирования и восстановления. Резервные копии будут шифроваться, что совместно с уже имеющимся шифрованием данных кредитных карт сильно затруднит атаку. Даже если злоумышленнику удастся украсть ленты с резервными копиями, ему придется сначала взломать шифр файловой системы, а потом еще — многоуровневое шифрование самой базы данных.
- В. **Неправильно:** Предложение *PASSWORD* команды *BACKUP* не предназначено для обеспечения безопасности. Эта защита очень слабая и предназначена только для предотвращения случайного восстановления.
- С. **Правильно:** Данные кредитных карт уже зашифрованы с помощью асимметричного ключа, который в свою очередь зашифрован главным ключом базы данных. Чтобы расшифровать данные кредитных карт, нужны оба ключа. Храня резервную копию главного ключа базы данных в закрытом помещении, отличном от помещения, в котором хранятся резервные копии, вы снизите вероятность того, что злоумышленникам удастся расшифровать данные кредитных карт, даже если они получат доступ к резервным копиям.
- Д. **Неправильно:** Хранение резервной копии главного ключа базы данных в другом каталоге никак не укрепляет защиту, так как у любого, имеющего доступ к каталогу с резервными копиями базы данных, есть также доступ к резервной копии главного ключа базы данных.

7. **Правильный ответ: В.**

- А. **Неправильно:** При использовании встроенной доставки журналов в SQL Server 2000 Enterprise Edition хранимая процедура *sp_resolve_logins* обновляла бы привязку SID к именам входа пользователей базы данных. Однако в SQL Server 2008 этой процедуры нет.
- В. **Правильно:** Команда *ALTER LOGIN* может использоваться для привязки SID к именам входа пользователей базы данных.
- С. **Неправильно:** Команда *GRANT* служит для определения разрешений имени входа или пользователя, но имена входа SQL Server могут неправильно сопоставляться пользователям базы данных, и команда *GRANT* эту проблему не решит.
- Д. **Неправильно:** Хотя хранимая процедура *sp_change_users_login* и обновляет привязку имени входа SQL Server к пользователю базы данных для обеспечения правильности разрешений базы данных, относящихся к име-

ни входа SQL Server, для этой цели должна использоваться команда *ALTER LOGIN*.

8. Правильный ответ: D.

- A. Неправильно:** На первый взгляд отказоустойчивый кластер с зеркальным отображением баз данных кажется вполне разумной стратегией для всех трех баз данных. Однако механизм зеркального отображения обнаруживает сбой и осуществляет переход на другой ресурс существенно быстрее, чем отказоустойчивый кластер. Поэтому переход на другой ресурс в кластере приведет к каскадному переходу в механизме зеркального отображения. Такая комбинация не подходит из-за того, что зеркальное отображение не гарантирует перекрестную целостность баз данных.
- A. Неправильно:** В базе данных *Products* должна использоваться модель восстановления с неполным протоколированием. Поэтому любое решение, предусматривающее репликацию, не подходит.
- B. Неправильно:** Доставка журналов не обеспечивает перекрестную целостность баз данных *Customers* и *Bookings*.
- C. Правильно:** Отказоустойчивая кластеризация обеспечивает перекрестную целостность баз данных *Customers* и *Bookings* из-за того, что переход обеих баз данных на другой ресурс осуществляется одновременно. В базе данных *Products* можно включить или отключить модель восстановления с неполным протоколированием, не оказывая никакого влияния ни на кластеризацию, ни на доставку журналов. Доставка журналов может использоваться как дополнительный механизм перехода на другой ресурс, так как даже в случае полного отказа описанного механизма, разрешаются небольшие расхождения в базах данных *Customers* и *Bookings*.
- D. Неправильно:** Зеркальное отображение не обеспечивает перекрестную целостность баз данных *Customers* и *Bookings*.

Глава 17

Занятие 1

Закрепление материала

1. Правильный ответ: A.

- A. Правильно:** В процессе репликации транзакций изменения отправляются на сервер отчетов, но никаких изменений не приходит обратно от распространителя.
- B. Неправильно:** Репликация моментальных снимков может использоваться, но сервер отчетов будет недоступным при каждом сеансе репликации. Кроме того сервер отчетов поддерживается только в статическом состоянии на определенный момент времени, что не отвечает требованиям.
- C. Неправильно:** Репликация слиянием поддерживает передачу изменений с подписчика на распространитель, что не отвечает заявленным требованиям.

D. Неправильно: Одноранговая репликация представляет собой архитектуру репликации транзакций, поддерживающую изменения во всех сайтах, что не отвечает заявленным требованиям.

2. Правильный ответ: D.

A. Неправильно: Репликация транзакций не поддерживает передачу изменений от подписчика на распространитель, что не отвечает заявленным требованиям.

B. Неправильно: Если распространитель становится недоступным, немедленно обновляемый подписчик не в состоянии обработать никаких изменений, что не отвечает заявленным требованиям.

C. Неправильно: Репликация слиянием может удовлетворить заявленным требованиям, но она не обеспечивает лучшей производительности, чем репликация транзакций с обновлением подписчиков посредством очередей.

D. Правильно: Репликация транзакций с обновлением подписчиков посредством очередей позволяет вносить изменения на подписчике, даже когда издатель недоступен, а также обеспечивает максимальную производительность.

Занятие 2

Закрепление материала

1. Правильный ответ: C.

A. Неправильно: Все базы данных должны быть доступны для чтения и записи, что невозможно при зеркальном отображении баз данных.

B. Неправильно: В механизме репликации транзакций в конфигурации с одним издателем все операции записи выполняются на одной машине, что не отвечает требованиям по избыточности.

C. Правильно: Одноранговая репликация при необходимости позволяет чтение и запись на всех серверах, отвечает требованию по доступности и поддерживает обмен данными между более чем двумя серверами.

D. Неправильно: Обновление подписчиков посредством очередей позволяет вносить изменения на одном подписчике, но большая задержка и потребность всех серверов реплицировать данные друг с другом не отвечает заявленным требованиям.

Занятие 3

Закрепление материала

1. Правильный ответ: F.

A. Неправильно: В центральном офисе поддерживается полномочная часть данных о заказах и всегда хранится главная копия данных. Курьеры доставляют пакеты, поэтому самая правильная информация о получении и доставке есть у курьера. Поскольку имеются противоречия в требованиях к тому, какие данные являются самыми актуальными, нужно, по

крайней мере, две публикации, потому что механизм обнаружение конфликтов нельзя выбирать динамически в каждом цикле синхронизации. Все публикации должны создаваться в центральном офисе, чтобы ими можно было управлять. Поскольку центральный офис является «хозяйном» информации о заказах, информация из центрального офиса должна всегда брать верх в конфликтах, касающихся этой информации. По аналогичной причине в конфликте информации о получении и доставке должна «побеждать» информация курьера. Конфигурация, описанная в этом ответе, не удовлетворяет всем этим требованиям, поэтому такой вариант неприемлем.

- В. Неправильно:** В центральном офисе поддерживается полномочная часть данных о заказах и всегда хранится главная копия данных. Курьеры доставляют пакеты, поэтому самая правильная информация о получении и доставке есть у курьера. Поскольку имеются противоречия в требованиях к тому, какие данные являются самыми актуальными, нужно, по крайней мере, две публикации, потому что механизм обнаружение конфликтов нельзя выбирать динамически в каждом цикле синхронизации. Все публикации должны создаваться в центральном офисе, чтобы ими можно было управлять. Поскольку центральный офис является «хозяйном» информации о заказах, информация из центрального офиса должна всегда брать верх в конфликтах, касающихся этой информации. По аналогичной причине в конфликте информации о получении и доставке должна «побеждать» информация курьера. Конфигурация, описанная в этом ответе, не удовлетворяет всем этим требованиям, поэтому такой вариант неприемлем.
- С. Неправильно:** В центральном офисе поддерживается полномочная часть данных о заказах и всегда хранится главная копия данных. Курьеры доставляют пакеты, поэтому самая правильная информация о получении и доставке есть у курьера. Поскольку имеются противоречия в требованиях к тому, какие данные являются самыми актуальными, нужно по крайней мере две публикации, потому что механизм обнаружение конфликтов нельзя выбирать динамически в каждом цикле синхронизации. Все публикации должны создаваться в центральном офисе, чтобы ими можно было управлять. Поскольку центральный офис является «хозяйном» информации о заказах, информация из центрального офиса должна всегда брать верх в конфликтах, касающихся этой информации. По аналогичной причине в конфликте информации о получении и доставке должна «побеждать» информация курьера. Конфигурация, описанная в этом ответе, не удовлетворяет всем этим требованиям, поэтому такой вариант неприемлем.
- Д. Неправильно:** В центральном офисе поддерживается полномочная часть данных о заказах и всегда хранится главная копия данных. Курьеры доставляют пакеты, поэтому самая правильная информация о получении и доставке есть у курьера. Поскольку имеются противоречия в требованиях к тому, какие данные являются самыми актуальными, нужно, по

крайней мере, две публикации, потому что механизм обнаружение конфликтов нельзя выбирать динамически в каждом цикле синхронизации. Все публикации должны создаваться в центральном офисе, чтобы ими можно было управлять. Поскольку центральный офис является «хозяином» информации о заказах, информация из центрального офиса должна всегда брать верх в конфликтах, касающихся этой информации. По аналогичной причине в конфликте информации о получении и доставке должна «побеждать» информация курьера. Конфигурация, описанная в этом ответе, не удовлетворяет всем этим требованиям, поэтому такой вариант неприемлем.

- Е. Неправильно:** В центральном офисе поддерживается полномочная часть данных о заказах и всегда хранится главная копия данных. Курьеры доставляют пакеты, поэтому самая правильная информация о получении и доставке есть у курьера. Поскольку имеются противоречия в требованиях к тому, какие данные являются самыми актуальными, нужно, по крайней мере, две публикации, потому что механизм обнаружение конфликтов нельзя выбирать динамически в каждом цикле синхронизации. Все публикации должны создаваться в центральном офисе, чтобы ими можно было управлять. Поскольку центральный офис является «хозяином» информации о заказах, информация из центрального офиса должна всегда брать верх в конфликтах, касающихся этой информации. По аналогичной причине в конфликте информации о получении и доставке должна «побеждать» информация курьера. Конфигурация, описанная в этом ответе, не удовлетворяет всем этим требованиям, поэтому такой вариант неприемлем.
- Г. Правильно:** В центральном офисе поддерживается полномочная часть данных о заказах и всегда хранится главная копия данных. Курьеры доставляют пакеты, поэтому самая правильная информация о получении и доставке есть у курьера. Поскольку имеются противоречия в требованиях к тому, какие данные являются самыми актуальными, нужно, по крайней мере, две публикации, потому что механизм обнаружение конфликтов нельзя выбирать динамически в каждом цикле синхронизации. Все публикации должны создаваться в центральном офисе, чтобы ими можно было управлять. Поскольку центральный офис является «хозяином» информации о заказах, информация из центрального офиса должна всегда брать верх в конфликтах, касающихся этой информации. По аналогичной причине в конфликте информации о получении и доставке должна «побеждать» информация курьера. Конфигурация, описанная в этом ответе, удовлетворяет всем этим требованиям.

Упражнения

Упражнение 1. Планирование высокой доступности

1. Правильный ответ: С.

- А. Неправильно:** Хотя репликация транзакций с обновлением подписчиков посредством очередей и позволяет вносить изменения в источнике и при-

емнике, а также выполнять разрешение конфликтов, она не гарантирует гибкости, предоставляемой репликацией слиянием.

- В. **Неправильно:** Доставка журналов не позволяет вносить изменения как в источнике, так и на приемнике.
- С. **Правильно:** Репликация слиянием позволяет вносить изменения как в источнике, так и на приемнике, а также предоставляет развитые средства разрешения конфликтов, позволяющие усреднять число заказов, что дает возможность более точно отображать складские запасы.
- Д. **Неправильно:** При зеркальном отображении изменения зеркальной базы данных невозможны.

2. **Правильный ответ: А.**

- А. **Правильно:** Доставка журналов потребует минимум усилий, так как в ней используются резервные копии, которые уже создаются в базе данных. При этом база данных остается недоступной для приемника.
- В. **Неправильно:** Репликацию транзакций придется конфигурировать дополнительно, что потребует больше усилий, чем создание резервных копий, которые уже создаются в базе данных. Однако по умолчанию репликация транзакций не реплицирует все объекты базы данных. Придется создавать либо дополнительный код, чтобы получить имеющуюся в столбцах идентификационную информацию, либо сценарии, вводящие идентификационное свойство (что потребует значительно больше усилий, чем доставка журналов).
- С. **Неправильно:** Придется дополнительно сконфигурировать репликацию, что потребует больше усилий, чем создание резервных копий, которые уже создаются в базе данных. Однако по умолчанию репликация транзакций не реплицирует все объекты базы данных. Можно было бы выполнять изменения на подписчике и отправлять их на издатель, но это идет вразрез с бизнес-требованиями.
- Д. **Неправильно:** Придется дополнительно сконфигурировать репликацию, что потребует больше усилий, чем создание резервных копий, которые уже создаются в базе данных. Однако по умолчанию репликация транзакций не реплицирует все объекты базы данных. Можно было бы выполнять изменения на подписчике и отправлять их на издатель, но это идет вразрез с бизнес-требованиями.

3. **Правильный ответ: D.**

- А. **Неправильно:** RAID 5 обеспечивает минимальную отказоустойчивость за счет поддержки горячего резерва. При сбое хотя бы одного диска обе базы данных станут недоступными.
- В. **Неправильно:** RAID 0 не обеспечивает никакой отказоустойчивости, поэтому вообще не подходит в данной ситуации.
- С. **Неправильно:** RAID 0 обеспечивает полное дублирование всех дисков, но производительность не так высока, как в комбинации «RAID 1 + RAID 0 (зеркало с чередованием)».

D. **Правильно:** RAID 1 + RAID 0 (зеркало с чередованием) обеспечивает максимальное дублирование и производительность.

4. **Правильный ответ: В.**

A. **Неправильно:** Репликация слиянием позволяет вносить изменения как в источнике, так и на приемнике, что идет вразрез с бизнес-требованиями.

B. **Правильно:** Репликация транзакций поддерживает реплику в отделе маркетинга почти в режиме реального времени, а также предотвращает перенос изменений реплики в производственную базу данных.

C. **Неправильно:** Репликация моментальных снимков не обновляет данные так часто, как репликация транзакций, и будет вносить путаницу в отчеты, создаваемые в момент применения моментального снимка.

D. **Неправильно:** Немедленное обновление подписчиков позволит отправлять изменения обратно на издатель, что идет вразрез с бизнес-требованиями.

5. **Правильный ответ: А.**

A. **Правильно:** Репликация транзакций в одноранговой конфигурации предоставляет дополнительные возможности чтения и записи, а также поддерживает единую целостную копию данных во всей среде. Есть также задел на будущее, так к текущим двум базам данных в будущем с ростом нагрузки можно добавить дополнительные базы данных. Приложения надо написать так, чтобы операции записи секционировались между базами данных, чтобы предотвратить конфликты данных.

B. **Неправильно:** Репликация транзакций в двунаправленной конфигурации не обеспечивает никакого масштабирования.

C. **Неправильно:** Репликация транзакций обеспечивает масштабирование операций чтения, но не записи.

D. **Неправильно:** Репликация транзакций с обновлением подписчиков посредством очередей может обеспечить масштабирование операций чтения и записи до нескольких серверов, но при этом задержка намного больше, чем в одноранговой конфигурации, из-за чего пользователи будут наблюдать несогласованность своих данных.

Системные требования

Для выполнения упражнений этого курса мы не рекомендуем использовать основной рабочий компьютер, потому что придется вносить изменения в операционную систему и конфигурацию Microsoft SQL Server.

Требования к оборудованию

Для выполнения большинства упражнений этого курса достаточно одной машины, однако для выполнения упражнений в главе 14 потребуются как минимум три виртуальных машины. Компьютер или виртуальные машины должны (как минимум) отвечать следующим требованиям:

- компьютер, оснащенный процессором Pentium III или выше, для 32-разрядной машины тактовая частота должна 1 ГГц, а для 64-разрядной — 1,6 ГГц;
- 512 Мб оперативной памяти;
- 20 Гб свободного пространства на диске;
- DVD-дисковод;
- клавиатура, мышь Microsoft или другое совместимое указывающее устройство.

Требования к программному обеспечению

Для выполнения практических заданий этого курса необходимо перечисленное ниже программное обеспечение:

- Microsoft .NET Framework 3.5;
- одна из следующих операционных систем: Windows XP Professional Home, Tablet, Media Center или Professional с SP2, Windows Vista с SP1, Windows Server 2003 Standard или Enterprise с SP2, Windows Server 2008 Standard или Enterprise;
- Microsoft Data Access Components (MDAC) 2.8 SP1 или более поздняя версия;
- поддержка сетью протоколов общей памяти, именованных каналов (Named Pipes) или TCP/IP;
- Microsoft Internet Explorer 6.0 с SP1 или Windows Internet Explorer 7.0 или более поздней версии.

Для использования виртуальных машин в Windows надо установить Microsoft Virtual Server 2005 R2, Hyper-V или ПО виртуализации других поставщиков. Загрузить пробную версию Virtual Server 2005 R2, а также получить дополнительную информацию об этом продукте можно на веб-странице <http://www.microsoft.com/virtualserver>. Подробнее о Hyper-V см. веб-страницу <http://www.microsoft.com/hyper-v>.

Об авторе

Майк Хотек

Майк Хотек (Mike Hoteck) — президент расположенной в США компании MHS Enterprises, Inc., а также филиппинской корпорации FilAm Software Technology, Inc. Более 30 лет он занимается разработкой приложений. Будучи специалистом Microsoft по SQL Server, Майк выступил консультантом свыше чем в тысяче связанных с SQL Server проектов, а также разработал продукты и решения, охватывающие абсолютно все возможности SQL Server — реляционные базы данных, ETL, построение отчетов, OLAP и интеллектуальный анализ данных. Майк профессионально владеет 40 языками и платформами разработки — от Cobol, RPG, Fortran и LISP до Powerbuilder, Delphi, .NET и PHP. Выступил автором и соавтором восьми книг, семь из которых посвящены SQL Server, а также написал десятки статей в различных специализированных журналах. Когда он не занят консультированием по связанным с SQL Server проектам, выступлением на конференциях, ведением семинаров, созданием программ или обучением студентов, Майка можно найти за токарным станком его столярной мастерской.

Хотек Майк

**Microsoft SQL Server 2008.
Реализация и обслуживание
Учебный курс Microsoft**

Подготовлено к печати издательством «Русская редакция»
125362, Москва, ул. Свободы д. 17. Тел.: (495) 638-5-638
e-mail: info@rusedit.com, <http://www.rusedit.com>

 **РУССКАЯ РЕДАКЦИЯ**

Подписано в печать 21.12.2010 г.
Формат 70х100/16. Усл. физ. л. 36
Тираж 1500 экз. Заказ №

Санитарно-эпидемиологическое заключение на продукцию
№ 77.99.60.953.Д.005770.05.09 от 26.05.2009 г. выдано Федеральной службой
по надзору в сфере защиты прав потребителей и благополучия человека.
Отпечатано с готовых диапозитивов
в ГУП «Типография «Наука»
199034, Санкт-Петербург, 9 линия, 12

Microsoft®

SQL Server® 2008

Реализация
и обслуживание

MCTS

Учебный
курс
Microsoft®

Экзамен **70-432**

Официальное пособие для самоподготовки

Освоив материалы этого курса, вы научитесь устанавливать SQL Server в корпоративной среде, а также самостоятельно подготовиться к сдаче экзамена № 70-432 по программе сертификации Microsoft MCTS (Microsoft Certified Technology Specialist). С помощью этого официального учебного пособия Microsoft вы сможете эффективно организовать свой учебный процесс.

В этой книге:

- конфигурирование, управление, резервное копирование и восстановление баз данных;
- создание секционированных таблиц, индексов и индексированных представлений;
- конфигурирование служб и компонентов в процессе управления средой;
- предотвращение несанкционированного доступа на каждом уровне инфраструктуры безопасности;
- проектирование и развертывание решений высокой доступности;
- управление заданиями, предупреждениями и операторами Агента SQL Server;
- диагностика неполадок служб, параллелизма и выполнения заданий.



ISBN 978-5-7502-0402-1



9 785750 204021



На прилагаемом
компакт-диске:

- демонстрационная версия экзаменационных тестов с автоматическим подсчетом результатов и разъяснением верных и неверных ответов;
- упражнения, которые позволят вам на практике закрепить полученные знания;
- электронные книги на английском языке.

Издательство
Русская редакция
Москва,
3-я Хорошевская ул. 11
E-mail: info@rusedit.com
Internet: www.rusedit.com
Тел.: (495) 638-5-638

РУССКАЯ РЕДАКЦИЯ